

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET

SAMIR RIBIĆ

OPERATIVNI SISTEMI



Univerzitetsko izdanje

Sarajevo, 2019.

Autor: Prof. dr. Samir Ribić, dipl. ing. el.

Naziv djela: Operativni sistemi

Broj izdanja: 1. izdanje

Izdavač: Elektrotehnički fakultet Univerziteta u Sarajevu

Za izdavača: Prof. dr. Samim Konjicija, dipl. ing. el.

Glavni urednik: Prof. dr. Samir Ribić, dipl. ing. el.

Recenzenti:

Prof. dr. Saša Mrdović, dipl. ing. el.,

Elektrotehnički fakultet Univerziteta u Sarajevu

Doc. dr. Adis Alihodžić, dipl. mat.

Prirodno-matematički fakultet Univerziteta u Sarajevu

Ilustrator: Prof. dr. Samir Ribić, dipl. ing. el.

Računarska obrada i DTP: Prof. dr. Samir Ribić, dipl. ing. el.

Godina izdanja: 2019.

Tiraž: 150 primjeraka

Štampa: Kemgrafika Trade, d.o.o. Sarajevo

Odgovorno lice štamparije: Salih Češko

Odlukom Senata Univerziteta u Sarajevu br. 01-1101-60/18 od 19.12.2018 godine, ovaj udžbenik dobio je saglasnost da se izda kao univerzitetsko izdanje

CIP - Katalogizacija u publikaciji
Nacionalna i univerzitetska biblioteka
Bosne i Hercegovine, Sarajevo

004.451(075.8)

RIBIĆ, Samir
Operativni sistemi / Samir Ribić. -Univerzitetsko izd. -Sarajevo :
Elektrotehnički fakultet, 2019. - 400 str. : ilustr. ; 25 cm

Bibliografija: str. 391-392. -Registar.

ISBN 978-9958-629-73-0

COBISS.BH-ID 26775046

SADRŽAJ

PREDGOVOR	11
1.UVOD U OPERATIVNE SISTEME.....	13
1.1.Pojam operativnog sistema	13
1.2.Klasifikacija operativnih sistema	14
1.3.Podjela po broju korisnika, procesa i procesora	14
1.4.Podjela po načinu obrade	15
1.5.Sistemi s grupnom obradom	15
1.6.Interaktivni sistemi	16
1.7.Hibridni sistemi	17
1.8.Distribuirani sistemi	18
1.9.Sistemi u realnom vremenu	18
1.10.Podjela sistema po namjeni	18
1.11.Jezgro operativnog sistema	19
1.12.Podjela operativnih sistema po strukturi jezgra	19
1.13.Potpuno monolitni sistemi	19
1.14.Sistemi monolitnog jezgra	20
1.15.Unikernel	20
1.16.Slojeviti sistemi	20
1.17.Prstenasti sistem	21
1.18.Modularno jezgro	21
1.19.OS s mikro jezgrom	21
1.20.OS preko virtualnih mašina	22
1.21.Exokernel	24
1.22.Hibridno jezgro	24
1.23.Prekidi	24
1.24.Sistemske pozive	25
1.25.API pozivi	26
1.26.WINDOWS API	27
1.27.POSIX API	28
1.28.Popularnost operativnih sistema	28
2.UPRAVLJANJE PROCESIMA I NITIMA	29
2.1.Model procesa	29
2.2.Pokretanje procesa	32
2.3.Stanja procesa	33
2.4.Redovi raspoređivanja	34
2.5.Izmjena konteksta i dispečer	36
2.6.Završavanje procesa	37
2.7.Windows API funkcije za promjenu stanja procesa	38
2.8.Posix funkcije za upravljanje procesima	40
2.9.Programske niti	41
2.10.Načini implementacije niti	43
2.11.Windows funkcije za rad s nitima	44
2.12.POSIX niti	46
2.13.Zadaci	47
3.RASPOREĐIVANJE PROCESORA	49
3.1.Raspoređivač	49
3.2.Nivoi i vrijeme raspoređivanja	49

3.3.Ciljevi i kriteriji algoritama za raspoređivanje.....	50
3.4.Kategorije algoritma za raspoređivanje	52
3.5.Algoritmi za pozadinske (batch) sisteme.....	53
3.6.Prvi došao prvi uslužen (FCFS)	53
3.7.Najkraći posao prvi -SJF (Shortest Job First).....	54
3.8.Najkraće preostalo vrijeme SRTN	56
3.9.Najbolji odnos odziva sljedeći (HRRN).....	56
3.10.Prioritetno raspoređivanje bez istiskivanja.....	56
3.11.Algoritmi za interaktivne sisteme	57
3.12.Kružno (Round-Robin)raspoređivanje	57
3.13.Kooperativno raspoređivanje	58
3.14.Prioritetno raspoređivanje sa vremenskim prilagođivanjem prioriteta.....	58
3.15.Round Robin sa prioritetima	60
3.16.Višestruki redovi čekanja.....	61
3.17.Raspoređivanje sa više nivoa sa povratnom spregom	61
3.18.Garantovano raspoređivanje.....	62
3.19.Lutrijsko raspoređivanje.....	63
3.20.Raspoređivanje s fer raspodjelom.....	63
3.21.Potpuno fer raspoređivanje.....	64
3.22.Algoritmi za sisteme realnog vremena i multimedijalne sisteme	64
3.23.Ručno raspoređivanje.....	65
3.24.Prioritetno raspoređivanje sa istiskivanjem	65
3.25.Algoritam najraniji rok prvo	66
3.26.Raspoređivanje monotonom stopom	67
3.27. Algoritmi za više-procesorske sisteme	67
3.28.Particionirano raspoređivanje procesa	68
3.29.Globalno raspoređivanje procesa	68
3.30.Podjela opterećenja.....	68
3.31.Družbeno raspoređivanje.....	69
3.32.Dodjela posvećenog procesora	70
3.33.Dinamičko raspoređivanje	70
3.34.POSIX funkcije koje podešavaju algoritme raspoređivanja.....	71
3.35.Windows funkcije koje podešavaju algoritme raspoređivanja.....	71
3.36.Zadaci.....	72
4.SINHRONIZACIJA PROCESA.....	77
4.1.Dijeljenje i natjecanje.....	77
4.2.Kritična sekcija.....	78
4.3.Načini rješenja problema međusobnog isključivanja.....	78
4.4.Dijeljena varijabla.....	79
4.5.Striktna promjena	79
4.6.Peterson-ovo rješenje.....	80
4.7.Zabrana prekida.....	80
4.8.Instrukcije testiranja i postavljanja.....	81
4.9.Semafori sa zaposlenim čekanjem.....	82
4.10. Semafori sa blokiranjem i deblokiranjem procesa	83
4.11.Mutex-i.....	84
4.12.Windows API za sinhronizaciju	84
4.13.POSIX API za sinhronizaciju	86
4.14.Monitori	87
4.15.Barijere	88
4.16.Sinhronizacija procesa - Klasični problemi sinhronizacije	89
4.17.Problem proizvođač – potrošač	89
4.18.Problem pet filozofa	90

4.19. Problem čitača i pisaca	91
4.20. Među-procesna komunikacija	92
4.21. Dijeljene datoteke	92
4.22. Dijeljena memorija i memorijski mapirane datoteke	93
4.23. Cijevi	93
4.24. Prosljeđivanje poruka	95
4.25. Windows Clipboard i DDE	97
4.26. Signali i alarmi	97
4.27. Poziv udaljene procedure	99
4.28. Zadaci	100
5. UPRAVLJANJE RESURSIMA I POTPUNI ZASTOJI	101
5.1. Resursi	101
5.2. Potpuni zastoj	102
5.3. Modeliranje potpunog zastoja grafom alokacije resursa	103
5.4. Graf zastoja sa više resursa iste vrste	103
5.5. Strategije rješavanja potpunog zastoja	104
5.6. Nojev algoritam - ignoriranje problema	104
5.7. Detekcija i otklanjanje potpunog zastoja	105
5.8. Detekcija potpunog zastoja mjerenjem vremena	105
5.9. Detekcija potpunog zastoja grafom alokacije resursa	105
5.10. Detekcija potpunog zastoja matičnom metodom	106
5.11. Oporavak od potpunog zastoja	107
5.12. Izbjegavanje potpunog zastoja	108
5.13. Bankarov algoritam	109
5.14. Sprječavanje potpunog zastoja	109
5.15. Windows API za potpune zastoje	111
5.16. Zadaci	111
6. UPRAVLJANJE LINEARNOM MEMORIJOM	113
6.1. Memorijske adrese	113
6.2. Konfiguracija memorije u jednoprocensnim sistemima	115
6.3. Swapping	116
6.4. Višeprocensni sistemi sa fiksnim memorijskim particijama	116
6.5. Statička relokacija	117
6.6. Dinamička relokacija	118
6.7. Alokacija memorije sa promjenljivim particijama	118
6.8. Algoritmi za izbor slobodne particije	122
6.9. Sistem drugova (Buddy system)	123
6.10. Windows API za upravljanje memorijom	124
6.11. POSIX API za upravljanje memorijom	125
6.12. Dinamički dijeljene biblioteke	126
6.13. Windows API za dinamičke dijeljene biblioteke	127
6.14. Zadaci	128
7. DISKONTINUALNA ALOKACIJA MEMORIJE I VIRTUALNA MEMORIJA	131
7.1. Overlay	131
7.2. Odvojen prostor instrukcija i prostor podataka	132
7.3. Memorijske banke	133
7.4. Virtualna memorija	133
7.5. Segmentacija	134
7.6. Straničenje	137
7.7. Veličina stranice	140
7.8. Invertovana tabela stranica	140

7.9.Hijerarhijsko straničenje	141
7.10.Postupak kod greške stranice	142
7.11.Izbor fizičkih okvira	143
7.12.Izbor stranica za učitavanje.....	145
7.13.Vrijeme brisanja stranica	145
7.14.Algoritmi za izbor stranice za zamjenu.....	145
7.15.Slučajni izbor stranice	146
7.16.FIFO: prvi učitani okvir	146
7.17.LDF (Longest Distance First) - Najduža udaljenost prvo	146
7.18.Optimalni algoritam	147
7.19.NRU (Not Recently Used): okvir koji nije skoro korišten	147
7.20.Druga šansa (Second Chance).....	148
7.21.Satni (Clock page) algoritam	148
7.22.Najdavnije zadnji put korišten - Least recently used (LRU) algoritam	149
7.23.GClock	150
7.24.LFU/NFU i MFU: najrjeđe ili najčešće korišten okvir	150
7.25.Algoritam starenja.....	150
7.26.Satni algoritam sa starenjem	151
7.27. Adaptivna zamjena.....	151
7.28.Algoritam sa radnim skupom.....	152
7.29.WSClock algoritam zamjene stranica.....	154
7.30.Satni algoritam sa dvije kazaljke	155
7.31.Bélády anomalija	155
7.32.Dijeljene stranice.....	155
7.33.Windows API za upravljanje stranicama i dijeljenom memorijom	156
7.34.POSIX API za upravljanje stranicama i dijeljenom memorijom.....	157
7.35.Algoritmi zamjene segmenata	158
7.36.Segmentacija sa straničenjem	159
7.37.Intel i386 memorijska arhitektura.....	160
7.38.Zadaci	163
8. ULAZNO-IZLAZNI SISTEM RAČUNARA	167
8.1.Klasificiranje ulazno-izlaznih uređaja.....	167
8.2.Hardverska struktura U/I sistema	168
8.3.Struktura U/I uređaja	169
8.4. U/I kontroler na matičnoj ploči.....	170
8.5.Kontroler prekida	171
8.6.DMA kontroler.....	172
8.7.Memorijski mapirani i U/I mapirani uređaji.....	173
8.8.Interfejs između U/I kontrolera i uređaja.....	174
8.9.Softverska struktura U/I sistema	174
8.10.Obrađivači prekida.....	175
8.11.Drajveri.....	175
8.12.Principi drajverskih operacija	178
8.13.Windows i POSIX API za pristup perifernim uređajima	179
8.14.Primjer drajvera za Linux.....	180
8.15.Ostale funkcije U/I sistema u jezgri.....	181
8.16.U/I sistem u korisničkom prostoru	183
8.17.Zadaci	185
9. UREĐAJI RAZUMLJIVI KORISNIKU	187
9.1.Tastatura	187
9.2.Miš.....	187
9.3.Ekranski hardver	187

9.4.Windows API za ekran, tastaturu i miša.....	190
9.5. Rad sa ekranom, tastaturom i mišem u Unix sistemima.....	194
9.6.Znakovni terminali i tanki klijenti.....	197
9.7.Štampači.....	198
9.8.Proces štampanja u operativnim sistemima.....	202
9.9.Štampanje u Windows.....	202
9.10.Linux/Unix štampanje.....	203
9.11.Dodirni ekrani.....	204
9.12.Tekstualni korisnički interfejs.....	205
9.13.Tekstualni korisnički interfejs u MS DOS i Windows.....	206
9.14.Unix Shell.....	209
9.15.Osnovne Unix/Linux naredbe komandne linije.....	210
9.16.Programiranje Unix školjke.....	216
9.17.Grafičko korisničko sučelje.....	218
9.18.Unix GUI.....	219
9.19.Zadaci.....	220
10.UREĐAJI ZA KOMUNIKACIJU IZMEĐU RAČUNARA.....	221
10.1.Slojevi mrežnih modela.....	221
10.2.RS232.....	222
10.3.Telefonski, ADSL, VDSL i kablovski modemi.....	224
10.4.USB.....	226
10.5.Ethernet.....	227
10.6.Bežična mreža 802.11.....	229
10.7.Protokoli mrežnog sloja: IP protokol.....	230
10.8. Protokoli transportnog sloja TCP i UDP protokol.....	232
10.9.Aplikativni protokoli.....	232
10.10.Zadaci.....	234
11.UREĐAJI SEKUNDARNE MEMORIJE.....	235
11.1.Magnetni diskovi.....	235
11.2.Organizacija diska.....	235
11.3.Blokovi i zone.....	235
11.4.Klasifikacija diskova.....	237
11.5.Vrijeme pristupa disku.....	237
11.6.Formatiranje diska i pravljenje particija.....	238
11.7.Transformacija logičkih sektora u virtualne ili fizičke.....	239
11.8.Raspoređivanje zahtjeva za rad sa diskom.....	240
11.9.Raspoređivanje unutar jedne staze.....	240
11.10.FCFS.....	240
11.11.SSTF.....	241
11.12.SCAN, LOOK i S-LOOK.....	241
11.13.C-SCAN i C-LOOK.....	242
11.14.F-SCAN, F-LOOK i višestepeni SCAN.....	243
11.15.Raspoređivanje diska u multimedijalnim sistemima i kod nemehaničkih diskova.....	243
11.16.Keširanje.....	244
11.17.Povećanje pouzdanosti.....	245
11.18.RAID strukture.....	245
11.19.RAID 0.....	246
11.20.RAID 1.....	246
11.21.RAID 2.....	246
11.22.RAID 3.....	247
11.23.RAID 4.....	247
11.24.RAID 5.....	248

11.25.RAID 6	248
11.26.Hibridni RAID 0+1, 10 i 50	249
11.27.SSD diskovi.....	250
11.28.Pristup diskovima.....	251
11.29.Izmjenjivi mediji	252
11.30.Magnetni izmjenjivi mediji	253
11.31.Optički mediji CDROM, DVD.....	254
11.32.Izmjenjivi poluprovodnički mediji	255
11.33.Zadaci.....	255
12.LOGIČKA ORGANIZACIJA DATOTEČNOG SISTEMA	259
12.1. Datoteke.....	259
12.2.Datotečni atributi	259
12.3.Fizički sadržaj datoteke	262
12.4.Logička struktura datoteka	263
12.5.Logički pristup datotekama.....	264
12.6.Podaci o datoteci u memoriji	265
12.7.Memorijski mapirane datoteke	266
12.8.Direktoriji.....	266
12.9.Jednonivovski direktorij	267
12.10.Dvonivovski ili jedan direktorij po korisniku	268
12.11.Hijerarhijski sistem direktorija	268
12.12.Aciklični graf i dijeljene datoteke.....	269
12.13.Opšti graf direktorija	270
12.14.Čuvanje i pretraživanje direktorija	271
12.15.Operacije nad datotekama i direktorijima.....	271
12.16.Windows API za rad sa datotekama i direktorijima	271
12.17.POSIX API za rad sa datotekama i direktorijima	273
12.18.Zadaci.....	274
13. FIZIČKA ORGANIZACIJA DATOTEČNOG SISTEMA	275
13.1. Struktura particije.....	275
13.2.Veličina bloka	276
13.3.Metode alokacije datotečnih sistema	276
13.4.Kontinualna alokacija datoteka	276
13.5.ISO 9960 FS.....	277
13.6.Alokacija datoteka ulančanim listama.....	278
13.7.Alokacija sa ulančanim listama sa tablicom pokazivača	279
13.8.DOS/Windows FAT	279
13.9.Indeksna alokacija na jednom nivou.....	281
13.10.CP/M datotečni sistem.....	282
13.11.Indeksna alokacija s više nivoa.....	283
13.12. Unix datotečni sistem.....	283
13.13.Ekstentna alokacija datoteka	285
13.14.NTFS.....	285
13.15.Vođenje evidencije o slobodnim blokovima.....	287
13.16.Konzistentnost i pouzdanost sistema datoteka	289
13.17.Performanse datotečnog sistema	291
13.18.Zadaci.....	293
14.SIGURNOST OPERATIVNOG SISTEMA	299
14.1.Prijetnje	299
14.2.Prirodne katastrofe i hardverske greške	299
14.3.Ljudski faktor.....	299

14.4.	Softverske greške	299
14.5.	Zlonamjerni programi	300
14.6.	Principi zaštite računarskog sistema	300
14.7.	Metode identifikacije	300
14.8.	Prava pristupa	301
14.9.	Metode izolacije	303
14.10.	Metode provjere	304
14.11.	Kriptografija	305
14.12.	Umanjenje štete	308
14.13.	Sigurnost u Unix sistemima	309
14.14.	Sigurnost u Windows sistemima	310
14.15.	Zadaci	314
15.	ARHITEKTURA MICROSOFT-OVIH SISTEMA	315
15.1.	MS DOS	315
15.2.	Xenix, OS 2, Azure Sphere	318
15.3.	Windows	318
15.4.	Šesnaestbitne verzije Windows	318
15.5.	Windows 95, 98, ME	321
15.6.	Windows NT verzije	322
15.7.	Windows NT – Arhitektura	326
15.8.	Hardver	327
15.9.	HAL i Hyper V virtualizator	327
15.10.	NT Kernel	328
15.11.	NT Executive	329
15.12.	Dispečer sistemskih servisa	333
15.13.	Drajveri	334
15.14.	WIN 32 u kernel dijelu	335
15.15.	Prirodni API	335
15.16.	Serveri okruženja	335
15.17.	Server Win32	336
15.18.	Win32 i Win64 bazni API	336
15.19.	OLE, COM/DCOM, ActiveX	337
15.20.	DirectX	339
15.21.	Sistemske procesi	339
15.22.	Servisi	339
15.23.	NTVDM i WOW	340
15.24.	Školjka	341
15.25.	Framework .NET	341
15.26.	Sistemske alati i korisnički procesi	342
15.27.	Windows Runtime	342
15.28.	Windows CE, Phone, Mobile i Pocket PC	342
15.29.	Zadaci	344
16.	ARHITEKTURA UNIX I LINUX	345
16.1.	UNIX historijat	345
16.2.	POSIX	346
16.3.	Linux	346
16.4.	Komponente Linux sistema	347
16.5.	Upravljanje procesima u Linux	349
16.6.	Među-procesna komunikacija u Linux	352
16.7.	Sinhronizacija jezgra	352
16.8.	Upravljanje memorijom	353
16.9.	Datotečni sistemi	354

16.10.Drajveri uređaja.....	355
16.11.Osnovne biblioteke.....	356
16.12.Pozadinski procesi.....	356
16.13.Školjke i osnovne komande.....	358
16.14.Paketni menadžeri.....	359
16.15.X server i bazne grafičke biblioteke.....	359
16.16.Desktop okruženja.....	360
16.17.Korisničke aplikacije.....	361
16.18.Zadaci.....	361
17.OPERATIVNI SISTEMI ZA MOBILNE UREĐAJE.....	363
17.1.Google Android.....	363
17.2.Apple iOS.....	367
17.3.Drugi operativni sistemi za mobilne uređaje.....	368
18.IZVORNI KÔD MALOG JEZGRA.....	369
18.1.Boot sektor.....	369
18.2.Podprogrami za i/o portove.....	370
18.3.Video drajver.....	371
18.4.Drajver tastature.....	372
18.5.Drajver diska.....	374
18.6.Obrađivači interapta.....	375
18.7.Upravljanje memorijom.....	377
18.8.Datotečni sistem.....	380
18.9.Procesi.....	386
18.10.Sistemska poziv.....	389
18.11.Glavni program.....	390
19.LITERATURA.....	391
Knjige.....	391
Članci.....	391
20.INDEKS.....	393

PREDGOVOR

Operativni sistemi su već decenijama stvar svakodnevnice. Od nekadašnjih mainframe računara, preko personalnih računara, konzola za igru, medicinskih uređaja do pametnih telefona, svaki računarski bazirani uređaj ima neku vrstu pogonskog softvera koji omogućava izvršavanje ostalih programa. Svi programi u sistemu koriste usluge operativnog sistema, pa bi se bez pretjerivanja operativni sistem mogao smatrati najvažnijim dijelom računarskog softvera. Principi rada operativnih sistema se stoga izučavaju na gotovo svakom studiju računarstva u svijetu.

Ova knjiga je udžbenik za predmet Operativni sistemi na fakultetima i visokim školama, ali je namijenjena i svima koji pokazuju interes za dublje upoznavanje ove oblasti računarskih nauka. Obuhvata i historijske i moderne koncepte rada operativnih sistema, jer se mnogi moderni koncepti mogu lakše shvatiti ako se prethodno objasne nedostaci i prednosti ranijih, ali jednostavnijih, koncepata. Pojedini podsystemi operativnih sistema prikazani su kroz opise struktura podataka neophodnih za realizaciju njihove funkcionalnosti, algoritama koji na različite načine rješavaju probleme pojedinog podsistema, reference na systemske/API pozive vezane za taj podsistem u Windows i Unix/Linux sistemima, programske primjere upotrebe tih poziva (uglavnom u jeziku C kao najpopularnijem programskom jeziku za systemsko programiranje) i računske primjere koji ilustruju rad važnijih algoritama.

Knjiga ima osamnaest poglavlja koja pripadaju u šest većih cjelina. Ove cjeline opisuju glavne funkcionalnosti operativnih sistema i pravce naučnog istraživanja u ovoj oblasti. Prvu cjelinu sačinjava prvo poglavlje knjige koje predstavlja uvod u operativne sisteme, te navodi neke načine njihove klasifikacije. Druga cjelina opisuje tematiku upravljanja procesima. Toj cjelini pripadaju drugo, treće, četvrto i peto poglavlje. Procesu su opisani kroz prikaz koncepta procesa i struktura podataka koje opisuju procese u memoriji, algoritama njihovog raspoređivanja, tehnika sinhronizacije procesa i međuprocenjske komunikacije i problem potpunog zastoja. Treća cjelina je vezana za upravljanje memorijom i obuhvata šesto i sedmo poglavlje. U ovim poglavljima su objašnjeni jednostavni (linearni) i složeni (banke, segmenti, stranice) načini organizacije memorije, te algoritmi za realizaciju virtualne memorije. Četvrtu cjelinu čine poglavlja koja se bave ulazno/izlaznim uređajima, hardverskim i softverskim principima, najprije zajedničkim, a zatim i onim specifičnim za komunikaciju s korisnikom, mrežne komunikacije i uređajima za pohranu podataka. Peta cjelina je posvećena logičkoj i fizičkoj organizaciji datotečnih sistema, te sigurnosti i zaštiti operativnih sistema. U šestoj cjelini se prelazi na opis konkretnih sistema. Opisani su principi rada i organizacije operativnih sistema Windows, Linux i mobilnih sistema kao što je Android. Posljednje poglavlje sadrži izvorni kôd vrlo jednostavnog jezgra operativnog sistema koje se može samostalno pokrenuti.

Želim da se zahvalim kolegama mr. Nazifu Husoviću i dr. Enisi Brki za pripremu nastavnih materijala koji su bili polazna osnova za višegodišnji razvoj i stalno unapređenje ove knjige.

1.UVOD U OPERATIVNE SISTEME

1.1.Pojam operativnog sistema

Operativni sistem predstavlja sistemski softver koji upravlja hardverskim i softverskim resursima i pruža zajedničke usluge računarskim programima. Gotovo svi računarski programi zahtijevaju operativni sistem za rad, uz rijetke izuzetke poput *firmware*, programa koji je ugrađen u ROM računara ili drugog programibilnog uređaja.

Upravljanje hardverskim i softverskim resursima je briga o korištenju računarske opreme i programa u izvršenju u odgovarajućim trenucima. Čak i ako se u memoriji nalazi samo jedan program, operativni sistem mora da brine da li je on stao u memoriju, kako ga prebaciti sa diska, kako ga informisati da li je štampač na koji je on priključen postoji itd. Situacija se komplikuje kada imamo veći broj programa u memoriji (procesa). Samo je jedna tastatura i treba brinuti o njoj dodjeli samo jednom od procesa, inače sistem nema smisla. Pored toga, procesi dijele memoriju koja je ograničena, procesore kojih je manje nego što ima procesa, štampače... Kako spriječiti da dva procesa istovremeno štampaju na štampaču? Koliki dio memorije i na koji način dodijeliti pojedinom procesu? Kada koji od procesa treba da se izvršava a kada da miruje? Ako sistem koristi grafičko okruženje, kako iscrtati strelicu miša kada prelazi preko određenog prozora? Dodatnu komplikaciju predstavlja računarska mreža. Tada se treba brinuti i o pravima korisnika za pristup računaru, spajanju paketa podataka koji su pristigli ko zna kojim redoslijedom, različitim brzinama mrežnih uređaja... U ovom pasusu spomenuto je nekoliko hardverskih resursa(procesor, memorija, štampač, disk, tastatura) i softverskih resursa(proces, prozor, strelica miša, mrežni paket).

Druga glavna uloga operativnog sistema je pružanje zajedničkih usluga računarskim programima. Mnogi hardverski uređaji su teški za direktno programiranje, koje je uz to podložno greškama. Na primjer, čitanje podataka sa diska zahtijeva slanje komandi njegovom kontroleru uz dobro poznavanje broja ploča na disku i broja staza na svakoj od njih, a zatim prebacivanje podataka bajt po bajt uz brigu o vremenu kada koji dolazi. Čak i ako se uspije napraviti program koji na taj način pristupa disku, za drugačiji tip diska on neće raditi. Da ne govorimo o potencijalnim zahtjevima različitih programa prema disku i konfliktima među njima. Umjesto toga, operativni sistem pruža uslugu **sistemskog poziva**, koga jednostavno poziva korisnički program. Takvi pozivi su puno pogodniji od direktnog pristupa uređaju. U datom primjeru se pristup preko geometrijskih osobina diska i vremenske sinhronizacije zamjenjuje pristupom čitanja datoteke koja ima logičko ime i posmatra se kao niz bajtova, a o njenom fizičkom položaju brine OS.

U modernijem shvaćanju operativnih sistema, oni pružaju i usluge ne samo drugim programima nego i korisnicima kroz aplikacije koje se isporučuju sa njim, pa se postavlja pitanje koliko široku oblast softvera operativni sistem zauzima.

Po najužem shvaćanju u operativni sistem ulaze samo skup sistemskih poziva i apstrakcija hardvera, tj. ono što se inače naziva jezgrom. Ovo stanovište ima uporište i u tekstovima većine udžbenika operativnih sistema. Razlog za ovo mišlje-

nje je što se dijelovi operativnog sistema u korisničkom području mogu mijenjati po korisničkoj želji. Ovo je usko stanovište, jer većina korisnika operativni sistem vidi kroz njegovo korisničko sučelje. Reći da je Windows zapravo datoteka NTOSKRNL.EXE koja sadrži njegovo jezgro za većinu ljudi bi bilo prilično zbunjujuće, jer mu nedostaje način pokretanja korisničkih programa.

Najčešće shvaćanje uključuje u opseg operativnog sistema: jezgro, školjku i skup osnovnih sistemskih alata. Školjka može biti tekstualna ili grafička. Po ovom shvaćanju, u OS ulazi onaj minimum potreban za pokretanje aplikativnih programa.

Da li uključiti aplikativne programe kao sastavni dio operativnog sistema? Komercijalni operativni sistemi, kao što je Windows uključuju programe za pregled web stranica i programe za obradu teksta. Na suđenju 1998 Microsoft je tvrdio da je Internet Explorer sastavni dio operativnog sistema, jer bi njegovo brisanje dovelo do prestanka rada mnogih aplikacija. Po modernom shvaćanju, u operativni sistem ulazi sve što je njegov proizvođač uključio. Ovo shvaćanje je, međutim, previše široko iz ugla proučavanja operativnih sistema, jer bi moglo obuhvatiti gotovo sve teme računarskih nauka, s obzirom na sve aplikacije koje se isporučuju u jednom većem sistemu.

1.2.Klasifikacija operativnih sistema

Razne klasifikacije operativnih sistema prikazane su na slici.



Slika 1 Podjela operativnih sistema

Operativni sistemi se mogu dijeliti prema broju korisnika, broju procesa, licenci, načinu obrade, distribuciji rada procesora i drugih resursa ili namjeni.

1.3.Podjela po broju korisnika, procesa i procesora

Po broju korisnika koji ih istovremeno koriste, OS se dijele na jednokorisničke i višekorisničke. Jednokorisnički OS se obično koristi na stonim računarima, ručnim sistemima i konzolama za igru. Računar je predviđen za samo jednog korisnika, pa su problemi zaštite i dijeljenja resursa jednostavniji. Višekorisnički računari omogućavaju pristup više korisnika istovremeno (preko posebnih terminala ili softvera za terminalski pristup). U nekim slučajevima računarski sistem ima samo jednu tastaturu i ekran, pa ti korisnici ipak ne mogu raditi istovremeno, ali postoji mogućnost prebacivanja između korisnika posebnim komandama.

Prema broju procesa koji se mogu istovremeno izvršavati, operativni sistemi se dijele na jednoprocenane i višeprocenane. Jednoprocenani sistemi dopuštaju da se u memoriji smjesti samo jedan program, dok višeprocenani sistemi dopuštaju više programa u memoriji. Potkategorija jednoprocenih operativnih sistema su jednoprogramski. To su sistemi koji ne samo što u memoriji čuvaju samo jedan program, nego je to i jedini program ikada napisan za njih, što se sreće npr. u neprogramabilnim kalkulatorima ili sistemima koji pokreću ulične semafore.

Po broju CPU (procesora) operativne sisteme dijelimo na sisteme za jednoprocenorske i višeprocenorske računare. OS za višeprocenorske računare predviđeni su za sisteme sa više procesora smještenih u istom kućištu. Procesori komuniciraju putem dijeljene memorije. Postoje dva osnovna principa višeprocenorskih operativnih sistema. Prvi je simetrični, koji sve procesore vidi ravnopravno i svaki procesor izvršava i korisničke programe i operativni sistem. Drugi je asimetrično multiproceniranje, gdje jedan *master* procesor izvršava dijelove operativnog sistema i dodjeljuje zadatke ostalim procesorima.

Moguće je kombinovati neke od ovih podjela i dobiti kombinovane kategorije:

- Jednokorisnički jedno-procenani operativni sistemi, (CP/M, MS-DOS)
- Jednokorisnički više-procenani (MS Windows 98, Android)
- Višekorisnički više-procenani, (UNIX, OS/360, VMS, MS Windows 10)
- Višekorisnički više-procenani za jednoprocenorske računare: (Windows XP Home, Linux 1.0)
- Višekorisnički više-procenani za višeprocenorske sisteme (Windows Server 2016, Linux 2.4)

1.4.Podjela po načinu obrade

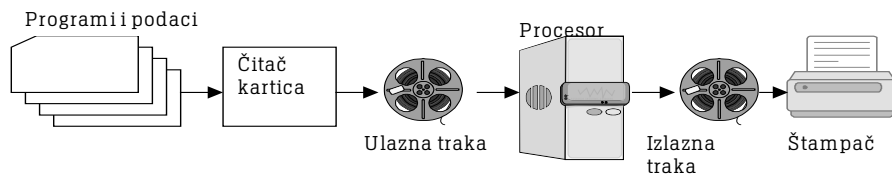
Ima više kategorija načina obrade, po kojima se mogu klasifikovati operativni sistemi.

- Sistemi sa grupnom obradom (engl. *batch*) kojima se predaju poslovi na izvršavanje pomoću ulaznih jedinica. Obrada se obavlja sekvencijalno bez komunikacije između korisnika i posla.
- Interaktivni sistemi u kojima korisnici mogu komunicirati sa računarom u dijeljenom vremenu.
- Hibridni sistemi koji podržavaju grupnu i interaktivnu obradu
- Sistemi u realnom vremenu kod kojih je vrijeme odziva aplikacije suštinsko.
- Distribuirani sistemi, koji dijele obradu između različitih računara.

1.5.Sistemi s grupnom obradom

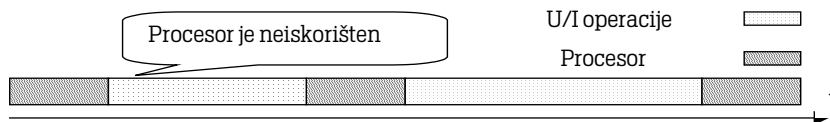
Sistemi s grupnom obradom (engl. *batch*) su najranija vrsta operativnih sistema iz 1960-ih i 70-ih. Ovakvi sistemi su davno iščezli iz upotrebe, ali ih treba spomenuti u proučavanju, jer je iz analize njihovih nedostataka došlo do značajnog razvoja ostalih sistema. Primjer ovakvog sistema je IBM 7090/94 IBSYS, sa ulaznim uređajima poput čitača kartica, izlaznim uređajima kao što je linijski štampač, i ulazno/izlaznim uređajima poput jedinice magnetne trake.

Slika 2 pokazuje tipičan tok podataka u ovakvim sistemima. Korisnik pripremi posao (engl. *job*) za obradu. Posao sadrži program i podatke. Podaci su obično uneseni na bušene kartice i zatim snimljeni na traku. Operater to postavi na sistem, i pokrene izvršavanje. Samo OS je stalno u memoriji. Zadatak ovog primitivnog operativnog sistema je da redom učitava poslove koji se nalaze u redu čekanja jedan po jedan. Svaki posao se učitava, pokrene i sačeka njegovo izvršenje. Korisnik u vrijeme izvršavanja ne sarađuje sa svojim programom. Po završetku rada, program šalje rezultate na izlaznu traku ili štampač.



Slika 2 Batch sistemi

U vrijeme *batch* sistema procesori su bili veoma skupi i njihova iskorištenost je bila značajan faktor, kako bi se investicija u računarski sistem što bolje isplatila. Često su računarski centri iznajmljivali računarsko vrijeme drugim korisnicima. Kako su u svim računarskim sistemima ulazno-izlazni uređaji znatno sporiji od procesora, uočeno je da iskorištenje procesora nije dovoljno. Slika 3 prikazuje vremenski dijagram u kome procesor čeka na ulazno/izlazne uređaje i za to vrijeme ne izvršava korisnički program.



Slika 3 Efikasnost serijskih batch sistema

Diskovi su omogućili da se poslovi smjeste na uređaj koji ima direktni pristup. To je dovelo do sistema kod kojih se također pripremi grupa zadataka, ali se oni sada ne moraju izvršavati istim redoslijedom. Kako je memorija porasla, u nju je sada postalo moguće smjestiti više poslova istovremeno, pa se ti sistemi zovu **multiprogramirani batch sistemi**. Ukoliko neki od poslova pređe u stanje čekanja, operativni sistem može pokrenuti izvršavanje drugog posla. Time se na neki način izvršava više poslova paralelno; npr. dok jedan čeka na završetak ulazno/izlazne operacije, procesor prelazi na izvršavanje drugog.

1.6. Interaktivni sistemi

U *Batch* sistemima korisnici nemaju mogućnost komunikacije sa poslom tokom njegovog izvršavanja, baš kao što pošiljalac pisma ne zna o toku svoje poruke do dobivanja odgovora. Porastom brzine procesora i količine memorije postalo je moguće da korisnik na teleprinteru ili ekranu vidi trenutno stanje izvršavanja njegovog programa i unosi podatke onog trenutka kada se to od njega traži. Time su postale moguće nove primjene računara: skladišno poslovanje, avionske rezervacije,

obrada teksta, unakrsne tablice, igre, video prikazivači itd. Nastala su tri tipa interaktivnih sistema.

- Daljnjim razvojem multiprogramiranih *batch* sistema nastali su sistemi u dijeljenom vremenu. Oni omogućavaju da više korisnika istovremeno koriste jedan računar preko terminala (Multics, Unix, VAX/VMS). U memoriji se nalazi više programa, baš kao i kod multiprogramiranih *batch* sistema. Ali programi se sada ne izvršavaju proizvoljno dugo. Tehnika dijeljenja vremena radi tako da se svaki program u memoriji izvršava u malim intervalima. CPU povremeno prebacuje izvršavanje sa programa na program. To prebacivanje se događa toliko često da korisnik ima osjećaj kontinualnog izvršavanja programa i utiče na njegov rad tokom izvršenja.
- Pad cijena komponenti doveo je računare u kancelarije i kuće. Najprije su nastali interaktivni računarski sistemi koji pružaju neprekidnu komunikaciju između korisnika i sistema, ali je u memoriji samo jedan program, kao kod *batch* sistema (primjer Apple DOS, CP/M, MS DOS, Nokia 30 OS). Unatoč manjoj brzini ovih računara u odnosu na tadašnje sisteme u dijeljenom vremenu, za mnoge korisnike u primjenama poput obrade teksta dobiven je osjećaj bržeg odziva jer se procesorsko vrijeme nije dijelilo s drugim korisnicima. Većina ovakvih sistema ima grafiku visoke rezolucije, što je otvorilo prostor za primjene poput tehničkog crtanja i igara.
- Daljnjim pojeftinjenjem opreme i rastom količine memorije i brzine procesora jednokorisničkim interaktivnim sistemima dodana je tehnika dijeljenja vremena i time mogućnost da korisnik istovremeno utiče na više procesa. Sistemi su obično opremljeni grafičkim korisničkim sučeljem (MacOS, MS Windows, Android).

Interaktivni sistemi uvode pojam procesa umjesto pojma posla. Proces je program u vrijeme izvršavanja. Pojmovi program i proces su slični, ali ipak različiti. Program sadrži samo instrukcije, a proces uključuje i stanje datoteka, memorije, ulazno/izlaznih uređaja i procesorskih registara u toku izvršenja. Program je poput kuharskog recepta, a proces je poput stvarnog kuhanja. Više procesa može biti aktivirano nad programskim kodom istog programa, ali isto tako neki programi mogu stajati na disku godinama, a da se ne pokrenu i time postanu procesi.

1.7.Hibridni sistemi

U prelaznoj fazi između *batch* i interaktivnih sistema, neki operativni sistemi kao što je OS/360 su nudili izbor između dvije vrste obrade. U međuvremenu, skup bušenih kartica sa poslovima zamijenile su datoteke s naredbama jer su se pojavili kontrolni jezici kao što je JCL za IBM Z/OS (iako je u njemu pojam "kartica" dobio značenje linije programa). Iako danas *batch* i hibridni operativni sistemi kao takvi ne postoje, principi *batch* obrade se još sreću u *web* i *cloud* aplikacijama. Uzmimo za primjer aplikaciju PDFExpress, koja priprema dokumente u PDF formatu za naučne simpozije. Pri korištenju ove aplikacije, korisnik preko web preglednika pošalje na server datoteku u formatu MS Word i nakon oko pola sata elektronskom poštom dobiva PDF dokument ili informaciju o greškama.

1.8.Distribuirani sistemi

Distribuirani sistemi (engl. *distributed systems*) dijele procese, datoteke i još neke resurse, ali ne i memoriju i sistemski sat. Korisnik izvana vidi sve kao jedan sistem. U stvarnosti, tu je više računara povezano u lokalnu ili globalnu mrežu sa zajedničkim operativnim sistemom koji upravlja zajedničkim resursima. Posjeta web stranici www.google.com izvana izgleda kao pristup jednom računar, a u stvari je riječ o distribuiranom sistemu u kome učestvuje više miliona računara.

1.9.Sistemi u realnom vremenu

Operativni sistemi u realnom vremenu su specijalni sistemi kod kojih se očekuje odgovor na systemske pozive u tačno definisanom vremenskom roku. Koriste se u industriji, medicinskoj dijagnostici, prikazu filmova, kontroli naučnih eksperimenata i slično. Ovi sistemi dobivaju podatke od senzora, raznih prekidača, pretvarača analognih i digitalnih signala i drugo.

1.10.Podjela sistema po namjeni

Prema namjeni, OS se mogu podijeliti u više kategorija iako mnogi sistemi istovremeno pripadaju u više njih.

- Sistemi za velike računare ili mainframe sistemi. Nakon početnih sistema sa grupnom obradom, ovi sistemi uglavnom omogućavaju istovremeni pristup preko terminala većem broju korisnika, koristeći tehniku multiprogramiranja. Primjeri ovih sistema su OS/360, z/OS, VAX/VMS, Unix.
- Personalni (*desktop*) operativni sistemi. Namijenjeni su jednom korisniku. Koriste tekstualno ili grafičko korisničko sučelje (GUI) . Sistemi su prilagodljivi i široko korišteni. Primjeri ovih operativnih sistema su CP/M, MS DOS, Amiga OS, MS Windows, MacOS, Novell NetWare, Linux.
- Mrežni operativni sistemi su namijenjeni komunikaciji računara povezanih u mrežu, ali sami računari imaju visok stepen autonomije. Korisnici pojedinih računara mogu pristupati resursima udaljenih mašina kroz udaljenu prijavu ili razmjenu datoteka. Svaki računar je samostalan i ima svoj OS. Različiti računari razmjenjuju podatke koristeći mrežne protokole. Danas je najpopularniji takav protokol TCP/IP.
- Serverski operativni sistemi su potkategorija mrežnih operativnih sistema. Namijenjeni su za računare koji pružaju podatke drugim računarima. Za razliku od *mainframe* računara, korisnički programi se ne izvršavaju na serverima interaktivno, ali pozadinski programi stalno komuniciraju sa korisničkim programima.
- Distribuirani sistemi (engl. *distributed systems*) obavljaju umrežavanje računara na takav način da se cio sistem vidi kao jedan računar, iako su oni kolekcija računara koji ne dijele memoriju i zajedno učestvuju u rješavanju problema.
- Udruženi sistemi (engl. *clustered systems*) su podvarijanta distribuiranih sistema sastavljenih od računara u lokalnoj mreži. Udruživanje može biti asimetrično i simetrično. Asimetrično udruživanje je namijenjeno za povećanje pouzdanosti. U ovom slučaju jedan server izvršava aplikaciju. Ostali serveri su

prateći (*monitoring*) i uključuju se u slučaju njegovog pada. Simetrično udruživanje, s druge strane, povećava performanse, jer su svi serveri učestvuju u izvršavanju aplikacije.

- Operativni sistemi za upravljanje u realnom vremenu (engl. *real-time systems*) su sistemi kod kojih se očekuje odgovor u tačno definiranom vremenskom intervalu. Odlikuju se veoma malom veličinom programskog koda, izuzetnom pouzdanošću i većom mogućnosti uticaja na način raspoređivanja procesa.
- Operativni sistemi za ručne i džepne uređaje (engl. *handheld systems*), se instaliraju u uređajima fizički malih dimenzija kao što su mobilni telefoni, tableti, digitalni rokovnici, pametni satovi itd. Ovi sistemi se odlikuju prenosivošću, ali imaju slabije karakteristike od personalnih računara (manja brzina, manje memorije, mali ekrani, nekomforni ulazni uređaji).
- Operativni sistemi za ugrađeni sisteme (*embedded systems*) su u uređajima čiji korisnik i ne zna da su u pitanju računarski sistemi (POS kase, pametne kartice, pametni televizori, mašine za pranje rublja, automobili, kalkulatori itd.).

1.11.Jezgro operativnog sistema

Jezgro (engl. *kernel*) je osnovna komponenta OS koja predstavlja njegov najniži sloj izveden u softveru. Zadatak mu je da izvršava kritične funkcije: upravljanje primarnom memorijom, upravljanje procesima, upravljanje prekidima, smještanje datoteka na disk, komunikacija s perifernim uređajima i drugo. Ono je veza između hardvera i ostalih slojeva operativnog sistema, tako da ono kod većine sistema direktno pristupa registrima perifernih uređaja. Neki operativni sistemi kao što je Windows NT ili sistemi bazirani na virtualnim mašinama, ipak ne pristupaju uvijek direktno hardveru, nego se između jezgra i hardvera dodaje još jedan sloj, sloj apstrakcije hardvera (engl. *Hardware Abstraction Layer*, HAL), odnosno virtualizator.

1.12.Podjela operativnih sistema po strukturi jezgra

Zavisno od toga koliki procenat funkcionalnosti operativnog sistema se izvršava u najvećem nivou privilegija jezgra, podjela operativnih sistema se može napraviti u tri osnovne grupe, koje se dalje mogu dijeliti. Prva grupa su monolitni sistemi. U monolitnim sistemima većina funkcionalnosti jezgra ili čak cijelog operativnog sistema se izvršava u režimu najvećih privilegija. Monolitne možemo podijeliti na potpuno monolitne, sisteme monolitnog jezgra, modularne, slojevite, prstenaste i *unikernel*. U drugu kategoriju spadaju sistemi sa mikro jezgrom. Kod ovih sistema se nastoji smjestiti u režim najveće privilegije što je moguće manje programskog koda. Njih dijelimo na mikro jezgro sa serverima, OS sa virtualnim mašinama, nano jezgra i *egzokernel*. Treću grupu predstavljaju jezgra koja su između ove dvije krajnosti, hibridna jezgra.

1.13.Potpuno monolitni sistemi

Najjednostavniji i najmanje fleksibilan operativni sistem je onaj u kome je apsolutno sve: jezgro, korisničko sučelje i sistemski alati, spakovano u jednu strukturu, koja je obično u ROM memoriji. Jedino što je odvojeno su korisnički procesi. Nikakve modifikacije sistema nisu moguće: dodavanje novih perifernih

uređaja ili novih alata bez izmjene cijelog OS. Primjer: kućni računari iz 1981. sa jezikom BASIC u ROM-u kao operativnim sistemom.

1.14. Sistemi monolitnog jezgra

Daleko najčešći način organizacije jezgra nema posebnu strukturu, kao što je slučaj sa MSDOS ili UNIX. OS je napisan kao skup potprograma, od kojih svaki može pozvati bilo koji drugi potprogram kad god je to potrebno. Ako se koristi ova tehnika, svaki potprogram u sistemu ima dobro definirano značenje parametara i rezultata, a svaki potprogram je ima pravo pozvati bilo koji drugi. Cijelo jezgro se izvršava kao jedinstveni proces i svi potprogrami jezgra imaju pune privilegije.

Monolitna jezgra se odlikuju dobrim performansama, jer je potrebno manje prelazaka između različitih nivoa privilegije. Ako npr. dijelu za upravljanje memorijom zatreba privremeno snimanje sadržaja memorije na disk, dovoljno je pozvati potprogram za snimanje na disk. Glavne mane monolitnog jezgra su:

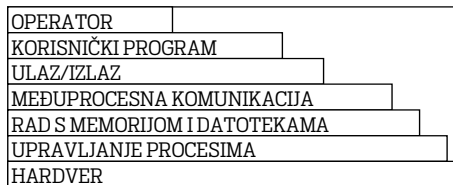
- Greška, čak i u nebitnom dijelu jezgra, npr. nekoj rijetko korištenoj mrežnoj funkciji, može dovesti do prestanka rada cijelog operativnog sistema.
- Razumijevanje principa rada jezgra je daleko teže zbog ogromnog broja međuzavisnosti potprograma koje je teško pratiti.
- Dodavanje nove funkcionalnosti (npr. podrška za veće diskove) zahtijeva pravljenje novog jezgra.
- Nepotrebne funkcionalnosti (npr. podrška za mrežnu karticu koju nemamo) zauzimaju memorijski prostor, a u nekim slučajevima troše i procesorsko vrijeme.

1.15. Unikernel

Ekstremni slučaj osobine monolitnog jezgra da cijelo jezgro ima maksimalne privilegije je *unikernel*. Kod ovog koncepta jezgra u jedinstvenu datoteku se spajaju jezgro, biblioteke i aplikativni program u zajedničkom memorijskom području. Time se dobivaju maksimalne performanse, ali se gube svi zaštitni mehanizmi, jer jezgro nije više ugroženo samo od samog sebe, nego i od korisničkog programa. Ali ako se koristi u kombinaciji sa hipervizorom (uskoro će biti objašnjen) onda se rješava i ovaj problem.

1.16. Slojeviti sistemi

U cilju povećanja razumijevanja rada jezgra, jedan način strukturiranja OS je urediti operativni sistem kroz više slojeva, tako da je svaki sloj postavljen logički jedan iznad drugog. Jedan operativni sistem strukture slojeva realizovan je na *Technische Hogeschool Eindhoven* u Nizozemskoj, autor Edgser Dijkstra. Sistem je imao 6 slojeva (Slika 4).



ARHITEKTURA THE SISTEMA

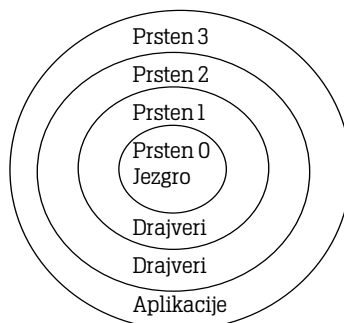
Slika 4: Slojeviti sistem

Sloj iznad hardvera se bavio dodjelom procesora, prebacivanjem između procesa kada je došlo do prekida ili isteklo predviđeno vrijeme procesa. Svaki sloj predstavlja

grupu funkcija operativnog sistema. Potprogrami koji pripadaju jednom sloju smiju pozivati samo potprogramme iz istog ili nižih slojeva. Time se smanjuje mogućnost da promjena jednog dijela operativnog sistema drastično utiče na njegov ostatak. Koncept slojeva ima i svoje nedostatke. Neke funkcionalnosti je teško uklopiti u postojeće slojeve. Neka, na primjer, operativnom sistemu treba dodati mrežne mogućnosti, koje treba uključiti u novi sloj. Ako se taj sloj postavi ispod sloja za rad sa memorijom i datotekama sistema, neće biti moguće čitanje datoteka za konfiguraciju mreže. Ako se mrežni sloj postavi iznad sloja za datoteke i memoriju, tada neće biti moguće dijeliti datoteke na mrežnim diskovima.

1.17.Prstenasti sistem

Varijanta koncepta slojeva se pojavila u sistemu MULTICS. Slojevi u MULTICS sistemi su opisani kao niz koncentričnih prstena (Slika 5), gdje unutrašnji prsten ima više privilegija od vanjskog. Kada potprogram u vanjskom prstenu želi pozvati rutinu u unutrašnjem prstenu, on mora pozvati sistemski poziv, izvršivši TRAP instrukciju čiji se parametri provjere. Za razliku od slojevitog sistema, prsteni ne grupišu potprogramme po funkcionalnosti nego po pravima. Neispravnost pojedinog drajvera, npr. za štampač, neće nužno srušiti cijelo jezgro jer se izvršavaju u odvojenim područjima privilegija, ali će ostali drajveri u istom prstenu i dalje biti ugroženi.



Slika 5: MULTICS arhitektura

1.18.Modularno jezgro

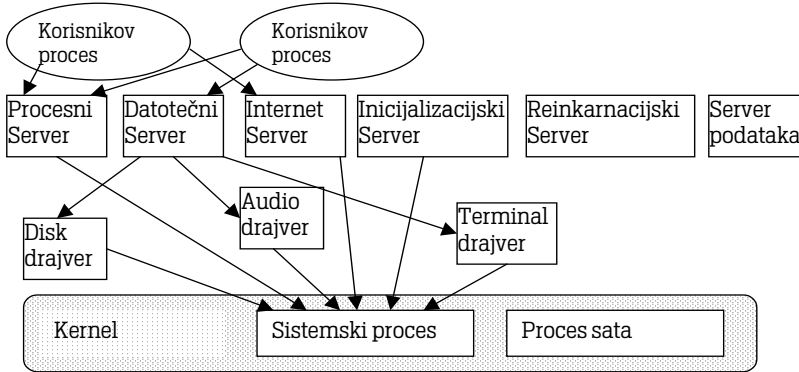
Linux je od verzije 2.0 zadržao koncept monolitnog jezgra, ali i riješio njegova dva nedostatka: teško dodavanje nove funkcionalnosti i zadržavanje nepotrebnih funkcionalnosti. Svi dijelovi jezgra, uključujući i drajvere su u istom području privilegija. Međutim, ne moraju biti svo vrijeme u memoriji. Jezgro se sastoji od glavne datoteke i većeg broja manjih datoteka (modula) koji obično predstavljaju drajvere za periferni uređaje ili neke mrežne protokole. U memoriju se ubacuju samo oni moduli koji su potrebni, npr. dovoljno je imati samo jedan drajver za mrežnu karticu u memoriji, i to za model koji posjedujemo. Moduli se mogu i izbaciti iz jezgra ako više nisu potrebni.

1.19.OS s mikro jezgrom

Neke nedostatke monolitnih jezgara možemo zaobići tako što će se ukloniti što je više moguće funkcionalnosti sa privilegijama jezgra, ostavljajući minimalno mikro jezgro (engl. *microkernel*). Većina operativnog sistema u tom slučaju se implementira u korisničkim procesima. Kada traži uslugu, npr. čitanje datoteke, korisnički proces poziva serverski proces, koji odradi svoj posao i vrati odgovor.

U OS sa mikro jezgrom, sve što jezgro radi je realizacija komunikacije između korisničkih procesa i serverskih procesa. OS je razbijen na dijelove, od kojih je svaki samo upravlja jednim aspektom sistema, kao što su usluge datoteka, raspoređivanje procesa, pristup terminalu, ili upravljanje memorijom. Time svaki dio postaje mali i upravljiv. Pošto se svi serverski procesi izvršavaju u korisničkom režimu, a ne u

režimu jezgra, oni ne pristupaju direktno hardveru. Posljedica je da npr. greška na serveru datotečnog sistema neće oboriti cijeli operativni sistem, nego samo taj podsistem. Podsistemi su u odvojenim područjima memorije, pa ne komuniciraju međusobno niti pozivom potprograma niti dijeljenom memorijom. Komunikacija između podsistema obavlja se razmjenom poruka koje raspoređuje mikro-jezgro.



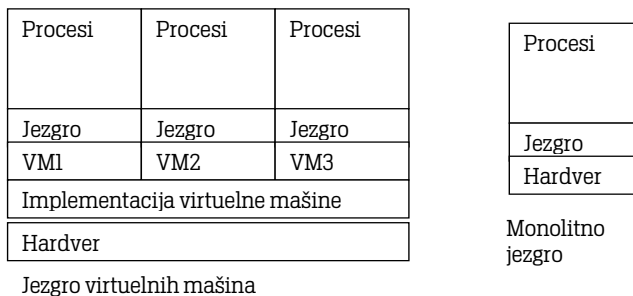
Slika 6 Arhitektura Minix sistema s mikro jezgrom

Mana mikro jezgra je u slabijim performansama, pošto se prelazak iz sistemskog u korisnički režim rada dešava prilično često. Najpoznatiji operativni sistemi sa mikro jezgrom su Mach, Minix, Symbian i Fuchsia. Slika 6 prikazuje osnovnu arhitekturu Minix 3 sistema. U jezgru se nalaze samo dva procesa, sistemski i proces sata. Sve ostalo, uključujući i drajvere perifernih uređaja i datotečni sistem se nalazi u korisničkom režimu rada, u formi odvojenih servera. Postoji samo pet sistemskih poziva, koji prelaze u režim jezgra, uglavnom vezanih za slanje i prijem poruka. Nije moguće direktno upisivati iz servera na memorijske apsolutne lokacije, nego se zahtjev za tim obrađuje u jezgri. Minix 3 povećava svoju pouzdanost koristeći reinkarnacijski server, koji proziva poznate podsisteme i u slučaju pada nekog od njih pokreće iz početka podsistem koji se ne odaziva, ako je to moguće.

Koriste se i termini nano jezgro i piko jezgro (*nanokernel*, *picokernel*). Kod ove vrste jezgra ne samo što je u režimu privilegija jezgra minimalna funkcionalnost, nego je ona realizovana sa što je moguće manje programskih instrukcija u usporedbi sa nekim operativnim sistemima klasifikovanim kao sistemima sa mikro jezgrom.

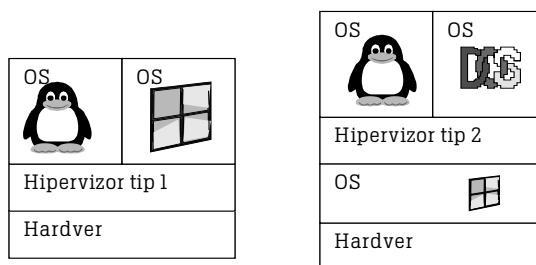
1.20.OS preko virtualnih mašina

U sistemu IBM VM/370 uveden je operativni sistem zasnovan na virtualnim mašinama. Umjesto jezgra, najbliže hardveru nalazi se monitor virtualnih mašina, koji realizuje virtualne mašine za ostale dijelove operativnog sistema. Za razliku od ostalih OS, te virtualne mašine ne predstavljaju adresni prostor jednog procesa sa datotekama, sistemskim pozivima i povećanim virtualnim područjima memorije. Ove virtualne mašine su potpuna kopija hardvera, i uključuju režime privilegija, ulazno/izlazne uređaje, prekide itd.



Slika 7 OS preko virtualnih mašina

Kako je ovakva virtualna mašina savršena simulacija pravog hardvera, ona može izvršiti bilo koji operativni sistem koji bi radio nad samim hardverom. Tako, dok klasično, monolitno jezgro izvršava jedan operativni sistem, ovim pristupom je moguće pokrenuti više operativnih sistema istovremeno na istom računaru (Slika 7). Na primjer neke virtualne mašine izvršavaju OS/360 za *batch* obradu ili obradu transakcija, dok ostale izvršavaju jednokorisnički operativni sisteme CMS.



Slika 8 Dvije vrste hipervizora

Od 2010. godine ovaj koncept operativnog sistema postao je moguć na personalnim računarima. Procesori Intel Core 2 imaju režim za virtualizaciju koji generiše izuzetke kada se izvršavaju privilegovane instrukcija. Pri obradi izuzetaka se simulira ponašanje prave mašine na virtualnoj mašini. Time postaje moguće realizovati monitor virtualnih mašina (zovu ga hipervizor tipa 1), čiji su primjeri Oracle VM i Microsoft Hyper V. Hipervizor tipa 1 se izvršava direktno nad hardverom. Za razliku od njega, hipervizor tipa 2, se izvršava kao korisnički program (npr. Qemu, VMWare) u već aktivnom operativnom sistemu i unutar njega izvršava drugi operativni sistem (Slika 8, desno).

OS preko virtualnih mašina je posebno koristan za serverske operativne sisteme. Firmama je često potrebno da imaju servere za elektronsku poštu, baze podataka, mrežno posredništvo i dijeljenje datoteka. Iz razloga pouzdanosti dobro bi bilo da je svaki od njih poseban računar, pa ako padne operativni sistem na računaru za elektronsku poštu, podaci iz baze su i dalje dostupni. Ali to znači i veću potrošnju električne energije, prostora i manju iskorištenost hardvera. Stavljanjem četiri virtualizovana operativna sistema nad hipervizorom u istoj fizičkoj mašini postiže se zaštita od rušenja jednog od operativnih sistema, a dovoljan je samo jedan računar.

1.21.Exokernel

U OS preko virtualnih mašina hipervizor mora da brine o tome koji je hardver zauzela svaka virtualna mašina. Svakoj virtualnoj mašini izgleda kao da ima svoj kompletan disk, koji ima jednak broj blokova kao i fizički disk. Zbog toga monitor virtualnih mašina treba pratiti koji blok tog diska je dodijeljen kojoj virtualnoj mašini i održavati tablice korištenih disk blokova. Jednostavnija varijanta sistema virtualnih mašina vidi se u sistemu Aegis kod koga svaki korisnik dobije djelomičnu kopiju fizičkog računara, koja ima podskup resursa. Tako se npr. prvoj virtualnoj mašini dodjeljuje prvih 1024 bloka na disku, druga dobija blokove diska između 1024 i 2047, i tako dalje. Slično se može podijeliti i fizička memorija. Na najnižem sloju, u režimu jezgra, je program *exokernel*. On dodjeljuje resurse za virtualne mašine i kontroliše njihovo korištenje kako bi bio siguran da neka mašina ne pokušava koristiti resurse dodijeljene drugoj. Svaka virtualna mašina može pokrenuti vlastiti operativni sistem, koji je ograničen na dio resursa. Prednost *exokernel* pristupa je u puno jednostavnijim tablicama mapiranja stvarnog hardvera u virtualni.

1.22.Hibridno jezgro

Hibridno jezgro spaja ideje monolitnog jezgra sa idejama mikro jezgra. Jezgro sistema je razdvojeno na osnovno jezgro, koje je vrlo ograničene funkcionalnosti (obrada prekida i izmjena procesa), uz dijelove za upravljanje resursima sistema. Za razliku od mikro-jezgra oba dijela ostaju u istom memorijskom prostoru. Pouzdanost je manja u odnosu na čisto mikro-jezgro ali je brzina veća. Ipak, sistem je bolje interno strukturiran od monolitnog jezgra. Najpoznatiji primjer hibridnog jezgra je Windows NT. Njegov podsistem koji se zove Kernel je mali, dok se dio sistema koji upravlja resursima naziva Windows Executive i on je u adresnom prostoru jezgra. Aplikativni programi, ne pozivaju Executive direktno sistemskim pozivima u jezgro, nego koristeći servere podsistema koji komuniciraju s jezgrom i pružaju Windows, OS/2 ili POSIX API (pri čemu je Windows API daleko najčešći).

1.23.Prekidi

Jezgro i korisnički procesi su u odvojenim memorijskim prostorima, procesor je u različitim režimima koji imaju različita pravila. Često je potrebno prelaziti između ta dva režima rada. Ako je procesor u režimu sa višim privilegijama (jezgro), on može preći u korisnički režim rada postavljanjem vrijednosti specijalnog registra načina rada. Iz korisničkog režima, međutim, ne može se na isti način preći u režim jezgra, zbog ograničenih prava. Stoga se prelazak iz korisničkog u režim jezgra odvija korištenjem prekida. Postoje tri vrste prekida: softverski prekidi, hardverski prekidi i izuzeci.

Softverski prekid se pokreće kada korisnički program izvrši specijalnu instrukciju. Ona prisiljava program da pređe u sistemski režim rada i nastavi izvršavanje od dobro poznate memorijske lokacije. Lokacija se određuje na osnovu broja prekida, koji je obično parametar ove instrukcije. Program koji se izvršava na toj lokaciji je pod kontrolom jezgra. Primjeri softverskog prekida su instrukcije **int** ili **sysenter** na Intel Pentium procesorima; **syscall** na AMD procesorima ili **SWI** na ARM procesorima.

Hardverski prekid nastaje kada vanjski događaj pošalje prekidni signal procesoru. Događaji mogu biti pritisak na taster, završetak direktnog prijenosa podataka sa diska u memoriju, periodični otkucaj sata itd. Hardverski prekidi mogu nastati u bilo koje vrijeme. Za razliku od softverskih prekida i izuzetaka, oni nisu vezani za instrukciju koja se trenutno izvršava.

Izuzeci se dešavaju ako korisnički program pokuša izvršiti instrukciju koja se smije izvršiti samo u režimu jezgra, ili pokušava pristupiti području memorije koje njemu ne pripada. Procesori imaju svoj skup izuzetaka koje poznaju. Izuzeci mogu biti kršenje pristupa memoriji, dijeljenje nulom, pokušaj izvršenja nepostojeće instrukcije, ili pristup nepostojećoj lokaciji memorije. U obradi izuzetaka jezgro može simulirati željeno ponašanje (npr. izvršenje instrukcije za aritmetiku sa realnim brojevima na sistemima koji nemaju instrukcije za to) ili sankcionisati neželjeno (prekinuti program koji pokušava pristupiti memoriji dodijeljenoj drugom procesu).

Svaki od prekida ima pridruženu tabelu sa lokacijama na koje procesor skače zavisno od vrste i numeričke oznake prekida. Kada se desi prekid, programski brojač se sačuva na *stack* i program nastavlja izvršavanje od lokacije u navedene u tabeli za navedeni prekid, čime počinje izvršavanje prekidne rutine. Istovremeno se izvrši prebacivanje iz korisničkog režima u režim jezgra. Tabela je pripremljena pri inicijalizaciji operativnog sistema. Nju može mijenjati samo operativni sistem i ona sadrži samo dobro poznate lokacije. Nakon ulaska u prekidnu rutinu, izvršava se jezgro operativnog sistema. Po njenom završetku, kada je operativni sistem spreman da vrati kontrolu korisničkom procesu, on izvršava instrukciju povratka (naredba IRET na Intel procesorima). To uključuje povratak procesora natrag u korisnički način rada i preuzima adresu sa vrha *stack* -a. Svakom od tri vrste prekida jezgro upravlja na drugi način.

- Kod softverskog prekida provjere se parametri koji su obično navedeni u procesorskim registrima. Ako parametri predstavljaju regularan način poziva usluge operativnog sistema, izvršava se odgovarajući sistemski poziv.
- Hardverski prekidi pokreću rutine koje obično očitavaju podatke koje je periferni uređaj poslao u međuvremenu i vraćaju procesor u stanje prije poziva
- Izuzeci se tretiraju zavisno od toga šta ih je izazvalo. Ako je izuzetak zbog nepostojeće instrukcije, može se simulirati njeno ponašanje sporijim, ali postojećim instrukcijama. Izuzeci zbog pristupa memoriji mogu nastati iz dva razloga. Jedan je pristup regularnoj memorijskoj lokaciji koja nije trenutno dostupna. U ovom slučaju, se sa diska učitava potreban sadržaj memorije i ponovi instrukcija. Ukoliko memorijska lokacija nije važeća, operativni sistem može poslati signal procesu ili ga prekinuti. Ako se pokušava izvršiti privilegovana instrukcija, akcija zavisi od toga da li je bila aktivna virtualizacija ili ne. U slučaju virtualizacije, simuliraju se akcije privilegovanih instrukcija, a ako nije virtualizacija u pitanju, proces može biti prekinut.

1.24.Sistemska poziv

Najčešća primjena softverskih prekida je sistemski poziv. Korisnički procesi normalno ne mogu pristupiti jezgru, ali program često treba čitati datoteku, slati

podatke nekom uređaju, ili možda pokrenuti neki drugi program. Za sve ovo je potrebna akcija jezgra operativnog sistema. Sistemski pozivi su definirane ulazne tačke u jezgru koji obezbjeđuju usluge korisničkim procesima. Realizacija sistemskih poziva koristi mehanizam softverskog prekida koji omogućuje procesu da se prebaci na dobro definirane tačke u jezgru.

Sistemski pozivi u Linux se pozivaju tako da se u EAX stavi redni broj sistemskog poziva, a njegovi parametri redom smještaju u registre EBX, ECX, EDX, ESI i EDI. Nakon ovoga se pozove softverski prekid broj 80h. Prvih 30 poziva je u tabeli.

1	exit	7	waitpid	13	time	19	lseek	25	stime
2	fork	8	creat	14	mknod	20	getpid	26	ptrace
3	read	9	link	15	chmod	21	mount	27	alarm
4	write	10	unlink	16	chown	22	umount	28	oldstat
5	open	11	execve	17	break	23	setuid	29	pause
6	close	12	chdir	18	oldstat	24	getuid	30	utime

Na primjer, sistemski poziv *getpid* (dobavljanje interne oznake procesa) na Intel/Linux sistemima se poziva unosom broja 20 u registar EAX (20 je redni broj *getpid* sistemskog poziva), i izvršavanjem instrukcije INT 80h, koja aktivira softverski prekid. Rezultat poziva će se dobiti u registru EAX. Sljedeći asemblerski program poziva tu funkciju.

```
mov    eax, 20 ; Poziv getpid
int    80h
```

I Windows NT i njegovi nasljednici uključujući Windows 10 imaju sličan način sistemskog poziva, ali je on vrlo malo poznat. Da se pozove usluga jezgra operativnog sistema koristi se jedna od tri instrukcije: *int 2Eh* (na 486 procesoru), *syscall* (na AMD procesorima) ili *sysenter* (na novijim procesorima novijim od Pentium II). Na primjer sistemski poziv *NtQuerySystemInformation* (poziv broj 97h) može se pozvati na sljedeći način:

```
mov    eax, 97h ;
lea    edx, [esp+4]
int    2Eh ; ili sysenter, syscall
ret    10h
```

1.25.API pozivi

Mada postoje načini da programer poziva systemske pozive, češće se koriste viši nivoi apstrakcije. Problem sa sistemskim pozivima je što su često loše dokumentovani i nekompatibilni između verzija operativnih sistema. Direktno pozivanje sistemskog poziva u Linux-u koristeći INT 80h će raditi samo na određenom procesoru, ili čak samo na određenoj verziji jezgra Linux-a za taj procesor.

Umjesto sistemskih poziva, prilikom programiranja, preporučuje se da programer koristi samo API (*application programming interface*) operativnog sistema, ili ako želi postići i manju ovisnost o operativnom sistemu, standardnu biblioteku programskog jezika. Sistemi pružaju biblioteke ili API koji se nalazi između korisničkog programa i operativnog sistema. Na sistemima sličnim Unix, taj API je obično dio standardne biblioteke jezika (*libc*). Jedan dio funkcija u toj biblioteci je omot za systemske pozive, skup funkcija koje se zovu kao sistemski pozivi koje

pozivaju, ali prosljeđuju parametre na način koji je pogodniji za jezik C nego za assemblerski jezik. Na Windows NT, API koji omotava systemske pozive se zove *Native API*, smješten u ntdll.dll biblioteci. *Native API* je slabo dokumentirani API koga koriste funkcije regularnog Windows API i neki systemski programi za Windows. Poziv funkcije omotač-biblioteke predstavlja običan poziv potprograma, koji sam po sebi ne prelazi u režim jezgra, nego prepakuje parametre i pozove odgovarajući softverski prekid. Na ovaj način, biblioteka koja postoji između jezgra OS i aplikacija, povećava prenosivost programa.

1.26. WINDOWS API

U slučaju Windows-a, programeri imaju više skupova API, raznih nivoa (Slika 9). Na slici navedene su verzije Windows-a u kojima je taj API skup moguće pozivati. Kao ilustracija, navede se ime funkcije koja pokreće novi proces u svakom od njih.

Naziv	Verzije Windows	Namjena	Funkcija za kreiranje procesa
Kernel API	NT,2000,XP, Vista,7,8,10	Funkcije u režimu jezgra koje pozivaju samo dražeri.	PspCreateProcess
Native API	NT,2000, XP, Vista, 7, 8,10	Klasični systemski poziv koji izaziva prelazak u režim jezgra, rijedak u aplikativnim programima	NtCreateProcess
Win32 API	95,98,ME,NT, 2000,XP, Vista, 7, 8,10	Najviše korišten API, u velikom broju dijeljenih datoteka i poziva se kroz funkcije u C. Uz njega su još i objektno bazirani API COM, DirectX, ActiveX,	CreateProcess
NET API	XP,Vista, 7, 8,10	Velika biblioteku za objektno-orijentirane jezike pruža priličnu neovisnost od programskog jezika	Process.Start
WinRT	8,8.1,10	Objektno-orijentirani API dizajniran za saradnju s drugim jezicima koristeći metapodatke	Windows.System.Launcher. LaunchFileAsync
Win16 API	1,2,3,95,98, ME,NT,2000, XP, Vista	Šesnaestbitna verzija Windows API	WinExec
DOS API	1,2,3,95,98, ME,NT,2000, XP, Vista	Programi za MSDOS	INT 21, AH=4Bh
Java API	98,ME,2000	Za izvršavanje programa u jeziku Java	Runtime.exec
OS/2 API	NT, 2000	Izvršavanje programa za OS/2	DosExecPgm
POSIX API	NT, 2000, XP	Izvršavanje programa za Unix	fork, execve

Slika 9 Aplikacijski programski interfejsi u verzijama operativnog sistema Windows

Najveći procenat računara s Windows operativnim sistemom poznaje Win32 API, pa će se u daljem tekstu reći nešto o njemu. Win32 API je dizajniran za upotrebu u C/C++ programima i predstavlja direktni način za vezu između Windows sistema i softverskih aplikacija.

Funkcionalnost koju nudi Win32 API dijeli se više kategorija. **Osnovne usluge** pružaju pristup najvažnijim resursima Windows sistema. Te usluge su datotečni sistem, upravljanje procesima i nitima, pristup uređajima, pristup Windows *registry*-ju, i praćenju grešaka. Većina ovih funkcija je u datotekama kernel32.dll i advapi32.dll. **Grafičke funkcije (GDI)** omogućavaju crtanje na ekranu, štampaču i datotekama sa vektorskom grafikom. Funkcije su locirane u datoteci gdi32.dll, a od Windows XP i u datoteci gdiplus.dll. **Korisničko sučelje** uključuje funkcije za

upravljanje prozorima, prijem ulaza miša i tastature, rad sa osnovnim kontrolama (dugmad, unosna polja i klizne trake), dodatnim kontrolama (trake napretka, prikazi stabla, kartice), standardne dijaloške prozore za otvaranje i snimanje datoteka, izbor štampača i fontova itd. Ove funkcije su u bibliotekama user32.dll, comctl32.dll, comdlg32.dll, shell32.dll i shlwapi.dll. **Mrežne usluge** omogućavaju pristup aplikativnih programa računarskoj mreži. U tu kategoriju spadaju Winsock (za pravljenje Internet servera), NetBIOS (mogućnost mrežnog imenovanja računara i dijeljenja datoteka), NetDDE (razmjena podataka preko mreže), RPC (pokretanje programa na daljinu) i mnogi drugi. **Web API** omogućava da se funkcije koje pruža Internet Explorer također mogu koristiti iz korisničkih programa. Ove funkcije se nalaze u grafičkoj kontroli koja uključuje web preglednik u aplikacije (shdocvw.dll i mshtml.dll). **Multimedijalni API** namijenjen je za aplikacije koje se bave zvukom i video zapisima. Lociran je u biblioteci WINMM.DLL i većem broju biblioteka pod imenom DirectX.

1.27.POSIX API

Sistemi srodni Unix sistemima koriste POSIX API, koji je standard za funkcije operativnog sistema. Ako aplikacije koriste ovaj API, njih je daleko lakše prenositi s jednog operativnog sistema na drugi, bez obzira koje se jezgro koristi. POSIX standard opisuje određene zajedničke usluge, koje odgovarajući operativni sistem mora pružiti softveru (npr. kako se kreiraju datoteke) zajedno sa konvencijama o tome kako se pozivaju iz programa. Program napisan za jedan POSIX-usklađeni OS će biti lakše prebačen na drugi POSIX usklađen OS nego na OS koji nije POSIX tipa. To je razlog zašto je mnogo lakše prebaciti aplikaciju sa FreeBSD na Linux nego sa FreeBSD na Windows.

1.28.Popularnost operativnih sistema

Na koje operativne sisteme obratiti posebnu pažnju? U trenutku pisanja ove knjige većina operativnih sistema ima jezgro bazirano na Windows, Linux ili Unix. Windows je dominantan na desktop i laptop računarima. Procjenjuje se da oko 88% aktivnih operativnih sistema za ove platforme u 2018 je Windows 10 ili Windows 7. Apple OS X, koji je baziran na Unix jezgri, instaliran je na oko 9%, dok je Linux (Ubuntu ili Chrome OS) instaliran na oko 2% desktop ili laptop računara. U oblasti pametnih telefona Linux platforma Android drži oko 70% tržišta, dok Apple IOS (Unix bazirano jezgro) pogoni oko 28% mobitela. Treći je Nokia OS sa 0.17% a Windows Mobile samo 0.15%. Izjednačeno je na tržištu tablet računara: trenutno 52% drži Apple iOS, a 48% Android. Linux jezgro pogoni praktično sve pametne televizore i uređaje Set top box (za prijem digitalne televizije na starim televizorima). Konzole za igru, predvodi Sony Playstation (Unix bazirano jezgro) sa 72% korisnika, slijedi XBOX (Windows jezgro) sa 12%, WII (interno razvijano jezgro) sa 10% i Android konzole 10%. U sistemima u realnom vremenu i ugrađenim sistemima se javljaju sve vrste platformi (Windows, Unix, Linux i ostale) u primjetnom procentu. Svih 500 najbržih superkompjuteru koristi neku verziju Linuxa. Računa se da Unix i Linux zajedno pogone 68% web servera, dok preostala 32% pripada Windowsu. Na mainframe računarima najčešće se instalira z/OS, slijede neke verzije Linuxa.

2.UPRAVLJANJE PROCESIMA I NITIMA

Izvršavanje korisničkih i sistemskih programa u memoriji koristi sve resurse računara, pa je stoga jedan od najvažnijih dijelova jezgra onaj koji je posvećen upravljanju programima u izvršenju – procesima.

Prvi računarski sistemi, npr. (Mark I, ili u mikro-računarskom svijetu CP/M), nisu dopuštali izvršenje više od jednog programa u jednom trenutku. Upravljanje resursima je bilo jednostavno, jer je taj jedini program imao punu kontrolu nad sistemom i dopušten pristup resursima bez ograničenja.

U višeprocесnim sistemima, istovremeno se dešava više aktivnosti, npr. čitanje sa diska, prikaz teksta na štampaču i izvršavanje različitih programa. Procesor stalno prelazi sa izvršavanja jednog programa na drugi. Iako se u jednom trenutku izvršava samo jedan program, unutar vremenskog intervala se obradi više programa. Time korisnik dobiva osjećaj paralelnog rada. On je u suštini kvazi-paralelan, dok se prava paralelnost postiže u više-procesorskim sistemima, kada različiti procesori dijele istu memoriju i istovremeno izvršavaju programe u njoj.

2.1.Model procesa

Proces je program u izvršenju. On nije samo skup instrukcija, nego uključuje trenutne aktivnosti koje se karakterišu sadržajem memorije, stanjem datoteka, registara procesora, a među njima se posebno ističe programski brojač. Programski brojač se uvećava nakon svake izvršene mašinske instrukcije i, osim kada se izvršava instrukcija skoka, možemo smatrati da se proces izvršava sekvencijalno.

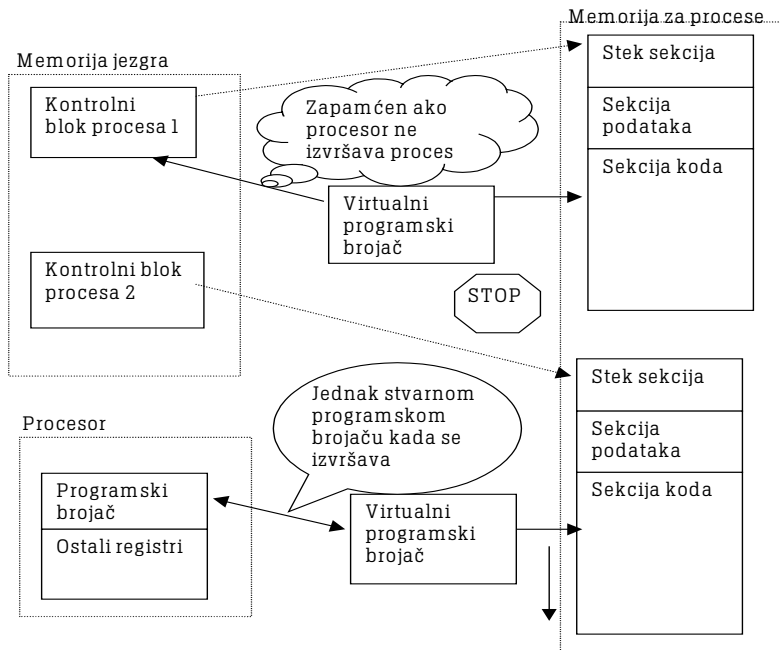
Operativni sistem, zajedno sa programima koji se pod njim izvršavaju možemo gledati kao skup sekvencijalnih procesa. Iako imamo samo jedan procesor, sa stanovišta svakog procesa, njemu izgleda da ima svoj vlastiti virtualni procesor. Stvarni, fizički procesor, prelazi sa izvršavanja jednog procesa na drugi, ali na način da se svaki može nastaviti tačno na mjestu gdje je prethodni put bio zaustavljen.

Model procesa će se predstaviti kroz tri elementa koji ga opisuju: njegovu memorijsku sliku, virtualni programski brojač i strukture u jezgru koje opisuju stanje procesa.

U dijelu memorije dodijeljenom procesu uočavaju se tri fundamentalna memorijska dijela.

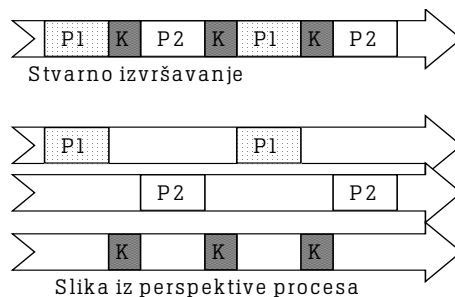
- Programska sekcija, koja se ne mijenja i koja sadrži programski kôd. Programski brojač može da pokazuje samo na memorijske lokacije u ovom dijelu memorije. Ovaj dio memorije je obično nepromjenjiv, a u nekim operativnim sistemima, ako je nad jednim programom pokrenuto više procesa, u memoriji je moguće čuvati jednu kopiju ovog memorijskog dijela.
- Stek sekcija (engl. *stack section*), koja sadrži privremene podatke (parametre za procedure, povratne adrese, lokalne promjenjive), je organizovana obično na kraju predviđenog memorijskog područja za proces. Koristi se specijalni registar zvani *stack pointer* i program pristupa podacima na memorijskim lokacijama koje su u njegovoj neposrednoj blizini. Ovo područje je promjenjivo kako u sadržaju, tako i u njegovoj veličini.

- Sekcija podataka (engl. *data section*), koja sadrži globalne promjenjive. Ovim memorijskim lokacijama se pristupa koristeći instrukcije koje navode njihove apsolutne ili relativne memorijske adrese. Sadržaj ovog područja je promjenjiv, ali veličina se mijenja razmjerno rijetko ili se u nekim slučajevima i ne može mijenjati.



Slika 10 Model procesa

Procesor izvršava programske instrukcije operativnog sistema i korisničke aplikacije.



Slika 11 Vremenski tok izvršenja, virtualna samostalnost

Slika 11 prikazuje kako programski brojač procesora tokom vremena pokazuje na dijelove procesa P1 i P2, kao i dijelove jezgra operativnog sistema K. Iako u stvarnosti postoji samo jedan programski brojač, procesi to vide kao da svaki od njih ima svoj virtualni programski brojač i nezavisno se izvršavaju.

Operativni sistem mora evidentirati podatke o svakom procesu, koristeći **tabelu procesa**, koja ima po jedan element za svaki proces. Taj element sadrži informacije o statusu procesa, vrijednosti programskog brojača, *stack pointer*-a, i ostalih registara procesa, zauzetosti memorije, stanju datoteka koje je otvorio proces, podacima o potrošenom vremenu i svim drugim informacijama koje se moraju snimiti kada proces prelazi iz stanja izvršavanja u stanje čekanja ili spremnosti. Element ove tabele se naziva kontrolni blok procesa (*PCB- process control block*) (Slika 12).

Pokazivači
Stanje procesa
Identifikator procesa
Programski brojač
Registri
Memorijska područja
Lista otvorenih datoteka
...

Slika 12 PCB

Informacije koje se čuvaju u PCB mogu uključiti:

- Trenutni status procesa: *ready*, *running*, *waiting*, i dr.;
- Ime ili numerički identifikator procesa (PID)
- Pokazivač na roditeljski proces (proces koji je kreirao ovaj proces)
- Pokazivač na naredni proces koji je u listi čekanja na procesor ili periferni uređaj
- Prioritet procesa
- Pokazivač na memorijsko područje dodijeljeno procesu
- Sačuvano stanje registara
- Podaci o upravljanju memorijom (tabele stranica, segmenata, ili bazni i limit registri)
- Lista otvorenih datoteka
- Lista ulazno/izlaznih uređaja dodijeljenih procesu
- Oznaka procesora na kome se izvršava
- Potrošeno procesorsko vrijeme

Neke od navedenih informacija se čuvaju u samom kontrolnom bloku procesa, dok su druge u posebnim strukturama, ali kontrolni blok pokazuje na njih. Najveći dio kontrolnog bloka procesa namijenjen je čuvanju konteksta procesa. Kontekst procesa čine podaci koji se čuvaju prilikom oduzimanja procesora. Prelazak na izvršavanje drugi proces traži snimanje stanja procesora za tekući proces i preuzimanje snimljenih podataka o stanju za novi proces. Kada dođe do izmjene konteksta (engl. *context switch*) jezgro snima kontekst starog procesa u njegov PCB. Iz PCB procesa koji je na redu za izvršavanje, pročita se sačuvani kontekst (stanje registara procesa) i upiše u procesor.

Tabela procesa objašnjava iluziju više paralelnih sekvencijalnih procesa na sistemu sa jednim CPU i više U/I uređaja. Kada dođe vrijeme za izvršavanje novog procesa, od trenutno izvršavanog se u tablici kontrolnih blokova procesa sačuva dovoljno informacija da se on može nastaviti tačno na onom mjestu gdje je zaustavljen, poput nastavka putovanja automobila nakon kratkog zaustavljanja na crvenom svjetlu saobraćajnog semafora. Ali, baš kao što zbog različitog nivoa saobraćajne gužve ne možemo proći kroz grad uvijek za jednako vrijeme, tako ni vrijeme izvršavanja pojedinog procesa u sistemima u kojima ih se više istovremeno izvršava ne možemo reprodukovati, jer ono zavisi od broja drugih procesa.

2.2. Pokretanje procesa

Postoje četiri vrste događaja koji uzrokuju pokretanje jednog procesa.

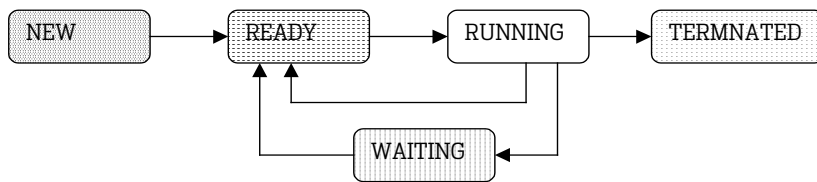
1. Prilikom **inicijalizacije sistema** pokreće se nekoliko procesa. Neki od tih procesa su procesi koji rade u prvom planu. Oni komuniciraju direktno sa korisnicima. Najvažniji među njima je školjka. Ostali procesi su pozadinski. Oni nemaju interakcije sa korisnicima. Npr. pozadinski proces može biti proces za prijem elektronske pošte, koji ne prikazuje ništa na ekranu niti traži podatke sa tastature, nego samo brine o porukama koje dolaze preko mreže. Ovi pozadinski procesi se nazivaju demoni (*daemons*), dok Windows koristi termin servis (*service*)
2. **Kreiranje iz drugog procesa** pokreće procese u kasnijoj fazi rada OS. Aktivni procesi mogu zvati sistemске pozive za stvaranje jednog ili više novih procesa. Tako Unix sistemi imaju sistemski poziv *fork*, a Windows sistemi poziv *CreateProcess*. Proces koji je pokrenuo drugi proces se tada zove roditeljski proces.
3. Proces se mogu pokrenuti i **od strane korisnika**, npr. izvođenjem komande ili klikom miša u grafičkoj školjci. U praksi je ovaj slučaj zapravo specijalni slučaj prethodnog, jer program koji vrši interakciju s korisnikom (školjka komandne linije ili grafičko okruženje) također poziva sistemski poziv.
4. U **batch sistemima** se poslovi pokreću kada sistem završi prethodni posao (ili on ode u stanje čekanja) i primijeti da ima dovoljno resursa za pokretanje narednog posla u čekanju.

Kada se proces pokrene, pripremi se područje u memoriji za njega i odgovarajuće kontrolne strukture u jezgru za njegovo praćenje. Proces dobiva jedinstveni **identifikator procesa** (PID), cijeli broj koji će kasnije lakše omogućiti njegovo praćenje. Inicijalni izgled njegovog područja u memoriji zavisi od dizajna sistema. U sistemima Unix tipa, napravi se kopija sadržaja memorije roditeljskog procesa kreatora u novo-kreiranom procesu, dok u drugim sistemima inicijalni sadržaj memorije kreiranog procesa se ostavlja praznim ili prebacuje sa diska. Proces roditelj može da nastavi rad ili se zaustavi dok novo-kreirani proces radi. Nakon stvaranja procesa može se sačuvati veza između roditeljskog i kreiranog procesa. Time se stvara hijerarhija procesa. Ona je karakteristična za UNIX sisteme. Kod Windows sistema ne postoji hijerarhija procesa, ali od verzije Windows 8 postoji hijerarhija grupa procesa (u Windows 8 grupa procesa se zove *job*).

2.3.Stanja procesa

Prilikom izvršenja procesa on mijenja stanja. Stanje procesa se definiše trenutnom aktivnošću procesa. Svaki proces može biti u jednom od sljedećih stanja (Slika 13).

- Stanje NEW predstavlja novo-kreirani proces. U ovom stanju procesu je dodijeljen kontrolni blok.
- Stanje RUNNING predstavlja proces koji se trenutno izvršava. Programski brojač procesora u tom trenutku pokazuje na instrukcije ovog procesa
- Stanje WAITING znači da proces čeka na pojavu događaja poput U/I operacije ili završetka drugog procesa.
- Stanje READY se odnosi na procese koji su spremni za izvršavanje na procesoru, ali se trenutno ne izvršavaju
- Stanje TERMINATED predstavlja završene procese, čiji PCB može biti obrisan. U specijalnom slučaju (zombi procesi) se PCB još neko vrijeme čuva.



Slika 13 Dijagram stanja procesa

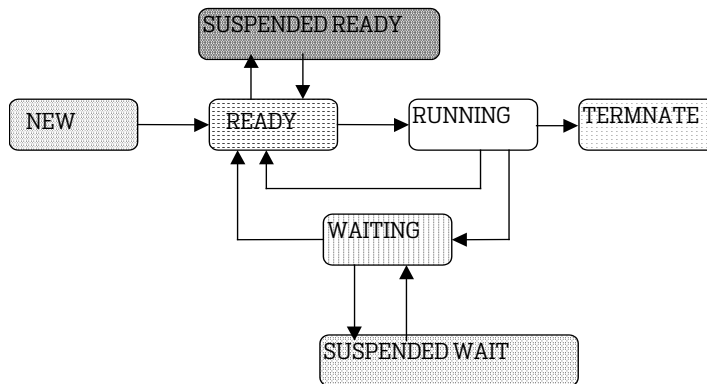
U stanjima READY i RUNNING proces je spreman za rad, ali u stanju READY procesor mu nije na raspolaganju jer izvršava drugi proces. U stanju WAITING proces se ne može izvršavati čak i ako nema drugih procesa, zato što on čeka na neki vanjski događaj. Stanje TERMINATED, kada je proces završio, može biti u dvije verzije: da više nema podataka o tom procesu ili da se podaci i njemu još čuvaju iako je proces završen (zombi procesi).

Na dijagramu vidimo šest prijelaza između stanja procesa.

- NEW→READY. Ovaj prijelaz se dešava kada je novo-kreirani proces spreman da se takmiči za resurse.
- READY→RUNNING. Raspoređivač je donio odluku da se proces izvršava ili nastavlja izvršavati na procesoru
- RUNNING→READY Procesu je isteklo predviđeno vrijeme na procesoru. Raspoređivač bira drugi proces za izvršenje, a proces se vraća u red spremnih procesa.
- RUNNING→WAITING Proces čeka na neki događaj bez koga ne može nastaviti izvršenje. Često proces poziva posebni sistemski poziv kojim se prebacuje u čekanje. Uglavnom se to dešava zbog čekanja na ulazne podatke koji još nisu na raspolaganju.

- WAITING→READY Desio se vanjski događaja na koji je proces čekao. Proces prelazi u red čekanja, a ako u tom trenutku ni jedan drugi proces nije aktivan pokreće se i prijelaz iz READY u RUNNING.
- RUNNING→TERMINATED Proces je pozvao poziv za završetak

Može se dodati i mogućnost privremenog prekida izvršenja procesa (engl. *suspend*), čime se proces prestaje takmičiti za resurse. Nakon stanja suspenzije proces se mora vratiti u stanje prije nje pa se uvode dva nova stanja: SUSPENDED READY i SUSPENDED WAIT (suspendovan u stanju čekanja na resurs). Time se uvodi prošireni dijagram stanja procesa.



Slika 14 Prošireni dijagram stanja procesa

Na proširenom dijagramu stanja se sada nalaze nove tranzicije. Za razliku od prijelaza u osnovnom dijagramu sva četiri nova prijelaza između stanja u proširenom dijagramu inicira korisnik.

- READY→SUSPENDED READY: Ovaj prijelaz se dešava ako korisnik suspenduje proces u toku njegovog izvršavanja ili ako je proces bio spreman za izvršavanje.
- WAIT→SUSPENDED WAIT Ovaj prijelaz se dešava ako korisnik suspenduje proces u dok je proces čekao na neki događaj.
- SUSPENDED READY→READY: Korisnik je tražio reaktiviranje procesa.
- SUSPENDED WAIT→WAIT: Korisnik je tražio reaktiviranje procesa, ali resurs neophodan za njegovo izvršenje nije oslobođen

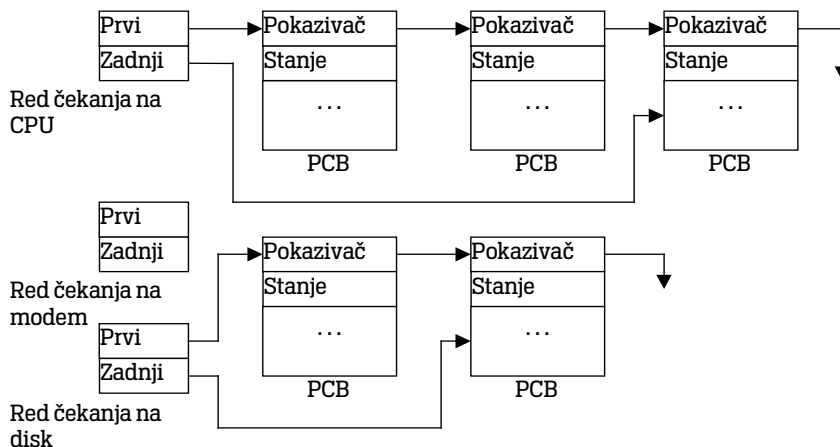
Promjena stanja procesa nije samo promjena vrijednosti polja statusa u kontrolnom bloku procesa, nego i prebacivanje procesa u odgovarajuće redove raspoređivanja.

2.4.Redovi raspoređivanja

Sama tabela procesa nije idealna struktura za određivanje redoslijeda njihovog izvršavanja, nego se dopunjava ulančanim strukturama podataka koji predstavljaju redove čekanja. Redovi čekanja su dinamičke strukture kao što su ulančane liste, stabla ili tablice koje sadrže kontrolne blokove procesa ili pokazivače na njih. Stanja

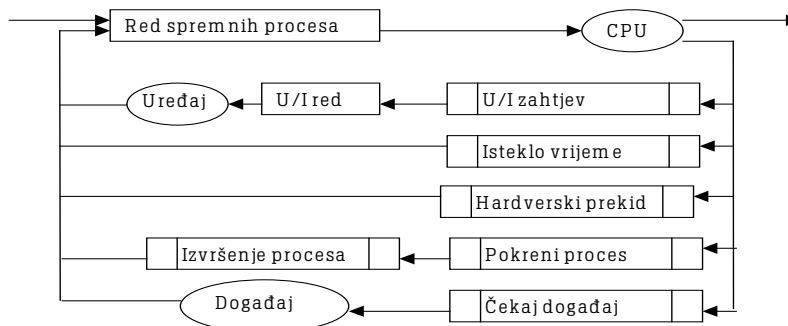
procesa i njihovi prelazi se preslikavaju u prebacivanje kontrolnog bloka procesa u odgovarajući red čekanja ili izbacivanje iz reda čekanja.

Tako na primjer, kada proces pređe u stanje READY, pokazivač na njegov kontrolni blok se stavi u red čekanja spremnih procesa (engl. *ready queue*). Ako je ovaj red uređen kao ulančana lista, u zaglavlju liste čuvaju se pokazivači na prvi i posljednji PCB u listi. Svaki PCB ima pokazivač koji ukazuje na sljedeći proces u redu čekanja spremnih procesa. Kada proces pređe u stanje WAIT, npr. zbog čekanja na ulaz/izlaz, čekanja na završetak procesa, čekanja na slobodnu memoriju itd., njegov kontrolni blok se prebacuje u dati red čekanja.



Slika 15 Redovi čekanja

Procesi koji se nalaze u glavnoj memoriji su spremni i čekaju na izvršenje, smješteni su u listu označenu kao "Red čekanja na CPU". Postoje i drugi redovi u sistemu. Lista procesa koji čekaju na neki uređaj se zove red čekanja na periferni uređaj. Svaki uređaj ima svoj vlastiti red. Slika 15 prikazuje red čekanja na CPU i različite redove čekanja na uređaje.



Slika 16 Događaji koji utiču na promjene redova

Uobičajen način predstavljanja kretanja između različitih redova kada se govori o raspoređivanju procesa je dijagram redova (Slika 16).

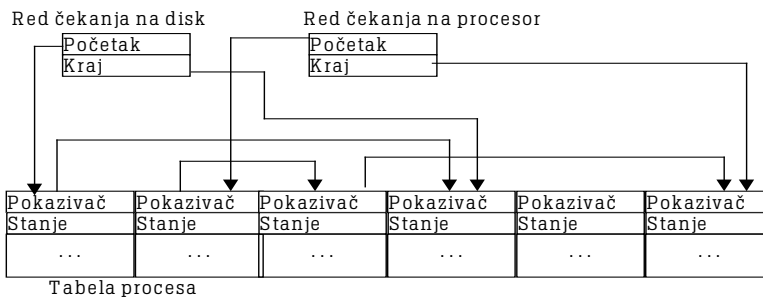
Svaki pravougaonik predstavlja red čekanja. Prisutna su dva tipa redova, red procesa koji su spremni za izvršenje na procesoru (*ready queue*) i redovi za uređaje (*I/O queue*). Kružna polja predstavljaju resurse koji služe redovima a strelice nam prikazuju tok procesa u sistemu.

Kada je proces došao na red za izvršavanje on se izvršava na procesoru dok se ne desi jedan od sljedećih događaja:

1. Vrijeme za proces (kvantum) je isteklo. Proces se vraća u red spremnih procesa
2. Proces je pokrenuo novi proces i čeka na njegovo završavanje. Novi proces se ubacuje u red spremnih procesa, a procesor se prepusta njemu ili trećem procesu.
3. Desio se hardverski prekid i prelazi se na izvršavanje u jezgru. Po završetku obrade kontrola se može vratiti procesu koji je prekinut ili prepustiti drugom procesu
4. Proces čeka na neki događaj, na primjer da mu drugi proces pošalje poruku. Proces se blokira dok se taj događaj ne desi
5. Proces je postavio zahtjev prema perifernom uređaju, nakon čega se prebacuje u red čekanja na ulazno-izlazni uređaj, tj. postaje blokiran.

Pošto je kontrolni blok procesa relativno velik (između nekoliko desetina i nekoliko hiljada bajta), s obzirom na to koliko često se ova stanja mijenjaju, ova prebacivanja se ne vrše kopiranjem čitavih kontrolnih blokova.

Umjesto toga se procesi čuvaju u tabeli procesa koristeći identifikator procesa kao indeks u ovoj tabeli, a neka od polja ove tabele su pokazivači koji pokazuju na druge kontrolne blokove praveći logičku strukturu. Prebacivanje procesa između redova čekanja svodi se na promjenu samo ovih pokazivača.



Slika 17 Fizički redovi čekanja

2.5.Izmjena konteksta i dispečer

Kada proces treba da pređe iz drugog stanja u stanje RUNNING ili iz stanja RUNNING u drugo stanje, dešava se izmjena konteksta. Izmjenu konteksta obavlja **dispečer**, koji se izvršava u jezgru i prepusta procesor procesu koga je za taj trenutak odabrao raspoređivač. Funkcije dispečera su izmjena konteksta, prelazak u

korisnički režim rada i skok na odgovarajuću lokaciju u korisničkom programu. Tokom izmjene konteksta se sačuva stanje procesora ili niti koja se prethodno izvršavala i učitava ranije sačuvano stanje za izvršavanje nove niti. Izmjena konteksta podrazumijeva, najjednostavnije rečeno, pomjeranje pokazivača na trenutni skup registara. Moderni procesori imaju jedinicu za upravljanje memorijom MMU koja sprječava da neki proces mijenja podatke u adresnom prostoru drugog procesa. Prebacivanje sa jednog procesa na drugi (promjena konteksta) znači reprogramiranje MMU da pokazuje na drugi adresni prostor i snimanje podataka o procesu, te vraćanje podataka o drugom procesu. Ovo reprogramiranje može da potraje, jer je potrebno izvjesno vrijeme da procesorski keševi budu napunjeni novim sadržajima memorijskih lokacija. Kašnjenje kod izmjene procesa predstavlja vrijeme da se zaustavi jedan proces i pokrene drugi. To vrijeme kod nekih algoritama raspoređivanja može biti značajan procenat ukupnog rada procesora. Stoga, dispečer, koji predstavlja čisti interni rad, treba da bude veoma brz. Neki procesori nemaju posebne instrukcije za izmjenu konteksta, pa se mora svaki registar procesora čuvati i učitavati odvojenim instrukcijama.

U primjeru jezgra u zadnjem poglavlju dispečer je realizovan koristeći *setjmp* i *longjmp* potprograme. U njima *setjmp* čuva neke registre na steku i zapamti mjesto gdje su sačuvani. S druge strane, *longjmp* kao parametar preuzme mjesto gdje su ti registri sačuvani i preuzme njihovu vrijednost.

Izmjena konteksta na Intel i386 i jačim procesorima se obavlja znatno brže zahvaljujući *task* registru. Procesori na PC računarima mogu obaviti izmjenu konteksta jednom instrukcijom skoka ili pozivom potprograma, ako je u opisu segmenta (o tome u 7. lekciji) segment predstavljen kao *task* segment. Tada će se snimiti svi registri na lokaciju koju pokazuje skriveni registar TS, u njega upisati nova adresa koja predstavlja adresu skoka i napuniti svi registri sa te adrese. Registri su raspoređeni prema strukturi prikazanoj na slici.

TSS	ESP	SS0	ESP	SS1	ESP	SS2	CR3	EIP	FLA	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI	ES	CS	SS	DS	FS	GS	LDT	IOM
0	4	8	12	16	20	24	28	32	36	40	44	48	52	56	60	64	68	72	76	80	84	88	92	96	AP

Slika 18 Struktura task segmenta na i386

2.6.Završavanje procesa

Proces završava rad iz jednog od sljedećih razloga:

1. **Dobrovoljni normalni izlaz:** Proces se završio tako što je sam pozvao sistemski poziv za završetak. Sistemskom pozivu za završetak procesa prosljeđuje se parametar koji označava da je proces ispravno završen.
2. **Dobrovoljni izlaz sa greškom:** Istom sistemskom pozivu za završetak procesa prosljeđuje se parametar koji označava neku grešku. Primjer je datoteka koja ne postoji.
3. **Fatalna greška:** Operativni sistem je uočio da program pokušava da radi nelegalnu aktivnost, npr. pokušava da pristupi nedopuštenom području memorije. Tada sistem prekida dati proces.
4. **Ubijen proces:** Od strane drugog procesa pošalje se signal procesu koji kaže da proces treba prekinuti.

5. **Pad operativnog sistema:** Nažalost, dešavaju se nepredviđeni događaji koji ponekad izazivaju rušenje cijelog računarskog sistema, pa time i navedenog procesa.

U svim slučajevima osim petog, memorija dodijeljena procesu se oslobađa. Kontrolne strukture u jezgu se također mogu osloboditi, ali se ponekad čuvaju još neko vrijeme.

2.7.Windows API funkcije za promjenu stanja procesa

Sistemske pozivi u Windows operativnim sistemima, vezani za upravljanje procesima, odlikuju se velikim brojem parametara radi preciznog podešavanja. U Windows API tipovi podataka parametara se ugrađuju u njihovo ime. Tako svi parametri koji su tipa duple riječi imaju ime koje počinje sa dw, parametri logičkog tipa počinju sa b, pointeri sa p ili lp, a rukovatelji sa h. Neke važnije funkcije date su u sljedećoj tabeli.

Funkcija i parametri	Značenje
CreateProcess(lpApplicationName, lpCommandLine, lpProcessAttributes, lpThreadAttributes, bInheritHandles, dwCreationFlags, lpEnvironment, lpCurrentDirectory, lpStartupInfo, lpProcessInformation)	Kreira novi proces sa svim parametrima koji ga opisuju.
ExitProcess(uExitCode)	Završava trenutni proces
GetCurrentProcessId ()	Dobavlja identifikator trenutnog procesa
OpenProcess(dwDesiredAccess, bInheritHandle, dwProcessId)	Otvora proces i vraća njegovu internu oznaku
GetModuleFileNameEx (hProcess, hModule, lpFileName, nSize)	Saznaje ime procesa iz interne oznake
TerminateProcess(hProcess, uExitCode)	Prekida izvršenje drugog procesa
EnumProcesses(pProcessIds, wArraySize, pBytesReturned)	Očitava identifikatore svih procesa

Kreiranje procesa obavlja se funkcijom **CreateProcess**. Ova funkcija ima deset složenih parametara, ali većini njih se može dodijeliti podrazumijevana prazna vrijednost. Najvažniji parametri su *lpApplicationName*, i *lpCommandLine* koji pokazuju na niz znakova sa imenom izvršnog programa procesa i njegovim parametrima iz komandne linije. Parametar *lpEnvironment* pokazuje na niz znakova koji sadrže konfiguracijske vrijednosti sistema oblika IME=VRIJEDNOST. Parametar *lpStartupInfo* je pokazivač na strukturu od 18 polja koji govore o veličini i položaju prozora aplikacije, boji tekstualnih aplikacija, preusmjerenju ulaza i drugo. Polje *cb* ove strukture mora biti inicijalizirano na njenu veličinu. Informacije o procesu se dobivaju preko posljednjeg parametra *lpProcessInformation* koji je pokazivač na strukturu od 4 polja u kojoj se nalaze identifikatori i rukovatelji procesa i njegove glavne niti.

Dobrovoljno završavanje procesa obavlja se funkcijom **ExitProcess**. Ova funkcija ima samo jedan parametar *uExitCode*, koji predstavlja izlazni kôd procesa, indikator je li proces ispravno završen.

Nasilno završavanje drugog procesa obavlja se funkcijom **TerminateProcess**. Parametri su *hProcess* (interna oznaka procesa koji se prekida) i *uExitCode* (izlazni kôd koji mu se proslijeđuje).

Kroz tablicu procesa se prolazi funkcijom **EnumProcesses**. Parametri su pokazivač na niz identifikatora procesa, veličina tog niza i pokazivač na vraćeni broj bajta. Iz identifikatora procesa može se dobiti njegov rukovatelj koji je parametar drugim funkcijama. To radi funkcija **OpenProcesses**. Parametri funkcije su konstante koje govore o pravima koja se traže u pristupu procesu, da li pristupiti rukovateljima na objekte i identifikator procesa.

Sljedeći primjer kreira novi proces, notepad.exe korištenjem funkcije *CreateProcess*. Nakon toga funkcijom *EnumProcesses* se preuzme lista procesa u memoriji. Krećući se kroz tu listu čitaju se informacije o identifikatoru i imenu svakog procesa i prikazuju na ekranu. Nakon pauze, kreirani proces se prekida, a zatim se izlazi iz pozivaoca.

```
#include <windows.h>
#include <psapi.h>
#include <stdio.h>
#include <stdlib.h>
int main() {
    char datoteka[300];
    DWORD pids[500];
    DWORD ukupno,duzimena;
    unsigned int i;
    int t;
    STARTUPINFO siStartupInfo;
    PROCESS_INFORMATION piProcessInfo;
    memset(&siStartupInfo, 0, sizeof(siStartupInfo));
    siStartupInfo.cb = sizeof(siStartupInfo);
    CreateProcess("c:\\windows\\notepad.exe", "",
        0, 0, FALSE,CREATE_DEFAULT_ERROR_MODE, 0, 0,
        &siStartupInfo, &piProcessInfo) ;
    EnumProcesses(pids,500,&ukupno);
    for(i=0;i<ukupno/sizeof(DWORD);i++) {
        HANDLE h = OpenProcess(PROCESS_QUERY_INFORMATION |
            PROCESS_VM_READ, 0,pids[i]);
        printf("PID = %d ",pids[i]);
        if (h !=NULL) {
            t= GetModuleFileNameEx( h,NULL,datoteka, 300);
            if (t>0)
                printf(" %s",datoteka);
            CloseHandle(h);
        }
        printf("\n");
    }
    getchar();
    TerminateProcess(piProcessInfo.hProcess,1);
    ExitProcess(0);
    return 0;
}
```

2.8.Posix funkcije za upravljanje procesima

Posix sistemski pozivi za upravljanje procesima su daleko jednostavniji od ekvivalentnih poziva Windows API. Funkcije imaju malo parametara i kratko ime. Najvažnije funkcije za rad s pojedinačnim procesima su date u tabeli.

Funkcija	Značenje
<code>fork()</code>	Kreiraj dijete proces
<code>execl(pcPath, pcArg,...)</code>	Izvrši program sa parametrima iz funkcije
<code>execvp(pcFile, pcArg,...)</code>	Izvrši program u PATH s parametrima iz funkcije
<code>execv(pcPath, pcArgv[])</code>	Izvrši program sa parametrima iz niza
<code>execvp(pcFile, pcArgv[])</code>	Izvrši program u PATH sa parametrima iz niza
<code>wait(idPid)</code>	Čekaj završetak procesa
<code>waitpid(idPid, piStatus, iOptions)</code>	Čekaj završetak procesa, uz informacije o statusu
<code>exit(iStatus)</code>	Završi trenutni proces
<code>int kill(idPid, iSig)</code>	Šalji signal procesu
<code>unsigned int sleep(uiSeconds)</code>	Spavaj navedeni broj sekundi
<code>int pause()</code>	Čekaj signal
<code>unsigned int alarm(uiSeconds)</code>	Postavi alarm za slanje signala
<code>getpid()</code>	Dobavi identifikaciju procesa
<code>getppid()</code>	Identifikacija roditeljskog procesa

Sistemski poziv **fork()** kreira novi proces. Kada proces pozove ovaj poziv, pokrenuti proces postaje "dijete" procesa koji ga je pokrenuo. Dijete proces dobiva kopije područja instrukcija i područja podataka od roditeljskog procesa. Pošto se područje instrukcija normalno ne mijenja, ne radi se fizičko kopiranje, nego se dio memorije dijeli između oba procesa i tako štedi prostor i vrijeme. Nakon `fork()` poziva, ako se ne želi da novi proces bude samo kopija roditelja, može se u memoriju iz datoteke učitati drugi proces nekim od `exec` poziva.

Poziv **wait()** prebacuje pozivajući proces u stanje WAIT dok dijete proces, naveden kao parametar ovog poziva ne završi rad. Rezultat ovog poziva će biti izlazni kôd dijete-procesa, nakon čega se dijete-proces može obrisati iz tablice procesa. Ako dijete-proces završi sa radom, a roditelj proces ga ne čeka sa `wait()`, tada se dijete proces ne briše odmah. Dijete-proces neće biti obrisani jer bi se u tom slučaju izgubio izlazni kôd. Tada dijete-proces postaje zombi-proces. Proces više nije u memoriju, ali se podaci o njemu još čuvaju. Ukoliko roditelj-proces završi bez `wait` poziva, tada će `init` proces (PID=1) preuzeti hijerarhiju nad dijete-procesom i obrisati zombi procese koje je preuzeo.

Sljedeći primjer za Unix sisteme pravi kopiju procesa roditelja pozivom `fork()`. Nakon toga roditelj i dijete se istovremeno izvršavaju ispisujući svoje identifikacijske oznake. Roditelj će sačekati da se kreirani proces završi, a dijete proces će se završiti pozivom `exit`.


```

#include <unistd.h> /* Symbolicke konstante */
#include <sys/types.h> /* Primitivni sistemski tipovi */
#include <errno.h> /* Greske */
#include <stdio.h> /* Input/Output */
#include <sys/wait.h> /* Naredbe cekanja */
#include <stdlib.h> /* Opste rutine */
int main() {
    pid_t dijete_pid; /* Ovdje se smjesta identifikator pid */
    int retval; /* dijete process: povratni kod */
    int status; /* roditelj proces: Izlazni status djeteta */
    dijete_pid = fork(); /* Kreiranje procesa */
    if (dijete_pid >= 0) { /* fork uspio */
        if (dijete_pid == 0) { /* fork() vraca 0 za dijete proces */
            printf("dijete: Ja sam dijete proces!\n");
            printf("dijete: Ovo je moj PID: %d\n", getpid());
            printf("dijete: PID mog roditelja je: %d\n", getppid());
            printf("dijete: Kopija dijete_pid je: %d\n", dijete_pid);
            printf("dijete: Pauza 1 sekunda...\n");
            sleep(1); /* 1 sekunda */
            printf("dijete: Unesi izlaznu vrijednost (0 to 255): ");
            scanf(" %d", &retval);
            printf("dijete: Gotovo!\n");
            exit(retval); /* dijete izlazi s povratnim kodom */
        }
        else { /* fork() vrati PID dijete procesa roditelju */
            printf("roditelj: Ja sam roditelj proces!\n");
            printf("roditelj: Ovo je moj PID: %d\n", getpid());
            printf("roditelj: Dijete_pid je %d\n", dijete_pid);
            printf("roditelj: Cekam da dijete izađe.\n");
            wait(&status); /* Ceka da dijete završi i cuvanje statusa */
            printf("roditelj: dijete izlazni kod: %d\n", (status)&0xff00>>8);
            printf("roditelj: Gotovo!\n");
            exit(0); /* roditelj izlazi */
        }
    }
    else { /* fork vrati -1 u slucaj greske */
        perror("fork"); /* Poruka o gresci */
        exit(0);
    }
}

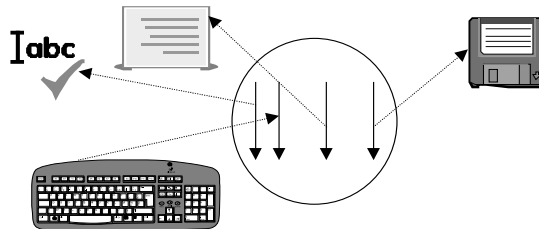
```

2.9. Programske niti

Već je rečeno da u modelu procesa postoji virtualni programski brojač koji se premapira u stvarni programski brojač kada se procesor dodijeli procesu, a čuva zapamćen u memoriji kada proces nije u stanju RUNNING. Njega možemo zamisliti kako se kreće od prve instrukcije programa i redom pokazuje na svaku instrukciju koja se izvršava. Vrlo sličan mehanizam bi se mogao koristiti da u procesu ima i više virtualnih programskih brojača, tako da u jednom procesu postoji više programskih tokova koji se izvršavaju sličnim mehanizmom izmjene stanja, redova čekanja i konteksta kao i procesi. Takvi programski tokovi se zovu niti (*thread*). Programska nit je jedan sekvencijalni tok kontrole unutar programa koji se može raspoređivati. Programske niti su slične *child* procesima, osim što sve programske niti vezane za jedan proces dijele isti memorijski prostor.

Prednost programskih niti je što je potrebno manje vremena za prebacivanje između programskih niti, nego između procesa, pošto nema potrebe za promjenom prostora memorije. Programske niti lakše komuniciraju međusobno jer dijele isti adresni prostor. Nit ima svoj virtualni programski brojač, skup registara i prostor u *stack* sekciji. Svaka nit dijeli sa drugim nitima pridruženim istom procesu kodnu sekciju, sekciju podataka, otvorene datoteke, signale i druge resurse operativnog sistema.

Kao i proces sa samo jednom niti, i programska nit može biti u jednom od nekoliko stanja: NEW, RUNNING, WAIT, READY ili TERMINATED. Prelasci između stanja niti su isti kao prelasci između stanja procesa.

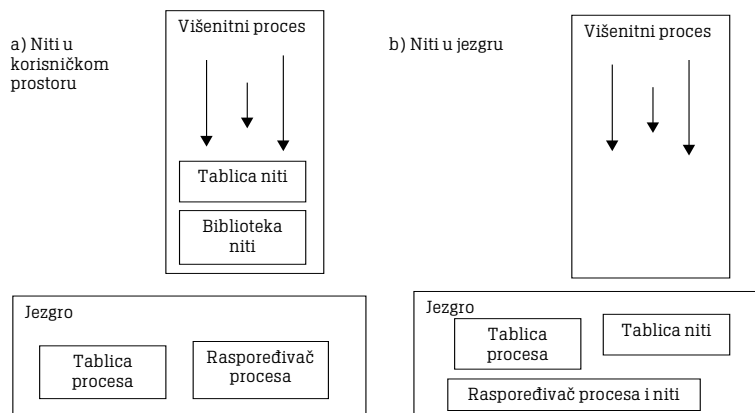


Slika 19 Višenitna obrada teksta

Programske niti pojednostavljaju neke programe. Tako na primjer, program za obradu teksta koji istovremeno radi sljedeće aktivnosti: prebacivanje otkucanog teksta sa tastature u memoriju, formatiranje izgleda stranice na ekranu, provjera pravopisa i povremeno snimanje na disk daleko je lakše napisati kao četiri odvojena potprograma koji se kvazi-paralelno izvršavaju, nego da se nakon svakog pritisnutog tastera provjerava da li je došlo vrijeme za automatsko snimanje podataka na disk, da li je formirana riječ za provjeru pravopisa ili da li je potrebno formatirati tekst na ekranu. Zapravo, većinu ovih aktivnosti moguće je realizovati dok je nit za unos sa tastature u stanju čekanja, jer korisnik ništa ne kuca, razmišljajući šta treba unijeti. Ovo se ne može uraditi sa četiri odvojena procesa, pošto sve četiri niti operišu na istom dokumentu. Međutim, kako postoje četiri niti umjesto četiri procesa, one dijele istu memoriju i zato sve imaju pristup dokumentu koji se uređuje.

Niti su posebno korisne u višeprocorskim računarima. Neka se neki program za matematički proračun, npr. program za račun numeričke vrijednosti složenog integrala (površine ispod krive) podijeli u više niti koje računaju različit dio ove površine. Pokretanjem svake od niti na različitom fizičkom procesoru program se ubrzava jer se dijelovi računa obavljaju istovremeno.

2.10.Načini implementacije niti



Slika 20: Načini implementacije niti

Dok pri raspoređivanju procesa nema dileme ko će obaviti to raspoređivanje (jezgro, naravno), kod niti koje se nalaze unutar procesa na raspolaganju postoje tri varijante. Kod raspoređivanja niti u **korisničkom prostoru** raspoređivanje obavlja sam proces (Slika 20a). Ovaj princip se primjenjuje ako jezgro operativnog sistema ne poznaje raspoređivanje niti ili ako se želi ubrzati raspoređivanje niti. Korisnički program se prevede i poveže zajedno s programskom bibliotekom koja obavlja raspoređivanje niti. Unutar memorijskog područja svakog procesa postoji tablica niti koja je slična tablici procesa. Raspoređivanjem niti u korisničkom području upravlja izvršni sistem procesa. On održava status niti i kada uoči da se do sada izvršavana nit prebacila u stanje blokiranosti, prebacuje izvršenje na novu nit unutar istog procesa. Prebacivanje na drugu nit se obavlja punjenjem registara procesora, što je brže nego sistemski poziv jezgra. Mana ovakvih niti je što, ako nit pozove blokirajući sistemski poziv (npr. čekanje na tastaturu), jezgro ne vidi nit, nego samo procese, pa blokira cijeli proces. To je šteta, jer za to vrijeme se ne mogu izvršavati druge niti iz tog procesa, iako je period dok se čeka blokirajući sistemski poziv pogodan za to.

Niti može raspoređivati i **jezgro** (Slika 20b). U ovom slučaju, kad treba kreirati ili uništi postojeću nit, izvrši se sistemski poziv za kreiranje ili uništavanje niti. Tabela o svakoj niti se čuva u području jezgra. Informacije koje ona čuva su iste kao i kod niti u korisničkom prostoru, samo je mjesto gdje se one nalaze drugo. Niti u jezgri nemaju problem sa blokirajućim sistemskim pozivima. Blokirajući sistemski poziv ne blokira druge niti. Glavna mana niti u jezgri je njihovo sporije kreiranje i raspoređivanje, jer se obavlja poziv u jezgro. Zbog relativno veće cijene kreiranja i uništavanja niti u jezgri, neki sistemi recikliraju svoje programske niti. Kad su niti uništene označe se kao neaktivne, ali se ne brišu. Kasnije, umjesto kreiranja nove niti, samo se reaktivira ranije kreirana.

Hibridna implementacija uključuje realizaciju niti u oba područja uz odgovarajuće preslikavanje skupova niti u korisničkom prostoru u odgovarajuće niti u jezgri. U hibridnoj implementaciji koriste se programske niti u jezgri i preusmjeravaju korisničke niti na neke ili sve niti u jezgri. Hibridna implementacija pruža tri

načina kako se niti u korisničkom prostoru pridružuju nitima u jezgru. U načinu **više u jednu**, sve niti iz korisničkog prostora se povezuju u jednu nit jezgru. Time se sistem svodi na upravljanje nitima u korisničkom prostoru. Ako neka nit pozove blokirajući sistemski poziv, blokiraće se cijeli proces, tj. sve niti. Način **jedna u jednu** mapira svaku nit korisničkog prostora u jednu nit u jezgru. Na ovaj način osigurano je mnogo bolje istovremeno izvršavanje niti, a više niti iz jezgra se mogu izvršavati paralelno na višeprocorskoj arhitekturi. Način **više u više** mapira više niti u korisničkom prostoru u manji ili isti broj niti u jezgru i ovaj model pruža najviše mogućnosti.

2.11.Windows funkcije za rad s nitima

Windows od verzije NT 3.1 podržava niti u jezgru, koje se raspoređuju sa istiskivanjem, ravnopravno procesima. Neke važnije funkcije za rad s nitima u jezgru date su u sljedećoj tabeli.

Funkcija	Značenje
CreateThread(lpThreadAttributes, dwStackSize, lpStartAddress, lpParameter, dwCreationFlags, lpThreadId)	Kreira programsku nit i vraća rukovatelj na nju
ExitThread(dwExitCode)	Završava nit koja je pozvala ovu funkciju
GetCurrentThread()	Vraća rukovatelj na nit pozivaoc funkcije
ResumeThread(hThread)	Nastavlja suspendovanu nit
Sleep(dwMilliseconds)	Suspenduje nit na određeno vrijeme
SuspendThread(hThread)	Suspenduje navedenu nit
SwitchToThread()	Dobrovoljno predaje izvršenje drugoj niti
TerminateThread(hThread, dwExitCode)	Prekida navedenu nit

Prilikom kreiranja niti funkcijom **CreateThread**, prvi parametar je NULL ako nije potrebno prosljeđivati rukovatelje na objekte u kreirane procese. Drugi parametar određuje veličinu područja u *stack* sekciji za navedenu nit. Kao treći parametar funkcije CreateThread prosljeđuje se funkcija, koja će se izvršavati kada započne izvršavanje niti. Ova funkcija mora biti pisana da ima jedan parametar i to pokazivač na tip *void*. Iako to izgleda ozbiljno ograničenje, složeniji parametri se mogu proslijediti prema niti pravljenjem strukture i zatim prosljeđivanjem pokazivača na nju. Parametar funkcije koja se izvršava na nit se prosljeđuje kao četvrti parametar funkcije CreateThread. Peti parametar su kreacijske zastavice. Ako je on 0 nit se starta odmah, a ako je CREATE_SUSPENDED nit je inicijalno suspendovana. Posljednji parametar predstavlja pointer na mjesto gdje će se smjestiti identifikator niti. Ova funkcija vraća rukovatelj niti koga koriste druge funkcije za rad s nitima

Sljedeći primjer je dvonitna aplikacija, kviz iz historije. U glavnom programu se pokreću dvije niti. Prva od njih izvršava funkciju *rjesavaj* koja u petlji postavlja 6 pitanja i nakon njih završava nit, uz sabiranje tačnih odgovore. Druga nit izvršava funkciju *prekini* koja je uspavana 10 sekundi i poziva *TerminateThread()* funkciju da prekine prvu nit kada je vrijeme za test isteklo. Glavni program kreira obje niti i čeka da se završi nit sa pitanjima, pa ispisuje rezultat.

```

#include <stdio.h>
#include <windows.h>
char * pitanja[]={"sarajevski atentat","Kolumbo u Americi",
"covjek na Mjesecu", "prvi let avionom","francuska revolucija",
"napad na Perl Harbur"};
int odgovori[]={1914,1492,1969,1903,1792,1941};
int tacno;
HANDLE hrjesavac,hprekidac;
DWORD WINAPI rjesavaj( LPVOID lpParam ) {
    int i,godina;
    for (i=0;i<6;i++) {
        printf("U kojoj godini je %s?",pitanja[i]);
        scanf("%d",&godina);
        if (godina==odgovori[i])
            tacno++;
    }
    ExitThread(0);
}
DWORD WINAPI prekini( LPVOID lpParam ) {
    Sleep(10000);
    TerminateThread(hrjesavac,1);
}
void main() {
    tacno=0;
    hrjesavac=CreateThread(NULL,65536,rjesavaj,NULL,0,NULL);
    hprekidac=CreateThread(NULL,65536,prekini,NULL,0,NULL);
    WaitForSingleObject(hrjesavac, INFINITE);
    printf("\nImali ste %d tacnih odgovora.\n",tacno);
}

```

Niti u korisničkom prostoru u Windows API se zovu vlakna (*fiber*), i uvedene su sa Windows XP. Iako su inferiorne nitima u jezgru (*thread*), uvedene su radi lakšeg prenošenja aplikacija s drugih operativnih sistema ili pravljenje vlastitog algoritma raspoređivanja. Ovo su važnije funkcije za rad s vlaknima.

Funkcija	Značenje
ConvertThreadToFiber(lpParameter)	Kreira vlakno iz niti
CreateFiber(dwStackSize, lpStartAddress, lpParameter)	Kreira objekt vlakna
SwitchToFiber(lpFiber)	Prelazi na potprogram vlakna
DeleteFiber(lpFiber)	Uklanja postojeće vlakno

Svako vlakno ima svoju izvršnu funkciju i dobrovoljno poziva takvu funkciju druge niti pozivom funkcije *SwitchToFiber*. Početno vlakno se mora kreirati iz glavne niti (u jezgru) programa funkcijom *ConvertThreadToFiber*.

Ovaj primjer ilustruje kako se koriste vlakna. Dva potprograma Ping i Pong naizmjenično pozivaju jedan drugog. Ovdje je razlika u odnosu na prosti poziv potprograma u tome što se, kada je potprogram napušten sa *SwitchToFiber*, zapamti pozicija programskog brojača zajedno sa lokalnim varijablama. Novo pokretanje tog potprograma sa *SwitchToFiber* nastavlja njegovo izvršenje od mjesta napuštanja.

```
#include<windows.h>
#include <stdio.h>
HANDLE hping,hpong;

VOID CALLBACK Ping( LPVOID
lpParameter){
    int i;
    for (i=1;i<=5;i++) {
        printf("%d Ping\n",i);
        SwitchToFiber(hpong);
    }
}

VOID CALLBACK Pong( LPVOID
lpParameter){
    int i;
    for (i=3;i<=8;i++) {
        printf("%d Pong\n",i);
        SwitchToFiber(hping);
    }
}

void main() {
    hping=CreateFiber(65535,Ping,NULL);
    hpong=CreateFiber(65535,Pong,NULL);
    ConvertThreadToFiber(NULL);
    SwitchToFiber(hping);
}
```

2.12.POSIX niti

Jezgra sistema Unix porodice su različite unutrašnje arhitekture, pa se niti implementiraju kao dodatna biblioteka. Ova biblioteka kasnije može biti implementirana da koristi postojeće niti u jezgri, ili ako jezgro ne podržava niti, biti implementirana u korisničkom prostoru. Najvažnije funkcije za rad sa nitima date su u sljedećoj tabeli.

Funkcija	Značenje
<code>pthread_create (pThread, pAttr, pStart_routine,pArg)</code>	Kreira programsku nit.
<code>pthread_exit (iStatus)</code>	Prekida nit koja je pozvala ovu funkciju i prosljeđuje status nitima povezanim na nju
<code>pthread_cancel (pThread)</code>	Prekida navedenu nit
<code>pthread_attr_init (pAttr)</code>	Kreira objekt s atributima niti
<code>pthread_attr_destroy (pAttr)</code>	Uništava objekt s atributima niti
<code>pthread_join (iThreadid,ppStatus)</code>	Čeka da se navedena nit završi, oslobodi njene resurse i nastavi izvršenje trenutne niti
<code>pthread_detach (iThreadid)</code>	Oslobađa resurse završene niti ili najavljuje oslobađanje kada bude završena

Kreiranje niti se obavlja funkcijom *pthread_create*. Njen prvi parametar je pointer na varijablu koja će sadržati identifikator niti nakon njenog kreiranja. Drugi parametar je pointer na strukturu atributa niti kojim se može uticati na ponašanje niti, u većini slučajeva može se staviti NULL. Treći parametar pokazuje na funkciju u kojoj se nit izvršava, a četvrti je argument koji se prosljeđuje toj funkciji. Pozivalac nakon kreiranja niti može pozvati *pthread_join* funkciju da sačeka završetak date niti.

Sljedeći primjer kreira dvije niti koje koriste istu funkciju u kojoj se izvršavaju. Nitima je prosljeđen parametar koji se ispisuje u petlji.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void *poruka( void *ptr ) {
    int i;
    char *tekst;
    tekst = (char *) ptr;
    for (i=0;i<20;i++)
        printf("%s \n", tekst);
}
main() {
    pthread_t thread1, thread2;
    char *p1 = "Thread 1";
    char *p2 = "Thread 2";
    pthread_create( &thread1, NULL, poruka, (void*) p1);
    pthread_create( &thread2, NULL, poruka, (void*) p2);
    pthread_join( thread1, NULL);
    pthread_join( thread2, NULL);
    exit(0);
}

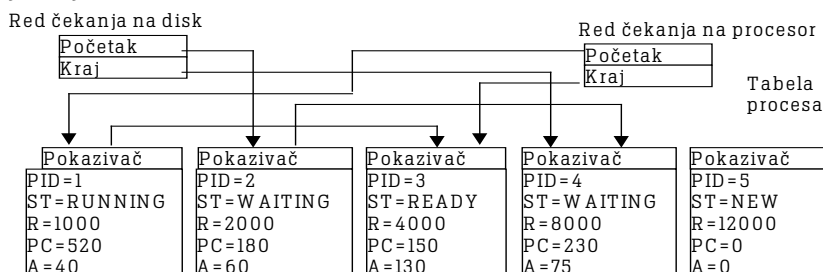
```

2.13.Zadaci

Zadatak 1

Neka se u memoriji nalazilo četiri procesa. Centralni procesor ima tri registra: programski brojač PC, akumulator A i relokacijski registar R. Relokacijski registar se sabira sa programskim brojačem u procesoru da se dobije fizička adresa i pokazuje na početnu adresu procesa. Proces 1 zauzima područje od adrese 1000, proces 2 zauzima područje od adrese 2000, proces 3 zauzima područje od adrese 4000, proces 4 zauzima područje od adrese 8000. Proces 2 je pozvao poziv za pristup disku kada mu je programski brojač pokazivao vrijednost 180, a akumulator imao vrijednost 60. Proces 4 je pozvao poziv za pristup disku kada mu je programski brojač pokazivao vrijednost 230, a akumulator imao vrijednost 75. Proces 3 je u trenutku kada mu je programski brojač bio 150, a akumulator 130 izgubio vrijeme izvršavanja i prepustio izvršenje procesu 1. Nacrtati tabelu procesa sa redovima čekanja u trenutku kada je proces 1 izvršavajući instrukciju na programskom brojaču 520 sa akumulatorom jednakim 40 kreirao novi proces 5. Proces 5 će zauzeti područje od adrese 12000, ali još nije stavljen u redove čekanja, a njegov programski brojač imaće početnu vrijednost 0.

Rješenje



Zadatak 2

Koliko procesa će nastati sljedećim Unix programskim kodom?

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid, pid1;
    fflush(stdout);
    pid = fork();
    fflush(stdout);
    pid1 = fork();
    return 0;
}
```

Rješenje: Početni proces će doći do linije `pid=fork()` i kreirati dijete proces. Sada dva procesa (roditelj i dijete) dolaze odvojeno do linije `pid1=fork()` i svaki od njih kreira još po jedan dijete proces. Stoga, ukupno će biti četiri procesa.

Zadatak 3: Napišite u jeziku C programe za Windows i Unix koji kreiraju jednu nit koja ispisuje poruku “Višenitni program” 1000 puta.

Rješenje:

```
#include <stdio.h>
#include <windows.h>
DWORD WINAPI helloFunc(LPVOID arg ) {
    int i;
    for (i=1;i<1000;i++)
        printf("Višenitni program\n");
    return 0;
}
main() {
    HANDLE hThread = CreateThread(NULL, 0, helloFunc, NULL, 0, NULL );
    sleep(1000);
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void * helloFunc ( void *ptr ) {
    int i;
    for (i=1;i<1000;i++)
        printf("Višenitni program\n");
    return 0;
}
main(){
    pthread_t thread;
    int iret1;
    iret1 = pthread_create( &thread1, NULL, helloFunc,NULL);
    pthread_join( thread1, NULL);
    exit(0);
}
```


3.RASPOREĐIVANJE PROCESORA

Kada se više procesa nalazi u memoriji, u datom trenutku vremena se može izvršavati samo onoliko njih koliko ima fizičkih procesora. Procesi koji su u memoriji, ali trenutno im nije dodijeljen procesor tada se nalaze u redu čekanja. Odluka o tome kada će procesima ili nitima biti dodijeljen procesor se zove **raspoređivanje procesora**.

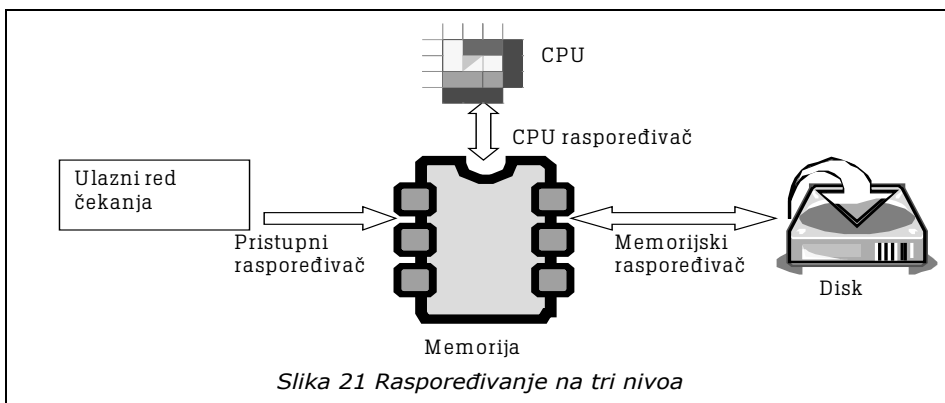
3.1.Raspoređivač

Raspoređivač (engl. *scheduler*) donosi odluku kojem procesu se dodjeljuje procesor u kojem trenutku. Raspoređivač tu odluku može donijeti na više načina, koristeći odgovarajući algoritam raspoređivanja. Raspoređivanje procesa se obavlja dok se izvršava programski kôd jezgra, najčešće u toku izvršavanja sistemskog poziva ili prekida (*interrupt*). Neki operativni sistemi imaju više vrsta raspoređivača. U *batch* sistemu često je više procesa nego što može biti izvršeno odmah. Ovi procesi se tipično prebacuju na disk gdje se čuvaju do kasnijeg izvršenja. Dugoročni (engl. *long-term*) raspoređivač poslova izabere proces i puni ga u memoriju za izvršenje. Kratkoročni raspoređivač poslova (engl. *short-term scheduler*) bira između procesa koji su spremni za izvršenje i dodjeljuje CPU jednom od njih. Osnovna razlika je u frekvenciji rada ova dva raspoređivača.

3.2.Nivoi i vrijeme raspoređivanja

Ovisno o tome sa kojom frekvencijom se poziva raspoređivač, sistemi mogu imati raspoređivače na tri, dva ili jednom nivou.

Batch sistemi dozvoljavaju raspoređivanje na tri različita nivoa (Slika 21). Ako poslovi stižu u sistem oni se inicijalno smještaju u ulazni red čekanja smješten na disku.



Pristupni raspoređivač odlučuje koji od poslova treba uvesti u sistem. Ostali se drže u ulaznom redu čekanja dok se ne izaberu. Tipični algoritam za upravljanje pristupom sistemu bi mogao posmatrati poslove vezane za procesor i poslove vezane za U/I i odabirati njihovu kombinaciju da se postigne što bolja balansiranost resursa ili birati procese prema njihovom očekivanom vremenu izvršenja.

Kada se posao prihvati da uđe u sistem, on se može natjecati za procesor. Međutim, moglo bi se desiti da je broj procesa tako veliki da za njih nema mjesta u memoriji. U tom slučaju, neke od procesa treba izbaciti na disk (operacija *swap out*). Drugi nivo raspoređivanja je **raspoređivač memorije**. On odlučuje o tome koje procese treba ostaviti u memoriji a koje bi trebalo držati na disku. Pošto je operacija dovođenja procesa sa diska dugo traje, ove odluke ne bi trebalo da se donose češće od jednom u sekundi. Kriteriji za držanje procesa u memoriji ili na disku su proteklo vrijeme od zadnjeg izbacivanja ili ubacivanja procesa, prioritet procesa, veličina, potrošeno procesorsko vrijeme, ukupan planirani broj procesa u memoriji itd.

Treći nivo raspoređivanja je određivanje jednog od spremnih procesa u memoriji da bi se izvršio na procesoru. Često se ovaj raspoređivač zove **raspoređivačem procesora** i on je onaj na koji se obično misli kada se govori o raspoređivaču. Trenutak kada proces mijenja stanje zbog prelaska u drugi red čekanja ili završetka izvršavanja je pogodan trenutak za poziv raspoređivača, ali ih ima još. Potencijalni trenuci za poziv ovog raspoređivača su:

1. Kada se novi proces kreira,
2. Kada proces završi svoje izvođenje,
3. Kada se proces blokira radi čekanja na ulazno/izlazni uređaj, slobodnu memoriju ili događaj
4. Kada se desi U/I prekid, koji obično znači da je U/I uređaj završio posao. Tada se proces koji se blokirao zbog tog uređaja može ponovo dovesti u red spremnih procesa.
5. Kada se desi prekid sata u kome je izmjereno da se proces izvršavao suviše dugo.
6. Pri nekom sistemskom pozivu čija je sporedna aktivnost također poziv raspoređivača.

Kada je proces završio s svojim izvršavanjem ili se blokirao (slučajevi 2 i 4) mora se izabrati sljedeći proces koji će se dovesti na procesor. U ostalim slučajevima raspoređivanje ne mora da se odmah uradi, iako je uobičajeno da se tada radi.

Na modernim personalnim sistemima koji imaju upravljanje memorijom preko straničenja raspoređivač radi samo na trećem nivou.

3.3. Ciljevi i kriteriji algoritama za raspoređivanje

Algoritmi raspoređivanja imaju više ciljeva koje trebaju ispuniti. Ti ciljevi su:

1. **Poštenost** znači da svaki proces treba dobiti šansu da se izvršava i da procesi iste kategorije dobivaju približno jednako procesorsko vrijeme.
2. **Postizanje prioriteta**, suprotno prvom kriteriju, znači da procesi trebaju dobiti više procesorskog vremena ako su većeg prioriteta.
3. **Ispunjenje pravila** je jako zavisno od sistema. Neki sistemi zahtijevaju da se pokrenuti proces započne izvršavati bez obzira sa kakvim zakašnjenjem, dok kod nekih je bolje da se proces uopće ne izvršava nego da kasni. No, koje god pravilo odabrali, treba ga poštovati.

4. **Skalabilnost** se odnosi na prilagođenost algoritma različitim stepenima opterećenja, na primjer povećanje broja procesa da ne usporava drastično raspoređivač.
5. **Balansiranje resursa** znači održavanje svih dijelova sistema zauzetim kada je to moguće. Ako procesor i ulazno/izlazni uređaji mogu raditi cijelo vrijeme i istovremeno, više poslova može biti završeno u jedinici vremena nego u sistemu gdje su komponente nezaposlene. Ako se mogu uočiti procesi koji većinu vremena rade ulazno izlazne operacije (U/I vezani) i procesi koji većinu vremena obavljaju račun, a manje koriste U/I uređaje (U/I vezani), bolje balansiranje resursa se postiže ako se istovremeno izvršavaju procesorski vezani i U/I vezani procesi nego ako su svi procesi koji se izvršavaju jednog tipa.
6. **Transparentnost** se odnosi na višeprocorske i distribuirane sisteme. Ovaj kriterij znači da za korisnika ne treba biti velike razlike između načina raspoređivanja na jednom ili drugom procesoru ili lokalnom i udaljenom računaru.
7. Maksimiziranje ili minimiziranje vrijednosti **pojedinih kriterija algoritama za raspoređivanje**: Uspješnost algoritma za raspoređivanje može se mjeriti i nekim numeričkim kriterijima koje treba držati što većim ili što manjim. Ti kriteriji su:
 - a) **Vrijeme boravka** u sistemu predstavlja srednje vrijeme boravka procesa u sistemu od momenta kada je ušao u sistem do momenta kada završi svoje izvođenje.
 - b) **Vrijeme čekanja** je srednje vrijeme koje proces provede u redu čekanja na procesor.
 - c) **Propusnost** je broj poslova koji se završi u jedinici vremena (recimo jednom satu). Kažemo da je propusnost veća ako se završi 10 poslova u nekom vremenskom periodu od sistema koji to uradi sa 5 poslova u istom periodu.
 - d) **Srednje opterećenje** je prosječan broj procesa koji se nalaze u redu čekanja.
 - e) **Iskorištenost procesora** predstavlja vrijeme provedeno u izvršavanju procesa u jedinici vremena. Na primjer, ako je računar u periodu od 1000 s proveo 750 s u izvršavanju korisničkih procesa, a preostalih 250 s u praznom hodu, kažemo da je iskorištenje procesora 75%.
 - f) **Vrijeme odziva** je vrijeme između zadavanja komande i dobivanja prvih rezultata. Ovo nije isto što i vrijeme boravka u sistemu koje se odnosi na vrijeme od početka do završetka cijelog procesa.
 - g) **Poštovanje rokova** predstavlja procenat ispunjenja rokova u nekoj vremenskoj jedinici u kome su procesi izvršili svoje zadatke.
 - h) **Predvidljivost** predstavlja procenat sigurnosti s kojim možemo tvrditi da će procesi poštovati postavljene rokove izvršenja za vrijeme života sistema.

Neki od ovih ciljeva su u kontradikciji s drugim. Na primjer, maksimiziranje cilja ispunjenja prioriteta, dovodi do minimiziranja poštenosti, jer nisko prioritetni procesi neće dobiti šansu da se izvršavaju. U nekim slučajevima ta kontradikcija nije tako očita. Minimiziranje vremena boravka u sistemu bi obično povećavalo propusnost. Ali, ako u sistemu postoje kratki i dugi procesi i raspoređivač stalno bira kratke procese na štetu dugih, on može postići izvrsnu propusnost (mnogo kratkih

poslova u nekom periodu vremena) ali na račun ogromnih vremena boravka u sistemu za duge poslove.

Očito je nemoguće ispuniti sve ciljeve, ali to i nije namjera. Različiti operativni sistemi imaju različite ciljeve.

Svi sistemi za cilj imaju poštenost, balansiranje resursa i ispunjenje pravila koja taj sistem zahtijeva. Pored ovoga svaka vrsta sistema ima pojedine specijalne ciljeve.

Pozadinski (*batch*) sistemi, ali i moderni *cloud* sistemi, su se često naplaćivali po vremenu korištenja ili imali dugo vrijeme od pokretanja procesa do njegovog završetka. Stoga je kod ovakvih sistema cilj povećati propusnost, smanjiti vrijeme boravka procesa u sistemu i povećati iskorištenje procesora.

U interaktivnim sistemima treba prije svega smanjiti vrijeme odziva, što predstavlja vrijeme između izdavanja komande i dobivanja rezultata jer je veoma nezgodno čekati 20 sekundi nakon svakog pritiska na taster prilikom obrade teksta. Posljedica je da se veći prioritet kod ovih sistema daje aplikacijama koje direktno komuniciraju s korisnikom.

U sistemima realnog vremena, ako računar kontroliše uređaj koji proizvodi podatke određenim intenzitetom i ne uspije da izvrši proces koji sakuplja podatke to može završiti sa gubljenjem podataka. Stoga, glavni zahtjevi od sistema realnog vremena su poštovanje rokova i predvidljivost.

U višeprocorskim i distribuiranim sistemima ciljevi su povećanje iskorištenosti, transparentnost i skalabilnost.

Nekada se ciljevi raspoređivača na istom sistemu mogu mijenjati u pojedinim situacijama. Na primjer, na *laptop* računaru dok je priključen na mrežno napajanje, može se povećavati iskorištenost procesora, ali pri prelasku na baterijsko napajanje iskorištenost treba smanjiti radi manje potrošnje električne energije. Ali kada padne nivo napajanja do tog nivoa da se mora snimiti memorija na disk i ugasi računar, tada se odustaje od cilja dobrog vremena odziva i pojačava cilj ispunjenja prioriteta.

3.4. Kategorije algoritma za raspoređivanje

Algoritme za raspoređivanje možemo dijeliti po njihovom odnosu prema trenucima izmjene konteksta i po namjeni.

Prema trenucima izmjene konteksta, algoritmi raspoređivanja se mogu podijeliti u dvije kategorije: **sa istiskivanjem** (*preemptive*) i **bez istiskivanja** (*non-preemptive*). Kod algoritama raspoređivanja bez istiskivanja proces se izabere za izvršavanje i pusti se da se izvršava dok samog sebe ne blokira (zbog U/I zahtjeva ili zbog čekanja na drugi proces) ili dok on dobrovoljno oslobodi procesor (sistemskim pozivom ili završetkom procesa). Takav način raspoređivanja je bio npr. u Windows 3.1 i u mnogim sistemima u realnom vremenu. Ovakvi raspoređivači su jednostavniji, ali loše napisani programi, na primjer oni koji su se zaglavili u petlji, mogu srušiti cijeli sistem.

Kod algoritma za raspoređivanje sa istiskivanjem, izabrani proces se izvršava fiksno vrijeme, ukoliko se u međuvremenu nije blokirao ili završio. Na kraju tog

vremenskog intervala, proces se skida sa procesora i raspoređivač bira drugi proces za izvršenje, ako je raspoloživ. Linux i Windows NT imaju ovakve raspoređivače.

Prema namjeni algoritme dijelimo po tome za kakvo su okruženje, tj. vrstu operativnih sistema pogodni. Različiti algoritmi su potrebni zbog različitih ciljeva koji imaju pojedina okruženja. Stoga postoje:

- Algoritmi za pozadinske (*batch*) sisteme
- Algoritmi za interaktivne sisteme
- Algoritmi za sisteme realnog vremena i multimedijalne sisteme
- Algoritmi za više-procesorske sisteme
- Algoritmi za distribuirane sisteme

U nastavku će biti predstavljeni algoritmi pogodni za različita okruženja. Algoritmi će biti ocjenjivani na primjerima koji uključuju različite trenutke dolaska procesa kroz tablice koje predstavljaju njihovo vrijeme boravka u sistemu, na procesoru i u čekanju, te kroz Ganttove dijagrame. Ganttov dijagram se inače koristi kod vođenja projekata i predstavlja koji se događaji dešavaju u pojedinom trenutku.

3.5.Algoritmi za pozadinske (*batch*) sisteme

Kod ovih sistema, cilj je imati što veću propusnost, veće iskorištenje hardvera i manje vrijeme boravka u sistemu. Pošto vrijeme odziva nije bitno, jer nema interakcije s korisnikom, za ove sisteme su pogodni algoritmi bez istiskivanja, koji imaju rjeđe izmjene između procesa, što poboljšava performanse jer ima manje internog rada.

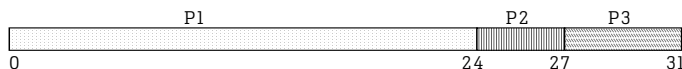
3.6.Prvi došao prvi uslužen (FCFS)

Napisano pravilo ponašanja ljudi u javnim ustanovama (npr. poštama, prodavaonicama), u kojima se čeka na neku uslugu u redu, je da oni koji su prije stali u red imaju prednost. Ovakav pristup je lako pratiti i organizirati, uprkos nekim nedostacima ovog pristupa (kupac s velikom korpom punom artikala će zaustaviti kupce iza njega sa malim brojem artikala, iako bi se više kupaca obradilo da su oni bili pušteni ranije).

Na bazi redoslijeda usluga po vremenu zahtjeva nastao je najjednostavniji od svih algoritama za raspoređivanje Prvi došao – prvi uslužen, skraćenica FCFS (engl. *first-come first-served*). U ovom algoritmu, procesi izvršavaju na procesoru onim redoslijedom kojim su pokrenuti. Svi procesi su povezani u red čekanja. Prvi kreirani proces dobije procesor i koristi ga koliko mu je potrebno. Ako se drugi procesi trebaju izvršavati, oni se smještaju kako dolaze na kraj reda čekanja. Ako se proces koji je zauzeo procesor završi ili blokira, onda se prepušta izvršavanje procesu koji je na početku reda čekanja. Ukoliko su se stekli uslovi za deblokiranje procesa, do tada blokirani proces se stavlja na kraj reda čekanja na procesor.

Slika 22 prikazuje vremena dolaska i trajanja tri procesa u nekim vremenskim jedinicama (npr. sekundama), uz Ganttov dijagram na kome se vidi kada će se koji proces dovesti na izvršavanje. Na slici se vidi da će zbog trajanja procesa P1, procesi P2 i P3 provesti dosta vremena u čekanju.

Proces	Vrijeme dolaska	Trajanje	Redoslijed	Vrijeme u sistemu	Vrijeme čekanja
P1	0	24	1	$24 - 0 = 24$	$24 - 24 = 0$
P2	3	3	2	$27 - 3 = 24$	$24 - 3 = 21$
P3	4	4	3	$31 - 4 = 27$	$27 - 4 = 23$



Srednje vrijeme čekanja $(0 + 21 + 23)/3 = 14,669$

Slika 22 Algoritam FCFS – prvi došao, prvi uslužen

Ovaj algoritam se lako razumije i programira, jer se raspoređivanje svodi na dodavanje procesa na kraj liste i preuzimanje procesa sa njenog početka. Glavni nedostatak algoritma je je slabo vrijeme odziva, što ga čini neprimjenjivim za interaktivne sisteme. Slika 22 pokazuje kako procesi P2 i P3 ne mogu da daju prvi odgovor sve dok se proces P1 ne završi. Drugi nedostatak FCFS algoritma je **efekt konvoja**, koji negativno utiče na balansiranu upotrebu resursa. On se dešava ako se procesi različitih profila povremeno blokiraju. Neka se npr. istovremeno izvršavaju jedan CPU vezani proces (npr. račun inverzne matrice) i više U/I vezanih procesa (npr. čekanje na podatke sa magnetne trake). U/I vezani poslovi dolaze na izvršenje i nakon kratkog izvršenja se blokiraju, čekajući na U/I. Nakon toga CPU vezani proces dolazi na početak reda čekanja i izvršava se duže vrijeme. U međuvremenu, stiže odziv od U/I uređaja i U/I vezani procesi se vraćaju u red čekanja, ne radeći ništa, jer čekaju na CPU vezani proces. Kada se CPU vezani proces završi, U/I vezani procesi ponovo dobijaju procesor, ali i brzo ga napuste, jer ponovo moraju čekati na U/I uređaj. Sada CPU nije iskorišten, jer se nijedan proces ne izvršava na njemu. Treći nedostatak je što srednje vrijeme čekanja nije optimalno.

3.7. Najkraći posao prvi -SJF (Shortest Job First)

U primjeru s redom čekanja u samoposluzi sa kupcem sa velikom korpom, iza koga je drugi kupac s jednim artiklom, ponekad se dešava da kupac s velikom korpom velikodušno propusti kupca iza sebe, svjestan da će kupcu s jednim artiklom ustupanje reda uštedjeti dosta vremena, a da će kupcu sa velikom korpom tih dodatnih pola minute potrebnih da se obradi drugi kupac biti zanemarivo u usporedbi s npr. 10 minuta da se obradi njegova korpa.

Ovakva strategija nas vodi do algoritma **Najkraći posao prvi**. Kada se nakupilo više procesa u redu čekanja i treba birati koji će se naredni izvršavati, odabira se onaj čije je vrijeme izvršavanja najkraće. Kao i kod FCFS, izabrani proces se izvršava do kraja ili blokiranja, dakle algoritam je bez istiskivanja. Ako su svi procesi ubačeni u sistem u isto vrijeme, ovaj algoritam ima optimalno srednje vrijeme čekanja. Dokaz: Neka ima n procesa, dostupnih u trenutku $t=0$ koji se izvršavaju $tr_1, tr_2, \dots, tr_n$ vremenskih jedinica i trebaju se završiti u vrijeme $tf_1, tf_2, \dots, tf_n$. Uočimo da je trenutak završetka procesa ujedno i trenutak početka narednog, pa važi

$$tf_1 = 0 + tr_1$$

$$tf_2 = tf_1 + tr_2 = tr_1 + tr_2$$

$$tf_k = \sum_{i=1}^k tr_i$$

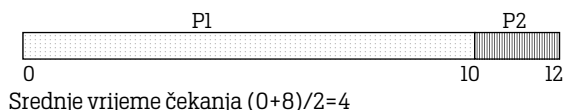
Srednje vrijeme čekanja je stoga

$$tw = \frac{1}{n} \sum_{i=1}^n (tf_i - tr_i) = \frac{1}{n} \sum_{k=1}^n \sum_{i=1}^{k-1} tr_i = \frac{1}{n} \sum_{i=1}^{n-1} (n-i) tr_i$$

Navedena suma je minimalna ako važi da je $tr_i \leq tr_{i+1}$

Ovaj algoritam ima ograničenu primjenu jer je potrebno znati vrijeme izvršavanja svakog procesa unaprijed. To je moguće u nekim sistemima realnog vremena, u interaktivnim sistemima je nemoguće, dok se u *batch* sistemima vrijeme izvršavanja može procijeniti na dva načina. Prvi način pretpostavlja da će dužina izvršavanja procesa biti proporcionalna veličini podataka koje on obrađuje, na primjer veličini ulazne datoteke. Drugi način uzima u obzir ranija vremena izvršavanja tog procesa na bazi statističke analize.

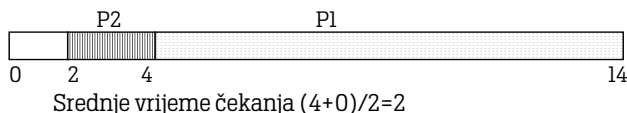
Proces	Vrijeme dolaska	Trajanje	Redoslijed	Vrijeme u sistemu	Vrijeme čekanja
P1	0	10	1	10-0=10	10-10=0
P2	2	2	2	12-2=10	10-2=8



Slika 23: SJF, kraći posao došao dok duži još traje

Važno je ukazati na činjenicu da je SJF optimalan samo kada su svi procesi raspoloživi simultano (recimo, svi počinju da se izvršavaju u isto vrijeme). Kao primjer, razmotrimo 2 procesa, P1 i P2, sa vremenima dolazaka 0 i 2 i vremenima izvršenja 10 i 2. Pošto je u trenutku 0 je raspoloživ samo proces A, imamo situaciju sa srednjim vremenom čekanja 4 (Slika 23). Ali, ako se sačeka dolazak kraćeg posla (P2), imaćemo manje srednje vrijeme čekanja, 2 vremenske jedinice (Slika 24).

Proces	Vrijeme dolaska	Trajanje	Redoslijed	Vrijeme u sistemu	Vrijeme čekanja
P1	0	10	2	14-0=14	14-10=4
P2	2	2	1	4-2=2	2-2=0



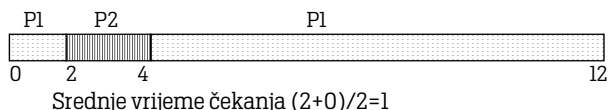
Slika 24: SJF, raspoloživost procesa u isto vrijeme

Dakle, optimalno srednje vrijeme čekanja bi imao algoritam koji daje prednost kraćim poslovima, ali koristio i procesor dok se takvi procesi ne pojave.

3.8. Najkraće preostalo vrijeme SRTN

Modifikacijom SJF algoritma u verziju sa istiskivanjem dolazi se do algoritma SRTN (engl. *shortest remaining time next*). Raspoređivanje se obavlja u trenucima kada se procesi završavaju, blokiraju ali i kada ulaze u sistem. Raspoređivač uvijek bira onaj proces čije je preostalo vrijeme za izvršavanje najkraće. Ako novom procesu treba manje vremena da se izvrši nego procesu koji se trenutno izvršava, tekući proces se prebacuje u red čekanja i novi proces započinje izvođenje. Slika 25 pokazuje prethodni primjer sa dva procesa, koristeći ovaj algoritam. Uočava se znatno manje srednje vrijeme čekanja.

Proces	Vrijeme dolaska	Trajanje	Redoslijed	Vrijeme u sistemu	Vrijeme čekanja
P1	0	10	1	$12-0=12$	$12-10=2$
P2	2	2	2	$4-2=2$	$2-2=0$



Slika 25: Algoritam SRTN, najkraće preostalo vrijeme sljedeće

Ovaj algoritam je optimalan i kada procesi nisu raspoloživi simultano. Nažalost, primjenjiv je samo u ograničenom broju slučajeva, jer i ovdje vrijeme izvršavanja mora biti poznato unaprijed. Ozbiljan problem s ovim algoritmom je izglednjivanje procesa. Ako se izvršava proces kome treba mnogo vremena, a stalno dolaze kraći procesi relativno brzo i prekidaju ga, nastavak izvršavanja dugačkog procesa će biti stalno odlagan.

3.9. Najbolji odnos odziva sljedeći (HRRN)

Problem izglednjivanja procesa kod SRTN algoritma doveo je do prijedloga algoritma koji daje šansu za izvršavanje i procesima koji su dugo u čekanju. Algoritam Najbolji odnos odziva sljedeće (*Highest response ratio next*), HRRN raspoređivanje obavlja u isto vrijeme kada i SRTN algoritam, pri pokretanju, završetku procesa i blokiranju. Ali, ne bira se proces s najkraćim preostalim vremenom, nego proces s najvećim količnikom vremena provedenog u čekanju i preostalog vremena izvršenja. Jasno, ovaj algoritam nema najmanje srednje vrijeme čekanja, ali bolje ispunjava cilj poštenosti.

3.10. Prioritetno raspoređivanje bez istiskivanja

Umjesto teško mjerljivog vremena izvršenja, može se uvesti i drugi kriterij za izbor procesa. Na primjer, procesi u iznajmljenom računarskom centru za koje je plaćeno više mogu započeti ranije od onih koji su pokrenuti u testne svrhe. Tada se svakom procesu dodjeljuje određeni prioritet, numerička vrijednost koja određuje redoslijed izvršavanja. Stvar konvencije je da li će manja ili veća vrijednost parametra prioriteta uticati na ranije izvršenje procesa. U ovim algoritmima raspoređivanje se obavlja u trenucima završavanja i blokiranja procesa. Prioritet se može odrediti interno i eksterno. Interno definisan prioritet je zasnovan na nekim

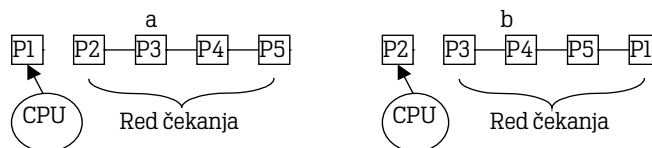
osobinama procesa u sistemu (primijetimo da je SJF specijalni slučaj ovog algoritma), dok eksterni prioritet se određuje od strane sistemskog operatora.

U ranim operativnim sistemima za personalne računare (MS DOS i FreeDOS) sreće se pojednostavljena verzija ovakvog raspoređivača. Kada se iz jednog procesa (najčešće komandnog interpretera) pokrene drugi proces, taj novopokrenuti proces uvijek ima veći prioritet. Na proces pokretač se neće vraćati sve dok se pokrenuti proces ne završi.

3.11.Algoritmi za interaktivne sisteme

Suprotno pozadinskim sistemima, u interaktivnim sistemima cilj je što brži odziv sistema. Neprihvatljivo je da neki proces predugo okupira procesor i time zablokira sve ostale. Stoga kod ovih sistema obično se oduzima procesor procesu na bazi sata, kako bi svi procesi dobili šansu za izvršavanje.

3.12.Kružno (Round-Robin)raspoređivanje



Slika 26: Princip Round-robin algoritma

Do sada opisani algoritmi raspoređivanja su puštali procese da se izvršavaju relativno dugo vrijeme. No, ovakav način raspoređivanja nije pogodan u sistemima koji interaktivno rade s korisnicima. Može se i pretpostaviti kako bi naporno bilo čekati 20 minuta do početka procesa koji direktno radi s korisnikom. Upotreba prioritetskog algoritma sa istiskivanjem, uz postavljanje najvećeg prioriteta, interaktivnom korisniku bi omogućila tom korisniku da brzo dobije odziv za svoju aplikaciju, ali nije rješenje ako postoje dva korisnika sa istim prioritetom i željom za interaktivni rad. Jedan od njih bi i dalje morao čekati završetak drugog procesa. Rješenje je, naravno, u osnovnom principu sistema u dijeljenom vremenu: dodjela relativno kratkog perioda kada se neki proces izvršava na procesoru, a zatim hardverski prekid na bazi sata i prelazak na drugi proces.

Na bazi toga, nastao je jednostavan i pošten algoritam **Round robin** ili kružno raspoređivanje. U ovom algoritmu svaki proces dobija jedan vremenski interval, koji se naziva **kvantum**, za vrijeme koga proces može da se izvršava. Raspoređivač se poziva kada se proces blokira ili završi, ali i kada mu je istekao taj vremenski interval, što se provjerava u prekidu generiranom sistemskim satom. Nakon tih događaja, proces se prebacuje u red čekanja, a izvršava se naredni proces u redu. Izbor procesa koji se sljedeći izvršava bazira se na redu čekanja (Slika 26a). Kada proces iskoristi svoj kvantum, on se postavlja na kraj reda (Slika 26b). Proces koji se tek pokreće dolazi na sam kraj liste čekanja.

Kvantum ne treba biti ni previše kratak ni previše dug. Ako je prekratak, tada postaje usporediv s vremenom izmjene konteksta između procesa. Neka je npr. kvantum jednak vremenu izmjene konteksta. Tada sistem jednako vremena potroši

na stvarno izvršavanje procesa koliko i na prebacivanje između njih, što smanjuje koristan rad procesora na 50%. Ako je kvantum previše dug, tada se algoritam približava FCFS algoritmu i vrijeme odziva procesa postaje neprihvatljivo za interaktivne korisnike.

Koliki onda kvantum odabrati? Pokazalo se, s obzirom na ljudsku reakciju koja je oko 0,1s, da kvantum od 10ms zadovoljava da se desetak procesa izvršava paralelno na jednom procesoru a da se stekne osjećaj da trenutnog odziva. Stoga je ovaj kvantum dobar za radne stanice. On je prekratak za servere, jer je reda brzine pristupa disku, i uglavnom korisnik ne bi stigao dobiti podatak s datotečnog servera za vrijeme njegovog kvantuma, pa se na serverskim računarima uzima kvantum od 100ms.

3.13. Kooperativno raspoređivanje

Ako operativni sistem nije dizajniran da omogući prekidanje procesa usred njegovog izvršavanja, *Round robin* nije moguć. To je slučaj s ranijim verzijama Windows (do 3.1) i MacOS. Windows 3.1 je samo grafička školjka za MS DOS. MS DOS nije imao mogućnost da se potprogrami jezgra mogu pozivati ako se neki potprogram jezgra već izvršava. Da bi se omogućio višeprocetni rad, a zadržala podloga dizajnirana za jednoprocesni operativni sistem, definirana su pravila samim procesima da povremeno pozivaju neki sistemski poziv koji će obaviti izmjenu konteksta. U slučaju Windows-a 3.1 svi programi moraju izvršavati sljedeću petlju.

```
while (GetMessage(&message, NULL, 0, 0) > 0) {  
    TranslateMessage(&message);  
    DispatchMessage(&message);  
}
```

Ovom petljom se očitavaju poruke smještene u red, a funkcijom *DispatchMessage* se te poruke prosljeđuju procesima koji izvršavaju rutine za obradu poruke namijenjene njima. Time svaki proces u svom izvršavanju dobrovoljno prepušta procesor drugim procesima svaki put kada pozove *DispatchMessage*.

U primjeru jezgra na kraju knjige, implementirana je ovakva vrsta raspoređivača. U tom jezgru se koristi *Round Robin* redoslijed, ali se izmjena između procesa ili sistemskih niti obavlja jedino pri izvršenju nekih sistemskih poziva.

Mana kooperativnih raspoređivača je što se programi moraju pisati strogo uzimajući u obzir postojanje takvih raspoređivača. Nije npr. dopustivo da se proces izvršava bez povremenog poziva specijalnog sistemskog poziva koji obavlja raspoređivanje. Takav proces bi zaustavio izvršavanje svih ostalih procesa.

3.14. Prioritetno raspoređivanje sa vremenskim prilagođivanjem prioriteta

Prioritetno raspoređivanje opisano u verziji za pozadinske sisteme je nepogodno za interaktivne sisteme jer pokretanje procesa višeg prioriteta u potpunosti zaustavlja procese nižeg prioriteta. S druge strane, osnovna verzija *Round robin* algoritma sve procese posmatra ravnopravno, a različiti prioriteti su često potrebni, kako bi se značajnijim procesima dalo više procesorskog vremena. Da bi spriječili visoko prioritetne procese da se ne izvršavaju previše dugo, raspoređivač može smanjiti prioritet tekućeg procesa koji se izvršava na svakom prekidu sata. U

modifikovanoj verziji prioritetnog raspoređivanja procesima se dodjeljuje vrijednost koja se zove kredit. Kredit se inicijalno računa na bazi prioriteta procesa. Nakon svakog prekida sata kredit se smanjuje određenom formulom. Zavisno od dizajna algoritma može se proces izvršavati dok mu kredit ne dostigne vrijednost 0 ili dok ne padne ispod vrijednosti kredita koju ima neki drugi proces. U prvoj varijanti kredit ima više funkciju kvantuma, a u drugoj funkciju prioriteta. Kada svi krediti dođu do nule ponovo se računaju.

Prioriteti iz kojih se računaju kvantumi se pridružuju procesima statički i dinamički. Statički prioritet se dodjeljuje od strane administratora ili korisnika sistema prema potrebama. Npr., procesi koji vrše arhiviranje podataka mogu imati niži prioritet od procesa koji neposredno rade s korisnikom. Primjer postavljanja ovakvih prioriteta je komanda *nice* u Unix sistemima ili promjena prioriteta procesa u Windows *Task Manageru*. Prioriteti se mogu mijenjati i dinamički da bi se postigli određeni ciljevi. Na primjer, ako raspoređivač uoči da je neki proces uglavnom provodio U/I operacije i samo zakratko zatražio procesor može mu se povećati prioritet kako bi što prije obavio posao na procesoru i ponovo se posvetio U/I uređajima jer će takav proces manje trošiti resurse nego kada bi se duže izvršavao. Slično, u većini interaktivnih sistema dinamički se dodjeljuje veći prioritet onom procesu koji ima fokus tastature.

Ovakav način raspoređivanja primijenjen je u ranijim Linux sistemima. Raspoređivač realizovan u verziji 0.11 je toliko kratak, da ga možemo pogledati u izvornom kodu na jednoj strani ove knjige. Raspoređivač se poziva na više mjesta, između ostalog i u obradi prekida sata što se vidi iz funkcije *do_timer* koja se poziva na svaki prekid sata. U svakom prekidu se brojač kredita umanjuje za 1 i kada dostigne nulu, poziva se raspoređivač.

Glavni dio raspoređivača je u funkciji *schedule()*. U listi *p* se čuvaju kontrolni blokovi svih procesa. U prvoj *for* petlji ovaj raspoređivač najprije provjeri da li ima nekih blokiranih procesa koji čekaju da prime signale ili alarme i ako su im stigli signali, promijeni im se stanje u *TASK_RUNNING*. Nakon ovoga se provjeri ima li još aktivnih procesa sa kreditom većim od nule. Unutar *while(1)* petlje se gleda koji od procesa ima trenutno najveći kredit (polje *counter*) i to dodjeli varijabli *c*, a varijabli *next* se dodijeli pozicija tog procesa u listi kontrolnih blokova.

Ako varijabla *c* na kraju ima vrijednost 0, to znači da svi procesi u stanju *TASK_RUNNING* imaju potrošene kredite. U tom slučaju se svim procesima kredit računa po formuli:
$$Kredit := \frac{Kredit}{2} + Prioritet$$

Glavna mana ovog algoritma je u slaboj skalabilnosti. Pri svakom donošenju odluke o narednom procesu mora se proći kroz cijelu listu procesa. Ako broj procesa raste, ovo pretraživanje postaje sve sporije i sporije. Tako je vremenom u Linux-u povećavana složenost raspoređivača i funkcija izbora, što je problem učinilo još izraženijim, pa je od verzije 2.6 zamijenjen algoritmom $O(1)$, a od 2.6.23 algoritmom CFS (potpuno fer raspoređivanje).

```

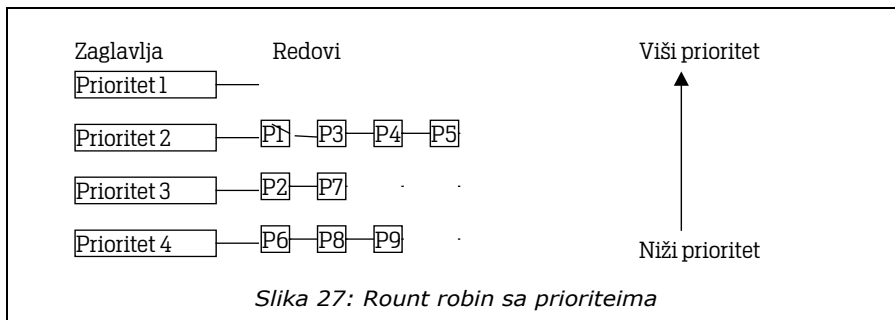
void do_timer(long cpl)
{ /* .... */
    if (--current->counter>0) return;
    current->counter=0;
    if (!cpl) return;
    schedule();
}
void schedule(void)
{
    int i,next,c;
    struct task_struct ** p;
    for(p = &LAST_TASK ; p > &FIRST_TASK ; --p)
        if (*p) { /* Ako proces postoji, provjeri ima li signala */
            if ((*p)->alarm && (*p)->alarm < jiffies) {
                (*p)->signal |= (1<<(SIGALRM-1));
                (*p)->alarm = 0;
            }
            if (((*p)->signal & ~(_BLOCKABLE & (*p)->blocked)) &&
                (*p)->state==TASK_INTERRUPTIBLE)
                (*p)->state=TASK_RUNNING;
        }
    while (1) {
        c = -1; /* najveći kredit je trenutno minimalan */
        next = 0;
        i = NR_TASKS; /* kreni od kraja liste */
        p = &task[NR_TASKS];
        while (--i) {
            if (!*--p) /* preskoči prazne kontrolne blokove */
                continue;
            if ((*p)->state == TASK_RUNNING && (*p)->counter > c)
                c = (*p)->counter, next = i; /* bolji kandidat */
        }
        if (c) break;
        for(p = &LAST_TASK ; p > &FIRST_TASK ; --p)
            if (*p)
                (*p)->counter = ((*p)->counter >> 1) + (*p)->priority;
    }
    switch_to(next);
}

```

3.15.Round Robin sa prioritetima

Dizajn prioriteta može biti i takav da niže prioritetni procesi ne trebaju uopće da dobivaju procesor dok god postoje procesi većeg prioriteta, ali da procesi istog prioriteta trebaju svi da se izvršavaju ako nema važnijih procesa (a ne samo jedan od njih kao kod prioritetnog algoritma bez istiskivanja). Na primjer, normalni korisnički procesi trebaju da se smjenjuju s pristojnim vremenom odziva, a neki pozadinski procesi kao što je automatska instalacija nove verzije operativnog sistema treba da radi samo kada nema drugih procesa. U takvim slučajevima može se ići i na ovakvu shemu raspoređivanja. Za svaki prioritet se formira poseban red čekanja. Unutar iste klase prioriteta koristi se *Round robin* raspoređivanje, a prioritetno raspoređivanje među klasama. Slika 27 prikazuje sistem sa četiri klase prioriteta. Algoritam raspoređivanja radi na sljedeći način: dok god ima izvršivih procesa u klasi većeg prioriteta, pokrene se svaki proces u klasi za kvantum na *round*

robin način, i ne brine o klasama nižeg prioriteta. U ovom primjeru razmjenjivali bi se procesi P1, P3, P4 i P5, pošto nema procesa najvećeg prioriteta.

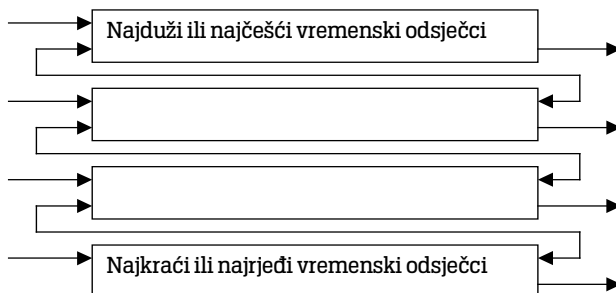


Slika 27: Round robin sa prioritetima

3.16. Višestruki redovi čekanja

Round robin sa prioritetima je specijalni slučaj višestrukih redova čekanja. Za svaki nivo prioriteta može se definirati poseban red čekanja i izvršavaju se procesi najprioritetnijeg reda čekanja koji nije prazan. Glavna razlika u odnosu na *Round robin* sa prioritetima je što svaki red čekanja može imati različit algoritam raspoređivanja. Na primjer, u jednom redu čekanja se koristi *Round robin* sa kraćim kvantom, u drugom *Round robin* s dužim kvantom, a u trećem FCFS. Logika ovog pristupa je da pozadinskim procesima treba veći kvantum, u ekstremnom slučaju neograničen. Kada nema prioritetnijih interaktivnih procesa, tada se niže prioritetni procesi izvršavaju sa parametrima koji su njima bolje prilagođeni.

3.17. Raspoređivanje sa više nivoa sa povratnom spregom



Slika 28 Višenivovsko raspoređivanje s različitim vremenskim odsječima

U tehnici usluživanja sa višestrukim redovima čekanja ne postoji kretanje procesa između nivoa. To znači da se nadolazeći procesi smještaju u redove čekanja na osnovu unaprijed date klasifikacije i ostaje u ovom redu čekanja dok god postoji. Ponašanje procesa u praksi se, međutim, mijenja i zato je uvedena tehnika raspoređivanja na više nivoa sa povratnom spregom (engl. *Multi-level-Feedback*). Kod ove tehnike raspoređivanja prioritet procesa se može mijenjati na osnovu rezultata korištenja procesora. Ova tehnika sprječava izgladnjivanje procesa i pokazuje dobre osobine u borbi sa "konvojskim" efektom. Raspoređivanje među različitim redovima čekanja može se izvesti promjenom dužine ili frekvencije

vremenskog odsječka. Raspoređivanje unutar svakog reda čekanja se, također, može mijenjati (Slika 28).

Prvi primjer ovakvog raspoređivača bio je u operativnom sistemu CTTS na računar IBM 7094 iz 1962. godine. Kako je u memoriji mogao biti samo jedan proces, uvedeno je više redova čekanja takvih da je svaki red čekanja imao kvantume tako raspoređene da procesi najvećeg prioriteta imaju najkraće kvantume, a najmanje prioriteta najduže. Na prvi pogled, ovo je nelogična odluka, ali je prilagođena maloj memoriji. Što je proces duže u sistemu, njegov prioritet pada u nižu klasu sa svakom izmjenom procesa, ali kada dođe na red za izvršenje dobiva sve duži i duži kvantum. Posljedica je da je potrebno manje snimanja procesa na disk ako se proces izvršava duže vremena.

Drugi primjer ovog raspoređivača primijenjen je u Windows NT seriji. Windows poznaje 32 nivoa prioriteta niti (ujedno i procesa, jer svaki proces ima bar jednu nit). U ovom operativnom sistemu (za razliku od Unix sistema) manji broj predstavlja manji prioritet. Prioriteti se dijele u tri klase: sistemski (nivo 0), varijabilni (nivoi 1 do 15) i klasa realnog vremena (nivoi 16 do 31). Niti sistemskog nivoa (*idle process*) se izvršavaju samo kada su svi ostali procesi neaktivni. Niti i procesi klase realnog vremena ne mijenjaju svoj prioritet u toku rada, i procesi u ovoj klasi se ponašaju kao kod algoritma *Round robin* sa prioritetima (pa mogu zagušiti cijeli sistem). Međutim, procesi i niti varijabilne klase povećavaju svoj prioritet na maksimalni u svojoj klasi (*priority boost*) u sljedećim događajima:

- Kada se proces normalne klase prioriteta se prebaci u prvi plan (odabrani prozor),
- Kada prozor primi ulaz: poruke od sistemskog sata, miša ili tastature.
- Kada se zadovolje uslovi čekanja blokirane niti.

Nakon isteka kvantuma za nit kojoj je ovako povišen prioritet, prioritet niti opada, sve dok ne dođe do svog baznog prioriteta.

3.18. Garantovano raspoređivanje

Round robin daje jednako vrijeme izvršenja svakom od procesa koristeći prekide na bazi sata. No, nekada ovakav način postizanja cilja pravičnosti nije moguć, npr. u distribuiranim sistemima koji nemaju jedinstveni sistemski sat. Ovaj cilj se može aproksimativno postići i sljedećim adaptivnim algoritmom. Najprije se izračuna relativna količina vremena koja treba biti dodijeljena procesu. Na primjer, ako u sistemu ima n procesa, svaki proces bi mogao dobiti $\frac{1}{n}$ procesorskog vremena.

Nazovimo vrijeme kada je izračunata ova relativna količina vremena za sve procese trenutkom obećanja. U svakom trenutku kada se poziva raspoređivač, izračuna se ukupno vrijeme proteklo od trenutka obećanja t_s , a za sve procese koliko je taj proces proveo u izvršavanju t_r . Za svaki proces se izračuna količnik $\frac{nt_r}{t_s}$ i što je on manji, to

je obećanje lošije ispunjeno. Stoga se za naredno izvršavanje bira proces s najmanjim ovakvim odnosom. Glavna prednost ovog algoritma je što se može definirati i za druge ciljeve osim ravnomjerne raspodjele vremena među procesima, npr. za ravnomjernu raspodjelu vremena među korisnicima ili različita vremena

izvršavanja procesa zbog različitih prioriteta. Algoritam je relativno težak za realizaciju ako se broj procesa mijenja i nije dobro skalabilan jer se često mora računati ponašanje svih procesa. Algoritam je loš po kriteriju ispunjenja pravila jer dopušta i postavljanje nerealnih ciljeva, npr. može se postaviti obećanje da svaki od 5 procesa u jednoprocorskom sistemu dobije po 70% ukupnog procesorskog vremena.

3.19. Lutrijsko raspoređivanje

Svi dosadašnji algoritmi su bili deterministički. Sada će se preći na jedan stohastički algoritam. Ovaj algoritam se zove lutrijsko raspoređivanje. U ovom algoritmu se procesima daju tiketi, brojevi, za procesorsko vrijeme. Kad god odluka o raspoređivanju treba da se napravi, lutrijski tiket se bira slučajno i proces koji drži taj tiket dobiva naredni vremenski kvantum. Procesi većeg prioriteta mogu dobiti dodatne tikete da bi povećali svoju šansu za dobitak. Ako proces drži 20% ukupnog broja tiketa, na duže staze, dobiva 20 procenata procesorskog vremena.

Dopušteno je i poklanjanje tiketa. Na primjer, roditeljski proces se blokirao čekajući završetak djetetog procesa. Tada roditeljski proces može prepustiti tikete djetetog procesu da bi ubrzao njegovo izvršenje.

Slika 29 pokazuje kako je ovaj algoritam skalabilan. Neka poslovi niskog prioriteta dobivaju po 1 tiket, a poslovi visokog prioriteta po 10 tiketa. Kako se povećava broj procesa različitih vrsta prioriteta, svaki od njih opet dobiva vrijeme proporcionalno svom prioritetu i broju procesa određene prioritetne klase. Mana algoritma je što se ne smije primjenjivati u sistemima gdje se očekuje determinizam u raspoređivanju.

Procesa visokog prioriteta	Procesa niskog prioriteta	%CPU za 1 proces visokog prioriteta	%CPU za 1 procese niskog prioriteta
1	1	91	9
0	2	0	50
2	0	50	0
10	1	9,9	0,99
1	10	50	6

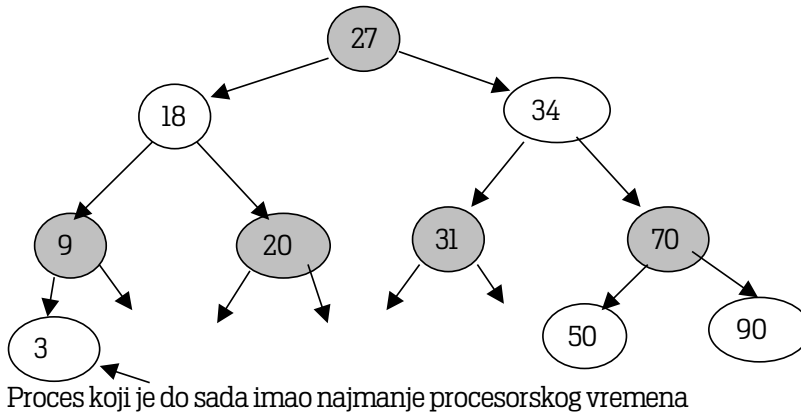
Slika 29 Lutrijsko raspoređivanje

3.20 Raspoređivanje s fer raspodjelom

U višekorisničkim sistemima može se prevariti poštenost *Round robin* algoritma, pokretanjem više procesa od strane istog korisnika. Neka je npr. Bob pokrenuo jedan proces (B1), Alisa 2 procesa (A1, A2), a Trudy 7 procesa (T1, T2, T3, T4, T5, T6, T7). Neka je sekvenca izvršavanja npr. A1,B1,T1,A2, T2, T3, T4, T5, T6, T7, A1,B1,T1,A2, T2, T3, T4, T5, T6, T7,... Iako je uočiti da Trudy dobiva 70% ukupnog procesorskog vremena. Ovaj efekt se može izbjeći ako se *Round robin* pravila primijene nad korisnicima, ne samo nad procesima. Tako se može nakon svakog kvantuma dodijeliti procesor procesu narednog korisnika a svaki put kada taj korisnik dobije procesor primijeniti *Round robin* pravilo između procesa tog korisnika. U ovom primjeru imamo sekvencu izvršavanja A1,B1,T1,A2,B1, T2, A1,B1,T3, A2,B1,T4, A1,B1, T5, A2,B1,T6, A1,B1, T7, A2,B1,T1,... i svi korisnici dobijaju jednako dodijeljeno vrijeme za izvršenje.

3.21 Potpuno fer raspoređivanje

U trenutnim verzijama jezgra Linux-a, počevši od verzije 2.6.23 koristi se kompleksan raspoređivač CFS, potpuno fer raspodjela. Riječ je o kombinaciji garantovanog raspoređivanja i raspoređivanja s fer raspodjelom. Podjela se obavlja i između korisnika da oni dobiju količinu ukupnog procesorskog vremena prema svom prioritetu, a i između procesa unutar vremena dodijeljenog korisniku.



Slika 30: Stablo korištenosti CPU u Potpuno fer algoritmu

Za razliku od većine algoritama raspoređivanja, podaci o procesima potrebni za raspoređivanje se ne čuvaju u ulančanoj listi nego u balansiranom binarnom crno-crvenom stablu. To ubrzava pretraživanje procesa. Čvorovi u ovom stablu su indeksirani po vremenu u nanosekundama koliko se proces izvršavao na procesoru. Za svaki proces se računa i maksimalno vrijeme izvršenja koje bi imao da je dobio idealno izvršavanje na procesoru, prema vremenu dodijeljenom njegovom vlasniku i broju procesa koje je vlasnik pokrenuo. Raspoređivač tada pogleda krajnji lijevi čvor balansiranog stabla i time bira proces čije je do sada potrošeno vrijeme minimalno. Kada se proces završio, on se izbacuje iz stabla. Ako je proces dostigao maksimalno vrijeme izvršenja ili je blokiran, ubacuje se u stablo prema novom proteklom vremenu izvršenja. Nakon ovih postupaka se bira novi krajnji lijevi čvor.

3.22. Algoritmi za sisteme realnog vremena i multimedijalne sisteme

Ovdje je cilj poštovanje rokova i predvidljivost vremena izvršavanja. Proces i ovim sistemima se izvršavaju relativno kratko i često se sami blokiraju kada obrade podatke. Obično je mali broj procesa u sistemu i uglavnom je unaprijed poznat. Kod ovih sistema vrijeme igra veoma važnu ulogu. Različiti uređaji u ovim sistemima šalju podatke centralnom računar i on mora da ih obradi u planiranom vremenu. Značenje riječi "mora" u prethodnoj rečenici je ipak različito između različitih sistema. Prema prihvatljivosti kašnjenja, sistemi realnog vremena su svrstani u dvije kategorije: **strogi** (*hard*) sistemi u realnom vremenu, gdje se krajnji rokovi (engl. *deadline*) moraju postići, i **relaksirani** (*soft*) sistemi u realnom vremenu gdje je nepostizanje krajnjeg odgovora u predviđenom intervalu je nepoželjno ali ipak

tolerirano. Primjer prvog slučaja je sistem za upravljanje ABS kočnicama u automobilu, a drugog server za prikaz video zapisa.

3.23. Ručno raspoređivanje

Broj procesa koji postoje u ovakvim sistemima je mali i njihovo ponašanje je poznato unaprijed. Na primjer, izmjereno je prateći sve grane uslova i petlji da se proces, koji na bazi primljenog podatka od senzora temperature šalje upravljački signal aktuatoru za pokretanje ventila, za bilo koje ulazne podatke izvršava maksimalno 0,5 s.

Ako je vrijeme odziva poznato za sve procese, onda se mogu ručno napraviti statičke tabele koje kažu u koje vrijeme se pokreću koji procesi. Na primjer, u uređaju za digitalizaciju TV programa snimljenom u starom PAL sistemu može se navesti da se svakih 64 μ s pokrene proces koji dohvata signal u memoriju i traje 52 μ s. To se ponovi 304 puta i pokrene proces koji će snimljeni kadar poslati koderu za njegovu kompresiju i ima rok od 5,12 ms za to (na starim televizorima s katodnom cijevi to je vrijeme predviđeno za vraćanje topa elektrona).

Pri pravljenju statičkih tabela raspoređivanja treba proračunati da li je to uopće moguće. Događaji na koje mora odgovoriti sistem u realnom vremenu mogu se podijeliti na **periodične** (koji se dešavaju u pravilnim intervalima) i **neperiodične** (koji se dešavaju nepredvidivo). Ne uzimajući u obzir vrijeme izmjene konteksta, ako postoji više različitih periodičnih događaja, a događaj i se dešava sa periodom T_i i zahtijeva C_i sekundi procesorskog vremena da bi se obradio događaj, onda je obrada ovih događaja moguća ako važi
$$\sum_{i=1}^n \frac{C_i}{T_i} \leq 1$$

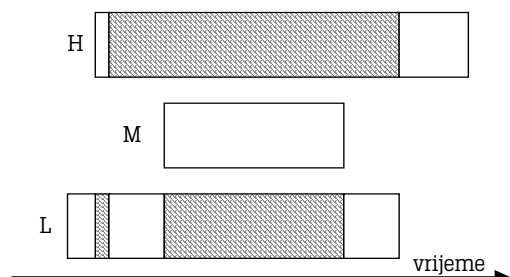
Na primjer neka u avionskom sistemu u realnom vremenu očitavamo atmosferski pritisak svake sekunde, a obrada tog posla traje 0,2 s. Pozicija se očitava svakih 0,5 s, a obrada traje 0,1 s. Brzina se očitava svakih 0,7 s, a obrada traje 0,1 s. Ove procese je moguće rasporediti jer je $0,2s/1s + 0,1s/0,5s + 0,1s/0,7s = 0.542 < 1$

3.24. Prioritetno raspoređivanje sa istiskivanjem

Umjesto statičkih tabela, raspoređivanje u sistemima realnog vremena može se prepustiti i operativnom sistemu. Prioritetni algoritam bez istiskivanja, opisan za *batch* sisteme se ne smije koristiti u sistemima realnog vremena, jer eventualni prioritetniji proces ne može prekinuti proces nižeg prioriteta koji je već zauzeo procesor. Nepogodno je i prioritetno raspoređivanje s postupnim prilagođavanjem prioriteta, namijenjeno za interaktivne sisteme, jer je trenutak pokretanja procesa teže predvidiv. Zato se koristi verzija prioritetnog raspoređivanja koja je analogna SRTN algoritmu, samo što se prioritet uzima umjesto preostalog vremena: izmjena procesa se dešava u trenutku pokretanja procesa pri praznom sistemu, završetka ili suspendovanja procesa ako ima procesa u čekanju, ali i trenutku pokretanja prioritetnijeg procesa.

U svemirskoj misiji *Mars Pathfinder* 1997, instaliran je operativni sistem u realnom vremenu VXWorks koji je koristio ovakvu vrstu raspoređivanja. Sistem je jednom prestao da radi zbog problema **prioritetne inverzije** koja se ponekad može desiti sa ovim raspoređivanjem.

Situacija prioritete inverzije može se npr. desiti kada imamo procese visokog (H), srednjeg (M) i niskog (L) prioriteta. Procesi H i L dijele isti resurs (Slika 31). Neka se prvo izvršavao L, i zauzeo resurs. Nakon toga se izvršava H, i istisne proces L. Kada H zatraži resurs, on mora biti blokiran jer je resurs zauzet. Zato on prepušta izvršavanje procesu nižeg prioriteta L da oslobodi resurs. Ali prije nego je L oslobodio resurs, pojavio se M i izbacio L. Najprioritetniji proces H mora čekati da se završe i M i L. Kada je konačno dobio svoj resurs, bilo je već kasno.



Slika 31: Prioritetna inverzija u RT sistemima

Rješenje ovog problema je nasljeđe prioriteta. Kada proces višeg prioriteta zatraži resurs koga drži proces nižeg prioriteta, procesu nižeg prioriteta se dodjeljuje maksimalni prioritet da ga ništa ne prekida dok ne oslobodi resurs koji treba procesu višeg prioriteta.

3.25.Algoritam najraniji rok prvo

U ovom algoritmu, procesi koji se mogu rasporediti se raspoređuju tako da se prije izvršavaju oni procesi čiji rok završetka dolazi ranije. Algoritam omogućava da se izvrši veći broj procesa u odnosu na statički definirane prioritete. Ali pri velikom opterećenju, skup procesa čiji uslovi izvršenja neće biti zadovoljeni je teško predvidljiv, što za sisteme u realnom vremenu može biti kobno.

Da se uporedi ponašanje ovog algoritma u odnosu na statičke prioritete, posmatraće se dva periodična procesa A i B, pri čemu A dolazi svakih 40 ms, i svaki odsječak traje po 30 ms, dok B dolazi svakih 100 ms i svaki odsječak traje 50 ms. Rokovi procesa su takvi da se obrada odsječka mora završiti prije narednog odsječka. Periodi i rokovi mogu se prikazati tabelom.

Proces	Vrijeme dolaska	Vrijeme izvršenja	Rok završetka
A1	0	20	40
A2	40	20	80
A3	80	20	120
A4	120	20	160
B1	0	50	100
B2	100	50	200
B3	200	50	300
...	...		

U trenutku 0 se bira proces A1, jer on ima raniji rok završetka. Slično, u trenutku $t=40$ ms bira se proces A2, kome je rok 80 ms i prekida proces B1 čiji je rok 100 ms, što je kasnije.

A1		B1		A2		B1			A3		B2	A4		B2				A5	
0	10	20	30	40	50	60	70	80	90	100	110	120	130	140	150	160	170	180	190

Prioritetno raspoređivanje sa istiskivanjem u ovom primjeru ne bi bilo izvodivo. Neka procesi tipa A imaju prioritet nad procesima tipa B. Proces B1 bi mogao započeti tek kada je A1 završen, u trenutku $t=20$ ms. U trenutku $t=40$ ms ponovo ga prekida A2, a kada se B1 nastavi u trenutku $t=60$ ms, izvršava se do trenutka $t=80$ ms kada ga prekida B3. Preostalih 10ms svog vremena proces B1 neće stići uraditi jer je u trenutku završetka procesa A3 rok istekao. Slično ovome, ako bi procesi tipa B imao prioritet, vidi se da proces A1 neće biti obrađen prije dolaska procesa A2.

3.26. Raspoređivanje monotonom stopom

Nešto jednostavniji kriterij od roka procesa je raspoređivanje bazirano prema periodu periodičnih poslova. Pri raspoređivanju se veći prioritet dodjeljuje onim procesima čija učestalost pojave je veća (tj. period je kraći). Npr., ako se događaji od senzora pozicije šalju svakih 10ms, a događaji od senzora temperature svakih 100 ms, uvijek će u raspoređivanju imati prednost obrada senzora pozicije.

Izvodivost ovog algoritma je manja od izvodivosti algoritma Najraniji rok prvo. Pokazano je da za n periodičnih procesa perioda T_i , od kojih svaki traje C_i vremena, a rokovi su jednaki periodu važi:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(\sqrt[n]{2} - 1)$$

Pošto je

$$\lim_{n \rightarrow \infty} n(\sqrt[n]{2} - 1) = \ln 2 \approx 0,69147$$

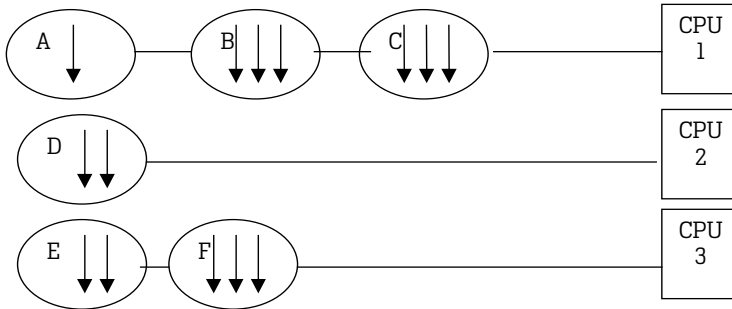
to znači da raspoređivanje ovim algoritmom u najgorem slučaju daje iskoristivost procesora od 69%. Preostalih 31% se može dodijeliti procesima koji nisu tipa realnog vremena.

3.27. Algoritmi za više-procesorske sisteme

Višeprocorski sistemi mogu kombinovati različite zahtjeve, npr. da neki procesori u njima koriste algoritme za interaktivne procese, a neki za pozadinske. Ponekad je moguće povećavati brzinu pojedinačnog procesa izvršavajući paralelno njegove niti na različitim procesorima, ali samo do određene granice. Kod ovih sistema javljaju se dileme načina iskorištenja procesora: koliko fizičkih procesora dodijeliti jednom procesu i koliko redova čekanja na procesore imati u sistemu. Dodavanje više procesora znatno poboljšava performanse računarskih sistema. Neka sistem koji se razmatra ima jednu memoriju i više procesora. Naravno, potrebne su određene promjene u algoritmima raspoređivanja za njih, jer je potrebno odabrati i procesor. Situacija je različita zavisno od toga da li se proces zajedno s

svim nitima cijeli dodjeljuje jednom od procesora ili se niti istog procesa dijele između procesora.

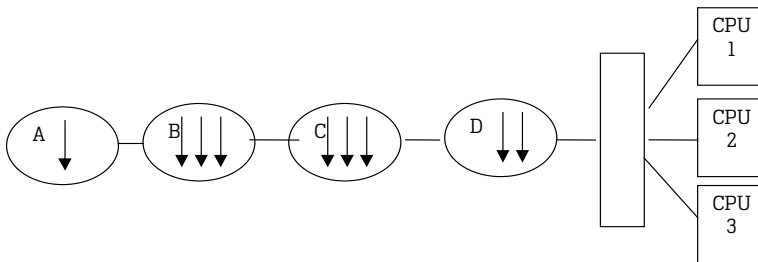
3.28.Particionirano raspoređivanje procesa



Slika 32: Particionirano raspoređivanje

U particioniranom raspoređivanju, proces zajedno sa svim nitima je trajno dodijeljen jednom procesoru od pokretanja do njegovog završetka. Namjenski kratkoročni red čekanja se održava za svaki procesor. Unutar reda čekanja za pojedini procesor koriste se algoritmi raspoređivanja namijenjeni jednom procesoru. Prednost ovog pristupa je da može biti manje posla oko raspoređivanja, jer je zadatak procesoru dodijeljen jednom i za sve vrijeme.

3.29.Globalno raspoređivanje procesa



Slika 33: Globalno raspoređivanje procesa

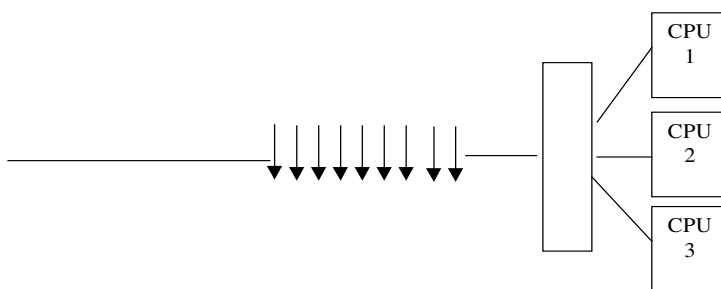
Nedostatak particioniranog dodjeljivanja je da jedan procesor može biti u stanju mirovanja, sa praznim redom čekanja, dok drugi procesor ima veći red čekanja. Da bi se spriječila ova situacija, može se koristiti zajednički red čekanja (Slika 33). Svi procesi idu u jedan globalni red i mogu dobiti bilo koji od procesora. Tako, za života procesa, proces može biti izvršen na različitim procesorima u različitim vremenima.

3.30.Podjela opterećenja

Višenitni procesi u višeprocorskom okruženju mogu se rasporediti da pripadaju različitim procesorima. Podjela opterećenja je pristup koji omogućuje ravnomjerno raspoređenje niti između različitih procesora. U zajedničkom redu čekanja uvezane su niti koje pripadaju različitim procesima i redom dodjeljuju nekom od procesora kada god on postaje slobodan.

Mogu se koristiti različiti algoritmi.

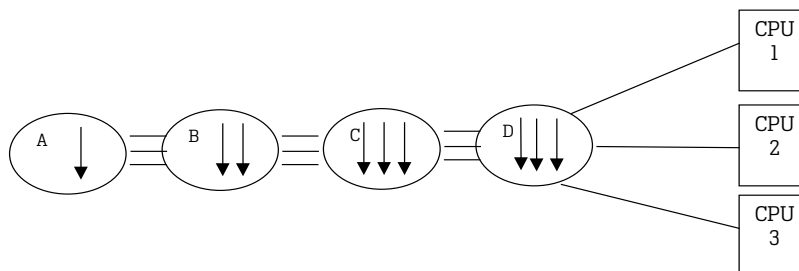
- FCFS. Svaki proces koji se izvršava kada dolazi u red čekanja u njega stavi sve svoje niti. Kako koji procesor postaje slobodan, dodjeljuje mu se naredna slobodna nit. Dodijeljena nit se nastavlja na datom procesoru do svog završetka.
- Prvo najmanji broj niti: Prioritet se daje procesima koji imaju manji broj neraspoređenih niti kako bi se brže pokrenuli takvi procesi. Dodijeljena nit se nastavlja na datom procesoru do svog završetka.
- Prvo najmanji broj niti sa istiskivanjem: Prioritet se daje procesima koji imaju manji broj neraspoređenih niti, ali se pri tome prekida neki proces koji je aktivirao više niti.



Slika 34: Podjela opterećenja

Jedna od mana ovog pristupa je što centralni red čekanja kome pristupaju različiti procesori zahtijeva kontrolu uzajamnim isključenjem, što predstavlja usko grlo. Drugi nedostatak je što blokirane niti nakon reaktiviranja obično ne nastavljaju izvršavanje na istom procesoru, čime njegov memorijski keš postaje manje efikasan. Treće, ako više niti pripadaju istom procesu, mala je vjerojatnoća da će se sve one izvršavati u isto vrijeme.

3.31. Družbeno raspoređivanje



Slika 35: Družbeno raspoređivanje

Različite niti od istog procesa u podjeli opterećenja ne moraju se izvršavati istovremeno, što pravi problem u njihovoj vremenskoj sinhronizaciji. Družbeno raspoređivanje (engl. *gang scheduling*) je način istovremenog raspoređivanja svih niti koje pripadaju istom procesu. Razlog za ovo je manje izmjena konteksta kada je

potrebna saradnja između niti i što veći stepen paralelizma unutar pojedinačnog procesa.

Pokušaće se rasporediti M procesa na N procesora, a svaki proces ima N ili manje niti. Procesi se mogu izvršavati npr. koristeći *Round Robin* algoritam, tako da svaki od njih dobija sve procesore. Uočava se da fiksni kvantum ne iskorištava najbolje sistem. Neka npr. imamo procese A, B, C i D koji redom imaju po 1, 2, 3 odnosno 3 niti. U *Round Robin* sa kvantomom 1, dobijamo sljedeće raspoređivanje.

CPU1	A1	B1	C1	D1	A1	B1	C1	D1	A1	B1	C1	D1
CPU2		B2	C2	D2		B2	C2	D2		B2	C2	D2
CPU3			C3	D3			C3	D3			C3	D3
t=	0	1	2	3	4	5	6	7	8	9	10	11

U ovom primjeru se vidi da je iskorištenje sistema 75%, jer se procesori CPU2 i CPU3 ne koriste svo vrijeme. Pametnim grupiranjem niti se može bolje iskoristiti sistem, kao u sljedećem redoslijedu.

CPU1	A1	C1	D1	A1	C1	D1	A1	C1	D1	A1	C1	D1
CPU2	B1	C2	D2	B1	C2	D2	B1	C2	D2	B1	C2	D2
CPU3	B2	C3	D3	B2	C3	D3	B2	C3	D3	B2	C3	D3
t=	0	1	2	3	4	5	6	7	8	9	10	11

3.32. Dodjela posvećenog procesora

U ekstremnoj varijanti društvenog raspoređivanja nakon što je nit dodijeljena procesoru, ona se izvršava na njemu sve do završetka aplikacije. Na prvi pogled, ovo je neracionalno. Ako se nit prebaci u stanje čekanja radi ulazno/izlaznih operacija ili sinhronizacije s drugom niti, tada njen procesor ostaje neiskorišten. Ali, ako imamo sistem velikog paralelizma, npr. više stotina procesora, iskorištenost procesora prestaje biti važna metrika performansi sistema. Zadržavanjem niti na istom procesoru smanjuje se broj izmjena konteksta, što takođe ubrzava proces.

Konačno, broj niti koje čine proces svakako ima svoju gornju granicu, pa se može proračunati da li je veća dobit od čuvanja niti na istom procesoru ili korištenju tog procesora za različite niti. Ta granica je **Amdalov zakon**. On kaže da ako se program može realizovati tako da se $p\%$ programa mora izvršiti sekvencijalno, a $(100-p)\%$ programa može paralelizovati, onda se ukupno vrijeme izvršenja ne može paralelizacijom smanjiti na manje od $p\%$. Na primjer, neka od 10 sati izvršenja programa na jedno-procesorskom računar, 1 sat otpada na sekvencijalni dio koda. Sa 2 procesora, posao se može obaviti za $1 + 9/2 = 5,5$ sati, sa 5 procesora za $1 + 9/5 = 2,8$ sati. No daljnja ubrzanja nisu velika. Sa 100 procesora proces se obavi za 1,09 sati, sa 1000 procesora za 1,009 sati.

3.33. Dinamičko raspoređivanje

Nekada je praktičnije odluku o dodjeli procesora pojedinim nitima prepustiti aplikaciji, jer njen programer bolje zna od operativnog sistema koje niti bi se trebale takmičiti za koji procesor. Na primjer, niti koje se aktiviraju na ulazno/izlazne operacije se mogu sve dodijeliti istom procesoru, ali je niti koje učestvuju u inverziji velike matrice bolje rasporediti na različite procesore. Zato mnogi operativni sistemi imaju systemske pozive kojim se niti dodijele preporučeni procesori i tako zaobiđu u toku rada dodijeljeni procesori od strane operativnog sistema. Tako Windows ima

poziv **SetThreadAffinityMask**, kojim se može uticati na preporučeni skup procesora koji mogu da izvršavaju pojedinu nit, koristeći binarni broj kao parametar u kome svaki bit odgovara jednom od procesora.

3.34.POSIX funkcije koje podešavaju algoritme raspoređivanja

Neke funkcije koje utiču na algoritam raspoređivanja procesa su date na sljedećoj listi. U Unix sistemima manji broj prioriteta znači veći prioritet.

Funkcija	Značenje
getpriority(which, who)	Očitava trenutni prioritet procesa
setpriority(which, who, int prio)	Postavlja prioritet procesa
nice(ilnc)	Proces pridodaje dio prioriteta samom sebi

Prvi parametar, *which*, u pozivima *setpriority* i *getpriority* može biti `PRIO_PROCESS` (proces), `PRIO_PGRP` (svi procesi grupe korisnika) i `PRIO_USER` (svi procesi jednog korisnika) određuje čega je identifikator drugi parametar. Sljedeći promjer povećava prioritet procesa na najveći prioritet (-20)

```
#include <sys/resource.h>
int main(void) {
    int ret= setpriority(PRIO_PROCESS, getpid(), -20);
}
```

3.35.Windows funkcije koje podešavaju algoritme raspoređivanja

Sljedeće Windows API funkcije mogu da mijenjaju ili očitavaju parametre algoritma raspoređivanja. U Windows sistemima manji broj prioriteta znači manji prioritet, pri čemu je najviši prioritet 15, a najniži -15.

Funkcija	Značenje
GetCurrentProcessorNumber()	Vraća redni broj procesora koji izvršava proces
GetPriorityClass(hProcess)	Vraća kategoriju prioriteta procesa
GetProcessTimes(hProcess,lpCreationTime, lpExitTime, lpKernelTime,lpUserTime)	Vraća koliko je vremena proces proveo u pojedinim režimima OS
SetPriorityClass(hProcess,dwPriorityClass)	Postavlja klasu prioriteta procesa
GetThreadPriority(hThread)	Vraća prioritet niti
SetThreadPriority(hThread,nPriority)	Postavlja prioritet niti

Sljedeći primjer mijenja prioritet trenutnog procesa u `NORMAL_PRIORITY_CLASS`. Sa ovom vrstom prioriteta Windows koristi raspoređivanje s višestrukim redovima sa povratnom spregom. Ako se klasa promijeni na `REALTIME_PRIORITY_CLASS`, prelazi se na prioritetno raspoređivanje sa istiskivanjem. Rezultat je gotovo potpuno zaglavljivanje na jednoprocorskim sistemima.

```
#include <windows.h>
#include <stdio.h>
int main( void ) {
    DWORD dwError, dwPriClass;
    SetPriorityClass(GetCurrentProcess(),NORMAL_PRIORITY_CLASS);
    dwPriClass = GetPriorityClass(GetCurrentProcess());
    printf("Trenutna klasa prioriteta je 0x%x\n", dwPriClass);
    while (1) {};
    return 0;
}
```

3.36.Zadaci

Zadatak 1.

Neka su ubačena u sistem četiri procesa s sljedećim vremenima dolaska, trajanja, prioritetima (manji broj veći prioritet).

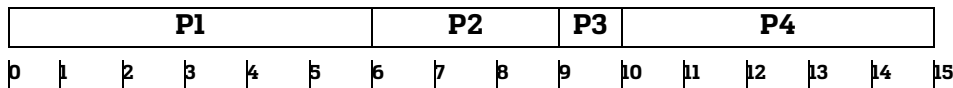
Proces	Vrijeme dolaska	Trajanje	Prioritet
P1	0	6	2
P2	2	3	1
P3	3	1	2
P4	7	5	1

Treba ih rasporediti u jednoprocesorskom sistemu. Ne uzimajući u obzir vrijeme izmjene konteksta, izračunati srednje vrijeme čekanja za algoritme a) FCFS, b) SJF, c) SRTF, d) Prioritetni bez istiskivanja e) Prioritetni sa istiskivanjem f) *Round Robin* $q=2$ g) *Round Robin*, $q=2$ s višestrukim prioritetima. Nacrtati Ganttove dijagrame izvršavanja procesa. Ako dva procesa imaju isti osnovni kriterij raspoređivanja, odabrati proces s manjim rednim brojem.

Rješenje:

a) FCFS

Kod ovog algoritma procesi će se izvršiti onim redom kojim su pristigli. Ganttov dijagram izgleda ovako.

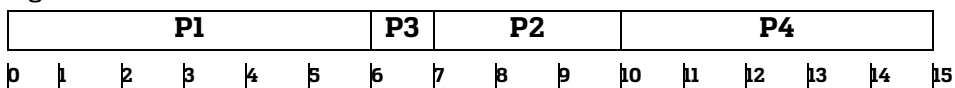


Proces	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	0	6	6	$6-0=6$	$6-6=0$
P2	2	3	9	$9-2=7$	$7-3=4$
P3	3	1	10	$10-3=7$	$7-1=6$
P4	7	5	15	$15-7=8$	$8-5=3$

Srednje vrijeme čekanja je $(0+4+6+3)/4=3,25$

b) SJF

Kod ovog algoritma novi proces ne prekida trenutno izvršavani, ali nakon njegovog završetka bira se najkraći od u međuvremenu pristiglih. Ganttov dijagram izgleda ovako.

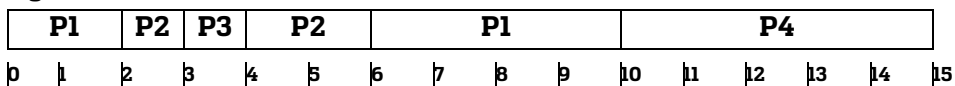


Proces	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	0	6	6	6-0=6	6-6=0
P2	2	3	10	10-2=8	8-3=5
P3	3	1	7	7-3=4	4-1=3
P4	7	5	15	15-7=8	8-5=3

Srednje vrijeme čekanja je $(0+5+3+3)/4=2,75$

c) SRTF

Kod ovog algoritma novi proces prekida trenutno izvršavani, ako mu je vrijeme izvršavanja kraće od preostalog vremena trenutnog procesa. Ganttov dijagram izgleda ovako.

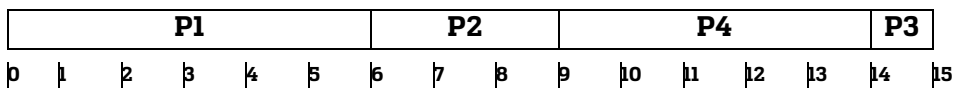


Proces	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	0	6	10	10-0=10	10-6=4
P2	2	3	6	6-2=4	4-3=1
P3	3	1	4	4-3=1	1-1=0
P4	7	5	15	15-7=8	8-5=3

Srednje vrijeme čekanja je $(4+1+0+3)/4=2$

d) Prioritetni bez istiskivanja.

Kod ovog algoritma novi prioritetniji proces ne prekida trenutno izvršavani, ali se nakon njegovog završetka bira najkraći od u međuvremenu pristiglih. Ganttov dijagram izgleda ovako.



Proces	Prioritet	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	2	0	6	6	6-0=6	6-6=0
P2	1	2	3	9	9-2=7	7-3=4
P3	2	3	1	15	15-3=12	12-1=11
P4	1	7	5	14	14-7=7	7-5=2

Srednje vrijeme čekanja je $(0+4+11+2)/4=4,25$

e) Prioritetni sa istiskivanjem

Kod ovog algoritma novi proces prekida trenutno izvršavani, ako je prioritetniji od trenutnog procesa. Ganttov dijagram izgleda ovako.

P1		P2			P1		P4					P1		P3	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Proces	Prioritet	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	2	0	6	14	14-0=14	14-6=8
P2	1	2	3	5	5-2=3	3-3=0
P3	2	3	1	15	15-3=12	12-1=11
P4	1	7	5	12	12-7=5	5-5=0

Srednje vrijeme čekanja je $(8+0+11+0)/4=4,75$

f) Round Robin

Stanje redova čekanja dato je u sljedećoj tabeli.

Trenutak	Izvršava se	Red čekanja
0	P1	-
2	P1	P2
3	P1	P2-P3
4	P2	P3-P1
6	P3	P1-P2
7	P1	P2-P4
9	P2	P4
10	P4	-
12	P4	-
14	P4	-
15	-	-

Ganttov dijagram izgleda ovako.

P1				P2		P3	P1	P2	P4						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Proces	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	0	6	9	9-0=9	9-6=3
P2	2	3	10	10-2=8	8-3=5
P3	3	1	7	7-3=4	4-1=3
P4	7	5	15	15-7=8	8-5=3

Srednje vrijeme čekanja je $(3+5+3+3)/4=3,5$

g) Round Robin s višestrukim redovima prioriteta

P1		P2			P1		P4					P3		P1	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Trenutak	Izvršava se	Red čekanja	Blokirani red			
0	P1	-	-			
2	P2		P1			
3	P2		P1-P3			
4	P2		P1-P3			
5	P1	P3				
7	P4		P3-P1			
9	P4		P3-P1			
11	P4		P3-P1			
12	P3	-				
13	P1	-				
15	-					
Proces	Prioritet	Vrijeme dolaska	Trajanje	Vrijeme završetka	Vrijeme u sistemu	Vrijeme čekanja
P1	2	0	6	15	15-0=15	15-6=9
P2	1	2	3	5	5-2=3	3-3=0
P3	2	3	1	13	13-3=10	10-1=9
P4	1	7	5	12	12-7=5	5-5=0

Srednje vrijeme čekanja je $(9+0+9+0)/4=4,5$

Zadatak 2. Neka u lutrijskom raspoređivanju postoji 9 procesa tipa A kojima je dodijeljeno po 2 tiketa i 10 procesa tipa B kojima je dodijeljeno po 8 tiketa. Koliko procentualno vremena se izvršava jedan proces tipa A i jedan proces tipa B?

Rješenje:

Ukupno ima $9 \times 2 + 10 \times 8 = 98$ tiketa. Statistički gledano, proces tipa A bi se trebao izvršavati $2/98 = 2,04\%$ vremena, a proces tipa B $8/98 = 8,16\%$ vremena.

Zadatak 3. Neka u avionskom sistemu u realnom vremenu očitavamo atmosferski pritisak svake sekunde, a obrada tog posla traje 0,3 s. Pozicija se očitava svakih 0,5 s, a obrada traje 0,1 s. Brzina se očitava svakih 0,7 s, a obrada traje 0,1s. Može li se obaviti raspoređivanje monotonom stopom?

Rješenje:

$0,3s/1s + 0,1s/0,5s + 0,1s/0,7s = 0.642 < 1$. Dakle, može se rasporediti monotonom stopom

Zadatak 4. Neka u sistemu u realnom vremenu proces P1 ima period signala 8s i vrijeme obrade od 1s, proces P2 period 5s i vrijeme obrade 2s, proces P3 period 10s i vrijeme obrade 2s. Rasporediti procese algoritmima Najraniji rok prvo i Monotona stopa. Ovo su trenuci dolaska:

P1a P2a P3a					P2b			P1b		P2c P3b					P2d	P1c			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Rješenje:

U algoritmu Najraniji rok prvo se bira proces čija sljedeća perioda dolazi najranije, jer do tog trenutka se mora obraditi podatak. Ovo je Ganttov dijagram.

P2a		P1a	P3a			P2b			P1b		P2c		P3b			P2d		P1c		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	

b) U algoritmu Monotona stopa prvo bira se proces **čiji period dolaska** je najkraći.

Pla		P2a		P3a		P2b			Plb			P2c		P3b			P2d		Plc			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19			

Zadatak 5: U četveroprocesorskom sistemu svi procesori su slobodni u trenutku 0. U čekanju su proces P1 koji ima 3 niti i traje 5 vremenskih jedinica, proces P2 sa 1 niti i traje 2 vremenske jedinice i proces P3 koji ima 2 niti i traje 3 vremenske jedinice. Rasporediti procese algoritmima a) najmanji broj niti prvo b) društveno raspoređivanje sa Round robin raspodjelom, quantum=2

Rješenje:

a) Najmanji broj niti prvo

CPU1	P2	P2	P1	P1	P1	P1	P1	
CPU2	P3	P3	P3	P1	P1	P1	P1	P1
CPU3	P3	P3	P3					
CPU4	P1	P1	P1	P1	P1			
T=	0	1	2	3	4	5	6	7

b) Društveno raspoređivanje, RR q=2

CPU1	P1	P1	P3	P3	P1	P1	P3	P1
CPU2	P1	P1	P3	P3	P1	P1	P3	P1
CPU3	P1	P1			P1	P1		P1
CPU4	P2	P2						
T=	0	1	2	3	4	5	6	7

4.SINHRONIZACIJA PROCESA

4.1.Dijeljenje i natjecanje

Bez obzira da li je riječ o nitima unutar jednog procesa, odvojenim procesima unutar jednog operativnih sistema ili pristupu zajedničkoj datoteci kroz mrežu odvojenih računara, procesi i niti često dijele resurse. Kada se više procesa izvršava istovremeno u sistemu, a dijele neki resurs, mogući su konflikti i dosta nepredvidljivo ponašanje programa. Kao primjer, uzeće se program koji u dvije POSIX niti puni niz podacima, konkretno upisuje vrijednost indeksa niza u element niza.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int brojevi[2000];
int brojac=0;
void dodaj() {
    int poz;
    poz=brojac;
    brojevi[poz]=poz;
    brojac++;
    usleep(2);
}
void * duzi(){
    int i;
    for(i=0;i<1500;i++)
        dodaj();
}
void * kraci() {
    int i;
    for(i=0;i<500;i++)
        dodaj();
}
int main() {
    int s,i;
    pthread_t t1, t2;
    pthread_create( &t1, NULL, &duzi, NULL);
    pthread_create( &t2, NULL, &kraci, NULL);
    pthread_join( t1, NULL);
    pthread_join( t2, NULL);
    s=0;
    for(i=0;i<2000;i++)
        s=s+brojevi[i];
    printf ("%d \n",s);
    return 0;
}
```

Jedna nit poziva potprogram koji se zove duzi(), a druga nit poziva potprogram koji se zove kraci(). Obije niti pozivaju potprogram dodaj(), pri čemu jedna nit puni 1500 elemenata, a druga 500 elemenata jedan po jedan. Potprogram dodaj() čita vrijednost globalne varijable brojac i upisuje vrijednost te varijable na zadatu poziciju. Kada se niti završe, petljom na kraju main() funkcije sumira se sadržaj niza s ubačenim podacima. U datom primjeru prikazana suma bi trebala biti 1999000,

što se računa prostom formulom za sumu aritmetičke progresije. Međutim, kada se program na nekim Unix ili Linux sistemima više puta pokrene, javljaju se i druge vrijednosti, što znači da je neki od podataka pogrešno ubačen u niz.

Pažljivi pogled na program otkriva da, ako se funkcije `duzi()` i `kraci()` izvršavaju istovremeno unutar pokrenutih niti, postaje moguć sljedeći scenarij. Neka je potprogram `duzi()` pozvao potprogram `dodaj()` i u izvršenju došao do linije `poz=brojac;`. Tamo je pročitana vrijednost varijable `brojac` i u tom trenutku se desila izmjena između niti. Programski brojčalo pređe na izvršavanje potprograma `kraci()` koji također poziva potprogram `dodaj()`. Ako se i on u tom trenutku nastavi izvršavati na svojoj liniji `poz=brojac;` sada obje niti imaju jednaku vrijednost varijable `brojac` u svojim lokalnim varijablama `poz` i jedna će vrijednost prebrisati drugu.

Da se izbjegnu ovakve situacije moraju se uočiti dijelovi programa u kojima samo jedan od procesa ili niti smije pristupa dijeljenim podacima. Takva disciplina pisanja programa se zove međusobno isključivanje.

4.2.Kritična sekcija

Dio programskog koda u kome se pristupa resursima tako da je potrebno međusobno isključivanje naziva se **kritična sekcija**. U prethodnom primjeru kritična sekcija i u potprogramu `duzi()` i u potprogramu `kraci()` je dio programa koji se izvršava od poziva potprograma `dodaj()` do izlaska iz tog potprograma. Nekada, čak i jedna linija programa može predstavljati kritičnu sekciju, pošto se jedna instrukcija u jeziku visokog nivoa realizuje sa više instrukcija u mašinskom jeziku. Ukoliko se dešava da dva procesa ulaze u kritičnu sekciju u isto vrijeme tada se to stanje naziva stanjem natjecanja. Dobro rješenje problema kritične sekcije treba da zadovoljava uslove:

1. U kritičnoj sekciji se smije izvršavati najviše jedan proces ili nit.
2. Osiguranje uzajamnog isključivanja treba da radi i za jednoprocorske i višeprocorske sisteme i ne smije se zasnivati na vremenu boravka u kritičnoj sekciji,
3. Proces koji se izvršava izvan kritične sekcije ne smije da blokira druge procese
4. Proces ne smije beskonačno čekati da bi ušao u kritičnu sekciju
5. Proces se ne smije ni beskonačno zadržati u kritičnoj sekciji kako ne bi blokirao druge procese.

4.3.Načini rješenja problema međusobnog isključivanja

Sada će se na primjeru gore spomenutog programa koji preko dvije niti ubacuje podatke u niz, razmotriti neka rješenja problema kritične sekcije. Neka od rješenja su ispravna, a neka nisu, ali će biti navedena jer pokazuju put kako se došlo do ispravnih rješenja. U ovom dijelu će biti prikazane različite metode za postizanje međusobnog isključivanja procesa. Time, dok jedan proces ažurira dijeljenu memoriju u svojoj kritičnoj sekciji, drugi procesi neće moći ulaziti u istu kritičnu sekciju i smetati mu.

4.4.Dijeljena varijabla

Prva ideja je da se uvede jedna zajednička varijabla koja je u početku ima vrijednost 0. Kada proces želi ući u kritičnu sekciju on testira tu varijablu. Ako je ona jednaka 0, proces je postavi na vrijednost 1 i nastavlja sa izvršavanjem kritične sekcije. Ako je dijeljena varijabla već imala vrijednost 1, proces mora da čeka sve dok ona ne postane jednaka 0. Efektivno, vrijednost 0 znači da nijedan proces nije u kritičnoj sekciji, a vrijednost 1 znači da ima proces u kritičnoj sekciji.

```
int lock=0;
void * duzi() {
    int i;
    for(i=0;i<1500;i++) {
        while (lock==1) {};
        lock=1;
        dodaj();
        lock=0;
    }
}
```

```
void * kraci() {
    int i,poz;
    for(i=0;i<500;i++) {
        while (lock==1) {};
        lock=1;
        dodaj();
        lock=0;
    }
}
```

Ovo rješenje, međutim, je samo smanjilo vjerovatnoću konflikta, ali nije riješilo problem uzajamnog isključivanja. Neka jedna nit pročita vrijednost varijable *lock* i vidi da je ona jednaka 0. Izađe se iz petlje, ali prije nego što je postavi na 1, raspoređivač pređe na izvršenje druge niti koja uoči da je *lock*=0. Zbog nezgodnog trenutka raspoređivanja obje niti su u kritičnoj sekciji i opisani konflikti su opet mogući. Prvi uslov dobro riješene kritične sekcije nije zadovoljen.

4.5.Striktna promjena

Problem rješenja s dijeljenom varijablom je bio što dvije niti čitaju istu vrijednost varijable *lock* u nekom posebnom trenutku i ulaze u kritičnu sekciju. Šta bi bilo ako se umjesto toga uvede pravilo da svaka nit ulazi u kritičnu sekciju samo kada je na nju red, poput poteza u šahu ili izmjene željezničkih skretnica. Sljedeći primjer koristi varijablu *potez* da se čeka ako je red na drugu nit u kritičnoj sekciji, tako da se u potprogramu *duzi()* čeka sve dok varijabla *potez* ne dobije vrijednost 0. Po završetku kritične sekcije *potez* se daje niti koja izvršava potprogram *kraci()*.

```
int potez=0;
void * duzi() {
    int i,poz;
    for(i=0;i<1500;i++) {
        while (potez!=0) {};
        dodaj();
        potez=1;
    }
}
```

```
void * kraci() {
    int i,poz;
    for(i=0;i<500;i++) {
        while (potez!=1) {};
        dodaj();
        potez=0;
    }
}
```

Ovo rješenje ne radi. Program se zaglavi kada se nit koja izvršava potprogram *kraci()* završi. Program bi radio samo kada bi oba potprograma bila jednako duga (npr. da petlje u oba potprograma imaju po 1000 iteracija). Po završetku potprograma *kraci()* potprogram *duzi()* više neće dobiti priliku da varijabla *potez* dobije vrijednost 0 nakon što ju je *duzi()* postavio na 1. Treći uslov dobro riješene kritične sekcije nije zadovoljen.

4.6. Peterson-ovo rješenje

Iako nijedno od prethodna dva rješenja nije zadovoljavajuće, kombinacijom njihovog pristupa došlo se do nekoliko drugih algoritama kao što su Dekkerov, Petersonov i pekarski algoritam koji problem ispravno rješavaju. U Petersonovom rješenju kritične sekcije sve niti ili procesi koji učestvuju u kritičnoj sekciji imaju vlastito rezervirano mjesto za najavu unutar niza u kome se navodi da se želi učestvovati u kritičnoj sekciji. Nit najavljuje da želi pristupiti u kritičnu sekciju upisom logičke jedinice na svoje mjesto u nizu najave i upisom svog rednog broja u varijablu potez. Samo varijabla potez je dijeljena između obje niti. U petlji se čeka ako je potez na toj niti, ali je i druga nit najavila ulazak u kritičnu sekciju. Po izlasku iz kritične sekcije, u niz najave se stavlja vrijednost logičke nule.

```
#define TRUE 1
#define FALSE 0
int potez=1,najava[2]={0,0};
void * duzi() {
    int i,poz;
    for(i=0;i<1500;i++) {
        najava[0]=TRUE;
        potez=0;
        while(potez==0 && najava[1]) {};
        dodaj();
        najava[0]=FALSE;
    }
}
```

```
void * kraci() {
    int i,poz;
    for(i=0;i<500;i++) {
        najava[1]=TRUE;
        potez=1;
        while (potez==1 &&najava[0]) {};
        dodaj();
        najava[1]=FALSE;
    }
}
```

Ovo rješenje zadovoljava uslove kritične sekcije. Ako se slučajno desi izmjena konteksta u trenutku kada prva nit izvršava `potez=0` i pređe na liniju `potez=1` u drugoj niti varijabla `potez` će biti prebrisana, ali će se druga nit tada zaustaviti u `while` petlji jer je `najava[0]` u toj situaciji jednaka `TRUE`. Algoritam je znatno komplikovaniji u varijanti kada učestvuje više procesa ili niti.

4.7. Zabrana prekida

Neka rješenja se zasnivaju na specifičnostima instrukcija koje centralni procesor pruža, pa se mogu realizovati samo na nekim računarskim sistemima. Glavni razlog stanja natjecanja je izmjena konteksta procesa ili niti pod uticajem prekida sata realnog vremena. Ako se taj prekid zabrani, dobije se jednostavno rješenje problema kritične sekcije. Na Intelovim procesorima, prekidi se zabranjuju naredbom `CLI`, a dopuštaju naredbom `STI`. Šta bi se desilo da se one ubace u kritičnu sekciju?

```
void * duzi() {
    int i;
    for(i=0;i<1500;i++) {
        asm ("cli");
        dodaj();
        asm ("sti");
    }
}
```

```
void * kraci() {
    int i;
    for(i=0;i<500;i++) {
        asm ("cli");
        dodaj();
        asm("sti");
    }
}
```

Naredba `asm` je proširenje `gcc` kompajlera koja omogućava ubacivanje asemblerskih instrukcija u C program. Eksperiment s ovim rješenjem u Linux-u dovodi do fatalne greške završetka programa. Korisnički programi u većini operativnih sistema koji imaju zaštitu nemaju pravo da isključuju prekide, jer bi to

potencijalno zaustavilo odziv cijelog operativnog sistema. Pored toga, u višeprocorskim sistemima naredba za isključenje prekida se odnosi samo na jedan procesor. Ovo rješenje ne zadovoljava uslov broj 2 za kritičnu sekciju. Ponekad se, ipak koristi na jednoprocorskim operativnim sistemima isključivo u kratkim kritičnim sekcijama u jezgu.

4.8.Instrukcije testiranja i postavljanja

Dizajn svih mikroprocesora je da se prekidi (*interrupt*) ne započinju prije nego se izvrši trenutna instrukcija na procesoru bez obzira što složenost instrukcija varira, od instrukcije koja ne radi ništa do instrukcija koje računaju transcendentne funkcije. Pokazalo se da bi instrukcija, koja u jednoj operaciji zapamti raniju vrijednost memorijske lokacije i postavi je na vrijednost 1, bila dovoljna za realizaciju petlje čekanja u kritičnoj sekciji. Takva instrukcija u literaturi o operativnim sistemima se zove TSL (engl. *Test and Set Lock*). U višeprocorskim sistemima, jedan od procesora izvršava TSL instrukciju i zaključa memorijsku sabirnicu zabranivši pristupe memoriji drugim procesorima dok se instrukcija ne završi. Na Intelovim procesorima ovakva instrukcija se zove BTS, (*bit test and set*). Jedan od parametara ove instrukcije je registar ili memorijska lokacija koja se testira, a drugi predstavlja redni broj bita koji se testira i postavlja na 1 na tom registru ili lokaciji. Rezultat testiranja se prebaci u indikator CF koji se nakon toga koristi u uslovnom skoku. Ako se ovoj instrukciji doda prefiks LOCK, tada je i sabirnica zaključana u višeprocorskim sistemima.

Sada će se napraviti dva sinkronizacijska potprograma. Potprogram `sacekajlock()` izvršava petlju napisanu u asemblerskom jeziku. Ona ima samo dvije instrukcije. Prva instrukcija kopira u CF najmanje značajan bit varijable `lock1`, a druga skače na prethodnu instrukciju ako je CF (a time i raniji najmanje značajan bit varijable `lock1`) bio jednak 1. Time će `sacekajlock` čekati u petlji sve dok neko ne pozove potprogram `oslobodilock()` koji će postaviti sve bite varijable `lock1` na nulu.

```
int lock1=0;
```

```
void oslobodilock() {
    lock1=0;
}
```

```
void sacekajlock() {
    asm ( "wait1: ;"
        " LOCK BTS $0,lock1 ;"
        " JC wait1;" );
}
```

Verzije potprograma `duzi()` i `kraci()` koristeći sinhronizaciju pomoću TSL instrukcije sada izgledaju ovako.

```
void * duzi() {
    for(int i=0;i<1500;i++) {
        sacekajlock();
        dodaj();
        oslobodilock();
    }
}
```

```
void * kraci() {
    for(int i=0;i<500;i++) {
        sacekajlock();
        dodaj();
        oslobodilock();
    }
}
```

Ako procesor nema ovakvu instrukciju, alternativno se mogu koristiti instrukcije koje zamjenjuju vrijednosti memorijskih lokacija ili lokacije i registra. Rješenje sa TSL instrukcijom zadovoljava sve uslove kritične sekcije.

4.9.Semafori sa zaposlenim čekanjem

Uzajamno isključivanje je specijalni slučaj limitiranja broja procesa koji mogu ući u kritičnu sekciju pa je E. W. Dijkstra (1965) uveo specijalne cjelobrojne varijable koje se zovu semafori i koje između ostalog mogu da čuvaju taj broj. Nad semaforima su pored inicijalizacije dopuštene samo dvije operacije: *wait* i *signal*. Pojednostavljeno, one rade sljedeće:

```
void wait(semaphore * sem) {
    while (*sem<=0) {}
    (*sem) --;
}
```

```
void signal(int * sem)
{
    (*sem) ++;
}
```

Prva operacija (u literaturi poznata kao *P*, *wait* ili *down*) čeka u petlji ako semafor nema pozitivnu vrijednost, a zatim umanjuje vrijednost semafora za 1. Ona se poziva prije ulaska u kritičnu sekciju. Druga operacija (*V*, *signal* ili *up*) uvećava semafor za 1 i poziva se nakon kritične sekcije. Po definiciji, ove operacije se moraju desiti atomski. To znači da ne smije biti uslova natjecanja u operaciji nad semaforom. Zato se u samim operacijama semafora mora obaviti zaključavanje, npr. TSL instrukcijom ili (ako je moguće) isključenjem prekida dok se analizira varijabla *sem*. Uz dodatak zaključavanja, ove operacije izgledaju ovako.

```
typedef int semaphore;
void wait(semaphore * sem) {
    while (1) {
        sacekajlock();
        if (*sem>0) {
            (*sem) --;
            oslobodilock();
            return;
        }
        else {
            oslobodilock();
            continue;
        }
    }
}
```

```
void signal(int * sem)
{
    sacekajlock();
    (*sem) ++;
    oslobodilock();
}
```

Nakon što su uvedene operacije nad semaforima, verzije potprograma *duzi()* i *kraci()* koristeći sinhronizaciju pomoću semafora sada izgledaju ovako.

```
semaphore sem=1;
void * duzi() {
    for(int i=0;i<1500;i++) {
        wait(&sem);
        dodaj();
        signal(&sem);
    }
}
```

```
void * kraci() {
    for(int i<500;i++) {
        wait(&sem);
        dodaj();
        signal(&sem);
    }
}
```

Uzajamno isključivanje sa semaforima postiže se postavljanjem poziva *wait* nad semaforom prije kritične sekcije a *signal* nakon nje.

Prednost semafora nad Petersonovim rješenjem je što će se uz pomoć semafora na isti način riješiti međusobno isključivanje kada je potrebno međusobno isključivati dvije niti na ulazu u kritičnu sekciju ili kad ih je veći broj. Drugo, semafori se mogu koristiti ne samo za međusobno isključivanje, nego i za postizanje

redosljeda, da jedan proces ili nit čeka da drugi završi neki događaj, kao i za situacije kada nije potrebno međusobno isključivanje nego ograničavanje broja procesa (npr. maksimalno 8 procesa) koji izvršavaju jedan dio koda.

Rješenje sa semaforima ispunjava uslove kritične sekcije. Kada se ovo rješenje isproba vidi se da je ono prilično sporo. Pored čekanja cijelog semafora da postane pozitivne vrijednosti, čeka se i na zaključavanje i otključavanje pri svakom pristupu semaforskoj varijabli. Zapravo i ostala do sada predložena rješenja su spora. Kod svih rješenja, osim isključenja prekida, kada proces želi da uđe u kritičnu sekciju, on provjerava da vidi da li je ulaz dozvoljen. Ako nije, proces se vrti u petlji i čeka svoj ulazak.

Ako je raspoređivač tipa prioritetni (sa ili bez istiskivanja) ili *Round robin* sa prioritetima, javlja se i problem prioritetne inverzije. Neka je u toj situaciji proces nižeg prioriteta ušao u kritičnu sekciju i još nije izašao iz nje. Raspoređivač je prebacio izvršenje na proces višeg prioriteta koji ulazi u čekanje na kritičnu sekciju. Sada se proces višeg prioriteta vrti u petlji čekajući da dobije ulaz, koga neće dobiti nikada, jer ga proces nižeg prioriteta ne može osloboditi, jer zbog algoritma raspoređivanja ne može doći na red.

4.10. Semafori sa blokiranjem i deblokiranjem procesa

Problem sporosti semafora zbog zaposlenog čekanja se može riješiti ako se operacije *wait* i *signal* povežu sa raspoređivačem. Umjesto da se nepotrebno troši vrijeme čekajući u petlji na oslobađanje kritične sekcije, proces ili nit koji mora čekati se izbaci iz reda spremnih procesa u raspoređivaču procesa ili niti i prebaci u jedan red čekanja vezan za semafor. Ovakvi semafori nisu više proste cjelobrojne varijable, nego strukture koje se sastoje iz vrijednosti semafora i pokazivača na listu kontrolnih blokova procesa ili niti. Moguća programska struktura za opis semafora izgleda ovako.

```
typedef struct
{
    int value;
    struct PCB * Q;
} semaphore;
```

Za razliku od semafora sa zaposlenim čekanjem, semafori sa blokiranjem procesa mogu imati i negativne vrijednosti u polju *value*. Pozitivna vrijednost semafora znači da je semafor otvoren za još toliko poziva *wait* operacije. Vrijednost 0 znači da je semafor u zatvorenom stanju, ali ni jedan proces nije blokiran. Negativna vrijednost semafora $-x$ znači da je x procesa trenutno u blokiranom stanju.

Neka postoje zamišljene funkcije: *DisableSwitching*, koja privremeno isključuje izmjenu poslova u raspoređivaču, *EnableSwitching* koji uključuje izmjenu, *GetCurrentProcess* koja vraća pokazivač na trenutno izvršavani proces, *AddQueue* koja ubacuje pokazivač na kontrolni blok procesa ili niti u listu, *RemoveFirstFromQueue* koja izbacuje prvi takav pokazivač iz liste i vraća njegovu vrijednost, *block*, koja blokira proces ili nit i *wakeup* koja reaktivira blokirani proces.

Osnovne tri operacije nad semaforima sa blokiranjem izgledaju približno ovako.

```

InitSemaphore(semaphore * Sem,int
val) {
    DisableSwitching();
    Sem->value = val;
    Sem->Q = empty;
    EnableSwitching();
}

Wait(semaphore * Sem) {
    PCB * P;
    DisableSwitching();
    P=GetCurrentProcess();
    Sem->value--;
    if (Sem->value < 0) {
        AddQueue (Sem->Q,P);
        block(P);
    }
    EnableSwitching();
}

```

```

Signal(semaphore * Sem) {
    PCB * P;
    DisableSwitching();
    Sem->value++;
    if (Sem->value <= 0) {
        P=RemoveFirstFromQueue(Sem->Q);
        wakeup(P);
    }
    EnableSwitching();
}

```

Način korištenja je isti kao i kod semafora sa zaposlenim čekanjem, ali su rezultati daleko bolji kada se koristi semafor sa blokiranjem i deblokiranjem procesa.

4.11.Mutex-i

Specijalna verzija semafora koja dopušta vrijednosti 0 i 1 zove se *mutex*. Naziv potiče od riječi *mutual exclusion* što znači uzajamno isključivanje. Vrijednosti 0 i 1 odgovaraju stanjima zaključanog ili otključanog *mutex*-a. Kada proces ili nit želi da pristupi kritičnoj sekciji onda on koristi funkciju `lock()`. Ukoliko je tada *mutex* otključan, pristup kritičnoj sekciji je tada dozvoljen. Iz funkcije se uspješno vraća i proces ili nit može ući u kritičnu sekciju. Ako je *mutex* zaključan, pristup kritičnoj sekciji nije dozvoljen. Pozivalac se blokira, sve dok proces koji se izvršava u kritičnoj sekciji ne pozove `unlock()` i time izađe iz nje. Ako je više procesa ili niti pozvalo `lock()` sa istim *mutex*-om dok je kritična sekcija bila zaključana, po otključavanju kritične sekcije odabira se jedan proces ili nit za deblokiranje, npr. onaj s najvećim prioritetom ili onaj koji je prvi pozvao `lock()`. *Mutex*-i se mogu realizovati kao semafori sa blokiranjem procesa ili korištenjem TSL instrukcije.

4.12.Windows API za sinhronizaciju

Windows API poznaje semafore i *mutex*-e i ako se njihova imena navode u obliku niza znakova kao parametra, može se koristiti i između procesa, ne samo niti. Za uzajamno isključivanje između niti pod Windows-om se koriste objekti tipa kritične sekcije. Funkcije Windows API-ja za sinhronizaciju su date u tabeli.

Funkcija	Značenje
InitializeCriticalSection (lpCriticalSection)	Inicijalizira kritičnu sekciju
EnterCriticalSection(lpCriticalSection)	Čeka na ulaz u kritičnu sekciju i uđe kada je slobodna
TryEnterCriticalSection(lpCriticalSection)	Pokušava ući u kritičnu sekciju bez čekanja
LeaveCriticalSection(lpCriticalSection)	Otključava kritičnu sekciju
DeleteCriticalSection(lpCriticalSection)	Briše kritičnu sekciju
WaitForSingleObject(hHandle, dwMilliseconds)	Čeka oslobađanje sinhronizacijskog objekta ili navedeno vrijeme
WaitForMultipleObjects(nCount, lpHandles, bWaitAll, dwMilliseconds)	Čeka na oslobađanje bar jednog ili svih sinhronizacijskih objekata
CreateMutex(lpMutexAttributes, bInitialOwner, lpName)	Kreira objekt uzajamnog isključenja
ReleaseMutex(hMutex)	Otvora postojeći Mutex
CloseHandle(hObject)	Zatvara sinhronizacijski objekt
CreateSemaphore(lpSemaphoreAttributes, lInitialCount, lMaximumCount, lpName)	Kreira semafor
ReleaseSemaphore(hSemaphore, lReleaseCount, lpPreviousCount)	Uvećava vrijednost semafora

Sve funkcije za rad sa kritičnim sekcijama imaju po jedan parametar tipa `LPCriticalSection`. Varijablu koja će služiti za uzajamno isključivanje treba najprije inicijalizirati funkcijom *InitializeCriticalSection*. Ako se želi realizovati kritična sekcija sa zaposlenim čekanjem, poziva se funkcija *TryEnterCriticalSection*. Ako ova funkcija vrati nulu, znači da je druga nit u kritičnoj sekciji i da treba ponoviti čekanje. Uzajamno isključivanje s blokiranjem niti se postiže pozivom *EnterCriticalSection*. Oslobađanje kritične sekcije se obavlja funkcijom *LeaveCriticalSection*. Objekte za uzajamno isključivanje u kritičnoj sekciji treba na kraju obrisati funkcijom *DeleteCriticalSection*.

U Windows sistemima sinhronizacijski objekti imaju globalna imena kako bi bili prepoznati u različitim procesima. To ime se navodi kao niz znakova u zadnjem parametru *CreateMutex* ili *CreateSemaphore* funkcije. Ove funkcije vraćaju rukovatelj tipa `HANDLE` koji se koristi u ostalim funkcijama za rad sa semaforima i drugim sinhronizacijskim objektima. Ako semafor ili *mutex* već postoji kreiran pod tim imenom, on postaje dostupan za oba procesa. Prvi parametar funkcija za kreiranje može definirati prava ostalih programa nad semaforom ili biti `NULL` za podrazumijevane vrijednosti. Kao drugi parametar *CreateSemaphore* navodi se početna vrijednost, a treći predstavlja maksimalnu vrijednost semafora. Ako je drugi parametar *CreateMutex* funkcije bio `TRUE`, program koji pozvao tu funkciju odmah dobiva *mutex*. Zauzimanje i *mutex*-a i semafora ili eventualno blokiranje procesa obavlja se funkcijom *WaitForSingleObject*. Prvi parametar ove funkcije je rukovatelj na *mutex* ili semafor, drugi parametar je maksimalno vrijeme čekanja, nakon čega se nastavlja izvršenje, osim ako je on jednak `INFINITE`. Signaliziranje semafora se obavlja funkcijom *ReleaseSemaphore*, koja može kao drugi parametar imati

vrijednost za koju se uvećava semafor, a kao treći pointer na varijablu gdje se ranija vrijednost semafora pamti. Otključavanje *mutex*-a obavlja funkcija *ReleaseMutex*.

U sljedećem primjeru su dva programa *sem1.c* i *sem2.c*. U programu *sem1* unose se razni brojeve, ali tek kada se unese broj 20, primjer *sem2.c* će se također završiti s porukom Konačno dvadeset. API funkcije *CreateSemaphore*, *WaitForSingleObject* i *ReleaseSemaphore* obavljaju klasične operacije *InitialSem*, *wait* i *signal*, ali imaju više parametara.

```
// sem1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
HANDLE hSemaphore;
LONG cMax = 10;
LONG cPreviousCount;
int main(int argc, char *argv[]){
    int broj;
    hSemaphore = CreateSemaphore(
        NULL, 0, INFINITE
        "DVADESETKA");
    if (hSemaphore == NULL) {
        printf("Greska");    }
    do {
        scanf ("%d",&broj);
    } while (broj !=20);
    if (!ReleaseSemaphore(
        hSemaphore, 1, NULL) ) {
        printf("Neuspjio release");
    }
    return 0;
}
```

```
// sem2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
HANDLE hSemaphore;
LONG cMax = 10;
LONG cPreviousCount;
int main(int argc, char *argv[]){
    hSemaphore = CreateSemaphore(
        NULL, 0, INFINITE
        "DVADESETKA");
    if (hSemaphore == NULL) {
        printf("Greska");
    }
    WaitForSingleObject(hSemaphore,
        INFINITE);
    printf("Konacno dvadeset \n");
    return 0;
}
```

4.13.POSIX API za sinhronizaciju

U Posix API za sinhronizaciju semaforima koriste se sljedeće funkcije.

Funkcija	Značenje
<i>sem_open</i> (pcName, iOflag, iNmode, iValue)	Kreira imenovani semafor ili se povezuje na njega
<i>sem_init</i> (pSem, iPshared, iValue)	Inicijalizira neimenovani semafor
<i>sem_close</i> (pSem)	Završava vezu s otvorenim semaforom
<i>sem_unlink</i> (pcName)	Završava vezu s otvorenim semaforom i uklanja ga kada ga zadnji proces prestane koristiti
<i>sem_destroy</i> (pSem)	Uklanja neimenovani semafor iz memorije
<i>sem_getvalue</i> (pSem, pValue)	Kopira vrijednost semafora u cijeli broj
<i>sem_wait</i> (pSem)	Blokira se kada semafor drži drugi proces
<i>sem_trywait</i> (pSem)	Vraća grešku kada semafor drži drugi proces
<i>sem_post</i> (pSem)	Povećava brojač semafora

Sada će se napisati iznova primjer sa dvije niti duzi i kraci, no sada sa korištenjem Linux semafora. Uzajamno isključivanje u kritičnoj sekciji postignuto je funkcijama *sem_wait* i *sem_post*.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define THREAD 0
#define PROCESS 1
int brojevi[2000];
int brojac=0;
sem_t sem;
void dodaj() {
    int poz;
    poz=brojac;
    brojac++;
    brojevi[poz]=poz;
    usleep(2);
}
void * duzi() {
    int i,poz;
    for(i=0;i<1500;i++) {
        sem_wait(&sem);
        dodaj();
        sem_post(&sem);
    }
}

void * kraci() {
    int i,poz;
    for(i=0;i<500;i++) {
        sem_wait(&sem);
        dodaj();
        sem_post(&sem);
    }
}

int main() {
    int rt1, rt2, s,i;
    pthread_t t1, t2;
    /* Kreiraj dvije niti */
    sem_init(&sem,THREAD,1);
    pthread_create( &t1, NULL, &duzi, NULL);
    pthread_create( &t2, NULL, &kraci, NULL);
    /* Sačekaj da se obje završe */
    pthread_join( t1, NULL);
    pthread_join( t2, NULL);
    s=0;
    for(i=0;i<2000;i++)
        s=s+brojevi[i];
    printf ("%d \n",s);
    return 0;
}

```

4.14.Monitori

Kada je višenitni program složeniji, moguće su i veće greške. Na primjer, ako treba više potprograma sa kritičnim sekcijama uzajamno isključiti, na početku svakog potprograma stavljaju se pozivi rutina koji zaključavaju kritičnu sekciju, a na kraju potprograma pozivi koji je otključavaju. Dovoljno je na jednom od njih zaboraviti dodati potrebne pozive da rezultat bude nezaštićena kritična sekcija ili zastoj sistema. Da bi bilo jednostavnije pisanje korektnih programa, Hansen i Hoare predložili su sinhronizacijsku mogućnost na koju su nazvali monitor. Uvođenje monitora zahtijeva promjenu programskih jezika.

Monitor predstavlja skup potprograma, promjenljivih i struktura podataka koje su sve grupirane u posebnu vrstu modula. Po tome više liči na objekte u C++ nego na semafore, ali se razlikuju i od objekata jer nema klasa i nasljeđivanja. Procesi ili niti koji se izvršavaju izvan monitora mogu zvati potprogramme u monitoru, ali ne mogu direktno pristupiti varijablama unutar monitora. Najvažnija osobina monitora je da kada neka nit ili proces pozove potprogram koji je u monitoru, nijedna druga nit ili proces ne može pozvati potprogram iz tog monitora, dok se ne završi poziv monitorskog potprograma prve niti.

Neka je jezik C proširen monitorima, uvođenjem ključne riječi monitor. U listingu ispod je dat primjer monitora koji ima jednu varijablu i tri potprograma.

```

monitor example
{
    int x;
    void init(int n) {
        x=n;
    }
    void increment () {
        x++ ;
    }
    void decrement () {
        x-- ;
    }
}

```

Neka se sada iz neke niti pozove potprogram `example.increment()`. Promijenjeni kompajler za C će dodati u generiranom kodu odgovarajuće pozive za zaključavanje i otključavanje *mutex*-a prije i nakon svakog takvog poziva. Ni jedna druga nit neće moći pozvati potprograme `example.init()`, `example.increment()` ili `example.decrement()` dok se poziv `example.increment()` ne završi.

Monitori rješavaju problem uzajamnog isključivanja, ali ne mogu uticati na redoslijed izvršavanja. Zato su uvedene **uslovne varijable** i dvije operacije koje se mogu izvoditi nad njima, *waitc* i *signalc*. Kada se pozove *waitc* nad uslovnom varijablom, proces ili nit koji je pozvao ovaj poziv se blokira sve dok neka nit ne pozove *signalc*. Iako su operacije s njima slične operacijama s semaforima, uslovne varijable nisu brojači. One ne akumuliraju događaje za kasniju upotrebu na način kako to rade semafori. U slučaju semafora inicijaliziranog na 0, ako se *wait* obavi nakon *signal*, program koji je pozvao *wait* će nastaviti s radom, dok kod uslovnih varijabli neće, nego će čekati na novi *signalc* iskaz.

Primjeri programskih jezika koji podržavaju monitore su Java, Ada 95 i C#. Klasa u jeziku Java se ponaša monitor tako što se pri deklaraciji metode doda ključna riječ *synchronized*, čime se postiže međusobno isključivanje poziva ovih metoda iz različitih niti. Metodama `wait()` i `notify()` omogućava se funkcionalnost uslovnih varijabli.

4.15.Barijere

Neke aplikacije su podijeljene u faze i imaju pravilo da se nijedan proces ili nit ne smije izvršavati u sljedećoj fazi ako svi procesi ili niti nisu spremni da uđu u tu fazu. Na primjer program koji računa određeni integral neke funkcije numeričkom metodom zasniva se na podjeli područja integracije ispod krive na manje površine koje su približne jednostavnijim geometrijskim likovima poput trapeza. Površine tih trapeza se mogu računati jedna za drugom, ali se mogu računati i istovremeno, ako se koristi višeprocorski računar. U tom slučaju svaka površina se računa u posebnoj niti. Ali konačno rješenje ne možemo dobiti dok nisu izračunate sve manje površine koje tek tada saberemo u jedinstvenu sumu. Mjesto u programu (u ovom primjeru to je trenutak prije sabiranja parcijalnih površina) gdje sve niti moraju da čekaju dok sve ne dođu do tog mjesta zove se barijera. Kada jedan proces u grupi procesa dostigne barijeru, on se blokira dok svi procesi ne dostignu istu barijeru.

Barijere se mogu realizovati semaforima sa blokiranjem procesa. Sljedeća funkcija se može pozvati na mjestu gdje se barijera očekuje.


```
int count = 0;
semaphore mutex=1;
semaphore barrier=0;
void barijera(int n) { //n je broj niti
    wait(mutex);
    count ++;
    signal(mutex);
    if (count == n) { // Ovo će izvršiti samo nit koja zadnja dolazi
        signal(barrier); // Oslobodi jednu nit
    }
    wait(barrier); // Sada se čeka
    signal(barrier); // Nit kada je završila čekanje, oslobađa sljedeću
}
```

4.16.Sinhronizacija procesa - Klasični problemi sinhronizacije

Većina literature o operativnim sistemima i konkurentnom programiranju spominje tri problema sinhronizacije koji su nastali iz prakse ili teoretskog razmišljanja. Tako se svaki predloženi sinhronizacijski mehanizam testira na tri sljedeća problema.

- Problem proizvođač – potrošač
- Problem čitač – pisac
- Problem filozofa koji jedu

4.17.Problem proizvođač – potrošač

Problem proizvođača i potrošača, poznat i kao problem ograničenog bafera se jako često javlja u praksi operativnih sistema. Neka dva procesa dijele zajedničko memorijsko područje (bafer) fiksne veličine. Jedan od njih, proizvođač, stavlja podatke u bafer, a drugi ih uzima (kažemo koristi). Na primjer, u drajverima za tastaturu proizvođač se izvršava unutar obrade prekida svaki put kada se pritisne taster. Tada se oznaka tastera upiše na sljedeće slobodno mjesto u baferu. Potrošač se poziva u sistemskom pozivu read() kada se očitava ubačeni znak i izbriše iz bafera.

U ovom problemu postoje dva specijalna slučaja. Proizvođač mora da se zaustavi ako u baferu nema više mjesta, najbolje da se blokira. Potrošač se blokira ako nema nijednog podatka u baferu koji već nije pročitao.

Problem je lako rješiv korištenjem semafora. Sljedeći primjer rješenja problema proizvođača i potrošača pretpostavlja da proizvođač generiše slučajne podatke u kružni bafer veličine N elemenata, a potrošač ih prikazuje na ekranu. Koriste se tri semafora. Semafor *proizvedeno* se postavi na 0. Uvećava se svaki put kada proizvođač generiše novi podatak, a umanjuje kada potrošač čita taj podatak. Semafor *slobodno* se inicijalizira na N, umanjuje se kada proizvođač generiše podatak a uvećava nakon što ga potrošač iskoristi. Semafor *mut* postiže uzajamno isključivanje u kritičnoj sekciji samog upisa u niz podataka.

```

semaphore mut=1;
semaphore proizvedeno=0;
semaphore slobodno=N;
int pod;
int buffer[N],proizpos=0,potrpos=0;
void proizvojac() {
    while (1) {
        wait(&slobodno);
        wait(&mut);
        pod=rand(10);
        buffer[(proizpos++)%N]=pod;
        signal(&mut);
        signal(&proizvedeno);
    }
}

```

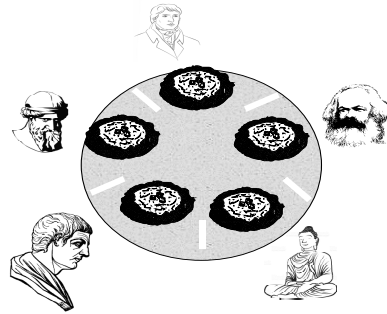
```

void potrosac() {
    while (1) {
        wait(&proizvedeno);
        wait(&mut);
        pod= buffer[(potrpos++)%N];
        printf("%d",pod);
        signal(&mut);
        signal(&slobodno);
    }
}

```

4.18.Problem pet filozofa

Drugi klasični problem sinhronizacije je problem pet filozofa, koga je prvi formulirao E. Dijkstra. Za razliku od prethodnog, ovaj problem nije direktno korišten u OS, ali ilustrira problem potpunog zastoja zbog nadmetanja za resurse. Pet filozofa sjede za okruglim stolom (Slika 36). Svaki filozof ima tanjir sa špagetima. Za jedenje špageta potrebno je koristiti dvije viljuške. Između dva tanjira se nalazi po jedna viljuška. Filozofi naizmjenično razmišljaju i jedu. Za jelo su potrebne dvije viljuške. Po završetku jela viljuške se oslobađaju. Problem je što je svaka viljuška dijeljena sa susjedom i samo jedan filozof može da je uzme. Ako drugi filozof drži viljušku, filozof kome viljuška treba mora da čeka. Pretpostavimo da se svaka viljuška modelira jednim semaforom, pri čemu vrijednost 1 znači da je viljuška slobodna. Ponašanje svakog filozofa modelira se sljedećom funkcijom.



Slika 36 Pet filozofa koji jedu

```

semaphore viljuska[5]={1,1,1,1,1};
void filozof(int i) { /* i: broj filozofa, od 0 do 4 */
    while (1) {
        printf("Filozof %d razmislija.\n",i);
        sleep(random(10)); /* Pauza */
        wait(&(viljuska[i])); /* uzima lijevu viljušku */
        wait(&(viljuska[(i+1)%5])); /* uzima desnu viljušku */
        printf("Filozof %d jede.\n",i);
        sleep(random(10)); /* Pauza */
        signal(&(viljuska[i])); /* spušta lijevu viljušku */
        signal(&(viljuska[(i+1)%5])); /* spušta desnu viljušku */
    }
}

```

Ako pokrenemo u pet niti kopiju funkcije za svakog filozofa može se desiti da je svih pet niti gotovo istovremeno izvršilo naredbu `wait (&viljuska[i])`. Tada

nijedan filozof ne može preći na narednu naredbu za uzimanje sljedeće viljuške i dolazi do tzv. **potpunog zastoja**. Analizirano je više rješenja ovog problema. Nisu dobra ni rješenja u kojima se uzima lijeva viljuška, provjerava da li je desna slobodna i ako nije spušta se lijeva viljuška i ponovo čeka, jer se može desiti da svi filozofi to rade istovremeno, neograničeno.

Jednostavno ispravno rješenje je ograničenje da maksimalno četiri filozofa mogu da jedu uvođenjem semafora *limit* koji se postavi na 4.

```
semaphore viljuska[5]={1,1,1,1,1};
semaphore limit=4;
void filozof(int i) { /* i: broj filozofa, od 0 do 4 */
    while (1) {
        printf("Filozof %d razmislija.\n",i);
        sleep(random(10)); /* Pauza */
        wait(&limit);
        wait(&(viljuska[i])); /* uzima lijevu viljušku */
        wait(&(viljuska[(i+1)%5])); /* uzima desnu viljušku */
        printf("Filozof %d jede.\n ",i);
        sleep(random(10)); /* Pauza */
        signal(&limit);
        signal(&(viljuska[i])); /* spušta lijevu viljušku */
        signal(&(viljuska[(i+1)%5])); /* spušta desnu viljušku */
    }
}
```

Drugo rješenje je postavljanje *mutex*-a na dohvaćanje obje viljuške (koji predstavlja konobara koji dopušta filozofima da jedu) tako da je uzimanje dvije viljuške uvijek garantovano. Treće rješenje je da se ne uzima uvijek prvo lijeva pa desna viljuška nego prvo viljuška s manjim rednim brojem, a zatim ona s većim rednim brojem, što bi značilo da filozof 4 prvo uzima viljušku 0, pa onda viljušku 4.

4.19. Problem čitača i pisaca

Problem čitača i pisaca dolazi iz sistema za upravljanje bazama podataka. U ovakvom sistemu, više procesa pristupa bazi podataka. Neki od njih čitaju, a neki pišu podatke u bazu. Dok se upisuje u bazu podataka ni jedan drugi proces ne smije pristupati bazi, jer se mogu dobiti nekonzistentni podaci. S druge strane, ako pristupaju samo čitači tada ne treba ograničavati njihov pristup.

U sljedećem rješenju, prvi čitač koji dobije pristup bazi podataka izvršava operaciju *wait()* na semaforu *db*. Sljedeći čitači treba samo da povećaju brojač čitača za jedan. Kada čitači završe rad s bazom podataka, oni smanjuju brojač za jedan i posljednji čitač izvršava *signal()* proceduru koja omogućuje eventualno blokiranim piscima, ako ih ima, da dobiju pristup bazi.

```

Semaphore mut=1;
semaphore db=1;
int citaca=0;

void pisac() {
    while (1) {
        analizapodataka();
        wait(&mut);
        upisubazu();
        signal(&mut);
    }
}

void citac() {
    while (1) {
        wait(&mut);
        if (++citaca == 1)
            wait(&db);
        signal(&mut);
        citanjeizbaze();
        wait(&mut);
        if (--citaca == 0)
            signal(&db);
        signal(&mut);
        analizapodataka();
    }
}

```

4.20.Među-procesna komunikacija

Za razliku od sinhronizacije, koja se bavi problemom natjecanja za resurse između procesa, među-procesna komunikacija se bavi problemom suradnje između procesa dijeljenjem podataka. Kod niti ovaj problem nije mnogo bitan, jer niti automatski dijele memoriju unutar istog procesa. Proces, međutim, imaju odvojene memorijske prostore, što je uglavnom korisno i sigurno, ali se kada treba dijeliti podatke moraju koristiti različite tehnike. Tehnike za među-procesnu komunikaciju osiguravaju metode pomoću kojih više procesa mogu da komuniciraju jedan sa drugim. Sljedeća tabela nabroja neke mehanizme među-procesne komunikacije uz operativne sisteme koji ih podržavaju.

Mehanizam	Windows	Unix/Linux
Dijeljene datoteke	Da	Da
Neimenovane i imenovane cijevi	Da	Da
Poruke	Da	Da
Dijeljena memorija mapiranjem datoteka	Da	Da
Signali	Ne	Da
Mrežni <i>socketi</i>	Da	Da
<i>Clipboard</i>	Da	Uslovno
COM	Da	Ne
DDE	Da	Ne
RPC	Da	Da

Neki od mehanizama će biti predstavljani u daljnjem tekstu.

4.21.Dijeljene datoteke

Svaki operativni sistem koji podržava rad sa datotekama omogućava da se one koriste za komunikaciju između procesa. Ova komunikacija se može obavljati i kada su procesi aktivni, ali i ako neki od procesa kome treba poslati poruku još nije ni započeo. Na disku se kreira zajednička datoteka kojoj pristupaju različiti procesi. Datoteka je podijeljena u slogove čiji sadržaj predstavlja podatke koje jedan proces šalje drugom. Na primjer, prvi slog ove datoteke može sadržati podatak koga treba primiti proces broj 1. Kako većina modernih operativnih sistema poznaje i mrežne diskove, ovaj pristup se može koristiti i za među-procesnu komunikaciju između odvojenih računara.

Teškoće nastaju kada više procesa treba da upisuje u isti slog, na primjer, dva procesa imaju podatke za isti proces. Tada se dešavaju isti problemi konkurentnosti kao sa dijeljenom memorijom. Ukoliko se dijeljene datoteke koriste za među-procesnu komunikaciju na istom računaru tada se pristup pojedinom slogu treba zaštititi koristeći *mutex* ili semafor. Ukoliko se dijeljena datoteka koristi za komunikaciju između odvojenih računara na mreži, datoteku koja se nalazi na zajedničkom disku treba **zaključavati** prilikom pristupa dijeljenom podatku. Zaključavanje se postiže sistemskim pozivom poput *fncntl()* na Unix sistemima ili *LockFile* na Windows sistemima. Parametri ovih poziva uključuju oznaku datoteke, početnu poziciju u datoteci koja se zaključava i dužinu područja datoteke koje se zaključava. Kada se pozove zaključavanje, drugim procesima se zabranjuje pristup tom dijelu datoteke i za njih poziv funkcije za zaključavanje vraća vrijednost koja predstavlja neuspjeh. U tom slučaju treba odustati od pristupa tom dijelu datoteke ili pokušati ponovo dok onaj proces koji je zaključao datoteku ne pozove poziv za njeno otključavanje.

4.22.Dijeljena memorija i memorijski mapirane datoteke

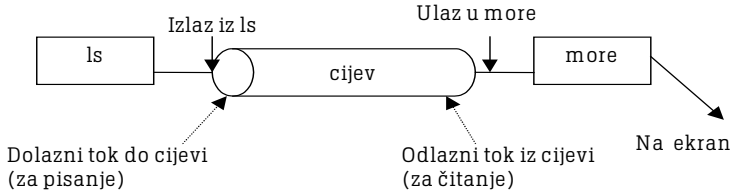
Glavna mana upotreba datoteka za komunikaciju između procesa je sporost, pošto se koristi disk koji je mnogo sporiji od memorije. Ova sporost dolazi do izražaja pogotovo ukoliko je potrebno često zaključavati i otključavati dijeljenu datoteku u radu preko mreže, jer dolazi do izražaja i mrežni saobraćaj. Ako su procesi koji trebaju međusobno komunicirati na istom računaru i u istoj fizičkoj RAM memoriji, znatno je brže koristiti tu memoriju za dijeljenje podataka. Procesu su normalno u memorijskom području izoliranom od drugih procesa, ali se može zatražiti od operativnog sistema da dodijeli zajednički dio memorije kome mogu pristupiti različiti procesi. O načinu uspostavljanja dijeljene memorije biće više riječi u poglavlju o upravljanju memorijom.

4.23.Cijevi

Cijevi (pipe) predstavljaju jednosmjernan komunikacioni kanal između dva procesa u UNIX sistemima. Cijev ima dva kraja: jedan za pisanje i jedan za čitanje. Koristeći cijevi, jedan proces može puniti bajte na kraj za pisanje, a neki drugi proces može preuzimati ovaj tok bajta na kraju za čitanje. Pisanje podataka u cijevi i njihovo čitanje se obavlja slično pisanju i čitanju datoteka.

UNIX poznaje dvije vrste cijevi, neimenovane i imenovane. Neimenovane cijevi se koriste uglavnom da se razmjene poruke između povezanih procesa (na primjer, između dva procesa dijete koji imaju isti proces roditelj. One se često koriste u komandnoj školjci, na primjer, kada se unese komanda koja prosljeđuje izlaz iz programa *ls* programu *more* (Slika 37).

ls more



Slika 37 Komunikacija između aplikacija pomoću cijevi

Neimenovana cijev se kreira korištenjem sistemskog poziva `pipe()`. Parametar ovog poziva pokazuje na niz od dva elementa. Sistemski poziv `pipe()` vraća 0 ako je cijev uspješno kreirana ili -1 u slučaju greške. Ako je poziv bio uspješan, onda prvi element proslijeđenog niza `fd[0]` dobiva datotečni deskriptor za čitanje a `fd[1]` dobiva datotečni deskriptor za pisanje na kreiranoj cijevi. Nakon toga se komunikacija obavlja pozivima `read` i `write`.

Sljedeći primjer kreira dijete proces kome roditelj proces nakon toga pošalje jednu poruku. Oba procesa trebaju zatvoriti one strane cijevi koje ne koriste.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main(void){
    int    fd[2], nbytes;
    pid_t  childpid;
    char    string[] = "Poruka\n";
    char    prijem[80];
    pipe(fd);
    childpid = fork();
    if(childpid == 0) {
        close(fd[0]); /* Dijete proces zatvara ulaznu stranu cijevi */
        write(fd[1], string, (strlen(string)+1)); /* Posalji niz */
        exit(0);
    }
    else {
        close(fd[1]); /* Roditelj proces zatvara izlaznu stranu cijevi */
        nbytes = read(fd[0], prijem, sizeof(prijem)); /* Citaj niz */
        printf("Primljeno: %s", prijem);
    }
    return(0);
}
```

Imenovane cijevi mogu ostati aktivne i kada je proces završen, a pogodnije su kada nam treba više vidova komunikacije između procesa. Cijevi se ponašaju kao datoteke sa sekvencijalnim pristupom. Imenovana cijev se kreira pozivom `mkfifo`, koja kao prvi parametar ima ime pomoćne datoteke koja će služiti za komunikaciju, a drugi parametar predstavlja prava pristupa nad njom.

Sljedeći primjer koristi dva odvojena procesa `pipesender.c` i `pipereceiver.c`. Program `pipereceiver` treba prvi pokrenuti i on će kreirati imenovanu cijev funkcijom `mkfifo`, otvoriti komunikacijsku datoteku funkcijom `open` i zatim čekati u pozivu `read` dok se ne izvrši program `pipesender`. Ovaj program otvara komunikacijsku datoteku, šalje u nju podatke i zatvara je.

```
// pipesender.c
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#define MAX_LINE 80
int main(int argc, char** argv) {
    char line[MAX_LINE];
    int pipe;
    pipe=open("/tmp/pipe",O_WRONLY);
    printf("Unesi liniju: ");
    fgets(line, MAX_LINE, stdin);
    write(pipe, line, strlen(line));
    close(pipe);
    return 0;
}
```

```
// pipereceiver.c
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#define MAX_LINE 80
int main(int argc, char** argv) {
    char line[MAX_LINE];
    int pipe;
    mkfifo("/tmp/pipe", 0666);
    pipe=open("/tmp/pipe", O_RDONLY);
    read(pipe, line, MAX_LINE);
    printf("Primio %s",line);
    close(pipe);
    return 0;
}
```

Cijevi u Windows-u služe za komunikaciju aplikacija koristeći funkcije za pisanje datoteka. Principijelno se koriste slično Unix cijevima, ali se koriste druge funkcije, koje su date u tabeli.

Funkcija	Namjena
CreatePipe(phReadPipe, phWritePipe, lpPipeAttributes, nSize)	Kreira neimenovanu cijev
CreateNamedPipe(lpName, dwOpenMode, dwPipeMode, nMaxInstances, nOutBufferSize, nInBufferSize, nDefaultTimeOut, lpSecurityAttributes)	Kreira imenovanu cijev
ConnectNamedPipe(hNamedPipe,lpOverlapped);	Postavlja server za imenovanu cijev

4.24.Prosljeđivanje poruka

Sljedeća metoda među-procesne komunikacije se naziva prosljeđivanje poruka. Pojam poruke je dovoljno generalan da se koristi, sa različitim načinima implementacije, između podsistema operativnih sistema na mikro-jezgru ili hibridnom jezgru, između procesa ili niti na istom operativnom sistemu, odnosno između različitih računara na globalnim i lokalnim mrežama. Metoda koristi dvije funkcije, *send* i *receive*. Poziv *send*(odredište,poruka) će poslati poruku ka datom odredištu dok će poziv *receive*(izvor, poruka) primiti poruku od navedenog pošiljaoca. Ako nema poruka, proces koji očekuje poruke će preći u stanje blokiranja sve dok ne za njega na stigne neka poruka. Može se dodati i funkcija *tryreceive*(izvor,poruka) koja završava odmah sa kodom greške ako nema poruka ili preuzimanjem poruke ako je ima.

Za razliku od dijeljene memorije, kod koje se podatak od pošiljaoca odmah dostavlja primaocu, poruke imaju odgovarajuće redove čekanja na njihov prijem. Kada pošiljalac pošalje poruku, njena kopija se smjesti u takav red. Poruka ostaje u redu čekanja dok ne bude pročitana, nakon čega se briše iz njega. Svaki primalac može imati svoj red poruka koje su mu namijenjene, a može postojati i jedan

zajednički red poruka koji se zove **mailbox**. U slučaju komunikacije sa zajedničkim redom poruka u pozivima *send*, *receive* i *tryreceive* se umjesto primaoca ili pošiljaoca navodi oznaka *mailbox-a*, a stvarni primalac i pošiljalac su smješteni u tijelo poruke..

Sistem prosljeđivanja poruka pruža više mogućnosti za među-procesnu komunikaciju u odnosu na dijeljenu memoriju i cijevi, jer se može koristiti i u distribuiranim sistemima. Ponekad poruke pojednostavljaju komunikaciju, jer u nekim slučajevima ne dolazi do natjecanja za resurse do kojeg bi došlo kada bi se koristila dijeljena memorija. Međutim, javlja se više problema kod dizajna sistema poruka, pogotovo ako se sistem poruka koristi za komunikaciju između procesa koji su različitim računarima unutar mreže. Na primjer, poruke mogu biti izgubljene. Da se osigura od gubitaka poruka, primalac može poslati posebnu poruku potvrde uspješnog prijema, nakon što je primljena poruka s podatkom. Ako pošiljalac ne primi poruku potvrde prijema unutar nekog vremenskog intervala, on vrši ponovno slanje poruke (retransmisija poruke). Mogući su razni scenariji:

- Ispravan slučaj: Pošiljalac je poslao poruku, primalac je primio poruku i poslao potvrdu koja je stigla do pošiljaoca.
- Pošiljalac je poslao poruku, koju primalac nije primio. Pošiljalac nakon što je vrijeme isteklo, šalje ponovo poruku, primalac je prihvaća i šalje potvrdu.
- Pošiljalac je poslao poruku. Primalac ju je primio i poslao potvrdu, ali se potvrda izgubila. Pošiljalac šalje podatak ponovo. Primalac je dobiva stoga dva puta i jednu mora ignorirati. Da bi znao da je to dva puta poslana ista poruka, u tijelu poruke mora postojati i specijalni niz brojeva, identifikator poruke, koji čini poruku uvijek jedinstvenom.
- Pošiljalac šalje poruku, ali primalac uopće nije u režimu prijema (završio se taj dio programa ili cijeli program). Pošiljalac pokušava ponovo i ponovo da šalje poruku. Da ne bi ostao vječno u režimu slanja poruka, može se dodati pravilo da nakon određenog broja retransmisija pošiljalac prestane sa slanjem.
- Primalac čeka na poruke, a pošiljalac dugo vremena ne šalje ništa. Može se uvesti pravilo da nakon isteka nekoliko minuta primalac izađe iz čekanja jer to znači da je pošiljalac završio.
- Neka od poruka se promijenila u toku transfera. Moguće su vrlo nepredvidljive situacije. Na primjer, pošiljalac šalje dva puta istu poruku jer nije stigla potvrda od primaoca, ali pri drugoj poruci se promijenio identifikator poruke. Primalac je primio dvije, za njega različite, poruke. Da bi se ovo spriječilo, uvode se i kontrolne sume nad porukama i provjeravaju prilikom prijema.

Zbog potrebe za kompleksnijim protokolom koji bi obradio gore navedene slučajeve, komunikacija porukama je sporiji vid među-procesne komunikacije od dijeljene memorije i cijevi.

Sve Windows aplikacije koje koriste prozore, komuniciraju putem poruka. Poruke su predstavljene strukturom tipa MSG čiji elementi sadrže oznaku prozora kome je namijenjena, oznaku same poruke, dva dodatna parametra koji se prosljeđuju, vrijeme kada je poruka prosljeđena i pozicija na ekranu ako je poruka nastala uslijed

djelovanja miša. Postoji veliki broj sistemskih poruka, ali mogu se definirati i korisničke. U ovoj tabeli su date funkcije za rad sa Windows porukama. Primjer upotrebe ovih funkcija će biti u poglavlju o grafičkom korisničkom sučelju.

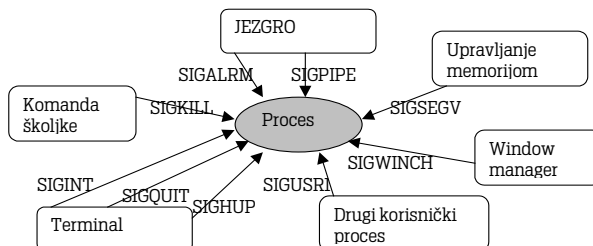
Funkcija	Značenje
GetMessage(lpMsg,hWnd,wMsgFilterMin,wMsgFilterMax)	Preuzima sljedeću poruku u redu čekanja na sve poruke između prozora i čeka ako nema poruka
PeekMessage(lpMsg,hWnd,wMsgFilterMin,wMsgFilterMax,wRemoveMsg)	Preuzima sljedeću poruku u redu čekanja na sve poruke između prozora ali ne čeka ako nema poruka
DispatchMessage(lpmgs)	Šalje narednu poruku u redu čekanja, a prozoru kome je namijenjena
SendMessage(hWnd,Msg,wParam,lParam)	Direktno šalje poruku navedenom prozoru

4.25.Windows Clipboard i DDE

U operativnim sistemima sa grafičkim korisničkim sučeljem koristi se među-procesna komunikacija koja je inicirana od strane korisnika. Korisnik može radeći u jednoj aplikaciji npr. Microsoft Office odabrati pojedine dijelove teksta ili slika, pritisnuti taster Ctrl+C ili opcijom *Copy* iz menija, nakon čega dati podaci mogu biti dostupni drugim aplikacijama. Podaci se smještaju u posebno zajedničko područje memorije koje se zove odlagalište ili **clipboard**. U drugoj aplikaciji se tasterom Ctrl+V ili opcijom *Paste* u meniju ubacuju podaci. *Clipboard* podržava više različitih formata podataka: bit-mapirane slike, upute za iscrtavanje likova (metadatoteke), tekst i Unicode tekst.

DDE (*Dynamic Data Exchange*) predstavlja mehanizam među-procesne komunikacije slanjem onih tipova podataka koji se mogu naći unutar *clipboard* koristeći sistem Windows poruka. Poruke tipa WM_DDE_DATA koje se šalju između aplikacija u svom parametru mogu pokazivati na podatke koji se šalju pozivom funkcije *SendMessage*. Pored ovog tipa poruka, postoje i tipovi poruka za uspostavljanje DDE komunikacije, izvršavanje zahtjeva na serveru, prekid komunikacije i potvrdu prijema podataka.

4.26.Signal i alarmi



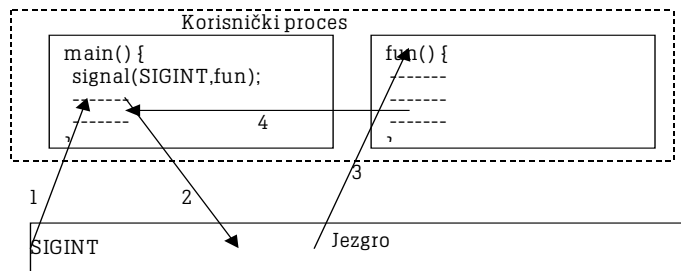
Slika 38 Signali

Signal je neka vrsta prekida programa u Unix sistemima. Oni predstavljaju vrlo rudimentarnu među-procesnu komunikaciju. Signali se dešavaju kada se pritisne CTRL+C za prekid programa i tada se šalje signal SIGINT ka programu. Slično, kada

se desi fatalna greška prilikom pristupa nedozvoljenom segmentu, tada se šalje signal SIGSEGV (Slika 38).

Uobičajena akcija pri prijemu signala je završetak izvršavanja programa (za signal SIGINT) ili snimanje slike memorije u trenutku završetka izvršavanja (engl. *dump core*) i završetak izvršavanja (za signal SIGSEV). Ove uobičajene akcije se mogu predefinirati.

Kada se generiše signal, operativni sistem prekida izvršavanje tekućeg procesa. Spašava se stanje prekinutog procesa i poziva obrađivač signala. Po završetku izvođenja obrađivača signala, operativni sistem preuzima kontrolu i vraća stanje prekinutog procesa (Slika 39). Ako akcija obrađivača nije bila završetak procesa, proces obično nastavlja na mjestu gdje je prekinut.



Slika 39: Reakcija na signal

Obrađivač signala se može postaviti funkcijom `signal()`. Sljedeći program postavlja obrađivač za signal SIGINT.

```
#include <signal.h>
#include <stdio.h>
void ctrl_c_obradjivac(int dummy) {
    printf("Pritisnuli ste ctrl-c\n");
    signal(SIGINT, ctrl_c_obradjivac);
}
main() {
    signal(SIGINT, ctrl_c_obradjivac);
    while(1) { }
}
```

Kada korisnik pritisne CTRL+C, operativni sistem će spasiti stanje programa i izvršiti potprogram `ctrl_c_obradjivac`. Po povratku iz obrađivača signala, operativni sistem nastavlja izvršavanje programa od mjesta gdje je taj program prekinut. Na kraju obrađivača ponovo se poziva `signal()` funkcija kako bi i naredni pritisci na CTRL+C pokretali obrađivač.

Posebna vrsta signala je alarm. Funkcija `alarm(n)` po povratku poslije `n` sekundi će uzrokovati da se desi signal SIGALRM. Ako ste postavili obrađivač za ovaj signal onda ga je moguće uhvatiti i uraditi akciju po želji. Ovaj primjer pokazuje kako UNIX može slati alarm svake sekunde.

```
#include <signal.h>
#include <stdio.h>
int i=0,  seconds;
void alarm_obradjivac(int dummy) {
    seconds++;
    printf("%d sekundi je proslo: i = %d\n", seconds, i);
    signal(SIGALRM, alarm_obradjivac);
    alarm(1);
}
main() {
    seconds = 0;
    signal(SIGALRM, alarm_obradjivac);
    alarm(1);
    while (1) {
        i++;
    }
}
```

4.27.Poziv udaljene procedure

Do sada spomenuti mehanizmi komunikacije između procesa su se zasnivali na tome da jedan proces pripremi podatke i pošalje ih drugom. Često, međutim, drugom procesu podaci trebaju samo na njegov zahtjev. Neka npr. jedan program ima meteorološke podatke o svakom mjestu na zemlji, a program koji ih koristi npr. prikazuje informacije o mjestu u koje korisnik ulazi na njegovom mobitelu. Nema nikakve potrebe da program šalje podatke o vremenu u svim mjestima svijeta svakom programu, koji je sa njim povezan, ali je potrebno da program klijent koji pokazuje informacije o gradovima povremeno pozove uslugu udaljenog meteorološkog programa servera. Ovo se svodi na odgovarajući protokol sa porukama, ali se on može sakriti tako da program klijent pozove potprogram uobičajenom sintaksom, npr. ako parametri potprograma predstavljaju geografske koordinate.

```
temperatura=trenutnatemperatura(44.125,18.130);
```

No, ovaj poziv potprograma samo sintaksom djeluje kao obični potprogram. Zapravo se aplikacije povežu sa odgovarajućim bibliotekama koje na poziv potprograma koji su vezani za udaljeni proces odrade sljedeće aktivnosti:

- Pokupe se ulazni parametri prosljeđeni pozivu RPC. Pošto udaljeni računar može biti sasvim druge računarske arhitekture, parametri se konvertuju u neutralni format. Na primjer, ne prenose se pointeri, a brojevi se prevode u ASCII oblik.
- Ako je server koji obrađuje potprogram na udaljenom računaru, klijent prosljeđuje podatke udaljenom računaru i naziv njegovog potprograma koji se poziva koristeći njegovu mrežnu adresu. Ako se nalazi na istom računaru, tada se prosto poziva potprogram,
- Na udaljenom računaru se očekuje prijem zahtjeva i ako se zaključi da je namijenjen pravom potprogramu, obrade se parametri, izvrši potprogram i vrate rezultati klijentu.

4.28.Zadaci

Zadatak 1: Izračunati približno $\int_0^{1.5} \sin(x^2) dx$ aproksimacijom integriranja površina 3 trapeza. Koristeći Win32 API napišite višenitnu verziju, tako da se račun svakog trapeza i prikaz konačne suma izvrši u posebnoj niti. Uz pomoć semafora postići međusobno isključivanje u kritičnoj sekciji, i da nit koja ispisuje konačnu sumu sačeka na ostale niti.

Rješenje

```
#include <stdio.h>
#include <windows.h>
#include <math.h>
HANDLE hMut,ht1,ht2,ht3;
float f (float x) {
    return (sin(x*x));
}
float sum;
void trapez (float min, float max) {
    WaitForSingleObject(hMut, INFINITE);
    sum += 0.5 *(f(min) + f(max)) *(max-min); /* add the endpoints */
    ReleaseSemaphore( hMut, 1, NULL);
}
DWORD WINAPI  t1 ( LPVOID lpParam ) {
    trapez(0,0.5);
    ReleaseSemaphore( ht1, 1, NULL); return 0;
}
DWORD WINAPI  t2 ( LPVOID lpParam ) {
    trapez(0.5,1);
    ReleaseSemaphore( ht2, 1, NULL); return 0;
}
DWORD WINAPI  t3 ( LPVOID lpParam ) {
    trapez(1,1.5);
    ReleaseSemaphore( ht3, 1, NULL); return 0;
}
DWORD WINAPI  t4 ( LPVOID lpParam ) {
    WaitForSingleObject(ht1, INFINITE);
    WaitForSingleObject(ht2, INFINITE);
    WaitForSingleObject(ht3, INFINITE);
    printf("Integral %f",sum);
    return 0;
}
void main() {
    sum=0.0;
    hMut = CreateSemaphore(NULL, 1, INFINITE,NULL);
    ht1 = CreateSemaphore(NULL, 0, INFINITE,NULL);
    ht2 = CreateSemaphore(NULL, 0, INFINITE,NULL);
    ht3 = CreateSemaphore(NULL, 0, INFINITE,NULL);
    CreateThread(NULL,65536,t1,NULL,0,NULL);
    CreateThread(NULL,65536,t2,NULL,0,NULL);
    CreateThread(NULL,65536,t3,NULL,0,NULL);
    WaitForSingleObject(CreateThread(NULL,65536,t4,NULL,0,NULL),INFINITE);
}
```

5.UPRAVLJANJE RESURSIMA I POTPUNI ZASTOJI

5.1.Resursi

Resurs je nešto što treba procesu za pravilno izvršavanje. To može biti i hardverski uređaj (štampač, jedinica magnetne trake, memorija), ili softverska jedinica informacije (dijeljena varijabla, slog u bazi podataka). U svakom slučaju, riječ je o objektima u sistemu čiji je broj ograničen, pa se ne mogu dodjeljivati procesima tek tako. Već je viđeno u poglavlju o konkurentnosti da se procesi natječu za resurse, pa se korištenje jednog resursa mora obaviti sa određenom disciplinom, kako bi procesi znali da li su resursi raspoloživi. Tipično, pristup resursu sastoji se iz tri koraka:

1. **Zahtjev za resursom** se postiže odgovarajućim sistemskim pozivom koji zavisi od vrste resursa. Na primjer, zahtjev za memorijom u Windows sistemu se obavlja pozivom *VirtualAlloc()*, a zahtjev za datotekom u Unix-u se obavlja pozivom *open()*. Ako je resurs zauzet, proces koji ga zahtijeva mora čekati dok ne postane slobodan. Ovo čekanje može biti zaposleno čekanje, ako je sistemski poziv vratio grešku, a može se i uspavati proces dok resurs ne postane slobodan. Ako to sam poziv ne radi, mogu se i eksplicitno pozvati semafori ili drugi mehanizmi zaključavanja resursa.

2. **Korištenje resursa** (npr. upis podataka u dodijeljenu memoriju) postaje moguće kada je zahtjev bio uspješan.

3. **Oslobađanje resursa** je potrebno pozvati kada se resurs više ne koristi.

Postoje dvije vrste resursa: oduzivi i neoduzivi resursi. Oduzivi resurs (engl. *preemptable*) može biti oduzet od procesa, bez štetnih posljedica. Primjer takvog resursa je memorija. Sadržaj memorije koja je dodijeljena procesu se snimi na disk, proces se suspenduje, a njegova memorija oslobodi i dodijeli drugom procesu. S druge strane, neoduzivi resursi (engl. *non-preemptable*) se ne mogu oduzeti, a da to ne izazove problem. Ekstremni primjer neoduzivog resursa je CD pisač. Ako se prekine proces pisanja prije zatvaranja sesije, CD postaje neupotrebljiv. Slično, ne može se oduzeti ni štampač usred štampanja dokumenta.

Ako nekim procesima treba više resursa situacija može biti složenija. Za primjer uzeće se *mainframe* računar sa dvije jedinice magnetne trake: traka A i traka B. Neka su pokrenuta istovremeno dva potprograma koji trebaju da kopiraju sa jedne trake na drugu. Pošto se trake premotavaju radi dolaska do prave datoteke, u trenutku pristupa jedinici trake mora se semaforom spriječiti da drugi proces ili nit pristupa traci u tom trenutku. Neka je konvencija takva da se semaforom ili *mutex*-om zaključa prvo izvorna jedinica trake, a zatim odredišna, što predstavlja zahtjev za resursima. Nakon toga se pročita datoteka sa izvorne jedinice trake i snimi na odredišnu jedinicu trake (korištenje). Po završetku kopiranja, otključa se semafor odredišne jedinice trake i na kraju semafor izvorne jedinice trake. (oslobađanje).

U primjeru koji slijedi se istovremeno pokreću potprogrami *saAnaB()* i *saBnaA()*. Zavisno od ponašanja raspoređivača, moguća je situacija u kojoj oba potprograma uspješno odrade svoj zadatak. Na primjer, prvi potprogram završi *wait(&trakaA)* i *wait(&trakaB)* i dobiva pristup ka obje jedinice trake. Drugi potprogram se sada ne

može miješati jer će ulazak u semaforne funkcije `wait(&trakaB)` izazvati čekanje. Prvi potprogram može uspješno kopirati datoteku i osloboditi oba semafora i drugi potprogram može odraditi zadatak.

```
Semaphore
trakaA=1, trakaB=1;

void saAnaB() {
    wait(&trakaA);
    wait(&trakaB);
    procitaj("A: vijesti.txt");
    snimi("B: vijesti.txt");
    signal(&trakaB);
    signal(&trakaA);
}
```

```
void saBnaA() {
    wait(&trakaB);
    wait(&trakaA);
    procitaj("B: novosti.txt");
    snimi("A: novosti.txt");
    signal(&trakaA);
    signal(&trakaB);
}
```

5.2.Potpuni zastoj

Ali, može se desiti i sljedeća situacija. Prvi potprogram je izvršio `wait(&trakaA)`, a onda je drugi potprogram izvršio `wait(&trakaB)`. Sada nijedan potprogram ne može izvršiti sljedeći `wait` poziv jer su oba semafora na nuli. Oba potprograma čekaju i to neograničeno i oni su u **potpunom zastoju**, pojam koji je već spomenut u problemu pet filozofa. U ovom primjeru resurs koji je izazvao potpuni zastoj je bio U/I uređaj, jedinica trake. Resurs može biti i softverski, poput sloga u datoteci žiro računa banke. Transakcija se obavlja tako što se zaključa dio datoteke koji čuva iznos uplatioca, pa dio datoteke koji čuva iznos korisnika sredstava, umanji iznos uplatioca, uveća iznos korisnika, i otključaju oba dijela datoteke. Npr. neka je firma PetrolPumpa uplatila troškove službenog putovanja na račun firme SuperTurist, a baš u tom trenutku firma SuperTurist platila troškove goriva svog autobusa firmi PetrolPumpa. Može se desiti takva nezgodna situacija da je svako od plaćanja zaključalo samo jedan od potrebna dva sloga u datoteci žiro računa i da se čeka da drugi postane dostupan.

Potpuni zastoj se može definirati na sljedeći način:

Skup procesa se nalazi u stanju potpunog zastoja ako svaki proces u skupu čeka na neki događaj koji može proizvesti jedino neki proces iz tog skupa.

Prethodna definicija je dosta široka, i obuhvata i situaciju kada neki proces čeka na završetak drugog procesa koji se zaglavio u beskonačnoj petlji. Detekcija ovakvih problema je dokazano matematički nemoguća u opštem slučaju (*halting problem*).

Ako se pod događajima ograniči na prisvajanje i oslobađanje resursa, ova definicija dovodi do sljedećih Coffmanovih uslova koji moraju biti ispunjeni za potpuni zastoj:

1. Uslov **uzajamnog isključivanja** znači da svaki resurs ili je dodijeljen tačno jednom procesu ili je dostupan. Ako neki proces traži resurs koji je već dodijeljen drugom procesu, proces tražilac mora čekati dok se resurs ne oslobodi.
2. Uslov **neoduzivosti resursa** znači da resursi koji su prethodno dodijeljeni ne mogu biti oduzeti procesu ukoliko ih proces samostalno ne oslobodi.
3. Uslov **stanja prisvajanja resursa i čekanja na drugi** znači da proces, koji trenutno zauzima ranije dodijeljene resurse, može zatražiti i nove resurse.

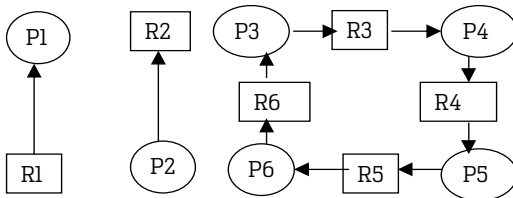
4. Uslov **kružnog čekanja** podrazumijeva da dva ili više procesa, čekaju na neki resurs koji drži sljedeći član lanca. Na primjer: proces P1 zauzeo štampač i želi skener. Skener je već dodijeljen procesu P2 a proces P2 traži slog u datoteci. Slog u datoteci je dodijeljen procesu P3 a proces P3 traži štampač.

Sva četiri uslova moraju biti prisutna da bi došlo do potpunog zastoja. Ako bar jedan nije ispunjen, to nije potpuni zastoj.

5.3.Modeliranje potpunog zastoja grafom alokacije resursa

Tri od spomenuta četiri uslova se mogu grafički modelirati pomoću **Holtovih grafova alokacije resursa**. Ovi grafovi se sastoje od skupa čvorova i skupa strelica. Postoje dvije vrste čvorova: procesi predstavljeni krugovima i resursi predstavljeni pravugaonicima. Strelica od čvora koji predstavlja resurs prema čvoru koji predstavlja proces znači da je proces dobio resurs koji je tražio. Strelica od procesa ka resursu znači da proces čeka da dobije zahtijevani resurs.

Slika 40 prikazuje modeliranje potpunog zastoja pomoću grafova. U ovom primjeru proces P3 čeka na resurs R3, ali je R3 već dodijeljen procesu P4. Proces P4 ne može osloboditi resurs R3 zato što čeka na resurs R4 koji je zauzet od strane procesa P5. Proces P5 čeka na resurs R5, koga drži P6. P6 traži R6 koji je već dodijeljen procesu P3. Sva četiri procesa će vječno čekati.



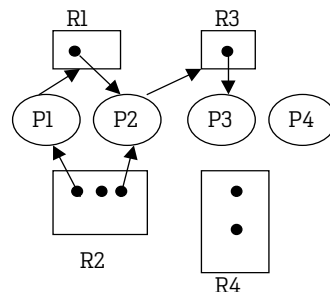
Slika 40 Modeliranje zastoja pomoću grafova

Sa grafa se vizualno može uočiti stanje zastoja:

- ukoliko graf ne sadrži nijednu zatvorenu konturu, zastoja sigurno nema
- ako graf sadrži najmanje jednu zatvorenu konturu to indicira mogućnost zastoja. Pošto ovi dijagrami ne prikazuju eksternu mogućnost prekida procesa ili oduzimanja resursa, oni samo predstavljaju mogućnost potpunog zastoja, ali ne i 100% dokaz da je u pitanju potpuni zastoj.

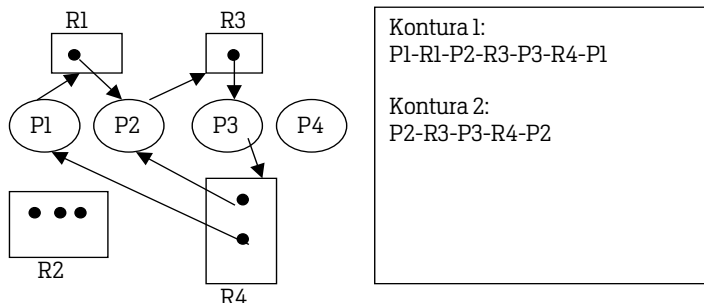
5.4.Graf zastoja sa više resursa iste vrste

Kada postoji više primjeraka resursa iste vrste, u blokovima koji predstavljaju resurse koriste se tačke koje predstavljaju pojedinačnu kopiju resursa. Slika 41 opisuje sistem u kome postoji jedna kopija resursa R1, tri kopije R2, jedna kopija R3 i dvije kopije R4. Proces P1 je zauzeo jednu kopiju R2 i čeka na oslobodjenje resursa R1. Proces P2 je zauzeo jednu kopiju R1 i jednu kopiju R2, a čeka na oslobodjenje resursa R3. Proces P3 je zauzeo R3. P4 ne zahtijeva niti drži resurse.



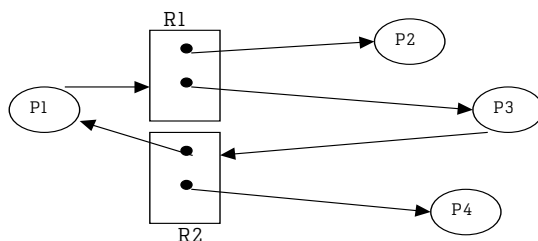
Slika 41 Modeliranje više resursa istog tipa

Slika 42 predstavlja potpuni zastoj. Na njoj su uočljive čak dvije zatvorene konture.



Slika 42 Potpuni zastoј sa više vrsta resursa

Sljedeći primjer (Slika 43) pokazuje da ako ima više kopija resursa iste vrste (i da je procesu svejedno koja će mu biti dodijeljena) tada pojava kružnog čekanja ne znači nužno potpuni zastoј. Čim proces P4 završi s radom ili dobrovoljno oslobodi resurs R2, tada se može ispuniti zahtjev za resursom R2 od strane procesa P3. Kada se P3 završi, on može osloboditi i R1. Postoji i drugi način razrješenja zatvorene konture, ako se prvo završi P2.



Slika 43 Primjer petlje bez potpunog zastoja

5.5.Strategije rješavanja potpunog zastoja

Ima jedna analogija između potpunog zastoja i reakcije pacijenata ili lekara na ljudske bolesti. Bolest se nekada ignoriše (npr. sitna oštećenja kože koja nisu vrijedna posjete lekaru), prepoznaje i liječi (npr. bakterijske infekcije), sprječava (vakcinacija i stil života) ili izbjegava (ne putuje se u područja endemskih epidemija). Ova četiri pristupa se javljaju i u kontekstu potpunog zastoja.

5.6.Nojev algoritam - ignoriranje problema

Najjednostavniji algoritam je nojev algoritam. Posmatrajući nojeve kako kopaju rupe u pijesku za jaja neki ljudi su zaključili da noj prosto ignoriše opasnosti savane, zavlacenjem glave u pijesak i praveći se da opasnost ne postoji. Tako se može gledati i na problem potpunog zastoja. Mišljenja o ovom "algoritmu" su podijeljena. Matematičari uglavnom smatraju da potpuni zastoји trebaju biti spriječeni bez obzira na cijenu, dok inženjeri porede koliko se često pojavljuje potpuni zastoј a koliko često sistem ne funkcioniše iz drugih razloga. Ako se potpuni zastoј uslijed zauzimanja resursa javi rijetko, a drugi hardverski ili softverski problemi se javljaju češće, ulaganja u otklanjanje ili sprječavanje potpunog zastoja se ne isplate. Čak se mogu smatrati i štetnim, jer stalne provjere da li je došlo ili bi

moglo doći do potpunog zastoja troše više hardverskih resursa. Stoga Unix i Windows do verzije XP, ignorišu ovaj problem, jer većini korisnika povremeno pojavljivanje potpunog zastoja neće smetati. Ali u sistemima realnog vremena sama pomisao da se softverski sistem može zagušiti je alarmantna.

Jedna od varijanti je i ne brinuti o potpunom zastoju u normalnom radu, jer algoritmi njegove detekcije troše previše resursa, ali prilikom razvoja operativnog sistema posvetiti pažnju i ovom problemu za kritične dijelove operativnog sistema. Tako na primjer, postoji program za verifikaciju drajvera za Windows koji se pokrene prilikom njihovih testiranja i upozorava na uzroke potpunog zastoja.

5.7.Detekcija i otklanjanje potpunog zastoja

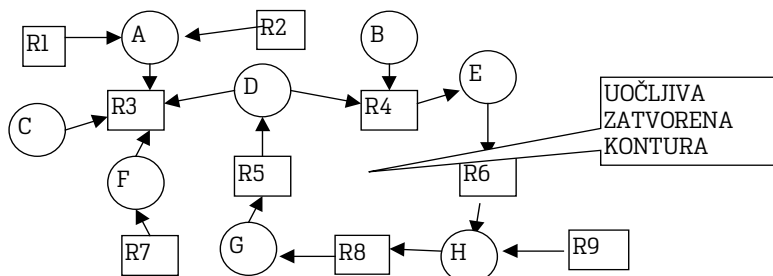
Umjesto ignorisanja, drugi pristup je prepoznavanje, a zatim otklanjanje potpunog zastoja. U ovom pristupu operativni sistem ne pokušava spriječiti pojavu potpunog zastoja. U ovom slučaju, potpuni zastoji su mogući, a sistem ih pokušava prepoznati i otkloniti. Stoga se mora osigurati algoritam detekcije (koji ispituje da li je došlo do zastoja) i algoritam za oporavak ako je došlo do potpunog zastoja.

5.8.Detekcija potpunog zastoja mjerenjem vremena

Jedan od najjednostavnijih i najčešće korištenih načina detekcije potpunog zastoja je uočiti da su neki procesi previše u stanju čekanja. Na primjer, modifikovati rutinu za čekanje na semafor tako da nakon isteka izvjesnog vremena izađe sa greškom uz pretpostavku da je u pitanju potpuni zastoj. Tako na primjer, Windows API funkcija *WaitForSingleObject()* ima kao drugi parametar maksimalan broj milisekundi koje se provedu u čekanju. Kada to vrijeme istekne, funkcija vraća vrijednost *WAIT_TIMEOUT*.

Mana ovog načina detekcije potpunog zastoja je što će i slučajevi koji nisu potpuni zastoj biti detektovani kao takvi, ukoliko je korištenje zauzetog resursa duže od vremena koje je navedeno kao maksimalno dopušteno vrijeme čekanja procesa na dati resurs. Ovaj pristup ne dokazuje koji skup procesa predstavlja potpuni zastoj.

5.9.Detekcija potpunog zastoja grafom alokacije resursa



Slika 44 Detekcija potpunog zastoja grafovima

Ako graf alokacije resursa sadrži jednu ili više zatvorenih kontura (Slika 44) sistem se vjerovatno nalazi u stanju potpunog zastoja.

U primjeru sa slike uočava se kontura koja odgovara potpunom zastoju u kome učestvuju procesi D, E, G i H.

Na papiru je lako prepoznati zatvorenu konturu, ali kako to uraditi u jezgri operativnog sistema? Algoritam koji to radi kreće od jednog izabranog čvora. Graf se tada posmatra kao stablo sa korijenom u tom čvoru uzimajući u obzir samo izlazne veze iz čvora i primjenjuje način prolaska kroz stablo algoritmom prvo dubinska pretraga. Ovaj algoritam u pseudo kodu izgleda ovako:

```
procedure IspitajČvor(Graf G, Čvor v):
```

```
  Označi Čvor v kao ispitivan
```

```
  Za svaku izlaznu vezu iz čvora v prema čvoru w
```

```
    Ako čvor w nije označen kao ispitivan Rekurzivno zovi IspitajČvor(G,w)
```

```
    Ako je čvor w označen kao ispitivan, to predstavlja da postoji kontura
```

Ako se ova rutina pozvana za izabrani čvor vrati bez indikacije da postoji kontura, i ako ovaj test prođe za sve početne čvorove, to znači da sistem nije u potpunom zastoju.

U primjeru (Slika 44) ako se krene iz čvora R1, tada rutina IspitajČvor će redom rekurzivnim pozivom ispitati R1, A i R3 i završiti. Ali ako se krene iz čvora D, ispitaće se redom D, R3, R4, E, R6, H, R8, G, R5 i uočiti da je D već posjećen. Prema tome, sistem je u mogućem potpunom zastoju.

5.10. Detekcija potpunog zastoja matričnom metodom

Kada postoji više kopija jednog resursa, koristi se algoritam baziran na matricama za prepoznavanje potpunog zastoja. Neka postoje procesi P_1 do P_n . Postoji m vrsta resursa. U svakom trenutku su neki resursi zauzeti. Vektor resursa $\mathbf{R}$ sadrži za svaki resurs i broj slobodnih kopija. Matrica alokacije $\mathbf{A}$ je dimenzija $n \times m$ kao i matrica novih zahtjeva $\mathbf{N}$. Element A_{ij} matrice $\mathbf{A}$ sadrži broj resursa klase j koje zauzima proces P_i . Analogno je N_{ij} broj resursa tipa j koje je proces i zahtijevao, ali mu još nisu dodijeljeni.

$$A = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \dots & \dots & \dots & \dots \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{bmatrix}, N = \begin{bmatrix} N_{11} & N_{12} & \dots & N_{1m} \\ N_{21} & N_{22} & \dots & N_{2m} \\ \dots & \dots & \dots & \dots \\ N_{n1} & N_{n2} & \dots & N_{nm} \end{bmatrix}, R = [R_1 \quad R_2 \quad \dots \quad R_m]$$

Definišimo poređenje vektora (relaciju $\leq$) tako da važi $\mathbf{A} \leq \mathbf{B}$, ako je svaki element vektora $\mathbf{A}$, manji ili jednak od elementa vektora $\mathbf{B}$ na odgovarajućoj poziciji.

Na početku algoritma su svi procesi neoznačeni. Kada se jedan proces označi, to znači da nije uključen u potpuni zastoj i može nastaviti s radom. Po završetku algoritma ostaju neoznačeni procesi koji učestvuju u nekom potpunom zastoju.

Algoritam za detekciju potpunog zastoja glasi:

1. Označi sve procese P_i čiji je i -ti red matrice $\mathbf{A}$ jednak nula-vektoru, jer taj proces ne koristi resurse.
2. Nađi bar jedan neoznačeni proces P_i , kod koga važi za red matrice $\mathbf{N}_i \leq \mathbf{R}$
3. Ako takav proces postoji, saberi red matrice $\mathbf{A}_i$ sa $\mathbf{R}$, i rezultat dodijeli vektoru $\mathbf{R}$. označi proces P_i , pa idi na korak 1.

4. Kada nema takvog procesa završi algoritam.

Sistem nije u potpunom zastoju ako su svi procesi označeni.

Primjer: Neka postoje tri procesa i 4 vrste resursa. Matrice su sljedeće:

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}, N = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}, R = [4 \quad 2 \quad 1 \quad 0]$$

Prvi proces ima red A_1 čiji su svi elementi jednaki 0, pa ga označavamo da nije učesnik u potpunom zastoju. Zahtjev drugog procesa (red $A_2 \leq R$) se može ispuniti, pa se vektor R sabere sa redom $A_2 = [2 \ 0 \ 0 \ 1]$, jer će po završetku njegove upotrebe resursa oni postati dostupni drugim procesima. Proces P_2 se također označi. Sada je $R = [6 \ 2 \ 1 \ 1]$. Preostao je P_3 . Važi da je $A_3 \leq R$ pa se označi i ovaj proces, a vektor R sabere sa A_3 . Sada je $R = [6 \ 3 \ 3 \ 0]$. Svi procesi su postali označeni i sistem nije u potpunom zastoju.

5.11.Oporavak od potpunog zastoja

Ako se koristio neki od načina za detekciju potpunog zastoja i potpuni zastoj je bio prepoznat, pa ga treba otkloniti. Pošto je potpuni zastoj nastao u situaciji kada neki proces traži resurs koji mu ne može biti dodijeljen, jer pripada drugom procesu, logično se uzima rješenje koje bi oslobodilo resurs oduzimanjem od jednog procesa ili prekidanjem procesa.

Rješenje da se **resurs oduzme od procesa** i dodijeli drugom procesu je izvodivo u samo nekim rijetkim situacijama. Za primjer, uzeće se situacija da je proces 1 zauzeo jedinicu trake A, preuzeo njenu strukturu u internu memoriju, i zatražio jedinicu trake B. Jedinicu trake B već drži proces 2, a još želi jedinicu trake A. Oduzme li se jedinica trake A od procesa 1, preuzeta struktura sadržaja trake je nevažeca, jer će je proces 2 promijeniti nakon toga. Stoga se ne može garantovati uspješan nastavak rada procesa 1. Ovaj pristup je pouzdan samo u slučajevima kada je tok izvođenja programa u potpunosti poznat (npr. *batch* sistemi čiji programi uvijek počinju alokacijom svih resursa koje će koristiti i završavaju njihovim oslobađanjem).

Puno jednostavnije rješenje je **prekidanje procesa**. Ovo prekidanje može raditi administrator sistema, kada on donosi odluku koji proces treba prekinuti, npr. Unix naredbom *kill*. Nakon ovoga zauzeti resursi procesa se oslobađaju i dodjeljuju drugim procesima koji su čekali na njih. Alternativno, odluku o izboru procesa koji će se prekinuti može donijeti i operativni sistem, na bazi kriterija kao što su prioritet procesa, vrsta zauzetog resursa, proteklo vrijeme itd. Mana pristupa je gubitak odrađenog posla prekinutog procesa. Ukoliko prekid jednog procesa nije doveo do oslobađanja njegovih resursa ili ako je nastalo više ciklusa čekanja na resurse, a oslobođen je samo jedan, treba prekinuti više procesa koji su u ciklusu čekanja na resurse ili čak sve njih.

Restart sistema ili podsistema se preporučuje kao rješenje potpunog zastoja u složenijim slučajevima. Primjeri su preporuke da se Java virtualna mašina ponovo pokrene ako višenitne aplikacije uđu u potpuni zastoj ili u distribuiranim sistemima

da se zaglavljeni čvorovi ponovo pokrenu. Slično, ako se potpuni zastoj desio između drajvera u jezgru, također je *restart* sistema jedino rješenje

Problem gubitka dosadašnjeg rada koji se javlja u rješenjima prekidanja procesa ili ponovnog pokretanja sistema se može djelomično ublažiti metodom oporavka **snimanjem kontrolnih tačaka**. U pojedinim trenucima se čuva stanje cijelog sistema, na primjer snima se memorija svih procesa na disk. Kada se desi potpuni zastoj, procesi se vraćaju u stanje snimljeno na onoj kontrolnoj tački prije nego su resursi zauzeti i nastavljaju, pa je izgubljen samo dio izvršavanja procesa. Pristup se obično ne koristi u operativnim sistemima, jer je snimanje cijele memorije sporo, a puna kontrolna tačka treba snimiti ne samo sadržaj memorije, nego i sadržaj diska, jer se i on mijenja u toku rada procesa. Ali u oblasti sistema za upravljanje bazama podataka, kod kojih se potpuni zastoji uslijed zaključavanja slogova često javljaju, ovaj pristup je vrlo popularan. U ovim sistemima se podaci baze podataka čuvaju u dva dijela: datotekama s podacima i **transakcijskim logovima**. Kontrolnu tačku predstavlja početak transakcije. Kada se zapisuju podaci, najprije se smještaju u transakcijske logove, i tek kada su sasvim potvrđeni prebacuju se u datoteke s podacima. Ako se desio zastoj, tada se gube samo podaci koji su u logu.

5.12. Izbjegavanje potpunog zastoja

Ako je u sistemu poznato koje će resurse proces zahtijevati, može se napraviti redoslijed opsluživanja zahtjeva tako da se izbjegne potpuni zastoj. Potpuni zastoj se može izbjeći, ali samo ukoliko sistem unaprijed posjeduje potrebne informacije. Te informacije su broj resursa koje je proces već zauzeo, ali i resursa koje će koristiti kasnije. Pri dodjeli raspoloživog resursa nije dovoljno prosto dodijeliti resurs procesu ako je on raspoloživ u tom trenutku, nego i paziti da li bi takva dodjela dovela do potpunog zastoja. Da to nije svejedno, vidi se na sljedećem primjeru (Slika 45a).

a)	Ima	Želi još
P1	4	7
P2	3	3
P3	3	6
Slobodno: 3		

b)	Ima	Želi još
P1	4	7
P2	4	2
P3	3	6
Slobodno: 2		

c)	Ima	Želi još
P1	5	6
P2	3	3
P3	3	6
Slobodno: 2		

Slika 45 Sigurno i nesigurno stanje

Od ukupno 13 kopija resursa, 3 su slobodne, a ostalih 10 raspoređeno između procesa, tako da je P1 već zauzeo 4 resursa, a P2 i P3 po 3 resursa. Svaki od procesa želi još resursa pored ovoga. Kome je sigurno dodijeliti? Neka je upravljač resursima dodijelio jednu kopiju resursa procesu P2 (Slika 45b). Može li se ovdje pronaći redoslijed dodjeljivanja resursa koji je siguran? Da, ako se preostala 2 resursa dodijele procesu P2. Kada on završi, oslobodiće 6 resursa (4 koja je već držao i 2 nova). Sada može da se zadovolji zahtjev za resursima procesa P3 koji traži upravo 6 resursa. Završetkom procesa P3, i proces P1 može da se izvršava jer će tada imati 9 raspoloživih resursa, a treba mu 7.

Slika 45c, međutim, pokazuje da bi bilo greška dodijeliti resurs procesu P1 umjesto P2, jer se više neće imati dovoljno resursa da se zadovolje zahtjevi niti jednog procesa.

U slučajevima a) i b) postoji redoslijed dodjele resursa procesima takav da neće biti potpunog zastoja. Stanje u kome postoji takav redoslijed dodjele resursa koji

neće dovesti do potpunog zastoja se zove **sigurno stanje**. Ako takav redoslijed ne postoji stanje nije sigurno, kao što je slučaj c).

5.13. Bankarov algoritam

Sretna okolnost za banke je što različitim subjektima novac treba u različito vrijeme. Distributeri plina imaju najveće prihode u januaru, građevinari u maju, hotelijeri u julu, a privatne škole u septembru i tada polažu veće iznose na žiro račune da koriste novac za kasnije. Banka onda u međuvremenu može davati kredite drugim subjektima kojima je novac potreban, ali moraju uvijek paziti pri davanju kredita da li bi ti krediti doveli do ugrožavanja likvidnosti banke. Bankarov algoritam (engl. *Banker's Algorithm*) predstavlja algoritam za izbjegavanje potpunog zastoja. Ovaj algoritam je sličan algoritmu za detekciju potpunog zastoja, ali se izvršava u različitom trenutku. Bankarov algoritam, u verziji kada postoji više vrsta resursa i više kopija jednog resursa, baziran je na matricama za prepoznavanje potencijalnog potpunog zastoja među procesima P_1 do P_n . Neka postoje procesi P_1 do P_n . Postoji m vrsta resursa. U svakom trenutku su neki resursi zauzeti. Vektor resursa $\mathbf{R}$ sadrži za svaki resurs i broj slobodnih kopija R_i . Matrica alokacije $\mathbf{A}$ je dimenzija $n \times m$ kao i matrica novih zahtjeva $\mathbf{N}$. Element A_{ij} matrice $\mathbf{A}$ sadrži broj resursa tipa j koje zauzima proces P_i . Analogno je N_{ij} broj resursa tipa j koje je proces i zahtijevao, ali mu još nisu dodijeljeni.

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \dots & \dots & \dots & \dots \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{bmatrix}, \mathbf{N} = \begin{bmatrix} N_{11} & N_{12} & \dots & N_{1m} \\ N_{21} & N_{22} & \dots & N_{2m} \\ \dots & \dots & \dots & \dots \\ N_{n1} & N_{n2} & \dots & N_{nm} \end{bmatrix}, \mathbf{R} = [R_1 \quad R_2 \quad \dots \quad R_m]$$

Definišimo poređenje vektora (relaciju $\leq$) tako da važi $\mathbf{A} \leq \mathbf{B}$, ako je svaki element vektora $\mathbf{A}$, manji ili jednak od elementa vektora $\mathbf{B}$ na odgovarajućoj poziciji.

Na početku algoritma su svi procesi neoznačeni. Algoritam detekcije sigurnog stanja izgleda ovako:

1. Nađi bar jedan neoznačeni proces P_i , kod koga važi za red matrice $\mathbf{N}_i \leq \mathbf{R}$
2. Ako takav proces postoji, saberi red matrice $\mathbf{A}_i$ sa $\mathbf{R}$, i rezultat dodijeli vektoru $\mathbf{R}$. označi proces P_i , pa idi na korak 1.
3. Kada nema takvog procesa završi algoritam.

Ako su na kraju svi procesi označeni, sistem je u sigurnom stanju i postoji bar jedan redoslijed izvršavanja procesa i dodjele resursa koji ne vodi potpunom zastoju. Ali ako ima neoznačenih procesa sistem nije u sigurnom stanju.

Ovaj algoritam se koristi samo u veoma malom broju sistema jer je beskoristan u praksi za sisteme gdje broj procesa nije konstantan, npr. ako se oni naknadno pokreću. Osim toga rijetko se unaprijed zna koliko će resursa biti potrebno procesu.

5.14. Sprječavanje potpunog zastoja

U manjim sistemima može se spriječiti potpuni zastoj između korisničkih procesa ili nekih dijelova jezgra ako se onemogućiti neki od četiri uslova za nastanak potpunog zastoja. Ako neki od njih nije ispunjen, tada nemamo potpuni zastoj izazvan zauzimanjem resursa.

Mnogi operativni sistemi za pojedine resurse koriste specijalizovane procese koji čine da **uslov uzajamnog isključivanja** bude neispunjen. Ako je neki proces zauzeo resurs prije svih ostalih i zauzeo ga u cijelosti samo za sebe, a ne traži druge resurse koje bi neki procesi još tražili, taj proces neće biti uzrok potpunog zastoja. Najbolji primjer za to je printer *spooler* (*lpd* u Unix-u, *spoolsv* Windows-u). To je jedini program koji koristi printer i on brine o redoslijedu štampanja dokumenata. Nijedan drugi program ne može zauzeti printer i stoga printer neće uticati na potpuni zastoj. Slično ovome, sistemi za upravljanje bazama podataka su jedini programi koji kao resurs koriste datoteke u kojima se podaci čuvaju. Ali, specijalizovani program ne može se koristiti na primjer za dijeljene varijable između dva procesa ili niti.

Drugi uslov potpunog zastoja, da su **neki resursi neoduzivi** je dat sa razlogom. Npr., CD pisač i štampač su po prirodi takvi uređaji da se ne mogu oduzeti u proizvoljnom trenutku, nego samo u trenucima kada upis na njih nije počeo ili je već završen, jer će u protivnom CD-ovi ili papir biti upropašteni. Stoga, sprječavanje ovog uslova uglavnom nije moguće, osim ako se neoduzivi resurs zamijeni srodnim oduzivim: npr. fizička magnetna traka zamijeni simuliranom trakom u datoteci.

Ako možemo spriječiti da proces čeka na resurse dok druge resurse drži zauzetim možemo spriječiti potpuni zastoj, kroz sprječavanje **stanja prisvajanja i čekanja**. Umjesto da procesi uzimaju jedan po jedan resurs i tako pri zahtijevanju dođu do nekog već zauzetog, jedno rješenje je navesti listu resursa koji se koriste i pri startu programa pokuša zauzeti sve njih. Ako bar jedan od njih nije slobodan, proces se neće pokrenuti. Primjer gdje bi se moglo ovako raditi vidi se u strukturi programa pisanih u programskom jeziku COBOL. U dijelu programa ENVIRONMENT DIVISION navedeni su svi periferni uređaji i datoteke koje program koristi i generirani izvršni program može sadržati listu resursa koje proces traži. Pošto većina drugih programskih jezika nije tako dizajnirana, primjenjuje se pristup **dvofaznog zaključavanja**. U prvoj fazi proces zauzima resurs po resurs, ali ako naiđe na neki već zauzeti, oslobađa sve resurse koje je zauzeo i ponovo pokušava sa zauzimanjem resursa. Tek kada je uspio zauzeti sve resurse, proces počinje sa drugom fazom, korištenjem resursa. Oba pristupa sprječavanja uslova stanja prisvajanja i čekanja imaju dvije ozbiljne mane. Prvi je što su neupotrebljivi kada procesi ne znaju unaprijed koje će resurse koristiti, npr. ime datoteke koja se obrađuje se unosi od strane korisnika nakon starta programa. Drugi nedostatak je što zauzeti resursi na početku programa se možda još neće koristiti, ali ne mogu biti korišteni ni u drugim procesima koji ih trebaju.

Pogled na početni primjer sa kopiranjem sa trake na traku daje jedan način za sprječavanje **kružnog čekanja**. Problem u tom primjeru je što je jedna nit prvo zauzimala traku A, a zatim traku B, a druga prvo traku B a zatim traku A. Kada bi oba procesa tražila prvo traku A, pa onda traku B samo jedan od njih bi dobio oba resursa i mogao raditi i ne bi bilo potpunog zastoja. Ovo bi se moglo generalizovati u pravilo redoslijeda uzimanja resursa, da svaki resurs ima svoj redni broj i da je dopušteno samo određenim redoslijedom, da se prvo alociraju resursi sa najmanjim rednim brojem, a onda ide po rastućim rednim brojevima. Pristup je primjenjiv u sistemima koji imaju malo resursa i koji se neće mijenjati, na primjer, u kontrolnom

medicinskom sistemu, ali ne i za sisteme opće namjene koji omogućavaju pojavu novih resursa u budućnosti.

Zaključak je da nijedan od pristupa rješavanju problema potpunog zastoja ga ne rješava u potpunosti. Neka rješenja kvare komfor programiranja, ili neefikasno koriste resurse, a neka se ne mogu primijeniti u određenoj vrsti sistema. Ipak, problem je uspješno minimiziran u praksi.

5.15.Windows API za potpune zastoje

Prije Windows XP, problem potpunog zastoja je ignorisan. Od Windows Vista uvedene su tri funkcije za detekciju potpunog zastoja u višenitnim programima. Funkcije su date u sljedećoj tabeli.

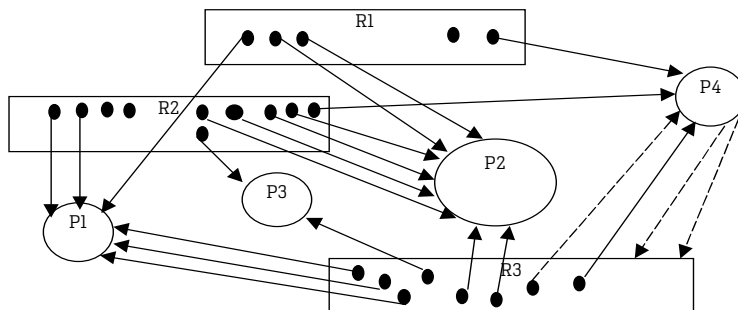
Funkcija	Značenje
OpenThreadWaitChainSession(dwFlags, pfCallback)	Otvora sesiju za dobivanje liste niti koje su u čekanju
GetThreadWaitChain(hWctHandle, pContext, dwFlags, dwThreadId, lpNodeCount, pNodeInfoArray, lpIsCycle)	Dobavlja lanac čekanja na niti
CloseThreadWaitChainSession(hWctHandle)	Zatvara sesiju za dobivanje liste niti koje su u čekanju

5.16.Zadaci

Zadatak 1. Neka postoje četiri procesa i tri vrste resursa. Resursi su dodijeljeni procesima kao u matrici A. Pored toga raspoloživo je još $R=(1,2,1)$ resursa. Nacrtati dijagram dodjele resursa procesima. Zatim do crtati stanje ako proces p4 zatraži još 3 instance resursa R3.

A	R1	R2	R3
P1	1	2	3
P2	2	4	2
P3	0	1	1
P4	1	1	1

Rješenje: Već dodijeljeni resursi su prikazani punim, a naknadno zahtijevani isprekidanim linijama. Uočimo da se jedan dostupni resurs R3 dodijelio u naknadnom zahtjevu procesa P4, a ostala dva su na čekanju.



Zadatak 2. Neka postoji 3 vrste resursa i 4 procesa. Matrice alokacije (A), maksimalnih zahtjeva (M) i vektor raspoloživosti (R) su

A	R1	R2	R3
p1	0	5	1
p2	1	0	1
p3	2	1	5
p4	3	5	4

M	R1	R2	R3
p1	0	7	3
p2	1	0	1
p3	4	3	5
p4	4	7	4

$$R = (2, 1, 0)$$

Pokazati da li je sistem u sigurnom stanju.

Rješenje: Matrica potreba N se dobiva oduzimanjem matrica maksimalnih zahtjeva i matrice alokacije. $N = M - A$

N	R1	R2	R3
P1	0	2	2
P2	0	0	0
P3	2	2	0
P4	1	2	0

Vidi se da se može izvršiti proces P2, pošto je red $N_2 \leq R$ i nakon njegovog završetka resurse iz reda A_2 možemo dodijeliti vektoru R koji sada iznosi $R = (3, 1, 1)$. Ali nakon ovoga ni jedan proces ne može ispuniti zahtjev za resursom R2, pa sistem nije u sigurnom stanju.

Zadatak 3. Neka postoji 3 vrste resursa i 4 procesa. Matrice alokacije (A), maksimalnih zahtjeva (M) i vektor raspoloživosti (R) su:

A	R1	R2	R3
p1	1	4	0
p2	6	2	2
p3	5	3	5
p4	6	6	1

M	R1	R2	R3
p1	2	6	0
p2	8	3	3
p3	5	4	6
p4	7	8	2

$$R = (2, 1, 2)$$

Pokazati da li je sistem u sigurnom stanju.

Rješenje: Matrica potreba N se dobiva oduzimanjem matrica maksimalnih zahtjeva i matrice alokacije. $N = M - A$

N	R1	R2	R3
P1	1	2	0
P2	2	1	1
P3	0	1	1
P4	1	2	1

Vidi se da se ne može izvršiti proces P1, ali može P2, i nakon njegovog završetka resurse iz matrice A možemo pridodati vektoru R.

Nakon izvršenja P2, $R = (8, 3, 4)$

Može se izvršiti P1 i nakon njega $R = (9, 7, 4)$

Nakon izvršenja P3, $R = (14, 10, 9)$

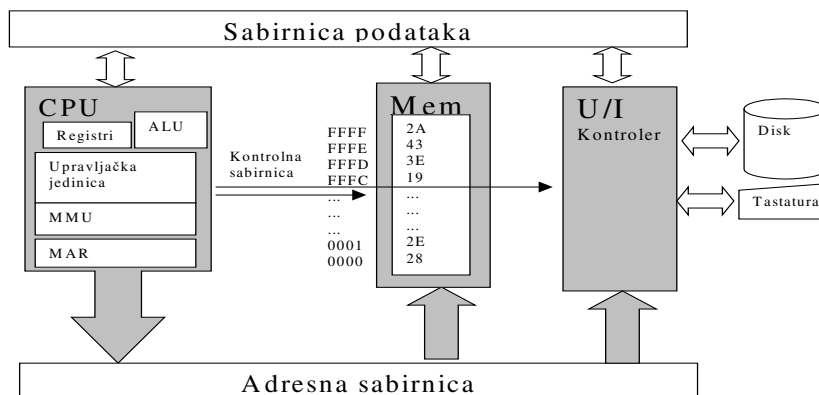
Nakon izvršenja P4, $R = (20, 16, 10)$

Mogu se izvršiti sva četiri procesa. Sistem je u sigurnom stanju.

6.UPRAVLJANJE LINEARNOM MEMORIJOM

6.1.Memorijske adrese

Slika 46 predstavlja pojednostavljen prikaz računarskog sistema. Procesor (CPU) izvršava instrukcije koje se nalaze u centralnoj memoriji. Memorija je podijeljena na ćelije veličine obično jedan bajt, mada postoje i sistemi kod kojih se veličina ćelije razlikuje od jednog bajta. Svaka memorijska ćelija ima svoj jedinstveni redni broj koji se zove adresa i koji služi za pristup pridruženoj memorijskoj ćeliji. Kada memorijski adresni registar (MAR) koji se nalazi u procesoru sadrži određenu binarnu vrijednost, tu vrijednost vide i memorijska integrirana kola i ona predstavlja adresu memorijske ćelije kojoj se pristupa. Zavisno od stanja kontrolne sabirnice, memorijska kola mogu da ne reaguju, mogu da izbace vrijednost prozvana memorijske ćelije na sabirnicu podataka ili da upišu na tu memorijsku ćeliju vrijednost koja se nalazi na sabirnici podataka.



Slika 46: Arhitektura računarskog sistema

Pri izvršavanju programa, procesor proziva odgovarajuće memorijske lokacije kako je navedeno u instrukcijama programa. U sasvim jednostavnim sistemima memorijska lokacija koja je navedena u instrukciji programa je ujedno ona koja se pojavljuje na adresnoj sabirnici. Na primjer, na računarima sa procesorom Z80 (korišten u CP/M sistemima) instrukcija sa heksadecimalnim kodovima 38 4E 12 (instrukcija LD A,(&124E)) će na adresnu sabirnicu preko MAR postaviti vrijednost heksadecimalno 124E (binarno 0001001001001110), sačekati da se vrijednost memorijske ćelije ove lokacije pojavi na sabirnici podataka i upisati tu vrijednost u registar A. U ovom slučaju, memorijska lokacija koja je bila navedena u instrukciji je upravo memorijska lokacija koja je prozivana. Moderniji sistemi, međutim, imaju jedinicu za upravljanje memorijom (MMU), koja će obaviti neku transformaciju između memorijske lokacije koja je navedena u instrukciji i one koja se pojavljuje na MAR. O ovim transformacijama biće još riječi.

Neke memorijske lokacije ne pripadaju RAM-u, nego njihove adrese na adresnoj sabirnici osluškuje U/I kontroler i komunicira sa perifernim uređajima pridruženim toj adresi. Ako je neka adresa pridružena npr. ulazu tastature, tada će

čitanje podatka sa te adrese dovesti do prepoznavanja koda pritisnutog tastera. Preko U/I kontrolera se priključuju i diskovi. Iako i oni čuvaju podatke, pa ih u širem smislu smatramo memorijom, način rada sa njima je znatno drugačiji od rada sa memorijom koja je dostupna direktnim pristupom njenim adresama.

Prema dostupnosti memoriju dijelimo na primarnu, sekundarnu, tercijarnu i *off-line* memoriju. Memorija kojoj se pristupa bez korištenja U/I kontrolera direktno preko adresiranja memorijskih lokacija ili podataka u samom procesoru se zove **primarna memorija**. Primarna memorija se tipično sastoji od tri tipa memorije: procesorski registri, keš memorija i glavna (interna) memorija. Registri nemaju memorijske adrese, nego se nalaze u samom mikroprocesoru. Predstavljaju najbržu memoriju i nad njihovim sadržajem se obavljaju aritmetičke i logičke operacije. Keš memorija se nalazi također u mikroprocesoru, ali joj se pristupa koristeći memorijske lokacije, a poseban uređaj brine o tome da se češće korištene memorijske ćelije čuvaju pored glavne memorije i u keš memoriji. Glavna memorija može biti izmjenjiva (RAM – *Random Access Memory*) i neizmjenjiva (ROM – *Read Only Memory*). Operativni sistemi obično brinu o organizaciji RAM memorije, dok organizacija ROM memorije nekad jeste, a nekad nije posao operativnog sistema. Vrijeme pristupa primarnoj memoriji se mjeri u nanosekundama. Kapacitet registara je samo nekoliko desetina bajta, keš memorije su veličine nekoliko megabajta, a glavne memorije reda gigabajta.

Sekundarna memorija zahtijeva upotrebu U/I kontrolera, ali je i dalje dostupna bez dodatne ljudske ili mašinske intervencije. Tipični predstavnik ovakve memorije je tvrdi disk. Kapacitet joj je znatno veći od primarne memorije, reda terabajta. Ali brzina pristupa je znatno manja i mjeri se u mikrosekundama (SSD diskovi) ili milisekundama (tvrdi diskovi).

Tercijarna memorija nije stalno spojena na računare. Riječ je o jedinicama trake od kojih svaka ima veliki kapacitet (npr. više terabajta) i čuvaju rijetko korištene podatke, npr. stare snimke video zapisa. Podacima se i dalje pristupa automatski, tako da robot odabere potrebnu traku i ubaci je, što traje nekoliko sekundi.

Off-line **memorija** zahtijeva ljudsku intervenciju za pristup na ubacivanju medija: CD-ovi, diskete, DVD-ovi, *BlueRay*, USB diskovi i slično. Kapacitet i brzina pojedinačnog medija su manji od kapaciteta sekundarnih memorija, vrijeme traženja može se mjeriti i minutama (možda i danima ako se ne zna tačno gdje je traženi podatak), ali zbog njihovih malih dimenzija i izmjenjivosti ukupna količina podataka koja se može smjestiti je proizvoljna (npr. u 1 m³ se može ubaciti 30000 četveroslojnih DVD diskova od kojih je svaki kapaciteta 12,5 GB.)

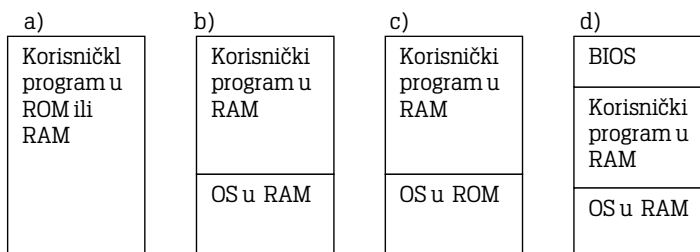
U ovom poglavlju će prvenstveno biti riječi o internoj memoriji i njenom odnosu sa operativim sistemom. Procesor može izvršavati samo procese koji su u internoj memoriji. Upoznaćemo različite načine organizacije memorije u operativnim sistemima od jednostavnijih ka složenijim.

6.2.Konfiguracija memorije u jednoprocесnim sistemima

Prvo će se upoznati sistemi kod kojih je memorijska adresa koja se navodi u programima ista kao i fizička adresa. Kod ovog pristupa moguće je u memoriji imati samo jedan korisnički program.

Najjednostavniji način je da u internoj memoriji postoji samo jedan proces (Slika 47 a). Tako se programiralo pedesetih godina prošlog vijeka na *mainframe* sistemima, prvom kućnom računar u Altair 8800, i ponovo na prvim konzolama za igru kao što je Atari 2060, pa i danas u nekim sistemima razvijenim iz hobija ili specijalizovanim ugrađenim sistemima. Korisnik ubaci posebnu ploču ili čip sa programom (u slučaju Altaira 8800 još teže: ukuca prekidačima bit po bit program u mašinskom jeziku) i taj program tada dobiva apsolutnu kontrolu nad računarom. Program ne mora da koristi operativni sistem ni drajvere, nego se oslanja se isključivo na sebe. U svaki program se mora ugraditi kôd na niskom nivou koji pristupa perifernim uređajima.

Već od prvih *batch* sistema, i kućnih računara, a kasnije i ugrađenih sistema situacija je olakšana uvođenjem operativnog sistema, sa kojim jedan korisnički program dijeli memoriju. Kada se računar uključi, učitava se operativni sistem koji sadrži potrebne drajvere koje više ne treba ugrađivati u svaki program, kao i neki komandni interpreter (u slučaju MacOS grafička školjka) koji služi za pokretanje programa. Pri definiranju pravila operativnog sistema odredi se koje memorijske adrese smije koristiti korisnički program, a koje su određene za operativni sistem. Na primjer, u CP/M sistemima je bilo definisano da korisnički programi moraju početi od fizičke adrese (heksadekadecimalno) 0100 i na većini sistema završiti na adresi DC00, dok ostatak memorije pripada operativnom sistemu. Kada se korisnički program završi, njegova memorija postaje slobodna za drugi program, a izvršavanje se prepušta ponovo komadnom interpreteru.



Slika 47 Konfiguracije memorije kod jednoprocесnih sistema

Tokom razvoja jednoprocесnih sistema pojavila su se još tri pristupa za raspodjelu memorije između operativnog sistema i korisničkog procesa.

U drugom pristupu i operativni sistem i korisnički program su u RAM-u, memoriji koja se može čitati i pisati (Slika 47 b), samo u različitim područjima. Model je bio upotrebljavan u prvim mikroračunarima tokom rane faze razvoja operativnog sistema, dok je on još pun grešaka. Problem pristupa je u unosu operativnog sistema nakon nestanka električne energije, jer ako nema ROM memorije, dobar dio

operativnog sistema bi se morao unositi preko specijalne komandne ploče bit po bit u mašinskom jeziku.

Treći pristup se upotrebljava u ugrađenim sistemima, kućnim računarima 80'ih (Apple II, ZX Spectrum, C64) i mnogim današnjim džepnim računarima. OS se nalazi u ROM-u (memorija koja se može samo čitati) na nižim adresama memorije a korisnički program na višim adresama (Slika 47 c). Na nekim sistemima je obrnuto, ROM je na višim adresama memorije. Ovakvi sistemi su vrlo robusni. Softverski se ne može oštetiti operativni sistem, koji je otporan na nestanak električne energije. Mana pristupa je u nefleksibilnosti: otkrivene greške u operativnom sistemu se ne mogu popraviti niti sistem proširiti novim mogućnostima, (npr. podrškom za nove tipove štampača) bez intervencije u servisu na matičnoj ploči.

Četvrti pristup, korišten u CP/M i MS DOS sistemima u ROM stavlja samo osnovni skup funkcija za rad sa uređajima i *boot* program koji sa diska učitava operativni sistem u RAM. Po cijenu sporijeg starta operativnog sistema, on je sada izmjenjiv. Ovakva ROM memorija se zove BIOS (*Basic Input Output System*). OS se oslanja na BIOS, a korisnički programi mogu pozivati i funkcije OS-a i funkcije BIOS-a. (Slika 47 d).

6.3.Swapping

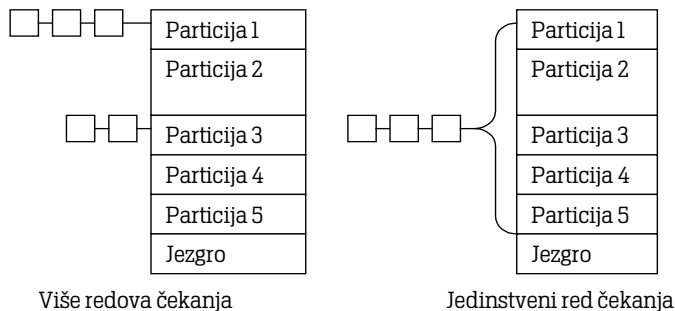
Dalje poboljšanje jednog procesnih sistema može se postići uvođenjem suspendovanja procesa. U memoriji se nalazi stalno jedan proces, ali se ne mora čekati njegovo potpuno završavanje da se memorija dodijeli drugom procesu. Umjesto toga, kada se želi preći na drugi proces, sadržaj kompletnog procesa se snima na disk a drugi proces ubacuje u memoriju iz njegove programske datoteke ili ranije snimljenog sadržaja. Kada se želi vratiti na prvi proces, sada se taj proces ne mora izvršavati od samog njegovog početka, nego se učitava sa diska njegov ranije snimljeni sadržaj i program nastavlja izvršavanje od mjesta gdje je stao.

Tehnika koja snima na disk sadržaj memorije cijelog procesa i učitava ga nazad kada je potrebno se zove *swapping* (izmjena). Ova tehnika se može koristiti i u višeprocensnim sistemima.

6.4.Višeprocensni sistemi sa fiksnim memorijskim particijama

Ako se ima dovoljno memorije, može se u memoriju smjestiti istovremeno više procesa. Time se postiže brže prebacivanje između procesa u odnosu na primjenu *swapping* kod jednog procesnih sistema, jer se ne gubi vrijeme na snimanje i učitavanje procesa na disk. Prvo će se razmotriti najjednostavnije proširenje modela sa slike (Slika 47 d) na višeprocensni sistem. Jedan dio memorije se rezerviše za BIOS i OS, a dio predviđen za korisnički program se podijeli na više manjih dijelova, **particija**, ako je moguće različitih veličina. U svaku particiju će se učitati proces koji je spreman za izvršavanje. U jednostavnijem slučaju, broj particija i njihova veličina su unaprijed poznati, kao i memorijske lokacije na kojima počinje pojedina particija. Ovaj pristup je primijenjen na *batch* operativnom sistemu OS/360 MFT.

Pošto ima više potencijalnih procesa za izvršavanje od raspoloživih particija, procesi moraju biti u redovima čekanja da dobiju svoju particiju. Ovi redovi čekanja su slični redovima čekanja na procesor. Za organizaciju redova čekanja postoje dvije mogućnosti (Slika 48).



Slika 48 Redovi čekanja kod multiprogramiranja s fiksnim particijama

1. Svaka particija ima svoju listu čekanja. Kod ovog pristupa programi mogu biti unaprijed pripremljeni da se izvršavaju na adresama koje odgovaraju određenoj particiji i veličini date particije.
2. Postoji jedinstveni red čekanja za sve particije. Kada neka particija postaje slobodna, upravljač memorije pregleda red čekanja i bira slijedeći proces iz liste čekanja. Može se birati prvi proces koji može da stane u oslobođenu listu čekanja (što je brže i pravednije), najveći proces iz liste čekanja koji može da stane u oslobođenu particiju (što bolje iskorištava memoriju) ili kombinacija ova dva pristupa.

Jedina prednost ovog pristupa organizaciji memorije je jednostavnost. Mana je što će se često dešavati da su kraći procesi smješteni u particiju koja je veća od njih, ali dio memorije koji predstavlja razliku između veličine particije i veličine procesa ostaje neiskorišten. U memoriji se istovremeno može nalaziti samo onoliko procesa koliko ima particija. Proces koji su veći od najveće particije se ne mogu nikako pokretati, čak i ako su sve particije prazne i njihova ukupna veličina je veća od veličine tog procesa.

6.5.Statička relokacija

Slučaj višeprocenih sistema sa fiksnim particijama sa jedinstvenim redom čekanja otvara novo pitanje. Neka program sa diska treba učitati u memorijsku particiju koja mu je dodijeljena. Program je dizajniran da se izvršava u particiji koja počinje od adrese 1000h i na određenom mjestu pristupa lokaciji 123Ah npr. instrukcijom LD A,(&123A) na Z80 procesoru. Ali, kada je proces nakon čekanja u redu na particije došao da se izvršava, ispostavilo se da je njegova prirodna particija zauzeta, ali je slobodna particija koja počinje od adrese 3000h. Program punilac (*loader*) sada mora da ima tabelu sa svim mjestima u programu koje treba promijeniti da bi se program izvršavao u drugoj particiji. Sva takva mjesta treba sabrati sa razlikom između početnih adresa dodijeljene i preferirane particije, pa bi

u datom primjeru instrukcija postala LD A, (&323A). Ovaj postupak se zove **statička relokalacija**, može se obaviti samo jednom, isključivo pri pokretanju procesa (ne i kada bi još bio potreban, npr. prilikom vraćanja procesa koji je *swapping*-om snimljen na disk) i razmjerno je spor, ali nekad jedini moguć.

6.6. Dinamička relokalacija

Kada bi se razdvojile adrese memorijskih lokacija koje su navedene u programu i memorijske adrese koje se zaista šalju memorijskim integriranim kolima, a proces transformacije između njih obavljao hardverski, dobio bi se daleko brži i fleksibilniji sistem. Adresa koja se navodi u procesorskoj instrukciji se zove logička adresa, a adresa same memorijske ćelije je fizička adresa. Logičku adresu također nazivaju i virtualnom adresom. Skup svih logičkih adresa unutar jednog procesa naziva se logički ili virtualni adresni prostor. Skup svih fizičkih adresa unutar računarskog sistema naziva se adresni prostor. Mapiranje između ova dva prostora obavlja se pomoću hardverske jedinice za upravljanje memorijom, skraćeno MMU (engl. *Memory Management Unit*). Relokacija koja se koristi uz pomoć MMU se zove **dinamička relokalacija** i u suštini se obavlja tokom cijelog izvršenja programa. Najjednostavniji način mapiranja između logičkog i fizičkog adresnog prostora je upotrebom **relokacionog registra**, koji određuje početnu fizičku adresu procesa. Svaka logička adresa koju zahtijeva proces sabira se sa vrijednošću relokacijskog registra i taj zbir predstavlja fizičku adresu. Primjer procesora koji je imao upravo ovakav relokacijski registar je CDC 6600, gdje se relokacijski registar zove RA. Ako korisnički program počinje od logičke adrese 0, tada svaka logička adresa predstavlja adresu od početka particije. Ako je proces *swapping* tehnikom bio prebačen na disk i treba da se učita na drugu fizičku lokaciju u odnosu na onu kada je bio prebačen, dovoljno ga je učitati i prilikom prepuštanja procesora tom procesu postaviti relokacijski registar na novu vrijednost.

Uz relokacijski registar može se dodati i **limit registar** koji predstavlja najveću dopuštenu fizičku adresu kojoj se može pristupiti. Na ovaj način se rješava i problem zaštite između procesa, da proces ne može pristupiti memoriji drugih procesa ili samog jezgra. Kada je izračunata fizička adresa kojoj proces želi pristupiti (sabiranjem logičke adrese i relokacijskog registra), jednostavno se hardverski uspoređuje sa sadržajem limit registra. Ako se pristupa adresi koja je manja ili jednaka od limit registra, pristup se normalno obavlja, a u protivnom se ignoriše ili generiše skok u jezgro operativnog sistema koje obrađuje nelegalni zahtjev.

6.7. Alokacija memorije sa promjenljivim particijama

U sistemima sa fiksnim particijama, broj, veličina, i fizička lokacija početka svih particija su određeni unaprijed od strane sistem operatora, prilikom pokretanja operativnog sistema. Jedna od mana pristupa se vidi u sljedećem primjeru. Postoji 5 particija, jedna od 50 kilobajta u kojoj je proces velik 40 kilobajta i 4 particije od po 10 kilobajta sa procesima velikim po 8 K. U redu čekanja je 10 procesa velikih po 1 kilobajt. Oni se ne mogu staviti u memoriju jer su particije zauzete, iako 18 kilobajta memorije stoji neiskorišteno. Ovaj problem se zove problem **interne fragmentacije**. Interna fragmentacija se događa ako se procesu dodjeli veći blok memorije od onog koji je zahtijevao. Ostatak bloka memorije ostaje neiskorišten ne samo tom procesu,

nego i ostalim procesima u sistemu, sve dok proces ne oslobodi memoriju. Ovakva fragmentacija se dešava u sistemima s fiksnim particijama.

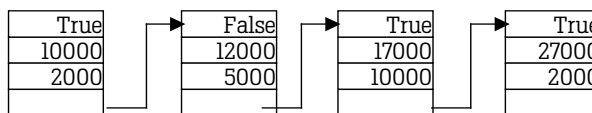
Logično unapređenje i bolje iskorišćenje memorije bi bilo da svaki proces zauzme onoliko kontinualne memorije koliko mu treba, ni manje ni više. Kod **promjenljivih particija** ne zna se unaprijed ni broj, ni veličina, ni lokacija particija, nego se ti parametri određuju dinamički prilikom pokretanja i izvršenja procesa. Particije se kreiraju i oslobađaju po potrebi. Ovaj pristup je korišten u mnogim starijim operativnim sistemima, npr. Unix sistemima na računarima koji nemaju podršku za straničenje. Pored toga koristi se u gotovo svim operativnim sistemima za alokaciju logičkih adresa memorije unutar procesa (čak i ako operativni sistem ima napredniji pristup za transformaciju logičkih u fizičke adrese), na primjer za realizaciju *malloc()* funkcije u C.

Ali svaka veća fleksibilnost u operativnim sistemima zahtijeva kompleksnije algoritme za realizaciju. Prije svega, potrebno je voditi računa gdje particije počinju i gdje se završavaju, te da li su slobodne ili zauzete. Prvi pristup za evidenciju particija je **upotreba ulančane liste**. Ova lista se čuva u memorijskom području jezgra operativnog sistema ili u zoni pojedinog procesa ako se algoritam koristi za alokaciju memorije unutar procesa. Svaka alocirana ili slobodna memorijska particija vezana je za jedan element ove ulančane liste. Elementi ulančane liste su slogovi određene strukture (Slika 49).

Zauzeta	početna adresa	veličina	Sljedeći
---------	----------------	----------	----------

Slika 49 Struktura sloga za upravljanje memorijom

Prvo polje sloga označava zauzetost memorijske particije, logička vrijednost koja predstavlja da li je proces zauzeo particiju. Drugo polje sadrži početnu adresu memorijske particije koju opisuje dati slog. Treće polje veličinu opisane memorijske particije, a četvrto polje sadrži pokazivač na sljedeći slog. Slika 50 prikazuje pridruživanje zauzetih i slobodnih dijelova memorije ulančanoj listi.



Slika 50 Upravljanje slobodnim prostorom preko ulančane liste

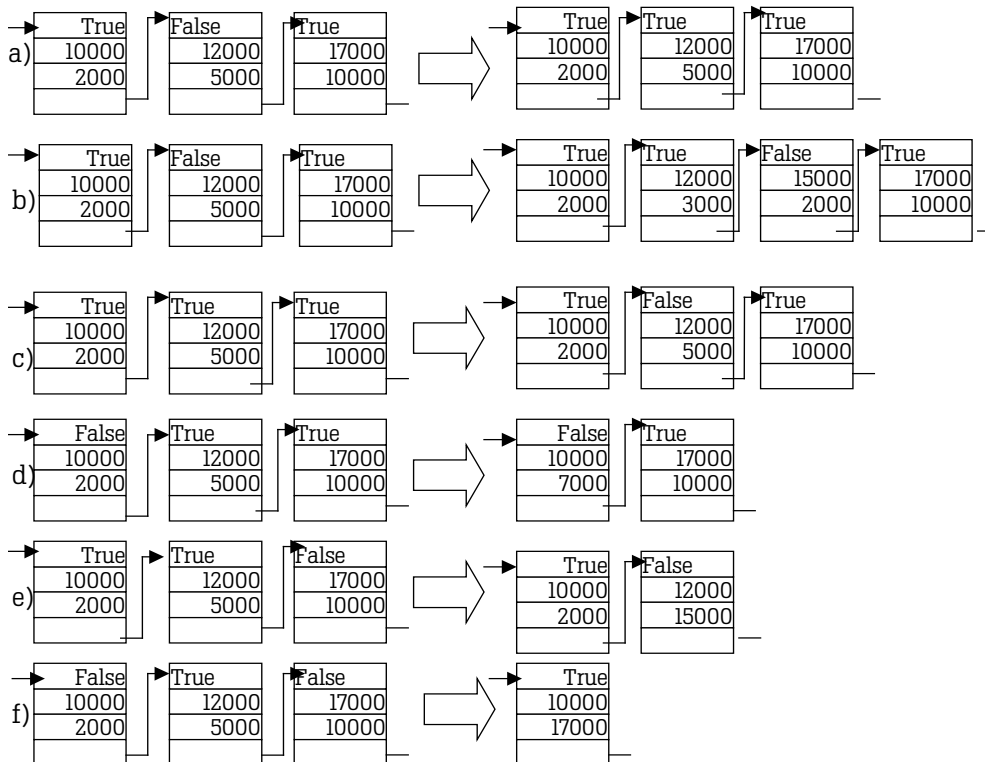
Ovdje je zauzeta particija koja počinje na adresi 10000 i zauzima 2000 bajta. Iza nje je slobodna particija od adrese 12000, dugačka 5000 bajta, a zatim slijede dvije zauzete particije, na adresama 17000 i 27000.

Slika 51 pokazuje da pri zauzimanju i oslobađanju memorijskih particija treba razlikovati šest slučajeva.

- Zahtijevana veličina particije koja se zauzima je jednaka veličini particije koja je bila slobodna** (u ovom primjeru 5000 bajta) i koja je određena da se dodijeli. U tom slučaju dovoljno je promijeniti polje zauzetosti odgovarajućeg sloga ulančane liste.
- Zahtijevana veličina particije koja se zauzima je manja od veličine particije koja je bila slobodna** (npr. 3000 bajta) i koja je određena da se dodijeli. Tada se stanje njenog sloga promijeni na zauzeto, dužina particije skрати na

zahtijevanu dužinu i ubaci novi član lanca koji predstavlja slobodnu particiju dužine koja je razlika između ranije dužine bivše slobodne particije i zahtijevane dužine.

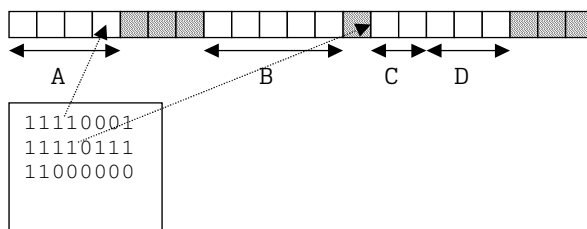
- c) Oslobađa se particija koja je locirana između dvije zauzete particije. Tada je dovoljno njeno stanje zauzetosti u listi promijeniti.
- d) Oslobađa se particija koja je locirana iza jedne slobodne, a ispred jedne zauzete particije. Jedan element liste se briše i spaja nova slobodna particija sa dosadašnjom.
- e) Oslobađa se particija koja je locirana iza jedne zauzete, a ispred jedne slobodne particije. Jedan element liste se briše i spaja nova slobodna particija sa dosadašnjom.
- f) Oslobađa se particija koja je između dvije slobodne particije. Brišu se dva elementa ulančane liste i formira jedna slobodna particija od ovih particija.



Slika 51 Uređivanje ulančane liste

Ako se ne želi pripremati poseban prostor u memoriji za smještanje liste slobodnih blokova, može se koristiti i drugi, ekvivalentan pristup preko **kontrolnih blokova memorije** (MCB). Umjesto u odvojenoj ulančanoj listi, podaci o alociranoj memoriji se čuvaju na istom mjestu gdje i sama alocirana memorija. Ispred svake particije se nalazi zaglavlje, kontrolni blok, koji sadrži stanje zauzetosti i dužinu particije. Pošto je kontrolni blok fiksne veličine (npr. 4 bajta) početna adresa

particije je poznata, jer je jednaka adresi gdje je kontrolni blok plus njegova veličina. Također je poznato gdje se nalazi sljedeći kontrolni blok jer je dovoljan podatak o dužini memorijske particije, one su odmah jedna iza druge. I u ovom pristupu treba brinuti o šest slučajeva ažuriranja kontrolnih blokova, kao kod ulančane liste, s tim što je proces oslobađanja particije nešto jednostavniji jer je brisanje elementa iz liste lakše realizovati. Pristup se ne može koristiti ako ima prekida u adresnom prostoru fizičke memorije: npr. prvih 32 K je popunjeno RAM čipovima, pa narednih 4K je predviđeno za U/I uređaje, a zatim narednih 8K je ponovo popunjeno RAM-om.



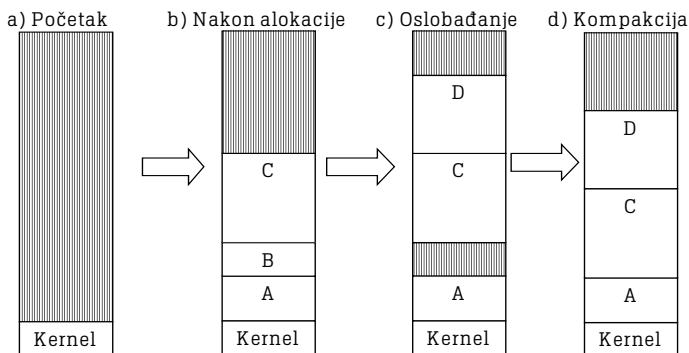
Slika 52Upravljanje slobodnim prostorom s bitmapama

U trećem pristupu, kod upravljanja memorijom pomoću **bit mape**, memorija se dijeli na komade iste veličine, na primjer 256 bajta. Svakom komadu odgovara jedan bit u bit mapi pri čemu 0 znači da je komad slobodan a 1 da je komad zauzet (Slika 52). Tako dolazimo do niza nula i jedinica (bitova) koji se zove bit mapa memorije.

Kada se particija alocira tada se kontinualni niz nula koje odgovaraju memorijskim lokacijama date particije mijenja u odgovarajući niz jedinica. Srodno ovome, kada se particija oslobađa tada se kontinualni niz jedinica koje odgovaraju memorijskim lokacijama date particije mijenja u odgovarajući niz nula. Veličina bitmape je obrnuto proporcionalna veličini komada, u ekstremnom slučaju, ako je komad jednak bajtu, tada bi osmina memorije otišla na bitmapu. Ako je komad velik, npr. 4 K, bitmapa je mala (0,003% ukupne memorije), ali se javlja interna fragmentacija, pošto posljednji komad memorije dodijeljen procesu ne mora biti u potpunosti iskorišten. Mana pristupa je što procesori nemaju dobrih brzih instrukcija za pretraživanje kontinualne sekvence bitova jednake vrijednosti, a koji su se raspodijelili u različite bajtove. Stoga je pretraživanje bitmape sporije od pretraživanja ulančane liste slobodnih blokova ili kontrolnih blokova, iako je njeno ažuriranje jednostavnije.

Alokacija memorije sa promjenjivim particijama dobro radi sve dok se ne pojavi potreba da se neke particije oslobode. Tada se javlja problem **eksterne fragmentacije**. Ona se dešava u sistemima sa kontinualnom dodjelom memorije, a u ovom slučaju u sistemima s promjenjivim particijama. Procesu se ne može dodijeliti zahtijevana količina memorije, jer u sistemu ne postoji kontinualni memorijski blok date veličine (bez obzira što je ukupna količina slobodne memorije veća od količine koju zahtijeva proces). Nakon što su učitani procesi A, B i C, a zatim je B oslobodio svoju memoriju, slobodno područje memorije više nije kontinualno (Slika 53 c). U ovom trenutku može se desiti da novi proces koji zahtijeva memoriju ne može stati ni u jednu od slobodnih particija, ali bi mogao stati kada bi se slobodne particije spojile u jednu veću particiju. Ovakva situacija predstavlja eksternu

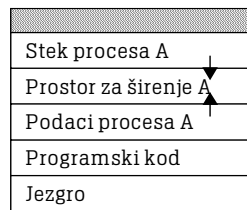
fragmentaciju. Problem eksterne fragmentacije mogao bi se riješiti tako što će se pomijerati skoro svi procesi kako bi se sve slobodne particije nalazile u memoriji zajedno u jednom kontinualnom bloku (Slika 53 d). Ova aktivnost se zove **kompakcija**. Kompakcija je moguća samo u sistemima koji podržavaju dinamičku relokaciju, ali i tada zahtijeva dosta CPU vremena pa se u normalnim uslovima ne koristi.



Slika 53 Pojava fragmentacije kod promjenjivih particija

Ponekad procesu u toku rada treba dodijeliti još memorije u odnosu na onu koja mu je već ranije dodijeljena. Pošto je i dalje potrebno kontinualno područje memorije, mogu se desiti dva slučaja. U prvom slučaju (npr. procesi A i D u situaciji na slici Slika 53 c) iza particije koja pripada procesu se nalazi slobodna particija dovoljne veličine. Tada se particija u kojoj je proces lako proširuje uz odgovarajuće ažuriranje ulančane liste ili bitmape. U drugom slučaju, ako nema odmah iza particije dodijeljene procesu odgovarajuće slobodne particije (proces C na slici Slika 53 c), tada se mora proces suspendovati, snimiti na disk, naći kontinualna particija nove veličine (ako je nema obaviti kompakciju) i vratiti proces u nađenu particiju.

Ako pođemo od pretpostavke da se većina procesa u periodu dok su aktivni povećava, u startu treba rezervirati više prostora u memoriji. Na slici Slika 54 uzeta je u obzir struktura procesa koja uključuje njegova tri fundamentalna dijela, programski kôd, područje podataka i područje stack-a. Područje podataka raste prema višim memorijskim adresama, a područje steka prema nižim memorijskim adresama. Zona između ova dva područja je rezervirana za širenje navedenog procesa.



Slika 54 Prostor za širenje

6.8.Algoritmi za izbor slobodne particije

Kada se treba odabrati slobodna particija da se dodijeli procesu u alokaciji sa promjenjivim particijama, postoji više algoritama za izbor slobodne particije koja će se dodijeliti procesu.

1. Kod **prvog uklapanja** (*first fit*) alokator kreće od početka liste slobodnih blokova ili bitmape i traži prvu slobodnu particiju u listi, odnosno prvu

sekvencu nula u bitmapi dovoljne dužine. Ta particija se dodjeljuje procesu, a ako je proces zahtijevao manje memorije nego što je veličina particije, formira se i odgovarajuća nezauzeta particija, šupljina. Ovo je dosta brz algoritam, ali vremenom na početku liste nastaje puno malih šupljina, pa pretraživanje liste postaje teže.

2. Algoritam **Sljedeće uklapanje** (*next fit*) pretražuje listu slobodnih blokova na sličan način kao prvo uklapanje, samo što ne započinje svaki put pretraživanje od početka liste, već od mjesta gdje je posljednji put dodijelio particiju. Ovaj algoritam izbjegava problem pojave malih šupljina na početku liste, pa je pretraživanje brže, ali ima manu što brzo potroši slobodni prostor na samom kraju memorije.
3. Algoritam **Najbolje uklapanje** (*best fit*) prolazi kroz cijelu listu i odabere šupljinu najmanje veličine u koju može stati zahtijevana količina memorije. Ovaj algoritam je sporiji od prvog i sljedećeg uklapanja jer mora svaki put preći cijelu listu. Ako su zahtjevi takvi da se nalaze šupljine prave veličine, algoritam dobro iskorištava memoriju. Ali u praksi mala je šansa da će šupljina biti u potpunosti iste veličine kao što je zahtijevano, pa dobivamo mnoštvo malih šupljina koja se ne mogu praktično iskoristiti bez spore kompakcije.
4. Algoritam **Najgore uklapanje** (*worst fit*) prolazi kroz cijelu listu i odabere šupljinu najveće veličine. Algoritam se zasniva na pretpostavci da ako se zauzme najveća raspoloživa šupljina, njen ostatak će biti dovoljno velik da bude upotrebljiv za kasnije zahtjeve. Međutim, simulacije su pokazale da je ovo najlošiji algoritam, jer će brzo potrošiti velike šupljine.
5. Algoritam **Brzo uklapanje** (*quick fit*) zasniva se na tome da se u praksi neke zahtijevane veličine particija češće javljaju (na primjer 1 K, 32 K ili 1 M), nego druge. Stoga, kada se god oslobodi particija takve karakteristične veličine, ona se ne opisuje u standardnoj ulančanoj listi slobodnih i zauzetih particija, nego se stavlja u posebnu listu za svaku od specijalnih dužina particija. Na ovaj način se brzo nalazi slobodna particija ako je ona jedne od uobičajenih veličina. Ažuriranje ovakvih listi je, međutim, dosta složeno.

6.9.Sistem drugova (Buddy system)

Sljedeći pristup za alokaciju memorije kombinuje neke osobine alociranja po fiksnim i promjenjivim particijama. **Sistem drugova** (*buddy system*) dopušta alokaciju particija samo one veličine koja je stepen broja 2. Algoritam radi na sljedeći način.

1. Neka je bio zahtjev za alokacijom particije veličine s u memoriji veličine 2^m bajta koju posmatramo kao jedan blok.
2. Ako je tražena količina memorije s takva da je $2^{m-1} < s \leq 2^m$, zauzmi čitav blok
3. U suprotnom blok se dijeli na dva jednaka dijela, umanjim m za 1, i idi na 2
4. Dobiven je najmanji blok koji je veći ili jednak zahtijevanoj veličini s .
5. Nađeni blok se ubacuje u ulančanu listu blokova određene veličine.

Pri oslobađanju memorije, provjere se susjedne slobodne particije iste veličine kao particija koja se oslobađa. Ako je particija koja se oslobađa veličine 2^m bajta, i počinje od memorijske adrese $p \cdot 2^m$ provjeriti da li na memorijskim adresama $(p-1) \cdot 2^m$ ili $(p+1) \cdot 2^m$ postoji još jedna šupljina veličine 2^m bajta. Ako postoji i ako se može formirati jedinstvena šupljina koja počinje na adresi djeljivoj sa 2^{m+1} , spajaju se ove dvije particije.

Primjer algoritma:

Sa 4M memorije, postojaće ukupno 23 liste ($2^{22}=4\text{ M}$). Inicijalno, postoji samo jedna šupljina veličine 4 M smještena u odgovarajuću listu.

4096 K

Zahtijeva se particija veličine 300 K. Najmanja particija koja može zadovoljiti zahtjev je ona veličine 512 KB jer je to stepen broja 2. U listi particija veličine 512 K nemamo takve particije, pa će se particija od 4 MB prepоловити u dvije particije po 2 MB. Jedna od njih se dalje polovi sve dok se ne dođe do dvije particije od 512 K, i prva od njih se zauzima. U listi veličina po 512 K nalaze se dvije particije, jedna zauzeta, druga slobodna. Liste veličina 1 M i 2 M imaju po jedan element, oba slobodna.

512K	512K	1024 K	2048 K
------	------	--------	--------

Ako se sada traži za novi proces 980 K memorije, ovaj zahtjev odmah opslužujemo iz liste veličina od 1M.

512K	512K	1024 K	2048 K
------	------	--------	--------

Ako je sada potrebno alocirati 200 K, potrebna je particija od 256 K. Lista particija ovih veličina je prazna, ali postoji nezauzeta particija sljedeće veće veličine od 512 K. Ta particija se polovi, briše iz liste particija od 512 K, ali se formira lista sa dvije particije od 256 K od kojih će jedna biti slobodna, a druga zauzeta.

512K	256K	256K	1024 K	2048 K
------	------	------	--------	--------

Sada će se osloboditi particija veličine 512 K.

512K	256K	256K	1024 K	2048 K
------	------	------	--------	--------

Ako se nakon ovoga oslobodi particija veličine 256 K, ona se spaja sa susjednom slobodnom particijom od 256K, a novostvorena particija od 512 K sa drugom particijom od 512K. Tako nastaje particija od 1024 K. Particije iz listi za veličine 256 K i 512 K nestaju.

1024K	1024 K	2048 K
-------	--------	--------

Kompakcija u *buddy* sistemu je jednostavna, traženje particije odgovarajuće veličine vrlo brzo, ali se u njemu javljaju i interna i eksterna fragmentacija. Linux koristi ovaj sistem za određivanje adresa fizičkih okvira kao sastavni dio straničenja.

6.10.Windows API za upravljanje memorijom

Windows API posjeduje više funkcija za zauzimanje i oslobađanje memorije unutar procesa.

Funkcija	Značenje
CopyMemory(pDestination,pSource,iLength)	Kopira blok memorije na drugo mjesto
FillMemory(pDestination, iLength, bFill);	Puni memoriju istim bajtom
GetProcessHeap()	Dobavlja alokacijsko područje za proces
VirtualAlloc(lpAddress, dwSize, dwAllocationType, dwProtect)	Alocira dio virtualne memorije za proces (zaokruženo na veličinu stranice)
VirtualFree(lpAddress, dwSize, dwFreeType)	Oslobađa alociranu virtualnu memoriju
HeapAlloc(hHeap, dwFlags, dwBytes)	Alocira dio memorije unutar procesa
HeapReAlloc(hHeap, dwFlags, lpMem, dwBytes)	Mijenja veličinu memorije za proces
HeapFree(hHeap, dwFlags, lpMem)	Oslobađa memoriju alociranu za proces
HeapSize(hHeap, dwFlags, lpMem)	Dobavi veličinu alociranog mem. bloka

Slijedi primjer realizacije pod Windows sistemom poznatih C funkcija *malloc* i *free* za alokaciju memorije unutar procesa. Windows ima veći broj funkcija za alociranje globalne i privatne memorije. Ovdje je korišten sistemski poziv *HeapAlloc*.

```
#include <windows.h>
#define heapNew GetProcessHeap()
void *malloc(size_t size){
    if (heapNew)
        return HeapAlloc(heapNew, HEAP_ZERO_MEMORY, size);
    return NULL;
}
void free(void *block) {
    if (block != NULL && heapNew)
        HeapFree(heapNew, 0, block);
}
```

6.11.POSIX API za upravljanje memorijom

Za razliku od oblasti procesa, Unix kompatibilni sistemi ne pružaju mnogo sistemskih poziva za upravljanje memorijom. Sve što je dostupno su funkcije koje povećavaju krajnju adresu sekcije podataka trenutnog procesa.

Funkcija	Značenje
int brk(void *last)	Postavi kraj područja memorije za proces na datu lokaciju
void *sbrk(int increment)	Uveća kraj područja memorije za proces za dati broj bajta

Aplikacije trebaju koristiti C funkcije *calloc()*, *malloc()*, *free()* i slično. Ove funkcije nisu realizovane u jezgru operativnog sistema, nego izvršni C sistem mora da ima algoritme za alokaciju memorije. Ipak, radi ilustracije principa rada, napisao se funkcija *malloc* i *free* za Unix sisteme. Ovdje se mora koristiti poziv *sbrk* za povećanje prostora za alokaciju memorije i ulančana listu slobodnih blokova. Lista je organizovana tako da se svaki član liste u memoriji nalazi neposredno ispred memorijskog sadržaja (kao kod MCB) ali ipak postoji pokazivač na naredni član liste. Funkcija *malloc* prvo provjerava veličinu i ako je ona nula, izađe se iz funkcije. Nakon toga se pozove *get_free_block()* koja prolazi kroz ulančanu listu slobodnih blokova algoritmom *first fit* i vraća nađeni slobodni blok potrebne veličine, i mijenja mu se status u slobodan. Vraća se memorijska pozicija neposredno iza elementa liste, što pokazuje na podatke. U cilju jednostavnosti uzima se cijeli slobodni blok iako je veće veličine od traženog. Ako nema već postojećeg slobodnog bloka, proširuje se sekcija podataka pozivom *sbrk* za veličinu zaglavlja i alociranog bloka. U dodijeljenom području napuni se zaglavlje bloka i vrati pokazivač na alociranu memoriju.

Funkcija *free* proglašava u zaglavlju memorijskog bloka čvor u listi slobodnim čvorom.

<pre>#define SIZE unsigned int; #define HEADER struct header_t HEADER { SIZE size; int is_free; HEADER *next; }; HEADER *head, *tail; HEADER *get_free_block(SIZE size){ HEADER *curr = head; while(curr) { if (curr->is_free && curr->size >= size) return curr; curr = curr->next; } return NULL; } void free(void *block) { HEADER *header; if (!block) return; header = (HEADER *)block - 1; header->is_free = 1; }</pre>	<pre>void *malloc(SIZE size) { SIZE total_size; void *block; struct header_t *header; if (!size) return NULL; header = get_free_block(size); if (header) { header->is_free = 0; return (void*)(header + 1); } total_size = sizeof(HEADER) + size; block = sbrk(total_size); if (block == (void*) -1) { return NULL; } header = block; header->size = size; header->is_free = 0; header->next = NULL; if (!head) head = header; if (tail) tail->next = header; tail = header; return (void*)(header + 1); }</pre>
--	---

6.12. Dinamički dijeljene biblioteke

Potprograme koje koriste skoro svi programi (npr. svi potprogrami iz standardne biblioteke jezika C, potprogrami za grafičko okruženje) je neracionalno ugraditi u svaki program. Neka npr. svih 10 programa u memoriji koristi *printf* funkciju čiji je programski kôd velik 6 K. Ako bi svaki program ugradio ovu funkciju posebno potrošilo bi se na to 60 K memorije. Dovoljno bi bilo imati samo jednu kopiju ove funkcije u memoriji dostupnu svim procesima, čime se postižu uštede u memoriji i na disku. Više potprograma koji se mogu pozvati iz različitih procesa grupišu se u dinamičke dijeljene biblioteke.

Adresa potprograma iz dinamičkih dijeljenih biblioteka nije poznata u trenutku prevođenja programa pa se koristi indirektni poziv potprograma. U jeziku C to su pointeri na funkcije, a i mnogi mašinski jezici imaju način poziva potprograma čija adresa nije navedena u samoj mašinskoj instrukciji poziva potprograma nego se uzima sa druge memorijske lokacije ili registra. U dinamičkoj biblioteci se čuvaju imena svih potprograma te biblioteke koji su dostupni programima, zajedno sa njihovim relativnim adresama od početka dinamičke biblioteke. Korisnički proces onda može koristiti systemske pozive koji vrate adresu potprograma prema njegovom imenu i prilagođenu logičkom adresnom prostoru tog procesa.

U jednoprocensnim sistemima, koji nemaju mogućnost relokacije, dijeljene biblioteke su korisne samo za uštedu prostora na disku, dok bi u memoriji njihova

upotreba trošila i više prostora nego da je riječ o ugrađenim potprogramima u same procese, jer će u memoriju biti ugrađeni i oni dijelovi biblioteke koje program ne koristi. Biblioteka se može učitati nakon korisničkog procesa ako on nije zauzeo cijelu raspoloživu memoriju. Nakon učitavanja obavlja se statička relokacija učitane biblioteke.

U višeprocensnim sistemima sa alokacijom s fiksnim, promjenjivim ili *buddy* particijama dinamička biblioteka se učitava u jednu od raspoloživih particija. Neka procesi koji su u različitim particijama koriste dinamičku relokaciju sa relokacijskim registrom. Kada proces traži adresu potprograma u dinamičkoj biblioteci, rezultat sistemskog poziva treba biti razlika između fizičke adrese potprograma i adrese početka particije u kojoj je proces koji je tražio adresu potprograma.

Situacija postaje složenija ako postoji zaštitni limit registar koji ograničava najvišu adresu particije. Dinamička biblioteka se nalazi u drugoj particiji. Svaki poziv potprograma u dinamičkoj biblioteci će rezultovati greškom zabrane pristupa i automatskim skokom u rutinu u jezgro za obradu greške. U toj rutini se može provjeriti da je skok u jezgro izazvan pozivom regularne dinamičke dijeljene biblioteke. Tada jezgro može privremeno isključiti limit registar, pozvati potprogram u dijeljenoj biblioteci, ponovo uključiti limit registar i vratiti kontrolu pozivaocu.

Segmentacija i straničenje, o kojima će biti riječi u narednom poglavlju, pružaju fleksibilniji način za mapiranje dinamičkih biblioteka u adresni prostor procesa.

6.13.Windows API za dinamičke dijeljene biblioteke

Windows omogućava dva načina povezivanja programa sa dinamičkim bibliotekama, implicitni i eksplicitni. U **implicitnom** načinu se unutar izvršne verzije programa nalazi spisak dinamičkih biblioteka i njihovih potprograma koje program koristi. Pri startu programa punilac poveže sve dinamičke biblioteke. Programer u jeziku C treba da koristi samo odgovarajuće datoteke sa zaglavljima i pravilno konfiguriran kompajler, pa i ne osjeti razliku u pozivu između potprograma u dinamičkim bibliotekama i običnih potprograma.

Eksplicitni način povezivanja zahtijeva poziv funkcija iz dolje navedene tabele. On se koristi ako se unaprijed ne zna da li je dinamička biblioteka instalirana u sistemu i za proširivanje mogućnosti programa u budućnosti pri čemu se unaprijed ne mora znati ni ime biblioteke ni ime funkcije. U narednoj tabeli su navedene API funkcije za korištenje eksplicitno povezanih dinamičkih biblioteka.

Funkcija	Naziv
LoadLibrary(lpFileName)	Učitava dinamičku biblioteku (.dll) u memoriju
FreeLibrary(hModule)	Oslobađa dinamičku biblioteku iz memorije
GetModuleFileName(hModule, lpFilename, nSize)	Dobavi puno ime biblioteke s stazom
GetProcAddress(hModule, lpProcName)	Traži adresu potprograma s datim imenom u dinamičkoj biblioteci

Sljedeći primjer eksplicitno učitava dinamičku biblioteku User32.dll i ako ju je učitao u njoj traži funkciju *MessageBoxA*. Adresa te funkcije se dodijeli pointeru na funkciju *MojaFunkcija* i pozova tu funkciju.

```
#include <windows.h>
typedef int(__stdcall *Fun)(HWND, LPCSTR, LPCSTR, UINT);
int main(void) {
    HINSTANCE hModule = LoadLibrary("user32.dll");
    Fun MojaFunkcija = 0;
    if(hModule != 0) {
        MojaFunkcija = (Fun)GetProcAddress(hModule, "MessageBoxA");
        MojaFunkcija(NULL, "Zdravo", "Poruka", 1);
    }
    return 0;
}
```

6.14. Zadaci

Zadatak 1: Neka se u sistemu koristi statička relokacija. Program je preveden da se podrazumijevano učitava na fizičku adresu 0. Adrese su 16 bitne i big endian tipa (na nižoj memorijskoj adresi nalazi se značajniji bajt). Neka zadnjih 8 bajta programskog koda predstavlja relokacijsku tablicu. Kako će program u memoriji izgledati ako se učita na heksadekadnu adresu A001? Heksadekadni prikaz programskog koda na disku izgleda ovako

A2	3E	00	02	4B	2C	10	18	77	01	1A	77	10	00	23	11
3E	40	00	20	00	10	11	01	00	02	00	09	00	0C	00	11

Rješenje: Tablica kaže da treba sabrati sa A001 vrijednosti (po 2 bajta) na relativnim pozicijama 0002, 0009, 000C i 0011. Nakon relokacije tablica se može odbaciti iz memorije. Stoga će memorija od adrese A001 izgledati ovako:

A2	3E	A0	03	4B	2C	10	18	77	A1	1B	77	B0	01	23	11
3E	E0	01	20	00	10	11	01								

Zadatak 2: Neka u sistemu sa fiksnim particijama postoje 4 particije:

Particija	Početna adresa	Veličina
0	0	10K
1	10240	8 K
2	18432	20 K
3	38912	32 K

Sve particije su zauzete i na izvršenje čekaju procesi P1-P5.

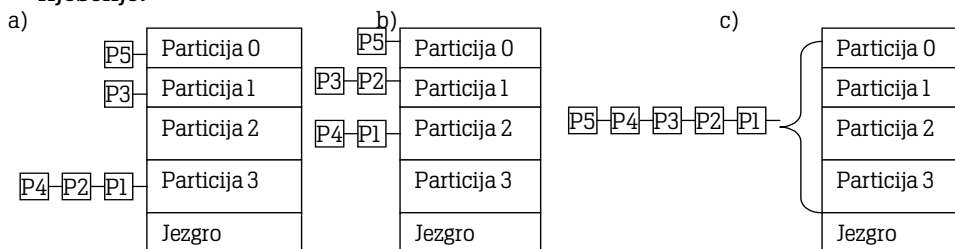
Proces	Redoslijed dolaska	Veličina	Podrazumijevana početna adresa
P1	1	12 K	38912
P2	2	6 K	38912
P3	3	7 K	10240
P4	4	18 K	38912
P5	5	10K	0

Prikažite redove čekanja na particije ako se koriste

- Odvojeni redovi čekanja na particije bez relokacije
- Odvojeni redovi čekanja na particije sa relokacijskim registrom

c) Jedinstveni red čekanja na particije

Rješenje:



Zadatak 3: Ako se koristi dinamička relokacija sa relokacionim registrom čija je vrijednost dekadno 32000, a limit registar ima vrijednost 35000, kojoj fizičkoj lokaciji pristupa program koji pristupa logičkoj lokaciji

a) 2000

b) 5000

Rješenje:

a) $32000 + 2000 = 34000$ b) $32000 + 5000 > 35000$, izaziva grešku pristupa

Zadatak 4: Ako imamo sistem s 8 memorijskih šupljina S1(20K), S2(6K), S3(8K), S4(30K), S5(27K), S6(9K), S7(29K), S8(20K), i želimo udovoljiti zahtjevima redom za 6, 11, 5 kilobajta, koje šupljine će biti korištene za algoritme:

a) *first fit*b) *best fit*c) *worst fit*?

Rješenje:

a) First fit

Zahtjev od 6K smjestiće se u šupljinu S1 koja će time biti smanjena na 14K

Zahtjev od 11K smjestiće se u šupljinu S1 koja će time biti smanjena na 3K

Zahtjev od 5K smjestiće se u šupljinu S2 koja će time biti smanjena na 1K

b) Best fit

Zahtjev od 6K smjestiće se u šupljinu S2 koja će time biti potpuno zauzeta

Zahtjev od 11K smjestiće se u šupljinu S1 koja će time biti smanjena na 9K

Zahtjev od 5K smjestiće se u šupljinu S3 koja će time biti smanjena na 3K

c) Worst fit

Zahtjev od 6K smjestiće se u šupljinu S4 koja će time biti smanjena na 24K

Zahtjev od 11K smjestiće se u šupljinu S7 koja će time biti smanjena na 18K

Zahtjev od 5K smjestiće se u šupljinu S5 koja će time biti smanjena na 22K

Zadatak 5: Neka je memorija *buddy* sistema alokacije inicijalno prazna i iznosi 262144 K. Nacrtati kako izgleda memorija dodijeljena zahtjevima od redom 69246K, 26046K, 45033K.

Rješenje:

Prva veličina veća od 69246 a koja je stepen broja 2 je 131072. Prva veličina veća od 26046 a koja je stepen broja 2 je 32768. Prva veličina koja je veća od 45033 a koja je stepen broja 2 je 65536. Kako se zahtjevi trebaju smjestiti na adrese djeljive s veličinom bloka u koji se smješta, adrese (**mjereno u kilobajtima**) gdje će se smjestiti zahtjevi su :

0:69246, 131072:26046, 196608:45033.

Iz ovog se vidi raspored memorije:

Z1	unused	Z2	unused	free	Z3	unused	
0	69246	131072	157118	163848	196608	241641	262143

Zadatak 6. Sistem ima 64 kilobajta memorije, od kojih su slobodna područja 2-15K i 22-28K. Označite slobodnu memoriju

- bitmapom (granularnost alokacije 1 K),
- kontrolnim blokovima (informacija o veličini bloka 2 bajta)
- ulančanom listom

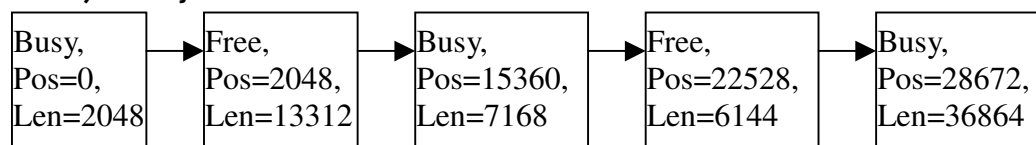
Rješenje:

a) Bitmapa će izgledati ovako: 11000000 00000001 11111100 00001111 11111111 11111111 11111111 11111111

b) U sljedećoj tabeli se vidi raspored kontrolnih blokova memorije

Sadržaj	2048 busy	Podaci	13312 free	Prazno	7168 busy	Podaci	6144 free	Prazno	36856 busy	Podaci
Adresa	0	2	2048	2050	15360	15362	22528	22530	28672	28674

c) Ovo je ulančana lista slobodnih i zauzetih blokova



7.DISKONTINUALNA ALOKACIJA MEMORIJE I VIRTUALNA MEMORIJA

Procesi po prirodi treba da u memoriji zauzimaju kontinualne blokove, tj. one koji su na susjednim logičkim adresama. Programi se izvršavaju instrukciju po instrukciju, koje se u memoriji nalaze na susjednim lokacijama ako nije bilo instrukcija skokova. Nizovi i strukture podataka su također zasnovani na susjednim lokacijama. Sasvim je prirodno mapirati kontinualni niz logičkih lokacija u kontinualni niz fizičkih lokacija. Nažalost, svaki od pristupa koji prevodi logičku u fizičku adresu prostom linearnom funkcijom (neizmijenjeno ili sabiranjem sa relokacijskim registrom) ima određene probleme.

Prvi problem je limit na veličinu fizičkog adresnog prostora i veličinu particije. Ako je memorijski adresni registar veličine npr. 16 bita (Z80), sistem ne može imati više od 65536 memorijskih lokacija. To znači da ni najveća kontinualna memorijska particija ne može biti veća od 64 kilobajta čak i ako je sama u sistemu i ako jezgro operativnog sistema zauzima 0 bajta. Postoje i sistemi (npr. Pentium III) čiji je memorijski adresni registar veće širine, 36 bita, ali logičke adrese navedene u instrukcijama imaju ograničenja na veličinu od 32 bita. U ovom slučaju ukupni adresni prostor je 64 G, ali pojedina particija je maksimalno velika 4 G.

Drugi problem je što sva četiri opisana pristupa sa linearnom transformacijom logičke u fizičku adresu imaju nekih praktičnih nedostataka. Jednoprocesni sistemi bez transformacije memorije su previše ograničeni sami po sebi, sistemi sa fiksnim particijama ograničavaju najveću moguću memorijsku particiju, sistemi sa promjenjivim particijama povremeno traže sporu kompakciju, a *buddy* sistem ima veliku internu fragmentaciju i neiskorišten prostor.

Sada će se vidjeti nekoliko historijskih i modernih tehnika koje po cijenu komplikovanijeg pristupa memoriji izbjegavaju ove nedostatke.

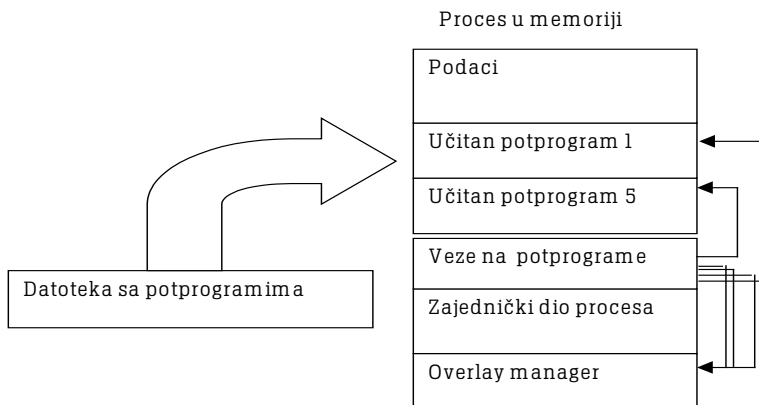
7.1.Overlay

Nekad se nema uticaja ni na dizajn računarskog sistema, ni izbor procesora ili memorije, nego samo na korisničke procese. Dešava se da je operativni sistem vrlo jednostavan i memorijski prostor limitiran, kao i veličina particije koja je dodijeljena procesu. Neka je particija limitirana, na npr. 32 kilobajta, a program sa raznim opcijama zauzima 80 kilobajta.

U ovom slučaju programer je prisiljen podijeliti svoj program u potprograme koji se snime na disk u jednu datoteku. U memoriji se odredi dio za potprograme koji će se povremeno učitavati iz te datoteke (Slika 55). Jedan dio programa i podaci koje on obrađuje su stalno u memoriji, a u program se ugradi i *overlay manager*. Njegova uloga je da sa diska učitava potprogram iz datoteke u memoriju u trenutku kada je on pozvan. Potprogrami koji se pozivaju su povezani pokazivačem koji pokazuje na njegov kôd u memoriji ili na *overlay manager* kada potprogram nije u memoriji. Kada se pozove potprogram koji nije u memoriji, tada se skoči u *overlay manager*. On pogleda veličinu potprograma i ako ima dovoljno prostora u dijelu memorije predviđenom za potprograme, ubaci potprogram u memoriju, obavi statičku relokaciju tog potprograma i preusmjeri pokazivač da pokazuje na učitani

potprogram. Ako nema dovoljno prostora, prije učitavanja novog potprograma mora se izbaciti neki ranije učitani potprogram i njegov pokazivač vratiti na *overlay manager*.

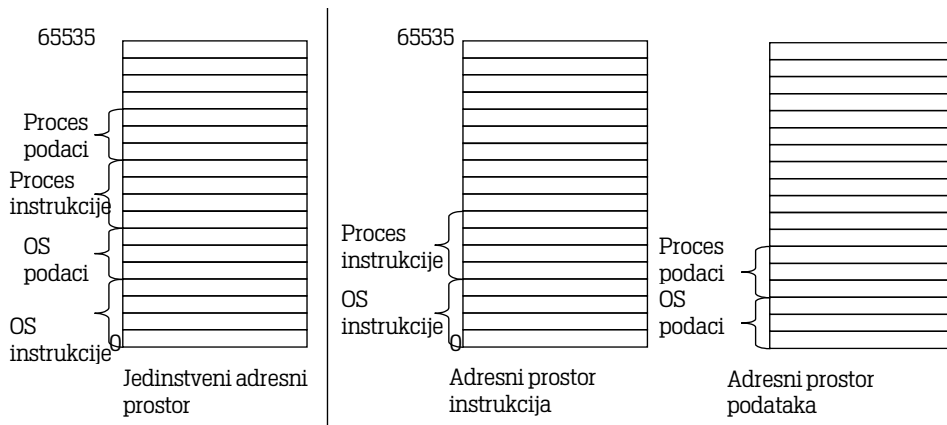
Mana pristupa je što se ne može koristiti za podatke, nego samo za potprograme i to određene strukture i veličine. Npr., ako potprogram poziva drugi potprogram koji je u *overlay* datoteci, potprogram pozivalac mora biti stalno u memoriji, kako bi povratak bio uspješan. Ako premali dio memorije procesa ostane dodijeljen za preklapljene potprograme, rad postaje spor zbog čestog učitavanja s diska.



Slika 55 Organizacija procesa sa overlay

7.2.Odvojen prostor instrukcija i prostor podataka

U klasičnoj, von Neumanovoj arhitekturi, koristi se zajednički memorijski prostor koji sadrži i programe i podatke. Ovo dobro funkcionira ukoliko je adresni prostor dovoljno velik (Slika 56a).



Slika 56 Zajednički i odvojeni adresni prostori za programe i podatke

Neki sistemi, prije svega mikrokontroleri, međutim, razdvajaju prostor za programske instrukcije i prostor za podatke u dvije odvojene memorije. Slika 56 b)

pokazuje odvojene adresne prostore. Takav pristup organizaciji memorije zove se **Harvard arhitektura**. Harvard arhitektura se smatra alternativom Von Neumanovoj. U Harvard arhitekturi program i podaci su potpuno odvojeni. U ovom primjeru memorija je time udvostručena, a širina adresne riječi ostala je ista. No, to ne mora biti slučaj. Isti procesor može npr. imati 24 bitnu širinu riječi za instrukcije a 4-bitnu za podatke. Ovakva arhitektura je uobičajena u mikrokontrolerima i kalkulatorima koji ne mijenjaju svoj osnovni program u ROM, a koriste vrlo malo RAM memorije.

7.3.Memorijske banke

Nekad je dizajner računarskog sistema ograničen osobinama procesora koji je izabran, ali ima uticaj na izbor memorije i razvoj operativnog sistema. Grafički programibilni kalkulator TI-84 Plus CSE iz 2013 ima za svoju primjenu mnogo memorije: 128 K RAM i 4 M ROM memorije, ali u cilju kompatibilnosti sa ranijim modelima koristi procesor Z80 čiji je adresni prostor samo 64 K koji nema MMU. Kako koristiti ovu znatno veću memoriju?

U ovakvim slučajevima koriste se **memorijske banke**. Banka je fiksna memorijska particija koja se bira iz skupa veće fizičke memorije i ubacuje u adresni prostor. Neka je u sistemu adresnog prostora od 64 K raspoloživo ukupno 4 M memorije. U tom sistemu je svaka banka velika 16 K, pa ukupno postoji 256 banki od kojih 4 istovremeno mogu biti u RAM memoriji. Odgovarajućim jednostavnim sklopom (koji nije dio mikroprocesora) bira se koja banka će biti u adresnom prostoru i na kojoj od 4 dopuštene memorijske adrese. Kada banka nije u adresnom memorijskim podacima u njoj se u tom trenutku ne može pristupiti, ali njen sadržaj nije obrisan, baš kao što se u knjizi od više stotina strana istovremeno vide samo dvije.

C000	Banka 14
8000	Banka 180
4000	Banka 32
0000	Banka 20

Slika 57 Memorijske banke

Ovdje nema ograničenja na količinu fizičke memorije koja može biti u sistemu, ali je njena fleksibilnost korištenja slična alokaciji sa fiksnim particijama. Programe koji zauzimaju više memorijskih banaka je teško pisati (programi i podaci se moraju sjeckati na dijelove), ali je izmjena između procesa veličine do jedne banke vrlo brza.

7.4.Virtualna memorija

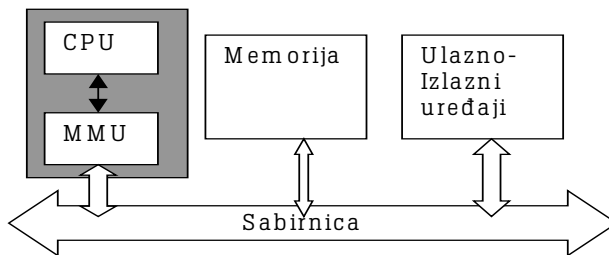
Virtualna memorija je tehnika upravljanja memorijom koja koristi i hardversku i softversku podršku i kod koje se adresni prostor procesa (virtualni adresni prostor), razlikuje od fizičkog adresnog prostora.

Iako i sistemi sa fiksnim i promjenjivim particijama i relokacijskim registrom zadovoljavaju ovu definiciju, oni se obično ne smatraju sistemima s virtualnom memorijom, jer je kod njih virtualni adresni prostor uvijek manji od fizičkog adresnog prostora.

Virtualna memorija dopušta jednom procesu da zauzme i više memorije od ukupne raspoložive primarne memorije. Tajna je u upotrebi diska za one dijelove procesa koji nisu trenutno u memoriji. Jedan dio procesa se čuva u primarnoj memoriji (RAM) a drugi dio je u sekundarnoj memoriji (disk), a sam proces ne zna

ništa o tome. Procesu se čini da ima dovoljno memorije i da se čitav proces nalazi u primarnoj memoriji. U stvarnosti će operativni sistem po potrebi snimati dijelove procesa na disk i učitavati ih svaki put kada se pristupa dijelu procesa koji nije trenutno u memoriji.

Realizacija virtualne memorije zahtijeva dodatni hardver. Proces generiše virtualnu memorijsku adresu. Virtualna adresa se proslijeđuje dijelu procesora ili eksternom integrisanom kolu, koji se zove jedinica za upravljanje memorijom (MMU – *Memory Managment Unit*). Ova jedinica pretvara virtualnu adresu u fizičku memorijsku adresu (Slika 58).



Slika 58 Položaj MMU u računarskom sistemu

Može se reći da MMU obavlja transformaciju virtualne adrese u fizičku adresu odgovarajućom funkcijom:

$$\text{FizičkaAdresa} = f(\text{VirtualnaAdresa})$$

Ako je funkcija f linearna, transformacija je jednostavna i brza, ali ograničena na zahtjev da kontinualno područje virtualne memorije odgovara kontinualnom području fizičke memorije. S druge strane, ako je funkcija f tabelarna funkcija, dobiva se puna fleksibilnost u preslikavanju između virtualne i fizičke memorije, ali tabela preslikavanja zauzima previše prostora.

Kompromis između ova dva ekstrema predstavljaju sistemi koji pružaju virtualnu memoriju koristeći kombinaciju relativno malih tabela transformacija i kontinualnih područja memorije. To su segmentacija, straničenje i segmentacija sa straničenjem koji će sada biti opisani.

7.5.Segmentacija

Ovaj pristup organizacije memorije kombinuje osobine relokacionih registara, banki i odvojenih adresnih prostora. Kada se koristi segmentacija, svakom se procesu dodijeli više linearnih adresnih prostora čija je najniža logička adresa jednaka 0, a najviša ide do određene maksimalne vrijednosti. Ti adresni prostori su dijelovi procesa i zovu se **segmenti**. Svaki segment predstavlja linearno područje memorije. Segmenti mogu predstavljati funkcionalne dijelove programa kao što su: grupa srodnih potprograma, *stack*, podaci, *heap*, ali segment može da sadrži i samo jedan potprogram ili niz cijelih brojeva ili nešto treće. Programer aplikacije je svjestan da se podacima pristupa iz dva dijela adrese: segmenta i pozicije unutar njega. Stoga je logička memorijska adresa ne jedan broj nego uređeni par oblika (segment, pozicija).

Dva pristupa postoje za realizaciju alokacije memorije preko segmenata i oba zahtijevaju hardversku podršku ugrađenu u mikroprocesor. Jedan pristup, je sličan kontinualnoj alokaciji s relokacionim registrom, samo što postoje tri ili **više relokacionih registara** namijenjenih različitim područjima memorije. Tako npr. Intel 8086 ima četiri takva registra koji se zovu segmentni registri: CS koji u memoriji pokazuje početak koda, DS koji pokazuje početak podataka, SS koji pokazuje početak područja za *stack* i ES koji se koristi za pristup području podataka kada se ne želi mijenjati DS. Na taj način instrukcije JMP 1000 (bezuslovni skok na memorijsku lokaciju 1000) i MOV AX,[1000] (čitanje vrijednosti memorijske lokacije 1000 i prebacivanje u registra AX) pristupaju različitim fizičkim lokacijama unutar istog procesa. Na ovom procesoru zadržane su 16 bitne logičke adrese u programima (što čini programe kraćim jer instrukcije zauzimaju manje prostora u memoriji, nego kada bi bile 32 bitne), a fizički adresni prostor je realizovan kroz 20 bitne adrese pa dopušta do 1 megabajt fizičke memorije. Segment ovdje predstavlja kontinualno područje od 64 kilobajta unutar fizičke memorije. Transformacija logičke u fizičku adresu je jednostavna i brza: vrijednost segmentnog registra relevantnog za instrukciju pomnožena sa 16 (što se postiže ožičenjem) se sabere sa logičkom adresom. Pristup ima dosta mana. Adresa početka segmenta je direktno vezana za relokacioni registar. Korisnički proces mora imati pravo da postavi adresu početka segmenta ako se njegovi podaci protežu u više segmenata. U tom slučaju ništa ga ne sprječava od pristupa području memorije koje je dodijeljeno drugom procesu ili jezgri i oštećenja drugih procesa ili jezgra OS. Dalje, segmenti su fiksne veličine. Stoga se ne mogu npr. koristiti nizovi i slične strukture čija veličina prelazi veličinu segmenta.

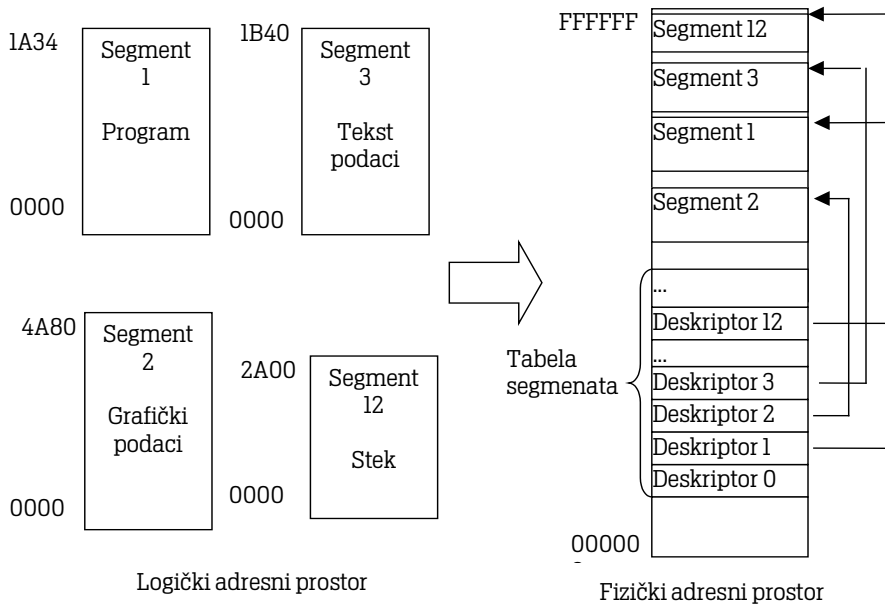
U drugom pristupu (korišten u Windows 3.1, u standardnom režimu) segmentaciji uvodi se **tabela segmenata** koja ima onoliko elemenata koliko je moguće imati segmenata u sistemu. Elementi tabele se zovu deskriptori. Deskriptor sadrži za svaki segment, kome se pristupa preko rednog broja njegovu početnu adresu, veličinu, vrstu, prava pristupa i podatak da li je segment prisutan u memoriji. Različiti segmenti mogu biti različitih veličina. Pošto svaki segment ima svoj adresni prostor (linearan), različiti segmenti ne utiču jedan na drugi.

Transformacija logičke adrese u fizičku se obavlja tako što se za dati redni broj segmenta iz odgovarajućeg deskriptora preuzme adresa početka segmenta i sa tom adresom sabere adresa unutar segmenta. Segmenti su stoga relokabilni, a korisnički program ne operiše s fizičkim adresama. Tabeli segmenata pristupa jezgro i raspoređuje logičke segmente po fizičkoj memoriji. Kako segmenti imaju svoju definisanu dužinu u jezgri, postignuta je i zaštita segmenata.

Segment može i da ne bude prisutan u memoriji. Kada se pristupa segmentu koji je u deskriptoru označen kao da nije u memoriji, generiše se prekid. U okviru obrade tog prekida može se neki od segmenata snimiti na disk i izbaciti iz memorije a zatim na njegovo mjesto u fizičkoj memoriji učitati sadržaj ranije snimljenog segmenta kome se željelo pristupiti. To je već opisana tehnika *swapping*, ali se ne prebacuje cijeli program nego samo jedan njegov segment. Ako program podijelimo u dovoljan broj segmenata čija pojedinačna veličina ne prelazi raspoloživu fizičku memoriju,

ali u ukupnom zbiru prelazi, može se na ovaj način realizovati proces sa podacima koji znatno prevazilazi fizičku memoriju.

Pri izmjeni konteksta između procesa, jezgro operativnog sistema može da za svaki proces promijeni registar koji predstavlja početak tabele segmenata, tako da svaki proces ima svoju vlastitu tabelu segmenata i time logičku organizaciju segmenata.



Slika 59 Transformacija logičkih u fizičku adresu kod segmentacije sa tabelom

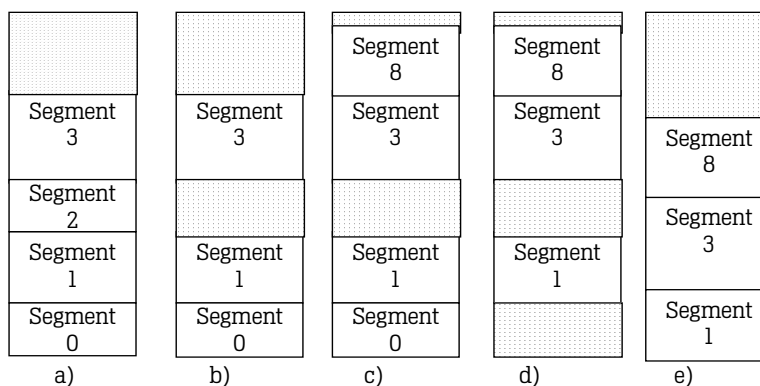
Kako se početna adresa svakog segmenta nalazi u memoriji, segmentacija sa tabelom segmenata na prvi pogled usporava pristup memoriji, jer bi se pri svakom pristupu memoriji moralo pročitati gdje segment počinje, pa tek onda pristupiti podatku. Da bi se to izbjeglo, u procesor se ugrađuje **deskriptorski keš**, koji pamti početne adrese zadnje pristupanih segmenata. Nakon što je promijenjen registar za izbor segmenta, procesor pročita njegovu početnu adresu iz deskriptora i ubuduće pri formiranju adrese koristi tako zapamćenu početnu adresu.

Prednosti segmentacije su:

- Program se može logički organizovati po različitim cjelinama
- Segmenti zauzimaju onoliko prostora koliko im treba i kada se učitavaju sa diska u virtualnoj memoriji odmah su dostupni u cijelosti u memoriji.
- Promjena segmenta se obavlja daleko rjeđe nego pristupanje memoriji. To čini izvodivim neke algoritme za realizaciju virtualne memorije koji su efikasni u izboru šta čuvati u fizičkoj memoriji ali zahtijevaju mnogo režijskog rada pri svakom pristupu memoriji kao što je LRU.
- Dinamičke dijeljene biblioteke je veoma lako realizovati koristeći segmente.

- Razni segmenti mogu imati razne tipove zaštite. Npr. segment programskog koda može se samo izvršavati, segment za program u toku razvoja treba čitanje, pisanje i izvršavanje, u *stack* smije se i pisati i čitati, segment sa bitmapiranim slikama možemo samo čitati (konstantni niz), segmenti koji pripadaju jezgri se mogu opremiti različitim pravima.

Mana segmentacije je što programer aplikacija, pogotovo ako se pišu u asemblerskom jeziku treba biti svjestan postojanja segmenata. Ponekad i u višim programskim jezicima se uvode jezička proširenja koja kažu da li podatke držati u posebnim segmentima ili ne, jer je prebacivanje podataka između različitih segmenata ponekad sporije nego unutar istog segmenta. Nažalost, ta proširenja nisu standardni dio programskih jezika pa njihova upotreba čini programe neprenosivim.



Slika 60 Eksterna fragmentacija kod segmentacije

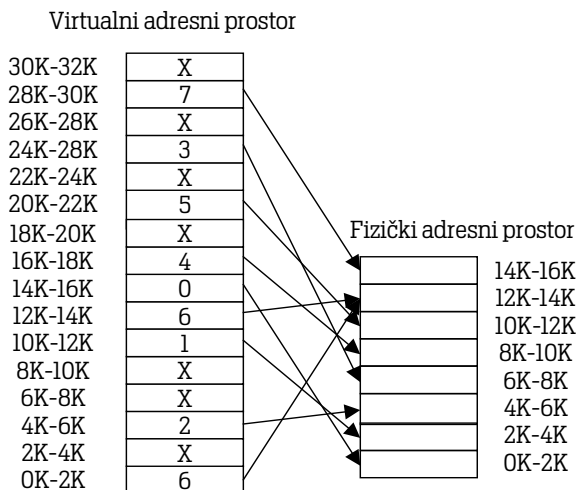
Druga mana se vidi na slici Slika 60. U početku su zauzeta 4 segmenata. Kako se segmenti izbacuju iz memorije i ubacuju ili alociraju, nakon izvjesnog vremena rada sistema, memorija je podijeljena na veliki broj blokova od kojih su neki segmenti a neki praznine. (Slika 60d). Javlja se problem eksterne fragmentacije, već opisan u sistemima s promjenjivim particijama. Kada se to desi, potrebno je premještati segmente u memoriji i obaviti njihovu kompakciju (Slika 60e). Najveća mana segmentacije je što ne dopušta segmente veće od fizičke memorije ili sistemskog limita za njihovu veličinu. Stoga na sistemima od npr. 4 M RAM-a sa segmentacijom se ne može računati inverzna matrica dimenzija 3000x3000 jer ne može stati u segment.

7.6. Straničenje

Najčešća tehnika za realizaciju virtualne memorije je **straničenje** (eng. *paging*). Kod ovog pristupa svaki proces ima kontinualan virtualni adresni prostor koji se u MMU transformiše u diskontinualna područja fizičke memorije.

Slika 61 pokazuje primjer preslikavanja virtualne u fizičku adresu tehnikom straničenja. Virtualni adresni prostor od 32 kilobajta je podijeljen na dijelove jednake veličine, koji se zovu stranice. Stranica u ovom primjeru je veličine 2 kilobajta. Fizička memorija od 16 kilobajta se dijeli na dijelove čija je veličina jednaka veličini stranice koji se zovu okviri. Stranice (engl. *page*) i okviri stranica (engl. *page fram*)

imaju **uvijek jednaku veličinu**. Stranica može biti mapirana na fizički okvir (primjer virtualna stranica 14 koja predstavlja područje virtualnih adresa između 28 K i 30 K mapirana je na fizički okvir 7 koji predstavlja područje fizičkih adresa između 14 K i 16 K) ili da nisu prisutne u memoriji.



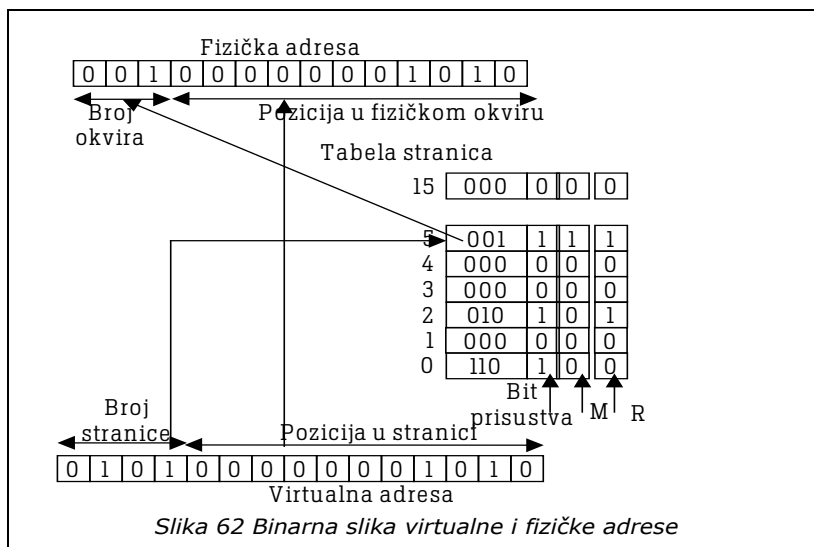
Slika 61 Preslikavanje virtualne adrese u fizičku

Iz virtualne adrese se direktno dobiva broj virtualne (logičke) stranice. Virtualna adresa se dijeli na broj virtualne stranice (biti na višim pozicijama) i poziciju unutar stranice (biti na nižim pozicijama). Iz broja stranice se dobiva fizička adresa korištenjem tabele stranica (engl. *page table*) koja preslikava virtualnu stranicu u fizičku stranicu. Ova tabela se dodjeljuje svakom procesu i mijenja se u toku izvršavanja procesa, kao i prilikom izmjene konteksta procesa. Broj virtualne stranice predstavlja indeks u tabeli stranica. U toj tabeli se nalazi broj odgovarajućeg okvira stranice, i informacija da li je stranica u fizičkoj memoriji. Broj okvira stranice i pozicija unutar stranice daju fizičku adresu koja se prosljeđuje memorijskim integrisanim kolima.

Neka proces pristupa logičkoj adresi dekadno 10250 (binarno 0101000000001010). (Slika 62). Ta adresa se dijeli na dva dijela. Četiri bita na najznačajnijim pozicijama predstavljaju indeks unutar tabele stranica. Preostalih 11 bita na nižim pozicijama logičke adrese predstavlja poziciju od početka stranice. To omogućava ukupno 16 virtualnih stranica (od 0 do 15), a sa 11 bita može se adresirati unutar svake stranice po $2^{11} = 2048$ bajta = 2KB memorije.

Pošto fizički okviri mogu počinjati samo na memorijskim adresama koje su djeljive sa veličinom stranice, u tabeli stranica se ne mora navoditi početna adresa fizičkog okvira nego samo njegov redni broj. U datom primjeru, kako ima 8 fizičkih okvira, redni broj okvira se može predstaviti sa $\log_2 8 = 3$ bita. Pored ovoga potreban je još jedan bit koji označava da li je navedena virtualna stranica u fizičkoj memoriji. Taj bit se zove **bit prisutnosti** (*present bit*) i ima npr. vrijednost 1 ako je stranica u fizičkoj memoriji, a 0 ako nije. Fizička adresa dobiva se na sljedeći način: na osnovu 4 najznačajnija bita adrese koju je poslao procesor (0101) dobije se indeks okvira

stranice (001). Pogleda se iz bita prisustva da li je navedena stranica učitana u fizičku memoriju. Ako jeste, iza broja okvira (001) dodaje se pozicija unutar stranice (0000001010), pa se dobije 0010000001010.



Slika 62 Binarna slika virtualne i fizičke adrese

U izračunatoj fizičkoj adresi tri najznačajnija bita adrese označavaju okvir stranice, a ostalih 11 bitova daju relativnu poziciju unutar tog okvira.

Situacija se razlikuje ako tražena virtualna stranica nije u fizičkoj memoriji. Ako se pokuša pristupiti stranici koja je u tabeli označena kao neprisutna, MMU pokreće procesorski izuzetak koji se zove greška stranice (engl. *page fault*). Operativni sistem tada bira jedan okvir stranice i piše njegov sadržaj nazad na disk. Poslije toga ubacuje stranicu kojoj se pokušalo pristupiti ali je prouzrokovala grešku stranice u upravo oslobođeni okvir stranice, prilagođava tabelu stranica i ponovo izvršava prekinutu komandu.

Da bi se znalo koji okvir je manje korišten, u tablicu stranica se pored rednog broja fizičkog okvira i bita prisutnosti dodaju **bit modifikacije M** i **bit referenciranja R**. Svaki put kada se čita ili piše podatak iz memorijskog područja date stranice, u tabeli stranica se bit R postavi na vrijednost 1. Ako se podatak upisuje u tom području, pored bita R, na vrijednost 1 se postavlja i bit modifikacije M. U tablici stranica se mogu nalaziti i **biti zaštite**, npr. biti koji određuju da se u toj stranici nalaze podaci koji se mogu samo čitati, ili da stranici smije pristupati samo jezgro.

Tabele stranica se drže u memoriji, jer su prevelike da se čuvaju u samom procesoru. Zbog toga ovako opisano straničenje bi imalo ogroman negativan uticaj na performanse sistema. Neka instrukcija upisuje sadržaj registra u memorijsku lokaciju. Bez straničenja su ovdje potrebna dva pristupa memoriji: jedan da se instrukcija pročita iz memorije i drugi da se upiše vrijednost. Sa straničenjem bi bilo potrebno šest pristupa memoriji: da se nađe u tabeli stranica gdje je instrukcija, upiše R bit za stranicu instrukcije u tabeli, pročita instrukcija iz memorije, nađe stranica u tabeli u koju se upisuje podatak, upiše M i R bit za tu stranicu u tabeli i upiše podatak u memoriju.

Da bi se izbjeglo ovoliko pristupanje memoriji više puta za istu instrukciju, uvodi se pamćenje elemenata tablice stranica sa kojim se trenutno radi u samom MMU. Skup specijalnih registara koji predstavljaju kopije najčešće korištenih elemenata tabele stranica zove se *Translation lookaside bafer* (TLB). On se nalazi unutar MMU ali se sastoji od relativno malog broja elemenata. Kada se jedna virtualna adresa pošalje MMU, on prvo provjerava da li će odgovarajući broj stranice pronaći u TLB. Ako se pronađe odgovarajući broj stranice, tada se broj okvira stranice dobije iz TLB. Ako se virtualna stranica ne nalazi u TLB, MMU tada normalno pristupa tabeli stranica, ali za buduće pristupe upisuje element tabele u TLB pri čemu izbacuje neki stari element tablice. Noviji procesori (npr. *Intel Sandy Bridge*) imaju dva nivoa TLB registara, pa se element tabele stranica može najbrže naći ako je u primarnim TLB registrima, zatim u sekundarnim TLB registrima, pa ako nije ni tu traži se da li je u keš memoriji prvog nivoa, pa u keš memoriji drugog nivoa, pa u keš memoriji trećeg nivoa i ako nije nađen ni tu tek se onda traži element tabele u RAM-u (pretraga u svim mjestima može početi istovremeno, pa se preuzima podatak od jedinice koja se prva javila).

7.7.Veličina stranice

Pri dizajnu procesora veličina stranice je obično fiksirana, ili pruža programeru operativnog sistema izbor između svega nekoliko mogućnosti. Na primjer, na Intel Core i7 procesoru, stranica može biti, zavisno od režima rada: 4K, 2M, 4M ili čak 1G. Prednost male veličine stranice je u manjoj unutrašnjoj fragmentacije koja vodi do velikog neiskorištenog prostora u memoriji. Nedostaci male veličine stranice je što programima treba puno stranica, a to dovodi do velike tabele stranica. Koja je optimalna veličina stranice?

Neka je prosječna veličina procesa s bajta, veličina stranice p bajta, a svaki element tabele stranica zauzima e bajta. Svaki proces zauzima onda u prosjeku

$\frac{s}{p}$ stranica i time $\frac{s \cdot e}{p}$ bajta u tabeli. Zbog interne fragmentacije se gubi u prosjeku

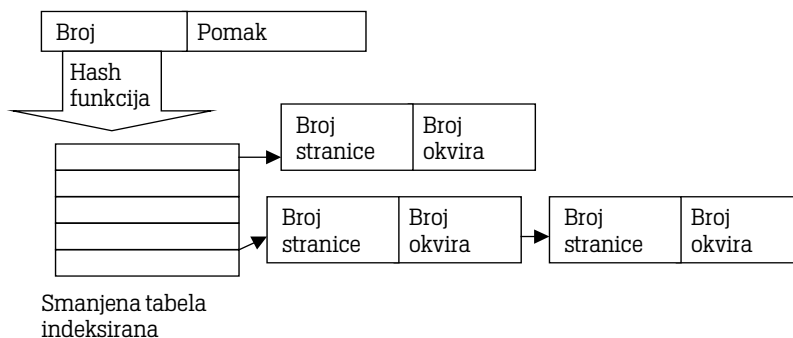
još $\frac{p}{2}$ bajta, tako da je ukupno prosječno dodatno zauzeće memorije

$potrošnja = \frac{s \cdot e}{p} + \frac{p}{2}$. Prvi sabirak je velik kad je stranica mala, drugi je velik kad je

stranica velika. Ako se nađe prvi izvod ovog izraza po p i izjednači sa 0 dobivamo optimalnu veličinu stranice kad je $p = \sqrt{2 \cdot s \cdot e}$.

7.8.Invertovana tabela stranica

Pored povećanja veličine stranice, postoje još dva načina za smanjenje veličine tabele stranica. Kod velikih adresnih prostora, za uobičajene veličine stranica, ukupan broj fizičkih okvira postaje veliki i tabela stranica bi zauzela većinu fizičke memorije. Npr. za 64 bitni virtualni adresni prostor i 4 kilobajtnu stranicu, potrebna veličina tabele je 2^{52} elemenata.



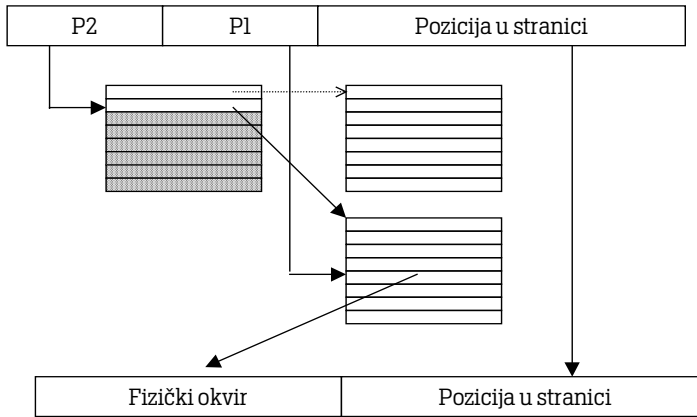
Slika 63 Invertovana tabela stranica

Da bi se smanjila veličina tabele stranica koriste se dvije tehnike: invertovana tabela stranica i straničenje na više nivoa. **Invertovana tabela stranica** korištena je na procesorima PowerPC, UltraSPARC i IA-64. Umjesto da se indeks u tabeli računa prema broju logičke stranice, on se računa prema broju fizičkog okvira. Pošto više logičkih stranica može biti mapirano u isti fizički okvir, virtualnu adresu postaje značajno komplikovanije preslikati na jednu adresu. Kada proces želi pristupiti virtualnoj stranici p , hardver ne može jednostavno naći fizički okvir stranice tako što koristi virtualni broj stranice kao indeks, nego mora pretražiti čitavu tabelu. Da bi se ta pretraga ubrzala koriste se *hash*-tabele sa vrijednostima koje se računaju iz broja logičke stranice. Sve logičke stranice koje imaju istu *hash* vrijednost su povezane u listu kao na slici (Slika 63). Kada se želi pristupiti logičkoj adresi, prvo se izračuna *hash* vrijednost iz broja logičke stranice i iz nje pozicija u tabeli. Element tabele pokazuje na ulančanu listu koja se pretraži da se nađe odgovarajuće preslikavanje iz broja stranice u broj fizičkog okvira. Kada je broj stranice pronađen, novi par koji se sastoji od virtualne stranice i fizičkog okvira stranice se unese u TLB što ubrzava kasniju pretragu.

7.9.Hijerarhijsko straničenje

Praktičniji način za smanjenje veličine tabele stranica je straničenje na dva ili više nivoa. Kod metoda straničenja na dva nivoa (Slika 64), logička adresa se dijeli na tri dijela: redni broj tabele stranica, broj stranice u tabeli i pozicija unutar stranice. Neka je proces velik 6 M virtualne memorije, adresni prostor je 32 bitni, a veličina stranice 4K (12 bitna adresa). Sa straničenjem na jednom nivou, tablica bi morala imati odgovor na pristup bilo kojoj memorijskoj adresi, s to predstavlja 2^{20} , tj, preko milion elemenata, iako bi samo prvih 1536 zaista pokazivalo na fizičku memorijsku lokaciju, a ostale bi imale vrijednosti da stranica nije prisutna. Tada bi u memoriji tablica stranica bila četiri megabajta za proces. Ako s druge strane logičku adresu podijelimo na dva nivoa straničenja, s deset bita za redni broj tablice (P2), deset bita za redni broj stranice u tablici (P1) i dvanaest bita za adresu unutar stranice, za predstavljanje procesa veličine 6 M u memoriji bilo bi dovoljne tri dodatne memorijske stranice: stranica s imenikom čija dva elementa pokazuju na tabele stranica, a ostalih 510 su prazni, jedna stranica s tabelom stranica čiji svi elementi pokazuju na fizičke adrese i druga stranica s tabelom stranica čijih polovina

elemenata pokazuju na fizičke adrese, a polovina na stranice koje nisu u memoriji. Na tablice stranica se u ovom slučaju troši 12 K.



Slika 64 Straničenje na dva nivoa

Za veće adresne prostore, može se ići i na više nivoa tablica stranica. Štoviše, samo tablica najvišeg nivoa mora stalno biti u memoriji. Tablice na nižim nivoima se mogu čuvati i izvan centralne memorije i ubacivati u memoriju tek kada se zaista pristupi memorijskoj lokaciji na koju trebaju pokazivati.

7.10. Postupak kod greške stranice

Kada instrukcija pristupi memorijskoj lokaciji koja je u tabeli stranica označena kao ne-dodijeljena fizičkom okviru, dešava se greška stranice. Greška stranice je hardverski izuzetak koji izaziva prekid izvršavanja procesa koji ju je izazvao. Procesor počinje da izvršava rutinu unutar jezgra operativnog sistema.

U rutini za obradu greške stranice najprije se sačuva stanje svih registara procesora u kontrolni blok procesa, kao i kod izmjene konteksta procesa. Neka je, na primjer, grešku stranice izazvao pokušaj čitanja sa virtualne adrese 20000, čija stranica je označena kao van memorije. Rutina za obradu greške stranice će pogledati koja logička adresa se pokušala čitati analizom instrukcije ili specijalnim skrivenim registrom. Analiza instrukcije može bit složen zadatak, jer jedna instrukcija može da pristupa jednoj memorijskoj lokaciji koja je unutar jedne stranice (onoj na kojoj je programski brojač), može se sastojati od više bajtova od kojih su neki na kraju jedne stranice, a neki na početku druge stranice, instrukcija može pripadati jednoj stranici a memorijska lokacija podataka koje ona čita drugoj, ali na nekim procesorima (VAX-11) čak i osam različitih memorijskih stranica može biti doticano jednom instrukcijom. Kada se saznaju sve lokacije kojima bi instrukcija mogla pristupiti, za svaku od tih lokacija se pogleda kojoj logičkoj stranici instrukcija pokušava pristupiti. Ako izabrana logička stranica prevazilazi predviđeni memorijski opseg virtualnog adresnog prostora procesa, proces se ubija odgovarajućim signalom, jer je ovu grešku stranice izazvala nedozvoljena adresa.

S druge strane, ako je navedena logička stranica označena kao neprisutna u RAM memoriji, ali u ispravnom opsegu, tada je ona dio virtualne memorije. Njen sadržaj

se može nalaziti u datoteci za virtualnu memoriju (kao rezultat ranijeg snimanja stranice), u datoteci sa programom koji odgovara procesu izazivaču greške stranice (prilikom pokretanja procesa i učitavanja njegovih dijelova) ili biti potpuno novi pa se ne treba učitavati sa diska.

Sada se treba naći fizički okvir kome će se pridružiti ova logička stranica. Moguća su dva slučaja:

- a) Postoji fizički okvir koji nije dodijeljen ni jednoj logičkoj stranici. U ovom slučaju se taj okvir dodjeljuje logičkoj stranici koja je izazvala grešku stranice.
- b) Svi fizički okviri su dodijeljeni nekoj od logičkih stranica. U ovom slučaju se odabira jedan od fizičkih okvira koji će biti izbačen, nekim od algoritama koji će biti kasnije opisan. Okvir predviđen za izbacivanje vjerovatno pripada nekom aktivnom procesu. Ako kopija sadržaja tog okvira ne postoji na disku ili ako postoji, ali je u međuvremenu promijenjena ($M \text{ bit u tabeli stranica}=1$), sadržaj tog okvira se snima u datoteku za virtualnu memoriju. Modifikuje se tabela stranica tako da logička stranica koja je pokazivala na izbačeni fizički okvir postane stranica koja nije u memoriji. Izbačeni fizički okvir se dodjeljuje logičkoj stranici koja je izazvala grešku stranice.

Nakon ovoga, treba popraviti tablicu stranica da logička stranica koja je izazvala grešku stranice pokazuje na fizički okvir koji joj je sada dodijeljen. Ukoliko je sadržaj logičke stranice već ranije postojao na disku, učitava se u navedeni fizički okvir.

Sve je spremno da proces proba ponovo pristupiti lokaciji. U kontrolnom bloku procesa se postavi polje zapamćenog programskog brojača da proba ponovo pristupiti lokaciji. Konačno pozove se dispečer da izvrši promjenu konteksta na proces koji je izazvao grešku stranice.

Sa ovakvim postupkom postiže se efekt kao da je ta memorijska lokacija uvijek bila u fizičkoj memoriji, ali pristup toj lokaciji zahtijeva značajan vremenski period. Stoga su istraživani algoritmi da se to dešava što je moguće rjeđe. Na učestalost grešaka stranice utiču načini izbora fizičkih okvira, izbora stranice za učitavanje, određivanja vremena izbacivanja stranica i izbora stranice za izbacivanje.

7.11.Izbor fizičkih okvira

Pri pokretanju procesa, može mu se dodijeliti određen broj fizičkih okvira u koje će on smještati svoje logičke stranice. Postavlja se pitanje koje fizičke okvire dodijeliti procesu i koliko njih.

Metoda **jednake raspodjele** je najjednostavnija metoda. Ako ima m okvira i n procesa, prema metodi jednake raspodjele, svakom se procesu dodjeljuje jednaka količina okvira m/n . Neka je recimo sistemska memorija jednaka 128 M, a izvršavaju se 4 procesa, svakom će biti dodijeljeno po 32 M. Mana metode se vidi u primjeru ako jedan proces traži samo 1 M virtualnog adresnog prostora, a drugi 80 M. Prvi će imati dosta neiskorištene memorije, a drugi će generirati mnogo grešaka stranica.

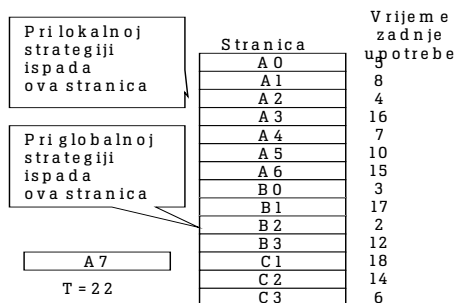
Prema **metodi proporcionalne raspodjele**, procesima koji su prijavili veći adresni prostor, dodjeljuje se i relativno više fizičkih okvira. Ako je m ukupan broj

okvira fizičke memorije, n broj procesa, a S_i -veličina virtualnog memorijskog prostora procesa p_i , tada se svakom procesu i dodjeljuje $a_i = \frac{S_i}{\sum_{i=1}^n S_i}$ fizičkih okvira.

Prioritetne metode dodjeljuju procesu više fizičkih okvira po nekom drugom kriteriju od njegove veličine. Tako na primjer, MS Windows daje više fizičkih okvira onom procesu s kojim korisnik trenutno radi (*front-end proces*).

Broj dodijeljenih okvira ne smije pasti ispod minimalne vrijednosti, koja dovodi do čestih grešaka stranica. Pojava čestog zamjenjivanja stranica, koja nastaje kao posljedica visokog stepena multiprogramiranja, naziva se **efekat zasićenja** (engl. *thrashing*). U tom slučaju se neki procesi moraju suspendovati.

Zavisno od dizajna sistema, dodijeljeni broj fizičkih okvira procesu može ostati isti tokom izvršavanja procesa ili se mijenjati prebacivanjem tih okvira između raznih procesa.



Slika 65 Lokalna i globalna strategija straničenja

U slučaju da broj dodijeljenih fizičkih okvira procesu ostaje isti, govorimo o **lokalnoj strategiji straničenja**. Ako dođe do greške stranice, moramo izbaciti stranicu u skupu dodijeljenih okvira. Prednost pristupa je u bržem izboru te stranice, jer se ne moraju čitati tablice stranica drugih procesa. Očigledni su nedostaci ove strategije: nekada je proces u stanju kada ima nedovoljno fizičkih okvira, a nekada u stanju kada ih ima više nego što treba. U prvom slučaju se dešavaju česte greške stranica, a u drugom slučaju drugi procesi kojima u tom trenutku treba više fizičke memorije ne mogu iskoristiti one fizičke okvire koji su dodijeljeni procesu koji ih drži a ne koristi.

Ako proces mijenja skup fizičkih okvira na račun drugih procesa, tada je riječ o **globalnoj strategiji straničenja**. Ako proces A generiše grešku stranice, stranicu koja će biti izbačena iz RAM se ne traži samo u skupu okvira pridruženih tom procesu, već u skupu svih okvira. Stoga se može desiti, da se izbací neka stranica iz okvira koji je bio dodijeljen procesu B, a na to mjesto ubaci tražena stranica procesa A. Ova strategija bolje koristi memoriju ali zahtijeva više pretraživanja, jer obično svaki proces ima svoju tabelu stranica, pa treba pri određivanju fizičkog okvira pretražiti sve tabele stranica (ili imati dodatnu tabelu indeksiranu po fizičkim okvirima).

Pogledajmo sliku Slika 65. Neka je korišten algoritam koji izbacuje onu stranicu koja je najranije zadnji puta korištena. Ako proces A proizvede grešku stranice, treba li algoritam zamjene u obzir uzeti samo njegove stranice ili treba pretražiti čitavu memoriju? U primjeru sa slike, ako se gledaju samo njegove stranice, kandidat za izbacivanje je stranica 2 procesa A, kao najranije zadnji put korištena stranica među stranicama ovog procesa. Ako je korištena lokalna strategija straničenja, kandidat za izbacivanje je stranica 2 procesa B, kao najranije zadnji put korištena stranica među svim stranicama.

7.12. Izbor stranica za učitavanje

Za izbor trenutka kada će se stranica učitavati u memoriju, postoje dvije osnovne strategije:

1. Kod **učitavanja po potrebi** (engl. *demand paging*) stranica se ubacuje sa diska u memoriju, samo onda kada je MMU traži. Ovaj pristup štedi radnu memoriju, jer se u njoj čuvaju samo stvarno potrebne stranice. Loša strana je sporo pokretanje programa, jer će u ranoj fazi program zahtijevati više memorijskih lokacija, a ovim algoritmom će se puniti jedan po jedan memorijski okvir bez iskorištenja optimalnog trenutka rotacije diska.

2. Kod **učitavanja sa predviđanjem** (engl. *anticipatory paging*) pokušava se pogoditi koje će stranice biti potrebne za rad procesa. Jezgro ili pozadinski program ubaci u memoriju te stranice, obično u vrijeme kada procesor nije zauzet izvršavanjem aplikacija. Ako je pretpostavka koje stranice treba učitati bila tačna, smanjiće se kasniji broj grešaka stranice i izvršavanje procesa dobiva na brzini. Ako je pretpostavka bila loša, od učitavanja nema nikakve koristi. Kako cijena kapacitet interne memorije raste, ovaj pristup je sve popularniji.

7.13. Vrijeme brisanja stranica

Slično varijantama vremena učitavanja (po potrebi i sa predviđanjem), i brisanje stranica iz radne memorije se može raditi u različitim vremenima. U jednoj varijanti, stranice se izbacuju iz memorije samo kada nekom od procesa zatreba određena memorijska stranica, a nema raspoloživog fizičkog okvira. Radi jednostavnosti, u analizi algoritama za izbor stranice za izbacivanje pretpostavlja se ovaj slučaj.

Pokazalo se da su performanse sistema bolje ako uvijek ima dovoljno slobodnih stranica. Da bi se to postiglo, umjesto da se neka stranica izbaci tek kada se desi greška druge stranice i svi okviri budu zauzeti, može se koristiti **pozadinski program** koji može stranice izbaciti i ranije. Taj program je uglavnom neaktivan, a onda izbaci stranice kada broj slobodnih okvira padne ispod nekog praga. Ovo će ubrzati kasniju dodjelu dodatne memorije procesima.

7.14. Algoritmi za izbor stranice za zamjenu

Ako je potrebno pristupiti stranici koja nije u memoriji, dešava se greška stranice i treba neku od stranica izbaciti iz radne memorije i u njen okvir upisati stranicu koja je traženo. Uz to, ako je sadržaj izabranog okvira izmijenjen, OS ga mora prebaciti na disk s ciljem aktualiziranja kopije koja se nalazi na disku. Nije svejedno koju stranicu izbacujemo, jer je disk znatno sporiji od memorije. Loš izbor stranice može negativno uticati na brzinu rada cijelog sistema. Možemo pretpostaviti da će stranica koja se

mnogo koristi vjerovatno vrlo brzo ponovo biti potrebna i ponovo dovesti do greške stranice, ako sistem izbaci takvu stranicu.

Slijedi opis nekoliko popularnih algoritama za izbor stranice koja će biti izbačena. Prvo će biti opisani algoritmi koji ne analiziraju dotadašnji pristup memoriji (slučajni, FIFO, LDF), zatim algoritmi koji analiziraju budući pristup (Optimalni), pa algoritmi koji uzimaju vrijeme dotadašnjeg pristupa memoriji (NRU, *Second Chance*, *Clock*, LRU), algoritme koji prate broj pristupa memoriji (*Gclock*, LFU, MFU, *Aging*), zatim algoritmi koji uzimaju u obzir i ponašanje stranica u memoriji i stranica na disku (Adaptivna zamjena) i na kraju algoritmi koji izbacuju više stranica (algoritam sa radnim skupom, *WSClock*, *Clock* s dvije kazaljke).

7.15.Slučajni izbor stranice

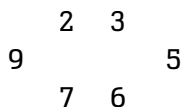
Najjednostavnije rješenje je izbaciti bilo koju stranicu koja je trenutno u radnoj memoriji. Osnovna prednost ovog algoritma je što nije potrebno dodatno vrijeme za pretraživanje stranica u tabelama radi donošenje odluke o tome koja stranica će biti izbačena, niti su potrebne dodatne memorijske strukture osim same tabele stranica. Algoritam ne vodi ni na koji način računa o korištenosti i značaju neke memorijske stranice, pa može rezultovati lošim performansama. Algoritam je korišten u OS/390 iz 1996. kao rezervni algoritam kada drugi korišteni algoritmi pokazuju loše performanse.

7.16.FIFO: prvi učitani okvir

Naredna logična ideja je izbaciti najstariju stranicu. Pored tabele stranica, održava se i lista svih stranica koje su trenutno u memoriji. Lista je poredana po vremenu prvog pristupa stranici, tako da se stranica koja je najstarija nalazi na početku liste, a na njenom kraju je ona koja je zadnja ubačena u memoriju. Kod greške stranice, stranica sa početka liste se uklanja a na kraj se dodaje nova stranica. Održavanje i pretraživanje ove liste je relativno brzo, a algoritam se može primijeniti i u jednostavnim sistemima straničenja koji nemaju informaciju o korištenju stranice. Pretpostavka algoritma je da stranice koje su davno učitane sadrže instrukcije koje su davno izvršene ili podatke koji više nisu potrebni. To ne mora biti tačno. Može se zbog toga izbaciti važna stranica iz memorije. Primjer predstavlja stranica koja pripada glavnoj petlji nekog programa, učitana je među prvim i najčešće se koristi. FIFO algoritam će je izbaciti i zatim ponovo vratiti. U nešto izmijenjenoj verziji, ovaj algoritam je korišten na operativnom sistemu VAX/VMS.

7.17.LDF (Longest Distance First) - Najduža udaljenost prvo

Iako je decenijama izgledalo da se na polju jednostavnih algoritama za zamjenu stranica koji ne uzimaju u obzir korištenost stranice nema šta više istraživati, Gyanendra Kumar je 2017. objavio rad o algoritmu LDF. Ideja algoritma je da se većina programa izvršava sekvencijalno, pa je pri izvršavanju programskog koda ili čitanju podataka iz logičke stranice k , vjerovatno ubrzo potrebna stranica $k+1$. Udaljenost između stranica se računa tako da sve logičke stranice kojima proces pristupa poredaju u krug u rastućem redoslijedu logičkog broja stranice, Na primjer logičke stranice 3,5,2,6,7,9 poredane u krug izgledaju ovako.



Udaljenost između stranica 9 i 6 je ovdje 2, putem preko stranice 7, a udaljenost između stranica 2 i 6 je 3, putem preko stranica 3, 5 i 6. Kada neka stranica nije u memoriji, a pristupa joj se, izbacuje se ona stranica koja je najdalja od nje. Ako postoje u memoriji dvije stranice koje su jednako udaljene od stranice koja je izazvala grešku, izbacuje se ona sa manjim rednim brojem. Algoritam se ponaša dobro za stranice s programskim kodom, ali za podatke koji se naizmjenično čitaju iz raznih krajeva memorije (npr. neki algoritmi za sortiranje) može imati loše performanse.

7.18. Optimalni algoritam

László Bélády je 1966. predstavio optimalni algoritam. U ovom algoritmu treba izbaciti onu stranicu za koju se smatra da će najkasnije biti potrebna u budućnosti ili još bolje ako neće uopće više biti potrebna. Ako će stranica broj 1 biti potrebna nakon 3 sekunde rada, a stranica broj 2 nakon 30 sekundi, bolje je izbaciti stranicu broj 2. Dokazano je da ovakva strategija rezultuje najmanjim brojem grešaka stranice, ali ima jedan ogroman nedostatak. Ovaj algoritam nije moguće realizovati, jer se ne može predvidjeti ponašanje programa u budućnosti. Upotrebnost vrijednost ovog algoritma je jedino za ocjenu drugih algoritama, nakon što je prethodno evidentirano kojim je stranicama pristupano i uočeno koliko grešaka stranica su ti algoritmi napravili. Ali, ako se budućnost ne može predvidjeti, prošlost se može pratiti, pa naredni algoritmi analiziraju dosadašnje pristupanje stranicama kako bi se približili optimalnom algoritmu.

7.19. NRU (Not Recently Used): okvir koji nije skoro korišten

Slučajni izbor stranice ne vodi računa o tome da li je stranici pristupano ili da li je mijenjana u međuvremenu. Znatno unapređenje ovog algoritma može se postići ako tabele stranica imaju informacije o tome. Mnogi sistemi za upravljanje memorijom podržavaju u tablici stranica dva statusna bita koji označavaju da li je sadržaju te stranice pristupano. Bit R (engl. *Referenced*) označava da li se pristupalo tom okviru. Bit M (engl. *modified*) znači da li je mijenjan sadržaj tog okvira. Bit R se postavlja automatski na 1 svaki put kad se čita ili piše bajt koji pripada toj stranici. Bit M se postavlja na 1 kada se piše podatak u bilo koji bajt koji pripada toj stranici. Kad je neki od ova dva bita postavljen na 1, on zadržava tu vrijednost sve dok operativni sistem ne promijeni njihovu vrijednost u tabeli stranica. Pri pravljenju tabele stranica, tokom pokretanja procesa, operativni sistem postavlja oba bita za sve stranice u tabeli na 0. U algoritmu NRU se periodično, na svaki prekid sata, R bit briše, ali ne i M bit. Tako je R bit postavljen samo kod stranica koje su u zadnje vrijeme korištene.

Kombinacijom R i M bita stranice se dijele u četiri klase:

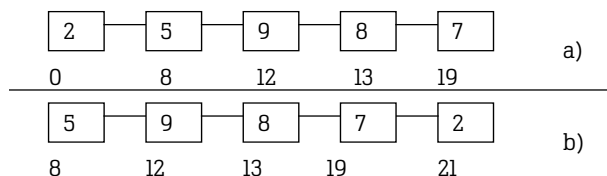
Kl.	R	M	Situacija
0	0	0	Ova stranica nije ni čitana ni mijenjana u zadnje vrijeme
1	0	1	Stranica je mijenjana u prošlosti, ali od zadnjeg čišćenja R bita nije joj u međuvremenu pristupano
2	1	0	Stranica je čitana od zadnjeg čišćenja R bita
3	1	1	Stranica je mijenjana od zadnjeg čišćenja R bita

Kada se pojavi greška stranice algoritam NRU (*Not Recently Used*) pregleda sve stranice i odabere za brisanje iz memorije bilo koju stranicu iz najniže klase. Time se smanjuje potreba za upisivanjem izbačenih stranica na disk i izbacivanjem nedavno korištenih stranica.

7.20.Druga šansa (Second Chance)

Glavna mana algoritma FIFO je što može izbaciti vrlo često korištenu stranicu jer je rano ubačena u operativnu memoriju. Algoritam je moguće znatno poboljšati uzimajući u obzir pored starosti stranice i stanje njenog bita pristupa, čime nastaje algoritam Druga šansa. Ovaj algoritam sprječava odbacivanje stranica koje se često koriste ispitujući R bit najstarije stranice. Ako je R=0, to znači da je stranica dugo u memoriji a da joj nije pristupano. Prema tome ona nije potrebna, može se izbaciti i u njen stranični okvir staviti nova. Ako je R=1, to znači da je stranica dugo u memoriji, ali je nedavno korištena. Tada će se staviti R=0 a stranica prebaciti na početak liste sortirane po vremena dolaska (tj. njeno vrijeme dolaska postavljamo na trenutno) i analizirati sljedeću stranicu. Znači, ako je stranica korištena dobiva još jednu šansu.

Rad ovog algoritma je prikazan na slici Slika 66a. Tu se vide stranice čuvane u ulančanoj listi i sortirane prema vremenu kad su stigle u memoriju.



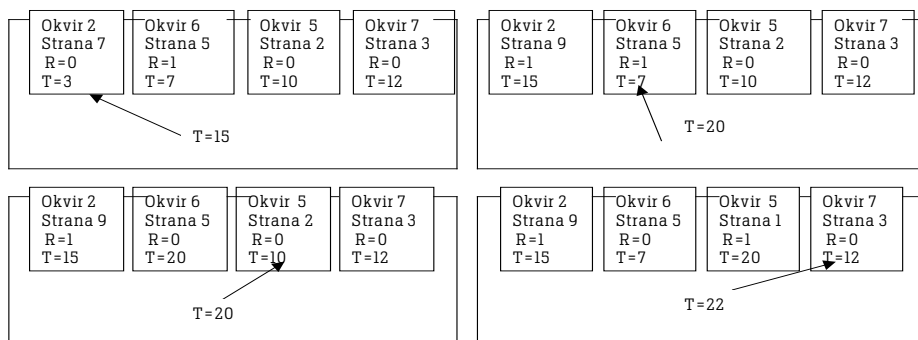
Slika 66 Second chance

Neka se nakon situacije (a) desila greška stranice u trenutku 21. Najstarija stranica je 2, koja je stigla u trenutku 0 (pokretanja procesa). Ako stranica 2 ima R=0, stranica se uklanja iz memorije. S druge strane ako je R bit postavljen (R=1), stranica 2 se stavlja na drugi kraj liste (Slika 66b) i njeno vrijeme ulaska u memoriju se postavlja na trenutno vrijeme (21). Za ovu stranicu R bit se postavlja na 0. Potraga za odgovarajućom stranicom nastavlja sa stranicom 5, koja je sljedeća po vremenu dolaska.

7.21.Satni (Clock page) algoritam

Algoritam Druga šansa se može ubrzati, ako se okviri stranica čuvaju u kružnoj listi, kao što je prikazano na slici Slika 67. Jedan pokazivač pokazuje na stranicu koja je najranije učitana. Kada dođe do greške stranice, provjeri se R bit stranice na koju pokazuje pokazivač. Ako je R=0 stranica se izbacuje a nova se ubacuje u okvir koji

je zauzimala izbačena stranica i pokazivač se pomjera za jedno mjesto. Ako je $R = 1$, tada se promijeni R da postane 0 i pokazivač se pomjera na sljedeću stranicu, čime se nastavlja traženje. Ako se pri tome obiđe cijeli krug, svi R biti su postali 0, pa će se stranica obrisati. Satni algoritam ima isti broj grešaka stranica kao algoritam druga šansa, i bira potpuno iste stranice, ali je brži u realizaciji jer se pomiče samo jedan pokazivač.



Slika 67 Satni algoritam

7.22. Najdavnije zadnji put korišten - Least recently used (LRU) algoritam

Budućnost se ne može predvidjeti, ali inverzijom ideje optimalnog algoritma može se ovaj algoritam aproksimirati. Tako umjesto odbacivanja stranice koja će najkasnije u budućnosti biti korištena, algoritam LRU (*Least Recently Used*) radi na principu da se odbacuje stranica koja se najranije zadnji put koristila u prošlosti. Ova inverzija je zasnovana na pretpostavci da stranice koje su se puno koristile u zadnjih nekoliko instrukcija će biti vjerovatno korištene u nekoliko budućih instrukcija.

Ovaj algoritam je dobar, ali teško ostvarljiv. Osnovni problem sa ovim algoritmom je što zahtijeva posebnu hardversku podršku. Tablica stranica treba za svaku stranicu, pored broja fizičkog okvira, bita prisustva i bitova R i M , da ažurira i polje o vremenu zadnjeg pristupa. Vrijeme se može evidentirati posebnim brojačem koji se uvećava nakon svake procesorske instrukcije ili memorijskog ciklusa i pri svakom čitanju ili pisanju u memorijsku stranicu treba prepisati vrijednost tog brojača u odgovarajući TLB element tablice stranica. Kada se desi greška stranica, pregledaju se brojači svih stranica i odabere među njima ona sa najmanjom vrijednosti brojača.

Ako tablica stranica nema odgovarajuće polje za vrijednost brojača, realizacija ovog algoritma je softverska i nedopustivo je spora. Tablica stranica bi u tom slučaju bila napravljena tako da svaki pristup memoriji izaziva grešku stranice. U potprogramu koji se poziva prilikom greške stranice, upisivalo bi se u dodatnu tabelu zadnje vrijeme pristupa odgovarajućoj stranici. To bi značilo usporenje rada računara više desetina puta.

7.23.GClock

Stranica kojoj je nedavno pristupano nije nužno stranica koja se često koristi. Sada ćemo pogledati algoritme koji analiziraju učestalost pristupa memorijskoj stranici. Prvi od njih, GClock, zahtijeva hardversku podršku. Tablica stranica, kao i kod algoritma Clock treba biti uređena u obliku kružne liste, a kao i kod LRU algoritma svaki njen element treba da ima dodatno polje brojača. Ali za razliku od LRU, gdje se u to polje upisuje vrijeme pristupa prilikom pristupanja memorijskoj stranici, kod GClock algoritma ovo polje se uvećava za 1 pri svakom pristupu memorijskoj stranici. Pokazivač pokazuje na najstariju stranicu, tj. na stranicu koja je najranije učitana. Kada dođe do greške stranice, pogleda se brojač stranice na koju pokazuje pokazivač. Ako je polje brojača jednako 0 stranica se izbacuje, a nova se ubacuje na njeno mjesto i pokazivač se pomjera za jedno mjesto. Ako je polje brojača veće od 0, tada se ono umanji za 1 i pokazivač se pomjera na sljedeću stranicu, čime se nastavlja traženje. Nakon obilaska jednog ili više krugova, ovo polje će dostići vrijednost 0 i stranica će biti izbačena.

7.24.LFU/NFU i MFU: najrjeđe ili najčešće korišten okvir

Ako nemamo potrebnu hardversku podršku sa brojem pristupa u tabeli stranica, GClock se ne može efikasno realizovati, ali može se probati njegova aproksimacija. Za svaku stranicu dodjeljuje se po jedan softverski brojač izvan tabele stranica koji se na početku postavlja na 0. Kod svakog prekida sata operativni sistem provjeri tablicu stranica u memoriji i sabere sa svakim brojačem odgovarajuće stranice njen R bit. Nakon toga se svi R biti postave na 0. Kada je potrebno neku stranicu izbaciti iz memorije, izbacuje se ona sa najmanjom vrijednosti brojača, jer se njoj najmanje pristupalo. Takav algoritam se zove LFU (*Least Frequentlty Used*) ili NFU (*Not Frequently Used*). Ovakav algoritam na prvi pogled predstavlja dobru aproksimaciju LRU algoritma. Međutim, nekada se dešavaju situacije da je pojedina stranica u prošlosti mnogo korištena, a onda dugo nije korištena, kao što je stranica u kojoj se nalaze instrukcije za inicijalizaciju programa. Kako ovaj brojač nije očišćen u međuvremenu, često korištene kasnije ubačene stranice neće moći dostići vrijednosti brojača onih stranica koje su ostale u memoriji "zbog stare slave". Posljedica je da će nove i često trenutno potrebne stranice biti izbačene iz memorije.

Alternativa je algoritam koji bi izbacivao stranice sa najvećom vrijednošću brojača MFU (*Most Frequently Used*). Logika algoritma je da stranica sa malim brojačem je tek počela da se koristi, pa je relativno nova, ali ovaj algoritam dovodi do puno veće štete: dešavaće se da se stranice izbacuju upravo u trenutku kada se najviše koriste.

7.25.Algoritam starenja

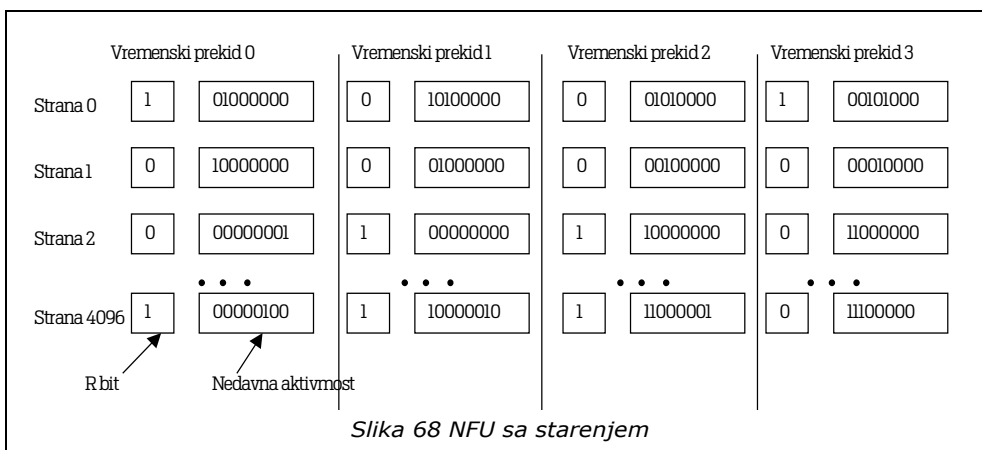
Nedostatak LFU algoritma (akumulirano pamćenje pristupa stranica) može se riješiti ako se sabiranje brojača sa vrijednošću R zamijeni operacijom pomicanja u desno. Ovaj algoritam je poznat pod imenom starenje (*aging*).

Pri svakom prekidu sata pomjeramo brojače tako da se bit R stranice za koju je vezan brojač stavi na mjesto najznačajnijeg bita brojača, dosadašnji najznačajniji bit

prelazi na drugu poziciju s lijeve strane, i svi ostali biti se tako pomjeraju u desno, pri čemu se najniži bit gubi. Efektivno n-bitni brojač se ažurira po formuli:

$$\text{brojač}^t = \frac{\text{brojač}}{2} + 2^{n-1} R$$

Primjer ovog algoritma pokazuje Slika 68. U prvom intervalu R biti imaju vrijednost 1, 0, 0,..., 1. Nakon pomjeranja brojača za jedan bit u desno i dodavanja R bita krajnjem lijevom bitu imao situaciju sa druge kolone slike Slika 68. Ostale kolone pokazuju šta se dešava nakon sljedeća 2 intervala. Kada je stranica dugo neaktivna, njen brojač će težiti nuli. Nema, međutim, načina da se ustanovi koliko dugo je stranica ostala neaktivna nakon što je brojač postao nula. Ravnopravni kandidati za brisanje su i stranica koja je zadnji put korištena prije 10 prekida sata i stranica koja je zadnji put korištena prije 10000 prekida sata, jer brojač prikazuje samo događaje u zadnjih 8 prekida sata.



7.26. Satni algoritam sa starenjem

Algoritam starenja pamti relativno mali prethodni broj pristupa. U Linux-u i Free BSD je korištena jedna varijacija Clock algoritma kombinovana sa brojačem starenja. Stranice su organizovane u kružnoj listi kao kod Clock algoritma, ali uz svaku stranicu postoji i brojač. Pokazivač pokazuje na jednu od stranica. Kada se pokrene algoritam za izbor stranica za brisanje (periodično ili na grešku stranice), provjeri se R bit stranice na koju pokazuje pokazivač. Ako je R=0 brojač za tu stranicu se umanjuje za 1. Ako je brojač postao 0, stranica se izbacuje a nova se ubacuje na njeno mjesto i pokazivač se pomjera na sljedeću stranicu. Ako je R =1, brojač se uvećava za 1, promijeni R da postane 0 i pokazivač se pomjera na sljedeću stranicu, eventualno kružeći kroz listu sve dok se ne dođe do stranice sa brojačem jednakim 0. Ovaj algoritam zadržava prednosti algoritma starenja uz duži dijapazon pamćenja događaja pristupa stranicama.

7.27. Adaptivna zamjena

Do sada opisivani algoritmi nisu uzimali u obzir čestoću izbacivanja pojedinih stranica, nego samo čestoću pristupa stranicama. U sljedećem primjeru vide se nedostaci tog zanemarivanja. Neka ima 3 fizička okvira i pristupalo se sljedećim

logičkim stranama: 1,2,8,4,1,5,9,7,1,6,3,10. Kod većine spomenutih algoritama, (FIFO, LRU, NFU itd.) stranica 1 će biti izbacivana jednako kao i ostale, iako je korištena 3 puta a ostale samo po jednom. Štoviše, uočljivo je da se stranici 1 pristupalo u pravilnim intervalima, npr. svaki put kada je njen proces dobio kvantum, i može se očekivati da će se tako i nastaviti. Stoga bi dobro bilo uočiti nakon njenog drugog ubacivanja sa diska da je ona loš kandidat za brisanje.

Jedan od algoritama koji izvjesno vrijeme prate i izbačene stranice je adaptivna zamjena. Kod ovog algoritma, vode se četiri liste stranica:

- T1: za nedavno pristupane stranice
- T2: za stranice kojima je pristupano bar dva puta
- B1: podaci o stranicama koje su izbačene iz T1 liste i memorije ali se još prate
- B2: podaci o stranicama koje su izbačene iz T2 liste i memorije ali se još prate

Algoritam se zove adaptivni jer se veličina ovih listi mijenja. Neka je p planirana veličina liste T1. Taj parametar se mijenja u toku rada i nekad favorizuje čestoću korištenja stranice, a nekad njenu nedavnu upotrebu. Prilikom greške stranice obave se sljedeća testiranja:

- Ako je $|T1| > p$, izbacuje se najranije zadnji put korištena stranica T1 i upisuje kao najkasnije korištena stranica u B1. Ovdje $|T1|$ predstavlja trenutnu veličinu liste T1.
- Ako je $|T1| < p$, izbacuje se najranije zadnji put korištena stranica T2 i upisuje kao najkasnije korištena stranica u B2.
- Ako je $|T1| = p$, i pristupana stranica je u B2 izbacuje se najranije zadnji put korištena stranica iz T1 i upisuje kao najkasnije korištena stranica u B1. Ako je $|B1| \geq |B2|$, umanjí p za 1; inače umanjí p za $|B1| - |B2|$.
- Ako je $|T1| = p$, i pristupana stranica je u B1 izbacuje se najranije zadnji put korištena stranica iz T2 i upisuje kao najkasnije korištena stranica u B2. Ako je $|B2| \geq |B1|$, uvećaj p za 1; inače uvećaj p za $|B2| - |B1|$.
- Ako je $|T1| = p$, i pristupana stranica nije uopće bila u memoriji izbacuje se najranije zadnji put korištena stranica T2 i upisuje kao najkasnije korištena stranica u B2.

Ovaj algoritam se odlikuje znatno manjim brojem greška stranica od LRU, ali je posao ažuriranja listi stranica složeniji. Algoritam je relativno nov pa je malo korišten dok se ne pronađu efikasnije varijante.

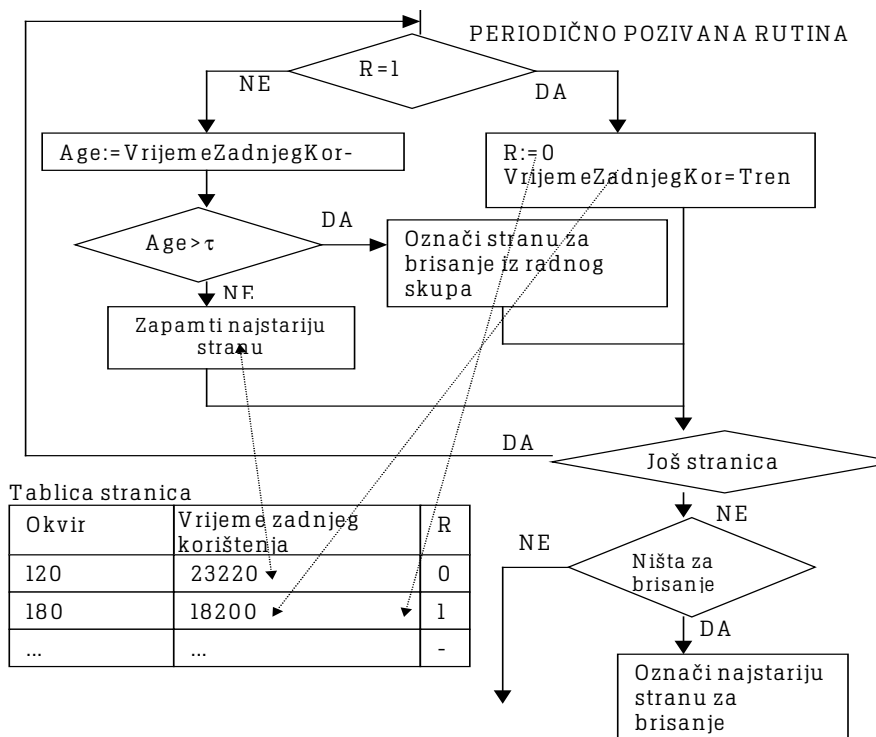
7.28. Algoritam sa radnim skupom

Do sada opisani algoritmi su odabirali jednu stranicu za brisanje iz radne memorije i pozivali su se prilikom greške stranice ili periodično. Pri tome su procesi zadržavali mnoge nepotrebne stranice u memoriji, dok se proces izbora stranice morao ponavljati svaki put kada se desi greška stranice. Sada će biti opisani algoritmi koji izbacuju više stranica koje ne zadovoljavaju kriterij čuvanja u memoriji, kako bi ostalim procesima pripalo više slobodne memorije.

Stranice koje proces u jednom određenom trenutku koristi zovu se **radni skup** (engl. *working set*). Njegova veličina varira tokom rada procesa. U početku je on prazan, (ako se koristi straničenje na zahtjev) i sistem generiše mnogo grešaka

stranica dok se one ne ubace u radnu memoriju. Nakon ubacivanja potrebnih stranica u radnu memoriju, broj grešaka stranica opada. Ali ako proces počne zahtijevati dodatnu memoriju, ili mu je oduzeta memorija kako bi se dodijelile drugom procesu, proces ponovo proizvodi mnogo grešaka stranice i radi sporo. Ovakvo ponašanje naziva se **thrashing**. Zaključak je da radni skup ne smije biti mali (npr. samo nekoliko stranica) jer će greške stranica biti prečeste. Ali radni skup ne treba biti ni prevelik, da npr. zauzima cijeli virtualni adresni prostor procesa. Pošto se u jednom trenutku pristupa manjem broju stranica dovoljno je da radni skup ima takvu veličinu da greške stranica budu rijetke.

Algoritmi zamjene stranica koji koriste radni skup, pored izbora stranice koja će biti obrisana, održavaju i radni skup na potrebnoj veličini. Sada će biti opisan jedan takav algoritam, baziran na LRU.



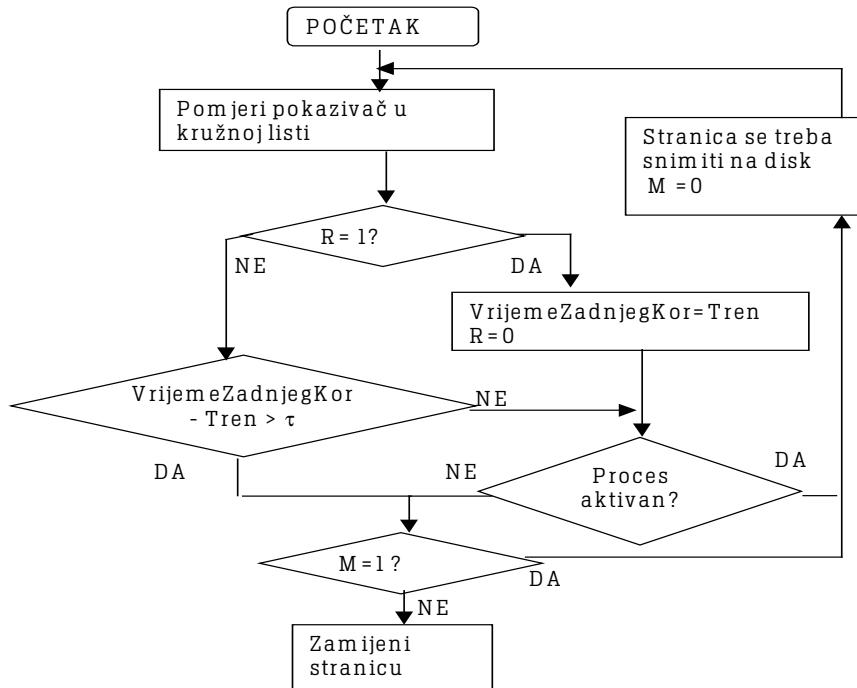
Slika 69 Algoritam radnog skupa

Rutina prikazana dijagramom toka na slici Slika 69 poziva se periodično, npr. svakih 50 ms. Uz tabelu stranica (sa bitima pristupa R), pamti se približno vrijeme zadnjeg pristupa stranici. Za razliku od LRU ovo vrijeme se ne mora evidentirati pri svakom pristupu memoriji, nego pri (daleko rjeđem) periodičnom pozivu rutine. Kod svakog elementa tablice se prvo ispituje R bit. Ako je ovaj bit postavljen na 1, taj bit se mijenja na 0, a u polje za vrijeme zadnjeg pristupa se unosi trenutno vrijeme. To znači da je stranici pristupano od zadnjeg izvršavanja rutine, pa stranica pripada radnom skupu i ne treba je izbaciti. Ukoliko je R=0 ta stranica je mogući kandidat za izbacivanje zato što joj nije pristupano u zadnjem vremenskom intervalu. Da se

proračuna da li stranica pripada u radni skup uzima se u obzir njena starost, razlika trenutnog vremena i vremena zadnjeg pristupa. Parametar τ reguliše veličinu radnog skupa i označava koliko se dugo od zadnjeg pristupa stranice drže u memoriji. Ukoliko je starost stranice veća od τ , ova stranica ne spada više u radni skup i može biti izbačena. Ako su sve stranice korištene u intervalu koji se smatra periodom čuvanja radnog skupa, izbacuje se najstarija među njima.

7.29.WSClock algoritam zamjene stranica

Baš kao što satni algoritam predstavlja bržu implementaciju algoritma druga šansa, može se napraviti i varijanta algoritma sa radnim skupom kombinovana sa satnim algoritmom. Ovaj algoritam se zove WSClock. Uz algoritam starenja smatra se najboljim i najviše korištenim algoritmom zamjene stranica.



Slika 70 Algoritam WSClock

U WSClock algoritmu stranice su poredane u kružnu listu kao u svim algoritmima Clock tipa. Na jedan od elemenata ove liste pokazuje pokazivač. Kada se desi greška stranice ili periodično čišćenje radnog skupa, provjeri se R bit stranice na koju pokazivač pokazuje. Ako je R bit postavljen stranica je korištena u zadnjem intervalu i treba da ostane u radnom skup. R bit se postavlja na 0, vrijeme dolaska na trenutno, a pokazivač se pomjera ka sljedećoj stranici i algoritam se ponavlja za ovu stranicu. Ako je R bit bio jednak 0, starost stranice veća od τ i ako ona nije promijenjena ($M=0$), stranica ne spada u radni skup. Ta se stranica izbacuje i zamjenjuje novom stranicom. Ako je sadržaj stranice promijenjen ($M=1$), stranica je predviđena za snimanje na disk i postavljanje bita M na 0 (čišćenje), ali malo

kasnije. Pokazivač se pomjera na sljedeću stranicu i proces se ponavlja. Ako je pokazivač obišao cijeli krug, tada se slučajno izbacuje neka od stranica.

7.30.Satni algoritam sa dvije kazaljke

Unix operativni sistem upravlja stranicama varijacijom satnog algoritma koji koristi dva pokazivača. Povremeno se poziva čišćenje stranica u dvije neovisne faze od kojih svaka ima svoj pokazivač na kružnu listu stranica. U prvoj fazi se R bit stranice na koju pokazuje prvi pokazivač postavi na 0. U drugoj fazi, nakon vremenske zadržke, se provjeri da li je R bit stranice na koju pokazuje drugi pokazivač jednak 0. Ako jeste, stranica se izbacuje, a ako nije, samo se prelazi na sljedeću stranicu. Tako se pošteđuju one stranice kojima je pristupano između prolaska prve i druge kazaljke. Na ponašanje algoritma znatno utiče razmak između mjesta gdje se kazaljke međusobno nalaze.

7.31.Bélády anomalija

Dugo se mislilo da što je više fizičkih okvira to će biti manje grešaka stranice. Međutim, to ne mora uvijek biti tačno. Laszlo Bélády je 1969. otkrio da FIFO u sistemu sa četiri okvira stranice ponekad proizvodi više grešaka stranice nego u sistemu sa tri okvira stranica. Ova situacija se naziva **Bélády anomalija**. Slika 71 prikazuje situaciju za jedan program sa 5 virtualnih stranica, i pristupa im se u sljedećem redoslijedu 012301401234. Na lijevoj strani se vidi kako sa tri okvira stranica nastaje ukupno 9 grešaka stranice. Na desnoj strani dobivamo 10 grešaka stranice sa 4 okvira. Godine 2010. je pokazano da se može konstruirati redoslijed pristupa stranicama u kome memorijski sistem sa većim brojem okvira generiše proizvoljan broj puta više grešaka stranice nego sistem sa manjim brojem okvira. Anomalija se ne javlja sa LRU i optimalnim algoritmom.

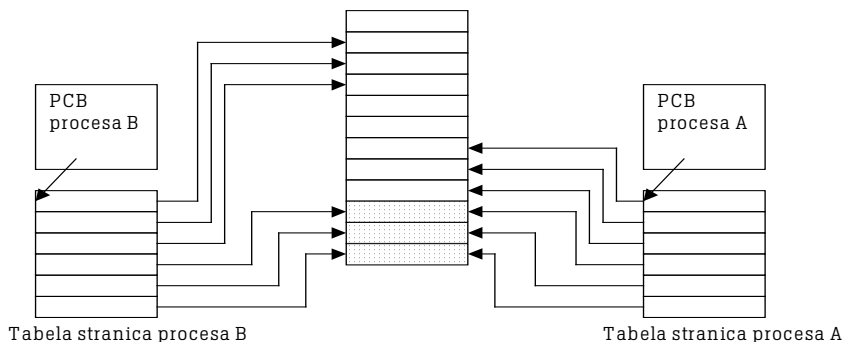
0	1	2	3	0	1	4	0	1	2	3	4
0	0	0	3	3	3	4	4	4	4	4	4
	1	1	1	0	0	0	0	0	2	2	2
		2	2	2	1	1	1	1	1	3	3
P	P	P	P	P	P	P			P	P	

0	1	2	3	0	1	4	0	1	2	3	4
0	0	0	0	0	0	4	4	4	4	3	3
	1	1	1	1	1	1	0	0	0	0	4
		2	2	2	2	2	2	1	1	1	1
			3	3	3	3	3	3	2	2	2
P	P	P	P				P	P	P	P	P

Slika 71 Beladyjeva anomalija

7.32.Dijeljene stranice

Dok jedna logička stranica procesa može biti smještena u jednom trenutku u samo jednom fizičkom okviru, jedan fizički okvir može da pripada različitim logičkim stranicama. Te logičke stranice mogu dalje pripadati istom ili različitim procesima. Takve stranice se zovu **dijeljene stranice**. Dijeljene stranice optimizuju upotrebu memorije. Na primjer, pri pozivu `fork()` u Unix sistemima, dijete proces nasljeđuje instrukcije programa roditeljskog procesa, ali treba da ima različite stranice podataka. Tada tablica stranica roditeljskog i dijete procesa može da pokazuje na iste fizičke okvire instrukcija programa, a na različite okvire memorije podataka. Očigledno je efikasnije stranice zajednički koristiti nego istovremeno imati više kopija iste stranice u memoriji. Problem pri tome je da sve stranice ne mogu istovremeno biti korištene. Dijeljene stranice omogućavaju i realizaciju dijeljenih biblioteka, mada na komplikovaniji način nego u slučaju segmentacije.



Slika 72 Dijeljena memorija

Ako se dopušta dijeljenje stranica između različitih procesa, pored tabela stranica za svaki proces, mora se uvesti i tabela fizičkih okvira sa **brojačem referenci**. Ova tabela će za svaki fizički okvir sadržati brojčani broj koliko logičkih stranica pokazuje na dati okvir. Ako neku stranicu treba izbaciti i njen okvir dodijeliti drugoj stranici, treba umanjiti za jedan brojčani broj za dati okvir. Tek kada taj brojčani broj postane 0, okvir se može dodijeliti drugoj logičkoj stranici.

7.33.Windows API za upravljanje stranicama i dijeljenom memorijom

Operativni sistemi koji koriste straničenje obično nemaju mnogo sistemskih ili API poziva koji se tiču straničenja. Razlog tome je što je straničenje dizajnirano da bude transparentno za korisničke programe, da se oni ponašaju kao da straničenja nema. U Windows-u je moguće djelomično uticati na dimenzije radnog skupa i zaštitu stranica. U sljedećoj tabeli su navedene funkcije ove namjene. Iako parametri veličine su u bajtima, njihova stvarna vrijednost se zaokružuje nakon prosljeđivanja na umnožak veličine stranice.

Funkcija i parametri	Opis
GetProcessWorkingSetSize(hProcess, LpMinimumWorkingSetSize, lpMaximumWorkingSetSize)	Dobavlja veličinu radnog skupa procesa u bajtima čija je oznaka data prvim parametrom na lokacije navedene u drugom i trećem
SetProcessWorkingSetSize(hProcess, dwMinimumWorkingSetSize, dwMaximumWorkingSetSize)	Postavlja veličinu radnog skupa procesa u bajtima čija je oznaka data prvim parametrom opseg naveden u drugom i trećem parametru.
VirtualLock (lpAddress,dwSize)	Fiksira virtualnu stranicu u memoriji
VirtualUnlock(lpAddress,dwSize)	Oslobađa fiksiranu virtualnu stranicu
VirtualProtect(lpAddress,dwSize,flNewProtect,lpflOldProtect)	Mijenja prava čitanja, pisanja, izvršavanja nad stranicom

Sljedeći primjer očitava i mijenja minimalnu i maksimalnu veličinu radnog skupa trenutno aktivnog procesa, uz stalno zadržavanje u memoriji stranice na kojoj se nalazi tekst koji treba biti sačuvan u memoriji. Iako je radni skup postavljen na

veličinu između 100000 i 150000 bajta njegove stvarne granice će biti zaokružene na vrijednost djeljivu sa 4096 (veličina stranice).

```
#include <windows.h>
#include <stdio.h>
char * stalno = "Ovo neka ostane";
void main(void){
    HANDLE hCurrProcess = GetCurrentProcess();
    SIZE_T lMin,lMax;
    VirtualLock(stalno,4096);
    SetProcessWorkingSetSize(hCurrProcess, 100000, 150000);
    if(GetProcessWorkingSetSize(hCurrProcess, &lMin, &lMax)) {
        printf("Minimalna velicina radnog skupa: %u\n", lMin);
        printf("Maksimalna velicina radnog skupa: %u\n", lMax);
    }
}
```

U narednom primjeru se nizu znakova dopušta promjena njegove vrijednosti, a zatim se promijeni da bude samo za čitanje. Naredni unos vrijednosti izaziva pad programa jer je stranica u kojoj je podatak zaštićena od upisa.

```
#include <windows.h>
#include <stdio.h>
char ime[4096];
void main () {
    DWORD OldProt;
    printf ("Unesi ime");
    scanf ("%s",ime);
    printf ("Zdravo, %s, ali ako sada probas unijeti novo padam!",ime);
    VirtualProtect (ime, 4096, PAGE_READONLY,&OldProt);
    scanf ("%s",ime);
    printf ("Ipak je %s uspio?!",ime);
}
```

7.34.POSIX API za upravljanje stranicama i dijeljenom memorijom

Sljedećim funkcijama se može podešavati pristup memorijskim stranicama koristeći POSIX kompatibilne operative sisteme

Funkcija i parametri	Opis
mprotect(pAddr, iLen, iProt)	Postavlja prava nad dijelom memorije
mlock(pAddr, iLen)	Zaključava/otključava dio virtualnog prostora procesa
munlock(pAddr, iLen)	
mlockall(iFlags)	Zaključava/otključava cijeli virtualni prostor procesa
munlockall()	
Mincore(pAddr, iLength, pVec)	Informiše da li je dio adresnog prostora u RAM-u
madvise(pAddr, iLength, iAdvice)	Predlaže jezgru način mapiranja stranica

Sljedeći primjer zaključava memorijsku stranicu sa lozinkom u memoriji koristeći *mlock*, kako ona ne bi išla u datoteku sa zamjenskim stranicama i bila pročitana od strane hakera koji bi je analizirao, a otključava je tek kada je ispražnjen njen sadržaj.

```
#include <windows.h>
#include <sys/mman.h>
char password[1024];
void main () {
    int i;
    mlock(password,4096);
    printf ("Unesi lozinku");
    scanf("%s",password);
    /* pozovi autentifikaciju */
    for (i=0;i<1024;i++) password[i]=' ';
    munlock(password,4096);
}
```

Iako je to vrlo rijetko potrebno, moguće je uticati na performanse straničenja pozivom *madvise*. Tim pozivom se može preporučiti Linux sistemu da unaprijed učita neke stranice, da optimizuje čitanje stranica za sekvencijalni ili direktni pristup. Sljedeći primjer obavlještava jezgro da

će se niz bajtova *muzika* čitati sekvencijalno, a niz bajtova *imenik* na preskok uz preporuku da se njegovih zadnjih 10 kilobajta što prije mapira u radnu memoriju. Preporuka sekvencijalnog čitanja utiče na učitavanje sa predviđanjem time što će jezgro znati da stranice treba ubacivati u memoriju jednu za drugom.

```
#include <sys/mman.h>
unsigned char muzika [200*1024],imenik [100*1024];
void main() {
    madvise(muzika, 200*1024, MADV_SEQUENTIAL);
    madvise(imenik, 100*1024, MADV_RANDOM);
    madvise(imenik+90*1024, 10*1024,MADV_WILLNEED );
}
```

7.35.Algoritmi zamjene segmenata

U sistemima kod kojih je memorija realizovana preko segmenata umjesto stranica, također je moguće postići virtualnu memoriju. Kada se pristupa segmentu koji je označen u tabeli da nije prisutan, taj segment se treba ubaciti u memoriju. Slobodno mjesto za njega se prvo potraži među šupljinama nekim algoritmom za traženje slobodne particije. Ako to nije moguće, neki od segmenata se izbacuje iz memorije i snima na disk. Objavljeno je vrlo malo ovih algoritama, za izbor segmenta, pošto se segmentacija rjeđe susreće od straničenja, a generalno su kompleksniji od srodnih algoritama za izbor stranice.

FIFO za segmentaciju: Najprije nađi slobodnu šupljinu u memoriji *First fit* algoritmom. Ako postoji takva šupljina za smještaj segmenta, dodijeli je. Ako ne postoji, ali je nezauzeta memorija u ukupnom iznosu manja od veličine segmenta koji se ubacuje, obavi kompakciju segmenata, i dodijeli dio slobodnog prostora novom segmentu. Ako nije bilo slobodne šupljine, ili kompakcija je ne može stvoriti, prođi kroz listu segmenata uređenoj po redoslijedu dolaska i nađi prvi segment dovoljne veličine koji se može zamijeniti segmentom koji ulazi u memoriju. Ako takav segment nije nađen, kreni ponovo od početka liste segmenata sortirane po vremenu dolaska i nađi niz segmenata koji su na susjednim memorijskim lokacijama i koji u zbiru zauzimaju manje od veličine traženog segmenta. Izbaci sve te segmente i memoriju dodijeli segmentu koji se ubacuje. Ako ni takav niz ne postoji izbaci

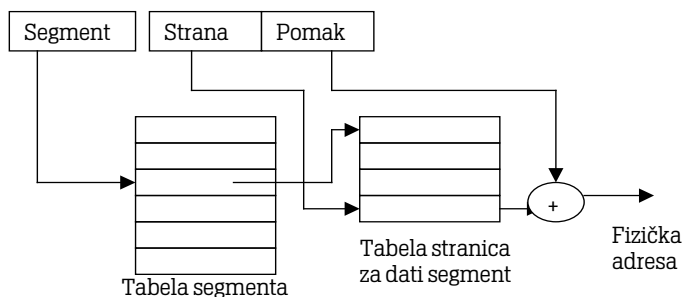
susjedne segmente iza najstarijeg koji počinje dovoljno daleko od kraja memorije, bez obzira kada su došli.

LRU za segmentaciju koristi isti princip kao FIFO za segmentaciju, osim što je lista segmenata koji se biraju za izbacivanje sortirana po vremenu upotrebe.. Ovu listu je lakše ažurirati nego kod straničenja, jer se može evidentirati trenutak kada aplikacija mijenja registar za izbor segmenta, i to vrijeme stavljati u listu, što je puno rjeđe od svakog pristupa memoriji,

7.36.Segmentacija sa straničenjem

Kada bi se usporedio koncept straničenja sa konceptom segmentacije, to su dva slična metoda organizacije memorije, ali imaju i međusobnih razlika. Najveća razlika je da aplikativni programer ne brine o straničenju. On piše programe kao da je memorija linearna, a operativni sistem i procesor brinu o njenoj podjeli na stranice i pretvaranju virtualnih adresa u fizičke. Programer na sistemima s segmentacijom, međutim, u svojim programima navodi dva dijela adrese, segment i stranicu. Druga razlika je u veličini. Sve stranice su međusobno iste veličine, a segmenti su različite veličine. Treća razlika je u broju adresnih prostora. Segmentacija ima više linearnih adresnih prostora, a straničenje samo jedan. Glavna prednost segmenata nad stranicama je što segmenti omogućavaju logičku podjelu procesa i zaštitu dijelova aplikacije, a stranice ne. Najveća mana segmentacije je što se često moraju radi iskorištenja segmentacije koristiti nestandardna proširenja programskih jezika. U segmentaciji se dešava eksterna a u straničenju unterna fragmentacija. Zaključuje se da oba pristupa imaju svoje prednosti i mane.

Neki računarski sistemi su opremljeni hardverom procesora i operativnim sistemom koji omogućava izbor između oba metoda ili njihovu kombinaciju. U kombinovanom sistemu straničenje/segmentacija, korisnički adresni prostora je razbijen u niz segmenata po planu programera (Slika 73). Svaki segment je dalje razbijen u niz stranica fiksne veličine koje su jednake dužini okvira glavne memorije. Ako je segment manji od dužine stranice, segment zauzima cijelu jednu stranicu.



Slika 73 Segmentacija sa straničenjem

Sa programerove tačke gledišta, logička adresa se još uvijek sastoji od uređenog para (*segment, pozicija*). Sa systemske tačke gledišta, adresa se dobiva iz segmenta, broja stranice i pozicije unutar stranice. Pristup je korišten u Multics, OS/2 i Windows 95 kada izvodi 16 bitne aplikacije.

7.37.Intel i386 memorijska arhitektura

Intelovi procesori počevši od 8086 pa do današnjih Core i7 su se razvijali evolutivno. Nastojalo se u svakoj novoj generaciji procesora očuvati kompatibilnost s prethodnim generacijama, kako bi programi pisani za ranije verzije procesora radile sa novim, s obzirom na ogromnu bazu softvera. Segmentaciju je bilo lakše dodati na raniji memorijski model naslijeđen od osmobitnih procesora, nego straničenje, pa je segmentacija prva nastala. Vremenom se došlo do 80386 procesora koji je na segmentaciju dodao straničenje i time omogućio realizaciju operativnih sistema koji koriste bilo koji model organizacije memorije: linearnu jednoprocensnu memoriju, banke, memoriju organizovanu po segmentima sa ili bez zaštite, memoriju sa razdvojenim adresnim prostorom za program i podatke, straničenje na dva nivoa, segmentaciju s straničenjem, sve prostom promjenom nekoliko registara i popunjavanjem tabela.

Kada je procesor u realnom režimu rada koristi se segmentacija bez zaštite. Kada je procesor u zaštićenom 32-bitnom režimu rada, omogućeni su segmentacija ili segmentacija sa straničenjem. Postoje dva registra koji mogu pokazivati na tabele segmenata. Registar LDTR pokazuje na tabelu segmenata za proces (*Local Descriptor Table*), dok registar GDTR pokazuje na tabelu segmenata koji pripadaju jezgri operativnog sistema (*Global Descriptor Table*).

Memorijska adresa se navodi iz dva dijela: segmenta i pozicije unutar njega. Pozicija je navedena u samoj instrukciji, dok je segment naveden unutar segmentnog registra. Podrazumijevano, sve instrukcije skokova i drugih instrukcija vezanih za programski brojač koriste segmentni registar CS, instrukcije koje čitaju podatke iz memorije koriste segmentni registar DS, dok instrukcije koje pristupaju području *stack*-a koriste segmentni registar SS. Segmentni registri ES, FS i GS se mogu koristiti ako se želi pristupiti podacima u različitim segmentima a ne žele se mijenjati podrazumijevani segmentni registri. Struktura segmentnog registra opisana je na slici Slika 74. Dva bita na najnižoj poziciji određuju nivo privilegija između 0 i 3 sa kojim se pristupa podacima ili programu u nekom segmentu. Bit DT na poziciji 2 govori da li je segment u tabeli na koju pokazuje LDTR ili GDTR. Ostalih 13 bita predstavljaju redni broj segmenta unutar jedne od ovih tabela.

Redni broj segmenta													DT	Privilegije	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Slika 74 Format selektora

Elementi tabela segmenata se zovu deskriptori. Deskriptor se sastoji od 8 bajtova, uključujući adresu početka segmenta, veličinu i druge informacije, prikazanih na slici Slika 75. Njegova struktura je rezultat mnogih evolucionih kompromisa. Na primjer, adresa početka segmenta je razbacana po bajtovima deskriptora, jer je prva verzija za procesor 80286 predviđela 2 bajta za početnu adresu, a na 80386 su se morali dodavati preostali bajti za početnu adresu za veći adresni prostor na neprirodnim mjestima.

Poz	Znacenje							
0	Bitovi 0-7 dužine segmenta							
1	Bitovi 8-15 dužine segmenta							
2	Bitovi 0-7 početka segmenta							
3	Bitovi 8-15 početka segmenta							
4	Bitovi 16-23 početka segmenta							
5	P	DPL		Tip deskriptora				
6	G	X	0	AVL	Bitovi 16 do 23 dužine segmenta			
7	Bitovi 24-31 početka segmenta							
	7	6	5	4	3	2	1	0

Slika 75 Format deskriptora

U petom i šestom bajtu značenja bitova su sljedeća. Bit P znači da je segment u memoriji. DPL su dva bita za nivo prava pristupa, koji određuju koje aplikacije mogu pristupiti segmentu. G (*granularity*) bit određuje mjernu jedinicu limita (0 predstavlja bajt, 1 predstavlja 4 kilobajta). Bit X zavisi od vrste deskriptora. Sa systemske nema značenja, za izvršne određuje da li je u pitanju 16 bitni (0) ili 32 bitni kôd (1), a za one s podacima se koristi samo kod segmenata koji se šire prema nižim adresama i služi za račun gornjeg limita. Bitovi AVL su namijenjeno za dodatne podatke. Tip deskriptora je polje od 5 bita koje ima tri različite interpretacije zavisno od vrste deskriptora (Slika 76).

Vrsta deskriptora					
Sistemska	0	Specijalna vrsta			
Podaci	1	0	E	W	A
Izvršni	1	1	C	R	A

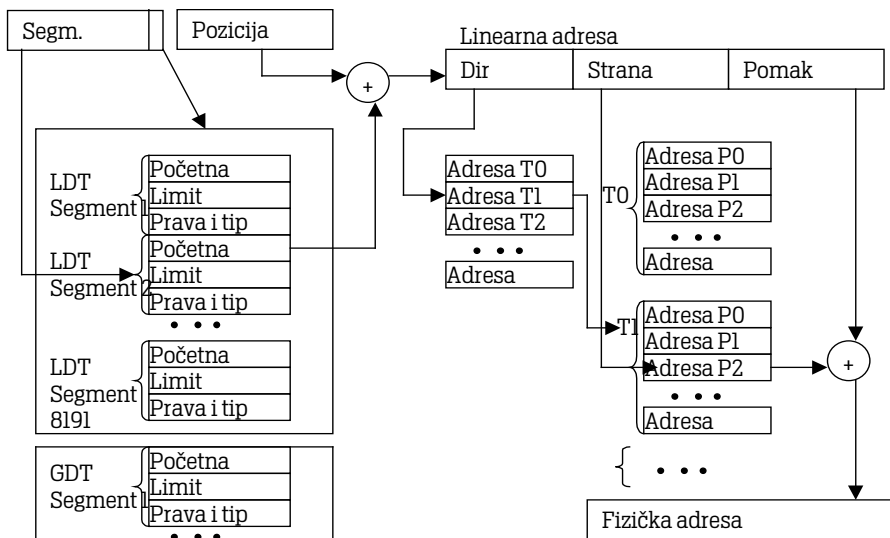
Slika 76 Tipovi deskriptora

U tipu deskriptora bit E predstavlja da li segment raste prema višim (0) ili nižim adresama (1). Bit A je 1 ako je segmentu pristupano. Bit W je 1 ako se u segment može pisati. Bit R je 1 ako se segment može čitati. Bit C je 1 ako skok u ovaj segment ne mijenja trenutni nivo privilegije.

Na lijevoj strani slike Slika 77 vidi se kako se konvertuje par (segment, pozicija) u linearnu adresu. Iz vrijednosti segmentnog registra sazna se na koju tabelu se segment odnosi. Viših trinaest bita segmentnog registra se pomnoži sa 8 (efektivno, zbog položaja u segmentnom registru dovoljno je njegovih nižih tri bita smatrati da su nule) i taj proizvod sabere sa odgovarajućom vrijednosti deskriptorskog registra LDTR ili GDTR. Osam bajta sa dobivene lokacije predstavlja deskriptor navedenog segmenta. Prije nastavka pristupa podacima obave se određene provjere u deskriptoru. Program će se prekinuti i prepustiti obrađivaču ako se desi neka od sljedećih situacija u deskriptoru:

- Zahtijevan je veći nivo privilegije u segmentnom registru od vrijednosti upisane u RPL bite deskriptora
- P bit je 0, segment nije u memoriji
- Pozicija unutar segmenta je veća od dužine segmenta. Kod većih segmenata dužina se ne mjeri u bajtima nego komadima od 4 kilobajta
- Pokušalo se pisati u segment namijenjen samo za čitanje

Ako je sve u redu, iz deskriptora se pročita početna adresa segmenta i sabere sa pozicijom u njemu. Time se dobiva **linearna adresa**.



Slika 77 Transformacija logičke adrese u fizičku na Intelovim procesorima

Ako je straničenje onemogućeno (to određuje registar CR0), linearna adresa se interpretira kao fizička i takva proslijeđuje memorijskim čipovima. Ako je straničenje omogućeno (vidi desnu stranu slike Slika 77), linearna adresa se interpretira kao virtualna i mapira u fizičku adresu korištenjem tabele stranica na dva nivoa, radi smanjenja veličine tabele stranica za male segmente. Linearna adresa podijeljena je u tri polja, *Dir* (10 bita), *Strana* (10 bita) i *Pomak* (12 bita). Svaki program ima **imenik stranica**, veličine 4 kilobajta, sa 1024 elementa. On se nalazi na adresi na koju pokazuje registar CR3. Elementi imenika stranica pokazuju na početke tabele stranica. Koji od tih elemenata će se čitati, određeno je poljem *Dir*. *Strana* polje se koristi kao indeks u tabeli stranica da bi se pronašla fizička adresa okvira stranice. Konačno, *Pomak* se dodaje na adrese okvira stranice da bi se dobila fizička adresa bajta ili riječi koja je potrebna. Elementi tabele stranica su 32-bitni, 20 bitova sadrži broj okvira stranice. Preostali bitovi sadrže bite pristupa, izmjene, bitove zaštite i bitove keširanja.

Radi ubrzanja ovog postupka procesor koristi deskriptorske keševe (koji pamte sadržaje deskriptora za trenutne vrijednosti segmentnih registara) i TLB registre (koji pamte više zadnjih transformacija logičkih adresa stranica u fizičke okvire).

Isključenjem straničenja, model se svodi na segmentaciju. Pravljenjem tabele segmenata sa samo jednim deskriptorom koji sadrži početnu adresu 0 i dužinu 4 gigabajta te postavljanjem svih segmentnih registara da pokazuju na njega model se svodi na čisto straničenje sa dva nivoa. Od Pentium Pro procesora postoji i režim rada u kome se faza straničenja obavlja kao straničenje na jednom nivou sa stranicama velikim 4 megabajta u 32 bitnom adresnom prostoru ili 2 megabajta u 36 bitnom adresnom prostoru.

U 64 bitnom režimu rada segmentacija sa straničenjem je zamijenjena hijerarhijskim straničenjem na 4 nivoa, a segmentni registri se ignorišu.

7.38.Zadaci

Zadatak 1: U sistemu se koristi straničenje s jednim nivoom. Veličina stranice je 2048 bajta. Logička adresa je velika 32 bita. Tabela stranica koja pretvara logičke u fizičke adrese izgleda ovako

0->47, 1->88, 2->36, 3->26, 4->49, 5->61, 6->68, 7->31, 8->10, 9->50

- Koja logička adresa odgovara fizičkoj adresi 20645?
- Koja fizička adresa odgovara logičkoj adresi 20337?
- Koliko se u prosjeku potroši bajta zbog straničenja, ako element tablice zauzima potreban cijeli broj bajtova?

Rješenje

a) Kada se podijeli fizička adresa 20645 s veličinom stranice, dobije se 10 i 165 ostatka. Fizički okvir 10 odgovara logičkoj stranici 8. Stoga, logička adresa je $8 \times 2048 + 165 = 16549$

b) Kada se podijeli Logička adresa 20337 i s veličinom stranice dobije se 9 i 1905 ostatka. Logička stranica 9 odgovara fizičkom okviru 50. Stoga je fizička adresa $50 \times 2048 + 1905 = 104305$.

c) Pošto je logička adresa velika 32 bita, a stranica 2048 bajta (11 bitna adresa), redni broj fizičkog okvira se može zapisati s 21 bitom. Prvi veći cijeli broj bajta je 3 bajta (24 bita). Posljednja stranica je u prosjeku 50% popunjena, a ova tabela ima 10 elemenata. Stoga se izgubi $10 \times 3 + 0.5 \times 2048 = 1054$ bajta.

Zadatak 2: U sistemu se koristi segmentacija s tabelom segmenata

Segment	0	1	2	3
Poč. adresa	1A00	2000	5000	4000
Dužina	04A0	0A50	02E5	0A30

Koju fizičku adresu imaju logičke adrese 1:3A5 i 2:30E? Kako će izgledati tablica nakon kompakcije?

Odgovor: 3A5 pripada segmentu 1, pa je ta adresa jednaka *offset*-u plus početak tog segmenta, tj $2000 + 3A5 = 23A5$. Adresa 30E je van segmenta 2, pa druga adresa izaziva grešku.

Nakon kompakcije treba da početne adrese narednog segmenta budu jednake zbiru početne adrese i dužine prethodnog segmenta.

Segment	0	1	2	3
Početna adresa	1A00	1EA0	28f0	2BD5
Dužina	04A0	0A50	02E5	0A30

Zadatak 3. U sistemu u kome je fizička memorija od 8192 bajtova podijeljena u stranice veličine 2048 bajtova, program je pristupao logičkim adresama 13326, 14388, 9875, 18868, 394, 14968, 6664, 12633, 4668, 6290, 14875, 16714, 1041, 14238, 12980. Izračunajte broj grešaka stranice za algoritme

- FIFO
- Second chance*
- Optimalni
- LRU

Rješenje:

a) Nakon dijeljenja logičke adrese s veličinom stranice i uzimanja cijelog dijela količnika, dobiva se referentni niz: 6, 7, 4, 9, 0, 7, 3, 6, 2, 3, 7, 8, 0, 6, 6. FIFO algoritam

izbacuje stranicu koja je najduže bila u fizičkom okviru. Zahtjevi koji će rezultovati greškom stranice su označeni znakom P u donjem redu.

	6	7	4	9	0	7	3	6	2	3	7	8	0	6	6
0	6	6	6	6	0	0	0	0	0	0	7	7	7	7	7
1	.	7	7	7	7	7	3	3	3	3	3	8	8	8	8
2	.	.	4	4	4	4	4	6	6	6	6	6	0	0	0
3	.	.	.	9	9	9	9	9	2	2	2	2	2	6	6
	P	P	P	P	P		P	P	P		P	P	P	P	

Grešaka stranice 12

b) *Second chance*

Kod ovog algoritma uz svaku stranicu se evidentira bit upotrebe (*). On se postavlja na 1 pri svakom pristupu stranici. Kada se treba stranica izbaciti, one koje imaju ovaj bit postavljen se neće izbaciti nego će se bit postaviti na 0, a vrijeme dolaska na trenutno vrijeme.

	6	7	4	9	0	7	3	6	2	3	7	8	0	6	6
0	6*	6*	6*	6*	0*	0*	0	0	2*	2*	2*	8*	8	8	8
1	.	7*	7*	7*	7	7*	7	7	7	7	7*	7	0*	0*	0*
2	.	.	4*	4*	4	4	3*	3*	3*	3*	3*	3	3	3	3
3	.	.	.	9*	9	9	9	6*	6*	6*	6*	6	6	6*	6*
	P	P	P	P	P		P	P	P			P	P		

Grešaka stranice 10

c) Optimalni algoritam. Ovdje je kandidat za izbacivanje stranica koja će se najkasnije u budućnosti koristiti.

p	6	7	4	9	0	7	3	6	2	3	7	8	0	6	6
0	6	6	6	6	6	6	6	6	2	2	2	8	8	6	6
1	.	7	7	7	7	7	7	7	7	7	7	7	7	7	7
2	.	.	4	4	0	0	0	0	0	0	0	0	0	0	0
3	.	.	.	9	9	9	3	3	3	3	3	3	3	3	3
	P	P	P	P	P		P		P			P		P	

Grešaka stranice 9

d) LRU. Kandidat za izbacivanje je stranica koja je najranije zadnji put korištena.

p	6	7	4	9	0	7	3	6	2	3	7	8	0	6	6
1	6	6	6	6	0	0	0	0	2	2	2	2	0	0	0
2	.	7	7	7	7	7	7	7	7	7	7	7	7	7	7
3	.	.	4	4	4	4	3	3	3	3	3	3	3	6	6
4	.	.	.	9	9	9	9	6	6	6	6	8	8	8	8
	P	P	P	P	P		P	P	P			P	P	P	

Grešaka stranice 11

Zadatak 4: Intelov MMU

Sve vrijednosti u ovom zadatku su u heksadekadnom brojnem sistemu. Na Pentiumu u zaštićenom režimu registri imaju sljedeću vrijednost: LDTR=06000000, GDTR=08000000, CR3=10000000, DS=0014, CS=0034 CR0=**80000001**. Ako instrukcija (npr. MOV AL,[2A64]) pristupa logičkoj adresi 2A64, kojoj fizičkoj adresi ona pristupa? Koja je vrijednost na toj adresi? Trenutno stanje memorije, gledajući apsolutne adrese je

```

.....
06000000 CD 20 FF 9F 00 9A EE FE 1D F0 4F 03 22 05 8A 03
06000010 22 05 17 03 22 93 0D 04 01 01 01 00 02 FF FF FF
.....
08000000 CA 20 FF 9F 00 9A E3 FE 1D F2 4F 08 23 05 8A 07
08000010 26 05 19 03 22 05 0D 04 01 02 01 00 02 FF FA FF
.....
10000020 3A 56 21 40 2A 38 42 18 2A 56 42 40 8E 48 42 18
10000030 2A 36 42 40 9A 48 42 18 7A 56 42 20 8E 48 42 18
10000040 23 60 42 40 4E A8 42 18 5A 56 42 40 8E 48 42 18
.....
40426860 C6 06 23 99 00 80 3E 1D 96 00 74 03 E9 99 00 E8
40426870 A6 01 E8 FF 03 75 19 80 3E C4 98 00 34 00 AD 0A
40426880 13 96 00 BA E9 89 75 03 E9 17 01 C6 06 1F 99 01
40426890 B8 00 6C BE 08 98 BB 21
.....
C6011D70 C6 06 23 99 00 80 3E 1D 96 00 74 03 E9 99 00 E8
C6011D80 A6 01 E8 FF 03 75 19 80 3E C4 98 00 34 00 AD 0A
C6011D90 13 96 00 BA E9 89 75 03 E9 17 01 C6 06 1F 99 01

```

Rješenje:

Prvo pogledamo najniži bit registra CR0. Ako je on jednak 0, radimo u realnom režimu rada i tada bi odgovor bio $DS*10+2A64=00002BA4$. Ali ovaj bit je 1, što predstavlja zaštićeni režim rada.

Prvo je potrebno odrediti linearnu adresu. Kako je riječ o dohvaćanju podatka iz memorije, u pitanju je segment podataka, DS. Format selektora segmenta, sa upisanom vrijednošću heksadekadno i binarno izgleda ovako:

0				0				1				4			
0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0
Redni broj segmenta u tabeli podataka												TAB		RPL	

Ovo znači da je u pitanju segment broj 0002 (binarno 10), a iz polja TAB koje ima vrijednost 1 saznajemo koju tablicu deskriptora gledamo. Vrijednost 0 znači GDTR, a 1 znači LDTR. Pošto je u pitanju 1, referentni registar je LDTR a on ima vrijednost LDTR=06000000.

Svaki element deskriptor tabele zauzima 8 bajtova. Stoga naš deskriptor počinje na adresi $LDTR+BrojSegmenta*8=06000010$. Označimo kosim slovima bajtove koji definišu početak segmenta, podvučenim limit segmenta a podebljano informacije.

06000010 22 05 17 03 22 **93** 0D 04

Iz bajta s vrijednošću 0D saznajemo da je *granularity* jednak 0

Segment počinje na adresi 04220317, dugačak je D0522 bajtova (*granularity* bit je 0). Iz bajta sa vrijednošću 93 razloženoj binarno saznajemo da je u pitanju segment podataka koji je prisutan u memoriji, za koga je nivo prava DPL=0, proširuje se prema višim adresama, upisiv je i pristupano mu je nedavno. Bit prisutnosti P indicira da je ovaj segment valjan. Linearnu adresu dobivamo kao zbir početka segmenta i virtualne adrese, tj. $04220317+2A64=04222D7B$.

1	0	0	1	0	0	1	1
P	DPL		1	0	E	W	A
			Tip deskriptora				

Pogledajmo sada najviši bit registra CR0. Ako je on 0, linearna adresa je i fizička adresa. Međutim, on je 1, pa imamo daljnju fazu, straničenje. Napišimo linearnu adresu na 32 bita. Grupišimo po 4 bita, pa u tri grupe koje imaju redom 10, 10 i 12 bita, a zatim svaku grupu u heksadekadno.

0	4	2	2	2	D	7	B
010		222			D7B		

Ovo nam kaže da se linearna adresa transformiše u tabelu broj 010, stranicu broj 222 i adresu unutar stranice D7B.

Registar CR3 ima vrijednost 10000000 i on pokazuje na imenik stranica. Svaki element ovog imenika se sastoji od 4 bajta sa sljedećim značenjem

Poz.	Značenje							
0	Avail	D	A	0	0	U/S	R/W	P
1	Bitovi 12-15 početka tabele stranica				Avail			
2	Bitovi 16-23 početka tabele stranica							
3	Bitovi 24-31 početka tabele stranica							

Bitovi 0-11 početka tabele stranica su jednaki 0. Bitovi *Avail* su na raspolaganju programeru, P predstavlja da je stranica prisutna u memoriji, U/S da li je za obične korisnike (0) ili supervizorski rad (1), R/W da li je za čitanje (0) ili za pisanje (1), D ima vrijednost 1 ako je stranica mijenjana, a A ima vrijednost 1 ako je čitana.

Kako imamo tabelu broj 010, a svaki element je dug 4 bajta, opis naše tabele stranica je na adresi $CR3 + BrojTabele * 4 = 10000040$.

Pogledajmo četiri bajta na toj adresi

10000040 23 60 42 40

Iz bajta s vrijednošću 23 iz njegovog najnižeg bita saznajemo da je ta tablica prisutna u memoriji. Iz niblova označenih podebljano zaključuje se da tabela stranica počinje na adresi 40426000 (najniža tri nibla su nule). Na adresu početka tabele stranica (40426000) pridodajemo redni broj stranice (222) pomnožen sa 4. i dobivamo da je opis naše stranice na adresi 40426888.

Struktura elementa tabele stranica je jednaka strukturi imenika stranica, osim što adresa početka tabele stranica predstavlja adresu početka stranice.

40426888 E9 17 01 C6

Poz.	Značenje							
0	Avail	D	A	0	0	U/S	R/W	P
1	Bitovi 12-15 početka stranice				Avail			
2	Bitovi 16-23 početka stranice							
3	Bitovi 24-31 početka stranice							

Stranica počinje na adresi C6011000. Iz bajta s vrijednošću E9 saznajemo da je ona prisutna u memoriji. Na ovu vrijednost dodajemo pomak, D7B.

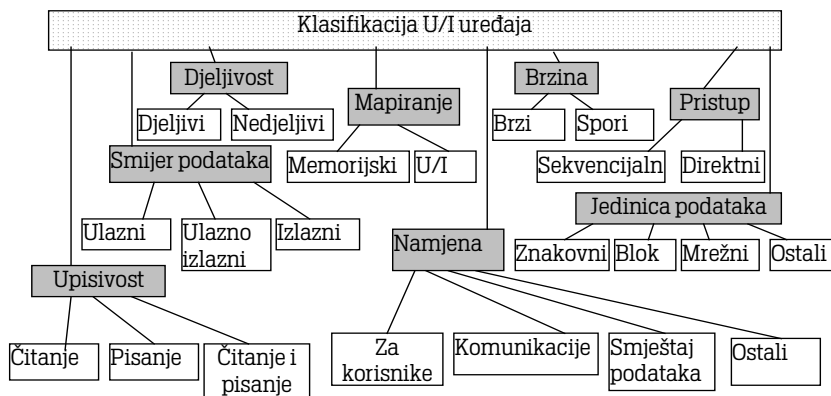
Dakle, logička adresa 2A64 se transformiše u fizičku adresu C6011D7B. Na toj adresi se nalazi bajt sa vrijednošću 03.

8.ULAZNO-IZLAZNI SISTEM RAČUNARA

Ulazno-izlazni sistem računara upravlja ulazno-izlaznim uređajima i osigurava vezu uređaja prema korisniku i ostatku sistema. Ulazno-izlazni uređaji se međusobno veoma razlikuju, ali su gotovo svi značajno sporiji od radne memorije i procesora. Stoga se mora osigurati efikasan i uniforman pristup U/I uređajima kako ne bi predstavljali usko grlo cijelog sistema.

8.1.Klasificiranje ulazno-izlaznih uređaja

Upravljanje ulazno-izlaznim uređajima je složen zadatak za operativni sistem najviše zbog njihove raznovrsnosti. Stoga je prilikom razvoja podsistema za ulazno izlazne uređaje najprije potrebno te uređaje klasifikovati na određeni način, jer će razlike u ponašanju uređaja uticati na način komunikacije operativnog sistema sa njima.



Slika 78 Klasifikacija U/I uređaja

Prema **namjeni** uređaji se dijele na uređaje za dugotrajno smještanje podataka (diskovi, trake, diskete, CD uređaji...), uređaje za prijenos podataka (mrežne kartice, ruteri, modemi, serijski portovi), uređaje koji pružaju sučelje ka korisniku (tastature, ekrani, video kartice, miševi, štampači, terminali, zvučne kartice) i ostali uređaji (uređaj sata).

Prema **smjeru prijenosa** uređaji se dijele na ulazne (miš, tastatura, tablet, skener, uređaj sata), izlazne (printer, monitor) i ulazno-izlazne (mrežna kartica, modem, disk, zvučna kartica). Ova kategorizacija se odnosi na glavni smjer podataka koji idu ka uređajima i od uređaja. Vrlo srodna ovoj kategorizaciji je **prema mogućnosti upisa** na one koji služe za čitanje i pisanje, zatim isključivo za čitanje i isključivo za pisanje. Ove dvije kategorizacije nisu identične. Na primjer, tastatura na IBM PC računarima je ulazni uređaj, ali nije isključivo za čitanje jer se tastaturi mogu poslati podaci koji pale i gase stanje CapsLock i NumLock lampica. Na nekim računarima je ona uređaj isključivo za čitanje.

Prema **osnovnoj jedinici prenesenih podataka** uređaji se dijele na znakovne, blokovske, mrežne i ostale uređaje. Ovakva podjela je posljedica razlika između ovih vrsta ulaznih i izlaznih uređaja u pogledu jedinice pristupa, načina pristupa i

upravljanja. Tako je za blokovske uređaje jedinica pristupa blok (fiksno 512 bajta, 1024 bajta i slično), za mrežne uređaje jedinica pristupa je paket (fiksirana ali podesiva veličina), a za znakovne uređaje jedinica pristupa je znak (tipično bajt). Stoga kod blokovskih uređaja nije moguće čitati u jednom pristupu manje od veličine bloka, npr. pojedinačne bajtove. Pored veličine podataka kojim se pristupa u jednom pozivu, ove grupacije uređaja se razlikuju i po tome što dok značajan broj blokovskih uređaja dozvoljava direktan pristup, znakovni uređaji podržavaju samo sekvencijalni pristup. Uređaj sata se po ovoj klasifikaciji ubraja u ostale uređaje. Od svih klasifikacija, ova klasifikacija najviše utiče na način programiranja drajvera za periferijske uređaje u operativnim sistemima.

Prema **metodi pristupa** uređaji se dijele na uređaje sa sekvencijalnim pristupom (tastatura, miš, magnetna traka, mrežna kartica, modem, matični štampač) i uređaje sa direktnim ili slučajnim pristupom (tvrdi disk, CD čitač, ekran, PostScript štampač). Uređaji sa sekvencijalnim pristupom prenose podatke fiksnim redoslijedom koje određuje sam uređaj, dok kod uređaja sa direktnim pristupom redoslijed određuje aplikacija. Uređaji sa direktnim pristupom dopuštaju keširanje, čuvanje podataka u memoriji, znatno prije nego što budu potrebni kako bi se ubrzao rad sa njima.

Prema **djeljivosti** uređaji mogu biti djeljivi i nedjeljivi. Djeljive uređaje može koristiti više procesa istovremeno (npr. diskovi, ekrani u grafičkim sistemima). Nedjeljive uređaje u jednom trenutku može koristiti samo jedan proces (npr. tastatura ili štampač).

Prema **načinu mapiranja** uređaje razlikujemo po tome da li su U/I mapirani ili memorijski mapirani. U/I mapirani uređaji koriste specijalne instrukcije mikroprocesora za pristup tim uređajima, dok memorijski mapirani uređaji koriste instrukcije mikroprocesora kojima on pristupa memoriji.

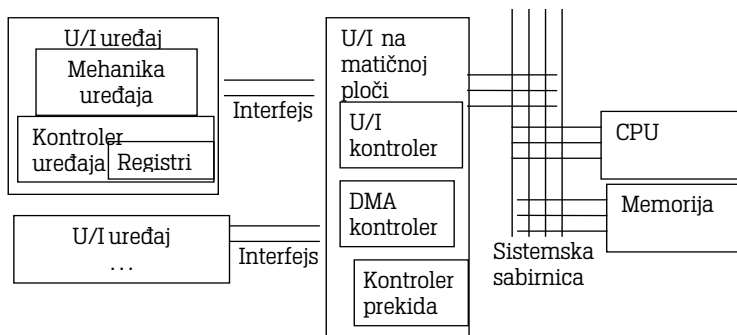
Grubo gledano, prema **brzini prijenosa** uređaji se mogu klasifikovati na brze i spore. Ali je ovu podjelu teško izvršiti jer se razlikuju od najsporijeg do najbržeg i milijardama puta (Slika 79).

Uređaj	Brzina prijenosa: Bajta u sekundi
Tastatura	10
Miš	10^2
56K modem	$7 \cdot 10^3$
Paralelni printer	10^5
IDE disk	$5 \cdot 10^6$
ISA sabirnica	$1,67 \cdot 10^7$
PCI sabirnica	$5,28 \cdot 10^8$
DDR3 RAM	$8,533 \cdot 10^9$
100Gb Ethernet	$8,533 \cdot 10^{10}$

Slika 79 Razlike u brzinama uređaja

8.2. Hardverska struktura U/I sistema

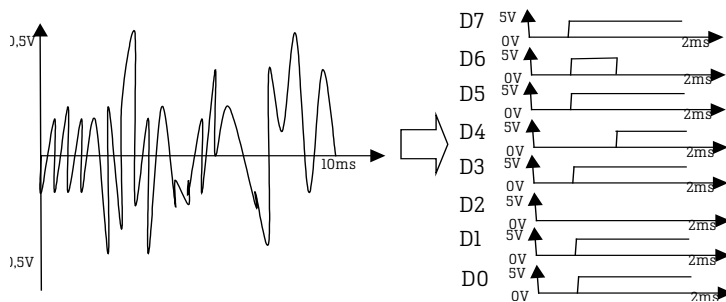
Računarski sistem sastoji iz tri osnovna funkcionalna bloka: CPU, memorije i ulazno/izlaznog sistema. Ulazno/izlazni (skraćeno U/I) sistem se dalje može rastaviti na tri dijela (Slika 80): U/I uređaj, interfejs i U/I kontroler.



Slika 80 U/I sistem, hardverska struktura

8.3. Struktura U/I uređaja

Mnogi **U/I uređaji** imaju dvije komponente: mehaničku komponentu i elektronsku komponentu. Mehanička komponenta uključuje motore diska ili štampača i kućište uređaja. Elektronska komponenta se zove kontroler uređaja. Zadatak kontrolera na samom uređaju je konverzija internog elektronskog signala kojeg generiše uređaj u digitalni format razumljiv računarskom sistemu, uz eventualnu korekciju grešaka. Ova konverzija može uključiti pretvaranje serijskog signala u paralelni ili obrnuto. Na primjer, ako je uređaj mehanički hard disk, i operacija je čitanje, prolazak ploče diska ispod glave će indukovati promjenjivu električnu struju. Promjene intenziteta te struje se prepoznaju kao logičke jedinice ili nule, a nakon što se sakupi npr. 8 takvih logičkih jedinica ili nula one se šalju na linije interfejsa prema centralnom procesoru kao jedan bajt koji se direktno čita.



Slika 81 Konverzija signala u kontroleru U/I uređaja

Svaki kontroler U/I uređaja ima jedan ili više registara u koje centralni procesor može upisati vrijednosti ili čitati ih. Sa uređajem se komunicira upisom i čitanjem vrijednosti tih registara. Ako su memorijski i ulazno-izlazni prostor odvojeni, procesor ima posebne instrukcije za pisanje i čitanje podataka sa određene lokacije u kontroleru uređaja. Ako se ulazno-izlazni prostor može tretirati kao memorija, upis i čitanje se obavljaju običnim instrukcijama za pristup memoriji. Registri ulazno-izlaznog uređaja dijele se u četiri grupe

- **Kontrolni registri:** Upis u ove registre postavlja režim rada perifernog uređaja. Na primjer, ako je uređaj zvučna kartica, upis u jedan od njenih kontrolnih registara postavlja jačinu zvuka.

- **Statusni registri:** Ovi registri se isključivo čitaju i opisuju status uređaja. Primjer predstavlja registar štampača koji indicira da nema papira.

- **Ulazni registri podataka:** Procesor ove registre isključivo može čitati i oni omogućavaju čitanje podataka sa perifernog uređaja. Na primjer, ako je uređaj disk, i nakon što su kontrolnim registrima izabrani sektori diska koji se žele čitati, uzastopnim čitanjem ulaznog registra podataka se čitaju bajtovi koji su stigli sa diska

- **Izlazni registri podataka:** Procesor u ove registre isključivo može pisati i oni omogućavaju slanje podataka perifernom uređaju. Na primjer, slanje narednog slova štampaču obavlja se upisom u njegov izlazni registar.

8.4. U/I kontroler na matičnoj ploči

Na matičnoj ploči računara nalazi se njegova **U/I komponenta**. Ona se sastoji od **U/I kontrolera, interrupt kontrolera i DMA kontrolera**.

Mnoge U/I uređaje nije moguće direktno povezati na sistemsku sabirnicu. Razlog je što postoji veliki broj različitih tipova uređaja, npr. monitori, diskovi, tastature. Svi rade na drugi način, a proizvode se nezavisno od procesora i matične ploče sa ciljem da se koriste na više različitih vrsta računarskih sistema. Dalje, brzina prijenosa podataka prema perifernim uređajima (i od njih) je puno manja u odnosu na brzinu CPU pa CPU ne može direktno komunicirati sa njima bez usporavanja čitavog sistema. Konačni, format podataka koji očekuju periferni uređaji se u mnogome razlikuje od signala na sabirnicama (npr. sistemske sabirnice imaju 64 linije za podatke, a modem samo dvije). Zbog svega ovoga računarski sistemi moraju koristiti U/I kontrolere, komponente koje povezuju CPU i periferne uređaje. Zadataci U/I kontrolera su:

- Kontrolisanje periferija i njegova sinhronizacija sa CPU. Mnogi uređaji imaju kontrolne registre kojima se može prijem podataka od uređaja zaustavljati ili statusne registre preko kojih se saznaje da li je uređaj spreman u tom trenutku. Na primjer, U/I kontroler za serijsku komunikaciju po RS232 standardu (UART) može se poslati kontrolni signal DTR koji javlja modemu da je računar spreman za prijem podataka. Kontroler uređaja u modemu tek kada primi taj sinhronizacijski signal počinje sa slanjem podataka određenim ritmom.

- Komunikacija kontrolera sa CPU kroz sistemsku sabirnicu. UI kontroler, kao i kontroler perifernog uređaja ima registre preko kojih CPU može komunicirati sa uređajem. Preko ovih registara U/I kontroler javlja procesoru stanje perifernih uređaja, i priprema podatke za proslijeđivanje uređajima ili procesoru.

- Komunikacija kontrolera sa perifernim uređajem kroz U/I interfejs, gdje se signali sa sabirnice konvertuju u signale koji odgovaraju standardu interfejsa i obrnuto. Interfejsi se međusobno mogu dosta razlikovati.

Funkcija bafera: Manja količina podataka se može čuvati u samom U/I kontroleru. Na primjer, podaci koji dolaze sa serijskog interfejsa pristižu bit po bit i u trenutku dok procesor ne komunicira sa U/I kontrolerom, sve dok se ne sakupi čitav bajt, nakon čega kontroler čuva sadržaj tog bajta dok ga procesor ne pročita.

- Detekcija greške. Primjer je provjera da li je tastatura priključena, i ako nije javljanje informacije o tome u posebnom registru.

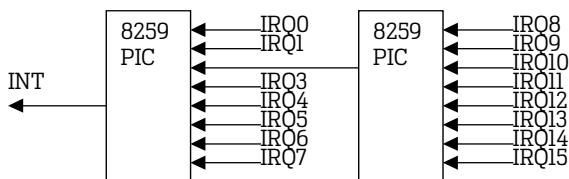
8.5.Kontroler prekida

Periferni uređaji su obično sporiji od centralnog procesora. Od vremena kada je dat zahtjev za prijemom podataka do vremena kada je uređaj zaista spreman da ih pošalje procesor ostaje neiskorišten. Da bi se povećala efikasnost procesora uvode se kontroleri prekida (*interrupt controller*). Uz pomoć ovog kontrolera omogućava se CPU da nastavi sa operacijama dok uređaj nije spreman da prenese podatke. Kada CPU želi da komunicira sa uređajem, on daje instrukciju odgovarajućem U/I kontroleru i nastavlja sa drugim operacijama. Kada je uređaj spreman, on će prekinuti CPU. Ovo otklanja potrebu da CPU periodično provjerava ulazne uređaje da vidi ima li spremnih podataka. Kada ulazni uređaj ima podatke (ili je izlazni uređaj postao spreman za prijem), odgovarajući U/I modul može prekinuti CPU da zahtjeva prijenos podataka.

Kontroler prekida šalje signal prekida procesoru preko prekidne linije (*engl. interrupt request line, IRQ*). Desiče se sljedeći događaji:

1. U/I kontroler šalje signal kontroleru prekida
2. Kontroler prekida šalje IRQ signal procesoru
3. Procesor završava izvršenje tekuće instrukcije.
4. Procesor potvrđuje prekid.
5. Procesor spašava svoje trenutno stanje (programski brojač i još neke registre)
6. Programski brojač se postavi na sekvencu instrukcija koje se bave prekidom.

Većina računarskih sistema ima više perifernih uređaja povezanih na njega, pa računar mora biti u mogućnosti da detektuje i obrađuje prekide od različitih uređaja. Ukoliko se više prekida desi istovremeno, kontroler prekida donosi odluku čiji prekid obraditi na bazi **prioriteta prekida**. Neki uređaji će zahtijevati odgovor CPU mnogo brže nego drugi, pa im se dodijeli veći prioritet. Prioriteti prekida su obično direktno povezani na linije prekida.



Slika 82 IRQ kontroleri

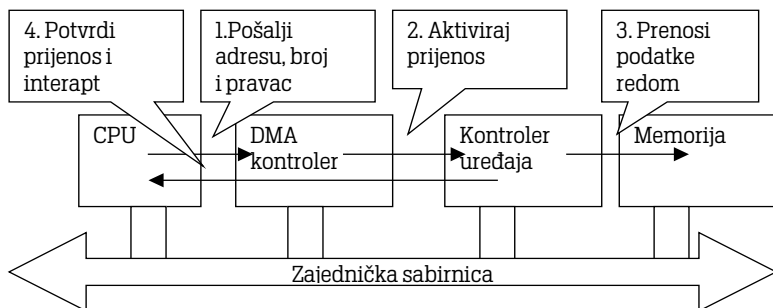
Na slici 82 je prikazano vezivanje kontrolera prekida prema procesoru. Kontroler prekida, npr. 8259 PIC prosljeđuje signal prekida sa jedne od linija vezanih prema uređajima na INT liniju procesora. Prioritet je određen linijom, pa je tako uređaj povezan na IRQ0 većeg prioriteta prekida od uređaja povezanog na IRQ1. Linija IRQ2 je povezana na dodatni kontroler prekida koji pruža mogućnost još 8 uređaja koji mogu generirati prekid. Za prekide koji dolaze sa tih uređaja, kontroler prekida mora na sabirnicu pružiti dodatnu informaciju procesoru koji je uređaj izazvao prekid. Kada CPU potvrdi prekid, taj uređaj postavi na sabirnicu svoju internu oznaku (vektor). Na bazi vektora, CPU potraži adresu odgovarajuće rutine za obradu prekida

generiranog od strane uređaja. Informacije o adresama svih prekidnih rutina čuvaju se u tabeli prekidnih vektora čiji svaki element sadrži adresu jedne prekidne rutine. Na PC računarima ta tabela obično ima 256 elemenata i koristi se ne samo za hardverski izazvane prekide od strane perifernog uređaja, nego i za sistemске pozive i izuzetke poput grešaka stranice i izvršenja nepostojećih instrukcija.

8.6.DMA kontroler

U mnogo slučajeva, podaci koji se šalju sa perifernog uređaja se prebacuju u memoriju. Procesor bi mogao da očitava podatke sa uređaja i upisuje ih u memoriju, ali to će biti neefikasno kod prijenosa velike količine podataka između perifernih uređaja i memorije. Zato je uveden dodatni uređaj, **DMA kontroler**. DMA kontroler koristi sistemsku sabirnicu i prenosi podatke između U/I kontrolera i memorije bez angažovanja CPU. Kad CPU želi prenijeti podatke, on pošalje instrukcije DMA kontroleru o smjeru prijenosa, U/I kontroleru koji je uključen, lokaciji podataka u memoriji i veličini bloka podataka koji se treba prenijeti. Procesor je nakon toga oslobođen za druge zadatke, a DMA će ga prekinuti kada se prijenos završi.

Jednostavan prijenos podataka na relaciji disk – memorija preko DMA kontrolera se može objasniti kroz četiri koraka slici Slika 83.



Slika 83 Postupak rada DMA kontrolera

Najprije procesor pošalje podatke DMA kontroleru o adresi u memoriji od koje se upisuju ili čitaju podaci, adresi uređaja sa kojim se komunicira, broju bajtova koji se prebacuje i pravcu prijenosa. Pošalje se i komanda kontroleru perifernog uređaja da započne sa generisanjem ili prijemom podataka. DMA kontroler će onda redom prozivati memorijsku lokaciju na adresnoj sabirnici, od prve koja mu je navedena i ako je riječ u čitanju sa uređaja, aktivirati i liniju koja zahtijeva upis u memoriju. Periferni uređaj će poslati podatak na sabirnicu podataka ili ga pročitati sa nje. DMA kontroler, povećava adresu memorijske lokacije i smanjuje broj bajtova koji se još moraju prenijeti. Sve dok je brojač veći od 0, ponavlja se prozivanje i upis/čitanje memorije. Kada je brojač 0, DMA kontroler generiše prekid i javlja CPU da je prijenos završen.

Iako procesor ne izvršava instrukcije tokom DMA prijenosa, to ne znači da ga DMA ne usporava. I DMA kontroler i procesor pristupaju sabirnici i prozivaju memorijske lokacije. CPU i DMA kontroler ne mogu koristiti sistemsku sabirnicu u isto vrijeme tako da mora postojati način dijeljenja sabirnice između njih. Koristi se jedan od slijedeća tri metoda:

- **Burst režim:** Čitav blok podataka se prenosi kroz jednu kontinualnu sekvencu. Kada je DMA kontroler dobio pravo na sistemsku sabirnicu, on prenosi sve podatke u bloku podataka (npr. 512 bajta) blokirajući procesor. Ovaj prijenos je najbrži, ali zadržava procesor prilično dug period vremena.
- **Režim krađe ciklusa:** U ovom režimu DMA blokira procesor prije prijenosa svakog bajta i odblokira ga nakon što je bajt prebačen. Time procesor nije mnogo neaktivan, ali je prijenos podataka manje brz nego u *burst* režimu rada.
- **Transparentni režim** zahtijeva najviše vremena za prijenos podataka, ali ne zaustavlja procesor, nego prenosi podatke isključivo kada procesor ne pristupa sabirnici. Realizacija ovog režima može biti dosta kompleksna.

8.7.Memorijski mapirani i U/I mapirani uređaji

Centralni procesor komunicira sa registrima U/I kontrolera, a u nekim slučajevima i registrima na U/I uređaju. Upisom u te registre operativni sistem može davati uređaju instrukcije, kao npr. zahtjev da započne prijenos podataka ili generiše ton određene frekvencije. Ovim registrima se može pristupiti na dva osnovna načina: memorijski mapirani pristup i U/I mapirani pristup perifernim uređajima.

Sa memorijski mapiranim pristupom, CPU vidi U/I kontroler kao običnu lokaciju u memoriji. Da bi poslao podatke U/I kontroleru, CPU piše ili čita podatke iz ove lokacije kao da je u memoriji. Npr., na računaru Commodore Amiga instrukcijom `move.b #252,$BFE001` se postavlja audio filter, koja izgleda kao instrukcija koja upisuje broj 252 na memorijsku lokaciju `BFE001`. Ali, pri pokušaju upisa tu lokaciju, U/I kontroler je prepoznao tu adresu, na kojoj inače nema memorijskih čipova i podatak prosljedio zvučnom čipu.

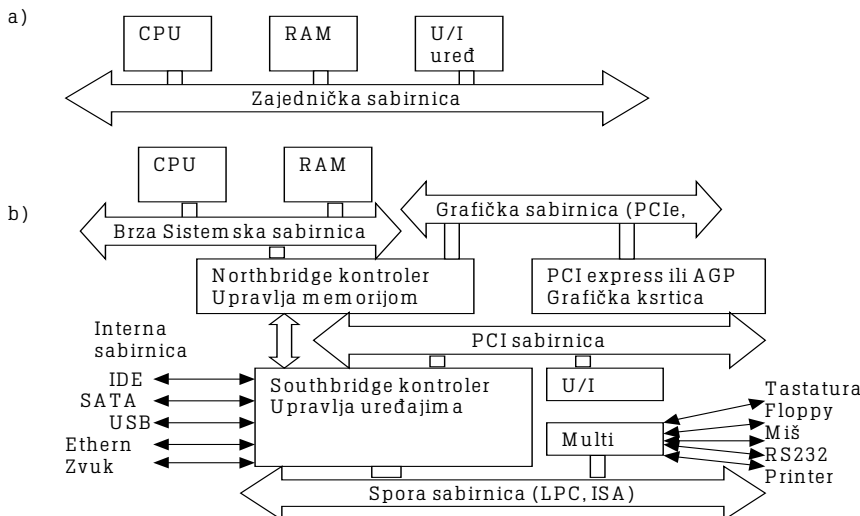
Sa U/I mapiranim pristupom, U/I kontroler ne dijeli adresni prostor sa memorijom, nego se koriste specijalne instrukcije za komunikaciju sa kontrolerom. Na primjer na PC računarima se izlaz na zvučnik šalje instrukcijom `out 42h,al`. Ovo daje više adresnog prostora i za memoriju i za U/I, ali zahtijeva dodatnu liniju procesora prema sabirnici da bi se znalo da li se pristupa memoriji ili U/I uređaju. Također može smanjiti fleksibilnost sa kojom CPU adresira U/I kontrolere.

Mnogi računari koriste oba pristupa, jer je neke uređaje praktičnije programirati kada su memorijski mapirani (obično ekran), a neke kada su U/I mapirani (npr. serijski port).

Pri projektovanju memorijski mapiranih uređaja mora se paziti na arhitekture s više sabirnica. Slika 84 prikazuje dvije arhitekture: *single-bus* arhitekturu i arhitekturu jednog modernog PC računara. Kod prve arhitekture adrese svih memorijskih lokacija prozivaju se preko jedne sabirnice. Memorijski mapirane periferni uređaje je lako realizovati, jer svaki zahtjev za memorijom dolazi na tu sabirnicu, pa zahtjev prepoznaju i memorija i U/I uređaj.

Kod modernih personalnih računara sve češća je arhitektura prikazana na donjem dijelu, gdje postoji brza sabirnicu za pristup CPU-a memoriji i više različitih sabirnica za pristup uređajima raznih brzina. Brza sistemka sabirnica je optimizirana na brzinu memorije i nisu pravljene kompromisi za ulazno-izlazne uređaje. Problem kod ovakve arhitekture je da U/I uređaji ne vide adrese na memorijskoj sabirnici i ne mogu na njih reagovati. Rješenje je u uvođenju dodatnih

kontrolera sabirnice (*Northbridge*, *Southbridge*) koji moraju biti informirani o postojanju memorijski mapiranog perifernog uređaja koji nije povezan na sabirnicu.



Slika 84 Sistemi sa jednom i više sabirnica

8.8. Interfejs između U/I kontrolera i uređaja

Kontroler na matičnoj ploči i kontroler uređaja međusobno su povezani interfejsima koji služe za prijenos podataka i kontrolnih signala. Dijele se u tri grupe:

Serijski interfejsi: U/I kontroler je jednom linijom podataka u jednom smjeru povezan na periferni uređaj, a podaci se prenose bit po bit. Ovo se koristi za sporije periferne uređaje kao što je tastatura ili uređaje koji koriste infrastrukturu vezanu za dvožične vodove (modemi, ADSL). Najvažniji predstavnici ovakvih interfejsa su Ethernet (mrežni uređaji), RS232 (za modeme i povezivanje različitih računara), SATA (za diskove), USB (za razne uređaje), PS/2 interfejs (tastature, miševi).

Paralelni interfejsi: U/I kontroler je višežično povezan sa perifernim uređajem, a biti podataka se prenose istovremeno za čitav bajt ili riječ. Ovakvi tipovi interfejsa se koriste za periferne uređaje velikih brzina kao što su grafičke kartice, kao i stariji štampači i diskovi. Popularni interfejsi ove vrste su ATAPI i SCSI (za diskove), priključci na ISA i PCI sabirnicu (za razne uređaje) i Centronix (za štampače). Paralelni interfejsi su po prirodi brži od serijskih. Ali kako su u novije vrijeme, porasle brzine kontrolera, serijski interfejsi poput SATA i USB su preuzeli i ulogu tradicionalno paralelnih interfejsa za štampače i diskove, jer su kablovi manji kao i zagrijavanje sistema.

Bežični interfejsi: Sa perifernim uređajem se komunicira koristeći radio ili infracrvene talase. Komunikacija je serijska, ali određeni načini kodiranja omogućavaju i paralelnu komunikaciju. Primjeri su *Bluetooth* i *WiFi*.

8.9. Softverska struktura U/I sistema

Ogromne razlike u hardverskim karakteristikama perifernih uređaja se prevazilaze strukturiranim dizajnom dijela operativnog sistema koji se bavi

ulazno/izlaznim uređajima. Softverski dio ulazno/izlaznog sistema može se podijeliti u četiri osnovne funkcionalnosti: obrađivače prekida, drajvere uređaja, ostale funkcije ulazno/izlaznog sistema u jezgru i funkcionalnosti ulazno/izlaznog sistema u korisničkom prostoru.

8.10.Obrađivači prekida

Obrađivači prekida su najniži sloj ulazno/izlaznog sistema. **Obrađivač prekida** ili servisna rutina prekida (ISR) je potptogram koji jezgro izvršava svaki put kada se desi specifični prekid. Svaki uređaj koji generiše prekid je povezan sa odgovarajućim obrađivačem. Obrađivač prekida je obično dio drajvera uređaja, ali ne mora biti (na nekim jednostavnijim operativnim sistemima obrađivači prekida su nezavisni od drajvera). Kada se desi prekid, procesor na bazi ulazne linije prekida ili oznake uređaja na sabirnici poziva odgovarajući obrađivač.

Ono po čemu se obrađivači prekida razlikuju od ostalih funkcija jezgra je poseban kontekst u kome se oni izvršavaju. Za vrijeme izvršavanja oni ne mogu pozivati blokirajuće pozive i predstavljaju kritičnu sekciju. Pošto se prekidi mogu desiti bilo kada, i obrađivači se mogu pozvati bilo kada. Pošto za vrijeme izvršavanja obrađivača ne mogu da se izvršavaju korisnički procesi niti drugi dijelovi jezgra, u kontekstu obrađivača mora se ostati što je moguće kraće vremena. Prema hardveru, prekid se mora obraditi sa što manjim zadržkom, a prema ostatku sistema obrađivač mora završiti posao što je prije moguće. Minimum što obrađivač mora odraditi je potvrda prijema prekida prema periferijskom uređaju, ali često imaju više posla.

Konflikt između dva cilja obrađivača; biti što kraći i odraditi što više posla; rješava se razdvajanjem obrađivača u dva dijela: gornja polovina i donja polovina obrađivača. U toku gornje polovine obrađivača obavljaju se poslovi koji su vremenski kritični: potvrda prijema prekida ili promjena stanja nekih registara kontrolera periferijskog uređaja. Nakon ove faze novi prekidi se dozvoljavaju, a jezgro ili korisnički program mogu nastaviti od mjesta gdje su prekinuti. Ovo može biti i mjesto kada se poziva raspoređivač procesa.

Donja polovina se izvršava kasnije, u pogodnijem trenutku, sa uključenim prekidima, kada se tipično obrađuju podaci stigili od periferijskog uređaja.

Na primjer, ako je uređaj mrežna kartica, pri prijemu paketa sa mreže mrežna kartica generiše prekid. Procesor nastavlja izvršenje rutine obrađivača prekida za mrežnu karticu. U toj rutini se potvrdi mrežnoj kartici prijem podataka i oni se kopiraju u glavnu memoriju. To se mora uraditi brzo, jer je glavna memorija puno veća od memorije mrežne kartice. Ako se to ne uradi na vrijeme, memorija mrežne kartice će se napuniti podacima novih paketa i oni će morati biti odbijeni. Nakon što su mrežni podaci prebačeni u glavnu memoriju, ostatak njihove obrade (na primjer prepoznavanje mrežnog pošiljaoca ili dešifriranje sadržaja) dovršiće se kasnije u donjoj polovini obrađivača prekida.

8.11.Drajveri

Najveći dio ulazno/izlaznog sistema u jezgru predstavljaju drajveri. Drajveri komuniciraju sa kontrolerima ulaznih i izlaznih uređaja i realizuju sve detalje i posebnosti funkcioniranja ovih kontrolera. Van drajvera su vidljive samo

operacije, kao što su, na primjer, operacije ulaza ili izlaza, koje omogućuju jednoobrazno korištenje ulaznih i izlaznih uređaja.

Uređaji su međusobno dosta različiti, ali postoji i određena sličnost u klasama uređaja, pa se za svaku klasu uređaja pišu odgovarajuće vrste drajvera. Osnovne kategorije drajvera su drajveri znakovnih uređaja, drajveri blokovskih uređaja, drajveri memorijskih uređaja i drajveri mrežnih uređaja.

Drajveri **znakovnih uređaja** su drajveri za uređaje kojima se podaci šalju serijski ili na takav način primaju od njih. Takvi uređaji su tastatura, RS232 serijski interfejs, štampač, modem, miš, zvučna kartica i slično. Ovakvim uređajima se može pristupati istim sistemskim pozivima kao kada se čita i piše datoteka. Stoga, drajveri znakovnih uređaja realizuju operacije:

- Inicijalizacija: Poziva se pri instalaciji drajvera. Ova operacija može registrovati i odgovarajući obrađivač prekida za dati uređaj.
- Finalizacija: Poziva se pri zatvaranju drajvera
- Otvaranje: koja se poziva kada program zahtijeva početni pristup uređaju.
- Zatvaranje: pokreće se pri zahtjevu za kraj pristupa uređaju
- Čitanje: poziva se svaki put kada se traži čitanje podataka od strane uređaja
- Pisanje: poziva se svaki put kada se šalju podaci uređaju
- I/O kontrola: poziva se ako je potrebno prebaciti uređaj u drugo stanje: npr. promijeniti brzinu serijske komunikacije,

Neka je korisnik u Unix sistemu otkucao komandu

```
echo "Zdravo" >/dev/lp0
```

Interpreter komandne linije će pozvati sistemski poziv *write* prema prividnoj datoteci koja se zove /dev/lp0. U okviru tog poziva navedeni su pokazivač na tekst i dužina teksta. To će pozvati odgovarajući potprogram u drajveru za pisanje ka uređaju pridruženom ovoj datoteci (u ovom primjeru linijski štampač).

Drajveri **blokovskih uređaja** (diskova) rade sa jedinicama podataka fiksne veličine. Njih najčešće poziva datotečni sistem. Drajveri blokovskih uređaja realizuju operacije:

- Inicijalizacija: Poziva se pri instalaciji drajvera.
- Finalizacija: Poziva se pri deaktiviranju drajvera
- Otvaranje: koja se poziva kada program zahtijeva početni pristup uređaju.
- Zatvaranje: pokreće se pri zahtjevu za kraj pristupa uređaju
- Asinhrono čitanje: zahtjev za čitanje se stavlja u red čekanja
- Asinhrono pisanje: zahtjev za pisanje se stavlja u red čekanja
- Definiranje strategije: određuje se način komunikacije drajvera sa hardverom, npr. da li će se koristiti DMA ili ne

Kao što se vidi, glavna razlika između drajvera blokovskih uređaja i drajvera znakovnih uređaja je u operacijama čitanja i pisanja. Dok se znakovnim uređajima podaci šalju ili primaju od njih u toku poziva *read* ili *write*, zahtjevi za prijem podataka od blokovskih uređaja (ili slanje prema njima) se stavljaju u red čekanja za asinhroni pristup. To omogućava da se koristi algoritam za raspoređivanje

pristupa disku koji minimizira kretanje glave diska ili čuvanje nekih blokova diska u RAM memoriji radi ubrzanog pristupa.

Neki uređaji, kao što je bafer video kartice su mnogo efikasniji ako im se pristupa koristeći direktni pristup memoriji. Drajveri **memorijskih uređaja** sadrže operacije:

- Inicijalizacija, finalizacija, otvaranje i zatvaranje, kako je već spomenuto
- Mapiranje dijela fizičke memorije u memoriju u korisničkom prostoru
- Operacija izmjene konteksta, može se pozvati asinhrono svaki put kada proces koji koristi memorijski uređaj promijeni status iz stanja izvršavanja.

Nakon mapiranja, upis podatka u neku lokaciju u korisničkom prostoru se manifestuje slanjem podatka uređaju. Kako bi više procesa moglo koristiti ovaj uređaj svaki proces ima vlastito mapiranje memorije uređaja u svoj virtualni prostor.

Drajveri **mrežnih uređaja**, kakve su *ethernet* kartice ili WiFi odašiljači sarađuju sa drajverima mrežnih protokola koji uređuju primljene i poslane pakete podataka u odgovarajući redoslijed. Operacije koje pružaju ovi drajveri su između ostalih:

- Slanje paketa: paket podataka se stavlja u red čekanja za slanje
- Postavljanje MAC adrese mrežne kartice
- *Multicast*: uključivanje prijema paketa koji su namijenjeni raznim primaocima
- Postavljanje rutine za obradu prekida
- Asinhroni prijem paketa koji se poziva iz prekidne rutine kada je paket stigao

Pri inicijalizaciji drajvera, registruje se skup potprograma koji realizuju ove operacije koje imaju pojedine vrste uređaja. Posebna tabela registrovanih drajvera sadrži slogove sa pokazivačima na odgovarajuće operacije. Zahvaljujući ovoj tabeli, moguće je u operativni sistem dodavati nove drajvere.

Drajveri moraju da reaguju i na prekide, koji stižu od kontrolera. Prekidi, npr. objavljuju da je završen prijenos podataka od kontrolera. U drajver se može ugraditi odgovarajući obrađivač prekida.

Na operativnim sistemima koji brinu o stanju napajanja, drajverima svih tipova dodaje se još jedna operacija koja se poziva kada uređaj treba da pređe u režim uštede energije. Autonomni računari kao što su *laptop* i *tablet* treba da brinu o potrošnji energije, s obzirom na ograničeni kapacitet baterija. Na primjer, kada se *laptop* računar prebaci iz režima rada sa mrežnim napajanjem u režim rada na baterijama poziva se odgovarajuća operacija drajvera koja može uraditi sljedeće mjere koje smanjuju potrošnju energije:

- Smanjenje osvjetljenja ekrana ili gašenje ekrana, tokom neaktivnosti korisnika
- Zaustavljanje motora hard diskova nakon perioda neaktivnosti
- Rjeđe prozivanje radio komunikacije
- Smanjenje frekvencije ili napona rada procesora

Pored grupiranja u ove četiri vrste drajvera, drajvere je moguće dalje grupirati u uže kategorije. Na primjer, drajveri za različite IDE diskove su međusobno veoma slični, ali se razlikuju u samo nekoliko instrukcija koje pristupaju registrima kontrolera (npr. noviji model diska dodao je komandu kontroleru koja uključuje

režim za uštedu energije, a sve ostalo je isto kao kod starijeg modela). Da bi se iskoristile zajedničke osobine drajvera, uvodi se način pisanja drajvera po slojevima. Drajeri višeg nivoa, npr. zajednički drajver za sve vrste IDE diskova, komuniciraju sa korisničkom aplikacijom. Oni mogu onda pozvati drajver nižeg nivoa koji poslane podatke prosljeđuje samom kontroleru uređaja.

8.12.Principi drajverskih operacija

Kod drajvera znakovnih uređaja, operacije čitanja i pisanja moguće je realizovati na tri osnovna načina.

Programirani ulaz/izlaz ili komunikacija prozivanjem je najjednostavnija metoda za komunikaciju između procesora i perifernog uređaja. U ovom pristupu, procesor je odgovoran za svu komunikaciju sa uređajem, koristeći čitanje i upis u registre uređaja. Neka npr. procesor šalje instrukciju odgovarajućem U/I uređaju da očekuje od njega podatke. Procesor onda mora čekati dok kontroler uređaja ne odgovori prije slanja podataka. Ovo može biti veoma neefikasno, jer bi procesor mogao za to vrijeme izvršavati druge korisničke procese. Ovakav način pristupa perifernom uređaju će se ilustrovati sljedećim primjerom.

Pretpostavimo da je uređaj memorijski mapiran na tako da se na adresama 100 i 101 nalaze njegovi registri podataka i statusni registri. Rutina *driver_write* se poziva kada je korisnički proces pozvao poziv *write* za slanje podataka uređaju. Drajver tada pozove *copy_from_user* rutinu koja prekopira podatke iz memorije korisničkog procesa u memoriju jezgra i kreće da šalje podatak, jedan po jedan znak. Uređaj će na kontrolni registar mapiran u memorijsku lokaciju ili U/I prostor upisati vrijednost READY kada je spreman, pa će se morati čekati prije slanja svakog znaka.

```
unsigned char * device_status = (unsigned char *)100;
unsigned char * device_data = (unsigned char *)101;
unsigned char p[BUFLN];
void driver_write(char * buffer,int count) {
    int i;
    copy_from_user(buffer, p, count);    /* p je buffer jezgra */
    for (i = 0; i < count; i++) {        /* petlja za svaki znak */
        while (*device_status != READY) ; /* ponovi dok nije spreman */
        *device_data = p[i];             /* pošalji jedan znak */
    }
    return_to_user();
}
```

Ako uređaj podržava prekide, komunikacija sa njim postaje znatno efikasnija. **U/I voden prekidima** ne zahtijeva procesorsko prozivanje da li je uređaj spreman, nego uređaj to sam javlja. Rutina za slanje podatka uređaju bi se sada sastojala iz dva dijela, sinhronog i asinhronog. Sinhroni dio je u *driver_write* rutini drajvera i poziva se kada je korisnički proces pozvao rutinu *write*. Ova rutina prekopira podatke i uključi prekide koje obrađuje drajver. Nakon toga pozove se raspoređivač kako bi se program pozivalac blokirao dok svi podaci ne budu poslani. Pretpostavka je da je signal spremnosti uređaja povezan na sistem prekida. Stoga, svaki put kada je uređaj spreman za prijem novog znaka, generirao bi se prekid. U rutini za obradu prekida bi se poslao novi znak, a kada je poslan i posljednji znak, deblokira se korisnički proces.

```

unsigned char * device_status = (unsigned char *)100;
unsigned char * device_data = (unsigned char *)101;
unsigned char p[BUFLen];
int i,cnt;
void driver_write(char * buffer,int count) {
    copy_from_user(buffer, p, count);
    enable_driver_interrupt();
    i=0;
    cnt=count;
    scheduler();
}
void driver_interrupthandler() {
    if(cnt-- == 0)
        unblock_user();
    else
        *device_data = p[i++];
    acknowledge_interrupt();
    return_from_interrupt();
}

```

Kada se koristi **DMA bazirana komunikacija**, drajver se također dijeli na sinhronu i asinhronu dijelove, kao i kod prekidima bazirane komunikacije. U ovom slučaju se prekidna rutina mnogo rjeđe poziva, u suštini samo pri završetku prijenosa. Poziv upisa podataka prema uređaju će postaviti DMA kontroler (početnu adresu bafera, koliko bajta se prenosi, I/O port kojim se obavlja transfer i pravac prijenosa) i reći mu da počne sa prijenosom podataka. Nakon toga se program pozivalac može uspavati. Kada je prijenos gotov, generiše se prekid koji vraća kontrolu programu pozivaocu. Evo verzija dijela drajvera sa DMA baziranom komunikaciju.

```

unsigned char * device_status = (unsigned char *)100;
unsigned char * device_data = (unsigned char *)101;
unsigned char p[BUFLen];
void driver_write(char * buffer,int count) {
    copy_from_user(buffer, p, count);
    set_up_DMA_controller(p,device_data,count,OUTPUT);
    enable_driver_interrupt();
    scheduler();
}
/* Ovaj dio se nalazi u prekidnoj rutini drajvera */
void driver_interrupthandler() {
    acknowledge_interrupt();
    unblock_user();
    return_from_interrupt();
}

```

8.13.Windows i POSIX API za pristup perifernim uređajima

Ako je potrebno iz Windows ili iz Unix programa poslati podatak perifernom uređaju poput serijskog porta, koriste se funkcije normalno namijenjene za rad s datotekama. U pozivu za otvaranje uređaja kao umjesto imena datoteke koristi se simboličko ime uređaja. Pozivi su navedeni u tabeli i izuzev poziva za slanje kontrolnog koda uređaju identični su pozivima za pristup datotekama.

Windows poziv	POSIX poziv	Namjena
CreateFile	open	Otvora U/I uređaj ili datoteku
WriteFile	write	Piše podatke na U/I uređaj
ReadFile	read	Čita podatke s U/I uređaja
DeviceIoControl(hDevice, dwIoControlCode, lpInBuffer, nInBufferSize, lpOutBuffer, nOutBufferSize, lpBytesReturned, lpOverlapped)	ioctl(iFd, iRequest, pBuf)	Šalje kontrolni kôd direktno drajveru periferijskog uređaja

8.14. Primjer drajvera za Linux

Kao ilustracija kako drajveri izgledaju daće se primjer jednog drajvera za Linux. U pitanju je znakovni uređaj koji metodom programiranog ulaza/izlaza šalje sadržaj uređaju povezanom na U/I adresu 80. Drajver počinje zaglavljima i globalnim varijablama.

```
#include <linux/kernel.h>
#include <linux/module.h>
/* Sve potrebne #include direktive nisu navedene */
static const char device_name[] = "sampledev";
static int device_file_major_number = 0;
char kbuf[1024];
```

Implementiraće se operacija pisanja na znakovni uređaj. U toj operaciji će se podaci iz korisničkog *buffer*-a kopirati u *buffer* u jezgru a zatim poslati uređaju.

```
static ssize_t device_file_write(struct file *filp,
const char *buf, size_t count, loff_t *offset) {
    int i, j;
    if(count > 1024) return -1;
    j = copy_from_user(kbuf, buf, count);
    for (i=0; i<count; i++)
        outb(kbuf[i], 0x80);
    return j;
}
```

Struktura *file_operations* sadrži pokazivače na funkcije koje implementiraju pojedine operacije, ovdje samo jednu.

```
static struct file_operations simple_driver_fops =
{
    .owner    = THIS_MODULE,
    .write    = device_file_write,
};
```

Inicijalizacija drajvera uključuje poziv njegove registracije.

```
int register_device(void) {
    int result = register_chrdev( 0, device_name, &simple_driver_fops );
    if( result < 0 ) {
        printk( KERN_WARNING "Error registering %i", result );
        return result;
    }
    device_file_major_number = result;
    printk( KERN_NOTICE "Registered device %i", device_file_major_number );
    return 0;
}
static int simple_driver_init(void){
    int result = register_device();
    return result;
}
```

Pri deaktiviranju drajvera potrebno ga je deregistrovati.

```
void unregister_device(void) {
    if(device_file_major_number != 0) {
        unregister_chrdev(device_file_major_number, device_name);
    }
}
static void simple_driver_exit(void) {
    unregister_device();
}
module_init(simple_driver_init);
module_exit(simple_driver_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Ja");
```

8.15.Ostale funkcije U/I sistema u jezgru

Izvan drajvera, a u režimu jezgra, obavljaju se još neke funkcionalnosti ulazno/izlaznog sistema. Neke od tih funkcionalnosti su: registrovanje drajvera, alokacija uređaja, raspoređivanje U/I operacija, smještanje u bafer, keširanje, praćenje statusa uređaja.

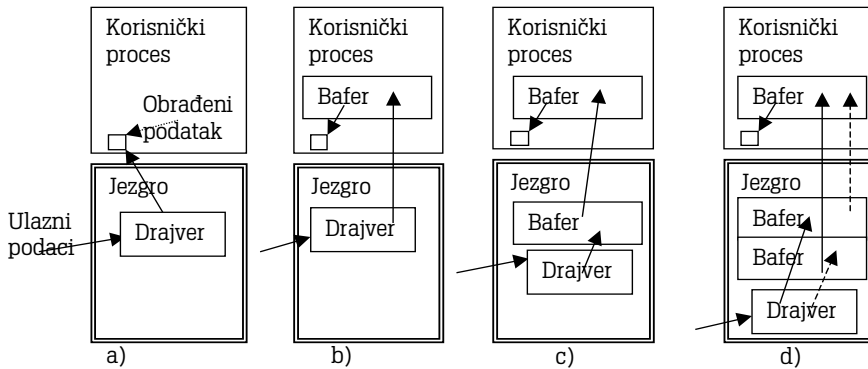
Registrovanje drajvera omogućava da se drajver učita u memoriju onda kada je potreban. Drajveri se čuvaju u spoljnoj memoriji i na zahtjev aplikacije ili korisnika učitaju u memoriju jezgra i inicijaliziraju. U nekim sistemima kao što je Linux drajveri moraju biti identični verziji jezgra, dok Windows i FreeBSD dopuštaju učitavanje drajvera pisanih za različite verzije jezgra.

Alokacija uređaja registruje koji je uređaj dodijeljen kojem procesu. Uređaji mogu biti posvećeni procesu ili dijeljeni. Posvećeni uređaji (trake, CD-ovi, štampači, terminali koji prikazuju preko cijelog ekrana) se dodjeljuju za duži period. Pošto drugi procesi ne mogu pristupati uređajima kada su dodijeljeni na ovaj način, ova alokacija se dopušta samo procesima koji imaju ovlaštenje za to. Dijeljeni uređaji (diskovi, grafički ekran) dopuštaju da im istovremeno pristupa više procesa.

Raspoređivanje ulazno-izlaznih operacija je slično raspoređivanju procesora. Kada proces zahtijeva ulazno/izlaznu operaciju, npr. želi da pošalje podatke modemu, proces se prebacuje iz reda čekanja na procesor u red čekanja na periferni uređaj, dok uređaj ne postane slobodan. Može se koristiti neki od algoritama, npr. FCFS da se odredi koji će proces dobiti pristup uređaju i kada. Kod blokovskih i mrežnih uređaja koji imaju asinhrona čitanja i pisanja, ne stavljaju se procesi u red čekanja, nego sami zahtjevi. Algoritmi za opsluživanje takvih zahtjeva mogu biti prilagođeni fizičkim karakteristikama uređaja. Na primjer, može se definirati redoslijed koji minimizira kretanje glave diska ili šalje mrežne pakete na način da se na odredištu mogu sastaviti na optimalan način.

Bafer je dio memorije a funkcioniše na principu proizvođač-potrošač i služi za čuvanje privremenih podataka prilikom prijenosa podataka između dva uređaja ili između uređaja i aplikacije. Mali baferi koji se nalaze u kontrolerima uređaja obično nisu dovoljni ako je razlika u brzini između proizvođača (npr. aplikativnog programa) i potrošača (npr. drajvera koji šalje podatke uređaju) znatna. Uvođenjem dodatne memorije podaci se čuvaju kako ih proizvođač generiše i šalju potrošaču kada je on spreman bez prevelikog usporjenja proizvođača.

Proizvođač i potrošač mogu biti u različitim kontekstima, npr. jedan od njih je obično u korisničkom prostoru, a drugi u jezgru. Na slici Slika 85 vide se različiti načini pozicioniranja bafera u memoriji. Neka podaci dolaze sa perifernog uređaja i šalju se korisničkom procesu.



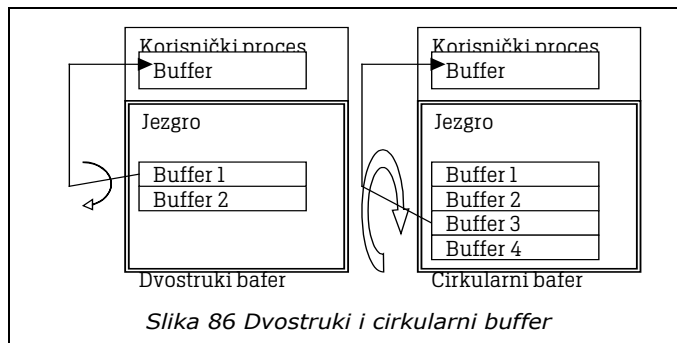
Slika 85 Komunikacija bez bafera, s baferima i dvostrukim baferima

Prva mogućnost (Slika 85 a) je da korisnički proces izvodi sistemski poziv *read*, onda se blokira i čeka dok dođe sljedeći znak. Rutina prekida daje znak dalje korisničkom programu i budi ga. Ovaj znak se obrađuje i čita se novi, nakon čega se proces ponovo blokira. Mana pristupa je u čestom prebacivanju iz korisničkog režima u režim jezgra. Korisnički proces je većinu vremena blokiran.

Slika 85 b) uvodi bafer u korisničkom prostoru. Kada proces pozove čitanje n podataka sa perifernog uređaja, on pruža bafer u svom području memorije. Proces se blokira, dok drajver upisuje podatke u memoriju procesa. Kada su svi podaci prebačeni proces se deblokira i podaci su mu odmah dostupni. Rješenje je dobro, ali nije uvijek izvodivo. Npr. neki računarski sistem imaju ograničenje opsega memorijskih adresa u kojima radi DMA (na starijim PC računarima to je prvih 16M, a na novijim to je prvih 4G memorije). Ako je proces mapiran u područje koje nije u zoni dokučivoj DMA kontroleru, transfer se ne može preusmjeriti u bafer u korisničkom prostoru. Pored toga, ovo rješenje omogućava aplikaciji da mijenja podatke u toku njihovog prijema ili slanja, što nije uvijek dopušteno.

Na slici Slika 85 c) bafer se stavi u jezgro. Performanse samog prebacivanja podataka su odlične. Kada se napuni bafer u jezgri, slijedi njegovo kopiranje u korisnički prostor. Problem se dešava sa novopridošlim znakovima koji će biti izgubljeni ili zaustavljeni dok traje ovo kopiranje. Najbolje performanse se postižu sa dva bafera u jezgri (Slika 85 d). Kada je jedan pun, počinje se puniti drugi bafer. Za to vrijeme se prvi bafer kopira u korisnički prostor.

Dvostruki bafer koristi dva systemska bafera umjesto jednog (Slika 86). Proces može prenijeti podatke iz jednog bafera u jezgro dok operativni sistem puni drugi bafer u jezgri. Ako je periferni uređaj prebrz (ili prespor ukoliko je riječ o slanju podataka prema uređaju), može biti potrebno i više buffera. Kod cirkularnog bafera se koristi više od dva bafera. Svaki bafer je jedna jedinica u cirkularnom baferu. Kada se jedan bafer napuni podacima, prelazi se na punjenje drugog, pa kada je i on pun na punjenje trećeg itd. Kada se napuni posljednji bafer, ponovo se puni prvi.



Keš (engl. *cache*) je područje memorije koje čuva kopiju podataka, najčešće sa diska. Pristup podacima u kešu znatno je brži od pristupa podacima na ulazno izlaznim uređajima. Keširanje je kopiranje podataka sa diska u keš memoriju čime se značajno poboljšavaju performanse U/I sistema. Keš se razlikuje od bafera po tome što bafer sadrži trenutno aktualne podatke, a keš bilo koju kopiju sa diska.

Praćenje statusa uređaja uključuje obradu grešaka koje se dešavaju prilikom ulazno/izlaznih operacija. Primjer greške je da je pozvan sistemski poziv za čitanje podataka sa miša koji u tom trenutku nije priključen. Ako je drajver prepoznao takvu situaciju, greška najčešće rezultuje izlaskom iz sistemskog poziva koji ju je prouzrokovao uz povratnu vrijednost koja indicira neuspjeh.

8.16.U/I sistem u korisničkom prostoru

Pored ulazno/izlaznih operacija u jezgru, dio ovih operacija je prebačen u korisnički prostor. Primjeri ulazno/izlaznog softvera u korisničkom prostoru su drajveri u korisničkom prostoru, softverske biblioteke za korisničko sučelje, *spooler*-i, mrežni servisi i mehanizam imenovanja uređaja u korisničkom prostoru. Ove operacije se obavljaju kao samostalni procesi, kao dinamičke biblioteke povezane sa korisničkim procesima ili se prozivaju iz jezgra prelaskom u korisnički režim rada.

Imenovanje uređaja omogućava korisničkim programima pristup uređajima na sličan način kao kada pristupaju datotekama. Jezgro osigurava odgovarajuće sistemske pozive preko kojih su se uređajima daje simboličko ime. U Unix kompatibilnim sistemima simbolička imena uređaja su smještene u direktoriju `/dev`. U tom direktoriju se kreiraju virtualne datoteke koje predstavljaju simbolička imena uređaja. Nekoliko standardnih imena uređaja u Linux sistemima, a slična su u ostalim Unix kompatibilnim sistemima na PC računarima, navedeno je u tabeli.

Blok Uređaj	Namjena	Znak uređaj	Namjena
<code>/dev/fd0</code>	Prvi floppy disk.	<code>/dev/dsp</code>	Digitalni zvučni izlaz
<code>/dev/hda</code>	Prvi IDE disk	<code>/dev/fb0</code>	Pristup video memoriji
<code>/dev/ht0</code>	IDE uređaj trake	<code>/dev/ttyl</code>	Prva konzola
<code>/dev/cdrom</code>	Glavni CDROM uređaj	<code>/dev/ttyS0</code>	Prvi serijski port
<code>/dev/loop0</code>	Datoteka koja simulira disk	<code>/dev/lp0</code>	Prvi paralelni štampač
<code>/dev/md0</code>	RAID struktura više diskova	<code>/dev/mixer</code>	Mikser zvučne kartice
<code>/dev/pda</code>	IDE disk, paralelni port	<code>/dev/zero</code>	Stalno šalje vrijednost 0
<code>/dev/pcd0</code>	CDROM, paralelni port	<code>/dev/psaux</code>	PS/2 miš
<code>/dev/sda</code>	Prvi SCSI disk	<code>/dev/random</code>	Šalje slučajne brojeve

Simbolička imena uređaja kreiraju se alatom *mknod* koji poziva istoimeni sistemski poziv. Interni brojevi koji identifikuju drajvere uređaja, koji se definišu unutar drajvera uređaja radi njegovog prepoznavanja, se na ovaj način pridružuju simboličkom imenu. Tako na primjer, da se definiše uređaj *mouse0*, koji je znakovnog tipa (c), a čiji su identifikacijski brojevi 13 (oznaka drajvera) i 32 (dodatna varijacijska informacija drajveru) navodi se komanda

```
mknod /dev/mouse0 c 13 32
```

Pri registraciji drajvera u Linux sistemu, interni broj drajvera se dobiva kao rezultat funkcije *register_chrdev* ili *register_blkdev*.

MS DOS ima nekoliko imenovanih uređaja. Tako ime CON: predstavlja konzolu (tastaturu i ekran), LPT1: predstavlja prvi štampač a COM1: serijski port.

Windows NT koristi manje poznat sistem za simboličko imenovanje uređaja. Njegov dio jezgra koji se zove ObjectManager imenuje uređaje interno. Tako na primjer prvi CD ROM uređaj se interno zove \Device\CdRom0, ali to ime nije vidljivo korisničkim programima i ne pripada datotečnom sistemu. Uređaji čija imena su dostupna korisničkim programima imaju globalno definirana alternativna imena. Ta imena počinju besmislenim imenom direktorija \\.\, kako ne bi greškom tražena u datotečnom sistemu. Neka globalno vidljiva imena uređaja su u sljedećoj tabeli.

Globalno Ime	Uređaj
\\.\PhysicalDrive0	Prvi fizički disk
\\.\COM1	Prvi serijski port
\\.\LPT1	Prvi paralelni port
\\.\DISPLAY1	Prvi ekranski uređaj.

Za mnoge uređaje pisanje drajvera u jezgru je nepotrebno težak zadatak. Često je za pisanje drajvera u jezgru potrebno na poseban način konfigurirati kompajler za programski jezik, a ovi drajveri moraju biti pisani vrlo pažljivo jer greške u njima imaju fatalne posljedice za cijeli operativni sistem. Često je za komunikaciju sa uređajem dovoljno da samo obradi prekid i da pristup memorijskom prostoru uređaja. Logika kontrole uređaja ne mora nužno biti u jezgru pa se za takve uređaje (npr. štampači, drajveri za grafičke kartice u Unix sistemima, industrijske kartice) realizuju **drajveri u korisničkom prostoru**. U jezgru se implementira samo veoma mali dio drajvera, dok se ostatak piše u korisničkom prostoru. Idealni uređaji za ovakav tip drajvera su oni kojima se pristupa mapiranjem memorije i kompletno se mogu kontrolirati pisanjem u tu memoriju, generišu prekide i ne zahtijevaju usluge podsistema u jezgru. No, ova vrsta drajvera nije pogodna ako treba koristiti usluge podsistema u jezgru. Na primjer, drajver za mrežnu karticu u korisničkom prostoru bi bio složeniji od drajvera u jezgru, jer bi trebalo realizovati u korisničkom prostoru i sve mrežne protokole koji ga koriste. Windows od verzije Vista podržava *User-Mode Driver Framework* (UMDF), mogućnost pravljenja drajvera u korisničkom režimu rada. Linux od verzije 2.6.24 također dopušta ovu vrstu drajvera za proizvoljne uređaje, uz dodatak već ranije postojećim drajverima u korisničkom prostoru: CUPS za štampače i X drajveri za grafičke kartice i miševe.

Softverske **biblioteke za korisnički interfejs** se povezuju sa korisničkim programima. Sarađujući sa drajverom za grafičku karticu, tastaturu i miša omogućavaju aplikativnim programima da pristupaju tim uređajima u tekstualnom ili grafičkom

režimu rada. U tekstualnom režimu rada, ove biblioteke realizuju funkcije kao što su *printf* i *scanf* u jeziku C. U grafičkom režimu uz pomoć ovih biblioteka se iscrtavaju linije, slike, prozori, liste za izbor podataka, kućice za popunjavanje itd.

Spooler je pozadinski proces najčešće vezan za štampače i crtače. On omogućava istovremeni pristup nedjeljivim uređajima tako što korisnički proces upisuje podatke namijenjene uređaju (npr. komande štampaču) u privremene datoteke na disku. Spooler gleda redoslijed dolaska tih privremenih datoteka i njihov sadržaj šalje redom jednu po jednu prema štampaču. Korištenjem spoolera, proces može brzo postaviti zahtjev u bafer, a nakon toga nastaviti sa drugim aktivnostima.

Mrežni servisi su pozadinski procesi koji koriste mrežne protokole. Ti procesi se izvršavaju na mrežnom aplikativnom sloju. Svaki servis pruža serverske ili klijentske usluge. Serverske usluge su dostupne računarima na mreži. Primjeri su *Domain Name System* (DNS) koji prevodi imena domena u adrese IP protokola, DHCP koji dodjeljuje mrežne konfiguracije računarima na mreži, servisi za slanje i prijem elektronske pošte, servisi za dijeljenje datoteka na mreži, dijeljenje štampača, Web serveri, servisi za otkrivanje računara na mreži, servisi za slanje kratkih poruka itd.

8.17.Zadaci

Zadatak 1. Koristeći systemske pozive *read* i *write* iz POSIX API napišite program u jeziku C koji šalje poruku "REQUEST" na prvi serijski komunikacijski port i zatim sa njega prima odziv koji je veličine 128 bajta.

Rješenje:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
void main() {
    char buf[128];
    int size;
    int fd = open("/dev/tty0", O_RDWR);
    write(fd, "REQUEST", 7);
    size = read(fd, buf, 128);
}
```

Zadatak 2: Napišite principijelni drajver uređaja sa koga se može čitati, ako je uređaj šesnaestbitni memorijski mapirani periferni uređaj, čiji je ulazni registar smješten na adresi 1098, a statusni registar na adresi 1099 ima 3. bit postavljen na 1 kada je uređaj spreman. Koristi se programirani U/I.

Rješenje:

```
unsigned char p[BUFLEN];
short int * status=(short int *) 1099;
short int * data=(short int *) 1098;
void driver_read(char * buffer,int count) {
    for (int i=0; i<count; i++) {
        while (!( *status & (1 << 3))) {};
        p[i]=*data;
    }
    copy_to_user (buffer,p,count);
    return_to_user();
}
```

Zadatak 3: Napišite principijelni drajver uređaja prema kome se može pisati, ako je uređaj šesnaestobitni U/I mapirani periferijski uređaj, čiji je izlazni registar smješten na adresu 1095, a kada je uređaj spreman on generiše prekid. Koristi se prekidima vođen U/I.

Rješenje:

```
unsigned char p[BUFLen];
int i,cnt;
void driver_write(char * buffer,int count) {
    copy_from_user(buffer, p, count);
    enable_driver_interrupt();
    i=0;
    cnt=count;
    scheduler();
}
void driver_interrupthandler() { /* U prekidnoj rutini drajvera */
    if(cnt-- == 0) unblock_user();
    else out(1095,p[i++]);
    acknowledge_interrupt();
    return_from_interrupt();
}
```

Zadatak 4: Napišite principijelni drajver uređaja prema kome se može pisati ako je uređaj šesnaestbitni memorijski mapirani periferijski uređaj, čiji je izlazni registar smješten na adresu 1365. Koristi se DMA baziran U/I. Port za izbor DMA adrese je 474, port za dužinu DMA bloka je 475, inicijalizira se postavljanjem 5. bita porta 476 a port uređaja se šalje na 477.

Rješenje:

```
...
short int * dmaaddr=(short int *) 474;
short int * dmalen=(short int *) 475;
short int * dmaset=(short int *) 476;
short int * dmaport=(short int *) 477;
short int * data=(short int *) 1365;
unsigned char p[BUFLen];
void driver_write(char * buffer,int count) {
    copy_from_user(buffer, p, count);
    *dmaaddr=p & 0xFFFF;    *dmaaddr= p >> 16;
    *dmalen=count & 0xFFFF;    *dmalen= count >> 16;
    *dmaport= &data;    *dmaset |= (1 << 5);
    enable_driver_interrupt();
    scheduler();
}
void driver_interrupthandler() { /* u prekidnoj rutini drajvera */
    acknowledge_interrupt();
    unblock_user();
    return_from_interrupt();
}
```

Zadatak 5: Koliko se procenata vremena potroši za obradu sata ako je frekvencija sata 140 Hz, a obrada pojedinog zahtjeva traje 1 ms?

Rješenje: Potroši se 14% vremena

9. UREĐAJI RAZUMLJIVI KORISNIKU

U ovom poglavlju će biti riječi o principima rada uređaja koji direktno komuniciraju sa korisnikom. Ovim uređajima se upravlja u korisničkom prostoru, kao i jezgru. Uz ovu temu je vezana oblast korisničkog sučelja.

9.1. Tastatura

Tastatura je osnovni ulazni uređaj personalnih, serverskih i *laptop* računara. Elektronski i mehanički, ovaj uređaj je razmjerno jednostavan. Sastoji se od 40-110 prekidača čije stanje (pritisnut/optušen) se detektuje jednim bitom. Tastature na PC računarima su uređaji koji imaju dva jednostavna kontrolera, od kojih je jedan na matičnoj ploči računara, a drugi na samoj tastaturi. Kontroler na tastaturi javlja koji je taster pritisnut ili otpušen. Tasteri se označavaju *scan* kodovima koji zavise od pozicije tastera na tastaturi. Tastatura može generirati prekid koji se dešava kada je taster pritisnut ili otpušen. Tada se može očitati stanje ulaznog porta tastature, očitati koji je taster pritisnut i sačuvati to u ulaznom baferu.

Drajver tastature očitava *scan* kodove za znakove koji su se sakupili u bufferu u međuvremenu. Treba ih konvertovati u razumljive znakove. Na primjer, kada se pritisne taster A, *scan* kôd (30) se stavi u registar. Drajver treba da odredi, na bazi ranijeg pritiska ili otpuštanja tastera da li je to a, A, CTRL-A, ALT-A,... U tu svrhu se koristi ASCII tabela, prilagođena različitim nacionalnim jezicima koja se može eksterno učitati.

Drajveri za tastaturu mogu da vraćaju samo unesene znakove (nekanonski ili sirovi režim) ili da prate razne kombinacije tastera sa specijalnim značenjem (kanonski ili kuhani režim). Na primjer u kanonskom režimu rada kombinacija CTRL-H briše prethodno uneseni znak.

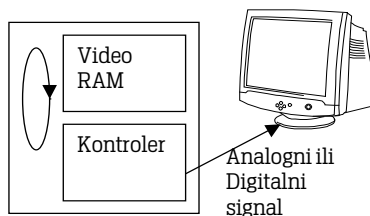
9.2. Miš

Miš je uređaj uključen na većini PC računara, ali se nekad zamjenjuje dodirnom pločom (*touchpad*) ili kuglicom (*trackball*) sa istom funkcionalnošću. Klasični tip miša ima gumenu loptu koja viri kroz rupu na dnu i rotira kada miš ide preko grube površine. Kako lopta rotira, ona trlja i okreće dva gumena valjka postavljena ortogonalno. Na bazi podatka koliko se pomjerio koji od valjaka dobiva se pomjeraj miša po x i y osi. Dati pomjeraji i status jednog do tri tastera na mišu se proslijeđuju matičnoj ploči u paketu koji je velik tri bajta. Pošto se valjci dosta prljaju, vremenom je prevladao noviji tip miša koji promjene pozicije po x i y osi detektuje promjenom intenziteta svjetla.

9.3. Ekranski hardver

Tekst ili slike koje se prikazuju na ekranu obično se predstavljaju kao pravougaona područja na ekranu, povezana sa odgovarajućim područjem u memoriji. Promjena vrijednosti memorije dodijeljene ekranu se automatski reflektuje na slici. Ovakav način prikaza grafike se zove **rasterska grafika**, za razliku od vektorske grafike kod koje se šalju komande uređaju poput komandi za iscrtavanje teksta ili linija. Rasterska grafika prikazuje se hardverskim uređajem koji se zove grafički adapter ili grafička kartica. On sadrži posebnu memoriju zvanu **video RAM**, koji čini dio

adresnog prostora računara i obraća se CPU na isti način kao i ostatak memorije (Slika 87). Slika ekrana je ovdje pohranjena u znakovnom režimu ili bit mapi.

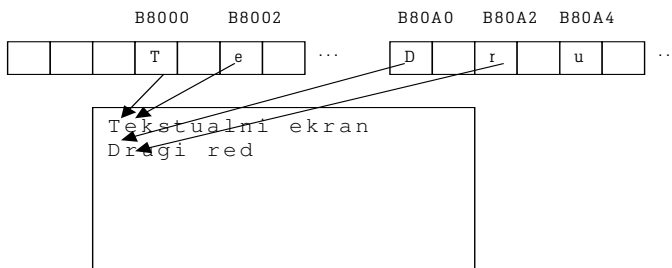


Slika 87 Generiranje slike

U znakovnom režimu, svaki bajt (ili 2 bajta) video RAM-a sadrži jedan znak (obično ASCII) koji se prikazuje. U režimu bit mape, svaki *pixel* (tačkica) na ekranu se predstavlja posebno u video RAM, sa 1 bit po *pixel*-u za najjednostavniju crno-bijelu sliku do 24 bita po *pixel*-u za visoku kvalitetu prikaza boja.

Pored video RAM-a, drugi važan dio grafičkog adaptera je čip koji se zove **video kontroler**. Ovaj čip očitava znakove ili bitove *pixel*-a iz video RAM-a i generira video signal koji se koristi za upravljanje monitorom. Monitor sa katodnom cijevi emituje snop elektrona koji ide horizontalno preko ekrana, osvjetljujući fosfornu pozadinu. Tipično ekran ima 480 do 1024 linija od vrha do dna, s 640-1200 *pixel*-a po liniji. Signal video kontrolera modulira snop elektrona, uz utvrđivanje da li će *pixel* biti svijetao ili taman. Kolor monitori imaju tri barijere, za crvenu, zelenu i plavu komponentu boje, koje su samostalno modulirane. Monitor sa tečnim kristalima radi na drugačijem principu. On također koristi tri komponente boja, ali se slika iscrtava dovodeći napon na pojedinu tačku kroz rešetku bita, što zakreće tečne kristale koji polarizuju svjetlost..

Na slici Slika 88 se vidi dio video RAM-a za prikaz u znakovnom režimu. Svaki znak na ekranu na slici zauzima dva bajta u RAM. Prvi bajt je ASCII kôd za znak koji će biti prikazani. Naredni bajt je atribut bajt, koji se koristi za određivanje boje znaka i njegove pozadine.



Slika 88 Video memorija tekstualnog ekrana

Drajver za prikaz na tekstualnom ekranu je razmjerno jednostavan, jer tekstualni ekran sadrži ASCII znakove. Primljeni znakovi koji se ispisuju se kopiraju na odgovarajuća mjesta u tekstualnoj video memoriji i pomjera kursor na sljedeće

mjesto za ispis (i sljedeću poziciju u video memoriji za smještanje narednog znaka). Izuzetak predstavljaju nevidljivi ASCII znakovi, oni sa kodom manjim od 32, kao što su znakovi koji predstavljaju prelazak u novi red (ASCII kôd 10) ili pomicanje za jedan znak ulijevo (ASCII kôd 8). Pri nailasku na takve znakove, drajver ne stavlja njihove kodove u memoriju ekrana, nego obavlja akcije poput promjene pozicije na ekranu. Treba još paziti na specijalni slučaj ispisa u najnižem redu ekrana, kada je potrebno pomjeriti cijelu video memoriju za jedan red naviše (*scroll*). Neki drajveri za tekstualni ekran podržavaju dodatne mogućnosti koje nisu sastavni dio ASCII skupa znakova. To su ANSI *escape* sekvence i prije svega su namijenjene za precizno podešavanje kursora na ekranu. Njih prihvata terminalski drajver, i imaju posebno ponašanje nakon ESC znaka. To je bajt s ASCII kodom 27. Neke od ovih sekvenci mogu imati parametre (Slika 89).

Escape sekvenca	Značenje
ESC [n A	Pomjeri se n linija gore
ESC [n B	Pomjeri se n linija dolje
ESC [n C	Pomjeri se n znakova desno
ESC [n D	Pomjeri se n znakova lijevo
ESC [m;n H	Pomjeri kursor na poziciju (m,n)
ESC [O J	Obriši ekran od kursora do kraja
ESC [O K	Obriši liniju od kursora do kraja
ESC [4 m	Režim podebljanih slova

Slika 89 ANSI sekvence

U režimu bit mape se svaki *pixel* na ekranu individualno kontroliše i predstavlja jednim ili više bitova u video RAM. Koliko bitova je potrebno, zavisi od broja boja koji se želi predstaviti. Sa jednim bitom po *pixel*-u mogu se imati samo dvije boje na ekranu (npr. crna i bijela) dok se sa n bita po *pixel*-u može imati 2^n boja. Skup boja se može izabrati i taj skup se zove **paleta**. Za $n=24$ može se imati oko 16 miliona različitih boja. Svaki *pixel* tada je predstavljen sa tri bajta koji predstavljaju crvenu, zelenu i plavu komponentu boje. Preko ovih komponenti se na crnoj pozadini ekrana mogu dobiti sve čovjekovom oku vidljive boje. Pored razlika u broju boja, grafički režimi se razlikuju po dimenzijama, i broju *pixel*-a koji se mogu prikazati po x i y osi. Najčešće su 320x200, 640x480 (VGA), 800x600 (SVGA), 1024x768 (XGA), 1280x1024 i 1600x1200.

Glavni nedostatak bit mapiranog pristupa je u velikoj upotrebi memorije. Na primjer, prikaz 1280x1024 u boji sa 24 bita po *pixel*-u zahtijeva gotovo 4 MB RAM-a, samo za čuvanje slike. Video RAM mora biti brz jer se on obično 50 puta u sekundi stalno u potpunosti pročita od strane video kontrolera. U cilju ubrzanja video kontrolera, primjenjuju se pristupi smanjenja ili povećanja njegovih mogućnosti. Smanjenje mogućnosti predviđa smanjenje broja tačaka (rezolucije) ili broja boja (uz odgovarajuću paletu) koje se istovremeno prikazuju na ekranu. Time se smanjuje veličina video memorije i samim tim posao koji video kontroler i centralni procesor moraju obaviti, ali nažalost i kvalitet slike. Porastom potreba za računarskim igrima i filmovima, krenulo se u pravcu povećanja mogućnosti grafičkih kontrolera. Pored mogućnosti da se centralnim procesorom iscrtaavaju likovi i slike na ekranu, sami grafički kontroleri dobivaju komande za crtanje punih linija, popunjavanje pravougaonika, prebacivanje pravougaonih dijelova slike s

jednog mjesta ekrana na drugo, iscrtavanje trouglova i drugih likova u prostoru sa projekcijom na ravan i sjenčanje trodimenzionalnih likova radi postizanja foto-realističnosti. Ugrađivanjem ovih mogućnosti u grafički kontroler, centralni procesor se oslobađa većine poslova vezanih za generiranje slike u realnom vremenu. Grafički kontroler se dalje ubrzava ugradnjom u njega većeg broja procesora koji paralelno obrađuju različite dijelove slike.

9.4.Windows API za ekran, tastaturu i miša

Programi za Windows su uglavnom grafički, ali operativni sistem podržava i tekstualni pristup. Tekstualne aplikacije se zovu konzolne aplikacije. Konzolne aplikacije u verzijama Windows 95 do Windows XP prikazuju podatke u tekstualnom režimu ekrana ili u grafičkom prozoru uz iscrtavanje teksta, dok počevši od Windows Vista isključivo su u grafičkom prozoru uz iscrtavanje teksta. Konzolne funkcije se pozivaju kada se u C programima koriste funkcije poput *scanf* i *printf*. Konzolne funkcije su date u sljedećoj tabeli.

Funkcija	Značenje
SetConsoleTitle(lpConsoleTitle)	Postavlja naslov konzolnog prozora
WriteConsole(hConsoleOutput, lpBuffer, nNumberOfCharsToWrite, lpNumberOfCharsWritten, lpReserved)	Ispisuje tekst na konzolni ekran
GetConsoleWindow()	Dobavlja rukovatelj konzolnog prozora
ReadConsole(hConsoleInput, lpBuffer, nNumberOfCharsToRead, pNumberOfCharsRead, lpReserved)	Unosi tekst sa tastature na konzolnom ekranu
PeekConsoleInput(hConsoleInput, lpBuffer, nLength, lpNumberOfEventsRead)	Čita unesene tastere bez njihovog brisanja
SetConsoleCursorPosition(hConsoleOutput, dwCursorPosition)	Postavlja poziciju kursora konzole
WriteConsoleInput(hConsoleInput, lpBuffer, nLength, lpNumberOfEventsWritten)	Šalje podatke na ulaz konzole

Sljedeći primjer ilustruje upotrebu ovih funkcija, traži od korisnika unos teksta i prikazuje ga na drugom mjestu konzolnog prozora.

```
#include <windows.h>
int main(void){
    char Poruka[500];
    COORD coord;
    HANDLE hIn = GetStdHandle(STD_INPUT_HANDLE);
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD Preneseno;
    WriteConsole(hOut, "Unesi tekst", 11, &Preneseno, NULL);
    ReadConsole(hIn, Poruka, sizeof(Poruka), &Preneseno, NULL);
    coord.X = 10;
    coord.Y = 8;
    SetConsoleCursorPosition(hOut, coord);
    WriteConsole(hOut, Poruka, strlen(Poruka), &Preneseno, NULL);
}
```

Spisak Windows API funkcija koje se bave prikazom na ekranu, korisničkim sučeljem, tastaturom i mišem je ogroman, čak i ako se ograniči samo na klasični Win32 API. U narednoj tabeli spomenuće se samo manji broj njih.

Funkcija	Namjena
CreateWindow(lpszClassName, lpszWindowName, dwStyle, x, y, nWidth, nHeight, hwndParent, hmenu, hinst, lpvParam)	Kreira prozor navedene klase, naslova, stila koodrinata, dimenzija sa pripadnošću drugom prozoru i instanci programa
DestroyWindow(hWnd)	Zatvara prozor i briše ga iz memorije
RegisterClass(lpwc)	Kreira prozorsku klasu preko strukture
EnumWindows(wundenmpc, lParam)	Proziva sve prozore na ekranu
GetMessage(lpmsg, hwnd, uMsgFilterMin, uMsgFilterMax)	Preuzima narednu poruku za prozor
DispatchMessage(lpmsg)	Prosljeđuje poruku prozoru na obradu
CreateMenu()	Kreira meni
AppendMenu(hMenu, uFlags, idNewItem, lpszNewItem)	Dodaje opcije meniju
SetMenu(hwnd, hmenu)	Pridružuje meni prozoru
ShowCursor(bShow)	Prikazuje ili sakriva strelicu miša
CheckDlgButton(hwndDlg, idButton, uCheck)	Mijenja kućicu za izbor i radio tastere
GetKeyState(nVirtKey)	Uzima stanje tastera na tastaturi
GetScrollPos(hwnd, nBar)	Čita poziciju kliznih traka prozora
CreateFontIndirect(lpf)	Kreira font sa osobinama iz strukture
GetDC(hwnd)	Preuzima kontekst uređaja za prozor
BitBlt(hdcDest, nXDest, nYDest, nWidth, nHeight, hdcSrc, nXSrc, nYSrc, dwRop)	Prebacuje bit mapiranu sliku na drugo mjesto na prozoru ili memoriji
CreateBitmap(nWidth, nHeight, uPlanes, uBitsPerPel, lpvBits)	Kreira bit mapiranu sliku određenih dimenzija
PlayEnhMetaFile(hdc, hemf, lpRect)	Izvršava zapamćene funkcije za crtanje
SetTextColor(hdc, crColor)	Mijenja boju iscrtavanja teksta
TextOut(hdc, nXStart, nYStart, lpszString, cbString)	Ispisuje tekst u grafičkom režimu na navedene koordinate
Arc(hdc, nLeft, nTop, nRight, nBottom, nStartX, nStartY, nEndX, nEndY)	Kružni luk od upisane kružnice pravougaonika sa početnom i krajnjom tačkom
BeginPaint(hWnd, lpPaintStruct)	Daje kontekst uređaja i zaključava crtanje
EndPaint(hWnd, lpPaintStruct)	Završava akcije crtanja
CreateHatchBrush(nHatchPattern, crColor)	Kreira šrafiranu četku za crtanje
ExtFloodFill(hdc, X, Y, crColor, uFillType)	Ispunjava zatvoreni lik šrafurom
Ellipse(hdc, nLeft, nTop, nRight, nBottom)	Crta upisanu elipsu u pravougaoniku
Rectangle(hdc, nLeft, nTop, nRight, nBottom)	Crta pravougaonik
PolyBezier(hdc, lpPoint, dwPoints)	Crta Bezierovu krivu
Polygon(hdc, lpPoint, nPoints)	Crta poligon sa vrhovima u niz
SetPixelV(hdc, X, Y, crColor)	Crta jednu tačku

Funkcije posvećene prozorima kao prvi parametar imaju rukovatelj prozorom koji se dobije kao rezultat funkcije *CreateWindow*. Funkcije namijenjene crtanju unutar prozora koriste kontekst uređaja kao prvi parametar, koji se obično dobije funkcijom *BeginPaint* ili *GetDC*. On se najčešće odnosi na korisničko područje prozora, ali se iste funkcije mogu koristiti i za izlaz na štampač, EMF datoteku vektorske grafike, okvire prozora itd. Tako na primjer za ispis teksta unutar prozora koristi se sljedeći uzorak koda:

```
HDC = GetDC ( hwnd ) ;
TextOut ( HDC,20,30,"Zdravo! ",7) ;
ReleaseDC ( hwnd,HDC ) ;
```

U prvoj naredbi se dobiva rukovatelj na kontekst uređaja, HDC. On predstavlja unutrašnji dio prozora, bez okvira i menija koji se iscrtava. Druga koristi kontekst uređaja da napiše liniju teksta na ekranu, navodeći (x , y) koordinate gdje string počinje, pokazivač na sam string, i njegovu dužinu. Trećom linijom se oslobađa kontekst uređaja.

Za ilustraciju GUI programiranja, ovdje će se uzeti Win32 API, koji je podržan od strane svih 32 - bitnih verzija Windowsa. Dok se aplikacije za konzolu pišu na tradicionalni način, grafičke aplikacije se pišu stilom upravljanim događajima. Razlog je što korisnički grafički programi moraju reagovati na različite događaje koje generiše miš (promjena veličine prozora, izbor opcije u meniju, klik na unosno polje itd.). Kao posljedica toga, Windows programi su orijentirani na poruke. Korisničke akcije koje uključuju upotrebu tastature ili miša su detektovane od strane Windows-a i pretvore se u poruke programu koje posjeduje prozor. Svaki program ima red poruka kome se šalju sve poruke koje se odnose na njega. Petlja glavnog programa preuzima narednu poruku i pozove interni potprogram da obradi tu vrstu poruke.

Sljedeći primjer pokazuje program u C koristeći Windows API koji prikazuje osnovni prozor na ekranu i jedan crveni pravougaonik unutar njega. Program se sastoji od četiri osnovna dijela: registracija prozorske klase, kreiranje i prikaz prozora, petlje koja prosljeđuje poruke i prozorske procedure koja ih obrađuje.

Glavni program počinje funkcijom *WinMain*. Prvi parametar je identifikacija programa za ostatak sistema. Drugi parametar je prisutan samo zbog kompatibilnosti sa Win16 i više se ne koristi. Treći parametar, *szCmd*, je nulom završen niz koji sadrži komandnu liniju kojom je pokrenut program (ime programa sa parametrima). Četvrti parametar, *iCmdShow*, govori da li početni prozor programa treba zauzimati cijeli ekran, dio ekrana, ili samo traku zadataka. Svaki prozor mora imati povezani objekt klase koja definira njegova svojstva. Objekt tipa *WNDCLASS* ima 10 polja. Najvažnije polje je *lpfnWndProc* koje je pokazivač na funkciju čiji je zadatak da upravlja porukama usmjerenim za ovaj prozor. Polje *lpszClassName* sadrži tekstualno ime klase prozora. Nakon što je *WNDCLASS* inicijaliziran, *RegisterClass* je pozvan da Windows zna koji potprogram pozvati za smještanje raznih događaja kroz red poruka.

Drugi glavni dio Windows programa je kreiranje i prikaz prozora.. *CreateWindow*, alocira memoriju za strukturu podataka prozora i vraća rukovatelj za referenciranje kasnije. Nakon njega se pozove *ShowWindow* koji obavi prikaz prozora na ekranu, i *UpdateWindow* koja dopuni prozor eventualno generiranim podprozorima.

U trećem dijelu dolazimo do glavne petlje programa, koji se sastoji od dobavljanja poruke, njene konverzije u drugu poruku, i prosljeđivanja poruke da Windows pozove proceduru za obradu. Ova procedura je ranije definisana kao polje u klasi prozora.

Pri slanju poruke se pozove potprogram *WndProc*, koji upravlja raznim porukama koje se mogu poslati do prozora. Prvi parametar je oznaka prozora koji se koristi. Drugi parametar je vrsta poruke. Treći i četvrti parametri mogu se koristiti za pružanje dodatnih informacija o poruci kroz dva broja *wParam* i *lParam* koji se

prosljede uz poruku (npr. oznaka koda tastera uz poruku tastature). Poruka WM_CREATE šalje se pri kreiranju prozora a WM_DESTROY pri zahtjevu za njegovo zatvaranje. Da Windows obavijesti program da mora ponovo iscrtati svoj prozor, on mu pošalje WM_PAINT poruku, koja također pruža informacije o tome koji dio prozora je prebrisan, u slučaju da je lakše generisati taj dio prozora umjesto cijelog. Postoji mnogo više vrsta poruka. Za ostale poruke treba pozvati *DefWindowProc* na kraju WNDPROC da se pozove podrazumijevana obrada.

```
#include <windows.h>
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam);
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
    LPSTR lpCmdLine, int nCmdShow) {
/* Dio 1: registracija prozorske klase */
    WNDCLASSEX wc;
    HWND hwnd;
    MSG Msg;
    wc.cbSize = sizeof(WNDCLASSEX);
    wc.style = 0; wc.lpfnWndProc = WndProc;
    wc.cbClsExtra = 0; wc.cbWndExtra = 0;
    wc.hInstance = hInstance;
    wc.hIcon = LoadIcon(NULL, IDI_APPLICATION);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);
    wc.lpszMenuName = NULL;
    wc.lpszClassName = "PROZOR";
    wc.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
    if(!RegisterClassEx(&wc)) {
        MessageBox(NULL, "Neuspjela registracija!", "Greska!",
            MB_ICONEXCLAMATION | MB_OK);
        return 0;
    }
/* Dio 2: kreiranje prozora */
    hwnd = CreateWindowEx( WS_EX_CLIENTEDGE, "PROZOR", "Pravougaonik",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 350, 200,
        NULL, NULL, hInstance, NULL);
    if(hwnd == NULL) {
        MessageBox(NULL, "Neuspjelo kreiranje!", "Greska!",
            MB_ICONEXCLAMATION | MB_OK);
        return 0;
    }
    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);
/* Dio 3: Petlja poruka */
    while(GetMessage(&Msg, NULL, 0, 0) > 0) {
        TranslateMessage(&Msg);
        DispatchMessage(&Msg);
    }
    return Msg.wParam;
}
```

```

/* Dio 4 prozorska procedura */
PAINTSTRUCT ps;
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM
lParam) {
    HDC hdc;
    HBRUSH NewBrush;
    switch(msg) {
        case WM_CLOSE:
            DestroyWindow(hwnd);
            break;
        case WM_PAINT:
            hdc=BeginPaint(hwnd,&ps);
            NewBrush = CreateSolidBrush(RGB(250, 25, 5));
            SelectObject(hdc, NewBrush);
            Rectangle(hdc,20, 20, 250, 125);
            EndPaint(hwnd,&ps);
            break;
        case WM_DESTROY:
            PostQuitMessage(0);
            break;
        default:
            return DefWindowProc(hwnd, msg, wParam, lParam);
    }
    return 0;
}

```

9.5. Rad sa ekranom, tastaturom i mišem u Unix sistemima

Za tekstualni ispis i očitavanje tastature se koriste standardne *read* i *write* funkcije, pri čemu je datotečni deskriptor STDIN i STDOUT. One se koriste kod PC ekrana i terminala.

U Unix sistemima se za grafiku koristi X sistem, razvijen na MIT-u. Program unutar X terminala koji prikuplja unos sa tastature ili miša i prihvata komande sa istog ili udaljenog računara naziva se X server. X server komunicira preko mreže sa X klijentom, programom koji radi na istom ili udaljenom računaru. Klijent šalje poruke tastature i miša i poziva komande X servera za crtanje na ekranu. Može izgledati čudno da se X server izvršava unutar terminala, a klijenti na udaljenom računaru, ali posao je X servera da prikazuje sliku, a aplikacija da ga pozivaju, pa je na taj način dobio ime server.

Također je moguće pokrenuti X Window System na vrhu UNIX ili nekog drugog operativnog sistema. Zapravo, mnogi UNIX sistemi pokreću X kao svoj standardni grafički sistem, čak i na samostalnoj mašini ili za pristup udaljenih računara preko Interneta. Protokol i rad sistema je identičan u svim slučajevima.

Za pristup osnovnim funkcijama koje pruža X server, klijent aplikacija koja poziva njegove usluge koristi biblioteku koja se zove *Xlib*. Neke funkcije ove biblioteke navedene su u sljedećoj tabeli.

Funkcija	Značenje
XOpenDisplay(display_name)	Otvara vezu sa serverom koji kontroliše ekran
XCloseDisplay(display)	Zatvara vezu sa X serverom
XCreateWindow(display, parent, x, y, width, height, border_width, depth, class, visual, valuemask, attributes)	Kreira podprozor navedenog prozora i vraća oznaku prozora.
XCreateSimpleWindow(display, parent, x, y, width, height, border_width, border, background)	Kreira podprozor nasljeđujući klasu, broj boja i izgled od roditeljskog prozora.
XRaiseWindow(display, window)	Postavlja prozor na vrh, tako da ga drugi prozori ne zaklanjaju
XLowerWindow(display, window)	Spušta prozor ispod ostalih prozora istog nivoa.
XSetForeground(display, gc, foreground)	Postavlja boju crtanja grafičkog konteksta.
XSetBackground(display, gc, background)	Postavlja boju pozadine grafičkog konteksta
XSetLineAttributes(display, gc, line_width, line_style, cap_style, join_style)	Postavlja debljinu i stil crtanja linija
XSetFillStyle(display, gc, fill_style)	Postavlja stil popunjavanja ikova
XSetFont(display, gc, font)	Postavlja trenutni oblik slova
XCLEARWindow(display, window)	Briše cijelo područje prozora
XDrawPoint(display, window, gc, x, y)	Crta tačku u području za crtanje
XDrawLine(display, window, gc, x1, y1, x2, y2)	Crta liniju između dvije tačke
XDrawRectangle(display, window, gc, x, y, width, height)	Crta okvir pravougaonika
XDrawArc(display, window, gc, x, y, width, height, angle1, angle2)	Crta eliptični luk nastao od elipse uokvirene pravougaonikom. Iscrtava se luk između dva navedena ugla u stepenima
XFillPolygon(display, window, gc, points, npoints, shape, mode)	Ispunjava poligon čije su koordinate rubova navedene u nizu points
XFillRectangle(display, window, gc, x, y, width, height)	Crta ispunjeni pravougaonik navedene relativne koordinate vrha i dimenzija
XLoadFont(display, name)	Učitava font
XDrawString(display, window, gc, x, y, string, length)	Ispisuje tekst
XSelectInput(display, window, event_mask)	Odabira događaje koji će se slati prozoru
XNextEvent(display, event_return)	Dohvati i ukloni sljedeći događaj iz reda čekanja
XPeekEvent(display, event_return)	Čita događaj iz reda čekanja bez uklanjanja
XGrabKeyboard(display, grab_window, owner_events, pointer_mode, keyboard_mode, time)	Dodjeljuje prozoru kontrolu nad tastaturom

Ovaj primjer pokazuje kako izgleda osnovni jednostavni X Window program koristeći Xlib. Njegov zadatak je da iscrtava nekoliko pravougaonika na ekranu..

```

#include <X11/Xutil.h>
#include <X11/Xos.h>
#include <X11/Xatom.h>
#include <X11/keysym.h>
Display *dis;
Window win;
XEvent report;
GC gc;
int main() {
    int i;
    dis = XOpenDisplay(NULL);
    win = XCreateSimpleWindow(dis, RootWindow(dis, 0),
        1, 1, 500, 500, 0, 0xFF0000L, 0xFF0000L);
    XMapWindow(dis, win);
    gc = XCreateGC(dis, win, 0, 0);
    XSetForeground(dis, gc, 0xFF0000L);
    XSelectInput(dis, win, ExposureMask | KeyPressMask | ButtonPressMask);
    while (1) {
        XNextEvent(dis, &report);
        switch (report.type) {
            case Expose:
                for ( i=1; i<200 ; i+=30)
                    XDrawRectangle(dis, win, gc, i,i, i,i);
                XFlush(dis);
                break;
            case KeyPress:
                if (XLookupKeysym(&report.xkey, 0) == XK_q) {
                    return 0;
                }
                break;
        }
    }
    return 0;
}

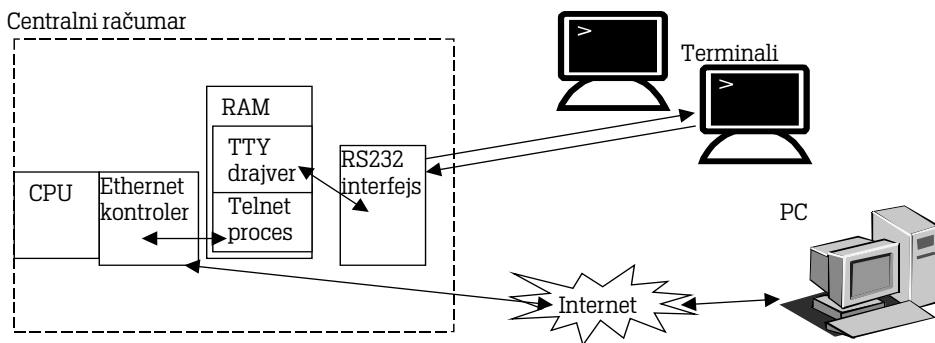
```

Funkcijom *XOpenDisplay* se uspostavlja veza s X serverom. Njen parametar je ime servera i broj ekrana, a ako je NULL, preuzima se iz varijable okruženja DISPLAY. Ime je u formatu ime_racunara.broj_servera.broj_ekrana, npr: video:0.1. Kao rezultat funkcije vrati se displej. Parametri funkcije *XCreateSimpleWindow*, koja kreira pravougaono područje, su redom: displej, oznaka roditeljskog prozora, x i y koordinata gornjeg lijevog ugla područja, širina i visina područja, boja ivice (RGB) i boja područja. Funkcijom *XMapWindow* prozor postaje vidljiv na ekranu. Parametri koji opisuju ponašanje likova (boja, tekstura) koji se iscrtavaju na ekranu opisuju se grafičkim kontekstom. On je uveden radi smanjenja saobraćaja između X servera i *Xlib* biblioteke, jer se čuva na serveru. Grafički kontekst se kreira s *XCreateGC*, čiji su parametri oznaka displeja, površine za crtanje (prozor), te konstante kojima se postavlja inicijalno stanje grafičkog konteksta. Mnoge funkcije kao parametre imaju displej, oznaku prozora, grafički kontekst ili kombinaciju ovih parametara. Tako npr. funkcijom *XSetForeground* se postavi boja, a s *XDrawRectangle* iscrtava pravougaonik unutar prozora. Slično Win32 API i X sistem je upravljan događajima. Funkcijom *XSelectInput* se određuje skup događaja na koje će prozor odgovarati. Tada ih je moguće raspoređivati u petlji koja poziva *XNextEvent* funkciju, koja kupi događaje

iz reda čekanja. Nakon očitavanja događaja odgovarajućom *switch/case* naredbom se on obradi prema njegovoj vrsti.

9.6. Znakovni terminali i tanki klijenti

Ranije su mnogo korišteni RS-232 terminali za komunikaciju s *mainframe* sistemima, bit po bit, preko serijske linije, između kojih se koristi eventualni telefonski modem. Terminal je uređaj vizualno sličan računarskom sistemu, kombinacija monitora, tastature, mikroprocesora i serijske komunikacije, ali njegov mikroprocesor ne izvršava operativni sistem niti korisnički program. Izvršavanje OS i programa obavlja centralni kompjuter, dok terminal samo brine o prijemu, slanju i prikazu podataka. Kompjuter i terminal su potpuno odvojeni, ali terminal ima sposobnost prikaza teksta koji mu se pošalje. Pojeftinjenje personalnih računara je učinilo da se i oni mogu koristiti kao terminali. To je najprije išlo također preko RS-232 interfejsa, a nakon toga su računari opremljeni mrežnim karticama i protokolima, što je komunikaciju učinilo daleko bržom (Slika 90).



Slika 90 Pristup preko znakovnog terminala

Jedan od prvih takvih mrežnih protokola je *telnet*. Protokol je izrazito jednostavan: primljeni sadržaj mrežnog paketa se prikazuje na ekranu (uz interpretaciju ANSI sekvenci ako je potrebno), a otkucani tekst na tastaturi se stavlja u mrežni paket i šalje na mrežu. Glavna prednost tog protokola je što omogućava pristup udaljenom računararu i najslabijim klijentskim računarima i što ima veoma mali mrežni saobraćaj. Ozbiljne mane su što ne podržava grafiku i što je tekst moguće lako špijunirati u mreži. Zato su nastali grafički mrežni protokoli kao što su X11 (za Unix sisteme) i Windows terminal server (za Windows sisteme), i kriptografisana verzija *telnet* protokola koja se zove *ssh*.

Pristup računararu preko terminala je skoro nestao, i zamijenjen mrežom personalnih računara, koji mogu ponekad pristupati serverskim računarima dijeleći podatke sa njima ili izvršavajući njegove udaljene programe. To je otvorilo mnoštvo novih mogućnosti, jer personalni računar može obavljati obradu podataka i kada centralni računar nije dostupan, korisnik može štampati dokumente direktno na svom stolu, a igre i multimedijalne aplikacije dobile su svoj zamah tek kada su se računari oslobodili od spore komunikacije centralnog računara i terminala. Ali, dovelo je i do problema sa održavanjem računarskih sistema, na primjer isti programi se sada moraju instalirati na mnogo računara umjesto samo na centralni, rezervne kopije podataka, programa i operativnih sistema nisu garantovano čuvane

kao što bi bile da se o njima brine samo na jednom mjestu, a uklanjanje računarskih virusa postaje daleko teže.

U međuvremenu, računarske mreže su postale mnogo brže i globalnije. Jedan program, *Web browser* postaje centar upotrebe računara i sve više programa se izvršava unutar njega. Tako nastaju **tanki klijenti**, računari koji se ponašaju slično terminalima. Oni mogu biti računari bez diskova i podizanje njihovih operativnih sistema se obavlja preko mreže sa servera ili imati disk na kome se nalazi redukovani operativni sistem (*Google Chromium, Windows Embedded*) sa instaliranim aplikacijama za terminalski pristup ili *web browser*-om koji služe kao osnovna školjka operativnog sistema. Još bliži konceptu klasičnog terminala su **zero klijenti**, koji nemaju operativni sistem, nego koriste specijalizovani procesor dizajniran za jedan od tri protokola za virtualni desktop PCoIP, HDX ili RemoteFX. Posao dekodiranja i prikaza se obavlja u hardveru, pa su ovi klijenti efikasniji od softverskih tankih klijenata, podižu se za nekoliko sekundi i imuni su na viruse, a ažuriranje njihovog softvera se dešava izuzetno rijetko.

9.7.Štampači

Dvije osnovne vrste štampača (eng. *printer*) su udarni štampači i beskontaktni štampači.

Udarni štampači su najstarija tehnologija štampe koja je još uvijek u aktivnoj proizvodnji. Udarni štampači su najfunkcionalniji u specijaliziranim okruženjima gdje je niska cijena štampe od suštinskog značaja. Tri najčešća oblika su matrični, lepezni i linijski štampači.

Tehnologija iza **matričnih štampača** je vrlo jednostavna. U radu se pritisne bubanj (gumeni cilindar) i povremeno povuče prema naprijed kako ispis napreduje. U elektro-magnetskom pogonu glava štampača se kreće preko papira i udara iglicama vrpcu štampača koja se nalazi između papira i glave štampača. Glava štampača na traci tiska tačkice tinte na papiru koje čine ljudski - čitljive znakove. Matrični štampači se razlikuju u rezoluciji štampe i ukupne kvalitete i imaju obično 9 ili 24 – iglice na glavi štampača. Budući da glava štampača mora udariti površinu papira sa dovoljno snage za prijenos tinte sa trakom na stranici, oni su idealni za okruženja koja moraju proizvesti karbon kopije kroz upotrebu posebnog papira.

Štampači s lepezom su slični pisaćim mašinama. Glava štampača sastoji se od metalnog ili plastičnog točka isječenog po laticama. Svaka latica ima određen broj slova (velikih i malih slova), brojeva ili interpunkcijskih znakova na njoj. Kada latica pogodi vrpcu štampača ovo rezultira ispisom tinte na papir. Ovi štampači su glasni i spori. Oni ne mogu ispisati grafiku, a ne mogu promijeniti fontove osim ako je točak štampača fizički zamijenjen. Pojavom laserskih štampača, ovi štampači se obično ne koriste u modernim računarskim okruženjima.

Linijski štampači imaju mehanizam koji omogućava da se istovremeno štampa više znakova na istoj liniji. Mehanizam može koristiti veliki rotirajući bubanj ili papir u lancu petlje. Kada se bubanj ili lanac rotira preko površini papira, elektromehanički čekići iza papira gurnu papir (zajedno sa trakom) na površinu bubnja ili lanca, iscrtavajući lik na površini bubnja ili lanca. Zbog prirode mehanizma ovi štampači

su vrlo brzi ali vrlo bučni i lošijeg kvaliteta otiska. Glavna primjena im je u masovnom štampanju računa.



Slika 91 Matrični, lepezni i linijski štampač

Kod **bekontaktnih štampača**, s papirom ne dolaze u dodir metalni dijelovi štampača nego samo boja. Ovakvi štampači su znatno tiši od udarnih i omogućuju bolji kvalitet štampe. Njihovi predstavnici su termalni, *inkjet*, laserski, sa sublimacijom boje, termalnog voska i čvrste tinte.

Termalni štampači koriste specijalni papir osjetljiv na temperaturu ili električni naboj. Glava štampača na pojedinoj tački papira zagrije to mjesto grijačem ili električnom varnicom, nakon čega papir na tom mjestu blago izgori i pojave se tamne tačke. Danas se najčešće koriste u registar kasama za štampanje računa. Tehnologija štampača je jeftina, ali odštampani tekst vremenom nestaje sam od sebe.

Inkjet štampač koristi jednu od najpopularnijih štampe tehnologije danas. Relativno niske cijene i višenamjenski ispis čine da su *inkjet* štampači dobar izbor za male firme i kućne urede. *Inkjet* štampači koriste boje na bazi vode koje se brzo suše i glavu štampača s nizom malih mlaznica koja rasprskava tintu na površini papira. *Inkjet* štampači mogu imati jednu mlaznicu (crno-bijeli ispis) ili četiri mlaznice koje predstavljaju boje cijan, magenta, žuta i crna. Ova kombinacija boja (naziva CMYK) omogućava ispis slika gotovo iste kvalitete kao razvoj u foto laboratoriju, kada se koristi papir određene vrste.

Laserski štampači su poznati po velikim mogućnostima izlaza i niskim troškovima po stranici. Laserski štampači dijele mnogo istih tehnologija kao fotokopirni aparat. Valjci povuku list papira iz ladice za papir i kroz valjak, što daje papiru elektrostatički naboj. U isto vrijeme ispisni bubanj ima suprotno naelektrisanje. Površina bubnja se zatim skenira laserom, isprazni površina bubnja i ostave samo one tačke koje odgovaraju željenom tekstu i slikama. Ovaj naboj se zatim koristi da prisili toner da se zadrži na površini bubnja. Papir i bubanj se zatim dovode u kontakt; njihovo različito naelektrisanje uzrokuje da se toner tada zadrži za papir. Konačno, papir putuje između valjaka, koji griju papir i tope toner, spajajući ga na površini papira.

Laserski štampač u boji temelji se na tradicionalnom laserskom monohromatskom štampanju, ali koristi dodatne komponente za stvaranje slike u

boji i dokumenata. Umjesto korištenja samo crnog tonera, laserski štampači u boji koriste CMYK toner kombinaciju. Bubanj za ispis ili rotira svake boje i postavlja toner jednu po jednu boju, ili postavi sve četiri boje, a zatim provuče papir kroz bubanj, prenošenjem kompletne slike na papir. Kolor laserski štampači dodaju grijač ulja za dodatno vezanje boje na papir što može dati različite stepene sjaja na gotovoj slici.



Slika 92 Termalni, ink-jet i laserski štampač

Za postizanje visokog kvaliteta u boji rjeđe se koriste još neke tehnologije. **Štampači termalnog voska** imaju pojas za pogon CMYK trake veličine lista papira i specijalno - premazani papir ili folije. Glava štampača sadrži grijače kojim se topi vosak boja na papir, dok on ide kroz štampač. **Štampači s sublimacijom boje** su slični štampačima termalnog voska, osim što koriste film boje difuzne plastike umjesto obojenog voska. Glava štampača zagrijava film u boji i isparava sliku na posebno premazani papir. **Štampači čvrste boje** su cijenjeni zbog svoje sposobnosti za ispis na raznim vrstama papira, što se koristi za izradu prototipova ambalaža. Ovi štampači koriste tvrdi tinte koja se istopi i prska kroz male mlaznice na glavama štampača. Papir se zatim šalje prema valjku grijaču što dodatno tjera tintu na papir.

Današnji štampači su u mogućnosti da obrađuju složene dokumente sa ugrađenim slikama, grafikovima i formulama na nekoliko jezika, sve na jednoj stranici. Zato su razvijeni jezici za opis stranice (*Page Description Language* - PDL) - specijalizirani jezik za formatiranje dokumenta specijalno napravljen za komunikaciju računara sa štampačem. Tokom godina, proizvođači štampača su razvili vlastite jezike za opisivanje formata dokumenata. Međutim, takvi jezici su primjenjivi samo na štampače koji su proizvođači stvorili sami. Ako bi, na primjer, poslali datoteku spremnu za štampu pripremljenu u PDL Epson štampača u profesionalnu štampariju, nema garancije da će datoteke biti kompatibilne sa štamparskom mašinom. PDL rade na istom principu kao i programski jezici. Kada je dokument spreman za ispis, PC ili radna stanicu uzima slike, tipografske informacije i raspored dokumenta, i koristi ih kao objekte koje šalje štampaču za obradu. Štampač potom prevodi ove objekte u rastere, niz skeniranih linija koje čine sliku dokumenta (tzv. *Raster Image Processing* ili RIP), i ispisuje izlaz na stranicu kao jednu sliku, zajedno sa tekstom i uključenom grafikom.

U doba matričnih štampača, priličnu popularnost je stekao Epsonov štampač FX-80, pa su gotovo svi štampači matričnog tipa drugih proizvođača bili kompatibilni

sa njim. Tako je nastao prvi popularni PDL, koji se zove ESC/P. Riječ je o ASCII ispisu teksta uz pomoćne kodove za definiranje vlastitih slova, ispis kosim slovima, podebljano ili ispis grafike. Pomoćni kodovi se navode iza ASCII koda 27 (heksadekadno 1b), koji se zove ESC. Matrični štampači pomjeraju valjak red po red, pa je i ovaj jezik tome prilagođen. Sljedeći primjer pokazuje slanje teksta na Unix sistemima sa priključenim Epson štampačem na paralelnom portu uz upotrebu kodova ESC 4 (kosa slova) i ESC 5 (kraj kosih slova).

```
#include <stdio.h>
int main()
{
    FILE *lp;
    char * poruka ="Ovaj tekst\x1b4prikazan koso\x1b5 pa normalno.";
    lp = fopen("/dev/lp0", "w");
    fprintf(lp,poruka);
    fclose(lp);
    return 0;
}
```

Laserski štampači nisu linijski orijentirani, nego pripremaju cijelu stranu. Pored toga njihove grafičke mogućnosti su znatno veće, pa je bio potreban moćniji PDL. U firmi Adobe, je razvijen PDL zvani PostScript, jezik za opisivanje i oblikovanje teksta i slika koje može obraditi štampač. Neobična osobina ovog jezika je što se za svaku njegovu naredbu prvo navode njeni parametri, pa onda sama naredba. Sljedeći primjer predstavlja PostScript program koji se šalje štampaču da nacrtatrougao na stranici sa tekstom unutar njega.

```
newpath
100 200 moveto
200 250 lineto
100 300 lineto
closepath stroke
/Times-Roman findfont
12 scalefont
setfont
100 250 moveto
(Tekst u trouglu) show
showpage
```

Mogućnosti Postscript-a su velike, ali u cilju smanjenja cijene štampača mnogi štampači imaju jednostavnije jezike za opis stranice. Hewlett-Packard je razvio *Printer Command Language* (ili PCL) za upotrebu u svojim laserskim i inkjet štampačima. Ovaj jezik, kao i ESC/P umjesto komandi koristi specijalne ESC kodove. Sljedeća sekvenca PCL kodova crta pravougaonik, gdje <esc> predstavlja bajt s ASCII vrijednošću 27.

```
<esc>&u300D<esc>*t300R<esc>*p300x300Y
<esc>*r3U<esc>*v2S<esc>*c300a300b5P
```

Porastom snage centralnog procesora i željom za smanjenjem cijena laserskih štampača pojavili su se *host* bazirani štampači. Njihov jezik za opis strane je izrazito jednostavan: bitmapa sastavljena od bitova koji predstavljaju tačke na papiru. Priprema ove bitmape obavlja se na centralnom računaru.

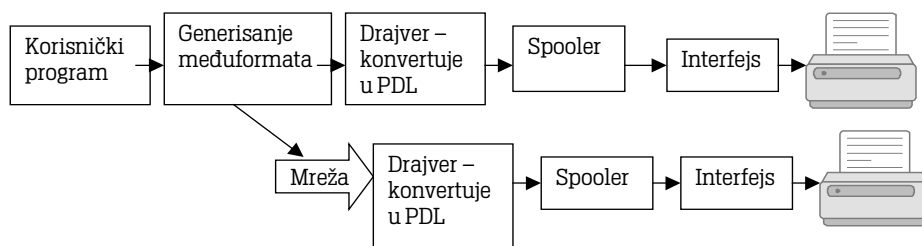
U zavisnosti od organizacionih potreba, može biti nepotrebno dodijeliti jedan štampač za svakog člana organizacije. Dok je lokalni štampač priključen preko

paralelnog ili USB kabel za svaku radnu stanicu idealno rješenje za korisnika, to je obično nije ekonomski isplativo. Proizvođači štampača su za ovo razvili grupne štampače. Ove mašine su obično izdržljive, brze, i imaju dug životni vijek potrošnog materijala. Grupni štampači obično su vezani za server štampe, samostalni uređaj koji se bavi poslovima štampe i preusmjerava izlaz na odgovarajući štampač kada je on dostupan.

9.8.Proces štampanja u operativnim sistemima

Raniji operativni sistemi nisu mnogo brinuli o raznovrsnim PDL koje imaju različiti štampači. Štampač se nije razlikovao od ostalih perifernih uređaja, pa je drajver bio u jezgu i izuzetno jednostavan. U principu sve što je drajver imao posla bilo je da provjeri da li je paralelni priključak za štampač spreman i, ako jeste, da na njega pošalje naredni bajt podataka. Brigu o različitim formatima jezika štampača vodili su aplikativni programi. Tako na primjer, svaki program za obradu teksta ili crtanje na MS DOS ili ranijim Unix sistemima se isporučivao sa desetinama drajvera za štampače, a i dalje je postojao rizik da neki od štampača nije podržan konkretnim programom.

Na modernim operativnim sistemima drajveri za štampače obično nisu u jezgu, ali se koriste zajednički drajveri za sve programe. Aplikativni programi generišu izlaz za štampu u jednom neutralnom formatu (ASCII, Windows Meta File ili PostScript) za trenutno aktivni štampač. Pripremljena datoteka za štampu se snima na disk i prosljeđuje drajveru štampača, ili se preko mrežnog interfejsa prosljeđuje udaljenom računaru i njegovom drajveru. Drajver štampača konvertuje iz neutralnog formata u PDL koga razumije štampač. Nakon toga se pripremljena datoteka prosljeđuje *spooler*-u koji upravlja štampačima prikopčanim na dati računar preko USB, paralelnog priključka ili štampačima koji imaju vlastitu mrežnu karticu. *Spooler* čuva kompletne dokumente namijenjene štampaču onim redom kako su pristizali i na kraju ih prosljeđuje prema štampaču nakon što on završi ranije dokumente.



Slika 93 Proces štampanja

9.9.Štampanje u Windows

Na Windows gotovo sve aplikacije koriste GDI, koji se može koristiti i za štampače, tako da je generiranje izgleda stranice na štampaču jednostavno. Kontekst uređaja štampača se dobije API pozivom *StartDoc* i koriste se pozivi za crtanje, poput *TextOut* i *BitBlt*. Zapamćene GDI komande se čuvaju u metadatoteci. Nakon funkcije *EndDoc*, privremena metadatoteka se dalje konvertuje u jezik štampača. Dobiveni podaci u jeziku štampača se pošalju programu *spooler*-u (*spoolsv.exe*). API pozivi za rad sa štampačem dati su u sljedećoj tabeli.

Funkcija	Opis
AbortDoc(hdc)	Prekida pripremu štampe
DeviceCapabilities(pDevice, pPort, fwCapability, pOutput, *pDevMode)	Vraća mogućnosti drajvera za štampač
EndDoc(hdc)	Završava pripremu štampe
EndPage(hdc)	Označava kraj stranice
StartDoc(hdc, *lpdi)	Počinje pripremu štampe
StartPage(hDC)	Označava početak strane
EnumPrinters(Flags, Name, Level, pPrinterEnum, cbBuf, pcbNeeded, pcReturned)	Prolazi kroz listu štampača i servera
GetDefaultPrinter(pszBuffer, pcchBuffer)	Ime podrazumijevanog štampača
GetPrinter(hPrinter, Level, pPrinter, cbBuf, pcbNeeded)	Vraća informaciju o navedenom štampaču
PrinterProperties(hWnd, hPrinter)	Prikazuje svojstva štampača
SetPrinter(hPrinter, Level, pPrinter, Command)	Postavlja podatke za navedeni štampač ili njegovo stanje

9.10.Linux/Unix štampanje

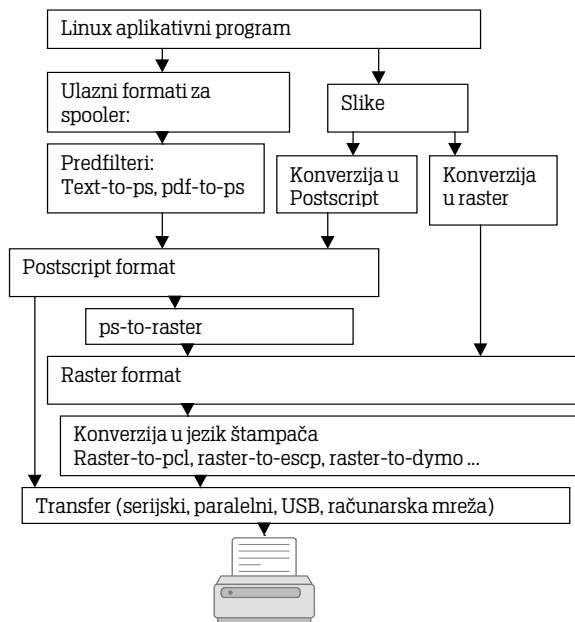
Unix arhitektura je starija od grafički orijentiranih štampača. Originalno, Unix nije imao *spooler*, nego se datoteka direktno slala na uređaj štampača naredbom poput sljedeće.

```
cat file > /dev/lp1
```

U kasnijim verzijama je uveden *lpd* printer *spooler*, ali je imao određenih problema. Većina Unix alata za štampu je prilagođena linijskim štampačima, a ne pametnim štampačima. To znači da nema standardnog interfejsa za programski izlaz. Mora se kreirati privremena datoteka i štampati iz linijskog režima

Situacija se promijenila uvođenjem CUPS (*Common Unix Printing System*). Arhitektura CUPS sistema prikazana je na slici Slika 94. CUPS je više od *spool* programa. To je kompletan sistem za upravljanje štampačima. Poznaje novi *Internet Printing Protocol* (IPP). Može se konfigurisati web, GUI i komandnim sučeljem. Opremljen je programima filtrima koji prevode različite tekstualne i grafičke formate u PostScript jezik, koji se može direktno slati takvim štampačima. Ako štampač nije PostScript tipa, programima CUPS-raster se konvertuje PostScript u format prilagođen štampaču. Raster formati predstavljaju prikaz stranice na papiru kodirajući pojedinačne tačke posebno., za razliku od PostCript formata koji kodira geometrijske likove i tekst. Kada je dokument u potpunosti prilagođen jeziku štampača, on se pozadinskim programima šalje kroz odgovarajući fizički interfejs ili mrežni protokol prema štampaču ili udaljenom serveru. Kada se koristi CUPS iz komandne linije, pripremljena datoteka se zatim štampa programom *lpr*. Npr, datoteka na podrazumijevani štampač, šalje se komandom:

```
lpr test.pdf
```



Slika 94 CUPS sistem štampanja

U CUPS sistemu se kao neutralni format koristi PostScript, dok Windows koristi EMF (*metafile*).

9.11. Dodirni ekrani

U novije vrijeme, sve popularniji ulazno/izlazni uređaj je dodirni ekran. Dodirni ekran je računarski monitor koji reaguje na dodir olovkom ili prstom. Koriste se četiri tipa principa rezistivni, kapacitivni, infracrveni i površinski akustični talas (SAW)

Kod **rezistivnog** dodirnog ekrana, ekran je napravljen od dva sloja provodnog materijala. Jedan sloj ima vertikalne, drugi horizontalne linije. Kada se pritisne gornji sloj on dođe u kontakt s drugim koji dopusti tok struji. Odgovarajuće horizontalne i vertikalne linije određuju poziciju na ekranu koja je dodirnuta.

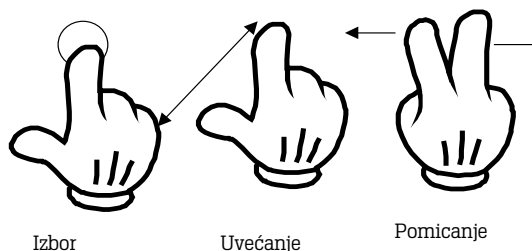
Kapacitivni dodirni ekran je napravljen od laminata preko cijelog staklenog ekrana. Laminat provodi struju u svim pravcima, a vrlo mala struja ide iz sva četiri ugla. Kada se ekran dodirne, struja teče u prst ili olovku. Lokacija dodira se odredi poređenjem koliko je jak tok elektriciteta iz svakog ugla

Infracrveni dodirni ekran je ekran s unakrsnim horizontalnim i vertikalnim zracima IC svjetla. Senzori suprotnih strana ekrana detektuju zraku. Kada korisnik prekine zraku dodirom ekrana, može se odrediti lokacija prekida.

Ekran sa **akustičkim površinskim valom** (SAW) emituje visoko frekventne zvučne talase horizontalno i vertikalno. Kada prst dodirne površinu, senzor prepozna prekid i odredi lokaciju dodira.

Najčešći način programiranja ulaza s ovim uređajima je da se za njih napišu drajveri slični drajveru za miša, pa da se uređaj ponaša kao miš.

Tableti su mali računari bez tastature kojima je dodirni ekran glavni ulazni uređaj. Jedan od načina da se prevaziđe nedostatak tastature su gesture. To su pokreti prstima preko dodirnog ekrana s posebnim značenjem (Slika 95).

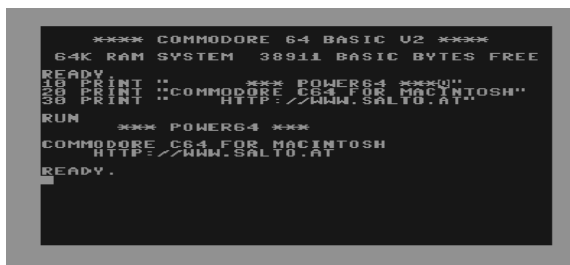


Slika 95 Gesture

9.12. Tekstualni korisnički interfejs

Obično izvan jezgra se izvršava aplikacija namijenjena korisnicima i to je ono što većina ljudi vidi kada se govori o operativnom sistemu. Ta aplikacija omogućava korisnicima da unose i pokreću programe.

Prvi kućni računari, kao i današnji programabilni kalkulatori, nisu imali operativne sisteme u smislu u kome se govori u ovoj knjizi. Umjesto toga koristi se interpreter za programski jezik u ROM-u (kao što su BASIC, Forth). Na slici je prikazan izgled ekrana starog računara Commodore 64, kod koga BASIC interpreter ima ulogu komandne školjke (Slika 96).



Slika 96 Izgled ekrana računara C64 sa BASIC-om

Prednost pristupa je u izuzetno brzom pokretanju računara, i mogućnosti da se odmah piše korisnički program, ali je mana u maloj fleksibilnosti.

Druga varijanta tekstualnog korisničkog interfejsa je primijenjena u operativnim sistemima kao što su CP/M, DOS, Unix ili VMS. Umjesto generalnog programskog jezika, kao što je BASIC, računar je opremljen specijaliziranim jezikom namijenjenim za pokretanje drugih aplikacija i manipulaciju datotekama. Ovakvi jezici se zovu **skriptni jezici**, a programi koji omogućavaju izvršavanje komandi i programa u takvim jezicima su **komandni interpreteri**. Većina komandnih interpretera radi tako što ispiše kratku poruku koja se zove prompt. Korisnik tada otkuca komandu oblika kao:

```
Naredba parametar1 parametar2 parametar3 ...
```

Naredbe su obično jedna od tri klase. **Interne naredbe** prepoznaje i obrađuje sam interpreter komandne linije i ne ovise o bilo kojoj vanjskoj izvršnoj datoteci. **Uključene naredbe** su izdvojene izvršne datoteke koja se općenito smatraju dijelom operativnog sistema i uvijek su uključene u OS. **Vanjske naredbe** su izvršne datoteke koje nisu dio osnovnog OS, ali su dodane naknadno od strane korisnika.

Parametri parametar1... parametarN se prosleđuju naredbi, odnosno eksternom programu. Parametri mogu biti argumenti (npr. imena datoteka koje se obrađuju naredbom) ili opcije (dodatno definišu naredbu). Ispred opcija se stavlja neki standardizovani simbol poput crtice ili kose crte.

Pojednostavljeni princip komandne školjke dat je sljedećim programom. Ova školjka ima samo jednu internu komandu, *exit*, ali kada se unese bilo koja druga komanda, pokrene se *fork* poziv koji kreira novi proces i zatim se sa *execvp* učitava pokrenuti program u njegov adresni prostor, dok se školjka suspenduje do završetka komande.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <string.h>
#define BUFFER 1024
int main(void) {
    char line[BUFFER];
    while(1) {
        printf("$ ");
        if(!fgets(line, BUFFER, stdin)) break;
        char *p = strchr(line, '\n');
        if (p) *p = 0;
        if(strcmp(line, "exit")==0) break;
        char *args[] = {line, (char*)0};
        int pid= fork();
        if(pid==0) {
            execvp(line, args);
            exit(1);
        } else {
            wait(NULL);
        }
    }
    return 0;
}
```

9.13. Tekstualni korisnički interfejs u MS DOS i Windows

Već od prve verzije MSDOS, pa sve do današnjeg Windows 10, postoji komandni interpreter koji se zove COMMAND.COM ili kasnije CMD.EXE. U MS DOS, koji gotovo da nema upravljanje procesima, COMMAND.COM se naizmjenično učitava s aplikativnim programom. U DOS memoriji se čuva u dva dijela. Tranzijentni dio interpretira komande i briše se iz memorije kada se učitava pozvani program. Rezidentni dio ostaje u memoriji i nakon završetka pozvanog programa ponovo učitava

tranzijentni dio COMMAND.COM sa diska Kada korisnik otkuca komandu, COMMAND.COM najprije provjeri ima li je među internim komandama i ako ima izvrši je, a ako nema, traži program sa istim imenom kao komanda u tekućem direktoriju ili direktorijima navedenim komandom PATH, a koji imaju ekstenzije .COM, .EXE ili .BAT.

Naredbe je moguće spajati u programe koji se zovu *batch* datoteke. Pored naredbi, *batch* datoteke u COMMAND.COM mogu imati četiri vrste varijabli :

- ERRORLEVEL - sadrži povratni kôd posljednjeg pokrenutog programa koji postavlja vrijednost (cijeli broj između 0 i 255). Većina programa imaju određeni konvenciju za njihove povratne kodove
- Varijable okruženja imaju formu %PROMENLJIVA%, a postavljaju se SET naredbom.
- Komandni parametri imaju oblik %0, %1,...%9. U početku sadrže naziv komande i prvih devet parametara komandne linije. Parametri desno od devetog se mogu dosegnuti pomoću SHIFT naredbe
- "for" varijable – se koriste u petljama i imaju format %%a kada se radi u *batch* datotekama

Postoji mogućnost preusmjeravanja ulaza ili izlaza komande u datoteku ili drugu komandu, sintaksom iz sljedeće tabele na slici Slika 97.

naredba < datoteka	Preusmjeravanje ulaza iz datoteke ili uređaja
naredba > datoteka	Preusmjeravanje izlaza, prebriše ciljnu datoteku ako postoji.
naredba >> datoteka	Preusmjeravanje izlaza, dodavanjem na ciljnu datoteku
naredba1 naredba2	Proslijedi izlaz iz naredbe1 ulazu u naredba2
naredba1 & naredba2	Komande razdvojene i izvršavaju se u sekvenci

Slika 97 Redirekcija u COMMAND.COM

Od Windows NT, COMMAND.COM je ostavljen za izvršavanje 16 bitnih DOS aplikacija, a uveden je **CMD.COM**. On je kompatibilan sa COMMAND.COM, ali ima više memorije na raspolaganju, naredbe SETLOCAL/ENDLOCAL za ograničenje područja varijabli okruženja, interne CALL i GOTO labele, izraze u SET i IF, poboljšanu FOR naredbu, historiju naredbi, dovršavanje naredbi uz pomoć TAB, i naredbe za pamćenje direktorija PUSH, POPD. Interne komande su date u tabeli na slici Slika 98. Komande koje postoje i u MS DOS interpreteru komandi COMMAND.COM označene su podebljano.

ASSOC Veza tipova datoteka i programa	BREAK Uključuje CTRL C taster
CALL Potprogram u <i>batch</i> programima	CD Mijenja trenutni direktorij
CHDIR Mijenja trenutni direktorij	CLS Briše ekran
COLOR Postavlja boju tekstualne konzole	COPY Kopira datoteku
DEL Briše datoteku	DIR Spisak svih datoteka u direktoriju
DATE Prikazuje ili mijenja trenutni datum	ECHO Poruka na ekranu
ELSE Dio IF ako uslov nije ispunjen	ENDLOCAL Izmjene varijabli globalne
ERASE Briše datoteku	EXIT Izlazak iz konzole
FOR Ponavlja komandu za više datoteka	FTYPE Veza ekstenzije s programima
GOTO Skok unutar BAT programa	IF Uslovna naredba
MD Kreira direktorij	MKDIR Kreira direktorij
MKLINK Kreira simbolički link na datoteku	MOVE Premješta datoteku
PATH Direktoriji gdje su izvršni programi	PAUSE Čeka na pritisak taster
POPD Vraća se u direktorij prije PUSH	PROMPT Tekst prije unosa komandi
PUSH Zapamti direktorij i pređi u njega	RD Obriši direktorij
REN Preimenuj datoteku	RENAME Preimenuj datoteku
REM Komentar, ignoriše se	RMDIR Obriši direktorij
SET Postavlja varijable okruženja	SETLOCAL Izmjena varijabli važi u BAT
SHIFT Pomjera parametre BAT programa	START Pokrene program prema datoteci
TIME Prikazuje/ mijenja trenutno vrijeme	TITLE Mijenja natpis konzole
TYPE Prikazuje tekst datoteke	VER Prikazuje verziju Windows ili DOS
VERIFY Provjera datoteke nakon kopiranja	VOL Prikazuje ime i serijski broj diska

Slika 98 Interne komande cmd.exe

Većina internih komandi ima parametre obično međusobno razdvojene razmacima. Na primjer, da se pređe u direktorij C:\users, prikaže trenutno vrijeme i zatim kopira datoteka primjer.dat u direktorij c:\tmp navode se komande

```
CD C:\USERS
TIME /T
COPY PRIMJER.DAT C:\TMP
```

Eksternih komandi ima oko 200, koje su uglavnom smještene u direktoriju C:\windows\system32, ali mnoge su bez efekta jer su preostale od MS DOS, uklonjene u novijim verzijama Windows ili jako rijetko potrebne. Neke eksterne komande su navedene su u sljedećoj tabeli.

ARP Status vezivanja IP adrese sa fizičkom	AT Izvršavanje komandi u zadano vrijeme
ATTRIB Promjena atributa datoteka	BCDEDIT Konfiguracija podizanja sistema
CACLS Promjena listi pristupa datotekama	CHCP Izbor kodne stranice za razne jezike
CHKDSK Provjera datotečnog sistema	CHKNTFS Provjera NTFS sistema
CHOICE Pruža izbor između opcija	CIPHER Šifruje i dešifruje datoteke
CLIP Preusmjerava komande u <i>clipboard</i>	CMD Nova kopija komandnog interpretera
COMP Poredi dvije datoteke	COMPACT Kompresuje NTFS datoteke
CONVERT Konvertuje FAT particiju u NTFS	DOSKEY Definiše kratice komandi
DISKPART Konfiguriše particije diska	DISKRAID Kreira RAID diskove
DRIVERQUERY Lista instaliranih drajvera	EXPAND Ekstrakcija datoteka iz CAB arhive
FC Poredi dvije datoteke i prikazuje razlike	FIND Pretraživanje teksta u datoteci
FINDSTR Traženje teksta u datoteci	FORFILES Prosljeđuje listu datoteka
FORMAT Formatira disk ili disketu	FSUTIL Popravak datotečnih sistema
GETMAC Čita adresu mrežne kartice	GPRESULT Podaci o grupnim pravima
HELP Kratka pomoć o komandama	ICACLS Podsi prava nad datotekama
IPCONFIG Konfiguracija mrežne kartice	LABEL Postavlja ime disk particije
MAKECAB Kompresija u CAB arhive	MODE Broj znakova u konzolnom ekranu
MORE Pauza nakon ispisa cijele konzole	NET Razne mrežne komande
OPENFILES Lista otvorenih datoteka	PING Test dostupnosti udaljenog računara
RECOVER Obnova oštećene datoteke	REG Promjena ključa <i>registry</i> -ja
REPLACE Zamjena datoteka novom	ROBOCOPY Napredno kopiranje datoteka
RUNDLL32 Izvršavanje potprograma iz DLL	SC Konfiguracija servisa
SCHTASKS Zakazivanje izvršenja procesa	SETX Promjena varijabli okruženja
SHUTDOWN Gašenje računara	SORT Sortiranje tekstualne datoteke
SUBST Dodjela diska direktoriju	SYSTEMINFO
TASKKILL Prekidanje programa u izvršenju	TASKLIST Lista programa u izvršenju
TIMEOUT Čeka pritisak tastera	TREE Prikaz direktorija u formi stabla
WHERE Traži datoteku unutar PATH	WMIC Konzola za administraciju
WSCRIPT Start VBScript i JScript programa	XCOPY Kopiranje datoteka i poddirektorija

Slika 99 Eksterne komande u Windows, komandna linija

POWERSHELL je novija školjka komandne linije, uveden od Windows 7 i Windows Server 2008. Njegove komande se zovu Cmdlet i ima znatno moćniji skriptni jezik od CMD. Svaki *cmdlet* ima *alias* koji liči na DOS ili Unix komande. Npr. *alias*-i za Remove-Item su *ri*, *del*, *rmdir*, *rd*, *rm*

9.14.Unix Shell

Pod Unix sistemima nastalo je više različitih komandnih jezika. Izvršavaju se u korisničkom prostoru memorije. Oni se mogu podijeliti u dvije kategorije. Prva kategorija se odlikuje jezikom sličnim Algol 68, i ovu grupaciju sačinjavaju Bourne shell (**sh** – najraniji), Almquist shell (**ash** - malo zahtjevan), Bourne-Again shell (**bash** – najpopularniji), Debian Almquist shell (**dash** – verzija ash za distribuciju Debian), Korn shell (**ksh** čiji je autor David Korn), Z shell (**zsh** – najkompletniji od svih), **busybox** – (u jednom programu su Almquist Shell i mali programi). Drugu kategoriju predstavljaju C shell (**cs**) i TENEX C shell (**tcsh**). Njihov skriptni jezik podsjeća na C.

Grafička okruženja su danas dosta napredovala u odnosu na rane Linux dane, ali je komandna linija i dalje popularna na Unix baziranim sistemima. Razlog je u velikoj fleksibilnosti, univerzalnosti i brzini komandnog okruženja. Aktivnosti iz

komandne linije se mogu automatizovati, tako da se može na primjer pripremiti skup komandi koje će kopirati veliki broj datoteka i za to vrijeme raditi nešto drugo. Ovo jeste teže naučiti od kopiranja mišem, ali ako se iste datoteke kopiraju svaki dan, ovo će uštedjeti mnogo vremena kasnije.

Naredbe se unose u jednu aktivnu liniju i pokreću pritiskom na taster Enter. Kad se naredba izvrši i ispiše se rezultat (ako naredba daje kakav ispis), pojavi se nova aktivna linija, a stara se pomakla prema gore. Krivo napisana naredba se može korigovati s tasterima Delete i Backspace.

Neke školjke imaju dodatne tastere. Taster Tab služi za dopunjavanje naredbe. Često je dovoljno otkucati prvih nekoliko slova naredbe ili njenog parametra, a zatim pritisnuti taster Tab a školjka će pokušati dopuniti komandu. Tasteri sa strelicama omogućavaju povratak na već unesene naredbe.

9.15.Osnovne Unix/Linux naredbe komandne linije

Sada će biti navedene najvažnije naredbe iz komandne linije na Linux i Unix sistemima.

Naredba *pwd* ispisuje kako se zove direktorij u kome se trenutno korisnik nalazi.

```
pwd
```

Korisničko ime pod ovom sesijom saznaje se naredbom *whoami*.

```
whoami
```

Ime računara se dobiva naredbom *hostname*, a verzija jezgra Linux-a naredbom *uname*.

```
hostname  
uname -a
```

Trenutno vrijeme i datum se saznaju naredbom *date*, a detaljniko kalendar naredbom *cal*.

```
date  
cal
```

Naredba *ls* bez parametara prikazuje spisak svih datoteka u tekućem direktoriju.

```
ls
```

Spisak svih datoteka uključujući i skrivene dobiva se parametrom *-a* naredbe *ls*.

```
ls -a
```

Da se dobije spisak svih datoteka u tekućem direktoriju i njegovim poddirektorijima sa detaljima o pravima, veličini, vlasništvu i vremenu kreiranja koristi se sljedeća naredba.

```
ls -lR
```

Svaka komanda ima parametar *-?* ili *--help* koji daje kratko uputstvo o toj komandi.

```
ls --help
```

Detaljniji opis svake naredbe možete dobiti koristeći komandu *man*. Ova komanda kao parametar ima ime komande o kojoj se traži uputstvo. Sljedeći primjer će prikazati priručnik sa svim parametrima naredbe *date* i načinom njihovog korištenja. Kroz ovaj priručnik kreće se strelicama gore/dolje ili PageUp/PageDown, a izlazi pritiskom na taster Q.

```
man date
```

Za prelazak u drugi direktorij koristi se komanda *cd*, čiji je parametar ime direktorija. Sljedeći primjer obavlja prelazak u direktorij */etc/apt*.

```
cd /etc/apt
```

Nekoliko direktorija imaju specijalna imena.

.	Trenutni direktorij
..	Direktorij koji se u hijerarhiji nalazi tačno iznad trenutnog
~	Početni direktorij za trenutnog korisnika
/	Korijenski direktorij cijelog datotečnog sistema

Sljedeća komanda prebacuje korisnika direktorij iznad trenutnog.

```
cd ..
```

Specijalni znak kose crte okrenut na lijevo (**) se koristi za unos znakova u komandnoj liniji koji mogu praviti probleme. Na primjer, razmak normalno razdvaja parametre i ako treba pristupiti direktoriju koji ima razmak u imenu (npr. */mnt/sda1/Program Files*), ispred razmaka se stavi ova kosa crta.

```
cd /mnt/sda1/Program\ Files
```

Ako se *cd* navede bez parametara, prelazi se u direktorij namijenjen trenutnom korisniku.

```
cd
```

Novi direktorij se kreira naredbom *mkdir*, koja kao parametar ima ime direktorija.

```
mkdir testni
```

Uz dodatni parametar *-p* ovom komandom kreiraju se i svi poddirektorije u navedenoj stazi ako ne postoje.

```
mkdir -p testni/pod1/pod2/pod3
```

Da se obriše datoteka koristi se komanda *rm*.

```
rm poruka.txt
```

Ako se uz ovu naredbu doda parametar *-r*, njom se mogu brisati i direktoriji.

```
rm testni -r
```

Datoteka se može kopirati u drugu datoteku ili direktorij naredbom *cp*. Ova naredba ima dva parametra, od kojih je prvi izvor, a drugi odredište. Sljedeći primjer, kopira datoteku *prva.txt* u direktorij */tmp*.

```
cp prva.txt /tmp
```

Istom naredbom može se napraviti druga kopija datoteke.

```
cp prva.txt druga.txt
```

Ako pri tome treba da nestane njena originalna kopija, dakle ne da se kopira nego da se premjesti treba koristiti komandu *mv*.

```
mv prva.txt /tmp
```

Ako odredište nije direktorij ova komanda se koristi za preimenovanje datoteka.

```
mv prva.txt druga.txt
```

Unix/Linux poznaje i veze na datoteke koje omogućavaju više kopija iste datoteke, ali da na disku koristi samo jedan prostor. To se zove simbolička veza. Simboličke veze se kreiraju komandom *ln*. Sljedećim primjerom će se kreirati datoteka *kopirana.txt* koja je identična naredbi *prva.txt*. Ako se nakon ove naredbe

mijenja sadržaj datoteke *prva.txt*, te će se promjene odraziti i na datoteku *kopirana.txt*.

```
ln prva.txt kopirana.txt
```

Za traženje neke datoteke u dubokom stablu direktorija koristi se naredba *find*.

```
find /home -name naziv.txt
```

Naredba *find* pruža širok izbor mogućih načina pretraga. Iza parametra *-name* se navodi regularni izraz s opisom imena datoteka koje se pretražuju, a iza parametra *-exec* može se navesti komanda koja se izvrši svaki put kada se pronađe datoteka čije ime zadovoljava dati uslov. Unutar parametra *-exec*, ime nađene datoteke se označava s otvorenom i zatvorenom vitičastom zagradom, a naredba završava znakovima kose crte okrenute nalijevo i tačka-zareza. Sljedeći primjer će iskopirati sve *.c* i *.h* datoteke iz svih poddirektorija direktorija */home/student* u direktorij */tmp/arhiva*.

```
find /home/student -name *. [hc] -exec cp {} /tmp/arhiva \;
```

Naredbom *which* saznaje se gdje se nalazi pojedina izvršna naredba. Sljedeći primjer pokazuje u kojem direktoriju se nalazi program *sort*.

```
which sort
```

Za prikaz poruke na ekranu koristi se naredba *echo*.

```
echo "Zdravo svima"
```

Za prikaz datoteke na ekranu koristi se naredba *cat*.

```
cat prva.txt
```

Ova naredba može da ima više parametara, koji predstavljaju datoteke koje se redom ispisuju.

```
cat prva.txt druga.txt treca.txt
```

Pošto veće datoteke prebrzo prelete na ekranu, prilikom ispisa, uvedene su komande *more* i *less*. Komandom *more* se prikazuje datoteka navedena kao njen parametar, praveći pauzu nakon svakog ispunjenog ekrana teksta, i čeka da pritisnete taster za razmak. Tasterom *Q* možete izaći iz programa *more*.

```
more prva.txt
```

Naredba *less* predstavlja unapređenje naredbe *more* i omogućava kretanje strelicama unazad kako bi se vidio već pregledani tekst.

```
less prva.txt
```

Za pregled samo prvih deset linija datoteke, koristi se naredba *head*.

```
head prva.txt
```

Za brisanje konzolnog prozora koristi se naredba *clear*.

```
clear
```

Za obnovu ispravnog funkcioniranja konzolnog prozora ako počne prikazivati čudne znakove umjesto slova koristi se *reset*.

```
reset
```

Da bi se izlaz iz neke komande umjesto na ekran proslijedio u datoteku, koristi se znak za redirekciju *>*, kao u sljedećem primjeru.

```
ls > spisak.txt
```

Ako se znak za redirekciju udupla, izlaz se nadovezuje na postojeću datoteku.

```
ls /etc >> spisak.txt
```

Da bi se izlaz iz jedne komande proslijedio kao ulaz u drugu komandu koristi se znak uspravne crte, npr. da se datoteka spisak.txt prikaže sortirano sa pauzom nakon svake strane koristi se sljedeći lanac od tri komande.

```
cat spisak.txt | sort | more
```

Ako komanda prihvata ulaz sa tastature, tada se taj ulaz može preusmjeriti da dolazi iz datoteke.

```
fdisk <priprema.txt
```

Ako je izlazni tekst takav da staje u jedan red, on može postati argument neke naredbe. U tom slučaju naredba se navodi između apostrofa okrenutih nadesno. Na primjer, da se vidi zauzeće prostora trenutnog direktorija mogu se kombinovati komande *du* (zauzeće navedenog direktorija) i *pwd* (ime trenutnog direktorija).

```
du `pwd`
```

Naredba *tar* spaja više datoteke u jednu ili raspakuje iz nje. To je korisno za arhiviranje i prenošenje preko Interneta. Kada se naredba *tar* koristi s parametrom *-c* tada se spaja više datoteka u jednu, a s parametrom *-x* se ovakva datoteka raspakuje. Izlaz iz naredbe *tar* se često prosljeđuje komandama *gzip*, *compress* ili *bzip2* koje zbirnu datoteku kompresuju. Sljedeći primjer kompresuje direktorij *home* u datoteku *home.tar.bz2*.

```
tar -c /home/ | bzip2 > home.tar.bz2
```

Datoteka kreirana na prethodni način se može dekompresovati i raspakovati sljedećom naredbom.

```
bzip2 -dc home.tar.bz2 | tar -x
```

Za pretraživanje podataka unutar tekstualne datoteke možete koristiti komandu *grep*. Ovoj komandi prvi glavni parametar predstavlja traženi niz znakova a drugi je datoteka u kojoj se on traži. Na primjer, pojava teksta "proizvod" u datoteci spisak.txt traži se sljedećom naredbom.

```
grep proizvod spisak.txt
```

Naredba *grep* je veoma moćna, jer omogućava pretraživanje složenijih izraza. Specijalni simboli *^* i *\$* predstavljaju početak i kraj linije. Prvi sljedeći primjer traži sve linije koje počinju riječju *root*, a drugi traži sve linije koje završavaju dvotačkom.

```
grep ^root /etc/passwd  
grep :$ /etc/passwd
```

Posebno je korisna mogućnost složenih regularnih izraza. Unutar uglastih zagrada se može navesti skup znakova koji mogu doći na datom mjestu, dok znak tačke predstavlja bilo koji znak. Sljedeći primjer će prikazati sve redove gdje se javljaju prezimena poput Milić, Mašić, Mišić, Mešić, Molać iz datoteke imenik.txt.

```
grep M.[lš].ć imenik.txt
```

Ako iza znaka tačke slijedi zvjezdica, proizvoljni znak se može više puta ponavljati. Tako, ukoliko pored prezimena navedenih u prethodnom primjeru trebaju izdvojiti iz datoteke imenik.txt prezimena Milanković, Malenković, Milanović, Mišanović itd., koristi se sljedeća naredba.

```
grep M.[lš].*ć imenik.txt
```

Izlaz iz naredbe *cat* se može proslijediti naredbi *sed* koja obavlja zamjene, brisanje i filtriranje linija. Najčešće korišteni parametar ove naredbe je komanda *s*

koja vrši zamjenu. Sljedeći primjer prikazuje datoteku `upute.txt` tako da se riječ kompjuter zamijeni riječju računar.

```
cat upute.txt | sed 's/kompjuter/računar/g'
```

Naredba `sort` sortira tekstualnu datoteku, dok `uniq` izbacuje ponovljene redove iz sortirane datoteke. Sljedeći primjer će napraviti uniju dvije datoteke.

```
cat skup1.txt skup2.txt | sort | uniq
```

Za informacije o broju linija, riječi i znakova u nekoj datoteci koristi se naredba `wc`, kao u sljedećem primjeru.

```
wc skup1.txt
```

Prelazak na drugog korisnika obavlja se komandom `su`. Ako ova komanda nema parametra podrazumijeva se prelazak na korisnika `root`. Sljedeći primjeri omogućavaju prelazak na korisničko ime `root` i na korisničko ime `mirko`.

```
su  
su mirko
```

Kako naredba `su` zahtijeva poznavanje lozinke drugog korisnika, npr. administratora `root`, širenje ove lozinke može predstavljati sigurnosni problem. Zato se u nekim Linux sistemima koristi program `sudo` koji daje pojedinim korisnicima (zabilježeno u bazi podataka kojim) pravo da postanu `root`, bez poznavanja lozinke korisnika `root`. Komande koje zahtijevaju veća prava mogu se pozvati s prefiksom `sudo`.

```
sudo su
```

Sada slijede neke administratorske komande koje su dostupne samo nakon `su` (ili `sudo su`) naredbe. Za dodavanje novog korisnika pod imenom `zlatko` otkucati sljedeću komandu i odgovoriti na pitanja o njegovoj lozinci, imenu i lokaciji.

```
adduser zlatko
```

Nad svakom datotekom su definirani vlasnik, grupa i ostali korisnici, te njihova prava. Da bi korisnik safet dobio vlasništvo nad datotekom `m.txt` koristi se komanda `chown`.

```
chown safet m.txt
```

Pripadnost datoteke grupi određuje se komandom `chgrp`. Da bi grupa studenti dobila vlasništvo nad datotekom `m.txt` koristi se komanda `chgrp`.

```
chgrp studenti m.txt
```

Komanda `chmod` dodjeljuje prava vlasniku, članovima grupe i ostalim korisnicima. Svakom od njih u navedenom redu odgovara jedna oktalna cifra prava. Najčešće se koriste cifre 0 (nikakva prava), 4 (samo za čitanje), 5 (čitanje i izvršenje), 6 (čitanje i pisanje), 7 (puna prava). Tako se vlasniku datoteke `m.txt` dodjeljuju puna prava (7), grupi prava čitanja i izvršenja (5), a ostalim samo čitanja (4) nad datotekom `m.txt` sljedećom komandom.

```
chmod 764 m.txt
```

Spisak aktivnih programa možete se dobiti komandom `ps`.

```
ps -A
```

Komanda `ps` će za svaki od procesa dati njegov identifikacioni broj. Taj broj može se koristiti da se nasilno prekine zaglavljani proces.

```
kill 2212
```

Ako iza komande slijedi znak `&`, ta komanda se izvršava bez komunikacije preko tastature.

```
cp velika odrediste &
```

Naredba *dd* se koristi za kopiranje koje uključuje i uređaje koji nisu prave datoteke. Njeni parametri *if*, *of*, *bs* i *count* redom predstavljaju ulaznu datoteku, izlaznu datoteku, veličinu blokova i broj blokova koji se koristi. Sljedeći primjer kopira cijeli sadržaj CD-a u datoteku.

```
dd if=/dev/cdrom of=snimak.iso
```

Da pristupite podacima na disk particijama koje nisu korištene u Linux datotečnom sistemu. Tako se particija treća particija prvog *hard* diska (`/dev/sda3`) povezuje s direktorijom `/mnt/disk` sljedećom komandom.

```
mount /dev/sda3 /mnt/disk
```

Naredbom *export* možete dodijeliti vrijednost promjenljivoj okruženja. Toj promjenljivoj se kasnije pristupa s znakom dolar ispred njenog imena. Sljedeće dvije naredbe će dodijeliti vrijednost varijabli *IME* i prikazati je.

```
export IME=testni  
echo $IME
```

Naredba *wget* preuzima datoteku s http site-a. Ona se najčešće koristi s jednim parametrom, URL koji predstavlja ime datoteke na udaljenom računaru. Uz parametar *-c* može se nastaviti prekinuto preuzimanje datoteke, a uz parametar *-limit-rate* može se ograničiti zauzeće mrežnih resursa. Sljedeći primjer koristi ove parametre.

```
wget -c -limit-rate=20k http://www.primjer.com/velika.iso
```

Da se prikaže stanje mrežnih interfejsa koristite komande *ethtool* (za žičane) i *iwconfig* (za bežične uređaje) ili *ifconfig* (za sve uređaje)

```
ethtool eth0  
iwconfig eth1  
ifconfig
```

Da se aktivira, odnosno deaktivira mrežni interfejs, koristi se naredba *ip link..*

```
ip link set dev eth0 up  
ip link set dev eth0 down
```

Spisak IP adresa se dobiva naredbom *ip addr*.

```
ip addr show
```

Naredbama *du* i *df* možete vidjeti zauzeće trenutnog direktorija i svih montiranih particija.

```
du  
df
```

Gašenje računara iz komandne linije obavlja se komandom *shutdown now*.

```
shutdown now.
```

Iz promijenjenog korisnika izlazi se sa *exit*. Ponavljanje ove komande napušta školjku.

```
exit
```

9.16. Programiranje Unix školjke

Shell skripte služe za automatizaciju čestih zadataka koji se inače ne mogu obaviti jednom prostom naredbom iz komandne linije. Najpopularnija školjka BASH omogućava pravljenje malih programa kojima se mogu ponavljati automatski često korištene naredbe. Ti programi se onda ponašaju kao nove komande školjke koje također imaju parametre. *Bash* skripte su tekstualne datoteke koje počinju linijom koja kaže gdje se nalazi interpreter komandi.

```
#!/bin/bash
```

U navedenom slučaju, program *bash* u direktoriju *bin* je interpreter naredbi.

Unutar programa u skriptnom jeziku mogu se koristiti parametri slično ostalim Unix komandama. Da se pročitaju parametri iz komandne linije, mogu se koristiti sljedeći specijalni skupovi znakova, Slika 100

\$0	ime datoteke s skriptom
\$#	ukupan broj parametara (>=0)
\$1	1. parametar
\$2	2. drugi parametar
\$*	svi parametri s kojim je skripta pozvana
\$\$	procesni identifikator (PID) školjke koji izvršava skriptu

Slika 100: *Bash*, specijalni paametri

Naredba *shift* pomjera parametre skripte: \$1 postaje raniji \$2, itd. a \$# se smanjuje za jedan (naravno, ako je bilo parametara). *Bash* skriptni jezik poznaje osnovne jezičke strukture. Najvažnija je *if*, koja uslovno izvršava dio koda. Njena struktura se sastoji *if* sekcije, obavezne *then* sekcije i opcionalnih *else* i *elif* sekcija. Ovdje se vide tri oblika, bez ovih sekcija, sa jednom ili obje.

		if naredba1
		then
		naredba2
		...
	if naredba1	elif naredba4
	then	naredba5
	naredba2	...
	naredba3	else
if naredba1	...	naredba6
then	else	naredba7
naredba2	naredba4	...
...	...	fi
fi	fi	

Naredba *if* izvršava zadatu naredbu i ako je ona vratila izlazni status 0 (logička vrijednost "tačno") izvršava se niz naredbi iza *then*. Moguće je navesti i jednu ili više naredbi *elif* koje testiraju dodatne uslove ukoliko raniji testovi nisu uspjeli. Na kraju, može se dodati naredba *else* koja izvršava niz naredbi koje iza nje slijede ako su sva ispitivanja uslova u *if/elif* klauzulama bila neuspješna. Kontrolna struktura *if* se završava ključnom riječju *fi*.

U *Bash* skriptama naredbe se mogu razdvajati novim redovima ili znakom tačka/zarez. Na primjer


```
if [ podjela == 0 ]
then
    echo $t
else
    ((t=$t/1024))
    echo $t K
fi
```

se može pisati i kao

```
if [ podjela == 0 ] ; then ; echo $t ; else ; ((t=$t/1024))
echo $t K ;fi
```

Kontrolnom strukturom *while* može se ponavljati niz naredbi dokle god je neki uslov zadovoljen, dok kontrolna struktura *until* ponavlja niz naredbi sve dok uslov nije ispunjen. Sintaksa ovih naredbi je

```
while naredba
do
niz naredbi
done
```

```
until naredba
do
niz naredbi
done
```

Naredba *test* je interna naredba koja se često koristi za ispitivanje uslova u kontrolnim strukturama. Argumenti naredbe *test* određuju vrstu testa. Izlazni status je 0 ako je test uspješan, a različit od nule ako je neuspješan. Slika 101 prikazuje sve moguće kombinacije uslova u naredbi *test*.

-e f	da li postoji datoteka/direktorij s imenom f
-f f	da li je f obična datoteka
-d f	da li je f direktorij
-r f	da li skripta može čitati datoteku ili direktorij f
-w f	da li skripta može pisati datoteku ili direktorij f
-x f	da li skripta može izvršavati datoteku ili direktorij f
-z str	da li je string str prazan
-n str	da li je string str neprazan
str1 = str2	jednakost stringova str1 i str2
str1 != str2	različitost stringova str1 i str2
str1 < str2	da li je str1 ispred str2 u abecednom redu
str1 > str2	da li je str1 iza str2 u abecednom redu
n1 -eq n2	da li su n1 i n2 jednaki
n1 -ne n2	da li su n1 i n2 različiti
n1 -lt n2	da li je n1 manji od n2 kao broj
n1 -le n2	da li je n1 manji ili jednak n2 kao broj
n1 -gt n2	da li je n1 veći n2 kao broj
n1 -ge n2	da li je n1 veći ili jednak od n2 kao broj
uslov1 -o uslov2	da li je zadovoljen bilo uslov1 bilo uslov2
uslov1 -a uslov2	testira da li je zadovoljen i uslov1 uslov2
! uslov	logička negacija zadovoljenosti uslova

Slika 101: Poređenje, parametri test naredbe

Naredba *test* se može i skraćeno zapisati uz pomoć uglastih zagrada. Npr. *test \$x -gt 2* se može zapisati kao *[\$x -gt 2]*. Obratite pažnju na razmake kojima su uglaste zagrade odvojene od susjednih riječi.

Naredba *for* se koristi za ponavljanje akcije nad skupom riječi ili datoteka:

```
for
ime_promjenljive [in lista_riječi]
do
niz
naredbi
done
```

Varijablama u *shell* programima dodjeljuju se vrijednosti tako što se odmah iza njenog imena dopiše bez razmaka znak "jednako", pa iza njega vrijednost varijable. Kada se želi očitati vrijednost varijable, navodi se znak \$ ispred njenog imena.

Shell programi mogu uključivati funkcije koje se pozivaju kao da su skripte u posebnoj datoteci. Definicija *shell* funkcije ima sljedeći oblik.

```
ime_funkcije () {
niz naredbi
}
```

Parametrima funkcije se može pristupiti na isti način kao parametrima same skripte, npr. \$2 predstavlja drugi parametar funkcije.

Sljedeći primjer u BASH skriptnom jeziku provjerava da li datoteka postoji i ako postoji prikazuje odgovarajuću poruku. Prvo je potrebno unijeti datoteku sljedećeg i snimiti je pod imenom "postoji".

```
#!/bin/bash
if [ $# -ne 1 ]
then
    echo "Upotreba - $0 datoteka"
    exit 1
fi
if [ -f $1 ]
then
    echo "$1 Datoteka postoji"
else
    echo "Datoteka $1 ne postoji"
fi
```

Kada je otkucana i snimljena datoteka "postoji", postaviće se pravo izvršavanja nad tom datotekom za korisnika koji ju je kreirao

```
chmod 755 postoji
```

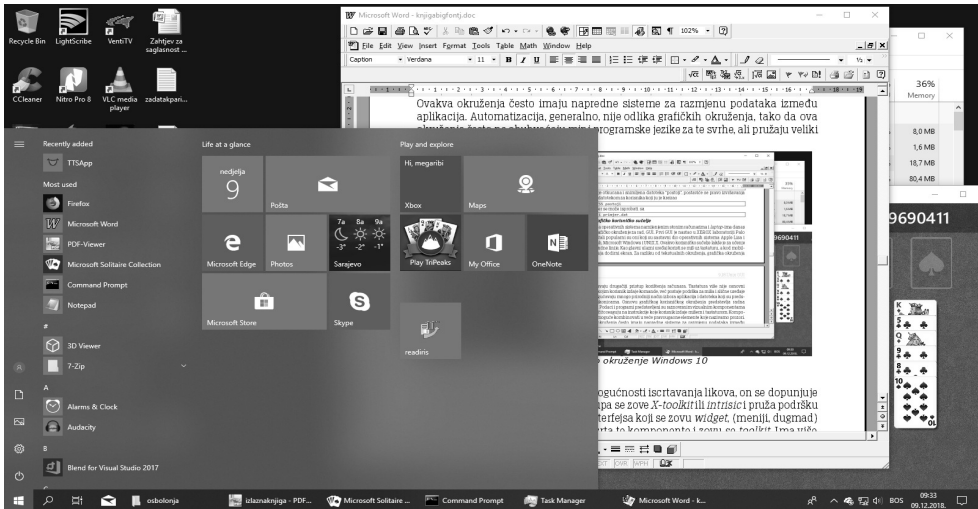
Primjer se može isprobati sa

```
./postoji primjer.dat
```

9.17.Grafičko korisničko sučelje

Većina operativnih sistema namijenjenim stonim računarima i *laptop*-ima danas koristi grafičko okruženje za rad, GUI. Prvi GUI je nastao u XEROX laboratoriji Palo Alto, a ostali popularni su oni koji su sastavni dio operativnih sistema Apple Lisa i Macintosh, Microsoft Windows i UNIX X. Ovakvo korisničko sučelje lakše je za učenje od komandne linije. Kao glavni ulazni uređaj koristi se miš uz tastaturu, a kod mobilnih uređaja dodirni ekran. Za razliku od tekstualnih okruženja, grafička okruženja omogućavaju drugačiji pristup korištenja računara. Tastatura više nije osnovni interfejs kojim korisnik izdaje komande, već postaje podrška za miša i slične uređaje koji omogućavaju mnogo prirodniji način izbora aplikacija i datoteka koji su predstavljeni ikoniciama. Osnovu grafičkog korisničkog okruženja predstavlja radna površina. Podaci i programi predstavljeni su raznovrsnim vizualnim komponentama

koje različito reaguju na instrukcije koje korisnik izdaje mišem i tastaturom. Komponente je moguće kombinovati u veće pravougaone elemente koje nazivamo prozori. Ovakva okruženja često imaju napredne sisteme za razmjenu podataka između aplikacija. Automatizacija, generalno, nije odlika grafičkih okruženja, tako da ova okruženja često ne obuhvaćaju mini programske jezike za te svrhe, ali pružaju veliki broj API funkcija za programiranje.

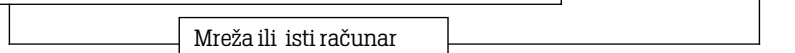


Slika 102 Grafičko okruženje Windows 10

9.18. Unix GUI

Pošto *Xlib* pruža samo osnovne mogućnosti iscrtavanja likova, on se dopunjuje sa još dvije grupe biblioteka. Jedna grupa se zove *X-toolkit* ili *intrinsic* i pruža podršku da se iscrtaju elementi korisničkog interfejsa koji se zovu *widget*, (meniji, dugmad) ali ih još ne crta. Druga grupa zaista crta te komponente i zovu se *toolkit*. Ima više *toolkit*-a (GTK, QT, Motif, FLTK, OpenLook ...). (Slika 103)

Window menadžer	Session manager	Display
Widget set (GTK+, QT, Fltk, Motif)		
X klijentska biblioteka (Xlib ili XCB)		



Slika 103 X Window sistem

Sam X sistem u kombinaciji s *toolkit* programima crta elemente grafičkog sučelja, ali je potreban i softver koji će crtati ukrase oko prozora, omogućiti njihovo pomjeranje, širenje, maksimiziranje uz dodatke elemenata kao što je Start dugme, ikona s značenjem i slično. Taj program se zove prozorski menadžer (*Window manager*), koji je promjenjiv u toku rada. Postoji više njih, kao što su minimalistički twm, Metacity, Compiz, kwm itd. Razlikuju se u izgledu i memorijskim zahtjevima.

Prozorski menadžer je obični pozadinski proces i registruje se s X serverom da se pokrene zajedno s njim.

Da bi korisnici mogli da se prijave na računarski sistem već u startu u grafičkom režimu rada, koristi se menadžer ekrana (*Display manager*). On može raditi lokalno ili preko mreže. Kada je menadžer ekrana aktivan, ovi korisnici startaju sesiju navodeći važeću kombinaciju korisničkog imena i lozinke. Menadžer ekrana također brine o pokretanju X servera pri podizanju ili eventualnom padu.

Menadžer sesija (*session manager*) čuva stanje aplikacije, tj. položaje i sadržaj svih otvorenih prozora. Korisnik može da se odjavi i ponovo prijavi s sačuvanim položajem prozora. Postoji poseban protokol za komunikaciju između menadžera prozora i menadžera sesija.

Ova tri menadžera se mogu pisati odvojeno. Ali, ako se pišu sa zajedničkim ciljem da čine jedinstvenu cjelinu, dobiva se *desktop* okruženje. U njemu sve aplikacije imaju isti izgled. Na primjer prozori za izbor datoteke ili štampača su jedinstveni, kao i načini iscrtavanja prozora. Pokretanje pojedinačnih programa (ali samo onih specijalno pisanih za to desktop okruženje ili bar *toolkit* na kom je njegov menadžer prozora zasnovan) je također ubrzano, jer su dinamičke biblioteke, koje oni koriste, već u memoriji dok traje sesija. Primjeri *desktop*-a su KDE, CDE, LXDE, EDE, Gnome itd.

9.19.Zadaci

Zadatak 1: Napisati BASH shell program koji kao parametar prima ime direktorija, prijavi grešku ako parametar nije direktorij. Ako je sve u redu, kopira sve datoteke u tom direktoriju na štampač (uređaj /dev/lp0).

Rješenje:

```
#!/bin/bash
if [ -d $1 ] ; then
  for i in $1 ; do
    if [ -f $i ] ; then ; cp $i /dev/lp0 ;    fi
  done
else
  echo $1 nije direktorij
  exit 1
fi
```

Zadatak 2: Napišite Bash program koji prima nekoliko parametara (do 9) i prikaže ih ako im je vrijednost veća od 20.

Rješenje

```
#!/bin/bash
for var in "$@"
do
  if [ $var -gt 20 ] ; then ; echo $var ; fi
done
```

10. UREĐAJI ZA KOMUNIKACIJU IZMEĐU RAČUNARA

Računari se mogu povezati sa drugim računarima, bilo korištenjem kablova koji povezuju samo dva računara bilo povezivanjem računara u velike računarske mreže. Računari koji učestvuju u mrežama mogu biti dosta različitog tipa i različitih operativnih sistema. Složena je situacija i sa uređajima, jedni uređaji su prilagođeni za telefonske linije, drugi su namijenjeni za bežične mreže, treći koriste satelite između kontinenata, a četvrti su ograničeni na kabl unutar prostorije. U nekim slučajevima samo dva računara koriste komunikacijsku liniju, a u nekim slučajevima linija se dijeli između mnogo učesnika.

10.1. Slojevi mrežnih modela

Složenost situacije koja se javlja u računarskim mrežama dovela je do potrebe za standardizacijom i podjelom standarda na slojeve. Slojevi predstavljaju zaokružen skup standarda i usluga koje se koriste od strane višeg sloja. Na primjer, sloj veze podataka zavisi od fizičkog sloja, a pruža usluge mrežnom sloju. Ali, dok se protokoli sloja veze podataka razlikuju za bežične mreže, telefonske linije ili *Ethernet* kartice, mrežnom sloju je svejedno kakvim medijem se prenose paketi podataka. ISO, organizacija za standarde definirala je sedam slojeva, od najnižeg ka najvišem.

1. Fizički sloj definira fizičke, električke, mehaničke i funkcionalne standarde za pristup fizičkom mediju tj. mreži. Protokoli i standardi ovog sloja definiraju parametre poput nivoa napona, oblika utičnica, oblika signala, načina predstavljanja pojedinačnih bitova na mediju itd.

2. Sloj veze podataka ima dva zadatka. Prvi je osiguravanje pouzdanog prijenosa podataka preko linije, između dva susjedna čvora. Na ovom sloju podaci se pakuju u okvire i takvi šalju na prijemnu stranu, gdje se oslobađaju okvira. Prilikom prijenosa podataka koristi se mehanizam potvrde prijema okvira. Drugi zadatak ovog sloja je razrješavanje problema kada više uređaja pristupa istom mediju.

3. Mrežni sloj brine se o prijenosu podataka, neovisno o mediju kojim su povezani. Za razliku od prethodnog sloja koji je zadužen za vezu između dva susjedna čvora (računara u vezi), na ovom sloju se podaci prenose preko cijele mreže. Ovaj sloj određuje i pravila kojim putem će podaci doći od izvora ka odredištu.

4. Transportni sloj prima podatke od višeg sloja, dijeli podatke u manje dijelove i prosljeđuje tako razbijene podatke mrežnom sloju. Ovo razbijanje je potrebno kako bi što više učesnika moglo koristiti komunikacijske uređaje, jer bi kod predugačkih poruka koje jedan učesnik šalje, svi ostali morali dugo čekati da uređaj bude slobodan. Kako se podaci sjećaju na manje komade (pakete), transportni sloj prati potvrdnim mehanizmom da li su svi paketi stigli na odredište, a ako nisu stigli da ih pošalje ponovo.

5. Sesijski sloj brine o uspostavljanju i prekidanju duže veze između dva računara (sesije) koja tokom koje se šalje više poruka prema transportnom sloju.

6. Prezentacijski sloj govori o formatu podataka (način zapisa brojeva, kompresija, kriptografija) koji se prenose.

7. Aplikativni sloj sadrži upravljačke funkcije i mehanizme za izvršavanje aplikacija, npr. udaljena prijava, elektronska pošta, web server, prijenos datoteka itd.

Kada se šalju podaci prema mreži, pripreme ih programi koji rade na najvišem, aplikativnom sloju i prosljede ih narednom nižem sloju. Svaki od slojeva može ispred ili iza podataka koje primi od višeg sloja dodati nove podatke (zaglavlja i kontrolne kodove na kraju) bitne za programe na odredišnoj strani koji rade na istom sloju. Fizički sloj može transformirati i pojedinačne bitove podataka radi karakteristika medija. Kada podaci dođu na odredište, oni se analiziraju na sloju veze podataka i ako su prihvaćeni prosljeđuju se višem, mrežnom sloju, bez podataka koje poslao sloj veze podataka pošiljaoca. Taj sloj dalje prosljeđuje podatke višem, transportnom sloju bez podataka koje je dodao mrežni sloj pošiljaoca i tako se nastavlja do aplikativnog sloja. Neki slojevi i ne moraju postojati, na primjer sesijski (5) i prezentacijski (6) sloj su nepotrebni u TCP/IP mrežama.

Uzmimo za primjer *dial-up* korisnika koji šalje elektronsku poštu. Aplikativni sloj na tekst poruke doda SMTP zaglavlje. Transportni sloj podijeli poruku na više manjih komada i svakom od njih doda TCP zaglavlje. Mrežni sloj doda IP zaglavlje za adresama krajnjeg odredišta. Konačno, sloj veze podataka pripremi podatke za slanje prvom susjedu, a to je računar sa modemom na drugoj strani i njemu pošalje PPP zaglavlje i završetak. Tako će se kratka *E-mail* poruka spremna za slanje na RS232 radi prosljeđivanja modemu sastojati od sljedećih elemenata:

PPP zaglavlje (sloj 2)	IP zaglavlje (sloj 3)	TCP zaglavlje (sloj 4)	SMTP zaglavlje (sloj 7)	Tekst poruke	PPP završetak (sloj 2)
---------------------------	--------------------------	---------------------------	----------------------------	-----------------	---------------------------

Na odredištu, kada se ovaj okvir primi, (gdje je npr. računar koji ima modem i *Ethernet* karticu) ukloniće se PPP zaglavlje i završetak i pročitati IP zaglavlje. Iz njega će se zaključiti da poruka nije namijenjena računarima u toj lokalnoj mreži. Zato zapakuje poruku navodeći kao odredište drugog sloja izlazni *router* u svojoj mreži koji je povezan *Ethernet* vezom.

Ethernet zagl. (sloj 2)	IP zaglavlje (sloj 3)	TCP zaglavlje (sloj 4)	SMTP zaglavlje (sloj 7)	Tekst poruke	Ethernet završe- tak (sloj 2)
----------------------------	--------------------------	---------------------------	----------------------------	-----------------	----------------------------------

Router skine *Ethernet* zaglavlje i završetak i zaključi iz IP zaglavlja da je odredište povezano iznajmljenom linijom koristeći HDLC protokol i pakira podatke ka njemu.

HDLC zaglavlje (sloj 2)	IP zaglavlje (sloj 3)	TCP zaglavlje (sloj 4)	SMTP zaglavlje (sloj 7)	Tekst poruke	HDLC završetak (sloj 2)
----------------------------	--------------------------	---------------------------	----------------------------	-----------------	----------------------------

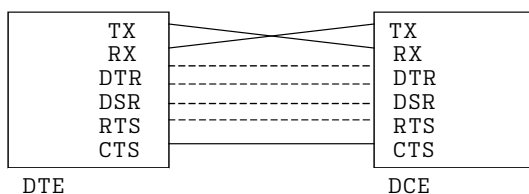
Odredište skine HDLC zaglavlje i završetak te iz IP zaglavlja zaključi da je poruka njemu namijenjena. Iz TCP zaglavlja prepozna pravi komad poruke, a iz SMTP zaglavlja ime korisnika kome je ona namijenjena i snimi je u odgovarajući direktorij za elektronsku poštu tog korisnika.

Sada će biti riječi o nekim češće korištenim komunikacijskim uređajima na fizičkom i sloju veze podataka.

10.2.RS232

RS232 je jedan od najstarijih serijskih komunikacionih standarda sa osiguranom jednostavnom povezivnošću i kompatibilnošću između različitih proizvođača. RS232 definiše signale kojim se povezuju DTE i DCE. Ovdje DTE označava *Data Terminal Equipment* i primjer za DTE je računar. DCE (*Data Communication Equipment*) označava opremu za razmjenu podataka, a primjer za DCE je modem. RS232 koristi serijsku komunikaciju, pri čemu se jedan bit podataka šalje u jednom trenutku duž jedne linije podataka. Serijska komunikacija zahtijeva manje žica od paralelne. Da

bismo uspostavili dvosmjernu komunikaciju, trebaju nam svega tri žice (RX, TX i GND). RX predstavlja liniju prijema signala, TX liniju slanja signala, a GND zajednički nulti potencijal.



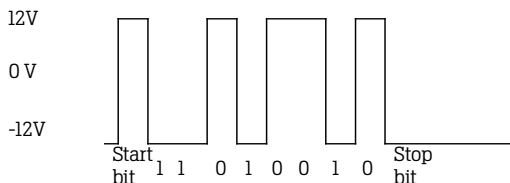
Slika 104 RS232 komunikacija

Prosti kabl sa tri žice ne garantuje da je prijemnik u datom trenutku bio spreman za prijem podatka. Zato se u RS232 uvode još dodatne linije za usaglašavanje (*handshaking*) između DTE i DCE. Te linije su: Spreman za slanje (RTS), Dopušteno slanje (CTS), Terminal spreman za prijem podataka (DTR) i Komunikacijski uređaj spreman za prijem (DSR).

Pretpostavimo da je računar (DTE) povezan sa modemom (DCE) preko RS232 interfejsa. Da bi poslali podatke sa računara na modem, obavi se sljedeća procedura.

1. Kada je modem spreman za prijem, postaviti će liniju DSR na 1.
2. Kada je računar spreman za slanje podataka, postaviće liniju RTS na 1.
3. Modem zatim postavlja CTS na 1, dopuštajući da se podaci mogu poslati pomoću računara (DTE).
4. Na kraju, računar šalje podatke linijom TX ka modemom.

Bajt podataka može se prenijeti u bilo kom trenutku pod uslovom da je prethodni bajt već prenesen. Nema signala sata za sinhronizaciju predajnika i prijemnika. Stoga se koriste start i stop bitovi da se informiše prijemnik o dolasku podataka. Logičko '1' u RS232 se predstavlja bilo kojim naponom između -15V i -3V, a logičko '0' se predstavlja bilo kojim naponom između +3V i +15V. Kada se šalje niz podataka, predajnik šalje startni bit prijemniku kako bi ga obavijestio da prijenos podataka počinje od sljedećeg bita. Startni bit je uvijek "0" tj. +12V. Nakon startnog bita, slijede biti podataka i eventualni bit pariteta. Nakon prenošenja podataka, predajnik šalje stop bit koji može biti dugačak 1 bit ili 1,5 bita ili 2 bita. Sljedeća slika prikazuje format prijenosa jednog bajta putem RS232.



Slika 105 RS232 signal

Prilikom prijema kompletnog bajta, kontroler RS232 prijenosa može da generiše prekid i omogućiti centralnom procesoru brzo čitanje podataka.

RS232 je jedan od najduže održanih računarskih standarda, ali ima ozbiljan limit na brzinu prijenosa podataka (115000 bita u sekundi). Zato je na mnogim računari-ma zamijenjen novijim USB standardom.

10.3. Telefonski, ADSL, VDSL i kablovski modemi

Telefonski sistem je mnogo ranije razvijen od računarskih sistema i potreba da se iskoristi postojeća infrastruktura telefonskih sistema dovela je do uvođenja modema. Telefonski modem se obično priključuje na RS232 interfejs. Modem se zasniva na pretvaranju sekvence električnog serijskog digitalnog signala u analogni signal pogodan za prijenos preko telefonskih linija, kao i pretvaranje telefonskog signala nazad u digitalni. Telefonska linija je prilagođena opsegu od 300 Hz do 4000 Hz, što odgovara ljudskom glasu. U suštini modem predstavlja interfejs između računara i okruženja telefona. Za modeme je razvijeno više standarda koji se razlikuju po brzinama prijenosa (od V.21 koji je dopuštao 300 bita u sekundi sve do V.92 koji postiže 56000 bita u sekundi).

Modem pretvara digitalni signal u analogni koristeći tehniku **modulacije**, koja predstavlja ubacivanje signala. Jedna od popularnih vrsta modulacija je kvadraturna amplitudna modulacija QAM. Nakon što modem prikupi n bita, on dodjeljuje jednu od 2^n mogućih vrijednosti za Q i I amplitude. Na primjer, u V.32 standardu, brzina 9600 bita u sekundi, 1001 sekvenca se prevodi u $Q=3, I=-1$, a sekvenca 1100 u $Q=1, I=1$. Tada se generiše signal proporcionalan $s(t)$ frekvencije f koji se mijenja u vremenu t i proslijedi na telefonsku liniju.

$$s(t) = I(t) \cos(2\pi ft) - Q(t) \sin(2\pi ft)$$

Frekvencija f za telefonske modeme je tipično 1650 Hz. Na odredištu se primljeni signal analizira u modemu određen broj puta u sekundi (npr. 1200, 2400, ili 3600 puta). Učestalost očitavanja signala se zove *baud rate*. Tada se na bazi izgleda primljenog signala rekonstruiše poslana sekvenca bita.

Telefonski sistem je namijenjen prije svega za prijenos glasa, sa ugrađenim filterima koji zabranjuju prolazak signala viših frekvencija kako ne bi smetali govoru. To ograničava noseću frekvenciju koja se može koristiti. Šumovi u signalu nastali iz termodinamičkih razloga ograničavaju dopušteni broj Q i I vrijednosti iz kojih se može prepoznati poslana sekvenca bita na jedinstven način. Teoretski limit klasičnog telefonskog sistema ako se koristi za prijenos podataka konverzijom digitalnog u analogni signal dugo je bio oko 35000 bita u sekundi.

Međutim, izbacivanjem filtra kada se ne koristi za govornu telefonsku komunikaciju, može se koristiti više frekvencija. Na primjer, sekvencu od 12 bita možemo podijeliti tako da se svaka 4 bita modulišu i šalju na različitim frekvencijama istovremeno (npr. prva 4 bita na 25000 Hz, druga 4 bita na 29312,5 Hz a treća 4 bita na 33625Hz, dakle svaka međusobno razdvojena po 4312.5 Hz). Time je brzina prijenosa povećana 3 puta. Ovakav način korištenja telefonske linije za prijenos podataka primijenjen je u ADSL (*Asymmetric Digital Subscriber Line*) modemima. U verziji ADSL 2+ ovakvi modemi koriste 452 frekvencije za prijem podataka i 50 frekvencija za slanje, uz *baud rate* 4000 očitavanja u sekundi i istovremenim prijenosom do 15 bita po svakoj od frekvencija. To omogućava brzinu prijenosa od 24

megabita u sekundi. Još noviji standard VDSL 2 (*Very-high-bit-rate digital subscriber line*) koristi do 8192 frekvencija, povećavajući brzine na 300 megabita u sekundi.

Još veće brzine postižu se modemima koji koriste koaksijalni kabl (inače namijenjen za prijenos televizijskog signala) ili optičke kablove. Sistemi kablovske televizije koriste frekvencije međusobno razmaknute po 6 MHz ili 8 MHz za svaki televizijski kanal ili prijenos podataka. Već upotreba jednog kanala od njih omogućava vrlo brzo očitavanje signala, preko 5 miliona puta u sekundi. Prenoseći istovremeno po 8 bita kablovskim modemom se uz upotrebu samo jednog kanala za prijenos podataka postižu brzine veće od 40 megabita u sekundi. Uz upotrebu više kanala postižu se višestruko veće brzine prijenosa (npr. po standardu DOCSIS 3.0 može se koristiti 8 kanala za prijem i 4 za slanje, čime se postižu brzine od 344 megabita u sekundi). Ipak, kod kablovske televizije često više korisnika koristi iste kablove, pa brzina prijenosa može znatno opasti kada više korisnika u zgradi ili ulici istovremeno pristupa Internetu, dok veza preko telefonskih linija nema taj problem.

Za komunikaciju računara sa ADSL i kablovskim modemima RS232 nema dovoljnu brzinu, pa se koriste USB ili *Ethernet*.

U sloju veze podataka za telefonski modem često se koriste protokoli SLIP (*Serial Line Internet Protocol*) i PPP (*Point to Point Protocol*). Prijenos podataka sa SLIP-om je vrlo jednostavan: ovaj protokol šalje okvir sastavljen samo od podataka koje treba poslati, nakon čega slijedi znak kraja prijenosa (tj. END znak, ASCII kôd 192). SLIP okvir izgleda ovako:

Podaci	END
--------	-----

Slika 106 SLIP okvir

Samo su dva izuzetka u podacima: ako je unutar njih bajt sa vrijednošću 192, umjesto njega se šalju vrijednosti 219 i 220, a ako je unutar njih bajt sa vrijednošću 219, šalju se vrijednosti 219 i 221.

Nažalost, ovako jednostavan protokol ne omogućava prijavu korisnika niti kontrolu grešaka u prijenosu. Alternativa je protokol PPP. Ovaj protokol zapravo definiše tri različita protokola. Prvi je LCP (*Link control protocol*), za uspostavljanje i testiranje veze. Drugi je grupa protokola NCP (*Network control protocol*) za konfigurisanje svih protokola mrežnog sloja koji se prenose koristeći PPP. Treći je enkapsulacija paketa iz viših slojeva u PPP okvir. PPP konekcija može početi tek nakon što je LCP i NCP protokolima veza uspostavljena i podešeni parametri mreže. Nakon toga se svaki paket koji je prosljeđen od mrežnog sloja pakuje u okvire prikazane na slici.

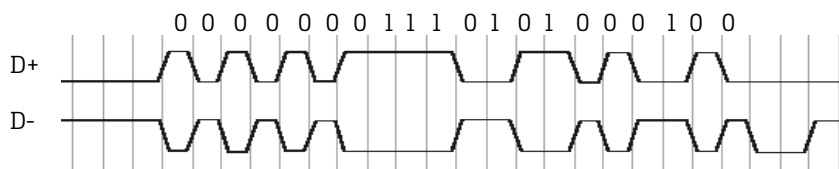
Flag	Adresa	Kontrola	Protokol	Podaci	Dopuna	CRC	Flag
8	8	8	8			16	8

Slika 107 PPP okvir

Polja Flag, Adresa i Kontrola (svako po 8 bita) uvijek imaju iste vrijednosti (redom 126, 255 i 3). U polju Protokol se navodi oznaka protokola mrežnog sloja. Za IP protokol to je vrijednost 33. Podaci su očekivano veličine do 1500 bajtova, a eventualno se kraći okviri mogu dopuniti do te veličine u polju Dopuna. Na kraju mrežnog okvira se kontrolna suma pošalje u polju CRC.

10.4.USB

USB (*Universal Serial Bus*) je uveden kao standard za priključivanje širokog broja uređaja uz vrlo mali broj žica. Koriste se samo četiri žice: napajanje VBUS, uzemljenje GND, signal D+ i negirani signal D-. Kada je D+ na potencijalu oko 3 V, tada je D- na potencijalu oko 0,2 V i obrnuto, kada je D+ na potencijalu oko 0,2 V, tada je D- na potencijalu oko 3 V. Nakon oduzimanja ove dvije vrijednosti dobiva se vrijednost signala očišćena od šumova. Za razliku od RS232 logička jedinica i nula nisu predstavljene vrijednostima napona signala, nego promjenom vrijednosti: ako je došlo do promjene vrijednosti signala D+ (a samim tim i D-) to predstavlja logičko 0, a ako u predviđenom vremenu nije došlo do te promjene, to predstavlja logičko 1.



Slika 108 USB signal

USB uređaji koji su povezani na računare imaju adrese između 1 i 128 koje dodjeljuje računar, dok je adresa 0 predviđena za računar na koga su povezani uređaji. Podaci koji se šalju preko USB sabirnice dijele se u pakete maksimalne veličine 8192 bita. Sloj veze podataka za USB uređaje definiše da se paketi prenose serijski standardizovanim brzinama (*Slow speed*, 1.5Mbps, *Full Speed* 12Mbps ili i brže). Postoji četiri vrste paketa (Slika 109).

SYNC 8	PID 8	ADDRESS 7	ENDPOINT 4	CRC5 5	EOP 3	Token paket
SYNC 8	PID 8	DATA 0-8192	CRC16 16	EOP 3	Data paket	
SYNC 8	PID 8	EOP 3	Handshake paket			
SYNC 8	PID 8	FRAME 11	CRC5 5	EOP 3	Start of frame paket	

Slika 109 Struktura USB paketa

Svaki paket počinje sinhronizacionim bajtom vrijednosti 80H, koji služi za generiranje taktnog signala. Nakon tog bajta šalje se polje za identifikaciju paketa *PID* (*Packet Identifier*). U polju PID prva 4 bita služe za identifikaciju vrste paketa, a sljedeća četiri bita su invertovani biti PID-a radi dodatne provjere njegove ispravnosti.

Token paketi označavaju vrstu podataka koji se prenose. *SETUP* paket (PID=1101) postavlja funkciju uređaja i sadrži tekuću i krajnju (*Endpoint*) adresu. *IN* paket (PID=1001) je prvi paket od periferijskog uređaja prema računaru. *OUT* paket (PID=0001) je prvi paket koji računar šalje periferijskom uređaju.

Data paketi sadrže podatke koji se prenose, u paketima veličine do 1 kilobajta. Naizmjenično se koriste dvije vrste ovih podataka, kao dodatna sigurnosna mjera za

lakše prepoznavanje izgubljenog paketa. Parni paketi podataka se šalje Data0 paketom (PID=0011). Neparni paketi podataka se šalju Data1 paketom (PID=1011).

Start of frame paket (PID=0101) predstavlja početak grupisanja paketa u okvire. On šalje 11 bitni podatak o broju okvira, kao i 5 bita CRC detekcije.

Handshake paketi se koriste za signalizaciju uspjeha ili neuspjeha transakcije. ACK paket (PID=0010) je potvrda prijemne strane da je paket primljen bez greške. NAK paket (PID=1010) je odgovor prijemnika da je paket neispravan. STALL paket (PID=1110) pokazuje da je neko odredište zagušeno tj. ne može da primi sve do tada poslano pakete.

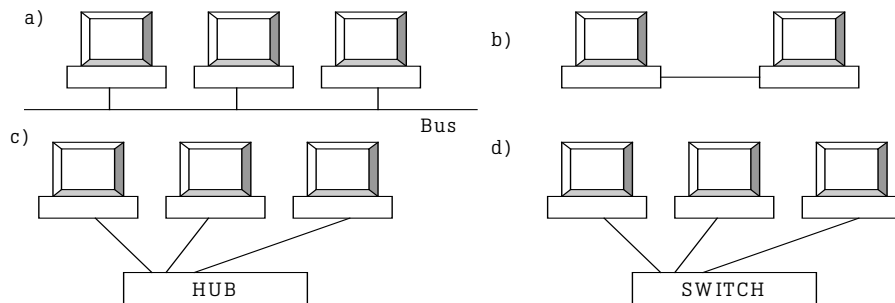
Prijenos podataka počinje tako što računar pošalje *token* paket u kome je definisan tip i smjer prijenosa, adresa USB uređaja i adresa krajnjeg odredišta. Uređaj prepozna svoju adresu iz adresnog polja i postaje spreman za prijem ili slanje podataka. Primalac odgovara slanjem ACK paketa izvještavajući o uspješnosti prijenosa. Kada je stigao ACK predajnik šalje sljedeće pakete, prvo se šalje paket Data0 pa paket Data1 i tako naizmjenično. Svi paketi imaju na kraju bite za redundantnu provjeru radi potvrde ispravnosti prijenosa. Komunikacija se obavlja u oba smjera i uvijek je pokrenuta od strane računara. Smjer prijenosa definiše računar slanjem IN ili OUT paketa. U IN paketu računar zahtijeva od eksternog USB uređaja da mu pošalje podatke. Nakon ovog paketa uređaj šalje *Data* paket, pri čemu se na kraju svakog primljenog paketa strana koja je vršila prijem šalje paket o (ne)uspješnosti slanja.

10.5. Ethernet

Najčešće korišteni hardverski i softverski standard za povezivanje računara u lokalnim mrežama je *Ethernet*. Dizajniran je za istovremeni pristup mreži većeg broja računara i drugih uređaja. Najčešće korištena vrsta kabla u *Ethernet* mrežama je UTP, uvijena parica. Ranije su se koristili i koaksijalni kablovi, a danas se sve više *Ethernet* koristi sa optičkim kablovima. Dizajniranje za male udaljenosti (oko 100 metara) osim ako je optički kabl u pitanju, kada je dopuštena udaljenost između čvorova u mreži oko 100 kilometara. Brzine prijenosa podataka se kreću od 1 megabit u sekundi do 400 gigabita u sekundi.

Računari koji koriste *Ethernet* mrežu uključuju određene adaptere koji se popularno zovu mrežne kartice. Zadatak mrežne kartice je da, prateći saobraćaj mreže, prepozna koji podaci su namijenjeni toj mrežnoj kartici i da, kada želi poslati podatke drugom računaru, postavi podatke na mrežu ukoliko je ona slobodna.

Rane verzije *Ethernet* mreže (Slika 110 a) povezivale su računare u lokalnoj mreži na jedan zajednički koaksijalni kabl – *bus*. Slanje podataka na mrežu svodi se na upis podataka na taj kabl. Prije slanja podataka mrežna kartica provjeri da li se podaci na tom kablju mijenjaju. Ako se mijenjaju, to je pouzdan znak da neko drugi već šalje podatke, pa će se odložiti slanje. Ali ako dvije mrežne kartice istovremeno uoče da je mreža slobodna, poslaće podatke istovremeno. Oba podatka će biti uništena. Ta situacija se zove kolizija. Mrežna kartica zato mora da provjerava da li su podaci koje je poslala na kabl jednaki onim koji se nalaze nakon toga na kablju. Ako nisu, to je indikator kolizije, prijenos podataka se prekida i čeka se novi trenutak da mreža bude slobodna.



Slika 110 Ethernet povezivanje

Unapređenje, ali znatno skuplje, je uvođenje *hub*-ova (Slika 110 c) i *switch*-eva (Slika 110 d). Kod ove dvije vrste uređaja svaki računar je odvojenim kablom povezan na centralno mjesto. To olakšava iskopčavanje i spajanje računara u mrežu, jer je kod koaksijalnog kabla potrebno kompletnu mrežu prekinuti kada god želimo dodati ili izbaciti računar iz mreže. **Hub** podatke primljene sa jedne linije šalje na sve linije. Stoga su kolizije jednako česte, možda i češće nego u *bus* varijanti. Ali dok je *hub* samo električno pojačalo, **switch**, je inteligentniji uređaj koji analizira izvor i odredište mrežnih okvira i šalje podatke samo na onu liniju gdje je odredište. Time su kolizije minimizirane ili čak nemoguće. U mrežama koje su sastavljene od samo dva računara sreće se i konfiguracija sa unakrsnim kablom između njih (Slika 110 b).

Fizičko predstavljanje bita u Ethernet mrežama se razlikuje između standarda. Tako na primjer, u 10 Mb mrežama logička jedinica se predstavlja kao prijelaz sa nižeg naponskog nivoa na viši, a logička nula kao prijelaz sa višeg naponskog nivoa na niži. U 100 Mb mrežama 4 bita se kodiraju sa 5 bita kako bi se lakše prepoznale sekvence jednakih vrijednosti kao što je 0000 i koju je teško razlikovati od neaktivne mreže. Zato se ova sekvenca predstavlja na 5 bita kao 11110.

Svaka mrežna kartica ima jedinstveni kôd nazvan MAC (*Media Access Control*) adresa. Ovo je binarni broj od 48 bita, koji su često predstavljeni u heksadecimalnom obliku kao PP-PP-PP-SS-SS-SS. Prva 24 bita (PP...) označavaju proizvođača, a naredna 24 bita (SS...) pripadaju serijskom broju kartice. Prema standardima ne može biti više kartica sa istim serijskim brojem, pa je MAC adresa jedinstvena.

Format *Ethernet* okvira na sloju veze podataka prikazan je na slici 111.

Preambula	SFD	Odredište	Izvor	Duž/Tip	Podaci i dopuna	CRC
56	8	48	48	16	368-12000	32

Slika 111 Ethernet okvir

Preambula predstavlja 56 alternirajućih nula i jedinica koje služe za sinhronizaciju između učesnika u mreži. Nakon nje se pošalje simbol početka okvira SFD, binarna sekvenca 10101011 (dekadno 213). Iza tog simbola na mrežu se pošalju MAC adrese odredišnog i izvorišnog mrežnog uređaja. Sljedeće polje Duž/Tip ima dva značenja. Ako je njegova vrijednost manja od dekadno 1500, onda ono predstavlja dužinu podataka u bajtima. Ako je veća od 1500, tada se uzima da je dužina tačno 1500 bajtova (12000 bita), a to polje predstavlja oznaku mrežnog protokola čiji se paketi prenose u polju podataka. Npr. vrijednost 2048 predstavlja IP V4 protokol, a

vrijednost 33079 predstavlja Novell IPX protokol. Najveći dio okvira sastoji se od njegovog sadržaja. Na kraju okvira nalazi se kontrolna suma.

10.6. Bežična mreža 802.11

Bežične komunikacije se koriste u širokom dijapazonu, počevši od povezivanja malih uređaja sa centralnim računarom (*Bluetooth* mreža), lokalnih mreža (802.11), gradskih mreža (802.16), radio amaterskih mreža (paket radio), mobilnih telefona (GSM, D-AMPS), satelitskih komunikacija i komunikacija sa svemirskim brodovima. Jedna od najčešće korištenih bežičnih mreža je 802.11, lokalna bežična mreža, poznata i pod imenom WiFi. 802.11 mreža podijeljena je na ćelije, u kojoj svakom ćelijom upravlja bazna stanica ili pristupna tačka (*Access Point*). Bazne stanice različitih ćelija se međusobno povezuju *Ethernet* vezom ili bežično.

Kada računar u bežičnoj mreži želi da šalje podatke, on provjerava da li trenutno ima prijenosa podataka. Ako je medij slobodan on počinje da šalje podatke. Prijemnik prima podatke, izračuna kontrolnu sumu (CRC) i pošalje potvrdni paket (ACK). Ako je pošiljalac primio ACK paket, to znači da nije bilo kolizije. Problem je što dva računara na različitim krajevima zgrade ne mogu čuti jedan drugog, ali pristupna tačka ih čuje oba. Za oba računara mreža je slobodna, pa počinju istovremeno slanje što rezultuje kolizijom. Da bi se smanjila mogućnost kolizije uvodi se virtualno prepoznavanje nosača signala. Računar koji želi poslati podatke prvo šalje kratki kontrolni paket RTS se podacima o izvoru, odredištu i dužini planirane transakcije. Prijemnik odgovara CTS paketom. Svi ostali računari u bežičnoj mreži koji su prepoznali RTS ili CTS pakete isključe rad umreži za vrijeme navedeno u paketima.

Preambula	SFD	PLCP Zaglavlje	Kont. okvira	Trajanje	Adresa 1	Adresa 2	Adresa 3	Sekvenca	Adresa 4	Podaci	CRC
80	16	40	16	16	48	48	48	16	16	<18496	32

Slika 112 802.11 okvir

Struktura okvira u kome se šalju podaci u 802.11 mrežama je nešto složenija od okvira u *Ethernet* mrežama. Preambula služi za sinhronizaciju i njena dužina se mijenja sa novim verzijama standarda, a kraj preambule je definisan SFD (*Start Frame Delimiter*) sekvencom od 16 bita. PLCP zaglavlje sadrži informacije o dužini paketa i propusnom opsegu. U polju Kontrola okvira svaki bit ima određeno značenje. Tu su podaci o verziji protokola, vrsti okvira, da li je sadržaj fragmentiran itd. Fragmenti su još manje jedinice od okvira. Polje Trajanje govori koliko vremena u mikrosekundama će se potrošiti na slanje sljedećih podataka, obično korišteno u RTS, CTS i ACK tipovima okvira. Za razliku od *Ethernet* protokola, u 802.11 okvirima javljaju se čak 4 adrese. Adresa 1 je obično 48-bitna oznaka neposrednog primaoca podataka, dok Adresa 2 sadrži neposrednog pošiljaoca. U mrežama u kojima se više bežičnih mreža spaja preko različitih pristupnih tačaka, neposredni pošiljalac i primalac nisu nužno sami računari koji su inicirali podatke ili kome su namijenjeni, nego to mogu biti uređaji pristupnih tačaka. U tom slučaju se koriste i Adresa 3 i Adresa 4 koje mogu zavisno od konfiguracije u polju Kontrola okvira čuvati adresu računara koji je inicirao podatke i računara kome su krajnje namijenjeni. Bežično okruženje je manje pouzdano od žičanog. Zbog čestih grešaka u komunikaciji, uvodi se kontrola toka i na nivou veze podataka, ne samo na transportnom nivou. U tu

svrhu, u polju Sekvenca nalazi se redni broj okvira koji se očekuje ili fragmenta na koji je podijeljen okvir.

Podaci iz ovih okvira se u raznim verzijama 802.11 standarda različito kodiraju i generišu radio talase. Koriste se frekvencije oko 5 GHz i 2.4 GHz. Područja u blizini tih frekvencija dijele se na kanale širine 22 MHz, a kanali na po 64 podnosača širine 312,5 kHz. Kada se šalju podaci, serija bitova, oni se modulišu kvadraturnom amplitudnom modulacijom po 2-6 bita za pojedinu podnosačku frekvenciju. Slično ADSL-u, jedna grupa bita se šalje na jednoj podnosačkoj frekvenciji, a druga na drugoj. Neki biti se, međutim, šalju i na više različitih frekvencija radi povećanja pouzdanosti. Tako prijemnik ne mora primiti sve podnosače da rekonstruiše informacije.

10.7. Protokoli mrežnog sloja: IP protokol

Adresiranje računara na sloju veze podataka prilagođeno je specifičnim mrežnim uređajima za koje je taj protokol dizajniran. Neki protokoli kao što je SLIP uopšte se ne bave adresiranjem na drugom sloju, jer im i ne treba: komuniciraju samo dva sagovornika. S druge strane, *Ethernet* protokol omogućava dovoljno jedinstvenih adresa za cijeli svijet, ali organizovanih po proizvođaču mrežne kartice, dakle neupotrebljivih za globalno nalaženje odredišta. Očigledno je potrebno dodijeliti adrese računarima u cijelom svijetu na način koji ne ovisi o prijenosnom mediju i adresama na sloju veze podataka.

Dosta jednostavno rješenje pružio je IP protokol V4. Adresa je broj od 32 bita koji se radi ugodnosti predstavlja kao četiri osmobarbitna broja. Prvih n bita predstavlja oznaku mreže a preostalih $32-n$ bita predstavlja oznaku računara unutar te mreže. Koliko bita predstavlja oznaku mreže određuje se dodatnim podatkom koji se zove mrežna maska. Tako oznaka IP adrese 218.22.14.3/24 znači da prvih 24 bita predstavlja oznaku mreže (dekadno 218.22.14, ili binarno 110111010 00010110 00001110), a preostalih 8 bita oznaku računara unutar mreže (dekadno 3 ili binarno 00000011). U specijalnom slučaju, oznaka računara unutar mreže koja se sastoji od svih bitova koji su nule predstavlja samu mrežu, a oznaka od svih bitova koji su jedinice predstavlja pakete koji se šalju svim čvorovima u mreži. Pokretljiva mrežna maska omogućava lako preusmjeravanje mrežnih paketa, prepoznavanje odredišta i podjelu mreža na manje mreže. Na primjer mrežu 77.0.0.0/8 (inače jedna od mreža dodijeljenih Evropi i Bliskom Istoku) moguće je podijeliti na milione raznih mreža različite veličine. Jedna od mogućih podjela je na četiri nove mreže; 77.0.0.0/10, 77.64.0.0/10, 77.128.0.0/10 i 77.192.0.0/10.

Uređaji koji gledaju pakete na mrežnom sloju (npr. pakete IP protokola) i odlučuju gdje će ih proslijediti zovu se *router*-i. To su specijalizovani računari sa operativnim sistemom prilagođenim za prosljeđivanje mrežnog saobraćaja. U njima se mogu nalaziti statičke tabele sa oznakom mrežnog priključka na koji se šalju primljeni paketi za odgovarajuće mreže (IP adrese sa maskom), a mogu se konfigurirati i da te tabele dinamički kreiraju na bazi brzine prijenosnog puta i dostupnosti veze. Tako, u slučaju prekida nekih od putova podataka (npr. nestalo električne energije u Budimpešti), *router* može da pronađe drugi, dostupan put.

IP protokol dodaje zaglavlje paketima koje je primio od transportnog sloja.

Verzija 4	Dužina IP zaglavlja 4	Različite usluge 6	Kontrola zagušenja 2	Ukupna dužina 16	Identifikacija 16	Flegovi 3
Pozicija fragmenta 13	Vrijeme života 8	Protokol 8	Suma zaglavlja 16	Izvorna Adresa 32	Odredišna adresa 32	Opcije 0 ili 32

Slika 113 IP V4 zaglavlje

Najvažnija polja su Izvorna Adresa i Odredišna adresa, koja sadrže 32 bitne IP adrese pošiljaoca i primaoca. Polje Protokol predstavlja oznaku protokola transportnog nivoa koji je proslijeđen u dijelu paketa sa podacima, za protokol TCP to je vrijednost 6, za UDP to je 17. Polje verzija sadrži vrijednost 4. U polju IHL navodi se dužina IP zaglavlja u 32-bitnim riječima, ako nema polja Opcije, navodi se 5 što predstavlja 20 bajta. Polje Različite usluge omogućava kategorizaciju paketa u 64 različite kategorije, čime pomaže *router*-u da odredi prioritete u proslijeđivanju paketa (npr. multimedijalni paketi se moraju slati ranije). Ako je polje kontrola zagušenja postavljeno na vrijednost različitu od 0, obavještava se odredište da ne odbacuje paket u slučaju zagušenja, nego da obavijesti pošiljaoca da se desilo zagušenje, koji će onda smanjiti brzinu slanja. Polje Ukupna dužina predstavlja veličinu cijelog paketa podataka, zajedno sa zaglavljem i podacima. Ako je dužina veća od one koju dopušta niži sloj veze podataka, paket se siječe u manje jedinice, fragmente. Ako se to dogodilo, pri ponovnom sastavljanju IP paketa se koristi polje Identifikacija kojim se jedinstveno prepozna paket od koga su fragmenti nastali. Poljem Flegovi može se zabraniti fragmentacija ili informisati primaoca da iza poslanog fragmenta slijedi još fragmenata. Polje Pozicija fragmenta kaže na kojem mjestu u nefragmentiranoj verziji paketa se nalazi početak fragmenta koji se šalje. Polje vrijeme života se umanjuje za 1 pri svakom proslijeđivanju paketa i kada dostigne nulu paket se odbacuje, kako bi se spriječili zalutali ili kružeći paketi. Polje Suma zaglavlja računa kontrolnu sumu od polja koja čine zaglavlje u cilju provjere njegove ispravnosti. Polje opcije može sadržati dodatne parametre IP zaglavlja, ali se ono rijetko koristi.

IP V4 pruža oko 4 milijarde različitih IP adresa. Taj broj se potrošio 2012, pa se pribjegava korištenju većeg broja privatnih IP adresa koje se u *router*-ima mijenjaju u manji broj javnih. Vremenom to rješenje neće biti dovoljno, pa se polako prelazi na IP V6 protokol. U ovom protokolu, veličina adrese je povećana na 128 bita, a struktura zaglavlja nešto jednostavnija, iako je samo zaglavlje veće.

Verzija 4	Različite usluge 6	Kontrola zagušenja 2	Oznaka toka 20	Veličina podataka 16	Protokol 8	Vrijeme života 8	Izvorna Adresa 128	Odredišna adresa 128
--------------	--------------------------	----------------------------	----------------------	----------------------------	---------------	------------------------	--------------------------	----------------------------

Slika 114 IP V6 zaglavlje

Značenje polja je jednako kao kod istoimenih polja u IP V4. Podjela paketa u fragmente nije predviđena, pa polja koja se odnose na fragmentaciju ne postoje. Novo polje Oznaka toka daje savjet *router*-ima da pakete sa istom vrijednosti tog polja proslijeđuju uvijek na isti način.

10.8. Protokoli transportnog sloja TCP i UDP protokol

Paketi koji se šalju IP protokolom obično su dio neke veće poruke, na primjer datoteke koja se prenosi sa jednog računara na drugi. Sljedeći sloj protokola, transportni sloj, brine o podjeli tih poruka na pakete i garantovanju njihove dostave. Najčešće se koriste dva protokola na ovom nivou: TCP (*Transmission Control Protocol*) namijenjen za poruke za koje se traži potvrda o njihovoj dostavi i UDP koji namijenjen za poruke koje se ne moraju potvrđivati.

Svaka TCP veza između dva sagovornika predstavljena identifikacijskim brojevima koji se zovu portovi, po jedan za svaku stranu u razgovoru. Sama IP adresa nije dovoljna jer su između dva sagovornika moguće različite vrste komunikacije (*web*, *E-mail*, prijenos datoteka, udaljena prijava itd.). Prilikom TCP komunikacije strane međusobno razmjenjuju pakete koji sadrže same podatke, dokle se stiglo sa slanjem i koliko je podataka primljeno. Kada pošiljalac pošalje nekoliko paketa ka primaocu, on očekuje od primaoca paket sa odgovorom koliko je podataka primljeno. TCP zaglavlje koje se nalazi ispred poruke izgleda ovako:

Port Izvora	Port Odredišta	Broj Sekvence	Broj potvrde	Veličina zaglavlja	0	Flegovi	Veličina prozora	Kontrolna suma	Urgent pointer	Opcije
16	16	32	32	4	3	9	16	16	16	0-32

Slika 115 TCP zaglavlje

Veza između dva sagovornika identifikovana je portovima izvora i odredišta. Nakon uspostavljanja komunikacije pošiljalac šalje poruke, razdvojene u nekoliko IP paketa. Pošiljalac šalje IP pakete i u polju Broj sekvence upiše koliko je već bajta poslao. Brojač sekvence ne mora krenuti od nule, nego od vrijednosti koja je bila u tom polju kada je poslan paket sa flegom SYN. Primalac šalje pakete sa flegom ACK, šaljući u polju Veličina prozora koliko bajta je spreman da primi odjednom, a u polju Broj potvrde pošalje naredni broj sekvence koji očekuje. Na primjer, pošiljalac je poslao četiri paketa sa brojevima sekvence 10, 1210, 2410 i 3610, a primalac odgovorio sa brojem potvrde 2410. To je indikacija pošiljaocu da se treći paket izgubio, a četvrti ili izgubio ili odbačen pošto je primalac očekivao treći paket. Tada pošiljalac ponovo šalje treći i četvrti paket. Polje Flegovi sadrži 9 bita, koji označavaju specijalne pakete. U ovom polju, pored spomenutih SYN i ACK, bit URG znači da postoji Urgent pointer, bit FIN koji indicira da je to zadnji paket koga šalje pošiljalac, bit PSH koji kaže da treba primljene pakete iz bafera što prije obraditi, bit RST koji znači da treba resetovati konekciju i još tri bita koja se koriste u kontroli zagušenja. Kontrolna suma se računa iz dijela zaglavlja i svih podataka. Polje Urgent pointer se vrlo rijetko koristi, npr. u Telnet protokolu da je unutar poruke negdje kôd *Break* tastera.

Za primjene koje ne zahtijevaju potvrdu prijema koristi se mnogo jednostavniji protokol UDP (*User Datagram Protocol*). UDP paket se sastoji od samo četiri polja.

Izvorni port	Odredišni port	Dužina zaglavlja i sadržaja	Kontrolna suma zaglavlja i sadržaja
32	32	16	16

Slika 116 UDP zaglavlje

10.9. Aplikativni protokoli

U polju sa podacima iza TCP ili UDP zaglavlja mogu se nalaziti posebne komande koje su dalje vezane za odgovarajući protokol aplikativnog nivoa. Protokoli na aplikativnom nivou se međusobno dosta razlikuju. Neki od njih koriste komande u

formi otvorenog teksta radi ostvarivanja određenih usluga, dok drugi koriste binarne formate. Većina protokola aplikativnog sloja je vezana uz poznati TCP ili UDP port na serverskoj strani.

Port	Protokol	Svrha	Enkapsulacija
20	FTP podaci	Prijenos datoteka	TCP
21	FTP komande	Prijenos datoteka	TCP
22	SSH	Šifrovana prijava na udaljeni računar	TCP
23	Telnet	Prijava na udaljeni računar otvorenim tekstom	TCP
25	SMTP	Slanje elektronske pošte	TCP
37	Time	Sinhronizacija časovnika	UDP/TCP
43	Whois	Informacije o vlasniku domene	TCP
53	DNS	Prevođenje razumljivog imena u IP adrese	UDP
67	BOOTP, DHCP	Podizanje i konfigurisanje računara preko mreže	UDP
69	TFTP	Jednostavni protokol za prijenos datoteka	TCP
70	Gopher	Tekstualne informacijske stranice	TCP
79	Finger	Podaci o pojedinačnom korisniku	TCP
80	HTTP	Tekstualne i grafičke informacijske stranice	TCP
110	POP3	Elektronska pošta za krajnjeg korisnika	TCP
111	NFS	Dijeljeni diskovi u Unix mrežama	TCP/UDP
119	NNTP	Novinske grupe	TCP
161	SNMP	Upravljanje mrežom	UDP
194	IRC	Poruke između korisnika	TCP
220	IMAP	Elektronska pošta stalno na serveru	TCP
512	LPD	Mrežno upravljanje štampačom	TCP

Slika 117 Važniji aplikativni protokoli

HTTP protokol iako dominantan je prilično jednostavan. Namijenjen je čitanju web stranica. Zasnovan je na svega devet komandi, od kojih se najčešće koriste četiri: GET (čitanje sadržaja web stranice), POST (slanje podataka), PUT (slanje web stranice) i DELETE (brisanje web stranice). HTTP zahtjev sa strane klijenta sadrži u prvom redu komandu, u narednim njene opcije (dijelove zaglavlja), zatim slijedi jedan prazan red i onda tijelo poruke (koje se javlja kod komandi POST i PUT). Kada *browser* želi da pročita web stranicu `www.hello.com/pub/index.html` poslaće sadržaj TCP poruke prema portu 80 servera, uz port klijenta koji sam ponudi.

```
GET /pub/index.html HTTP/1.1
Host: www.hello.com
```

Server odgovara porukom koja u sebi sadrži odgovarajuća HTTP zaglavlja i samim sadržajem web stranice. Prvi red odgovora daje status zahtjeva, naredni redovi pružaju informacije o sadržaju, zatim slijedi prazna linija i na kraju sam sadržaj. Dobiveni sadržaj je obično u prezentacijskom jeziku HTML.

```
HTTP/1.1 200 OK
Date: Sun, 10 Oct 2010 23:26:07 GMT
Content-Length: 41
Content-Type: text/html

<HTML><BODY>
Hello world!
</BODY></HTML>
```

I protokol za slanje elektronske pošte SMTP (*Simple mail transfer protocol*) zasnovan je na malom broju komandi. Najčešće se koriste komande HELO (za identifikaciju klijenta), MAIL (za navođenje *e-mail* adrese pošiljaoca poruke), RCPT (za navođenje *e-mail* adrese primaoca) i DATA (za definiranje teksta poruke koja završava redom koji u sebi sadrži samo tačku).

Da se ne moraju koristiti IP adrese koje su teške za pamtiti i promjenjive u vremenu, koristi se DNS protokol. On je baziran na UDP jer se zahtjevi šalju na više servera od kojih mali broj treba da odgovori. Zahtjevi uključuju razumljivo ime računara (npr. `www.example.com`) za koje server koji brine o tom imenu pošalje numeričku IPV4 ili IP V6 adresu.

10.10.Zadaci

Zadatak 1: Koliko računara može biti u mreži 128.10.128.0/22

Rješenje: Ukupno $2^{32-22}-2=1022$ (dvije IP adrese su za oznaku mreže i *broadcast*).

Zadatak 2: Mrežu 128.10.128.16/28 podijelite u 4 manje

Rješenje: 128.10.128.16 gledano binarno je

10000000.00001010.10000000.00010000

Četiri najniže binarne cifre pripadaju računarima unutar mreže. Prve dvije među njima će razlikovati podmreže, koje će kada vratimo u dekadni zapis biti:

10000000.00001010.10000000.00010000 : 128.10.128.16/30

10000000.00001010.10000000.00010100 : 128.10.128.20/30

10000000.00001010.10000000.00011000 : 128.10.128.24/30

10000000.00001010.10000000.00011100 : 128.10.128.28/30

Zadatak 3: Šta se zaključuje iz sljedeće sekvence bajtova koja je uhvaćena u mreži?

```
AA AA AA AA AA AA AA AB 01 00 5E 00 00 12 00 00 5E 00 01 01 08 00 45
00 00 56 85 F5 40 00 34 06 52 35 C0 A8 01 64 C0 A8 01 C8 9E 75 00 19
09 81 DC 65 22 79 A5 EC 80 18 82 18 04 49 00 00 01 01 08 0A 12 7D D9
29 06 70 6A FB 48 45 4C 4F 20 77 65 62 33 34 32 31 33 2E 6D 61 69 6C
2E 6D 75 64 2E 79 61 68 6F 6F 2E 63 6F 6D 0D 0A 97 1C 19 F8
```

Rješenje: U pitanju je Ethernet protokol: **AA AA AA AA AA AA AA AB** – Preambula i start bloka, **01 00 5E 00 00 12** – Odredišna MAC adresa 01:00:5E:00:00:12, **00 00 5E 00 01 01** – MAC adresa pošiljaoca 00:00:5E:00:01:01, **08 00** – Prenosi se IP V4 paket

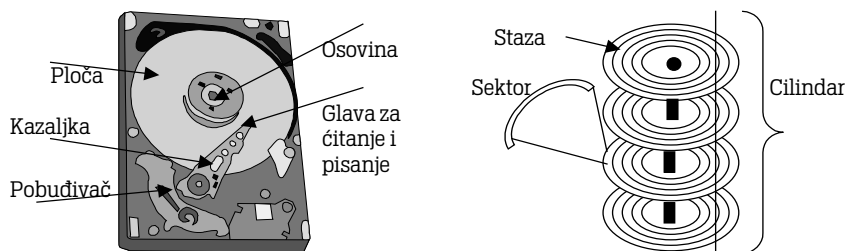
Sljedeći bajtovi se odnose na IP protokol: **45** – Verzija 4, zaglavlje $5*4=20$ bajtova, **00** – Nema diferencijacijskih usluga, **00 56** – Dužina paketa 86 bajtova, **85 F5** – Identifikacija 85 F5, **40 00** – Bez fragmentacije, **34** – Vrijeme života 56 prosljeđivanja, **06** – TCP protokol, **52 35** – Kontrolna suma zaglavlja, **C0 A8 01 64** – IP adresa izvora 192.168.1.100, **C0 A8 01 C8** – IP adresa odredišta 192.168.1.200. Sljedeći bajtovi se odnose na TCP protokol: **9E 75** – Izvorni port 40565, **00 19** – Odredišni port 25 SMTP, **09 81 DC 65** – Broj sekvence 159505509, **22 79 A5 EC** – Broj potvrde 578397676, **80** – Zaglavlja $8*4=32$ bajta, **18** – Flegovi Push, ACK **82 18** – Veličina prozora 33304, **04 49** – Kontrolna suma, **00 00** – Nema urgent pointera, **01 01 08 0A 12 7D D9 29 06 70 6A FB** – 12 bajtova opcija. Sljedeći bajtovi se odnose na SMTP protokol: **48 45 4C 4F 20** – HELO, **77 65 62 33 34 32 31 33 2E 6D 61 69 6C 2E 6D 75 64 2E 79 61 68 6F 6F 2E 63 6F 6D 0D 0A** – web34213.mail.mud.yahoo.com\r\n, Završni dio *Ethernet* okvira: **97 1C 19 F8** – Kontrolna suma

11. UREĐAJI SEKUNDARNE MEMORIJE

Sadržaj sekundarnih memorija se za razliku od interne radne memorije, ne gubi nakon isključivanja napajanja. Sekundarne memorije služe za smještanje operativnog sistema, podataka i programa.

11.1. Magnetni diskovi

Hard disk (ili tvrdi disk) je uređaj koji koristi namagnetisane ploče koje se brzo okreću i dugotrajno čuvaju sadržaj podataka. Mehanički dio diska (Slika 118) se sastoji od magnetnih ploča, glava za čitanje/pisanje, pobuđivačke kazaljke, pobuđivača i osovina. Tokom operacija čitanja i pisanja glava miruje dok ploča rotira ispod glave. Mehанизam pisanja zasnovan je na principu magnetnog uzorka.



Slika 118 Hard disk

11.2. Organizacija diska

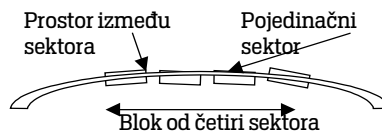
Podaci se zapisuju u koncentrične krugove na disku koji se zovu **staze** ili trake. Susjedne staze su odvojene međuprostorom, radi smanjenja njihove interferencije. Staze su podijeljene na **sektore** (fiksne ili varijabilne veličine), koji su međusobno razdvojeni međuprostorom. Uobičajeni su sektori fiksne veličine 512 bajta. Staze se broje od vanjske strane diska (broj 0) prema centru. S obzirom da se u pojedinom uređaju nalazi više ploča, uvodi se i pojam cilindra. Skup svih staza jednako udaljenih od osovine na svim pločama čine jedan **cilindar**.

Kapacitet diska predstavlja količinu podataka koja stane na disk. Mjeri se u bajtima. Ukoliko sve staze imaju jednak broj sektora, kapacitet je proizvod broja glava, cilindara (staza po ploči), broja sektora po stazi i broja bajtova po sektoru. U modernim diskovima, različite staze imaju različite brojeve sektora.

11.3. Blokovi i zone

Skupovi susjednih sektora nazivaju se blokovi (*block*) ili klasteri (*clusters*). Kako pokazuje Slika 119 blokovi se sastoje od više susjednih sektora lociranih na istoj stazi.

O grupiranju sektora u blokove ne odlučuje disk kontroler nego operativni sistem. Veličina blokova zavisi od izabranog datotečnog sistema. U idealnom slučaju će sadržaj datoteka biti zapisan u susjednim blokovima na stazi. U praksi će često sadržaj datoteka biti razbacan

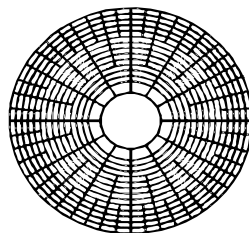


Slika 119 Grupiranje sektora u blokove

u međusobno udaljene blokove na disku. Ova pojava se naziva **fragmentacija** i dovodi do usporavanja pristupa datotekama.

Magnetna tehnologija stalno napreduje, pa je gustoća staza sve veća, što vodi efikasnijem korištenju magnetna. Prvi *hard* disk za personalne računare, proizveden 1980, ST-506 imao je kapacitet od 5 megabajta, sa 4 glave i 2601 sektora po ploči, uz brzinu pristupa 85 ms. U 2019. godini, ST14000VN0008 dostiže kapacitet od 14TB, uz prosječnu brzinu pristupa 4.16 ms, ima 16 glava, uz 1,7 milijardi sektora po ploči, a i manjih je dimenzija.

Radi povećavanja gustoće pakovanja sektora primjenjivane su razne metode. Uočeno je da unutrašnje staze, odnosno staze bliže centru, imaju manju dužinu pa se na vanjske staze stoga može smjestiti više sektora nego na unutrašnje. Zbog toga se uvodi tehnika zona na disku gdje se cilindri grupišu u zone iste gustoće (*ZBR - zone recording*) (Slika 120). Svi cilindri u istoj zoni imaju jednak broj sektora, ali cilindri iz različitih zona imaju različit broj sektora. Na taj način povećava se i kapacitet diska i brzina čitanja sa medija ali kontrolna elektronika ima složeniji zadatak.



Slika 120 Zonski zapis

Od 2014. godine gustoća zapisa je povećana uvođenjem zapisa oblika crijepa (*SMR - Shingled magnetic recording*) kod kojih se upis u jednu stazu preklapa dijelom i na dvije susjedne. To bi normalno usporavalo upis, jer se pri ažuriranju jedne staze moraju pročitati i obje susjedne staze, pa zatim upisati sve tri. No, pošto moderni diskovi imaju i veliku količinu RAM-a u kome će se nalaziti nedavno čitani podaci i specijalni softver, ovo usporenje se neće primijetiti.

Dalje povećanje kapaciteta će biti moguće povećanjem gustoće zapisa bita u samim sektorima. Ali, ako je manja površina koja se magnetiše, potrebni su materijali koji zahtijevaju jače magnetno polje da bi se upisali podaci. Pošto toliko magnetno polje na današnjim diskovima više nije moguće postići, u vrijeme pisanja ove knjige (jesen 2018) u razvoju su HAMR diskovi (*Heat assisted magnetic recording*) koji zagrijavanjem tačke upisa laserom promijene magnetna svojstva materijala, da omogućiti unos i slabijim poljem, a kada se tačka ohladi materijalu se vrate normalna magnetna svojstva i podaci ostaju permanentno upisani.

Iz ovog postaje jasno da formula za račun kapaciteta diska više nije tačna, jer broj sektora po stazi nije konstantan. Zbog toga, moderni diskovi uvode virtualnu geometriju diska koju koriste za komunikaciju sa operativnim sistemom, prema kojoj je broj sektora po stazi jednak za svaku stazu, a kontroler diska obavlja konverziju između fizičke i virtualne geometrije. Moderni diskovi pružaju dva režima adresiranja na disku: CHS (cilindar, glava i sektor) i LBA (logička adresa bloka). Kada se želi pristupiti sektoru na disku u CHS režimu, kontroleru se pošalju redni broj cilindra, glave i sektora unutar staze (na starijim diskovima stvarni brojevi, a na novijim virtualni brojevi). U LBA režimu se šalje samo jedna koordinata, apsolutni redni broj sektora na cijelom disku, a kontroler određuje stvarni broj sektora, cilindra i staze.

11.4. Klasifikacija diskova

Najvažnije tehničke karakteristike magnetskih diskova omogućavaju podjele diskova u razne kategorije.

Prema **načinu kretanja** glave diskove dijelimo na one sa fiksnim glavama (jedna po stazi) i sa pokretnom glavom (jedna po površini). Diskovi sa fiksnim glavama se odavno ne proizvode, zbog vrlo visoke cijene, a malog kapaciteta, ali postizali su daleko veće brzine od dominantnih diskova sa pokretnom glavom.

Po **zamjenjivosti** diskovi mogu biti zamjenjivi i nezamjenjivi. Nezamjenjivi diskovi se ne mogu vaditi u toku rada računara. Zamjenjivi se javljaju u dvije varijante: diskovi kod kojih je moguće uklanjanje samo sadržaja (diskete), ili diskovi koji dopuštaju uklanjanje uređaja bez prekida rada računara (USB diskovi).

Mehanizam glave omogućava klasifikaciju diskova na dva tipa. U prvom slučaju, magnetna glava dolazi u fizički kontakt sa površinom diska tokom operacija čitanja i pisanja (diskete). U drugom slučaju, (*hard disk*), glava za čitanje i pisanje je postavljena na fiksnu udaljenost iznad površine diska, što omogućava veće brzine rotiranja

Diskovi se mogu dijeliti i po **broju ploča** na diskove s jednom ili više ploča. Diskovi često sadrže više ploča koje se nalaze jedna iznad druge a sadrže i više ručica sa glavama za čitanje odnosno pisanje.

Pored ovoga, diskovi se mogu dijeliti i po **broju iskorištenih strana** svake ploče. Disk je jednostrani ako ima magnetni materijal i glavu za čitanje samo s jedne strane ploče, a dvostrani ako su magnetni materijal i glava za čitanje sa obje strane ploče.

11.5. Vrijeme pristupa disku

Prije čitanja i pisanja glava diska mora biti pozicionirana na željenu stazu i na početak željenog sektora na toj stazi. Za razliku od pristupa radnoj memoriji, za pristup podacima na disku potrebno je da prođe određeno vrijeme.

Vrijeme traženja je vrijeme potrebno da se glava diska pomjeri na zahtijevanu stazu. Ovo vrijeme se sastoji od dvije ključne komponente: vrijeme uspostavljanja, od izdavanja komande za transfer do početka kretanja glave i vrijeme koje je potrebno za prelazak do željene staze.

Kada je glava diska došla iznad željene staze, čeka se vrijeme potrebno da se početak željenog sektora poravna sa glavom. To vrijeme se naziva **rotaciono kašnjenje**. U najgorem slučaju, glava je došla iznad staze kada je željeni sektor upravo prošao, pa se mora sačekati cijela rotacija. U prosjeku se čeka pola rotacije. Diskovi rotiraju ugaonom brzinom koja se kreće od 300 rpm (*Revolutions Per Minute*) do 15,000 rpm, što vodi do rotacionih kašnjenja između 2ms i 100 ms.

Vrijeme prijenosa podataka na disk i sa diska zavisi od rotacione brzine i računa se po sljedećoj formuli:

$$T = \frac{b}{rN}$$

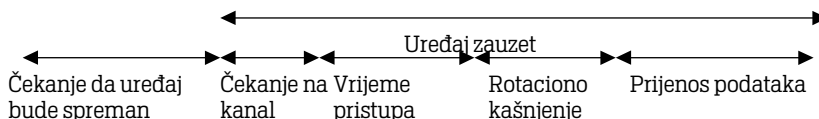
gdje je T - vrijeme prijenosa, b - broj bajta koji trebaju biti preneseni, N - broj bajta na stazi, r - rotaciona brzina. Kod tehnologije zona broj bajta po stazi je varijabilan a to komplikuje računanje.

Ukupno **prosječno vrijeme pristupa** može biti izraženo sa:

$$Ta = Ts + \frac{1}{2r} + \frac{b}{rN}$$

gdje je Ts prosječno vrijeme traženja.

Generalni vremenski dijagram dat je na slici Slika 121.



Slika 121 Vremenski dijagram pristupa disku

11.6.Formatiranje diska i pravljenje particija

Priprema diska za rad sastoji se od tri faze: formatiranje na niskom nivou, kreiranje particija i formatiranje particija. **Formatiranje na niskom nivou** je proces kod koga se na magnetni medij upisuju oznake koje predstavljaju granice staza i sektora. Na nekim vrlo starim diskovima (MFM, raniji SCSI) radio ga je kupac, ali se danas isključivo radi fabrički. U toku ovog formatiranja pripreme se mjesta za sektore, staze i međuprostor između njih. Sektor se sastoji od zaglavlja za njegovo prepoznavanje (*Preamble*), samog sadržaja i podataka za korekciju grešaka sektora (ECC) (Slika 122). Ovu strukturu vidi samo kontroler diska, prema računaru se proslijeđuje isključivo blok sa podacima.

Preamble	Podaci	ECC
----------	--------	-----

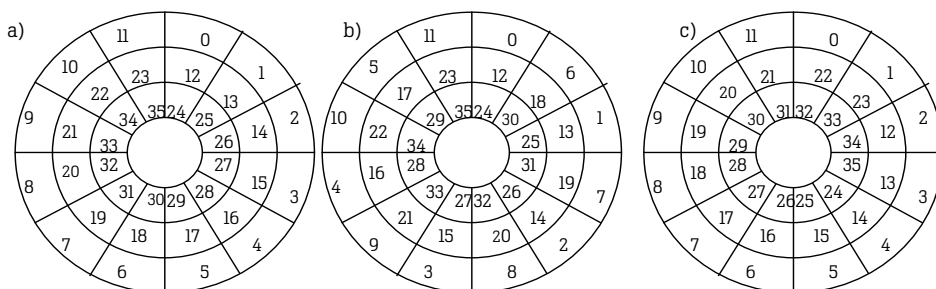
Slika 122 Sektor formatiran na niskom nivou sa redundantnim podacima

Naredni zadatak je dijeljenje diska na jednu ili više particija koje se ponašaju kao zasebni diskovi. Više particija se kreira kada se želi instalirati više operativnih sistema ili ako se žele razdvojiti sistemski podaci od korisničkih. Lista svih particija nalazi se na samom početku diska (sektor 0, staza 0, površina 0). Ovaj sektor se naziva **MBR**, glavni startni zapis (*engl. Master Boot Record*). Pri pokretanju računara, BIOS nakon inicijalizacije memorije pronade MBR na disku. Unutar MBR se nalazi kratki program koji pročita tabelu particija. Jedna među njima se zove **aktivna**. BIOS pročita MBR sa diska i pokrene izvršavanje u njemu programa, koji pročita prvi sektor aktivne particije. Taj sektor se zove **startni sektor** (*engl. boot sector*). U njemu se takođe nalazi mali program čijim pokretanjem započinje *bootstrap* rutina odnosno učitavanje operativnog sistema u memoriju. Na kraju MBR se nalaze tabela particija i signatura MBR. Tabela particija sadrži početnu i krajnju adresu svake particije, na PC računarima ima četiri elementa po 16 bajta.

Treća faza pripreme diska je **formatiranje particija**. U ovoj fazi se sektori koji pripadaju pojedinim particijama pune početnim vrijednostima koje su potrebne za realizaciju tabela za datotečne sisteme, ili particije za realizaciju virtualne memorije. Na primjer odredi se koji sektori u particiji će čuvati podatke o slobodnom prostoru, u kojima će biti sadržaj datoteka, a koji će govoriti o tome gdje se nalazi pojedina datoteka. Nekada se upiše i startni sektor.

11.7. Transformacija logičkih sektora u virtualne ili fizičke

Pored transformacije virtualnih koordinata diska (koristeći LBA broj ili cilindar/sektor/glava), u fizičke, koje obavlja disk kontroler, obavlja se i transformacija logičkog sektora u virtualni ili (na manjim i starijim diskovima) u fizičke. Ovu transformaciju obavlja datotečni sistem. Logički sektori se predstavljaju jednom koordinatom i broje se od početka particije na kojoj je datotečni sistem. Neka u narednim primjerima particija zauzima tri unutrašnje staze i svaka staza ima po 12 sektora. Ima više načina da se obavi transformacija logičkog u fizički sektor. Najjednostavnije je da se poredaju u jedan niz od logičkog sektora 0 svi sektori početne staze particije, pa svi sektori naredne staze itd. sve do zadnjeg sektora zadnje staze (Slika 123a). No, ponekad se radi poboljšanja performansi sekvencijalnog čitanja ova transformacija obavlja i drugačije.



Slika 123 Prosta transformacija, prepletanje i zakretanje

Kada se čita sektor po sektor, računar ima vremena da obradi primljeni sektor dok je glava diska u međuprostoru između sektora. Disk se u međuvremenu još uvijek okreće i na sporijim računarima može se desiti da je glava diska došla na naredni sektor prije nego je završen prijenos prethodnog sektora u bafere. Kada dođe zahtjev za čitanje narednog sektora, morala bi se sačekati cijela rotacija diska. Da bi se to izbjeglo, koristi se **prepletanje** (Slika 123b). U ovoj tehnici logički sektori se ne redaju sekvencijalno u fizičke, nego se preskače po jedan ili dva sektora. Na primjer, za disk čija staza ima 12 sektora, ako logički sektor broj 0 treba da se nalazi na fizičkom sektoru 0 određene staze 2, logički sektor broj 1 će se smjestiti na fizički sektor 2 te staze, a logički sektor broj 5 će se smjestiti u sektor 10 te staze. Zatim se popunjavaju preskočeni fizički sektori, pa će se logički sektor 6 smjestiti na fizički sektor 1 date staze, logički sektor 7 na fizički sektor 3 itd. Stoga kada je pročitana logički sektor 1, procesor obrađuje podatke dok je glava iznad logičkog sektora 7 i biće spreman da prima podatke prije nego glava diska dođe iznad logičkog sektora 2. Za čitanje cijele staze potrebne su dvije rotacije sa prepletanjem. Ako nema prepletanja, na sporom računar za čitanje cijele staze moglo bi čak biti potrebno onoliko rotacija koliko ima sektora u stazi. Međutim, na brzom računar, koji bi mogao obraditi podatke dok je glava između dva sektora, bila bi dovoljna jedna rotacija. Moderni procesori imaju dovoljno veliku brzinu pa je prepletanje suvišno.

Nekada se koristi i **zakretanje** (*skew*). Prilikom čitanja susjednih sektora koji se nalaze na različitim stazama (u navedenom primjeru nakon logičkog sektora 11 treba

preći na sektor 12), može se desiti da dok glava diska pređe na narednu stazu, uslijed rotacije diska, potrebiti sektor je već prošao i morao bi se sačekati cijeli krug da se on pojavi ispod glave. Radi ubrzanja sekvencijalnog čitanja, brojanje logičkog sektora po završetku staze se ne treba nastaviti od nultog sektora sljedeće staze, nego od onog fizičkog sektora koji bi bio na poziciji glave nakon što prođe vrijeme potrebno za njeno pozicioniranje na sljedeću stazu (Slika 123c).

11.8. Raspoređivanje zahtjeva za rad sa diskom

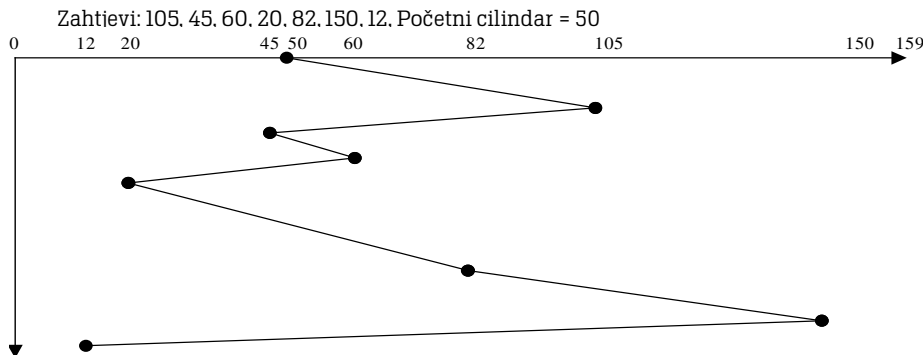
Dok disk obavlja transfer podataka, u međuvremenu procesi zahtijevaju nove podatke sa diska ili šalju prema disku. Svaki zahtjev sadrži informacije o tome da li je u pitanju operacija čitanja ili pisanja, poziciju bloka na disku, adresu memorijskog bafera gdje se nalaze podaci za slanje ili prijem i broj bajtova koje treba prenijeti. U multimedijalnim i sistemima u realnom vremenu zahtjevi mogu uključiti i rok kad trebaju biti ispunjeni. Ovi zahtjevi se moraju obaviti jedan po jedan, ali redoslijed opsluživanja ovih zahtjeva bitno utiče na ukupne performanse diska i na njega se može uticati. Postoji više algoritama za raspoređivanje zahtjeva prema disku: FCFS, SSTF, SCAN, C-SCAN, LOOK, S-LOOK C-LOOK, F-SCAN, višekoračni SCAN.

11.9. Raspoređivanje unutar jedne staze

Ako neki različiti zahtjevi za blokovima pripadaju istoj stazi, te je zahtjeve dovoljno sortirati u rastućem redoslijedu logičkog sektora unutar iste staze i opslužiti ih tim redoslijedom, da se svi zahtjevi mogu obraditi tokom jedne rotacije. U analizi daljnjih algoritama, svi zahtjevi za sektorima iz iste staze se svode na jedan zahtjev.

11.10. FCFS

Razmotrimo sljedeći primjer. Pretpostavimo da disk ima 160 cilindara i jednu glavu, i da imamo sekvencu zahtjeva za čitanjem cilindara 105, 45, 60, 20, 82, 150, 12. Pretpostavimo da je glava za čitanje/pisanje inicijalno postavljena na stazi 50. Šta bi se desilo da drajver diska opslužuje zahtjeve upravo tim redoslijedom, kao što je u najjednostavnijem algoritmu FCFS (*first come, first served*)? Ovaj algoritam, iako ravnopravno tretira sve zahtjeve, daje slabije performanse. Performanse zavise od pokretanja glave za čitanje i pisanje, a performanse će opadati ukoliko glava mora preći veći put.



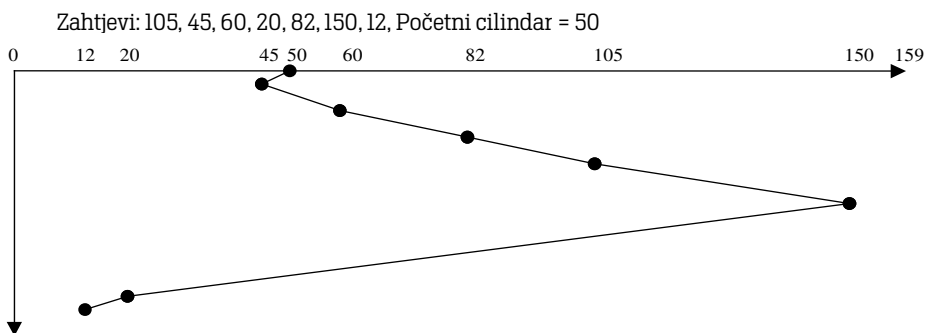
Slika 124 Algoritam FCFS

Slika 124 prikazuje dijagram kretanja glave diska, gdje horizontalna osa predstavlja cilindar, a vertikalna vrijeme.

U ovom primjeru glava za čitanje/pisanje će preći 438 cilindara. Zdravorazumski i poštenu algoritam očito ima loše performanse.

11.11. SSTF

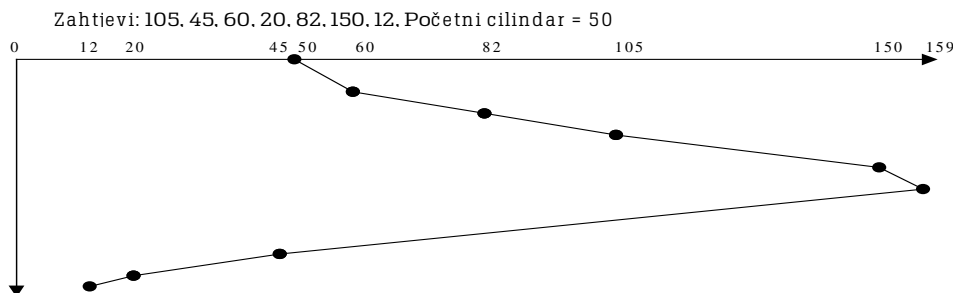
Ako se između pristiglih zahtjeva uvijek bira cilindar koji je najbliži trenutnoj poziciji glave, dobiva se SSTF algoritam (*shortest seek time first*). Postižu se bolji rezultati, za isti primjer kao kod FCFS, pređeni put je 248 cilindara (Slika 125). Pa ipak, lokalno najbliži cilindar ne znači najkraći ukupno pređeni put. Na slici se vidi da je glava diska preko cilindra 45 prešla dva puta. Drugo, ovaj algoritam, ima problem izgladnjivanja (*engl. starvation*). Ako stalno dolaze novi zahtjevi u red čekanja, glava se može zadržati dugo u jednoj zoni opslužujući zahtjeve za koje je potrebno malo kretanja, tako da se može desiti dugo čekanje na opsluživanje zahtjeva prema udaljenim cilindrima.



Slika 125 Algoritam SSTF

11.12. SCAN, LOOK i S-LOOK

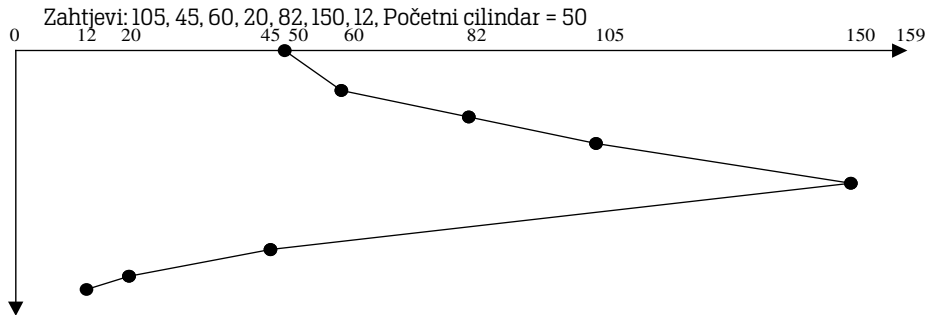
U velikim poslovnim zgradama i tržnim centrima u liftove ulaze novi ljudi na među-spratovima. Ako se lift kretao prema gore i ima još putnika u liftu u tom pravcu, nepošteno bi bilo da pridošlice promijene pravac lifta, nego oni moraju sačekati da lift završi sa penjanjem, pa tek onda početi spuštanje. SCAN algoritam zbog sličnosti sa ovim ponašanjem lifta se često naziva i elevator algoritam.



Slika 126 Elevator ili SCAN algoritam

Glava diska se naizmjenično pomiče od početnog cilindra na ploči ka krajnjem, pa onda od krajnjem ka početnom. Kada prelazi iznad pojedinih cilindara opslužuje zahtjeve prema njima. Ovaj algoritam rješava problem izgladnjivanja. Za spomenuti primjer, pređeni put je 256 cilindara (Slika 126).

Unapređenje SCAN algoritma je LOOK algoritam. Kod ovog algoritma se promjena pravca ne obavlja na krajevima ploče, nego kada su opsluženi svi zahtjevi za cilindrima u datom smjeru.

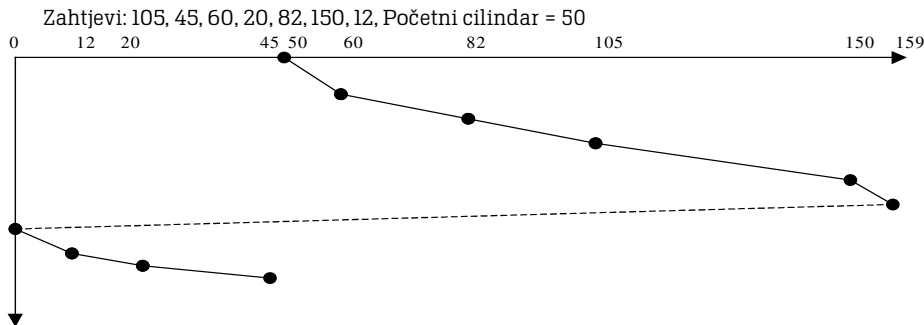


Slika 127 Algoritam LOOK

Za navedeni primjer LOOK algoritma, pređeni put je 243 cilindara (Slika 127).

U osnovnom LOOK algoritmu, početni smjer kretanja glave se preuzima od smjera u kome se glava diska kretala pri obradi ranijih zahtjeva. Varijanta S-LOOK određuje početni smjer da krene prema najbližem cilindru. Dalje se nastavlja kretanje u istom smjeru do najdaljeg zahtjeva, nakon čega se smjer mijenja.

11.13.C-SCAN i C-LOOK

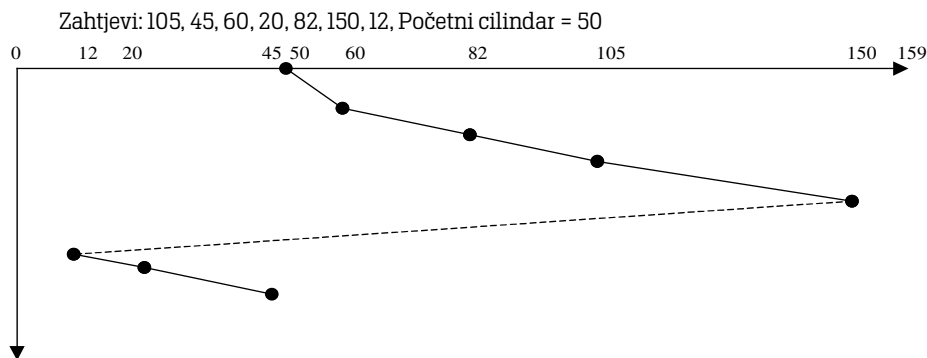


Slika 128 Algoritam C-SCAN

Kod klasičnog SCAN algoritma, zahtjevi za cilindrima koji su u sredini ploče će biti češće opsluživani nego zahtjevi prema cilindrima na krajevima diska. Kružni SCAN algoritam (C-SCAN), smanjuje favorizovanje centralnih cilindara. Kod ovog algoritma se zahtjevi opslužuju samo u jednom smjeru. Uzmimo za primjer da je taj smjer prema većim brojevima cilindra. Kad glava dođe do posljednjeg cilindra na ploči, vrati se na nulti, pri čemu tokom povratka ne opslužuje zahtjeve. Neki diskovi imaju mehanizam za brzo vraćanje glave. Uz raniji primjer, ako se

povratni put ne računa, (na nekim diskovima se može povratak realizovati dosta brzo), pređeno je 154 cilindra (Slika 128).

C-LOOK je varijanta algoritma LOOK koja opslužuje zahtjeve u jednom smjeru do posljednjeg zahtjeva u tom smjeru. Tada nastavlja od najdaljeg zahtjeva na drugoj strani ploče i ponovo opslužuje zahtjeve u istom smjeru. Ako se povratni put ne računa, CLOOK algoritam ima najkraći pređeni put (133 cilindra).



Slika 129 Algoritam C-LOOK

11.14. F-SCAN, F-LOOK i višestepeni SCAN

Iako u manjoj mjeri nego SSTF i kod varijanti SCAN algoritma može se desiti izgladnjivanje zahtjeva. Ako se glava diska nalazi iznad određenog cilindra i dok obrađuje zahtjev za sektorom koji pripadaju tom cilindru, pristižu novi zahtjevi za čitanje na istom cilindru. Glava diska će ostati zaustavljena na tom cilindru sve dok se ne završi serija zahtjeva za njim. Rješenje problema predstavljaju F-SCAN i N-step SCAN. Kod algoritama F-SCAN i F-LOOK postoje dva reda čekanja na zahtjeve diska. Dok se obrađuju zahtjevi za cilindrima u jednom redu čekanja mogu pristizati drugi zahtjevi, ali oni se ubacuju u drugi red čekanja. Zahtjevi u jednom redu čekanja se obrađuju SCAN ili LOOK algoritmima. Kada se svi obrade, onda se opslužuje drugi red čekanja, a prvi preuzima ulogu reda u koji se ubacuju u međuvremenu pristigli zahtjevi.

U N-step SCAN algoritmu, zahtjevi se ubacuju u centralni red čekanja. Od tih zahtjeva u centralnom redu se izabere maksimalno N zahtjeva prema osnovnom algoritmu raspoređivanja i obradi. Nakon toga se izabere i obradi narednih N zahtjeva iz centralnog reda čekanja.

11.15. Raspoređivanje diska u multimedijalnim sistemima i kod nemehaničkih diskova

Kod nemehaničkih (SSD, RAM disk) diskova vrijeme pristupa svakom sektoru je jednako. Sada pređeni put glave nema nikakvog udjela u vremenu pristupa, nego do izražaja dolazi raspoređivanje dirigovano vrstom aplikacije. Tako, npr. postoje i posebni algoritmi za raspoređivanje u multimedijalnim sistemima, poput video servera koji trebaju da emituju audio-video sadržaje sa što manje trzaja.

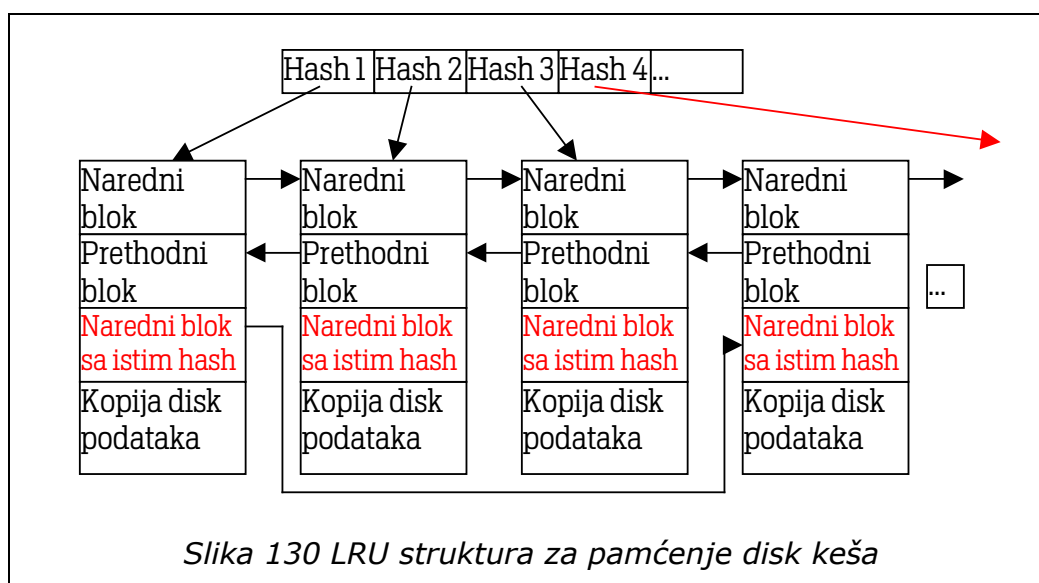
Jedan algoritam raspoređivanje zahtjeva na mehaničkim i nemehaničkim diskovima kod multimedijalnih sistema je **najraniji rok prvo**, *Earliest-Deadline-First* (EDF). Zahtjevi za disk blokovima se dopunjavaju podatkom u kojem trenutku je potrebno učitati taj blok. Redoslijed obrade zahtjeva se određuje prema roku kada je zahtjev potrebno ispuniti, tj. ranije potrebni zahtjevi se ranije obrađuju.

Kombinacijom EDF algoritma sa SCAN algoritmom dobiva se SCAN-EDF. Zahtjevi se sortiraju i opslužuju prema roku ispunjavanja. Ako ima više zahtjeva sa istim rokom, međusobno se raspoređuju po SCAN algoritmom.

Treći algoritam je prioritarno raspoređivanje. Kod ovog pristupa se redoslijed pristupa sektorima računa prema prioritetu procesa: prije će se pristupiti sektorima prioritarnijih procesa.

11.16. Keširanje

Ako računar raspolaže dovoljnim RAM-om najefikasnija tehnika za ubrzavanje pristupa disku je keširanje blokova. U ovom kontekstu, keš predstavlja skup blokova koji logički pripadaju disku ali se drže u RAM, jer je RAM brži i od mehaničkih i od SSD diskova. Prilikom obrade zahtjeva za čitanje provjeri se da li su traženi blokovi već u RAM kešu. Ako jesu, zahtjev za čitanje može se uraditi bez diskovnog pristupa. Ako blok nije u kešu, prvo ga je potrebno pročitati u keš a onda kopirati tamo gdje je potrebno, recimo, u korisnički bafer. Sljedeći zahtjevi za pristup tom bloku mogu se izvršiti iz keša. Pošto u kešu ima puno blokova, često i više hiljada, potrebno je brzo provjeriti da li je blok prisutan u kešu. Uobičajen način je da se napravi *hash* tabela. Iz rednog broja disk bloka izračuna se brzo neka vrijednost koja se zove *hash* (npr. ostatak rednog broja bloka pri dijeljenju sa 65537). Svi blokovi sa istom *hash* vrijednošću se smještaju u kratku povezanu listu, koja se sekvencijalno pretražuje. Prvi element te liste se lako nalazi iz *hash* vrijednosti.

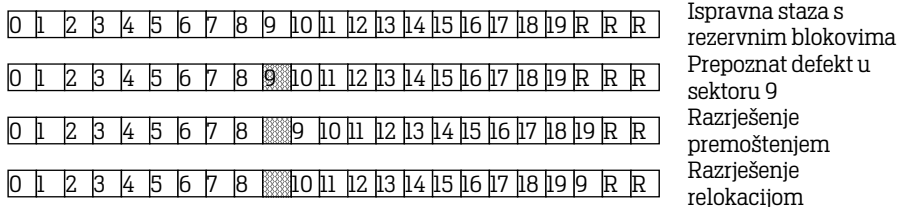


Pored pokazivača na naredni blok sa istom *hash* vrijednošću, lista blokova u kešu može imati i pokazivače na češće i rjeđe korištene blokove. Tako lista blokova može biti uređena po LRU redoslijedu, koji se (za razliku od LRU kod straničenja) može efikasno implementirati pošto su zahtjevi za čitanjem disk blokova daleko rjeđi od zahtjeva za čitanjem bajtova iz memorije, tada su blokovi koji su nedavno korišteni na vrhu liste a oni koji su davno korišteni na kraju liste.

11.17. Povećanje pouzdanosti

Povećanje pouzdanosti diskova može se postići na više načina, kao što su korekcionni kodovi u formatu niskog nivoa, mapiranje loših sektora, uvođenje RAID struktura itd.

Korekcionni kodovi (ECC) omogućavaju da se u slučaju neispravnosti jednog bita i dalje dobivaju ispravni podaci, odgovarajućim računom. To interno radi kontroler diska. Korekcionni kodovi su ugrađeni u sastavni dio sektora formatiranog na niskom nivou i nevidljivi su za operativni sistem.



Slika 131 Rezervni sektori

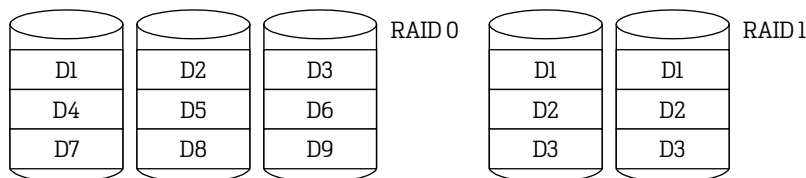
Veće greške se prepoznaju razlikovanjem upisane i naknadno pročitane vrijednosti. Ako se uoči takva razlika, sektor se proglašava neispravnim i treba ga izbaciti iz upotrebe. Prije formatiranja particije, nekoliko sektora na kraju svake staze se ostavi za rezervu. Kada je uočeno pisanje sektora u defektni sektor, posao se ponavlja aktiviranjem rezervnog sektora. Sljedeći primjer (Slika 131) pokazuje dva načina mapiranja lošeg sektora u rezervni. Prvi je razrješenje premošćenjem, uz preskakanje defektnog sektora. Ovaj način zahtijeva kopiranje svih sektora u stazi iza defektnog za jedan sektor naprijed ali je kasnije čitanje staze ostalo brzo. Drugi način je smještanje sektora u rezervni sektor, no ubuduće se ta staza neće moći čitati u toku jedne rotacije diska. Loše sektore može zamijeniti disk kontroler ili OS. U slučaju da to radi kontroler, on mijenja preambule sektora na stazi, dok operativni sistem, pravi drugačije tablice, često definisan datotečnim sistemom.

11.18. RAID strukture

Poboljšanje performansi i pouzdanosti dodatno se postiže korištenjem više diskova koji rade nezavisno i paralelno. Sa više diskova, odvojeni U/I zahtjevi se mogu obraditi istovremeno, sve dok se podaci koji se zahtijevaju nalaze na odvojenim diskovima. Pored performansi, korištenjem višestrukih diskova povećava se pouzdanost, ukoliko se na različitim diskovima nalazi više kopija podataka ili pomoćnih podataka preko kojih se mogu rekonstruisati izgubljeni podaci. Postoji nekoliko standarda koji omogućavaju definisanje jednog logičkog od više fizičkih diskova, koji se nazivaju RAID (*Redundant Array of Independent Disks*). Iako se za različite konfiguracije ovih diskova koristi termin "nivo" (od 0 do 6 i kombinovani), nije riječ o hijerarhiji ili verzijama standarda, nego samo o različitim konfiguracijama. Zajedničko za sve RAID nivoe je da se sastoje od više fizičkih diskova koji se vide kao jedan logički disk i da se podaci raspoređuju između različitih fizičkih diskova. Svi RAID nivoi osim RAID 0 imaju na nekom od diskova dodatne informacije koje omogućavaju obnovu podataka u slučaju pada nekog od diskova.

11.19.RAID 0

RAID 0 nema dodatne informacije za obnovu podataka u slučaju pada. Koristi se kada je potrebno povećati kapacitet i brzinu rada diskova (npr. u trenutku pisanja ove knjige diskovi od 32 terabajta još nisu bili u prodaji, ali se mogu nabaviti četiri diska od 8 terabajta i povezati u jedinstveni logički disk).



Slika 132 RAID 0 i RAID 1

Podaci u RAID 0 se raspoređuju na različite diskove. Svaki fizički disk može imati svoj red čekanja na zahtjeve. Ti zahtjevi mogu biti obrađeni nezavisno, pa se postižu bolje performanse u odnosu na jedan disk većeg kapaciteta. Svaki fizički disk je podijeljen na odsječke, koji predstavljaju skupove disk sektora. Odsječci se redom postavljaju na svaki od fizičkih diskova. Npr., ako je RAID 0 sastavljen od četiri fizička diska, logički odsječci 0, 4, 8, 12... će se smjestiti na jedan disk, odsječci 1, 5, 9, 13... na drugi, odsječci 2, 6, 10, 14... na treći a odsječci 3, 7, 11, 13,... na četvrti fizički disk. Transformacija logičkog u fizički sektor će u ovoj konfiguraciji dobiti još jednu koordinatu: sektor se transformiše u četvorku (disk, cilindar, glava, sektor). Pošto nema nikakvu redundanciju, otkaz bilo kog diska znači gubitak svih podataka.

11.20.RAID 1

Kod RAID 1 svi diskovi imaju identičan sadržaj. Koristi se izraz “ogledalo” (*engl. mirroring*). Svaki upis sektora na jednom disku reflektuje se na drugi disk (ili diskove) na istom mjestu sa istim podacima. Ovaj upis se može uraditi specijalnim disk kontrolerom paralelno, pa se ne gubi previše vremena za pisanje. RAID 1 ubrzava čitanje jer se zahtjev može proslijediti bilo kom diskova koji sadrže iste podatke. Time se može birati među diskovima koji će imati manje vrijeme traženja, ili podijeliti redove čekanja između različitih diskova. Pouzdanost je vrlo visoka. U sistemu sa n diskova, dopušteno je da ispadne čak $n-1$. Mana RAID 1 konfiguracije je cijena. Ako je RAID 1 sastavljen od 2 diska stvarna iskorištenost kapaciteta je 50%, a ako je sastavljen od 4 diska svega 25%.

11.21.RAID 2

U ovoj konfiguraciji odsječak je velik jedan bit. RAID 2 dijeli pojedinačne bitove podataka na odvojene fizičke diskove, što bi značilo da se svaka četverobitna riječ podataka razdvaja na 4 odvojena fizička diska sa podacima. Pored njih, potrebna su još 3 diska za korekcione Hamingove kodove. U slučaju pada jednog od diskova, iz približne vrijednosti kombinacije preostalih bitova i korekcionih kodova, može se rekonstruisati vrijednost tog bita. RAID 2 se ne koristi u praksi, jer je preskup. On koristi mnogo diskova. Broj diskova za paritete n za m diskova sa podacima, se dobija po iz nejednačine $2^n \geq m + n + 1$. Osim toga, oni moraju biti savršeno vremenski sinhronizovani.

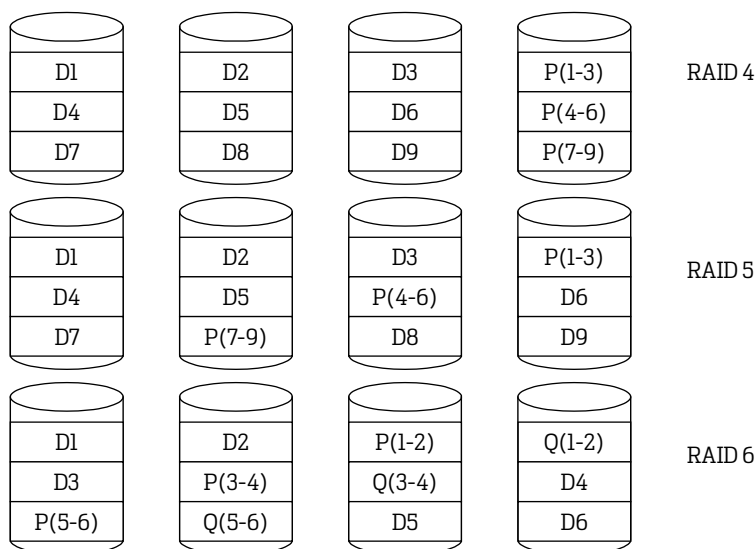
11.22. RAID 3

RAID 3 zahtijeva samo jedan dodatni disk, bez obzira koliko ima diskova s podacima. U ovoj konfiguraciji odsječak je veličine bajta. Na dodatnom disku se čuva bajt pariteta. U slučaju pada jednog diska, podaci se rekonstruišu uz pomoć diska pariteta i preostalih diskova sa podacima. Funkcija pariteta koja se koristi u RAID 3, 4 i 5 je ekskluzivna disjunkcija. Ova binarna funkcija ima vrijednost 1 ako su dva bita koja se uspoređuju međusobno različita, a 0 ako su ta dva bita međusobno jednaka. Na primjer, paritet između binarnih brojeva $A=11110010$ i $B=00100101$ je

A	1	1	1	1	0	0	1	0
B	0	0	1	0	0	1	0	1
$A \oplus B$	1	1	0	1	0	1	1	1

U slučaju da je podatak A izgubljen, on se i dalje može rekonstruirati iz podatka B i podatka $A \oplus B$ tako što važi $(A \oplus B) \oplus B = A$. Ako je podatak B izgubljen a podatak $A \oplus B$ i podatak A i dalje imamo, podatak B se dobiva iz $(A \oplus B) \oplus A = B$. Paritet preko ekskluzivne disjunkcije se može generalizovati na proizvoljan broj diskova.

RAID 3 je pogodan za spajanje uređaja koji mogu adresirati podatke na nivou bajtova, kao što je *Non-Volatile Memory*. Na klasičnim diskovima, koji adresiraju na nivou sektora, performanse su jako loše. Iskorištenost prostora je bolja. Za RAID 3 (kao i za RAID 4 i RAID 5) u sistemima sa 4 diska, 75% otpada na podatke, a 25% je redundantno.



Slika 133 RAID 4, i RAID 5, i RAID 6 redoslijed odsječaka s paritetima

11.23. RAID 4

RAID 4 koristi posvećeni disk pariteta kao i RAID 3, ali podatke po diskovima podataka ne raspoređuje po pojedinačnim bajtovima, nego po odsječcima veličine sektora ili većih disk blokova. To omogućuje lako uključivanje klasičnih diskova u

RAID konfiguraciju. Za proizvoljnih N odsječaka, raspoređenih na N diskova, dovoljan je jedan blok pariteta veličine odsječka. Čitanje ima dobre performanse, ali je pisanje sporije nego RAID 1 ili RAID 0, jer u svakom ciklusu upisa učestvuje i disk pariteta, koji postaje usko grlo.

11.24.RAID 5

Unapređenje RAID 4 konfiguracije postiže se sa RAID 5. Razlika između ova dva pristupa je što RAID 5 ne čuva sve odsječke sa paritetima na istom disku, nego se neki odsječci čuvaju na jednom, a neki na drugim diskovima (Slika 133). Tipično se koristi cirkularna šema alokacije. Uz RAID 1, konfiguracija RAID 5 je najpopularnija jer ima dobar odnos cijene, brzine, pouzdanosti i paralelizma.

11.25.RAID 6

Ukoliko se koriste prethodno opisane konfiguracije (osim RAID 1 u konfiguraciji sa tri i više diskova) svi podaci se gube ako otkazu dva diska. Konfiguracija RAID 6 je dizajnirana da izdrži ispad 2 diska, ali zahtijeva pored kapaciteta za podatke i dva dodatna diska. I ova konfiguracija dijeli diskove na odsječke. Kod nje se koristi računanje dva različita odsječka sa paritetima, koji se distribuiraju po različitim diskovima. Za podatke A i B, prvi paritet je ekskluzivna disjunkcija kao i kod RAID 3, RAID 4 i RAID 5.

$$P = A \oplus B$$

Drugi paritet se može računati kombinacijom ekskluzivne disjunkcije i funkcije koju ćemo nazvati $gm2$, što je skraćeno od Galoovo množenje sa 2.

$$Q = A \oplus gm2(B)$$

Ako nad osmobarbitnim binarnim brojem čije su cifre $b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$, primijeni funkcija $gm2$, rezultat će biti binarni broj čije su cifre $b_6 b_5 b_4 b_3 \oplus b_7 b_2 \oplus b_7 b_1 b_0 \oplus b_7 b_7$. Sljedeći primjer računa ova dva pariteta.

A	1	0	0	1	1	1	0	1
B	1	1	0	1	0	1	0	0
$P = A \oplus B$	0	1	0	0	1	0	0	1
$gm2(B)$	1	0	1	1	0	0	1	1
$Q = A \oplus gm2(B)$	0	0	1	0	1	1	1	0

U slučaju gubitaka podatka A ili B, uz pomoć diska koji čuva podatak P obnavljaju se podaci kao kod RAID 3, RAID 4 ili RAID 5. Na primjer ako su i dalje ispravni podaci A i P, podatak B se dobiva kao $B = A \oplus P$.

Ako se pokvario disk sa podatkom A kao i disk sa podatkom P, i dalje se može rekonstruirati A iz podataka B i Q računajući $A = Q \oplus gm2(B)$

Ako se pokvario disk sa podatkom B kao i disk sa podatkom P, i dalje se može rekonstruirati B iz podataka A i Q računajući $V = A \oplus Q = gm2(B)$. Pošto je $gm2$ bijektivna funkcija, za izračunatu vrijednost V može se iz tablice očitati vrijednost njene inverzne funkcije i na taj način rekonstruirati vrijednost A.

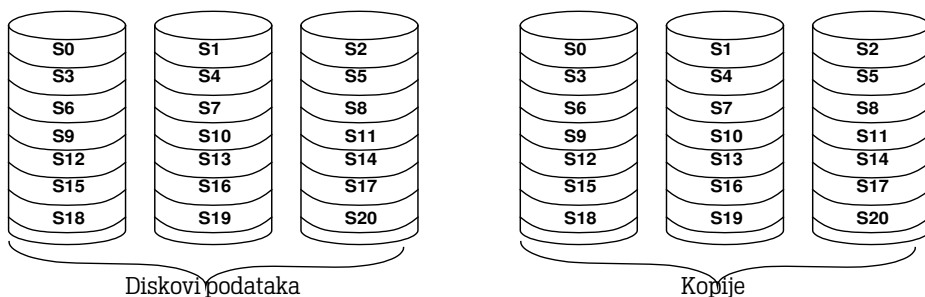
Najsloženiji slučaj je ako su se pokvarili diskovi sa podacima A i B, a ispravni su podaci P i Q. Tada se najprije izračuna $F = P \oplus Q = B \oplus gm2(B)$. Pokazuje se da je i ovo preslikavanje bijektivno, pa se iz prethodno pripremljene tablice može saznati kojoj

vrijednosti F odgovara koja vrijednost B. Nakon ovoga se iz vrijednosti P i B može dobiti vrijednost A.

Ako u RAID 6 učestvuje više diskova, npr. 6, funkcije pariteta nad podacima A, B, C i D su $P=A\oplus B\oplus C\oplus D$ i $Q=A\oplus \text{gm2}(B)\oplus \text{gm2}(\text{gm2}(C))\oplus \text{gm2}(\text{gm2}(\text{gm2}(D)))$. Proces obnove podataka je složeniji ali i dalje moguć bez obzira na to koji diskovi su ispali.

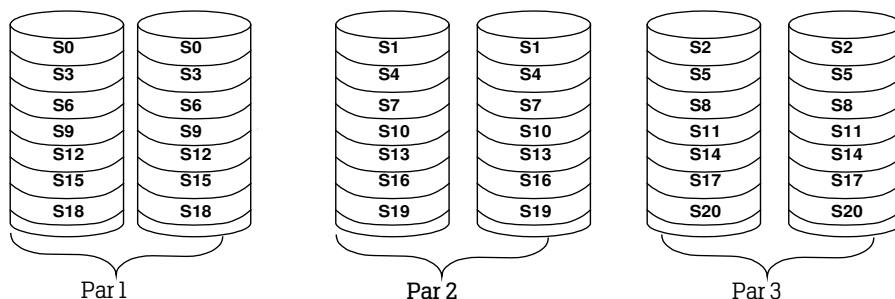
11.26. Hibridni RAID 0+1, 10 i 50

Kada je pouzdanost važnija od cijene (koja inače stalno pada) koriste se kombinovane RAID šeme. Najčešće se kombinuju RAID 0 i RAID 1 sa dva različita redosljeda ovih šema.



Slika 134 RAID 0+1

RAID 0+1 se pravi tako što poveže skup diskova sa podacima razdvojenim na odsječke na N različitih diskovima bez redundancije, kao što radi RAID 0. Kada je postignut željeni kapacitet i brzina, onda se poveća pouzdanost kupovinom još N diskova povezanih na isti način. Zatim se cijeli prvi skup diskova duplicira na drugi skup kao RAID 1 (Slika 134).

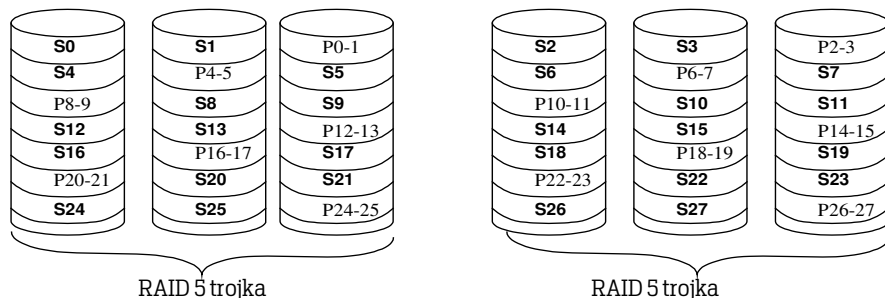


Slika 135 RAID 10

U RAID 1+0 ili RAID 10 radi se obrnutim redosljedom. Ovdje se najprije svi diskovi spoje u parove, kao kod RAID 1. Zatim se podaci dijele na odsječke i naizmjenično raspoređuju na parove. (Slika 135). Sistem se odlikuje velikom pouzdanošću, jer dopušta ispad više diskova pod uslovom da ostane bar jedna kopija.

Na sličan način se mogu kombinovati RAID 5 i RAID 0. Ova konfiguracija se zove RAID 50. Diskovi se grupišu u trojke, unutar kojih odsječci koriste RAID 5. Prva dva

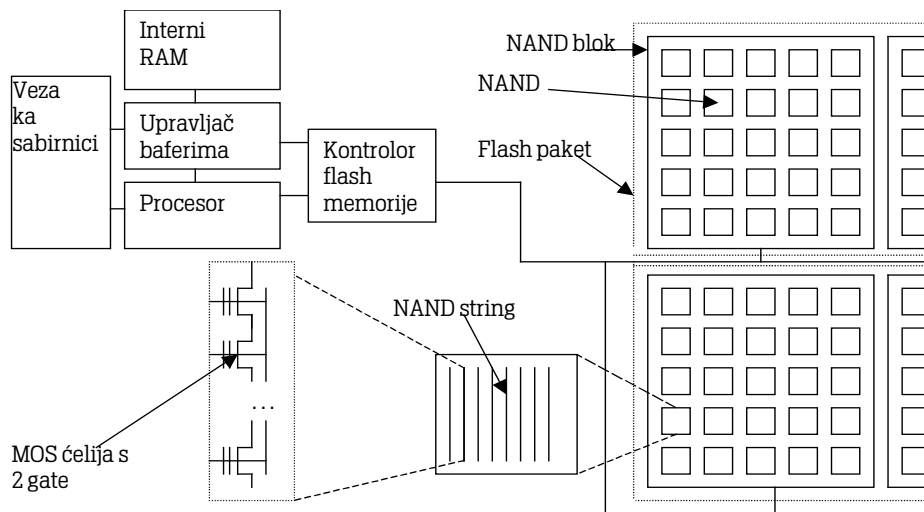
odsječka cijelog sistema se smjeste u prvu trojku, naredna dva u drugu, naredna dva u treću i tako ciklično. Time se poboljšavaju performanse RAID 5 konfiguracije.



Slika 136 RAID 50

11.27.SSD diskovi

Klasični tvrdi disk zasnovani su na kombinaciji mehaničkih i elektronskih komponenti. Mehanička komponenta je najuže grlo performansi modernih računarskih sistema. Stoga se u novije vrijeme magnetni diskovi zamjenjuju poluprovodničkim.



Slika 137 Struktura SSD diska

Jedna od najpopularnijih tehnologija za realizaciju čisto elektronskih diskova je SSD (*Solid-State Drive*). Bez mehaničkih dijelova povećava se brzina pristupa, izostaju vibracije i buka, smanjuje se potrošnja energije i zagrijavanje. Pojam *solid-state* označava upotrebu isključivo elektronskih elemenata i sklopova, a time je i način zapisa na ovim uređajima potpuno drugačiji od onog na klasičnim tvrdim diskovima. Algoritmi za optimizaciju kretanja glave i rotirajućih ploča, kao što su elevator, ili preplitanje su potpuno irelevantni, jer je vrijeme pristupa jednako, bez obzira kojem se sektoru pristupa

Za zapis informacija se koriste dvije vrste memorija, NAND *flash* i DRAM. Princip NAND *flash* je MOSFET s 2 gate, od kojih je jedan izolovan pa čuva podatke godinama.

SSD pohranjuju podatke u *flash* memorijske ćelije koje su raspoređene u nizove po 32-128 bita. Nizovi se smještaju u stranicama tipične veličine od 4 do 16 K. Stranice se grupišu u blokove obično 128 do 512 stranica, npr. 512 K po bloku. Blokovi se dalje grupišu u pakete. NAND memorijske ćelije mogu se direktno upisati samo kada su prazne (Slika 137). Ako one sadrže podatke, sadržaj mora biti izbrisan prije pisanja. Operacija SSD pisanja može se učiniti na jednoj stranici, ali zbog hardverskih ograničenja, naredbe brisanja (tj. postavljanja svih bitova na 1) uvijek utiču na cijeli blok. Stoga je pisanje podataka na prazne stranice na SSD vrlo brzo, ali znatno usporava ako podaci na stranicama već postoje, jer se prilikom izmjene podataka mora pročitati blok, obrisati pa zatim upisati nova vrijednost.

Zbog načina kako datotečni sistemi obrađuju operacije brisanja, označavanjem blokova podataka kao nekorištenih, SSD sam po sebi ne zna koji su sektori doista u upotrebi a koji se mogu smatrati slobodnim prostorom. Za mehaničke diskove nema razlike u pisanju preko praznog sektora ili sektora u kome se nalaze neki podaci, premda nekorišteni. Kod SSD diskova proces pisanja bi zahtijevao ciklus čitanja bloka u interni RAM keš, brisanja bloka u fleš memoriji, izmjene podataka u RAM kešu i upisa novih podataka. Ova se operacija mora obavljati nad cijelim blokom, iako je mijenjana samo jedna stranica.

Optimizacija operacije upisa je uvedena naredbom *trim*, koju drajver operativnog sistema šalje SSD kontroleru, obavještava SSD kontroler da određene stranice nisu u upotrebi, na primjer nakon brisanja datoteke. Kada kontroler zaključi da cijeli blok nije u upotrebi u budućim operacijama pisanja on ne mora da čita sadržaj tog bloka prije njegovog pisanja.

Kod *hard* diskova vrijeme života je vezano uglavnom za mehaničko habanje, pa na njega utiču čitanje, pisanje i prazan hod. Kod SSD diskova vrijeme života se mjeri se brojem ciklusa brisanja. Stoga drajveri u samom disku ili na operativnom sistemu, treba i da paze da je broj brisanja na svakom bloku uravnotežen da se uređaj ne pokvari prerano. Na primjer, datoteke povremeno treba prebacivati s jednog mjesta na disku na drugo, a posebno pažnju obratiti da se ne fiksira na jedno fizičko mjesto datoteka koja realizuje virtualnu memoriju, jer se ona mnogo čita i piše.

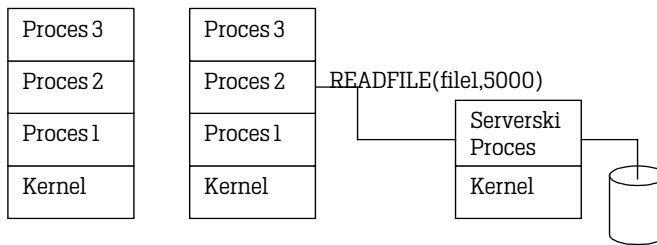
Pošto su magnetni diskovi su još uvijek jeftiniji, a SSD diskovi imaju daleko brže vrijeme pristupa, prave se hibridni diskovi. Hibridni disk koristi mali SSD kao bafer za veći magnetni disk. Izmijenjeni blokovi unutar SSD diska se upisuju na magnetni disk na bazi vremena koliko su dugo na SSD dijelu diska i kada se najavi gašenje.

11.28. Pristup diskovima

Diskovi se povezuju sa računarom na tri osnovna načina. Prvi način je **direktni priključak diska na računar** i pristupa im se koristeći sabirnice (npr., IDE, USB, ATA, SATA, SCSI, FireWire) i kontrolere periferijskih uređaja. Kada operativni sistem želi da čita ili piše podatke na ovako priključenim diskovima, on izvršava uobičajene instrukcije za ulaz/izlaz kao što je *out* za U/I mapirane periferijske uređaje. Koje

vrijednosti slati na koji *port* u kojem vremenskom trenutku zavisi od vrste diska i inteligencije njegovog kontrolera.

Mrežni priključak omogućava da jedan računar (zove se klijent) pristupa disku drugog računara. Koriste se protokoli bazirani na RPC (poziv udaljene procedure). Na klijentskom računaru izvršava se program za preusmjeravanje (*redirector*) koji kada očekuje pristup dijelu datoteke, pakuje zahtjev u mrežni paket i pošalje ga programu na drugom računaru koji se zove server. Server pronađe na svom lokalnom disku dio te datoteke i šalje ga nazad klijentu (Slika 138).



Slika 138 Mrežni priključak

Postoje posebni protokoli za ovu svrhu, kao što su NFS za Unix sisteme ili CIFS (ranije SMB) za Windows sisteme. Mrežno dijeljenje diskova omogućava da više korisnika na različitim računarima pristupa zajedničkim podacima, ali na svakom od njih mora postojati operativni sistem s realizovanim datotečnim sistemom.

Ukoliko je potrebno dijeljenje diskova između više računara, ali da se podacima na takvim diskovima pristupa na nivou bloka, a ne da se oslanja na postojeći datotečni sistem, koristi se treći pristup, **mreže za smještaj** (*storage networks*). Riječ je o posebnim, autonomnim diskovima koji se priključuju na računarsku mrežu, a ne ugrađuju ni u jedan računar. Koristeći posebne adaptere, pristup mrežnom disku je sličan pristupu lokalnom disku. S obzirom da diskovi ne izvršavaju druge aplikacije, za razliku od servera, postižu se bolje performanse od mrežnog priključka. Ovakva konfiguracija se koristi kada se želi smanjiti obim posla na administriranju pojedinačnih računara i datotečnih sistema na njima. Na primjer, ovako postaje moguće podići operativni sistem sa diska u mreži, pa se on ne mora instalirati na svaku radnu stanicu. Također, u slučaju kvara serverskog računara, drugi računar može preuzeti njegovu funkciju bez otvaranja računara i prebacivanja diska u njega.

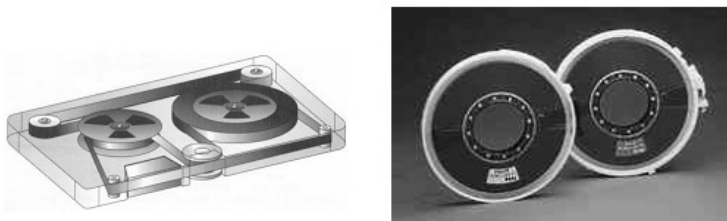
11.29. Izmjenjivi mediji

Diskovi sa izmjenjivim medijem zahtijevaju ljudsku intervenciju da bi se pristupilo podacima. Podaci se nalaze na mediju koji se može izvaditi iz uređaja i tako postići praktično neograničen kapacitet i prenosivost između različitih računara. Mediji se prave ili se pravili od četiri vrste materijala: magnetni (trake, diskete, izmjenjivi tvrdi diskovi), papirni (optički prepoznatljiv simbol, bušene kartice i bušene trake), poluprovodnički (stikovi, SD kartice) i optički (CD, DVD).

Kada se ovi mediji koriste za pisanje, operativni sistem treba paziti da korisnik može da ih iskopča iz računara kada to poželi. Većina modernih operativnih sistema

ne upisuje podatke odmah na diskove, nego ih čuva u memoriji, a zatim primjenjuje neki od algoritama za pristup disku kada je procesor manje opterećen. Ako se podaci namijenjeni izlazu nisu na vrijeme upisali, nego na primjer čuvali u baferima, podaci će biti izgubljeni ako se medij izvadi prije snimanja. Stoga, operativni sistem treba na izmjenjive medije da piše podatke što prije moguće. Ako to ne može biti slučaj, jer bi to drastično usporilo operativni sistem, treba tražiti od korisnika da najavi vađenje izmjenjivog uređaja. Ova akcija se zove demontiranje ili sigurno uklanjanje.

11.30. Magnetni izmjenjivi mediji

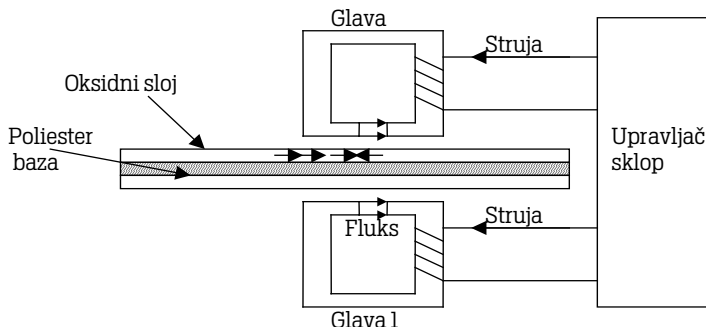


Slika 139 Magnetna traka u koturu i kaseti

Magnetne trake se koriste kao jeftini medij velikog kapaciteta. Dva tipa ovog medija su koturasti (koji se koristio na *mainframe* računarima, danas napušten pristup) i kasete (*cartridge*). U drugom slučaju traka je u kaseti uz korištenje standarda kao što su QUIC, DAT, DLT pa čak i VHS. (Slika 139).

Jeftini kućni računari iz perioda 1976-1990 su koristili audio kasete, uz jednostavni konvertor zvuka iz kasetofona u digitalni format. Na današnjim kasetama postižu se kapaciteti od više terabajta. Pristup podacima je isključivo sekvencijalni, pri čemu neki formati na isto mjesto na magnetnoj traci smještaju jedan bit ili po devet bita istovremeno. S obzirom na sporo vrijeme pristupa, magnetne trake se uglavnom koriste za rezervne kopije podataka.

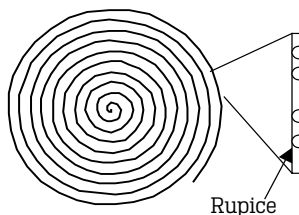
Drugi izmjenjivi magnetni medij je flopi disk. Sve do oko 1998. godine bili su dominantni izmjenjivi medij na personalnim računarima. Zbog sporosti i malog kapaciteta (varirao od 75K do 240 M) na današnjim personalnim računarima se rijetko ugrađuju, a jedina preostala namjena im je za podizanje operativnog sistema u situaciji kada to nije moguće uraditi s drugog uređaja zbog nedostatka drajvera. Proizvodili su se u raznim veličinama, mjerenim u inčima: 2, 2.5, 3, 3.5, 5.25 i 8". Flopi disk i magnetni hard disk imaju u suštini isti princip rada. I kod flopi diskova postoji jedna ili dvije glave koje se kreću preko površina koje su podijeljene na staze ili sektore. Sektori se sastoje od pojedinačnih bajtova i bitova. Bitovi na flopi diskovima se upisuju koristeći modifikovanu frekventnu modulaciju. Ovo kodiranje se bazira na kombinacijama jačeg (S) i slabijeg (W) magnetnog fluksa. Na taj način, binarno 1 se kodira kao slabiji signal iza koga slijedi jači, dok se binarno 0 kodira na jedan od dva načina: kao jači pa slabiji signal (ako je prethodni bit bio 0) ili kao produženi slabiji (ako je prethodni bit bio 1) (Slika 140).



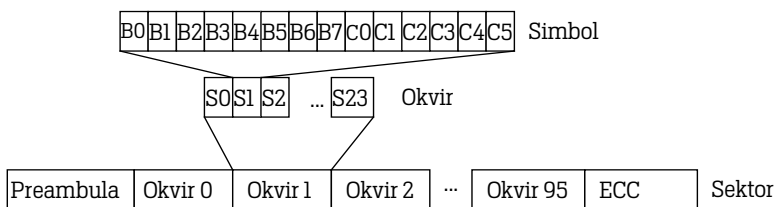
Slika 140 Princip rada floppy diska

11.31.Optički mediji CDROM, DVD

CD (engl. *Compact Disk*) je inicijalno dizajniran kao alternativa gramofonskim pločama za čuvanje digitalizovane muzike. Kasnije je upotrebljen i u računarima za čuvanje podataka koji se ne mijenjaju, najčešće za enciklopedijske podatke i instalaciju softvera. Sadržaj CD-a nije podijeljen na staze i sektore, nego samo na sektore. Sektori su poredani sekvencijalno u jednu spiralu, pri čemu je njihova veličina 2352 bajta. Obični CD se priprema tako što laser buši rupe na *master* disku pravljenoj od specijalnog stakla. Kalup je napravljen sa zaprekama gdje su rupe. Smola izlivena unutra ima isti uzorak rupe kao stakleni disk. Na vrh smole se stavi aluminijum. Rupice (udubljenja) i ravnine (neizgoreno područje) su raspoređeni u spiralama (Slika 141). Laser čita rupice i ravnine i prepoznaje ih kao bitove (0 i 1).



Slika 141 Struktura CD-a



Slika 142 Struktura sektora na CD

Pošto je na malu površinu nabijeno puno podataka, potrebno je poboljšati mogućnost korekcije grešaka na CD. Svaki bajt (8 bita podataka) se na ovom disku čuva kao 14 bitni simbol sa 6 bita za korekciju greške. Takva 42 simbola čine okvir, a grupa od 98 okvira čini CD-ROM sektor. Na kraju svakog sektora se nalaze dodatni korekcionni kodovi, a na njegovom početku preambula za prepoznavanje početka sektora. Logička struktura CD sektora prikazana je na slici Slika 142.

Početni trošak pravljenja CD-a je visok zbog potrebe za pravljenjem master CD-a, ali su kasniji štampani CD-ovi relativno jeftini. Za pravljenje manjih serija, pogodniji

su **upisivi CD**. Mogu se praviti s jednim ili više upisivanja. Prilikom upisa laser spaljuje tačke na sloju iznad reflektivnog zlatnog ili aluminijskog sloja. Pri kasnijem čitanju laserski zrak se reflektuje samo na nezatamnjanim mjestima i tako se razlikuju 0 i 1.

Inicijalno je namijenjen za digitalizovane filmove, DVD ima isti dizajn kao CD sa poboljšanjima uz veći kapacitet. Veći kapacitet u odnosu na CD postignut je manjim rupama, užom spiralom i upotrebom crvenog lasera. Izrađuje se u četiri varijante: jednostrani - jednoslojni (4.7 GB), jednostrani - dvoslojni (8.5 GB), dvostrani - jednoslojni (9.4 GB) i dvostrani - dvoslojni (17 GB). Prelaskom na još manje rupe, širinu spirale i laser ljubičaste svjetlosti, pojavio se optički uređaj s još većim kapacitetom. To je Blu Ray. Na jedan ovakav disk staje normalno 25 gigabajta, a kada se koriste četiri sloja, kapacitet raste sve do 128 gigabajta.

11.32. Izmjenjivi poluprovodnički mediji

Jedna od najčešćih primjena USB pruključka su džepni poluprovodnički diskovi. Diskovi se sastoje od SSD memorije i mikrokontrolera koji obezbjeđuje komunikaciju računara sa sadržajem diska koristeći serijsko slanje paketa sa diskovnim sadržajem.

11.33. Zadaci

Zadatak 1. Klasični disk se sastoji od 6 glava, 1360 cilindara po ploči i 128 sektora po traci. Formula za IDE diskove za konverziju iz CHS u LBA glasi

$$LBA = (C \cdot HPC + H) \cdot SPT + S - 1$$

Koja će LBA biti za cilindar 20, glavu 3, sektor 100? Izvedite formulu za konverziju iz LBA u CHS.

Rješenje: $LBA = (20 \cdot 6 + 3) \cdot 128 + 100 - 1 = 15843$

$$C = \left\lceil \frac{LBA}{SPT \cdot HPC} \right\rceil, H = \left\lceil \frac{LBA}{SPT} \right\rceil \bmod HPC, S = (LBA \bmod SPT) + 1$$

Zadatak 2. Disk koristi zonski zapis. Sastoji se od 6 glava i 1360 staza po glavi. Staze 0-339 imaju po 111 sektora, staze 340-679 imaju po 113 sektora, staze 680-1019 imaju po 115 sektora, staze 1020-1359 imaju po 117 sektora. Koju glavu, cilindar i sektor unutar staze će zauzeti logički sektor 253197, uz pretpostavku da se broji po glavama, pa cilindrima pa sektorima? Koliki je kapacitet ovog diska ako je sektor velik 512 bajta?

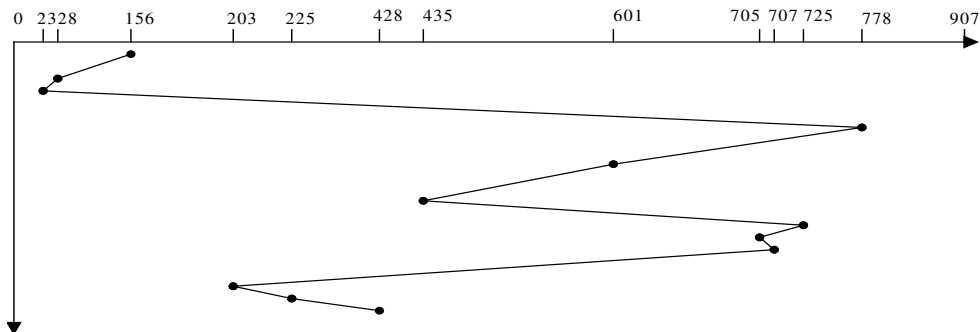
Rješenje: Svaka ploča ima četiri zone. Množeći broj staza u zoni sa brojem sektora u stazi dobijemo da zone imaju redom $Z_0=37740$, $Z_1=38420$, $Z_2=39100$ i $Z_3=39780$ sektora. Ukupno na svakoj ploči ima 155040 sektora. Kada se 253197 podijeli s 155040 dobije se 1 i 98157 ostatka. Dakle glava je 1. Pošto je $Z_0 + Z_1 < 98157 < Z_0 + Z_1 + Z_2$, sektor pripada zoni Z_2 . Relativna pozicija staze od početka zone se dobiva kada se od broja sektora unutar ploče oduzme veličina prethodnih zona, tj $98157 - Z_1 - Z_0 = 21997$. Kada ovaj broj podijelimo s brojem sektora po stazi za zonu Z_2 dobijemo $21997 / 115 = 191$ i 32 ostatka. Zona Z_2 počinje od staze 680, pa je staza = $680 + 191 = 871$, a sektor 32.

Ukupni kapacitet je jednak proizvodu broja glava i veličine ploče = $6 \cdot 155040 = 930240$ sektora, što daje kapacitet 476282880, tj 454M.

Zadatak 3. Glava diska čija ploča ima 908 cilindara (početni je 0, krajnji 907) se nalazila na cilindru 156, krećući se ka većim brojevima cilindra i pristizali su zahtjevi za cilindrima 28, 23, 778, 601, 435, 725, 705, 707, 203, 225, 428. Izračunati ukupni pređeni put glave diska i nacrtajte dijagrame kretanja glave diska za algoritme. a) FCFS b) SSTF c) SCAN d) LOOK e) C-SCAN f) C-LOOK. Kod C-SCAN i C-LOOK u pređeni put ne računati povratak glave.

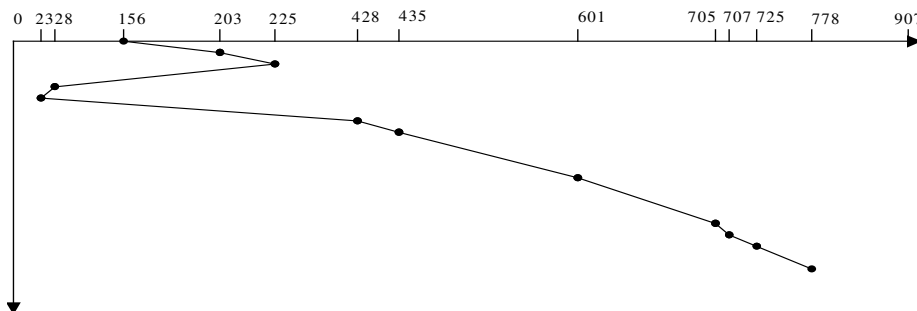
Rješenje

a) FCFS;



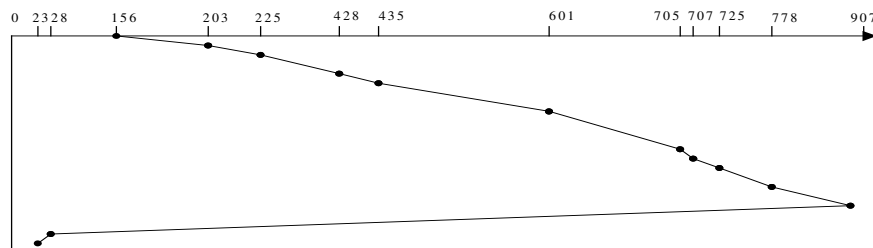
Glava je prešla $(156-28) + (28-23) + (778-23) + (778-601) + (601-435) + (725-435) + (725-705) + (707-705) + (707-203) + (225-203) + (428-225) = 2272$ cilindra

b) SSTF



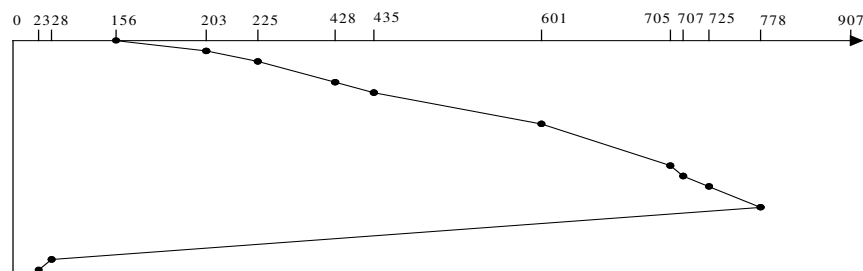
Glava je prešla $(225-156) + (225-23) + (778-23) = 1026$ cilindra

c) SCAN



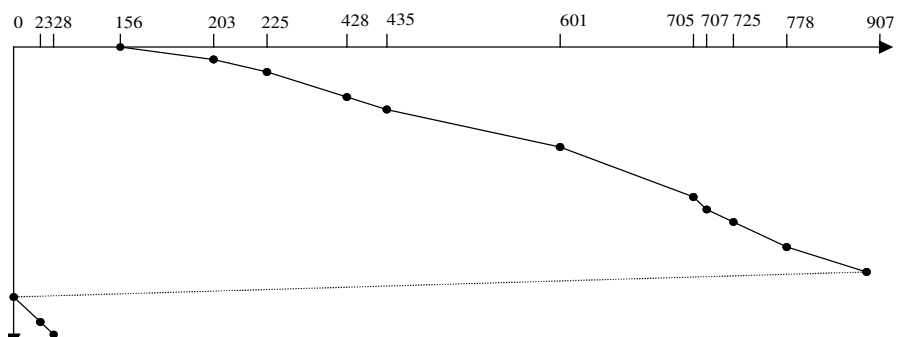
Glava je prešla $(907-156) + (907-23) = 1635$ cilindara

d) LOOK



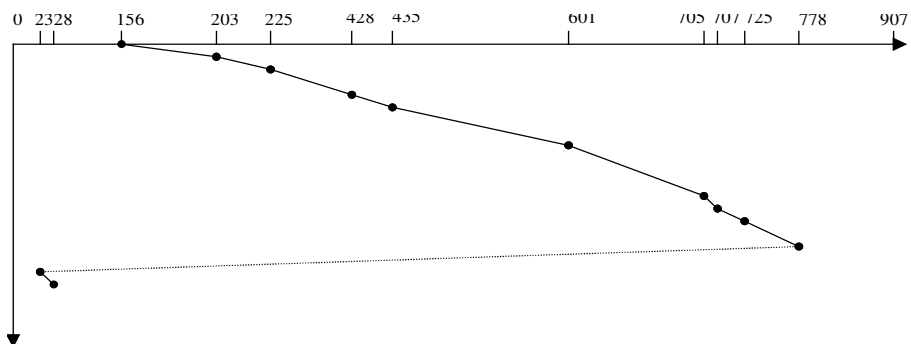
Glava je prešla $(778-156) + (778-23) = 1377$ cilindara

e) C-SCAN



Glava je prešla $(907-156) + (28-0) = 779$ cilindara

f) C-LOOK.

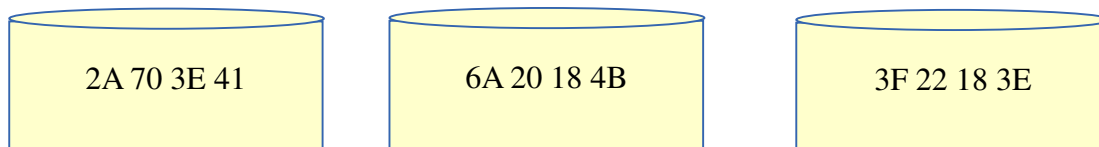


Glava je prešla $(778-156) + (28-23) = 627$ cilindara.

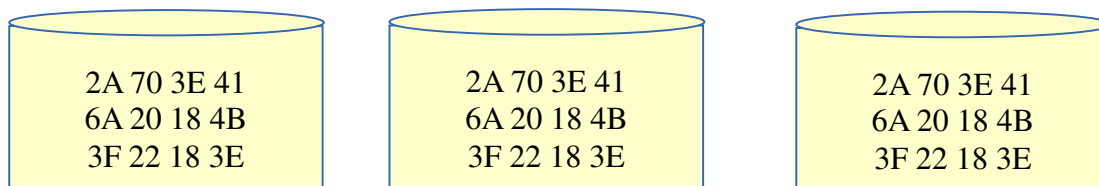
Zadatak 5. Neka je dat niz bajtova u heksadekadnim vrijednostima 2A 70 3E 41 6A 20 18 4B 3F 22 18 3E. Rasporedite ove bajtove na tri diska koristeći odsječke veličine 4 bajta (gdje je primjenjivo) i strukture a) RAID 0, b) RAID 1, c) RAID 2, d) RAID 3 e) RAID 4 f) RAID 5

Rješenje:

a) RAID 0, Odsječci se raspoređuju po svim diskovima

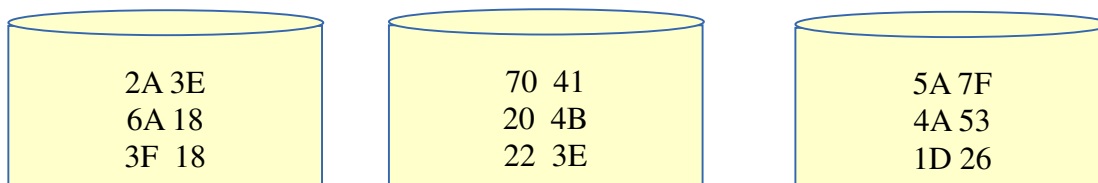


b) RAID 1, sadržaj se kopira na sve diskove

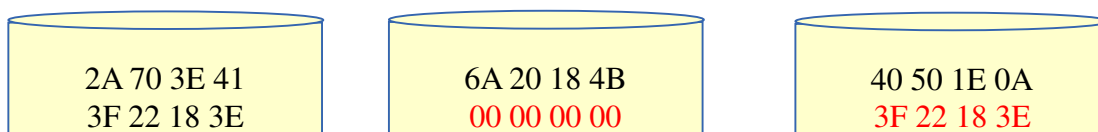


c) RAID 2: Nije izvodiv s ovim brojem diskova, potrebno 12 diskova: po 1 za svaki bit i 4 za paritete

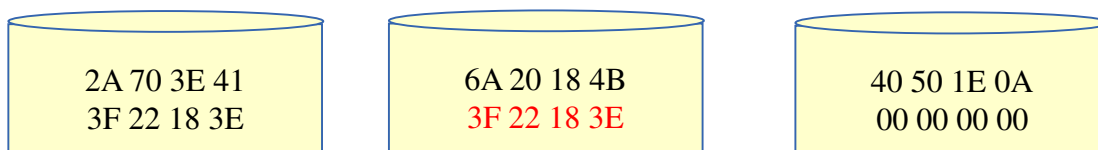
d) RAID 3: Raspoređuje se bajt po bajt, na treći disk ide paritet (XOR) između pojedinih bajtova



f) RAID 4: Odsječci se redaju na prva dva diska, treći je paritet između odsječaka. Drugi se dopunjava nulama.



g) Raid 5, odsječci se redaju na sva tri diska, paritet putuje



12.LOGIČKA ORGANIZACIJA DATOTEČNOG SISTEMA

Dok se proces izvršava, unutar adresnog prostora je moguće smjestiti ograničen skup informacija koje se brišu kada se proces završi. Za većinu primjena računara potrebno je podatke čuvati i nakon završetka procesa, a često je potrebno pristupiti i većoj količini podataka od raspoložive fizičke memorije. U tu svrhu, naravno, služe diskovi. Međutim, mogućnosti diskova za čuvanje podataka se uglavnom svode na pristup blokovima koristeći njihovu logičku ili fizičku poziciju. Ove pozicije je jako teško pamtit, jako zavise od vrste diska i nemaju nikakve veze sa samim podacima. Na primjer, podaci o uplaćenim računima za januar 2016. godine se mogu nalaziti na logičkim blokovima 180, 24300 i 78320. Evidentirati te pozicije u nekoj svesci bi bilo smiješno, kada bi te pozicije mogli čuvati i na samom disku. Tako se vrlo rano od pojave diskova došlo do ideje datotečnih sistema. To je način organizacije diska ili particije diska uz pomoć koga se saznaje gdje se nalaze određeni podaci, pri čemu zaokruženi skupovi podataka dobivaju smisljena imena. Diskovi se obično dijele na područja koja sadrže informacije o tome kako doći do blokova sa podacima i područja sa samim podacima. Glavni problem datotečnog sistema je određivanje tih područja, kako bi pristup podacima bio što brži i omogućilo se smještanje različitih veličina podataka (od skupova podataka od 0 bajta do više terabajta) uz što bolju iskorištenost prostora na disku. Ovi ciljevi su često suprotstavljeni. Na primjer, diskovi kao uređaji su napravljeni tako da je moguće čitati i pisati samo konačni broj blokova (sektora), a ne pojedinačnih bajtova. Ako u jedan sektor stavimo spisak sektora koji pripadaju podacima o uplaćenim računima za danas, brzo saznajemo gdje su ti sektori sa podacima. Međutim, ako je danas bilo malo uplaćenih računa, dio sektora sa pozicijama sektora podataka ostaje neiskorišten, dok ako je bilo mnogo uplaćenih računa, sa jednim rezerviranim sektorom neće se moći svi evidentirati.

12.1. Datoteke

Prema korisniku, datotečni sistem izgleda kao skup datoteka. **Datoteka** je imenovana kolekcija vezanih podataka koja prema korisniku izgleda kao prosti kontinualni blok informacija i kao takva je smještena na uređaju za pohranu podataka. Zadatak datotečnog sistema je mapiranje logičkih podataka koje datoteke čuvaju i fizičkih pozicija na diskovima.

Datotečni sistem opisuje datoteku koristeći tri grupe podataka: datotečne atribute, sadržaj datoteke i podatke o datoteci u memoriji.

12.2.Datotečni atributi

Datotečni sistem treba da čuva razne podatke o datotekama u određenim tablicama. Svaka datoteka ima ime i svoj sadržaj. Pored ovoga, operativni sistem čuva o datoteci još neke informacije, kao što su datum i vrijeme kreiranja datoteke, njena veličina, te način kako doći do njenog sadržaja. Ovi dodatni podaci se zovu **atributi datoteke**. Lista atributa značajno varira od sistema do sistema. U tabeli na slici Slika 143 prikazani su samo neki od atributa, ali treba znati da se tu mogu dodati i oni koji su određenim projektantima potrebni. Dokumentacija o datotečnim sistemima često ima uže ili šire stanovište o tome šta je atribut od ovog opisanog u tabeli. Tako npr. dokumentacija o NTFS pod pojmom atributa podrazumijeva ne

samo podatke o datoteci, nego i njen sadržaj i podatke o slobodnom prostoru na disku. S druge strane, dokumentacija o FAT sistemu termin atribut ograničava samo na šest bitova pristupa.

Kategorija	Atributi	Značenje
Imenovanje	Ime	Ime datoteke u ljudski čitljivom obliku
	Ekstenzija	Ekstenzija imena datoteke
	Verzija	Verzija datoteke
	Interni identifikator	Oznaka datoteke interna,
	Pripadnost direktoriju	Oznaka direktorija u kome je datoteka
Lociranje	Početni blok na disku	Redni broj bloka od koga počinje datoteka
	Pokazivači na blokove	Različiti pokazivači koji lociraju ostatak datoteke
Prava	Korisnička prava	Ko može pristupati datoteci i na koji način
	Lozinka	Lozinka s kojom se pristupa datoteci
	Kreator	Oznaka korisnika koji je kreirao datoteku
	Vlasnik	Oznaka korisnika koji je vlasnik datoteke
Pristup	Samo za čitanje	Da li se može samo čitati ili i upisivati u datoteku
	Samo za dodavanje	Da li je dodavanje moguće samo na kraj
	Skrivena	Da li će datoteka biti sakrivena od listanja
	Sistemska	Da li je datoteka sistemska
	Zaključana	Da li datoteci sada smije samo 1 proces pristupiti
	Arhivirana	Da li je datoteka mijenjana od zadnjeg backup-a
Struktura datoteke	ASCII/binarna	Da li je datoteka ASCII tipa ili ne
	Kriptografisana	Da li je datoteka šifrovanog sadržaja
	Kompresovana	Da li je specijalno kodirana radi smanjenja
	Rijetka	Da li se datoteka uglavnom sastoji od nula
	Slučajni pristup	Da li se može pozicionirati na proizvoljno mjesto
	Privremena	Da li se datoteka automatski briše po zatvaranju
	Specijalna	Namjena: obična datoteka, Direktorij, link
Slog struktura	Indeksirana	Da li se datoteka uključuje u indeksiranje
	Dužina sloga	Broj bajta u slogu
	Pozicija ključa	Pozicija ključa unutar svakog sloga
Vremena	Dužina ključa	Broj bajta u polju ključa
	Vrijeme kreiranja	Datum i vrijeme kreiranja datoteke
	Vrijeme pristupa	Datum i vrijeme posljednjeg pristupa datoteci
Veličine	Vrijeme izmjene	Datum i vrijeme posljednje promjene datoteke
	Trenutna veličina	Broj bajta u datoteci
	Maksimalna veličina	Broj bajta u datoteci može rasti do te veličine

Slika 143 Atributi datoteka

Najvažniji atributi su oni za imenovanje. Kada proces kreira datoteku, on joj dodijeli ime. Kada se proces završi, datoteka ostaje pod tim imenom da postoji na disku i može joj pristupiti drugi proces koristeći to ime. Pravila za imenovanje datoteke su različita od sistema do sistema. Imena su najčešće sastavljena od slova, cifara i specijalnih znakova. Dopusštena dužina imena u sistemima varira od 5 pa do više od 200 znakova. Neki sistemi razlikuju u imenu mala i velika slova (u Unix sistemima pismo.txt i Pismo.txt su dva različita imena) dok drugi ne (kao u MS-DOS-

u, Windows-u i CP/M-u). Mnogi datotečni sistemi podržavaju imena datoteka sa dva dijela međusobno odvojena tačkom, recimo prog.c. Dio koji se nalazi iza tačke se naziva ekstenzija i obično ukazuje na tip datoteke (u ovom slučaju kaže da datoteka predstavlja izvorni kod napisan u C jeziku). U nekim datotečnim sistemima može biti samo jedna ekstenzija (npr. u CP/M i MS-DOS-u) fiksne veličine (3 znaka). U ovom slučaju, ekstenzija je poseban atribut imenovanja. U Unix i Windows NT može postojati više ekstenzija različitih dužina (prog.c.bz2, gdje bz2 označava da je datoteka prog.c kompresovana) i one predstavljaju sastavni dio atributa imena. Verzija se javlja u nekim operativnim sistemima kao što je VAX/VMS. Svaki put kada se datoteka snimi, broj verzije se uveća za 1. VAX VMS snima zadnju verziju prije nego uveća brojač verzije i snimi novi sadržaj. Tako nastaje više datoteka sa istim imenom ali različitim brojem verzije PISMO.TXT;1 i PISMO.TXT;2. Atribut internog identifikatora predstavlja oznaku datoteke u formatu koji je operativnom sistemu brži za prepoznavanje od dužeg imena, npr. cijeli broj.

Atributi lociranja opisuju gdje se datoteka nalazi na disku. Obično je potreban podatak gdje se nalazi na disku početak datoteke, tj. njen prvi blok. Potrebni su, naravno i podaci o ostatku datoteke, gdje se nalaze svi blokovi na disku koji čine datoteku. Ima mnogo načina kako se ti podaci čuvaju.

Atributi prava su prisutni na datotečnim sistemima koji podržavaju više korisnika. Uz datoteke se mogu zakačiti različite liste sa spiskom prava koja imaju pojedinačno korisnici nad datotekom ili interne oznake vlasnika datoteke i pridruženih prava koja imaju vlasnici i ostali korisnici.

Atributi pristupa su obično vrijednosti koje mogu biti 0 ili 1. Na primjer, datoteke koje su označene kao sakrivene se ne prikazuju u spisku dostupnih datoteka. Zastavica arhiviranja nam govori da li je datoteka kopirana. *Backup* program briše tu zastavicu a operativni sistem je postavlja kod promjene datoteke. Na ovaj način, *backup* program zna koje datoteke treba kopirati. Atribut zaključanosti se može postaviti kada jedan proces otvori datoteku i time zabrani pristup toj datoteci drugim procesima.

Atributi koji opisuju strukturu datoteke također najčešće imaju vrijednosti 0 ili 1. Neki sistemi dopuštaju da se različito tretiraju ASCII i binarne datoteke, omogućavaju kompresiju i šifrovanje datoteka, ili imaju poseban tretman za uglavnom prazne datoteke. U nekim sistemima (npr. Windows Vista) postoji mogućnost brzog pretraživanja sadržaja datoteka indeksiranjem. Dok indeksiranje omogućava bržu pretragu datoteka, pravljenje samog indeksa je sporo i zahtijeva dosta dodatnog prostora, pa je atributom moguće isključiti neke datoteke iz tog procesa. Najvažniji atribut strukture označava sadržaj datoteke. Neki datotečni sistemi podržavaju specijalne datoteke kao što su direktoriji, simboličke veze i uređaji. Operacije nad ovakvim datotekama su drugačije od operacija nad običnim datotekama.

Atributi slogovske strukture, dužina sloga, pozicija ključa i dužine ključa su prisutni su samo u onim datotečnim sistemima koji poznaju strukturu datoteka kao niz slogova.

Različita vremena služe da ukažu na vrijeme kreiranja datoteke, posljednjeg modifikovanja i posljednjeg pristupa datotekama. Ona su korisna za različite svrhe.

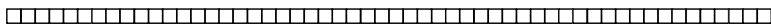
Na primjer, ako je izvorna datoteka modificirana poslije kreiranja objektne datoteke daje nam informaciju da je treba ponovno kompajlirati (*make program*).

Veličina datoteke govori koliko je velika datoteka koju opisuje taj atribut. U nekim starijim operativnim sistemima bila je potrebna maksimalna veličina datoteke da se upiše kod kreiranja nove datoteke kako bi operativni sistem rezervirao unaprijed maksimalni iznos smještajnog prostora za datoteku. Većina sistema mjeri veličinu u bajtima. Oni sistemi koji mjere veličinu datoteke u blokovima koriste specijalni znak EOF za indikaciju posljednjeg bajta podataka.

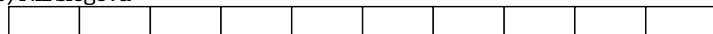
12.3.Fizički sadržaj datoteke

Datoteke mogu biti fizički strukturirane na jedan od nekoliko načina. Uobičajeni načini su prikazani na slici Slika 144.

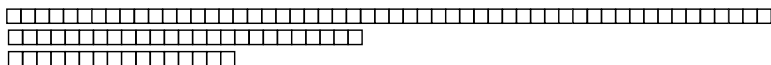
a) Niz bajtova



b) Niz slogova



c) Više nizova bajtova



Slika 144 Fizičke organizacije datoteka

Datoteka, na slici Slika 144a, predstavljena je sa nestrukturiranim nizom bajta. U ovom slučaju, operativni sistem ne zna ili ne brine o sadržaju unutar datoteke, koji u suštini predstavlja samo **niz bajta**. Značenje bajtima uglavnom daje korisnički program, a ne operativni sistem (iako je struktura nekih tipova datoteka poput izvršnih programa poznata i operativnom sistemu). Većina operativnih sistema opće namjene datoteke organizuje na ovaj način.

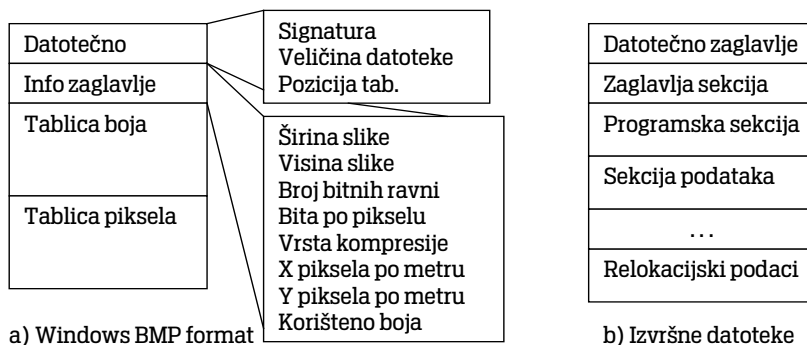
Druga mogućnost je da je datoteka fizički organizovana kao niz podataka takvih da je najmanja jedinica podataka veća od jednog bajta (Slika 144b). U ovom modelu, datoteka predstavlja **niz slogova** fiksne veličine, od koji svaki ima svoju internu strukturu. Osnovna ideja ovakve strukture podataka je da operacija čitanja vraća jedan slog i da operacija pisanja prepisuje ili dodaje jedan slog (primjer, čitanje programa sa 80 kolonskih kartica i štampanje istih na štampač sa širinom od 132 znaka). Ovakav pogled na datoteke je primijenjen u operativnim sistemima poput *Michigan Terminal System*, *Pick Operating System*.

Moderni operativni sistemi poznaju i smještanje **različitih tokova**, nezavisnih nizova bajtova u jednu datoteku (Slika 144c). Na primjer, Mac OS-X je razvijen za dva vrlo različita tipa procesora: Power PC i Intel Pentium. Stoga se u izvršnim datotekama nalaze dvije odvojene kopije izvršnog programa unutar jedne datoteke, od kojih svaka predstavlja sekvencu bajtova a vidljiva je na instalaciji operativnog sistema za jedan od ovih procesora. Na datotečnom sistemu NTFS za Windows NT datoteke su sastavljene od tokova koji se zovu atributi (konfuzija s istoimenim pojmom) tako da je moguće u istu datoteku smjestiti više nizova podataka, kao i informacija o samoj datoteci.

12.4.Logička struktura datoteka

Nestrukturirani niz (ili nizovi) bajtova u datoteci se dalje interpretira na različite načine prema namjeni datoteke. Tako recimo, Unix sistemi poznaju posebne datoteke koje predstavljaju spisak datoteka u direktoriju, datoteke koje služe samo za imenovanje perifernih uređaja blok i znakovnog tipa, datoteke koje predstavljaju simboličke veze na druge datoteke i obične datoteke. Windows NT u datotečnom sistemu NTFS poznaje datoteke koje mogu biti kompresovane ili kriptografisane, kao i specijalne datoteke za opis sadržaj datotečnog sistema.

Obične datoteke se po skupu bajtova koji se u njima nalaze dijele u dvije osnovne grupe: ASCII datoteke i binarne datoteke. ASCII datoteke se sastoje od linija teksta na kraju kojih se nalazi bajt sa kodom CR. Linije ne moraju biti iste veličine. Ove datoteke se prilikom prikaza na ekranu razumiju kao čitljivi tekst. Binarne datoteke predstavljaju niz bajta i nisu pogodne za štampanje. Pokušaj njihovog ispisa na ekranu (npr. Unix komandom *cat* ili Windows komandom *TYPE*) obično rezultuje besmislenim sadržajem na ekranu. Njihov sadržaj je, međutim, itekako smislen, ali nije namijenjen da se čita od strane čovjeka nego od strane aplikativnih programa i operativnog sistema.



Slika 145 Neki binarni formati

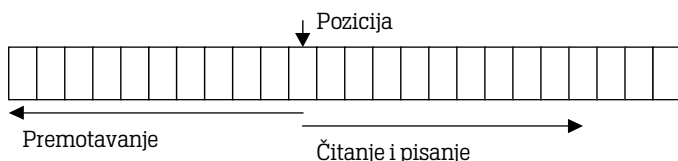
Na slici Slika 145a može se vidjeti struktura jedne binarne datoteke, BMP, datoteka grafičkih slika u Windows sistemu. Ove datoteke se sastoje od četiri osnovna dijela, prvo zaglavlje koje objašnjava gdje su ostali dijelovi, zaglavlje slike koje opisuje njene dimenzije i značenje bajtova koji predstavljaju piksele, bitove palete boja i bitove piksela.

Najvažnija grupa binarnih datoteka su izvršne datoteke. U njima je zapisan mašinski kôd i inicijalizirani podaci programa na disku koji se treba učitati u memoriji. Na Unix sistemima koristi se formati *a.out* i noviji *ELF* format, dok se na Windows sistemima koriste formati *NE (New executable)* za 16 bitne programe i *PE (Portable executable)* za 32 bitne programe. Ovi formati nisu identični, ali su međusobno jako slični. Svi se sastoje od datotečnog zaglavlja koje opisuje strukturu datoteke izvršnog programa, zaglavlja sekcija koje kaže gdje je koja sekcija i samih sekcija. Sekcije prate fundamentalne dijelove procesa u memoriji: kodnu sekciju, sekciju podataka i *stack* sekciju (koja nije upisana u izvršnu datoteku jer se kreira u toku rada procesa). Izvršni programi mogu sadržati i sekcije koje se ne ubacuju u

memoriju pri startu procesa, na primjer sekcije sa imenima simbola koje koriste programi za praćenje izvršenja (*debugger*) ili sekcije resursa u Windows PE formatu (namijenjen za čuvanje većih objekata poput bit-mapiranih slika koje se učitavaju po potrebi na zahtjev).

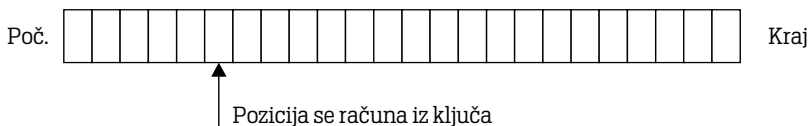
12.5.Logički pristup datotekama

Struktura datoteka kao i karakteristike medija na kome se one nalaze utiče i na način pristupa datotekama. Neke datoteke je moguće čitati samo sekvencijalnim pristupom. Taj pristup zahtijeva da se čitaju svi bajtovi ili slogovi fiksnim redoslijedom od početka. Pristup je uobičajen za ASCII datoteke, kao i za datoteke na magnetnim trakama. Kod sekvencijalnog pristupa nije dozvoljeno čitanje od proizvoljne pozicije niti preskakanje određenog broja slogova. Jedino se može postaviti na početak datoteke (Slika 146).



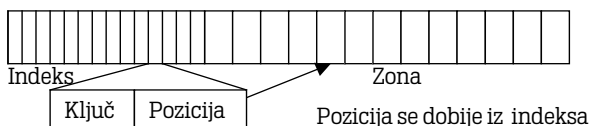
Slika 146 Sekvencijalni pristup datotekama

Datoteke, čiji se bajti ili slogovi mogu čitati u bilo kojem redoslijedu se nazivaju datoteke sa direktnim pristupom (Slika 147). Kod ovih datoteka se navodi jedan podatak, koji se zove ključ. Iz ključa se nekom formulom može izračunati pozicija u datoteci. Na primjer, ako datoteka sadrži spisak članova sportskog društva, sa podacima o rednom broju člana, imenu, datumu upisa i listi sportova koje trenira, i ti podaci za svakog sportistu zauzimaju 256 bajtova, onda se pozicija podataka o sportisti čiji je redni broj r lako dobiju na poziciji od početka datoteke $r \cdot 256$. Nakon pozicioniranja na pravo mjesto u datoteci, podaci se onda mogu pročitati i upisati.



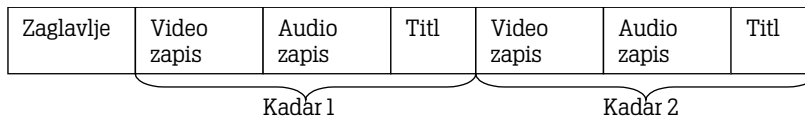
Slika 147 Direktni pristup datotekama

Indeksna organizacija datoteka (Slika 148) se zasniva na posebnom području (indeks) u kome se čuvaju pokazivači na pojedini dio datoteke prema navedenom ključu. U indeksnom području se nalaze uređeni parovi (vrijednost ključa, pozicija sloga). Ovakve datoteke se koriste u bazama podataka jer se odlikuju brzom mogućnosti pretrage. Indeksi mogu sadržati više različitih vrsta ključeva, a sami su organizovani kao niz uređenih parova ili (što omogućava veću brzinu) u obliku stabla.



Slika 148 Indeks-sekvencijalna datoteka

Multimedijalne datoteke (Slika 149), tj. datoteke s filmom i zvukom kombinuju sekvencijalni i direktni pristup. Većinu vremena im se pristupa sekvencijalno. Filmovi su kodirani na način da se jedan za drugim navode elementi slike trenutnog kadra i zvučni elementi, kadar za kadrom. Ali, većina njih ima i mogućnost premotavanja na kadar.

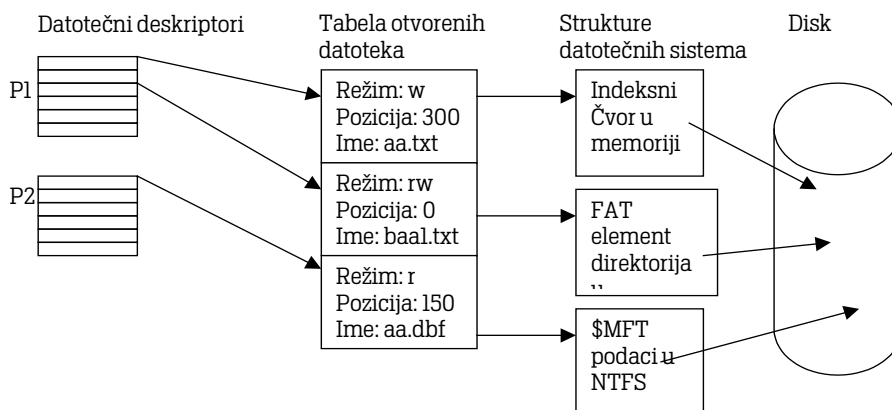


Slika 149 Struktura multimedijalne datoteke

12.6.Podaci o datoteci u memoriji

Većina operativnih sistema (Unix, Windows, MS DOS), pa i korisnički programi pisani u jeziku C, bez obzira u kojem operativnom sistemu su pokrenuti, se referenciraju na otvorenu datoteku koristeći jedan cjelobrojni ili pointerski identifikator datoteke, koji se zove datotečni deskriptor (*fd*). On je uveden umjesto stalnog prosljeđivanja sistemskim pozivima imena datoteke iz dva razloga. Jedan razlog je u brzini samog sistemskog poziva, jer je manje zahtjevno proslijediti jedan cijeli broj nego, npr. 30 znakova imena. Drugi razlog je što je mnogim programima potrebno otvoriti istu datoteku više puta istovremeno, na primjer prilikom njenog sortiranja praktično je kretati se kroz datoteku s dva različita pokazivača.

Kada želi prvi put pristupiti datoteci, proces pozove sistemski poziv za njeno otvaranje, npr. *open* (ime_datoteke, način_pristupa). Način pristupa može biti čitanje, pisanje, čitanje i pisanje, dodavanje, provjera postojanja datoteke itd. Ako operativni sistem dopusti otvaranje datoteke sa traženim načinom pristupa, (npr. datoteka postoji, i korisnički proces koji je tražio čitanje ima pravo na to), rezultat ove funkcije je **datotečni deskriptor** *fd*. Proces će dalje koristiti deskriptor kao parametar za kasnije operacije nad datotekom, na primjer *read(fd, buff, num)* ili *write(fd, buff, num)*. Posljednja među njima će biti *close(fd)* koja zatvara datoteku.



Slika 150 Tablice datotečnih deskriptora

Datotečni sistem održava podatke za svaku otvorenu datoteku. U memoriji se čuva trenutna pozicija sa koje se čitaju podaci iz datoteke, veza na strukture koje

omogućavaju pozicioniranje na bilo koji podatak u datoteci i neki atributi datoteke. Ako novi kreirani procesi nasljeđuju i otvorene datoteke od roditeljskih procesa (i time datotečne deskriptore), čuva se i podatak koliko procesa koristi isti datotečni deskriptor. Ova se struktura kreira prilikom otvaranja datoteke i briše se kod zatvaranja. Datotečni deskriptor na neki način pokazuje na te podatke, direktno ili indirektno. Podaci o svim otvorenim datotekama čuvaju se u **tabeli otvorenih datoteka** (engl. *Open file table*) (Slika 150).

12.7.Memorijski mapirane datoteke

Pristup datotekama je znatno drugačiji nego podacima u memoriji. Na primjer, da se uveća za 1 element niza smješten u memoriji, dovoljno je upisati vrijednost na odgovarajućem indeksu:

```
niz[240]++;
```

Kada je niz smješten u datoteci, mora se raditi puno više: otvoriti datoteku, pozicionirati na traženu poziciju, pročitati podatak u memoriju, promijeniti ga, opet se pozicionirati i upisati novi podatak i ako nema više operacija zatvoriti datoteku.

```
fd=open("niz",rw);
seek(fd,240*sizeof(int));
read(fd,&value,sizeof(int));
value++;
seek(fd,240*sizeof(int));
write(fd,&value,sizeof(int));
close(fd);
```

Podsistem za virtualnu memoriju svakako povremeno snima memorijske stranice u specijalnu datoteku ili disk particiju. Stoga su potrebne male izmjene da se sadržaj memorijske stranice automatski smješta u datoteku koju odabere korisnik. Na ovaj način se pristup podacima datoteke umjesto poziva za čitanje i pisanje dalje obavlja prostim čitanjem ili upisom na memorijsku lokaciju. Memorijski mapirane datoteke povezuju dijelove memorije sa dijelovima datoteke. Koristeći posebne pozive *map* i *unmap* postiže se da se upis podatka u memorijsku lokaciju reflektuje na datoteku koja je mapirana. Forsiranjem grešaka stranica navedeni dio memorije se upisuje u tu datoteku. Time imamo jednostavnost pristupa memoriji kombinovanu sa perzistentnošću pristupa datotekama.

Nedostatak ovog pristupa je što veličina datoteke mora biti djeljiva sa veličinom stranice ako sistem koristi straničenje ili proizvoljne veličine, ali manja od fizičke memorije ako se koristi segmentacija.

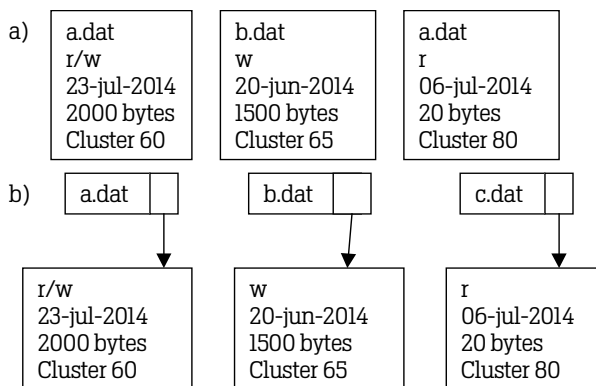
12.8.Direktoriji

Datoteke se grupišu u logičke skupove koji se zovu direktoriji, folderi, ili mape. U većini datotečnih sistema direktoriji su specijalizovane datoteke čiji je sadržaj spisak datoteka koje su članovi direktorija. Na nekim datotečnim sistemima taj spisak nije u datoteci, nego u odvojenom području particije. Direktorij tipično sadrži fiksni ili promjenjiv broj elemenata, po jedan za svaku datoteku.

Jedna mogućnost je prikazana na slici Slika 151a, gdje svaki element sadrži ime datoteke, datotečne attribute i lokacije na disku smještenih podataka. Druga mogućnost je prikazana na slici Slika 151 b, gdje jedan element direktorija sadrži

ime datoteke i pokazivač na drugu strukturu podataka, u kojoj se mogu naći atributi datoteke i lokacije na disku.

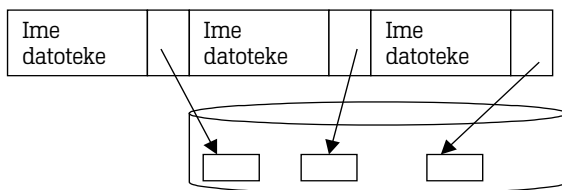
Kada se pozove sistemski poziv za otvaranje datoteke, pretražuje se odgovarajući direktorij sve dok ne nađe njeno ime. On tada vadi attribute datoteka kao i adrese na disku iz elementa direktorija (slučaj Slika 151a), ili iz strukture podataka na koju pokazuje polje u elementu direktorija, (slučaj Slika 151b), i stavlja ih u tabelu u glavnu memoriju. Sve sljedeće reference na datoteku koriste informacije zapisane u glavnoj memoriji.



Slika 151 Organizacije direktorija

12.9.Jednonivovski direktorij

Broj direktorija koji može postojati varira od sistema do sistema. Najjednostavniji oblik direktorija je jedan direktorij koji sadrži sve datoteke u datotečnom sistemu, kao što je to pokazano na slici Slika 152.



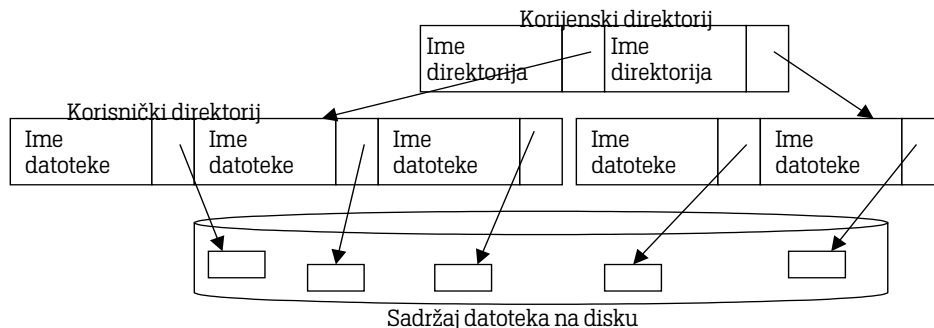
Slika 152 Direktorij na jednom nivou

Kod ranih personalnih računara to je bila uobičajena praksa, jer je postojao samo jedan korisnik, a kapacitet disketa kao osnovnog medija za čuvanje podataka i nije dopuštao mnogo datoteka. Ovakva organizacija direktorija je primijenjena npr. u Apple DOS za računare Apple II.

Problem sa jednim direktorijom u sistemu sa više korisnika je što različiti korisnici mogu slučajno dovesti do brisanja datoteka. Na primjer, kada korisnik A kreira datoteku *mailbox*, a kasnije korisnik B kreira istu datoteku, doći će do uništenja datoteke korisnika A. Još veći problem je ako se ovakva organizacija koristi na velikim diskovima, snalaženje u više hiljada datoteka lociranih na istom mjestu je izuzetno teško.

12.10.Dvonivovski ili jedan direktorij po korisniku

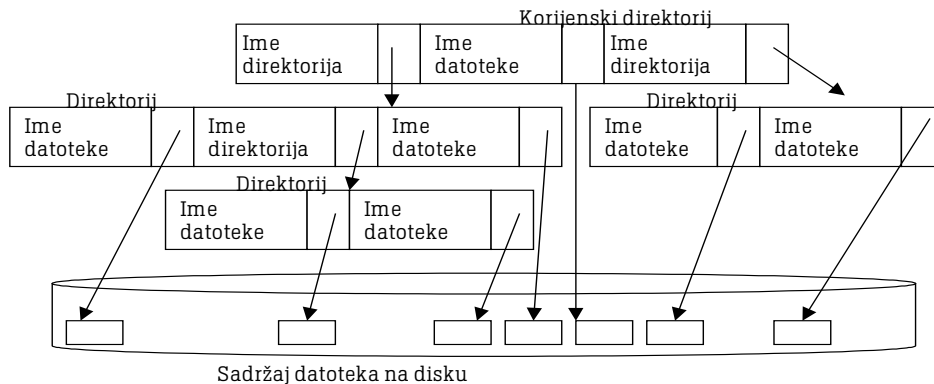
Nije potrebno mnogo promijeniti datotečni sistem da se uvede određeno grupiranje datoteka. Može se npr. svakom korisniku dodijeliti direktorij u okviru koga može kreirati svoje datoteke, a u spisku datoteka dodati jedan atribut koji kaže kojem direktoriju datoteka pripada. Na ovaj način ime datoteke koje je izabrao jedan korisnik ne utiče na istoimene datoteke drugih korisnika. Ovaj pristup je prikazan na slici Slika 153.



Slika 153 Dvonivovski (korisnički) direktorij

Pristup je korišten na operativnom sistemu CP/M. Tu je prosti jednonivovski direktorij proširen pomoću naredbe USER n , gdje je n broj između 0 i 15 i u suštini predstavlja jedan od 16 poddirektorija. Kada korisnik pokuša da otvori datoteku, operativni sistem zna koji je to korisnik i za njega zna koji direktorij da pretražuje.

12.11.Hijerarhijski sistem direktorija

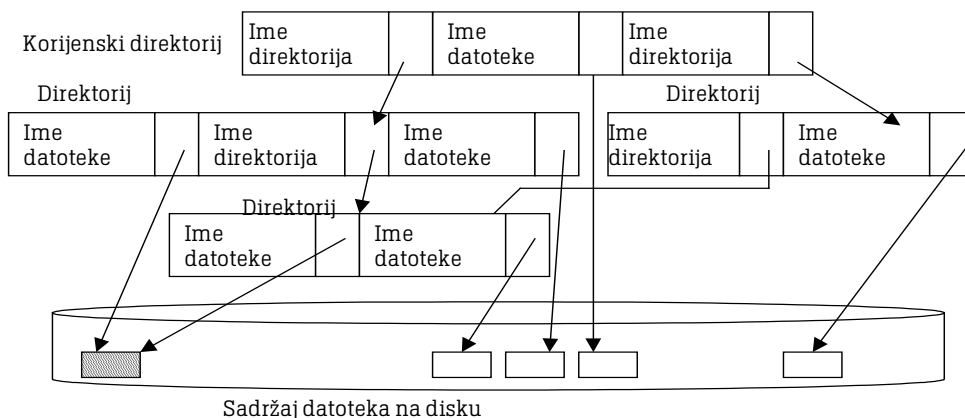


Slika 154 Hijerarhijski direktorij

Često jedan korisnik ima veliki broj datoteka, koje nije moguće lako organizovati unutar jednog direktorija. Pojavljuje se potreba grupisanja datoteka u manje grupe, recimo, direktor želi da izvještaje o prodaji grupiše u jedan direktorij, a putne naloge da čuva u drugom direktoriju. Za ovakav pristup je potrebno imati hijerarhijsku strukturu direktorija, poput stabla. Kod ovog pristupa (Slika 154), svaki korisnik može imati direktorija koliko mu treba tako da datoteke može grupisati na prirodniji način. Koriste ga MSDOS i Windows do verzije XP.

U hijerarhijskim direktorijima imena datoteka se obično navode na tri načina. Prvi način je **prosto navođenje** njenog imena, pri čemu se za pojedini proces u izvršenju odredi jedan direktorij da bude tzv. radni direktorij. Na ovaj način se može pristupiti samo datotekama koje se nalaze u radnom direktoriju. Drugi način je navođenje uz imena datoteke i svih direktorija kojem ona pripada. Taj način se zove **apsolutna staza**. Npr., datoteka `/home/mitar/Dokumenti/prijava.txt` u Unix sistemima se odnosi na datoteku `prijava.txt` lociranoj u direktoriju `Dokumenti`, koji je lociran u direktoriju `mitar`. Direktorij `mitar` je u direktoriju `home`, a taj direktorij je u korijenskom direktoriju. Na Windows i MS DOS sistemima se apsolutna staza navodi razdvajajući direktorije tupougлом kosom crtom (*backslash*) pri čemu svaka disk particija ima odvojen korijenski direktorij, npr. `C:\windows\system32\ntdll.dll`. Treći način se zove **relativna staza**. Kod ovog pristupa se ne polazi od korijenskog direktorija, nego od radnog, a za ispomoć se koriste u većini operativnih sistema i specijalna imena direktorija `“.”` `“..”` koji predstavljaju trenutni direktorij u hijerarhiji, odnosno jedan iznad. Npr. ako je u Unix sistemu radni direktorij `/home/mitar` tada je datoteka `Dokumenti/prijava.txt` drugi način pristupa datoteci `/home/mitar/Dokumenti/prijava.txt`, dok bi se datoteci `/bin/ls` u ovom slučaju moglo pristupiti i navodeći `../bin/ls`.

12.12.Aciklični graf i dijeljene datoteke



Slika 155 Aciklični graf

Neki datotečni sistemi dopuštaju da se datoteka istovremeno nalazi u više direktorija. Ovakav model pruža najviše mogućnosti, jer korisnici mogu da u svoji direktorijima imaju sve datoteke koje su im potrebne, bez trošenja prostora na višestruke kopije iste datoteke koja je potrebna za više korisnika.

Datotečni sistemi koji imaju strukturu acikličnog grafa direktorija omogućavaju dijeljene datoteke između direktorija. Na slici Slika 155 se vidi datoteka označena šrafurom čiji sadržaj istovremeno pripada u dva direktorija.

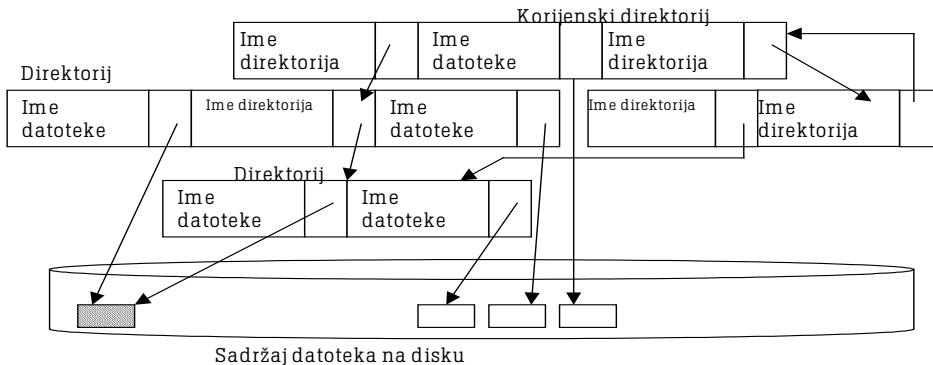
U Unix sistemima direktoriji su organizovani kao tablica uređenih parova koji se sastoje od imena datoteke i pokazivača indeksni čvor. U indeksnom čvoru, koji je izvan opisa direktorija, su ostali atributi datoteke, kao i brojač koji pokazuje koliko datoteka u opisu direktorija pokazuje na taj indeksni čvor. Brojač se uveća za jedan

kada se kreira nova veza na istu datoteku. U Unix sistemima ovo se postiže komandom *ln*. Brisanje datoteke ne znači nužno brisanje indeksnog čvora, nego umanjeње brojača. Kada brojač dostigne nulu, tek tada se izbaci indeksni čvor. Veza kod koje element direktorija pokazuje na sadržaj datoteke naziva se **čvrstom vezom**. Ovo je najbrži pristup realizaciji datoteka koje pripadaju u više direktorija, ali je moguć samo u slučaju da su direktoriji i datoteke u istom datotečnom sistemu. Čvrstom vezom se ne može pokazivati na datoteke koje se nalaze na mreži, drugim diskovima, pa i drugim particijama istog diska.

Drugi pristup je kreiranje nove datoteke čiji sadržaj predstavlja puno ime datoteke. Otraranje ove datoteke rezultuje otvaranjem datoteke čiji je ime unutar nje. Ova vrsta veze na datoteku se naziva **simbolička veza** u Unix-u, *shortcut* u Windows-u, i *alias* u Apple Mac OS. Prednosti simboličkih veza su mogućnosti postavljanja različitih prava nad istom datotekom za različite korisnike, povezivanje na datoteke koje su u drugim particijama i mogućnost implementacije i u datotečnim sistemima koji ne podržavaju čvrste veze (FAT). Mana je u dodatnom zauzeću prostora na disku (vrlo male datoteke zauzimaju bar jedan blok) i potencijalne situacije ako veza pokazuje na nepostojeću datoteku.

12.13.Opšti graf direktorija

Neki datotečni sistemi nemaju nikakva ograničenja na pripadnost datoteka direktorijima. Direktorij može sadržati druge direktorije, uključujući one kojima pripada. Slika 156 pokazuje takvu situaciju kod krajnjeg desnog direktorija na drugom nivou.



Slika 156 Opšti graf direktorija

Neki Linux datotečni sistemi, npr. UnionFS koriste ovu mogućnost kod distribucija koje se pokreću s CD-a. Preporučljivo je izbjegavanje ovakve strukture direktorija. Neka je npr. neko pokrenuo komandu koja briše sve poddirektorije trenutnog korisničkog direktorija. Ako se unutar njega nalazi čvrsta veza na korijenski direktorij, rezultat će biti brisanje cijele particije. Slično, ako se želi kopirati sadržaj direktorija sa poddirektorijima, rutina za kopiranje bi lako mogla završiti u neprekidnoj petlji.

12.14.Čuvanje i pretraživanje direktorija

Direktoriji se mogu čuvati u datotečnom sistemu kao posebna vrsta datoteke, sa specijalnim atributima ili u potpuno odvojenom području datotečnog sistema (CP/M).

U Unix sistemima smješteni su kao obične datoteke specijalne strukture. Njihov sadržaj je skup slogova koji se sastoji od broja indeksnog čvora i imena datoteke. Imena se završavaju sa “\0”, a indeksni čvorovi se koriste za nalaženje samog sadržaja datoteke.

Datoteka ili područje datotečnog sistema koje predstavlja direktorij može svoje elemente (opise datoteka) organizovati na više načina koji utiču na brzinu pretraživanja. Najjednostavnije je organizirati direktorij kao **linearni niz** ili listu što omogućava brzo dodavanje, ali sporu pretragu kod velikih direktorija. Pristup se koristi u FAT i originalnom Unix datotečnom sistemu. Koristeći **B* stabla** postiže se znatno brža pretraga uz komplikovano ažuriranje direktorija. Ovaj pristup je korišten u NTFS i Mac HFS, Neki datotečni sistemi koriste **hash tabelle** za organizaciju direktorija gdje se pozicija u tabeli se računa iz imena. To omogućava brzo nalaženje jedne datoteke čije se ime tačno zna. Pretraživanje između više datoteka čiji se samo dio imena poznaje (džoker znakovi) se mora raditi sekvencijalno. Primjer ove organizacije je Ext2FS.

12.15.Operacije nad datotekama i direktorijima

Operacije koje se obavljaju nad datotekama su otvaranje (priprema memorijskih struktura za pristup datoteci), kreiranje (pravljenje nove datoteke), čitanje (prebacivanje dijela datoteke u memoriju), pisanje (prebacivanje podataka iz memorije na datoteku), testiranje (provjera da li je datoteka postojeća i otvorena), brisanje (uklanjanje datoteke sa diska), pozicioniranje (promjena pozicije pokazivača na datoteku), otvaranje radi dodavanja (otvara datoteku za pisanje i pozicionira se na njen kraj), itd. Operacije koje se mogu raditi nad direktorijima su slične operacijama koje se mogu raditi nad datotekama, ali ne identične. Direktorije je moguće kreirati, otvoriti (radi analize sadržaja) zatvoriti, čitati sadržaj (spisak datoteka), preimenovati, obrisati, dodati čvrste veze na datoteke.

12.16.Windows API za rad sa datotekama i direktorijima

Najvažnije Funkcije Win32 API koje realizuju operacije nad datotekama i direktorijima nalaze se u KERNEL32.DLL biblioteci i date su u sljedećoj tabeli.

Funkcija	Značenje
CloseHandle(hObject)	Zatvara datoteku ili objekt
CopyFile(lpExistingFileName,lpNewFileName, FailIfExists)	Kopira datoteku
CreateDirectory(lpPathName,lpSecurityAttributes)	Kreira direktorij
CreateFile(lpFileName, dwDesiredAccess, dwShareMode, lpSecurityAttributes, dwCreationDisposition, dwFlagsAndAttributes, hTemplateFile)	Kreira ili otvara datoteku
DeleteFile(lpFileName)	Briše datoteku
FindClose(hFindFile)	Završava pretragu datoteka
FindFirstFile(lpFileName, lpFindFileData)	Traži prvu datoteku

Funkcija	Značenje
FindNextFile(lpFileName, lpFindFileData)	Traži sljedeću datoteku
FlushFileBuffers(hfile)	Forsira upis buffera
GetFileAttributes(lpFileName)	Čita attribute datoteke
GetFileInformationByHandle(hFile, lpFileInformation)	Uzima podatke o datoteci
GetFileSize(hFile,lpFileSizeHigh)	Dobavlja veličinu datoteke
LockFileEx(hFile,dwFlags,dwReserved, nNumberOfBytesToLockLow, nNumberOfBytesToLockHigh, lpOverlapped)	Zaključava dio datoteke od istovremenog pristupa
MoveFile(lpExistingFileName,lpNewFileName)	Premješta datoteku
ReadFile(hFile, lpBuffer, nNumberOfBytesToRead, lpNumberOfBytesRead,lpOverlapped)	Čita datoteku
RemoveDirectory(lpPathName)	Briše direktorij
SetEndOfFile(hFile)	Postavlja kraj datoteke
SetFileAttributes(lpFileName,dwFileAttributes)	Postavlja attribute datoteke
SetFilePointer(hFile,lDistanceToMove, lpDistanceToMoveHigh,dwMoveMethod)	Postavlja trenutnu poziciju u datoteci
UnlockFileEx (hFile, dwReserved, nNumberOfBytesToUnlockLow, nNumberOfBytesToUnlockHigh, lpOverlapped)	Otključava dio datoteke
WriteFile(hFile, lpBuffer, nNumberOfBytesToWrite, lpNumberOfBytesWritten,lpOverlapped)	Piše u datoteku

Funkcija `CreateFile`, koja kreira ili otvara postojeću datoteku i kao rezultat vraća njen datotečni deskriptor, koji se u Windows terminologiji zove file handle. Njen prvi parametar predstavlja ime datoteke (ASCII ili Unicode) koja se kreira ili otvara. Drugi parametar je režim otvaranja koji je obično `GENERIC_READ`, `GENERIC_WRITE`, ili njihova kombinacija. Treći parametar, ako je vrijednosti 0 zabranjuje drugim procesima otvaranje datoteke, dok vrijednosti `FILE_SHARE_DELETE`, `FILE_SHARE_READ` i `FILE_SHARE_WRITE` dopuštaju drugim procesima brisanje, čitanje ili pisanje datoteke. Četvrti parametar postavlja prava korisnika nad datotekom, najjednostavnije je da ovaj parametar bude `NULL`. Peti parametar može biti `CREATE_ALWAYS` (uvijek kreira novu datoteku, ako već postoji prazni joj sadržaj), `CREATE_NEW` (kreira datoteku samo ako ne postoji), `OPEN_ALWAYS` (ako datoteka postoji otvara je, ako ne postoji kreira je), `OPEN_EXISTING` (ako datoteka postoji otvara je, ako ne postoji to je greška) ili `TRUNCATE_EXISTING` (ako datoteka postoji prazni joj sadržaj, ako ne postoji to je greška). Šesti parametar su atributi koje će dobiti datoteka, kombinacijom konstanti kao što su `FILE_ATTRIBUTE_READONLY`, `FILE_ATTRIBUTE_HIDDEN`, dok zadnji parametar navodi drugu datoteku iz koje se ti atributi mogu preuzeti.

Pozicija datoteke na koju se postavlja u funkciji `SetFilePointer` se navodi u dva parametra, viših i nižih 32 bita pozicije. Funkcije za čitanje i pisanje datoteke `ReadFile` i `WriteFile`, pored 3 očekivana parametra (oznake datoteke, početne adrese sa koje se čita ili piše i dužine bloka koji se čita ili piše, imaju i dva dodatna parametra, od kojih četvrti prima broj zaista upisanih ili pročitanih bajtova, dok zadnji omogućava da se ova operacija obavi asinhrono, da korisnički program može nastaviti sa radom dok se obavlja prijenos podataka. Slijedi primjer C programa koji otvara jednu datoteku, pozicionira se na njen 20. bajt i ostatak sadržaja kopira u drugu datoteku, koristeći Win32 API.


```
#include <windows.h>
const int BUF_SIZE = 1024;
void main(int argc, char *argv[]) {
    HANDLE fileIn, fileOut;
    char buf[BUF_SIZE];
    DWORD nread,nwrote;
    fileIn = CreateFile("input.txt", GENERIC_READ,0,0,OPEN_EXISTING, 0, 0);
    fileOut = CreateFile("output.txt", GENERIC_WRITE,0,0,CREATE_NEW, 0, 0);
    if (fileIn == INVALID_HANDLE_VALUE || fileOut == INVALID_HANDLE_VALUE)
        exit(1);
    SetFilePointer(fileIn,20,0,FILE_BEGIN);
    while (ReadFile(fileIn, buf, BUF_SIZE, &nread, NULL) && nread > 0) {
        if ( ! WriteFile(fileOut, buf, nread,&nwrote, NULL))
            exit(2);
    }
    CloseHandle(fileIn);
    CloseHandle(fileOut);
    exit(0);
}
```

12.17.POSIX API za rad sa datotekama i direktorijima

Na Unix sistemima može se koristiti više sistemskih poziva namijenjenih za rad sa datotekama, uz dodatak onim koji su dio C biblioteke. Važnije POSIX funkcije za rad sa datotekama i direktorijima navedene su u sljedećoj tabeli.

Funkcija	Značenje
access(pPathname, iMode)	Provjeri prava nad datotekom
chdir(const char *path)	Mijenja trenutni direktorij
chmod(pPath, iMode)	Promijeni prava korisnika nad datotekom
chown(iPath, owner, group)	Promijeni vlasništvo datoteke
close(fd)	Zatvori datoteku
closedir(DIR *dir)	Zatvori direktorij
creat(pPathname, iMode)	Otvori i eventualno kreiraj datoteku
dup2(oldfd, newfd)	Duplicira deskriptor datoteke
fstat(fd, pBuf)	Pročitaj status datoteke
getcwd(pBuf, iSize)	Pročitaj trenutni direktorij
link(pOldpath, pNewpath)	Još jedno ime datoteke
lseek(fd, iOffset, iWhence)	Pozicioniraj datoteku
mkdir(pPathname, iMode)	Kreiraj direktorij
open(pPathname, iFlags)	Otvori datoteku
opendir(pName)	Otvori direktorij
read(fd, pBuf, iCount)	Čitanje datoteke
readdir(DIR *dir)	Čitaj element direktorija
rename(pOldpath, pNewpath)	Promijeni ime ili lokaciju datoteke
rewinddir(DIR *dir)	Reset direktorijskog prikaza
rmdir(pPathname)	Briši direktorij
stat(pFile_name, pBuf)	Pročitaj status datoteke
unlink(const char *pathname)	Ukloni dodatno ime a možda i datoteku
write(fd, pBuf, iCount)	Pisanje u datoteku

Datoteka se otvara pozivom *open*, čiji prvi parametar predstavlja ime datoteke, drugi predstavlja način pristupa i može biti *O_RDONLY* (samo za čitanje), *O_WRONLY* (samo za pisanje), *O_RDWR* (čitanje i pisanje), dok treći predstavlja prava koja se dodjeljuju datoteci (oktalni broj koji se koristi uz *chmod* komandu).

dvije funkcije vraćaju datotečni deskriptor koji je parametar pozivima za čitanje, pisanje i pozicioniranje.

Čita se pozivom *read*, kome su parametri datotečni deskriptor, adresa u memoriji gdje se podaci prebacuju i broj bajta koji se prebacuje. U datoteku se upisuje se pozivom *write*, parametri su mu datotečni deskriptor, adresa u memoriji odakle podaci smještaju u datoteku i broj bajta koji se prebacuje.

C program koji otvara jednu datoteku, pozicionira se na njen 20. bajt i ostatak sadržaja kopira u drugu datoteku, koristeći Posix API izgleda ovako.

```
#include <sys/types.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
const int BUF_SIZE = 1024;
void main(int argc, char *argv[]) {
    int fileIn, fileOut, nread, nwrote;
    char buf[BUF_SIZE];
    if ((fileIn = open("input.txt", O_RDONLY)) < 0)
        exit(1);
    lseek(fileIn, 20, SEEK_SET);
    if ((fileOut = creat("output.txt", 0777)) < 0)
        exit(2);
    while ((nread = read(fileIn, buf, BUF_SIZE)) > 0) {
        nwrote = write(fileOut, buf, nread);
        if (nwrote < 0)
            exit(2);
    }
    close(fileIn);    close(fileOut);
    exit(0);
}
```

12.18.Zadaci

Zadatak 1. Zadana je datoteka koja ima 100 blokova. Pretpostavimo da je kontrolni blok datoteke (indeksni blok, u slučaju indeksne alokacije) već u memoriji. Izračunati koliko se U/I operacija zahtijeva kod dodavanja ili brisanja bloka na početku, sredini i kraju datoteke, za kontinualnu, indeksnu ili alokaciju povezanu listama.

Rješenje:

	Kontinualna	Povezane liste	Indeksna
Dodavanje na početak	201	1	1
Dodavanje u sredinu	101	52	1
Dodavanje na kraj	1	3	1
Brisanje na početku	198	1	0
Brisanje u sredini	98	52	0
Brisanje na kraju	0	100	0

13. FIZIČKA ORGANIZACIJA DATOTEČNOG SISTEMA

Podaci koji se čuvaju u datotečni sistemima mogu se rasporediti na fizičkom disku na neograničen broj načina. Pravo pitanje realizacije je kako to postići na fleksibilan, prostorno i vremenski efikasan način.

13.1. Struktura particije

Nakon što je *hard* disk podijeljen na particije i *master boot record* sadrži particionu tabelu sa spiskom svih particija, sada je potrebno rasporediti podatke koje čuva datotečni sistem unutar pojedine particije. *Floppy* diskovi se ne dijele na particije, pa se ova podjela odnosi na sadržaj cijele diskete, ali princip je isti. Svaki datotečni sistem ima različit raspored particije. Na slici Slika 157 prikazana je logička organizacija diska koji je podijeljen na tri particije: FAT, NTFS i Minix. Svaka od particija je dalje podijeljena u specijalizovana područja čiji broj, veličina i sadržaj se razlikuju za svaki datotečni sistem, ali se ipak mogu grupirati po namjeni.

MBR i part. tabela	Particija 1: FAT					Particija 2: NTFS				Particija 3: Minix				
	Boot BPB	FAT 1	FAT 2	Korijenski direktorij	Sadržaj datoteka	\$BOOT		\$MFT	Datoteke	Boot	Super blok	Bit mape	Indeks. čvorovi	Sadržaj datotek

Slika 157 Tipična struktura datotečnog sistema

Boot područja se obično nalaze na početku particije. U njima se nalazi kratki program koji učitava jezgro operativnog sistema ili, ako je particija s podacima bez operativnog sistema, prijavljuje grešku. Na NTFS sistemu i ovo područje se smatra specijalnom datotekom.

Područje opisa particije sadrži podatke o datotečnom sistemu, kao što su veličina i pozicija ostalih područja, broju sektora koji čine blok, maksimalnom broju datoteka koje sistem može imati. Na Unix datotečnim sistemima (Minix, ext2fs, ReiserFS itd.) ovo područje se zove superblok. U FAT datotečnom sistemu ovo područje predstavlja dio *Boot* područja i zove se *BIOS Parameter Block* (BPB), dok u NTFS sistemu ovu ulogu ima specijalna datoteka \$MFT (*Master file table*).

Područje sadržaja datoteka je najvažnije područje i u njemu se nalaze podaci koje pojedine datoteke imaju. Kod većine datotečnih sistema ovdje su smješteni i direktoriji, tj. spiskovi datoteka koje oni sadrže.

Područja za praćenje alociranog prostora datoteka opisuju gdje se na disku nalazi sadržaj datoteka. Datoteke obično zauzimaju više blokova i mora postojati način da operativni sistem zna koji su to blokovi. U FAT datotečnim sistemima tu ulogu imaju FAT tabele FAT 1 i njena kopija FAT 2, Unix sistemi koriste indeksne čvorove, dok NTFS te podatke čuva u datoteci \$MFT.

Područja za praćenje slobodnog prostora opisuju koji blokovi na disku nisu dodijeljeni datotekama. Kod FAT datotečnog sistema ovo područje ne postoji, jer se ti podaci efikasno dobivaju iz FAT tabela. Unix datotečni sistemi koriste područje bitmapa. NTFS koristi specijalnu datoteku \$BITMAP. na način da je svakom bloku u particiji pridružen po jedan bit koji govori da li je on slobodan ili zauzet,

Područja direktorija opisuju sadržaj direktorija u slučaju da oni nisu u području sadržaja datoteka. To je npr. slučaj sa korijenskim direktorijem datotečnog sistema FAT 16. Sadržaj ostalih direktorija ovog datotečnog sistema je u području sadržaja datoteka.

Zavisno od toga koliko prostora se rezerviše za ova područja, koliko brz datotečni sistem želimo, koliko se interne memorije ima na raspolaganju, da li je važnija brzina čitanja i slično, razvijeno je više načina alokacije datoteka. Oni će uticati na strukturu područja za praćenje alociranog prostora i strukturu područja sadržaja datoteka.

13.2.Veličina bloka

Princip rada *hard* diskova, kao najčešćih periferijskih uređaja za smještaj podataka određuje da sva područja u particiji, kao i sve pojedinačne datoteke, moraju biti veličine koja je umnožak veličine fizičkog sektora. Teoretski, moguća je i bolja granularnost, na primjer da datoteke mogu počinjati na proizvoljnom bajtu na disku, ali bi performanse datotečnog sistema u tom slučaju bile katastrofalne. Koliko sektora treba da čini jedan blok? Ako je blok velik, na primjer, jednak veličini cilindra, tada to znači da svaka datoteka, čak i ona koja zauzima samo jedan bajt će zauzeti cijeli cilindar. S druge strane, korištenje malih blokova (npr. 256 bajta) ima za posljedicu da će se svaka datoteka sastojati od mnogo blokova. Čitanje svakog bloka zahtijeva kašnjenje kod pozicioniranja na stazu kao i rotaciono kašnjenje, tako da će čitanje datoteke, koja se sastoji od mnogo malih blokova, biti prilično sporo. Mali blokovi su, stoga, loši za performanse, ali dobri za iskorištenost diskovnog prostora u situacijama kada na disku ima mnogo malih datoteka. Potrebno je pronaći neku veličinu bloka koja predstavlja kompromis. Pokazalo se da veličina bloka od 4 KB predstavlja dobar izbor, ali za diskove čije su dimenzije u terabajtima, preporučljiva veličina bloka je 64 kilobajta.

13.3.Metode alokacije datotečnih sistema

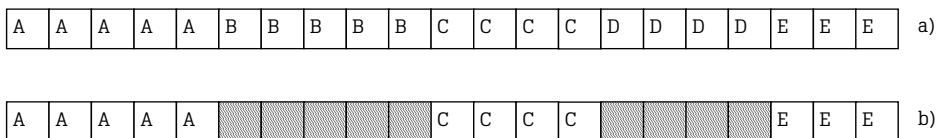
Sada će biti riječi o nekoliko načina dodjele blokova sa podacima pojedinim datotekama, njihovim prednostima i nedostacima, te po jednim datotečnim sistemom koji je tipični primjer takve alokacije. Opisat će se kontinualna, spregnuta, spregnuta sa tablicama, indeksna na jednom nivou, indeksna na više nivoa i ekstentna alokacija.

13.4.Kontinualna alokacija datoteka

Sve metode alokacije datoteka moraju koristiti atribut datoteke koji govori gdje se nalazi njen prvi blok. Najjednostavniji način alokacije je da naredni logički blokovi te datoteke budu smješteni odmah iza tog bloka na susjednim fizičkim blokovima. Posljedica alokacije je smještanje svake datoteke u niz susjednih blokova diska. Tako na disku sa blokovima veličine 2 K za datoteku veliku 9 K, čiji početak se nalazi na bloku broj 150, njen sadržaj će se rasporediti između 5 blokova: blokovi 150, 151, 152, 153 i 154.

Kontinualna alokacija ne zahtijeva posebno područje za evidentiranje blokova dodijeljenih datoteci. Evidencija se svodi na zapisivanje samo dva broja: adresa na disku prvog bloka datoteke i broj blokova u datoteci. Iz pozicije prvog bloka u datoteci, jednostavnim operacijama se može doći do željenog bloka. Performanse

kod čitanja su izuzetno dobre, jer cijela datoteka može biti pročitana u samo jednoj operaciji čitanja, pošto disk kontroler poznaje čitanje više sektora odjednom. Jednako brzo je i pisanje koje ne mijenja dužinu datoteke.



Slika 158 Kontinualna alokacija

Međutim, kontinualna alokacija se ponaša loše kada podatke treba često mijenjati. U početku, ova tehnika alokacije ne predstavlja problem, jer se datoteke uvijek pišu na početku praznog prostora za disk. Ako treba datoteku produžiti, a iza nje se odmah nalazi druga datoteka, moramo tada datoteku koju produžujemo prebaciti na drugo slobodno mjesto. Tako nastaju alocirana područja i šupljine kojima se upravlja istim algoritmima kao za pristup memoriji sa promjenjivim particijama (*first fit*, *best fit*). Kao i kod organizacije memorije s promjenjivim particijama, dolazi se u situaciju kada veliki procenat diska nije iskorišten, a neka veća datoteka se ne može zapisati. Kod memorije se tada pribjegava kompakciji, ali u slučaju diska ta operacija je veoma duga i nepoželjna.

S druge strane, ako se podaci ne mijenjaju često, na primjer na CD i DVD uređajima, čiji se sadržaj ne mijenja nakon snimanja, kontinualna alokacija je odlična, jer je pristup datoteci najbrži i ne zahtijeva dodatne alokacijske tablice.

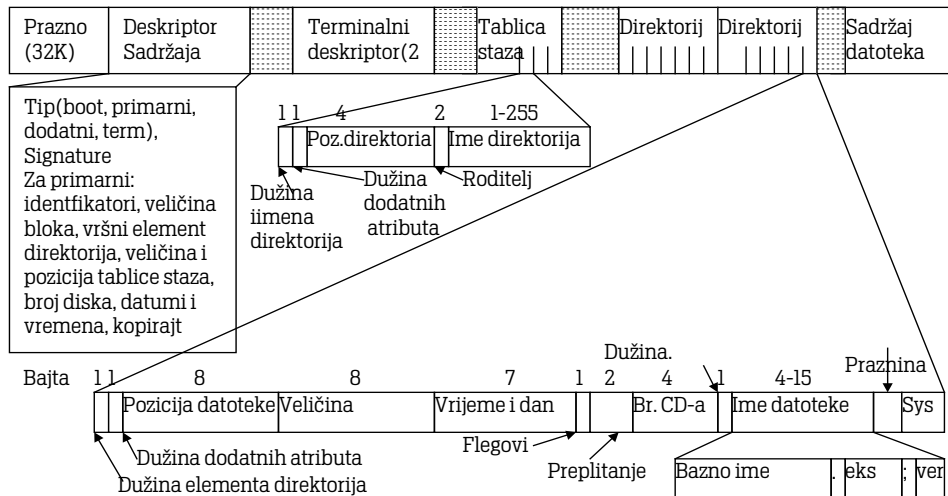
13.5.ISO 9960 FS

Kao primjer datotečnog sistema s kontinualnom alokacijom opisaće se ovaj sistem koji se koristi na CD ROM uređajima. CD je svakako uređaj kod koga proširivanje veličine datoteke nije moguće, pa prednosti kontinualne alokacije u brzini pristupa daleko premašuju eventualne mane. CD-ROM se sastoji od više staza (*track*) od kojih svaka može imati različit datotečni sistem, pa i miješati datotečne sisteme namijenjene za čuvanje datoteka i one za audio zapise. Ove staze, međutim, nisu koncentrični krugovi, nego nadovezane linearne sekvence sektora.

Zapis u formatu ISO 9960 se sastoji od sektora velikih 2352 bajta, od kojih su 2048 namijenjeni za podatke, a ostatak su zaglavlja i korekcija grešaka. Početni dio od 16 sektora se ne koristi. Zatim slijede deskriptori sadržaja, veliki po jedan sektor. Od njih je najvažniji deskriptor primarnog sadržaja (tip 1) koji sadrži podatke o veličini logičkih blokova, autorskim pravima na CD, verziji datotečnog sistema, lokaciji sadržaja direktorija, datumu i vremenu kreiranja CD-a itd. Jedan od deskriptora (tip 0) može sadržati *boot* program. Posljednji deskriptor koji se navodi je oznaka kraja skupa deskriptora (tip 255).

Svaki element direktorija ima strukturu kao na slici Slika 159. Prva dva polja definišu veličinu elementa direktorija i veličinu dodatnih atributa. Podaci o lokaciji datoteke i njenoj veličini na CD-u čuvaju se u dva formata, u *Big Endian* i u *Little Endian* formatu. Nakon toga se u odvojenim bajtovima navode dan, mjesec, godina, sat, minuta, sekunda i vremenska zona kreiranja datoteke. Zatim slijedi indikator bajt čiji pojedini bitovi govore da li je datoteka skrivena, koristi li dodatne attribute

ili je zadnja datoteka u direktoriju. Polje prepletanja omogućava da se kontinualna alokacija zamijeni ekstentnom koja se sastoji od više kontinualnih dijelova. Naredno polje omogućava da se navede oznaka CD-a kome pripada ta datoteka (ako nije na istom CD-u). Na samom kraju je ime datoteke sa dužinom ispred imena i oznakom verzije iza tačka/zareza.

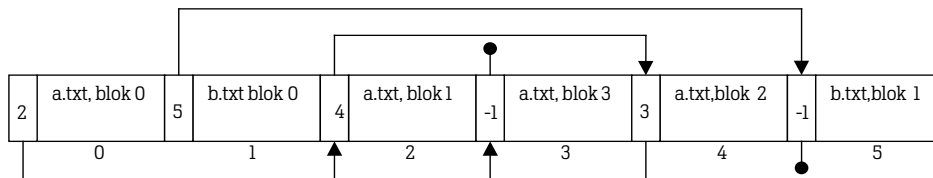


Slika 159 Struktura ISO 9960

Ovaj datotečni sistem ima ograničenja. Hijerarhija direktorija može da bude duboka do 8 direktorija. Imena datoteka su ograničena na maksimalno 8 ili 30 znakova, zavisno od odabranog nivoa kompatibilnosti. Ukupan broj direktorija može biti 65535. Pojedinačna datoteka može biti velika do 4 gigabajta. Zato je uvedeno nekoliko proširenja standarda: Joliet (Unicode imena datoteka), El-torito (mogućnost podizanja sistema sa CD), Rock-Ridge (duža imena datoteka i Unix prava), ISO 13490 (podrška više sesija, mogućnost dopunjavanja već snimljenog CD).

13.6.Alokacija datoteka ulančanim listama

Još jedan način alokacije datoteka koji ne zahtijeva dodatne tablice je alokacija ulančanim listama. kao što je to prikazano na slici Slika 160.



Slika 160 Alokacija ulančanom listom

Kod ove alokacione tehnike, svaki blok sa podacima pored sadržaja dijela datoteke ima na svom početku ili kraju i broj bloka u kome se datoteka nastavlja. Sa

ovom alokacijom nema problema sa traženjem dovoljno velikog kontinualnog bloka, jer bilo koji slobodan blok može biti iskorišten za produživanje datoteka.

Dovoljno je da se u elementu direktorija smjesti samo pozicija početnog bloka, a da se ostalim blokovima može prići koristeći listu. Pristup datotekama se obavlja sekvencijalno, a u slučaju da pristupamo datoteci slučajnim pristupom, pristup je izuzetno spor, jer je potrebno pročitati sve blokove prije traženog bloka da bi se došlo do njega.

Manje bitna mana ove alokacione tehnike, je što veličina podataka u bloku nije više jednaka stepenu broja 2, jer svaki blok na početku ima pokazivač koji pokazuje na sljedeći blok. To otežava spajanje blokova u memoriji.

Primjer ovakve alokacije je u operativnom sistemu KERNAL za *Commodore 64*. Na 18. stazi njegove diskete je direktorij s imenima datoteka i početnom stazom i sektorom svakog programa, a na početku svakog sektora je pokazivač na narednu traku i sektor (2 bajta) nakon čega slijedi 254 bajta podataka.

13.7. Alokacija sa ulančanim listama sa tablicom pokazivača

Alokacija sa ulančanim listama ima priličnu sporost u pristupu proizvoljnom bloku datoteke, jer se moraju pročitati svi prethodni blokovi, jer su pokazivači na nastavak lanca blokova smješteni u samim blokovima. Ako se pokazivači u lancu izdvoje u posebno područje particije, tada će prolazak kroz lanac biti znatno brži jer će biti moguće učitati više pokazivača (po mogućnosti sve) u RAM, a čitanje RAM-a je oko 10 000 puta brže od čitanja sa diska. Tabela u kojoj su smješteni ovi pokazivači zove se FAT (*File allocation table*). Kao u prethodnoj alokacionoj tehnici, dovoljno je imati pokazivač na prvi blok datoteke koji se drži u elementu direktorija bez obzira na veličinu datoteke. Osnovna mana, kod ove alokacione tehnike, je što cijela tabela mora biti u memoriji cijelo vrijeme, ako se želi postići direktni pristup ili ubrzati sekvencijalni. Kod veličine 32 GB sa blokovima velikim 1KB, tabela će imati preko 32 miliona elemenata. Svaki element tabele bi bio velik 4 bajta. Dakle, tabela će biti velika 128 MB. U praksi se zato u RAM memoriju ipak učitavaju samo dijelovi ove tabele, bilo eksplicitnim čitanjem dijelova tabele (MS DOS) bilo čuvanjem cijele tabele ali u virtualnoj memoriji (Windows).

2	5	4	-1	3	-1	a.txt, blok 0	b.txt blok 0	a.txt, blok 1	a.txt, blok 3	a.txt blok 2	b.txt blok 1
						0	1	2	3	4	5

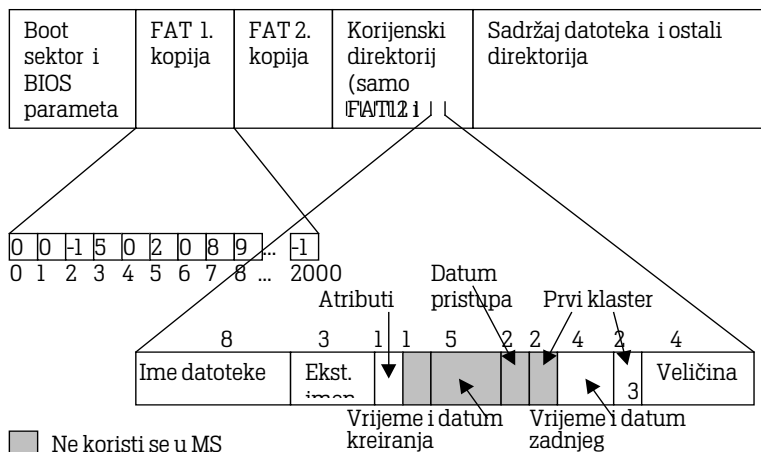
Slika 161 Alokacija ulančanom listom sa pokazivačem lanca u tabeli

Ipak, pristup je veoma popularan. MS-DOS i Windows 98 koriste FAT datotečni sistem kao svoj osnovni sistem, a i sve kasnije Windows verzije ga podržavaju. Zbog jednostavnosti strukture, to je standardni datotečni sistem na USB prijenosnim medijima.

13.8. DOS/Windows FAT

Najpoznatija realizacija datotečnog sistema sa FAT tablicama je ona korištena u MSDOS i Windows. FAT tablica se sastoji od elemenata velikih 12, 16 ili 32 bita.

Slika 162 prikazuje osnovnu strukturu FAT particije. Particija (ili disketa) s FAT datotečnim sistemom počinje *boot* blokom čiji se početni dio zove *BIOS parameter block*. U njemu se nalazi opis strukture datotečnog sistema, prije svega parametri o veličini blokova i potrebnih tablica. Nakon *boot* bloka, OS čuva na disku dvije kopije ove tabele pokazivača (*File Allocation Table*). U verzijama FAT 12 i FAT 16 iza ovih tabela se nalazi korijenski direktorij, pa područje podataka, dok u FAT 32 može stajati bit mapa slobodnih blokova ili odmah započeti područje podataka.



Slika 162 Struktura FAT particije

Svaki element FAT tabele je broj u *little endian* formatu, koji pokazuje na naredni član u lancu. U slučaju FAT 12 se dva pokazivača smjeste u 3 bajta.

Neka element s rednim brojem 0x38 pokazuje na element s rednim brojem 0x39, a on na element s rednim brojem 0x3A. Tada bi bajtovi koji odgovaraju elementima 0x38 i 0x39 imali vrijednosti kao na slici Slika 163:

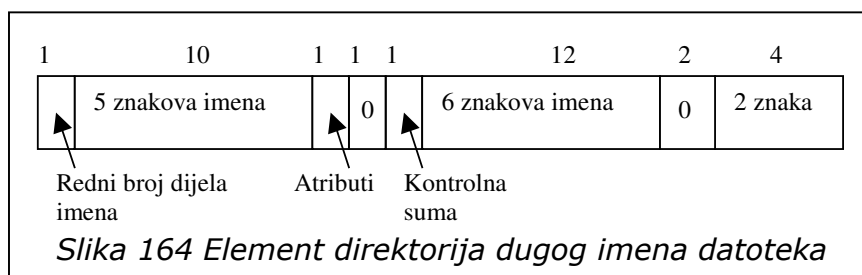
Format	Adresa od početka FAT tabele	Vrijednosti
FAT 12	0054	39 A0 03
FAT 16	0070	39 00 3A 00
FAT 32	0000E0	39 00 00 00 3A 00 00 00

Slika 163 Raspored bajtova u FAT tabeli

Element korijenskog direktorija u FAT datotečnom sistemu sastoji se od imena (8 bajta), ekstenzije imena (3 bajta), atributa datoteke (1 bajt čiji svaki bit predstavlja neki od atributa), neiskorištenog bajta (1 bajt), vremena i datuma kreiranja datoteke (5 bajta), datuma zadnjeg pristupa (2 bajta), viša 2 bajta početnog bloka (2 bajta), vremena i datuma zadnjeg pisanja (4 bajta), niža 2 bajta početnog bloka (2 bajta) i dužine datoteke (4 bajta).

Od Windows 95 i 98 struktura direktorija je proširena, dodavanjem podrške za duga imena datoteka. Postoje dva tipa elementa direktorija u Windows 98 datotečnom sistemu. Prvi od njih se zove osnovni element i osigurava ime u starom stilu, 8 + 3 znaka, koje je preuzeto od DOS-a. Ispred njega se u direktorijskoj strukturi smjeste elementi koji čuvaju dugo ime datoteka. Svaka datoteka ima element direktorija sa kratkim imenom, a mnoge mogu imati i elemente sa dugim imenom.

Slika 165 prikazuje alternativni oblik elementa direktorija koji može sadržati do 13 znakova dugog imena datoteke.



Da bi se dobilo dugo ime (npr. "Istraživanje proizvodnje nafte.doc") bilo je potrebno zapisati ga u posebne elemente direktorija, upisane ispred osnovnog imena u obrnutom redoslijedu (Slika 165). Pri tome se ime datoteke kodira koristeći *Unicode*, kako bi se omogućila i nacionalna slova u imenima. Polje atributa svakog elementa sa dugim imenom sadrži vrijednost 0x0f, koja je za prethodne sisteme nečitljiva i biće ignorirana ako se čita sa starim sistemima (na disketi na primjer). Najviši bit u polju rednog broja imena kaže sistemu koji je posljednji komad imena.

43	f	t	e	.	D	A	0	C	o	c	x					0	0				
2	p	r	o	i	Z	A	0	C	v	o	d	n	j	i		0	0	N	a		
1	I	s	t	r	a	A	0	C	7E	l	i	v	a	n	j		0	0	e	U	
I	S	T	R	A	I	~	l	D	O	C	A	0	S	Vrijeme i dan kreiranja		Dan pristu	MSB 1.blok	Vrijeme i dan pisanja		LSB 1.blok	Veličina datoteke

Slika 165 Dugo ime datoteke, položaj u tabeli direktorija

Mada izgleda veoma kompleksno, potrebno je bilo osigurati kompatibilnost sa starim sistemima.

Fiksna veličina tablica u FAT sistemu dovodi do bitnih ograničenja u maksimalnoj veličini datotečnog sistema. Što je blok veći, veći je i maksimalni kapacitet, ali je slabije iskorišten zbog interne fragmentacije (Slika 166).

Veličina bloka	FAT 12	FAT 16 (1. verzija)	FAT 16 (2.verzija)	FAT 32
512 bajta	2 MB	32 MB	32 MB	128 GB
1 KB	4 MB	32 MB	64 MB	256 GB
2 KB	8 MB	32 MB	128 MB	512 GB
4 KB	16 MB	32 MB	256 MB	1 TB
8 KB		32 MB	512 MB	2 TB
16 KB		32 MB	1 GB	2 TB
32 KB			2 GB	2 TB
64 KB			4 GB	2 TB

Slika 166 Maksimalni kapacitet FAT particije u zavisnosti od veličine bloka

Treba reći da maksimalni kapaciteti FAT 32 particija se mogu realizovati samo posebnim softverom, jer Windows 98 može formatirati FAT 32 particiju na maksimalno 2 TB, a noviji Windows 7 na daleko manjih 32 GB.

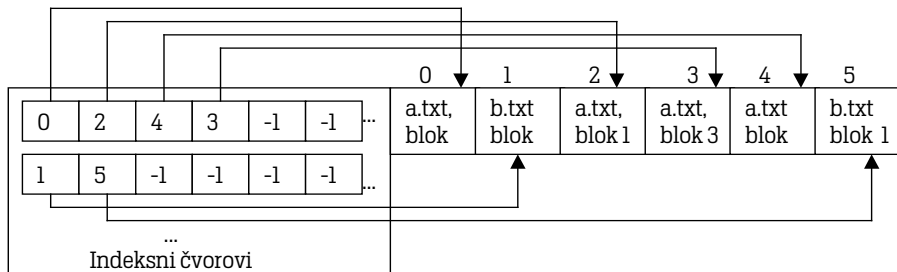
13.9. Indeksna alokacija na jednom nivou

Dosadašnji načini alokacije su čuvali informaciju o poziciji prvog bloka, ali pozicije ostalih blokova su se izvodile iz njega: prostim sabiranjem kod kontinualne alokacije, ili praćenjem lanca u disku ili memoriji kod spregnute alokacije. Nalaženje ostalih dijelova datoteke bi moglo biti mnogo brže od spregnute alokacije, a izmjene

datoteka fleksibilnije od kontinualne alokacije ako bi se negdje direktno čuvale pozicije svih blokova koji pripadaju datoteci. Prednost ove sheme u odnosu na ulančanu listu blokova datoteke u memoriji je da taj skup pokazivača treba da bude u memoriji samo za otvorene datoteke, a ne za cijelu particiju kao kod FAT tabela.

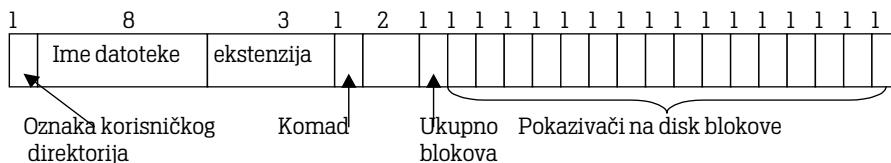
Skup pokazivača na sve blokove datoteke se može naći na jednom od dva mjesta u particiji. Jedan pristup je da se pokazivači nalaze u sadržaju direktorija zajedno sa ostalim podacima o datoteci (primjer: CP/M datotečni sistem). U drugoj varijanti, rezerviš se posebno područje, izvan sadržaja direktorija kod koga postoji poseban skup pokazivača. Elementi ovog područja se zovu **indeksni čvorovi**, skraćeno *i-node*. Svaki indeksni čvor se odnosi na pojedinu datoteku i sastoji se od više pokazivača od kojih se svaki odnosi na pojedini dio datoteke i pokazuje gdje se on fizički nalazi.

Najveći problem sa ovom vrstom alokacije je određivanje veličine indeksnog čvora ili elementa direktorija. Radi brze pretrage datoteka broj pokazivača u indeksnom čvoru ili elementu direktorija mora biti fiksna. To znači da ako je rezervirano premalo pokazivača, nije moguće zapisati velike datoteke, a ako ih je definisano previše onda će indeksni čvorovi biti ponekad veći od sadržaja datoteke koju opisuju, što je neracionalno.



Slika 167 Alokacija sa indeksnim čvorovima

13.10.CP/M datotečni sistem



Slika 168 Struktura CP/M direktorija

Jedan od prvih operativnih sistema za mikro računare, CP/M je koristio indeksnu alokaciju s jednim nivoom. Tablica elemenata direktorija, pored imena datoteka i polja *user code* (koje predstavlja redni broj direktorija) sadrži šesnaest pokazivača na blokove veličine između 128 bajta i 32K, te brojač koji kaže koliko blokova je iskorišteno (Slika 168). Ako je datoteka duža od 16 blokova, ponavlja se isti element u strukturi direktorija sa rednim brojem komada. Prema tome, datoteka teoretski može biti veličine maksimalno 4096 blokova (do 256 komada svaki po 16 blokova). Ovakav datotečni sistem ima ozbiljna ograničenja na maksimalne veličine datoteka, ali je zadovoljavao za tadašnje kapacitete disketa.

13.11. Indeksna alokacija s više nivoa

Proširenje maksimalne veličine datoteke indeksnom alokacijom postiže se uvođenjem indirektnih pokazivača. Umjesto da pokazivači u indeksnim čvorovima ili elementu direktorija pokazuju isključivo na blokove podataka, oni mogu pokazivati i na blokove pokazivača. Blok pokazivača je iste veličine kao blok podataka, ali su njegovi elementi pokazivači na druge blokove podataka. Pošto je jedan blok podataka obično znatno veći od indeksnog čvora ili elementa direktorija, u blok podataka može stati više pokazivača, a alociraće se samo ako je potrebno opisivati veće datoteke. Ideja se može proširiti da se koriste dvostruko indirektni pokazivači: pokazivač u indeksnom čvoru ili elementu direktorija pokazuje na blok sastavljen od pokazivača na nove blokove pokazivača, a svaki od novih blokova pokazivača sadrži elemente koji pokazuju na blokove podataka. Mogući su i trostruko indirektni pokazivači.

13.12. Unix datotečni sistem

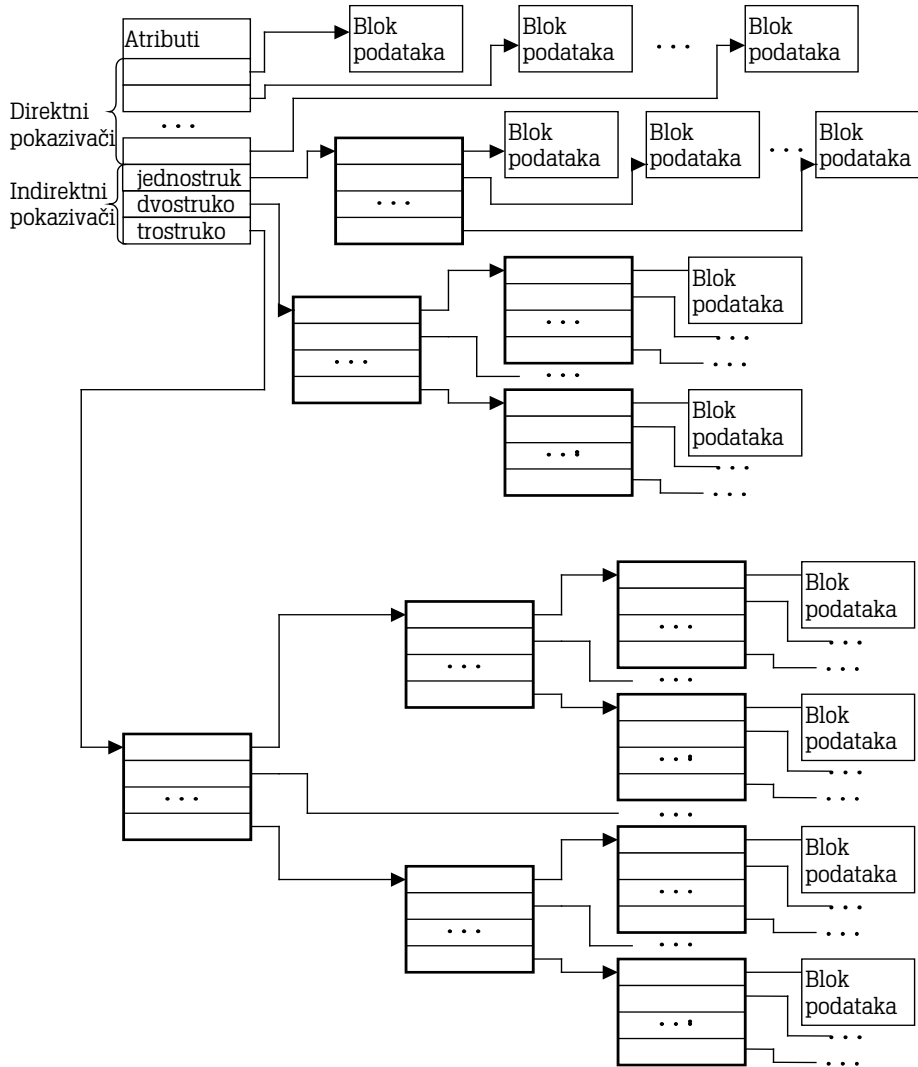
Indeksna alokacija na više nivoa primijenjena je u datotečnim sistemima kod svih operativnih sistema iz porodice Unix sistema. Jedno područje diska sadrži indeksne čvorove, relativno male veličine, po jedan čvor za svaku datoteku. Unutar indeksnog čvora se ne čuva ime datoteke, ali se čuvaju podaci o njenoj veličini, vremenima kreiranja i pokazivačima na blokove. Zavisno od podvarijanti datotečnog sistema, zaglavlje datoteke sadrži više pokazivača na blokove podataka (npr. 10), te pokazivače na blokove pokazivača, koji mogu biti jednostruko indirektni, dvostruko indirektni i eventualno trostruko indirektni pokazivači (Slika 169.)

U navedenom primjeru, ako postoji 10 direktnih pokazivača, jedan indirektni, jedan dvostruko indirektni i jedan trostruko indirektni, za pristup blokovima između 0 i 9, njihovu poziciju čitamo iz direktnog pokazivača. Neka je n broj pokazivača koji se mogu smjestiti u blok sa pokazivačima. Za pristup blokovima između 10 i $10+(n-1)$, njihove pozicije saznajemo iz pokazivača koji su smješteni u bloku na koji pokazuje jednostruko indirektni pokazivač. Za pristup blokovima između $10+n$ i $10+n+n^2-1$ najprije preko dvostruko indirektnog bloka pokazivača se nađe pokazivački blok prvog nivoa, u njemu pokazivački blok drugog nivoa, a iz pokazivačkog bloka drugog nivoa dođe do podataka. Za pristup podacima između bloka $10+n+n^2$ i $10+n+n^2+n^3-1$ se koriste tri nivoa indirekcije.

Prema tome, u ovakvim datotečnim sistemima, gornja granica veličine datoteke je određena s dva limita, zbog načina alokacije i veličine pokazivača, od kojih se uzima manji od njih. Ako je p veličina pokazivača, b veličina bloka, $n=b/p$ broj pokazivača u bloku, D broj direktnih pokazivača u indeksnom čvoru, I broj indirektnih pokazivača u indeksnom čvoru, V broj dvostruko indirektnih pokazivača u indeksnom čvoru, T broj trostruko indirektnih pokazivača u indeksnom čvoru, maksimalna veličina datoteke zbog načina alokacije je $M=b(D+In+Vn^2+Tn^3)$. Maksimalna veličina datoteke zbog veličine pokazivača je $M=2pb$.

Ovakav način alokacije datoteka se odlikuje dobrom skalabilnošću, jer je zadovoljavajući i za male i za velike datoteke. Blokovi se alociraju dinamički i indirektni blokovi postoje samo za velike datoteke. Pristup malim datotekama je

jako brz. Mana pristupa je što velike datoteke zahtijevaju puno pretraživanja da bi se pristupilo indirektnim blokovima.



Slika 169 Unix datotečni sistem, indeksna alokacija na 1,2,3 i 4 nivoa

Jednom kada se nađe indeksni čvor, može se pristupiti svim blokovima datoteke. U nekim Unix sistemima indeksni čvor je u rezervisanom dijelu diska, pa se datotekama pristupa interno, navodeći poziciju indeksa. Mana ovog pristupa je što indeksni čvor nije blizu podacima (i to zahtjeva kretanje glave diska) i što je broj indeksnih čvorova unaprijed određen u vrijeme formatiranja diska. U kasnijim verzijama kao što su *Berkeley fast file system* (FFS) ili Ext2 (linux) indeksni čvorovi su raspoređeni po disku na različitim mjestima, ali se njihova pozicija opet može izračunati iz njegovog broja.

Kako nalazimo broj indeksnog čvora za datoteku? On je naveden u strukturi datoteke koja predstavlja direktorij.

Struktura direktorija u tradicionalnom Unix sistemu je izuzetno jednostavna i prikazana je na slici Slika 170. Svaki element direktorija sadrži ime datoteke i broj indeksnog čvora vezanog za odgovarajuću datoteku. Sve dodatne informacije o tipu, veličini, vremenima i vlasništvu kao i diskovnim blokovima sadržaja datoteke su sadržane u indeksnom čvoru.

2	14
Broj indeksnog čvora	Ime datoteke

Slika 170 Element Unix direktorija

13.13. Ekstentna alokacija datoteka

Porastom kapaciteta diskova i potrebom za većim datotekama zbog predstavljanja video sadržaja, uočava se nedostatak indeksne u odnosu na kontinualnu alokaciju. Pristup velikoj datoteci koristeći indeksne čvorove s tri nivoa, često zahtijeva čitanje četiri bloka da bi se pročitao jedan, dok se kod kontinualne alokacije tom bloku može direktno pristupiti. Stoga, neki datotečni sistemi kombinuju indeksnu alokaciju na jednom nivou i kontinualnu alokaciju. Umjesto da pokazivači indeksnog čvora pokazuju na blokove fiksne veličine, oni pokazuju na kontinualne nizove blokova promjenjive veličine, koji se zovu ekstenti. Tako se u području direktorija čuva za svaki ekstent početni blok i koliko blokova on zauzima. Na primjer, u području direktorija se uz datoteku može navesti sljedeća tabela kao na slici Slika 171, koja će značiti da ova datoteka zauzima blokove 12, 13, 100, 101, 102, 103, 104, 105, 1200, 1201

Početni blok	Veličina
12	2
100	5
1200	2

Slika 171 Primjer ekstentne alokacije

Postojanje velikih kontinualnih blokova predstavlja malu fragmentaciju u smislu fizičke pozicije bloka. Mana ove alokacije je potencijalna eksterna fragmentacija kada disk postane skoro pun, pa se ne može pronaći dovoljno veliki kontinualni blok. Pored toga, potrebno je rezervirati više prostora u svakom elementu direktorijske strukture, nego kod drugih načina alokacije.

13.14. NTFS

NTFS (engl. *New Technology File System*) je standardni Microsoft-ov datotečni sistem. Ovaj datotečni sistem je jedan od najpoznatijih predstavnika ekstentne alokacije. Ovdje će se prikazati kratak opis ovog sistema sa nekim od problema sa kojima se suočava kao i rješenja tih problema. NTFS je dizajniran za pouzdanost, sigurnost i podršku velikim uređajima. Skalabilnost je postignuta upotrebom generičkih struktura podataka koje se omotaju oko struktura podataka specifičnog sadržaja. Za razliku od većine datotečnih sistema, u NTFS je sve datoteka (i bit mape, indeksi, direktorij) i može se nalaziti bilo gdje na disku. Samo *boot* sektor je na

fiksnom mjestu i pokazuje na \$MFT datoteku. Druga specifičnost je struktura same datoteke. U Unix-u, datoteka se predstavlja nizom bajta, dok je kod NTFS-a, datoteka **skup atributa** i svaki atribut predstavlja niz bajta. Aplikativni programi, kroz API pozive, datoteke i dalje vide kao nizove bajtova.

Poz	Ime	Opis
0	\$MFT	Sama MFT
1	\$MFTMirr	Kopija prvih elemenata MFT
2	\$LogFile	Žurnal koji prati izmjene metapodataka
3	\$Volume	Informacije o disku
4	\$AttrDef	Opisi svih atributa: identifikatori, imena i veličine
5	.	Korijenski direktorij
6	\$Bitmap	Alokacijski status svakog bloka
7	\$Boot	Boot sektor
8	\$BadClus	Blokovi s lošim sektorima
9	\$Secure	Informacije o pravima nad datotekama
10	\$Upcase	Verzija velikim slovima svakog Unicode znaka
11	\$Extend	Direktorij s datotekama s opcionim proširenjima

Slika 172 Sadržaj datoteke \$MFT

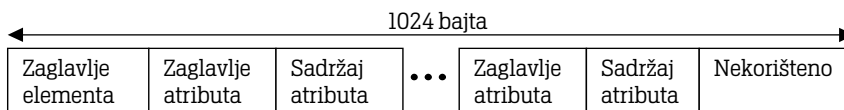
Najvažnija datoteka (iako nevidljiva bez specijalnih alata) u NTFS-a je \$MFT (engl. *Master File Table*). U njoj su opisane ostale datoteke, pri čemu opis svake zauzima po 1 KB u \$MFT-u i sastoji se od zaglavlja (42 bajta) i atributa. Dvanaest prvih datoteka u MFT su specijalne datoteke navedene u tabeli, na slici Slika 172

Struktura zaglavlja elementa koje opisuje pojedinu datoteku unutar \$MFT je prikazana na slici Slika 173.

Pozicija	Značenje	Bitno
0-3	Oznaka "FILE" za datoteku ili "BAAD" za loš blok	Ne
4-5	Pozicija u nizu popravaka	Da
6-7	Broj elemenata u nizu popravaka	Da
8-15	\$LogFile LSN	Ne
16-17	Broj sekvence	Ne
18-19	Broj logičkih veza	Ne
20-21	Pozicija prvog atributa	Da
22-23	Flegovi (u upotrebi, direktorij)	Da
24-27	Korištena veličina MFT elementa	Da
28-31	Alocirana veličina MFT elementa	Da
32-39	Veza datoteke na bazni slog	Ne
40-41	ID sljedećeg atributa	Ne
42-1023	Atributi i područje za popravke	Da

Slika 173 Struktura zaglavlja opisa datoteke u MFT

Nakon zaglavlja MFT elementa preostalih 1024-42 bajta popunjavaju njegovi atributi. Kako pokazuje Slika 174, svaki atribut ima dalje zaglavlje i sadržaj.



Slika 174 Struktura elementa MFT tabele

Zaglavlje atributa određuje njegov tip (numerički identifikator), veličinu i ime (Unicode UTF 16), flegove da li je kompresovan i šifrovan. Neki važniji atributi s navedenim identifikatorom su: \$FILE_NAME (48) za ime datoteke, \$BITMAP (176) za slobodne blokove, \$DATA (128) sa sadržajem, \$INDEX_ROOT (144) za opis direktorijske strukture, \$INDEX_ALLOCATION (160) za elemente direktorijske strukture.

Tijelo atributa se može smjestiti u samu MFT datoteku ili biti eksterni atribut (pokazuje na blok). Tijelo atributa može biti i sam sadržaj datoteke. Za kratke datoteke (do 700 bajta) podaci se mogu staviti i u sam MFT. Za duže datoteke, u zaglavlju \$DATA (tip 128) atributa polje *runlist* pokazuje na listu ekstenta.

Kako se u opisu datoteke može nalaziti više \$DATA atributa, NTFS datoteka može da ima više od jednog toka podataka. Višestruki tokovi podataka imaju svoju upotrebu, na primjer, velike grafičke slike mogu imati i male grafičke slike smještene u istoj datoteci (engl. *thumbnail*). Tok podataka može sadržati do 264 bajta.

Kada je \$DATA atribut nerezidentni, on pokazuje na seriju kontinualnih blokova memorije, pa se zato ovaj datotečni sistem ubraja u ekstentne sisteme, pri čemu se ovdje koristi terminologija *data-run* za svaki kontinualni niz sektora.

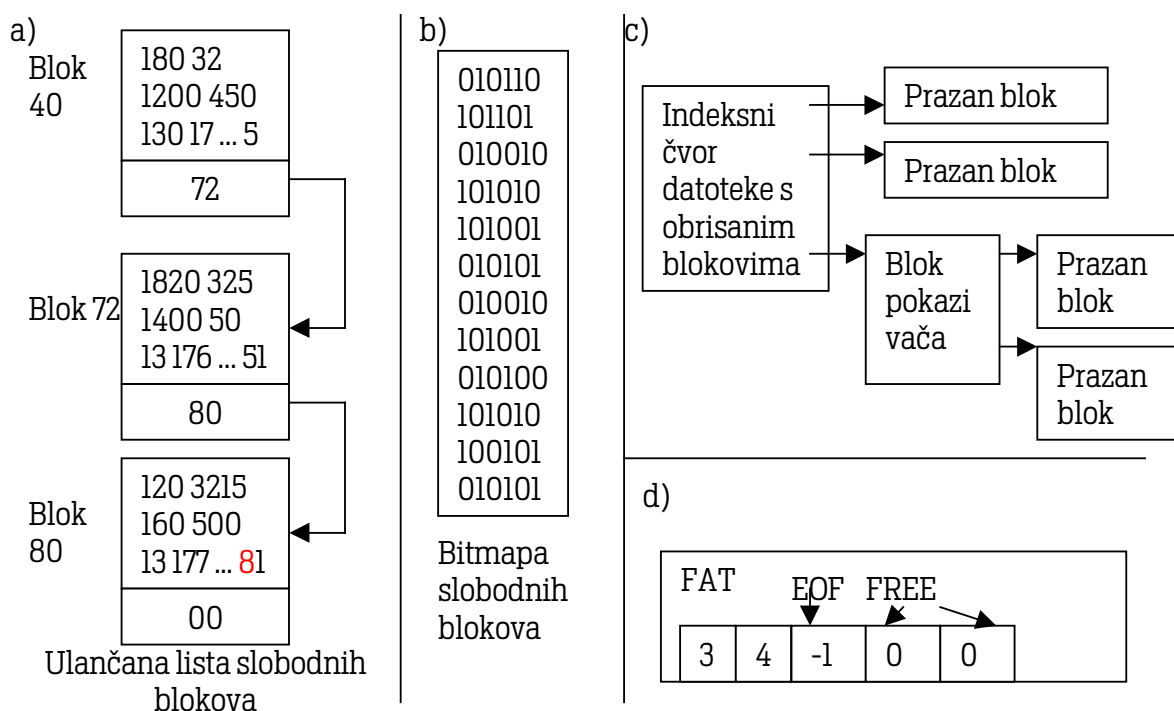
To važi i za samu \$MFT. Inicijalno se rezerviše 12,2% prostora na particiji za rast MFT-a. Dakle, on može rasti bez fragmentiranja sve dok se inicijalno pridružen prostor ne ispuni, poslije čega se rezerviše drugi veliki prostor. Dakle, ako MFT postane fragmentiran on će se sastojati od malog broja vrlo velikih fragmenata.

Direktoriji su organizovani kao balansirano B stablo u atributima \$INDEX_ROOT (rezidentni) i \$INDEX_ALLOCATION (nerezidentni) koji sadrže \$FILE_NAME attribute. Ovakva struktura omogućava brzo nalaženje datoteka i ta lista se automatski sortira pri dodavanju datoteke.

NTFS obezbjeđuje dodatne mogućnosti kao što su: složeni sistem zaštite, šifrovanje i kompresija podataka.

13.15. Vođenje evidencije o slobodnim blokovima

Ako zanemarimo stanje u kome su blokovi na disku neispravni, blok na disku može biti dodijeljen datotekama ili slobodan, pa na prvi pogled rješavanje problema alokacije blokova na disku, rješava i pitanje upravljanja slobodnim blokovima. Zaista, u nekim datotečnim sistemima kao što je FAT to i jeste slučaj. U većini datotečnih sistema, međutim, traženje slobodnog bloka na način da se pregledaju svi blokovi koji su dodijeljeni datotekama, pa na kraju slobodnim blokom smatraju oni koji nisu dodijeljeni ni jednoj datoteci bi bilo nepotrebno sporo. U cilju ubrzanja traženja slobodnog bloka koji će se dodijeliti novim datotekama vrijedi žrtvovati još malo prostora u particiji gdje će se evidentirati slobodni blokovi.



Slika 175 Upravljanje slobodnim prostorom

Pet metoda se koristi u ovu svrhu. Prva se sastoji od korištenja **ulančane liste diskovnih blokova** (Slika 175a), pri čemu svaki blok drži brojeve slobodnih diskovnih blokova. Sa 1 KB blokom i 32 bitnim brojem diskovnog bloka, svaki blok može držati brojeve 255 slobodnih blokova, pri čemu je jedno mjesto potrebno za pokazivač na sljedeći blok. Jedan disk veličine 256 GB treba listu slobodnih blokova od maksimalno 1,052,689 blokova da bi držao svih 2^{28} brojeva diskovnih blokova. Često se slobodni blokovi koriste da bi držali listu slobodnih blokova.

Kada se koristi metoda sa ulančanim listama, samo jedan blok pokazivača se drži u glavnoj memoriji. Kada se datoteka kreira, potrebni blokovi se uzimaju iz liste blokova sa pokazivačima. Kada se završi sa korištenjem datoteke, čitaju se novi blokovi pokazivača u memoriju sa diska. Slično, kada se obriše datoteka, blokovi se oslobađaju i dodaju u blok pokazivača na slobodne blokove. Kada se ovaj blok popuni, piše se na disk.

Druga tehnika za upravljanje slobodnim prostorom na disku je **bit mapa** (Slika 175b). Jedan disk sa n blokova zahtjeva mapu sa n bita. Slobodni blokovi su predstavljeni sa 1 u mapama, a alocirani blokovi sa 0 (ili obrnuto). Jedan 256 GB disk ima 228 blokova od 1 KB i zahtjeva 228 bita za mapu, koja zahtjeva 32,768 blokova. Nije iznenađenje da bit mapa zahtjeva manje prostora pošto koristi 1 bit po bloku prema 32 bita kod modela sa ulančanim listama. Samo ako je disk gotovo pun (tj. ima nekoliko slobodnih blokova) će model sa ulančanim listama zahtijevati manje blokova nego bit mapa.

Treća metoda je uvezana **lista indeksnih blokova** (Slika 175c). Koriste se specijalni indeksni čvorovi koji pokazuju na slobodne blokove. Svaki indeksni blok sadrži adrese slobodnih blokova i pokazivač na sljedeći indeksni blok. Veliki broj slobodnih blokova se može naći veoma brzo.

Četvrta metoda je korištenje podataka iz **druge strukture** za opis datoteka da se zaključi koji su blokovi slobodni (Slika 175d). Na primjer, u FAT datotečnom sistemu

FAT tablica navodi za svaki blok (blok) njegovog sljedbenika u lancu datoteke. No, ista tablica se može koristiti i za informaciju da li je neki blok slobodan ili neispravan, npr. ukoliko se neki blokovi ne mogu pojaviti u lancu, njihovi brojevi mogu se koristiti u FAT tabeli za oznaku slobodnog bloka. U FAT 16 sistemu vrijednosti 0, 0xFFFF i 0xFFFF u FAT tabeli ne predstavljaju brojeve blokova gdje se lanac nastavlja, nego redom predstavljaju slobodni blok, neispravan blok i blok na kraju FAT lanca.

Peta metoda je vezana za ekstentnu i kontinualnu alokaciju. Neki datotečni sistemi, kao što je NTFS, rezervišu **slobodni kontinualni prostor** za konkretnu datoteku određene veličine (npr. 64K za NTFS), čak i ako je ona kraća. Prilikom širenja datoteke, prvo se gleda da li ona ima slobodnog prostora ranije dodijeljenog za nju, a tek ako nema, gledaju se slobodni blokovi predstavljeni bit mapama.

13.16. Konzistentnost i pouzdanost sistema datoteka

Metode povećanja pouzdanosti na nivou disk blokova (prepoznavanje loših sektora i RAID sistemi) nisu same po sebi dovoljne za postizanje pouzdanosti datotečnog sistema. Datotečni sistem treba da bude, koliko je to moguće, zaštićen od fizičkih i korisničkih grešaka. Greške u ovom podsistemu imaju najgore posljedice od svih grešaka u operativnim sistemima, jer dok greška u upravljanju procesima ili memorijom uglavnom nemaju teže posljedice od ponovnog pokretanja operativnog sistema i gubitka nekoliko sati posla, greške u datotečnom sistemu mogu upropastiti podatke čuvane godinama.

Ranije spomenuta tehnika rezervnih sektora u stazi (koje prepoznaje disk kontroler ili drajver za rad sa diskom) uz premapiranje ostalih sektora da ne koriste oštećeni ima ograničenje ako su potrošeni svi rezervni sektori u toj stazi, a detektovan novi loš sektor u toku rada. Neki datotečni sistemi, stoga moraju također voditi evidenciju o lošim blokovima. Na primjer, u FAT datotečnom sistemu, postoji i specijalna vrijednost u FAT tabeli koja označava da je sektor loš i da se neće koristiti za dodjelu datotekama. U drugim datotečnim sistemima može se dodati specijalna tajna datoteka sastavljena isključivo od loših blokova. Treba, međutim, reći da ako je disk došao do takvog stanja da se loši blokovi moraju označavati na nivou datotečnog sistema da je bolje takav disk zamijeniti, jer je vjerovatnoća njegovog potpunog pada postala jako velika.

Datotečni sistem je, međutim, mnogo više od skupa blokova. Sastoji se od različitih tabela koje moraju biti usuglašene. Kada sve tabele imaju ispravne vrijednosti, datotečni sistem je **konzistentan**. Mnogi datotečni sistemi moraju obaviti više operacija na disku pri ažuriranju stanja datoteka. Primjer predstavlja situacija u kojoj se briše datoteka. Pri brisanju datoteke, ako je u pitanju Unix sistem, treba isprazniti indeksni čvor, sve blokove sa pokazivačima na drugom trećem i četvrtom nivou proglasiti slobodnim u bit mapi, sve blokove podataka proglasiti slobodnim u bit mapi, obrisati datoteku iz svih direktorija koji pokazuju na nju, a ako je riječ o sigurnom brisanju, popuniti nulama i sve bajtove sa podacima. Ili u FAT datotečnom sistemu, pri brisanju datoteke treba njen opis obrisati iz direktorija i proglasiti blokove u njenom bivšem lancu slobodnim u obije kopije FAT tabele. Samo ako je sve ovo urađeno operacija brisanja datoteke je uspješno završena. Ako u toku ove operacije nestane električne energije može se desiti da bit mapa pokazuje da je

blok slobodan, a da datoteka koja je zauzimala ovaj blok još uvijek postoji ili da indeksni čvor pokazuje da je blok zauzet. Nekonzistentni datotečni sistem je vrlo opasan, jer se tako može isti blok dodijeliti u dvije različite datoteke bez korisnikove namjere što dovodi do gubitka podataka.

Konzistentnost datotečnog sistema se povremeno provjerava specijalnim alatima kao što su Unix *fsck* ili Windows *chkdsk* ili *scandisk* u nekim verzijama. Ovi alati se mogu pokrenuti kad god je potrebno, posebno kod pada računara. Alati rade na sljedećem principu. Pripreme se dvije tabele blokova u memoriji: tabela zauzetih i tabela slobodnih blokova. Tabela slobodnih blokova se direktno dobije iz strukture za upravljanje slobodnim blokovima (ulančane liste slobodnih blokova ili bit mape). Tabela zauzetih blokova se dobiva analizirajući indeksne čvorove ili lance alokacije za svaku datoteku. Svaki put kada se zaključi da blok pripada datoteci povećava se odgovarajući element tabele zauzetih blokova za 1. Kada su pregledane sve strukture koje prate zauzete i slobodne blokove (npr. indeksni čvorovi i bit mape) mogu se usporediti ove dvije tabele u memoriji. Ako su svi blokovi ili zauzeti ili slobodni, ovaj dio datotečnog sistema je konzistentan. Ali, ako se pokazalo da je neki blok istovremeno zauzet ili slobodan, ili da nije ni zauzet ni slobodan, ili (najgore) da je zauzet sa tako da dvije datoteke pokazuju na njega, datotečni sistem je nekonzistentan i alat za popravku datotečnog sistema treba da sredi potrebne tabele.

Da se izbjegne test cijelog sistema na nekonzistentnost, uvodi se poseban dio datotečnog sistema ili specijalna datoteka koji se zove **žurnal**. Kada se zahtjev za višefaznom akcijom nad datotečnim sistemom, u žurnal se zabilježe unaprijed sve promjene koje se trebaju uraditi. U slučaju kraha, prilikom novog podizanja operativnog sistema, samo se pročita žurnal i ponove planirane aktivnosti. Promjene su atomske pa ili će se obaviti u cijelosti ili neće biti uopće započete, jer nisu ni upisane u žurnal. Primjeri ovakvih datotečnih sistema su NTFS i Ext4FS.

Kada god je to moguće, sadržaj datoteka treba kopirati na rezervno mjesto. Ovo kopiranje se zove **backup**. Time se obezbjeđuje oporavak u slučaju kvara diska, nepopravljivih grešaka u konzistentnosti datotečnog sistema, a naročito korisničkih grešaka poput slučajnog brisanja datoteke. Kopiranje se obavlja na vanjske medije poput magnetnih traka, optičkih diskova, mrežnih diskova a u najnovije vrijeme preko Internet servisa u oblaku. *Backup* može biti baziran na sljedećim principima:

- **Fizičko kopiranje cijelog diska:** Kod ovog pristupa *backup* program redom čita cijeli disk, sektor po sektor i upisuje svaki sektor u specijalnu datoteku na drugom mediju. Glavna prednost ovog pristupa je što će omogućiti oporavak sistema i nakon najtežih problema kao što je brisanje MBR područja, a mana što se nepotrebno kopiraju i prazni i loši blokovi, pa kopiranje zna potrajati.
- **Fizičko kopiranje particije:** Slično je prethodnom, ali kopira blok po blok samo odabrane particije. Dobivena datoteka sa kopijom sadržaja particije ima istu strukturu kao datotečni sistem. Na Linux sistemima takva datoteka se može montirati kao da je stvarni datotečni sistem (*loopfs*) i time naći i pojedinačni podatak.
- **Offline kopiranje particije** je fizičko kopiranje particije uz uslov da ona u tom trenutku nije uključena u datotečne sisteme koje operativni sistem prati. Kod fizičkog kopiranja montirane particije postoji rizik kopiranja nekonzistentnog

datotečnog sistema, na primjer ako se kopiranje indeksnih čvorova ili bit mape obavljalo baš u trenutku kada se brisala neka datoteka. Ako datotečnom sistemu zabranimo korištenje particije prije početka kopiranja (zaključavanje ili demontiranje) ovaj problem se izbjegava.

- **Kopiranje svih datoteka** u sistemu ima prednosti što kopira i one datoteke koje su na različitim fizičkim diskovima, lakše je naći pojedinačne datoteke za vraćanje, a obnova podataka je moguća i ako se novi kupljeni disk po kapacitetu razlikuje od onoga koji je kopiran. Ova strategija kopiranja je najsporiji način, ima problema sa kopiranjem otvorenih datoteka, nepotrebno kopira već kopirane datoteke i ne kopira neke metapodatke u datotečnom sistemu poput *boot* sektora
- **Kopiranje izmijenjenih datoteka** (inkrementalno kopiranje) koristi specijalni arhivni atribut u datotekama. Nakon što su sve datoteke kopirane, taj atribut im se postavi na 1 kao oznaka da su kopirane, a pri promjeni datoteke on se postavi na 0. Alternativno, može se gledati i vrijeme zadnje izmjene datoteke. Kada se pokrene novi *backup* sada se kopiraju samo izmijenjene datoteke.
- Kopiranje **datoteka pojedinih korisnika** kopira samo datoteke koje je kreirao određeni korisnik, npr. samo datoteke iz direktorija */home/petar*. Može se kombinovati sa inkrementalnim kopiranjem.
- **Inkrementalno kopiranje dijelova** datoteka se koristi u bazama podataka gdje se prati koji dijelovi datoteka su izmijenjeni i nakon prvog kopiranja cijele datoteke, dalje se snimaju samo njeni dijelovi.
- **Kopiranje metapodataka** kopira one dijelove datotečnog sistema koji nisu sadržaj datoteka i koristi se kao dopuna kopiranju datoteka

Datoteka ili skup datoteka u koju je obavljeno kopiranje se može dodatno kompresovati radi smanjenja prostora ili kriptografisati radi zaštite privatnosti podataka.

13.17. Performanse datotečnog sistema

U poglavlju o vanjskoj memoriji spomenute su neke tehnike za ubrzavanje rada diskova na nivou blokova: raspoređivanje zahtjeva radi smanjenja kretanja glave, keširanje blokova u memoriji računara i samom disk kontroleru, upotreba DMA i slično. Pametnim izborom organizacije datotečnog sistema, mogu se postići ubrzanja i na nivou datotečnog sistema. Takve tehnike su keširanje datotečnog sistema, čitanje unaprijed i fizičko smještanje dijelova datotečnog sistema.

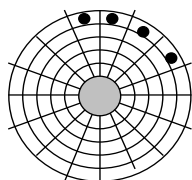
Neki operativni sistemi obavljaju keširanje ne na nivou fizičkih disk blokova (kako npr. radi Windows 95) nego na nivou dijelova datoteka (kako radi Windows NT) ili na nivou blokova uz uvažavanje strukture datotečnog sistema. Dok varijanta sa keširanjem fizičkih disk blokova omogućava brzo pretraživanje keša, prednosti keširanja dijelova datoteka su u jednakom načinu keširanja bez obzira gdje je datoteka: na lokalnom *hard* disku ili na mreži. Keširanje koje uzima u obzir strukturu datotečnog sistema čini postupak pouzdanijim, jer može odrediti da se važniji podaci, npr. indeksni čvorovi uvijek odmah upisuju na disk.

Slično straničenju sa predviđanjem, mogu se i blokovi datoteka ubacivati u keš i prije nego su potrebni. Ako datotečni sistem zaključi da se datoteka čita sekvencijalno, npr. ako nije pozvan poziv *seek*, on može prilikom čitanja bloka sa diska učitati i nekoliko blokova koji slijede iza njega i smjestiti ih u keš. Ako su ti blokovi i fizički blizu na disku, ovo čitanje se obavlja u jednoj operaciji. Onog trenutka kada se pokuša slučajni pristup, sa tom datotekom se više ne radi učitavanje unaprijed, jer postaje kontraproduktivno.

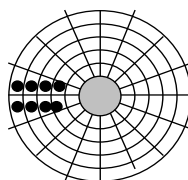
Tehnike koje povećavaju performanse datotečnog sistema na *hard* diskovima koje uzimaju u obzir fizičku poziciju diskova su aproksimacija kontinualne alokacije, defragmentacija i pametni izbor pozicije pojedinih područja datotečnog sistema.

Pri alociranju datoteka, čak i sa alokacionim tehnikama koje to ne zahtijevaju poput spregnute ili indeksne, treba nastojati da što više logičkih blokova bude fizički susjedno. U normalnom radu to se postiže analizom bit mape ili liste slobodnih blokova, npr. ako je bilo potrebno snimiti 50 kilobajta sa blokovima velikim po 1 kilobajt, bolje je odabrati onih 50 blokova koji su susjedni nego proizvoljnih 50 blokova, jer će to ubrzati kasnije sekvencijalno čitanje.

Ali i sa urednom primjenom prethodne tehnike, vremenom će datoteke biti rasute u više dijelova koji su fizički udaljeni na disku. **Defragmentacija** je postupak (obično se obavlja posebnim alatom) koji premješta logičke blokove datoteke tako da susjedni logički blokovi budu na susjednim fizičkim lokacijama. Ovaj postupak je posebno potreban kod FAT datotečnog sistema. Defragmentacija je dosta dugotrajna, ali kasnije dosta poboljšava performanse sistema.



a) Indeksni čvorovi
na krajnjoj stazi



b) Indeksni čvorovi
na svim stazama

Slika 176 Optimizacija lokacije indeksnih čvorova

Još jedan način poboljšanja performansi je vezan za raspored područja datotečnog sistema na fizičkom disku. U sistemu koji koristi indeksne čvorove ili neki ekvivalent indeksnim čvorovima čitanje čak i kratkih datoteka zahtijeva dva pristupa disku: jedan za indeksni čvor datoteke i jedan za njen blok podataka. Uobičajeno smještanje indeksnih čvorova na disk je da su svi indeksni čvorovi blizu početka particije, tako da će pisanje podataka zahtijevati stalno šetanje glave diska između cilindara sa indeksnim čvorom, cilindara sa bit mapom i cilindara sa blokovima podataka. (Slika 176a)

U kasnijim datotečnim sistemima poput Ext2fs. (Slika 176a), svaki cilindar sadrži bitmapu, indeksne čvorove i blokove podataka. Kada se formira nova datoteka, bilo koji indeksni čvor može biti izabran, mada se pokušava da se nađu blokovi i indeksni čvor na istom cilindru kako bi se minimiziralo kretanje glave. Ako to nije moguće, onda se koriste blokovi iz susjednog cilindra (njegova blizina zahtijeva manje vremena za pozicioniranje)

Izbor pravog datotečnog sistema za pravu primjenu veoma utiče na performanse. Primjer predstavlja izbor kontinualne alokacije kao principa datotečnog sistema na CD. Pošto se datoteke ne mogu proširivati, nema potrebe za naprednijim načinom alokacije.

13.18. Zadaci

Zadatak 1. Prazna particija je formatirana s blokovima velikim 1K. Na početku particije se nalaze Boot(BT), superblok(SB), bitmapa(BM) i zauzimaju po jedan blok. Nakon toga se nalazi eventualni blok s potrebnim tabelama za ovu vrstu alokacije. Prvi blok s podacima sadrži korijenski direktorij(RD) i u njega su redom stavljene datoteke d1 (5K), d2(6K), d3 (2K). Nakon toga je d1 uvećana za 2K. Nacrtati strukturu ove particije, prikazavši sadržaj blokova s metapodacima koji opisuju ove datoteke i pripadnost blokova datotekama. Koristi se a) kontinualna alokacija, b) ekstentna alokacija, c) spregnuta alokacija s pokazivačima u blokovima, d) FAT, e) indeksna s jednim nivoom, 5 pokazivača u direktoriju, f) indeksna s više nivoa, 3 direktna, 1 indirektni pokazivač u indeksnom čvoru.

Rješenje:

a) kontinualna alokacija, Raspored blokova na disku je na sljedećoj slici.

[illegible]

Sadržaj korijenskog direktorija (datoteka, početni blok, dužina)

$$(d_{1,17,7}) : (d_{2,9,6}) : (d_{3,15,2})$$

Specifičnost alokacije: produženje datoteke znači njeno kopiranje

b) ekstentna alokacija

[illegible]

Sadržaj korijenskog direktorija

dl početni blok 4 dužina 5 i početni blok 17 dužina 2

d2 početni blok 9, dužina 6

d3 početni blok 15, dužina 2

Specifičnost alokacije: u korijenskom direktoriju se navodi više dijelova

c) spregnuta alokacija s pokazivačima u blokovima

[illegible]

Sadržaj korijenskog direktorija (datoteka, početni blok, dužina)

$(d_{1,4}, 8) ; (d_{2,10}, 7); (d_{3,17}, 3)$

Specifičnost alokacije: Blokovi su u lancu, kako pokazivači takođe troše 2-4 bajta svakog bloka, svaka datoteka ima još po jedan blok, jer datoteka od 4K zauzima na disku 4K i 16 bajtova.

d)FAT

[illegible]

Sadržaj korijenskog direktorija (datoteka, početni blok, dužina)

(d1, 5, 7); (d2,10, 6); (d3,16,2)

Sadržaj FAT tabele

-1-1-1-1-16789181112131415-117-119-1

Specifičnost alokacije: Blokovi su u lancu, ali su pokazivači u posebnom bloku

e) indeksna s jednim nivoom

bt	Sb	bm	rd	d1	d1	d1	d1	d1	d2	d2	d2	d2	d2	d3	d3	d1	d1						
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	--	--	--	--	--	--

Sadržaj korijenskog direktorija

d1 Blokovi 4,5,6,7,8 (dužina 7)

d1 Blokovi 17,18 (dužina 7)

d2 Blokovi 9,10,11,12,13, (dužina 6)

d2 Blokovi 14, (dužina 6)

d3 Blokovi 15, 16, (dužina 2)

Specifičnost alokacije: Ako je nedovoljno linkova u elementu direktorija, on se ponavlja.

f) indeksna s više nivoa

bt	Sb	bm	in	Rd	i1	d1	d1	d1	d1	d1	i2	d2	d2	d2	d2	d2	d3	d3	d1	d1				
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	--	--	--	--

d1.txt, Indeksni čvor 1; d2.txt Indeksni čvor 2; d3.txt Indeksni čvor 3

Indeksni čvor 1: Direktni pokazivači 6,7,8, indirektni pokazivač 5

Indeksni čvor 2: Direktni pokazivači 12,13,14, indirektni pokazivač 11

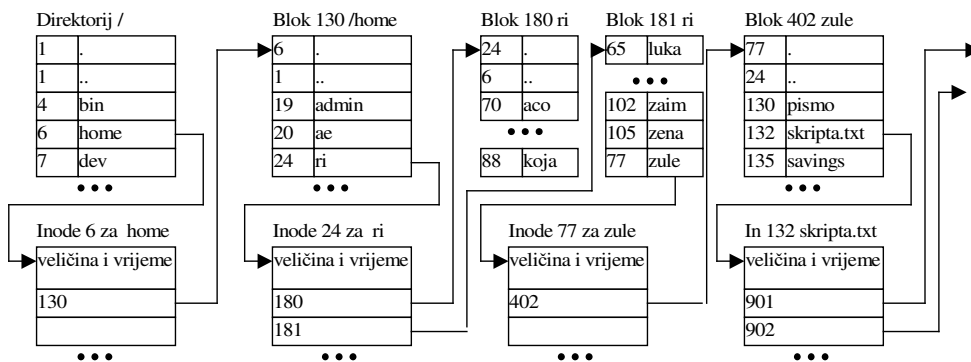
Indeksni čvor 3: Direktni pokazivači 18,19

Blok pokazivača 5: Pokazivači 9,10,20,21

Blok pokazivača 11: Pokazivači 15,16,17

Zadatak 2. Koliko je potrebno čitati blokova da bi se otvorila datoteka */home/ri/zule/skripta.txt* ? Direktorij *ri* zauzima dva bloka, a ostali po jedan. Nacrtati sliku na kojoj se vide svi blokovi kao i njihov sadržaj (indeksni čvor ili blok podataka).

Rješenje: Ukupno se čita do 9 blokova, prikazanih na slici. Brojevi blokova i indeksnih čvorova su odabrani proizvoljno u primjeru.



Zadatak 3. Razmotrimo indeksne čvorove kod UNIX-a sa 9 direktnih pokazivača, jednim indirektnim i jednim dvostruko indirektnim pokazivačem. Pretpostavimo da

je veličina bloka jednaka 8 KB i da je veličina pokazivača 4 bajta. Koristeći ovakav datotečni sistem, kolika je maksimalna veličina datoteke u ovom sistemu?

Rješenje: $9 \cdot 8 \cdot 1024 + (8 \cdot 1024 / 4) \cdot (8 \cdot 1024) + (8 \cdot 1024 / 4)^2 \cdot (8 \cdot 1024)$ bajta = 32G+16M+72K

Zadatak 4: Na particiju veličine 40M, treba smjestiti 80 datoteka od 100 bajtova, 10 datoteka od 20000 bajtova u korijenski direktorij koji ima mjesta za 128 datoteka. Blok zauzima 4 sektora. Koliko je zauzetih podataka na ovoj particiji ako je datotečni sistem

a) FAT 12, b) FAT 16, c) FAT 32

Ukoliko na nekom datotečnom sistemu nije moguće formatirati s ovom veličinom bloka, odrediti minimalnu veličinu bloka.

Rješenje:

a) FAT12: Sektor je 512 bajta, pa je blok velik 2K. Ukupno ima $40 \cdot 1024 \cdot 1024 / (2 \cdot 1024) = 20 \cdot 1024 = 20480$ blokova. To je više od 4095 koliko je maksimalno moguće sa 12 bita da se predstavi. Stoga se ova particija ne može formatirati s FAT 12.

Da bi se smjestilo 4096 bloka u 40M, jedan blok mora biti velik najmanje 10K, tj. 20 sektora. Ukupno ima $40 \cdot 1024 \cdot 1024 / (10 \cdot 1024) = 4096$ blokova (to je za 1 više od limita, ali se blokovi u rezerviranom području ne moraju adresirati. Jedan sektor je *boot* blok, 512 bajta. Postoje 2 FAT tabele koje opisuju 4096 blokova, svaki element po 12 bita, dakle svaka je velika 6144 bajta. Korijenski direktorij je fiksni i zauzima $128 \cdot 32 = 4096$ bajta. Datoteke zauzimaju cijeli broj bloka, tako da datoteke od 100 bajta zauzimaju po 1 blok svaka, a datoteke od 20000 bajta po 2 bloka svaka. Stoga, datoteke zauzimaju $80 \cdot 10240 + 10 \cdot 20480$. Prema tome, potrošeno je $512 + 2 \cdot 6144 + 4096 + 80 \cdot 10240 + 10 \cdot 20480 = 1040896$ bajta.

b) FAT16: Sektor je 512 bajta, pa je blok velik 2K. Ukupno ima $40 \cdot 1024 \cdot 1024 / (2 \cdot 1024) = 20 \cdot 1024 = 20480$ blokova, što je manje od 65535, limit za FAT 16. Jedan sektor je *boot* sektor, tj. 512 bajta. Postoje 2 FAT tabele, a svaki element je po 16 bita, stoga svaka od njih zauzima po 40960 bajtova. Korijenski direktorij je fiksni i zauzima $128 \cdot 32 = 4096$ bajta. Datoteke zauzimaju cijeli broj blokova, tako da datoteke od 100 bajta zauzimaju po 1 blok svaka, a datoteke od 20000 bajta po 10 blokova svaka. Stoga, datoteke zauzimaju $80 \cdot 2048 + 10 \cdot 20480$.

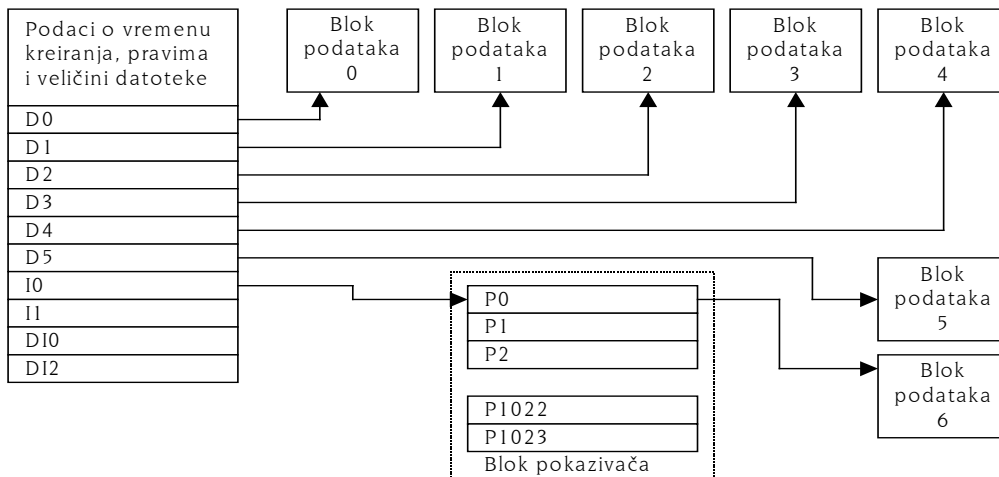
Stoga je potrošeno $512 + 2 \cdot 40960 + 4096 + 80 \cdot 2048 + 10 \cdot 20480 = 455168$ bajta

c) FAT 32: Sektor je 512 bajta, pa je blok velik 2K. Ukupno ima $40 \cdot 1024 \cdot 1024 / (2 \cdot 1024) = 20 \cdot 1024 = 20480$ blokova, što je manje od 268435445, limit za FAT32. Jedan sektor je *boot* sektor, tj. 512 bajta. Postoje 2 FAT tabele, a svaki element je po 32 bita, stoga svaka od njih zauzima po 81920 bajtova. Korijenski direktorij u FAT 32 nije fiksni, ali zauzima najmanje jedan blok, ili više zavisno od broja datoteka. Ukoliko datoteke nemaju duga imena, svaki element direktorija zauzima po 32 bajta, pa nam treba 2880 bajta, što je više od jednog bloka, stoga se korijenski direktorij smješta u dva bloka.

Prema tome, potrošeno je $512 + 2 \cdot 81920 + 4096 + 80 \cdot 2048 + 10 \cdot 20480 = 537088$ bajta

Zadatak 5. Dat je indeksni čvor veličine 64 bajta koji sadrži 6 direktnih pokazivača na blokove, 2 indirektna pokazivača i 2 duplo indirektna pokazivača. Svaki pokazivač ukazuje na blok od 4kbajta koji sadrži podatke ili listu 4-bajtnih pokazivača. a) Nacrtati shematski systemske i blokove podataka datoteke koja je velika 27K. b) Koliko memorije zauzimaju metapodaci za ovu datoteku?

Rješenje:a)



b) Metapodatke predstavljaju: indeksni čvor datoteke, 1 indirektni blok pokazivača, što ukupno čini $4K + 64 \text{ bajta} = 4160$ bajta

Zadatak 6. Ako je veličina bloka u datotečnom sistemu 2 kilobajta, a particija ima predviđenih 40000 blokova sa podacima, koliko prostora treba alocirati za a) bitmapu, b) ulančanu listu slobodnih blokova, c) evidentiranje slobodnog prostora kroz postojeći indeksni čvor sa slobodnim blokovima (10 direktnih, 1 indirektni, 1 dvostruko indirektni)

Rješenje

a) Za svaki blok nam treba po bit, pa se u 1 bajt može predstaviti bitmapa za 8 blokova. Za opis bitmape potrebno je $40000/8 = 5000$ bajtova, što znači da se bit mapa može smjestiti u 3 bloka, dakle $3 \cdot 2048 = 6144$ bajta.

b) Pošto je $40000 < 2^{16}$ za adresu bloka dovoljno je 2 bajta. Veličina varira od broja slobodnih blokova. U svakom bloku može se smjestiti $2048/2 = 1024$ pokazivača, od kojih je jedan na sljedeći blok pokazivača, a ostalih 1023 na slobodne blokove. Za potpuno praznu particiju pokazivače na slobodne blokove stavljamo u $\text{floor}(40000/1023) = 40$ blokova, što predstavlja 81920 bajta. Kod potpuno pune particije, zauzeće za listu je 0.

c) U svakom bloku može se smjestiti $2048/2 = 1024$ pokazivača. 10 od njih je u indeksnom čvoru, a preostalih 39990 u blokovima sa pokazivačima opisanih u $\text{floor}(39990/1024) = 40$ blokova. Jedan blok se može povezati na jednostruko indirektni pokazivač. Za lociranje ostalih 39 blokova **sa pokazivačima** treba dodati još jedan blok na koga pokazuje dvostruko indirektni pokazivač, što daje ukupno 41 blok, tj $41 \cdot 2048 = 83968$ bajtova.

Zadatak 7: Ovo je dio prikaza particije u heksadekadnom formatu.

```

00000000 EB 58 90 4D 53 44 4F 53 35 2E 30 00 02 08 7C 07
00000010 02 00 00 00 00 00 F8 00 00 3F 00 FF 00 38 00 00 00
00000020 C8 7F F1 00 42 3C 00 00 00 00 00 00 02 00 00 00
00000030 01 00 06 00 00 00 00 00 00 00 00 00 00 00 00 00
.....
0000EF800 F8 FF FF 0F FF FF FF FF FF FF 0F 04 00 00 00
0000EF810 05 00 00 00 06 00 00 00 07 00 00 00 08 00 00 00
.....
0001DEAB0 FF FF FF 0F FF FF FF 0F AF BC 03 00 B0 BC 03 00
0001DEAC0 B4 BC 03 00 B2 BC 03 00 B3 BC 03 00 FF FF FF 0F
0001DEAD0 B5 BC 03 00 B6 BC 03 00 B7 BC 03 00 B8 BC 03 00
.....
001000000 54 72 61 6E 73 63 65 6E 64 20 20 08 00 00 00 00
001000010 00 00 00 00 00 00 56 5A 7E 33 00 00 00 00 00 00
001000020 43 69 00 00 00 00 FF FF FF FF FF FF 0F 00 6D FF FF
.....
001000760 41 66 00 61 00 74 00 74 00 65 00 0F 00 A9 73 00
001000770 74 00 2E 00 74 00 78 00 74 00 00 00 00 00 FF FF
001000780 46 41 54 54 45 53 54 20 54 58 54 20 00 00 8C 5A
001000790 A9 48 25 4D 03 00 8C 5A A9 48 B1 BC B8 2F 00 00
.....
03CCAEFF0 92 9A 74 74 33 01 60 8B B8 A2 60 EB 1D 3F 85 74
03CCAF000 20 20 20 20 20 20 0A 20 0A 20 0A 20 56 69 6A 65
03CCAF010 73 74 69 20 0A 4E 61 6A 6E 6F 76 69 6A 65 0A 4C

```

Prateći strukture FAT sistema, doći do fizičkog sadržaja datoteke fattest.txt.

Rješenje: Prvo će se analizirati BIOS parametarski blok na početku particije

Poz.	Bajтови	Značenje
00	EB5890	JMP instrukcija
03	4D53444F53352E30	ASCII oznaka sadržaja 'MSDOS 5.0'
0B	0002	Broj bajta u sektoru: 200 bajta u sektoru
0D	08	Broj sektora u klasteru: 8
0E	7C07	Broj sektora u rezervisanom području: 77C
10	02	Broj FAT tabela: 2
11	0000	Datoteka u korijenskom direktoriju: FAT 32 ovo ignoriše
13	0000	Broj sektora u FAT16 ili FAT12: FAT 32 ovo ignoriše
15	F8	Izmjenjivost medija: Ponaša se kao <i>hard</i> disk
16	0000	Veličina tabela FAT16 ili FAT12: FAT 32 ovo ignoriše
18	3F00	Broj sektora u stazi: 3F fiktivna sektora
1A	FF00	Broj glava: FF fiktivnih glava
1C	38000000	Sektora prije početka particije: 00000038
20	C87FF100	Ukupno sektora u particiji: 00F17FC8 (oko 8 G)
24	423C0000	Veličina tabele FAT32: 00003C42 sektora
28	0000	Aktivna FAT tabela : 0, kopira se
2A	0000	Verzija FAT: 0
2C	02000000	Korijenski direktorij u klasteru 00000002

Pozicija korijenskog direktorija dobiva se po formuli:

KorijenskiDirektorij =
VeličinaSektora*(RezervisanoPodručje + BrojFATTabela
*VeličinaFATTabele) =

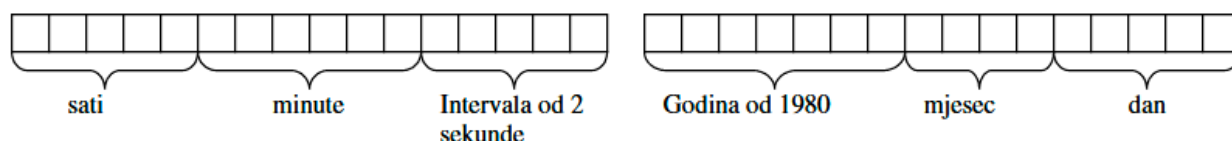
$$200*(77C+2*3C42)=01000000$$

Zaista, prvih nekoliko bajta od adrese 01000000, gledano kao ASCII kôd predstavljaju riječ Transcend (ime medija)

Sada treba naći one bajtove u korijenskom direktoriju vezanih za datoteku fat-test.txt. Heksadekadni ASCII kôd slova 'f' je 46, a slova 'a' je 41, pa zaključujemo da se element našeg direktorija nalazi na poziciji 001000780.

Rel	Bajтови	Značenje
00	4 6 4 1 5 4 5 4 4 5 5 3 5 4 2 0 5 4 5 8 5 4	Ime datoteke fattest.txt
0B	2 0	Atributi: Archive: 1, directory: 0, volume: 0, system: 0, hidden: 0, readonly: 0
0C	0 0	Rezervirano
0D	0 0	Vrijeme kreiranja desetinke: 0
0E	8 C 5 A	Vrijeme kreiranja h,m,s: 5A8C znači 11:20:24
10	A 9 4 8	Datum kreiranja: 48A9 znači 9.5.2016
12	2 5 4 D	Datum pristupa 4D25 znači 5.9.2018
14	0 3 0 0	Viša 2 bajta početnog klastera
16	8 C 5 A	Vrijeme pisanja: 5A8C znači 11:20:24
18	A 9 4 8	Datum pisanja: 48A9 znači 9.5.2016
1A	B 1 B C	Niža 2 bajta početnog klastera
1C	B 8 2 F 0 0 0 0	Veličina datoteke: 00002FB8

Datumi i vremena pakuju se u po 2 bajta na sljedeći način



Iz označenih vrijednosti, zaključuje se da je prvi klaster 0003BCB1. Njegova pozicija od početka particije računa se kao:

$$\text{FizičkaPozicija} = \text{VeličinaSektora} * (\text{PocetniKlaster} * \text{VeličinaKlastera} + \text{BrojFatTabela} * \text{VeličinaFATTabele} + \text{VeličinaRezervisanogPodručja} - 2 * \text{VeličinaKlastera}) =$$

$$= 200 * (0003BCB1 * 8 + 2 * 3C42 + 77C - 2 * 8) = 03CCAF000$$

Dakle na toj poziciji počinje sadržaj datoteke.

Sada trebamo naći nastavak FAT lanca. Prva FAT tabela počinje iza rezerviranog područja. Kako je u pitanju FAT 32 svaki element zauzima 4 bajta. Pozicija elementa FAT tabele dobiva se kao

$$\text{FATPozicijaKlastera} = \text{VeličinaSektora} * \text{VeličinaRezervisanogPodručja} + 4 * \text{Klaster} = 200 * 77C + 4 * 0003BCB1 = 001DEAC4$$

Na FAT poziciji klastera vide se 4 bajta B2 BC 03 00 iz čega se zaključuje da se lanac nastavlja na 0003BCB2. Na poziciji koja odgovara klasteru 0003BCB2 stoji da se lanac nastavlja na poziciji 0003BCB3. Na poziciji koja odgovara klasteru 0003BCB3 stoje bajtovi FF FF FF 0F što predstavlja kraj lanca.

14. SIGURNOST OPERATIVNOG SISTEMA

U poglavljima o memoriji i procesima govoreno je o zaštiti pojedinih memorijskih područja procesa i jezgra od drugih procesa. Sada će biti riječi o prijetnjama koje mogu negativno uticati na operativni sistem i podatke, te tehnikama borbe protiv njih.

14.1. Prijetnje

U datotečnim sistemima se čuvaju korektni podaci namijenjeni određenoj grupi korisnika, a zadatak procesa je da te podatke obrađuje. Moguće su različite prijetnje koje sprječavaju ovu osnovnu funkciju. Prijetnje mogu ugrožavati korektnost podataka, na primjer njihovim brisanjem ili ubacivanjem lažnih podataka. Može se ugroziti i povjerljivost, izlaganjem podataka ili programa onima kojima nisu namijenjeni. Također, može se ugroziti izvršavanje procesa, na primjer njegovim nasilnim prekidanjem ili forsiranjem njegovog pada uslijed preopterećenja.

Prijetnje možemo podijeliti u više kategorija: prirodne katastrofe, hardverske greške, ljudski faktor, softverske greške i zlonamjerni softver

14.2. Prirodne katastrofe i hardverske greške

Požari, zemljotresi, poplave, nestanak električne energije, kvarovi diskova, pregrijavanje napojnih jedinica, ili drugačije ponašanje elektroničkih komponenti uslijed starenja mogu nepredvidivo da ugroze korektnost podataka ili dostupnost računarskog sistema. Protiv ovih vrsta prijetnji ne može se dizajnirati zaštita samog operativnog sistema, ali ih treba uzeti u obzir kao sigurnosnu prijetnju i pripremiti odgovarajuće mjere. Na primjer redovan *backup* podataka na udaljene servere u Internet oblaku je efektivno čuvanje podataka u slučaju požara koji bi zahvatio cijelu zgradu sa serverima i arhivom traka.

14.3. Ljudski faktor

Ljudi su najveća prijetnja sigurnosti računarskog sistema, nenamjerno ili namjerno. Nenamjerne prijetnje su pogrešno otkucana komanda koja briše drugu datoteku umjesto one koja je trebala biti obrisana, neplanirano kopiranje na javni server privatnih podataka, ostavljanje na svom stolu ceduljice na kojoj piše lozinka, pogrešno unesen slog u bazu podataka i slično. Namjerne prijetnje obavljaju lica koja se nazivaju uljezima (engl. *intruders*). Oni žele da obavljaju aktivnosti za koje nisu ovlašteni, npr. čitaju sadržaje datoteka koje nisu za njih, onemogućavaju rad sistema, ili uništavaju podatke. Motivi za ovakve aktivnosti mogu biti radoznalost, zlonamjernost, koristoljublje, špijunaža, šale i slično.

14.4. Softverske greške

Programerske greške dovode po rušenja aplikacija, ali i do izlaganja podatka pa i rušenja operativnog sistema. Nema programa bez grešaka, ali dok se neke otkrivaju još u toku razvoja programa i utiču na funkcionalnost samo tog programa u kome je greška, neke greške mogu ugroziti sigurnost cijelog sistema. Najpoznatija takva greška je prekoračenje bafera. Ona se pored ovoga, može iskoristiti od strane napadača na sistem. U sljedećem primjeru:

```
char ime[8];  
int testirano;  
testirano=0;  
printf("Unesite podatak");  
scanf("%s", ime);
```

ako korisnik unese ime duže od 8 slova neće biti promijenjena samo varijabla ime, nego i varijabla testirano. Sličnim principom se mogu mijenjati i povratne adrese iz potprograma i tako forsirati izvršenje koda na inače nedostupnom mjestu. Da se ne ostave ovakve sigurnosne rupe, treba uvijek koristiti sistemske pozive koji provjeravaju dužine nizova, npr. *strncpy*.

14.5.Zlonamjerni programi

Zlonamjerni program predstavlja program, napisan da bi uzrokovao štetu i navođen je ciljano na sistem. Ima ih više vrsta

Virus predstavlja dio koda koji se može reprodukovati, pišući svoj kôd kao dodatak nekoj datoteci (što liči na sličan proces reprodukcije bioloških virusa). Pored vlastite reprodukcije, virusi mogu raditi i druge stvari. Na primjer, može štampati poruke, prikazati sliku na ekran, izvoditi muziku ili nešto ali i modifikovati datoteke, uništiti ih ili krasti datoteke slanjem elektronskom poštom.

Dok se virus širi lijepljenjem na drugi program i izvršava se kada se izvršava program sa kojim je virus povezan, **crv** predstavlja neovisan program. Crvi se šire korištenjem mreže za prijenos vlastitih kopija na druge računare, ali mogu da obavljaju kasnije slične operacija kao i virusi.

Trojanac je program koji izgleda kao normalni program (igra, korisnički program) ali izvršava i neke druge funkcije, možda se pokreće crv ili virus ili izvršava neku od zlonamjernih akcija. Efekti trojanskog konja treba da budu tihi i nečujni. Nasuprot crvima i virusima, trojance skidaju korisnici sa Interneta i čim se prepoznaju, biće izbrisani.

Logička bomba je tajna opcija nekog programa koja se pokreće pod određenim uslovima, na primjer knjigovodstveni programi koji prestaju da rade ako u bazi podataka ne nađu da je na žiro-račun autora uplaćen određeni iznos ili ako otkriju da je instaliran konkurentski program.

Spyware je softver koji šalje informacije o korisničkom računaru, na primjer koje web stranice se najčešće posjećuju sa tog računara. Ovakvi programi se najčešće instaliraju kao dodatak web preglednicima.

14.6.Principi zaštite računarskog sistema

Zaštita od prijetnji koje ugrožavaju sigurnost operativnog sistema, prvenstveno ljudskog faktora i malicioznog softvera, treba da bude zasnovana na javno poznatim algoritmima i pravilima. Zaštitni mehanizmi uključuju metode identifikacije korisnika, metode prava pristupa, metode izolacije između procesa i sistema, metode provjere sistema na sigurnost, metode ublažavanja posljedica, i metode šifrovanja povjerljivih podataka.

14.7.Metode identifikacije

Najčešći način identifikacije je zaštita **lozinkom**. Zaštita lozinkom je jednostavna za shvatiti, a i jednostavna za implementaciju. U većini sistema se

izvodi ovako: program za prijavu traži od korisnika da unese svoje ime i lozinku. Lozinka se odmah šifrira. Zatim program za prijavu čita datoteku lozinke u kojoj se nalazi niz linija, po jedna za korisnika, dok ne nađe liniju koja sadrži ime korisnika. Ako je šifrirana lozinka u datoteci ista kao prijavom unesena a zatim šifrirana lozinka, pristup je dozvoljen inače se odbija.

Da se ne bi lako otkrile od strane korisnika kome nisu namijenjene, treba izbjegavati lozinke koje su vezane za ime korisnika, poznate riječi u rječniku, fudbalske timove, gradove i slično. Preporučljivo je da je lozinka sastavljena od velikih i malih slova, specijalnih znakova i cifara, jer to povećava broj kombinacija koje treba probati ako se lozinka pogađa grubom silom.

Drugi način identifikacije su **kartice** sa magnetnom trakom na njoj. Kartica se ubaci u terminal koji onda kontroliše čija je kartica. Ova metoda se kombinuje sa lozinkom tako da se korisnik može prijaviti samo ako ima karticu i zna lozinku. Automatske mašine za izdavanja novca, obično, rade na ovom principu.

Na računarima sa dodirnim ekranom koristi se identifikacija **pokretima**. Na primjer, za otključavanje nekih Android mobitela korisnik prstom treba da spoji niz tačaka praveći izlomljenu liniju oblika koji samo on zna.

Biometrijske metode su identifikacija otiskom prsta, bojom glasa izgledom mrežnjače oka ili mjerenjem dužine prsta. Prednost ove identifikacije je u većoj sigurnosti od saznavanja lozinke, ali može predstavljati problem npr. u situacijama kada je korisnik u bolnici i hitno mu je potreban novac da plati operaciju slomljene noge, ali ne može pristupiti bankomatu koji obavlja identifikaciju otiskom prsta.

14.8. Prava pristupa

Resursi kojima upravlja operativni sistem treba da budu zaštićeni. Ovi resursi mogu da budu hardverski (tj. procesori, memorijski segmenti, disk uređaji, ili štampači), ili mogu da budu i softverski (tj. procesi, datoteke, baze podataka, ili semafori). Svaki resurs ima jedinstveno ime preko koga mu se pristupa i konačan skup operacija kojima im proces može pristupiti. Operacije *read* i *write* su odgovarajuće operacije za datoteke, operacije *wait()* i *signal()* su za semafore.

Korisnici pristupaju ovim resursima sa različitim pravima. Korisnik Asim možda ima pravo čitanja i pisanja određene datoteke, dok korisnik Belma tu istu datoteku smije samo čitati. Sve kombinacije prava korisnika nad resursima u sistemu opisuju se velikom **matricom kontrole pristupa**, (*access control matrix*), skraćeno ACM. Redovi ove matrice su korisnici, a kolone resursi kojima se upravlja. Svaki član ove matrice predstavlja konkretna prava koja navedeni korisnik ima nad određenim resursom. Slika 177 prikazuje jednu matricu kontrole pristupa.

	Dat1	Dat2	Dat3	Dat4	Dat5	Dat6	Dat7	Dat8	Dat9	Port1	Port2
Tom	R	RW	RW								
Alisa			R	RX	R			R	RX	R	
Stefan						RX	RW			R	W
Hajro								R	RX		

Slika 177 Matrica kontrole pristupa

U ovoj matrici se vidi da Stefan ima pravo čitanja i izvršavanja datoteke Dat6, čitanja i pisanja datoteke Dat7, čitanja sa porta Port1 i pisanja na Port2. Kada Stefan pokrene proces, taj proces dobiva navedena prava nad spomenutim resursima. U

praksi, smještanje matrice prava pristupa predstavlja smještanje rijetke matrice jer je velika i nepopunjena. Većina korisnika nema pristupa većini objekata, tako da smještanje vrlo velike, gotovo prazne matrice predstavlja gubitak smještajnog prostora. Dvije praktične tehnike se koriste za smještanje matrice.

Prva tehnika se sastoji od pridruživanja svakom resursu jedne uređene liste koja sadrži sve korisnike koji mogu pristupiti objektu i kako. Ova lista se naziva **listom kontrole pristupa** ili ACL. U primjeru matrice sa slike Slika 177 lista kontrole pristupa za datoteku Dat3 bi izgledala ovako.

Tom: RW ; Alisa: R

To znači da svaki proces koga je kreirao Tom može čitati i pisati ovu datoteku, procesi koje je kreirala Alisa čitati je, dok procesi koje su kreirali ostali procesi ne mogu je ni otvoriti. Datoteke Dat8 i Dat9 imaju identične kolone u matrici kontrole pristupa, pa mogu dijeliti listu kontrole pristupa.

Mnogi sistemi podržavaju koncept grupe korisnika. Grupe korisnika imaju imena i mogu se uključiti u ACL. Ovaj pristup znatno smanjuje veličinu listi kontrola pristupa. Na primjer ako Duje i Fikret imaju pravo čitanja i pisanja nad datotekom, a svi članovi grupe grafika imaju pristup za čitanje i izvršavanje, lista kontrole pristupa će biti kako slijedi.

duje: RW; fikret: RW; grafika: RX

U slučaju da liste kontrola pristupa mogu sadržati grupe, postavlja se pitanje šta ako je korisnik član više grupa navedenih u listi kontrole pristupa za dati resurs, ili je naveden i kao pojedinačni korisnik i kao član neke grupe? Zavisno od dizajna operativnog sistema primjenjuju se dva pristupa. Jedan je da čim se korisnik pronade na neki način u listi kontrole pristupa da se prekida traženje u toj listi i korisniku dodijele ta do sada nađena prava. U ovom primjeru, ako je Fikret ujedno i član grupe grafika, on dobiva pravo čitanja i pisanja. Drugi pristup je da se kumuliraju sva prava, pa bi tada Fikret dobio prava RW kao korisnik i RX kao član grupe grafika, što kumulativno rezultuje pravima RWX.

Nekada se desi da korisnik ili grupa ima neke dozvole pristupa s obzirom na datoteku koje vlasnik datoteke želi da poništi. Sa listom kontrole pristupa, relativno je jasno da se ponište prethodno dodijeljena prava. Sve što treba uraditi je da se naprave izmjene u ACL. Međutim, ako se lista kontrolira samo kod otvaranja, promjene će imati efekta tek kasnije kod ponovnog otvaranja, čak i u slučajevima kada korisnik više nije ovlašten da pristupi datoteci.

Drugi pristup pogleda na matricu kontrole pristupa je preko redova. Kada se ova metoda koristi, svakom procesu se pridružuje lista resursa kojima se može pristupiti zajedno sa indikacijom koje operacije su dozvoljene na svakom objektu. Ova lista se naziva **listom mogućnosti** (engl. *capabilities*) ili C- lista a pojedini elementi u listi se nazivaju mogućnostima. U primjeru matrice sa slike Slika 177 lista mogućnosti za svaki proces koga je kreirala Alisa izgleda ovako.

Dat3:R; Dat4:RX; Dat5:R; Dat8:R; Dat9:RX; Port1:R

Kada Alisin proces pristupi datoteci Dat5, provjeri se da li je ta datoteka u listi mogućnosti za taj proces (u ovom primjeru samo za čitanje). Liste mogućnosti moraju biti zaštićene od neovlaštenog pristupa korisnika pa se čuvaju u jezgru ili kriptografisane korisničkom prostoru.

14.9. Metode izolacije

Zaštita od prijetnji računarskom sistemu kao narednu komponentu treba da uključi izolaciju od prijetnji. Pošto prijetnje uključuju široki spektar od ljudskog faktora do zlonamjernog koda mora se djelovati na mnogo širem planu od operativnog sistema pojedinih računara.

Izolacija od **fizičkog pristupa** treba biti uključena u projektovanje računarskih mreža, barem za računare koji sadrže najvažnije podatke. Sve metode zaštite padaju ako uljez može slobodno doći do servera, odnijeti njegov disk ili podići alternativni operativni sistem sa CD-a zaobilazeći tako proces prijave. Ili iskoristiti trenutak kada je operater izašao na trenutak u WC bez odjavljivanja sa sistema da ubaci aplikaciju koja će simulirati ponovnu prijavu, a u stvari krađe njegovu lozinku. Ili oslušivati laptopom mrežni promet u nekriptografisanoj bežičnoj mreži pored zgrade ili na router-u koji se nalazi u Norveškoj, a povezuje evropsku i američku filijalu firme, kako bi se ulovile informacije koje su korisne berzanskim špekulantima. Mjere ove izolacije uključuju postupke kao što su čuvanje servera u posebnim zaključanim prostorijama, angažovanje osoblja za fizičku i informacionu sigurnost, video nadzor, zabranu otvorenih bežičnih mreža, stalnu edukaciju o potrebi sigurnosti itd.

Izolacija između procesa je skup različitih hardverskih i softverskih tehnika koje štite svaki proces od drugih procesa u operativnom sistemu. Ona sprječava da proces P1 upisuje ili čita podatke koji pripadaju procesu P2. Procesna izolacija se postiže virtualnim adresnim prostorom, gdje je adresni prostor procesa P1 različit od adresnog prostora procesa P2. Postojanje izolacije procesa smanjuje mogućnosti djelovanja zlonamjernog softvera i zlonamjernih korisnika. O ovim tehnikama je već bilo riječi u ovoj knjizi.

Tehnike pješćanika (*sandboxing*) su načini izolacije nedovoljno pouzdanog softvera od ostatka sistema. One se koriste za izvršavanje netestiranog softvera ili softvera iz nepovjerljivih izvora. Pješćanik pruža kontrolirani skup resursa programima koji se u njemu izvršavaju, na primjer odvojeni direktorij ili memorijsko područje, bez mogućnosti čitanja sa mreže. Oni često rade na principu virtualnog operativnog sistema koji simulira unutar posebne datoteke ponašanje računarskog sistema. Ako testirani softver sadrži virus, on se neće moći širiti dalje od područja pješćanika i zaraziti samo to područje. Druga tehnika pješćanika je pokretanje procesa sa pravima koja dopuštaju minimalan broj sistemskih poziva. Tako u Linux-u postoji poziv *seccomp()* kojim aplikacija prelazi u režim u kome je procesima dopušteno jedino pozivanje poziva *write()*, *read()*, *exit()* i *sigreturn()* nad otvorenim datotečnim deskriptorom.

Certifikacija je restrikcija softvera na onaj koji je neki autoritet proglasio sigurnim. Android sistem koristi centralni server za instalaciju softvera za mobilne uređaje, tako da su nove aplikacije koje se instaliraju prethodno pregledane i digitalno potpisane. Pri njihovoj instalaciji, korisnik se upozori na to koje će sve resurse aplikacija koristiti (npr. bežičnu mrežu i telefonske impulse). Windows XP i noviji koristi sertifikaciju za drajvere, kao najslabiju kariku u sigurnosti sistema: pri instalaciji drajvera administrator se upozorava ako pojedini drajver nije prošao test kontrole i nema certifikacijsku oznaku. Linux od verzije 2.6 ne dopušta instalaciju *kernel* modula koji nisu kompilirani zajedno sa ostatkom jezgra.

Izolacija izvršnog okruženja je preporuka da se umjesto programa direktno prevedenih u mašinski jezik i sistemskih poziva koriste interpretirani jezici poput Unix *shell* skripti, programa u Web jezicima kao što je JavaScript ili jezici sa izvršnim virtualnim mašinama poput .NET jezika (C#, C++/CLI, VisualBasic) ili Java jezika. Kompajleri za .NET jezike ne generišu direktno izvršni kôd, nego program u međuformatu koji se zove MSIL, dok kompajleri za Java generišu program u formatu za zamišljeni procesor koji se zove JVM. U trenutku pokretanja ovih programa oni se interno prevode u mašinski jezik procesora na kome se izvršavaju. Ovaj pristup povećava sigurnost, jer se pri tome mogu uključiti provjere prekoračenja bafera, kontrolisati da li je dio memorije alocirana a neoslobođen ili aktivirati okruženje pješčanika u kome se npr. zabranjuje pisanje na disk ili dodatno limitira pravo pisanja unutar adresnog prostora procesa (npr. da nije moguće mijenjati povratne adrese potprograma na steku).

Izolacija mrežnog pristupa postiže se posebnim softverom koji se zove **firewall**. Na svakom računaru u mreži, a pogotovo na onom koji predstavlja vezu mreže prema Internetu postavlja se program koji kontroliše mrežni saobraćaj i sprječava da sumnjivi mrežni paketi dođu do odredišnog računara. Najjednostavniji princip *firewall*-a je filtriranje paketa. Kod ovog pristupa se posmatraju izvorišne i odredišne IP adrese mrežnih paketa kao i njihova namjena (mrežni protokol). Ako mrežni paket dolazi sa lokacije koja nije dopuštena, on se odbacuje. Slično tome, odbacuju se paketi koji koriste neki od mrežnih protokola na računarima na kojima nisu predviđeni (npr. pokušaj udaljene prijave). Odbacuju se i pokušaji pristupa prema vani ako mrežni administrator ima informacije da je riječ o posjeti Internet stranicama koje su poznate po širenju virusa ili negativno utiču na produktivnost rada. Novije generacije *firewall*-a, u odlučivanju koje pakete treba propuštati, a koje ne treba, analiziraju ko je uspostavio komunikaciju. Na primjer, ako je korisnik iz lokalne mreže otvorio web stranicu na udaljenom serveru i web server poslao odgovor, firewall neće blokirati paket sa odgovorom, ali će blokirati sve pakete koji su došli spolja, a da nisu inicirani od lokalnog korisnika. Dodatne aktivnosti koje *firewall* programi uključuju su kontrola sadržaja samog mrežnog paketa da se uoče i odbace sumnjivi zahtjevi na legitimnim adresama i protokolima. *Firewall* treba odbacivati i pokušaje brzog slanja velikog broja mrežnih paketa jer oni mogu narušiti odziv sistema.

14.10. Metode provjere

I pored mjera izolacije, zlonamjerni softver ili ljudski uljez nalazi svoj put do operativnog sistema računara. Sada sistem sigurnosti treba da prepozna da je takva prijetnja postala ozbiljna i da preduzme odgovarajuću akciju.

Antivirus je program koji prepoznaje i uništava zlonamjerni kôd. Kada se pojavi nova datoteka u sistemu, (a povremeno se pregleda i cijeli datotečni sistem), antivirusni program analizira tu datoteku (ili sve datoteke) i testira da li je riječ o zlonamjernom softveru. Analiziraju se i ostala područja datotečnog sistema poput *boot* sektora, particione tabele i direktorijske strukture, jer se i u njih može sakriti zlonamjerni softver. Antivirusi rade na dva osnovna pristupa: preko rječnika i preko sumnjivog ponašanja pri čemu se ovi principi često kombinuju. Antivirusi na bazi rječnika traže karakteristični kôd koji imaju poznati virusi, crvi i trojanci. Na primjer,

virus Jerusalem za MS DOS prepoznaje se po svojih karakterističnih 1802 bajta koje dodaje na kraj svakog pokrenutog izvršnog programa. Dio tog karakterističnog koda je signatura virusa. Signatura treba biti veličine 50-300 bajta da se legitimne datoteke ne bi prepoznale kao virusi jer se slučajno u njima nalazi baš ta sekvenca koda, a da s druge strane pretraživanje ne traje predugo. Datoteke se porede sa listom signatura koja se ažurira svakodnevno. Ako se u datoteci pronađe niz bajtova identičan signaturi, to je velika vjerovatnoća da se u njoj nalazi zlonamjerni softver. Pristup preko signatura ne štiti od novih virusa čije signature još nisu evidentirane u listi signatura, a često ga varaju polimorfni virusi, oni koji sebe stalno mijenjaju kako ne bi imali stalnu signaturu. Antivirusi koji prate sumnjivo ponašanje prate pokušaje aktivnosti koje rade virusi i crvi kao što su pokušaji mijenjanja izvršnih programa, mijenjanja *boot* sektora, ubacivanja programa u direktorije namijenjene programima koji se startuju automatski prilikom prijave korisnika, instalacije dodataka u web browsere, kopiranja izvršivih programa kroz mrežu itd. Ovaj metod pomaže i protiv novih virusa, ali često zna pokrenuti lažnu uzbunu, pa se od korisnika traži da potvrdi da li je on zaista želio tu akciju koju je antivirus spriječio.

Praćenje aktivnosti i događaja u sistemu realizovano je kroz dnevnike (engl. log). Jezgro i drugi dijelovi operativnog sistema mogu se konfigurisati da bilježe na posebno mjesto događaje koji mogu poslužiti sistemskom administratoru kao izvor bitnih informacija. Takvo mjesto se u Windows sistemima zove *Event log*, a u Linux sistemima se upisuje u datoteke unutar direktorija */var/log*. Administrator povremeno gleda događaje kao što su neuspjele prijave u sistem, ili informacije o zadnjem ažuriranju liste virusnih signatura. Na osnovu ovih informacija zaključuje da li to neko pokušava provaliti probom lozinke i da li je prijetnja od novog virusa ozbiljna.

Testovi narušavanja sigurnosti se obavljaju u toku i nakon razvoja operativnog sistema u fazi održavanja. Specijalisti pokušavaju neke akcije koje narušavaju sigurnost, a analizira se i izvorni kôd operativnog sistema. Ako se uoči propust poput zaboravljenog korisničkog imena koje je kreirano u eksperimentalne svrhe i ostalo u finalnoj verziji sistema omogućivši prijavu bez lozinke ili greške poput pada mrežnog podsistema ako je poslan paket dužine 60000 bajtova, pripremaju se sigurnosne zakrpe.

Sigurnosne zakrpe su nove verzije sistemskih datoteka koje se povremeno distribuiraju i zamjenjuju one ranije koje su imale sigurnosne propuste. U doba kada su računarske mreže bile spore ili nepostojeće distribuirale su se na magnetnim ili optičkim medijima relativno rijetko (jednom ili dva puta godišnje), porastom korištenja mreža distribuirale su se preko mreže na zahtjev administratora sistema, a danas se obavljaju automatski često i više puta sedmično.

14.11. Kriptografija

Sljedeća barijera protiv sigurnosnih prijetnji je šifrovanje podataka. Koristi se za čuvanje nekih podataka na datotečnim sistemima u nečitljivom obliku od uljeza ali i korisnika koji imaju legitiman pristup. Pogotovo je potrebno u okruženjima koja se ne mogu štiti izolacijom i pravima pristupa, kao što su komunikacije bežičnom mrežom i Internet komunikacije. **Šifrovanje** izvornog teksta se obavlja pomoću određenog algoritma. Postoji više različitih algoritama za šifrovanje koji kao ulazne

podatke imaju izvorni tekst i ključ, koji osigurava tajnost podataka, dok kao izlazni podatak daju šifrovani tekst. Inverzni postupak od šifrovanja, kojim se od šifrovanog teksta dobije originalni izvorni tekst nazivamo **dešifrovanje**.

Kriptografske algoritme možemo podijeliti u sljedeće grupe:

- **Tajni algoritmi**: sigurnost se zasniva na tajnosti algoritma.
- Algoritmi zasnovani na **ključu**: algoritam je javno poznat, a ključ se čuva u tajnosti. Dije se na tri osnovna tipa
 - **Simetrični** - koristi se jedinstven ključ i za šifrovanje i za dešifrovanje.
 - **Asimetrični** - postoje dva ključa (ključ za dešifrovanje se razlikuje od ključa kojim je sadržaj šifrovan).
 - Hibridni - kombinacija su prethodna dva.
- Algoritmi za **jednosmjerno** šifrovanje preko *hash* funkcija: šifrovani podaci se ne mogu vratiti u čitljiv oblik

Tajni algoritmi nisu preporučljivi u računarskim sistemima, pošto postaju neupotrebljivi kada neko sazna njihov princip rada. Poznat je primjer iz II svjetskog rata kada je za komunikaciju korištena "Knjiga o Rebeki". Kada je trebalo poslati poruku generalu Romellu, riječi u rečenicama navođene preko strane, reda i pozicije u redu gdje se navedena riječ nalazi u knjizi. Engleski obavještajci su uočili sumnjivim što je u Portugalu kupljeno više primjeraka ove knjige i našli je kod njemačkog špijuna. Nijemci su morali napustiti ovu šifru.

Osnovna osobina **simetričnih kriptosistema sa tajnim ključem** je da se za šifrovanje i dešifrovanje poruka koristi isti ključ. Pošiljalac ima za cilj slanje poruke M primaocu preko nezaštićenog komunikacionog kanala. Pošiljalac A najprije generiše poruku M (izvorni tekst) koja se prosljeđuje algoritmu za šifrovanje. Tu se vrši šifrovanje poruke M uz korištenje ključa K dobivenog uz pomoć generatora ključa. Na taj način se kreira šifrovana poruka C . Potom se tako dobivena poruka komunikacionim kanalom šalje do primaoca. Postupak dešifrovanja se obavlja inverznim postupkom od šifrovanja pomoću istog ključa K koji je korišten prilikom šifrovanja. Nakon dešifrovanja se dobiva izvorna poruka M . Ukoliko na kanalu za prijenos postoji osoba N (napadač) koji može da presretne šifrovanu poruku i ukoliko dođe u posjed ključa može pročitati ili zloupotrijebiti izvornu poruku. Stoga obje strane moraju držati ključ u tajnosti, odnosno ključ se ne smije prenositi nezaštićenim komunikacionim kanalom, dok sama poruka smije.

Jedan od najjednostavnijih algoritama sa simetričnim tajnim ključem je XOR šifriranje. ASCII otvoreni tekst se šifrira obavljajući operaciju XOR sa ključem koji predstavlja više puta ponovljenu tajnu riječ. Primjer je dat u sljedećem programu.

```
#include <stdio.h>
#include <string.h>
void encryptDecrypt(char *input, char *output, char *key) {
    int i,n;
    n=strlen(key);
    for(i = 0; i < strlen(input); i++) {
        output[i] = input[i] ^ key[i % n];
    }
}
int main (int argc, char *argv[]) {
    char encrypted[1000], decrypted[1000] ;
    encryptDecrypt("Nemoj da se ovo zna", encrypted, "TEORIJA");
    printf("Sifrovano:%s\n", encrypted);
    encryptDecrypt(encrypted, decrypted, "TEORIJA");
    printf("Desifrovano:%s\n", decrypted);
}
```

Algoritam je jednostavan i brz, ali se smije koristiti samo za jednokratno šifrovanje sa slučajnim ključem jer će dešifrovanje teksta navodeći kao ključ očekivanu prvu riječ u tekstu (a puno pisama počinje riječju "Poštovani" dok binarne datoteke imaju standardne signature na početku) će kao rezultat otkriti ključ kojim se dalje mogu dešifrovati podaci. Za profesionalnu primjenu razvijeni su drugi algoritmi simetričnih kriptosistema kao što su DES, 3DES, DES-CBC, IDEA, RC5, RC6, AES i drugi.

Nedostatak simetričnih kriptosistema je što se ključ ne smije distribuirati nesigurnim kanalom. Razlika između simetričnih i **asimetričnih algoritama** je u tome što simetrični algoritmi koriste isti ključ za šifrovanje i dešifrovanje dok asimetrični algoritmi koriste jedan ključ za šifrovanje a drugi za dešifrovanje. Jedan od ključeva je tajni a drugi javni. Oba ključa su povezana pomoću jedinstvene jednosmjerne funkcije. Ne smije se izračunati tajni ključ iz javnog ključa u razumnom vremenu. Funkcija šifrovanja mora biti takva da se iz šifrovanog teksta ne može odrediti izvorni tekst, čak ako je poznat i ključ kojim su podaci šifrovani. Međutim, ukoliko se zna ključ za dešifrovanje Kd moguće je lako računanje izvornog teksta. Scenarij asimetričnog šifrovanja bi izgledao ovako: Pošiljalac šifrira poruku radi slanja primaocu upotrebom javnog ključa primaoca koji je svima dostupan (čak i uljezu). Međutim, uljez i pored toga što poznaje javni ključ ne može otkriti sadržaj poruke. Poruku može dešifrovati samo primalac korištenjem svog tajnog ključa.

Najpoznatiji asimetrični algoritam šifrovanja je **RSA**. Kod ovog algoritma, za generiranje javnog i tajnog ključa se koriste prosti brojevi. Tajni ključ predstavlja uređeni par brojeva (N,d). Javni ključ je također uređeni par brojeva (N,e). Treba uočiti da je broj N zajednički za oba ključa. Osoba koja šalje poruku vrši šifrovanje pomoću sljedeće formule

$$C = P^e \bmod N$$

gde su: P, izvorni tekst koji je prikazan u obliku broja; C broj koji predstavlja kriptovan tekst; brojevi e i N su komponente javnog ključa.

Kada se poruka primi potrebno je dešifrovati pomoću formule

$$P = C^d \bmod N$$

Brojevi N , e i d zadovoljavaju sljedeće uslove

1. Neka postoje dva proizvoljno odabrana prosta broja p i q .
2. $N = p \cdot q$
3. $e < N$
4. Brojevi e i $(p-1)(q-1)$ su uzajamno prosti
5. $(e \cdot d) \bmod [(p-1)(q-1)] = 1$
6. N je vrlo veliki broj (više stotina cifara)

Ako se zna javni ključ (brojevi N i e), tajni ključ se može izračunati ako se imaju p i q . Ali navedeni uslovi za brojeve N , e i d postižu da se p i q mogu saznati samo ako se broj N rastavi na proste faktore. Pošto je N veliki broj rastavljanje na faktore traje dugo, zavisno od snage računara za razbijanje šifre i veličine broja N , danima ali i milionima godina.

Jedna sigurnosna rupa sa ovako opisanom primjenom asimetričnog kriptografskog algoritma postoji. Uljez može slati podatke primaocu, bez znanja primaoca ko je zaista poslao poruku. Međutim, kada se podaci dva puta šifruju: pošiljaočevim tajnim ključem, pa zatim primaočevim javnim, a na odredištu dešifruju najprije primaočevim tajnim ključem a zatim pošiljaočevim javnim ključem, postiže se dokaz o autentičnosti. Najveća mana RSA algoritma je sporost, pošto zahtijeva manipulacije sa vrlo velikim brojevima.

Ponekad se ne mora saznati originalna poruka. Primjer je smještanje lozinki za prijavu na sistem u datoteku sa lozinkama. Kada se definiše lozinka, izračuna se njena *hash* vrijednost nekom **kriptografskom hash funkcijom**. Takvim funkcijama je nemoguće u razumnom vremenu naći inverznu funkciju, a nisu ni injektivne: više lozinki mogu imati istu *hash* vrijednost. Umjesto same lozinke, u datoteku lozinki se stavi *hash* vrijednost lozinke. Kada se korisnik prijavi na sistem na otkucanu lozinku se primijeni ista *hash* funkcija i usporedi sa sačuvanom vrijednošću. Primjeri algoritama koji implementiraju ovakve funkcije su SHA-3, MD5, Whirpool. Ove funkcije se koriste i za potvrdu integriteta podataka. Ako postoji rizik da će neko mijenjati datoteku u toku slanja, može se izračunati vrijednost ove funkcije nad cijelom porukom koja se šalje i poslati je zajedno sa porukom. Na odredištu se ponovi *hash* funkcija nad primljenom porukom. Ove funkcije su takve da mala promjena poruke rezultuje drastično različitom vrijednošću *hash* funkcije. Ako su poslana i primljena *hash* vrijednost jednake podaci nisu mijenjani.

14.12.Umanjenje štete

Neke sigurnosne rizike (npr. požar ili sistemskog administratora koga je uljez podmitio da mu da veća prava) nismo izbjegli i desila se šteta. U tom slučaju mora se obaviti obnova, npr. zamjena pokvarenih dijelova računara, kupovina novih servera, čišćenje sistema od virusa, sudske tužbe, otpuštanje odgovornih s posla, zavisno od vrste problema. Da bi šteta koju je narušavanje sigurnosti sistema bila što manja, prije nastanka štete potrebne su sljedeće mjere.

- Redovan *backup* sistema i podataka koji se čuva na traci i prebacuje izvan zgrade u regularnim intervalima, ili kopira na udaljene lokacije mrežom;
- Upotreba sistema kao što je RAID za povećanje tolerancije na kvarove;
- Zaliha rezervnih dijelova;

- Uređaji za zaštitu napajanja, uređaji za neprekidno napajanje i generatori;
- Alarmi za požar i automatski uređaji za gašenje;
- Antivirusni softver i ostale sigurnosne mjere;

14.13. Sigurnost u Unix sistemima

Sigurnosni model u Unix sistemima je usko vezan s pravilom da su gotovo svi objekti datoteke u jednom virtualnom korijenskom direktoriju. Stoga se pravila koja se odnose na datoteke mogu primijeniti i na direktorije, mrežne *socket*-e, znakovne uređaje, blok uređaje, cijevi, a u nekim sistemima i na procese i parametre jezgra. Svi ovi objekti se identifikuju imenom koje može imati stazu sa poddirektorijima, npr. */usr/bin/wget* ili */dev/lp0*. Dok jedno ime predstavlja samo jedan objekt, obrnuto ne važi. Objekt može imati istovremeno više imena, što je postignuto linkovima u datotečnom sistemu kroz *ln* komandu.

Za razliku od obavezne kontrole pristupa (MAC) kod koje prava nad objektima određuje isključivo sistemski administrator, Unix koristi diskrecijsku kontrolu pristupa (DAC). Kod ove kontrole pristupa svaki objekt (npr. datoteka) ima vlasnika, koji odlučuje kakva prava nad objektom mogu da imaju korisnici. Vlasnik je također jedan od korisnika. Pri pokretanju procesa, procesi pripadaju onom korisniku koji ga je pokrenuo. Stoga, kada proces pristupi nekom objektu, proces ima iste privilegije nad tim objektom kao njegov kreator.

Korisnici su definirani numeričkim korisničkim identifikatorom (UID). Taj identifikator je obično šesnaestbitni ili tridesetdvo-bitni broj. Posebni korisnik se zove *root*. Njegov UID je 0 i kada on pokreće procese, kontrola pristupa se zaobilazi. Ovaj korisnik ima posebna prava u cijelom sistemu. On može promijeniti vlasništvo nad datotekama, konfigurirati uređaje, dijelove mreže, instalirati i mijenjati softver, pristupati svim datotekama. Može kreirati i brisati sve datoteke, korisnike i grupe korisnika, i mijenjati vlasništvo nad trenutno izvršavanim procesom.

Unix je odavno zamišljen kao višekorisnički sistem, i upotrebu *root* korisnika treba svesti na najmanju moguću mjeru. Ovaj korisnik je česta meta napadača sistema. Napadači traže način kako obični korisnici mogu da dobiju privilegije koje ima *root*, kroz sigurnosne rupe. Problem je što običnim korisnicima ponekad zatreba *root* pravo. Na primjer, na Unix sistemima sadržaj diskete treba montirati u datotečni sistem, a to je isključivo administratorsko pravo. Ili, korisnik želi da promijeni vlastitu lozinku za prijavu na sistem, što također nije moguće, jer prava čitanja i pisanja nad datotekom lozinki ima samo *root*. Stoga postoji nekoliko sistemskih alata da korisnik privremeno promijeni svoja prava. Oni se moraju koristiti s velikom pažnjom. Jedan takav alat su *setuid* i *setgid* programi. Normalno proces nasljeđuje prava i UID korisnika koji ga je pokrenuo, ali uz pomoć *setuid* programa se može navesti da pokretanje datog procesa nasljeđuje prava i UID koji imaju vlasnik datoteke s izvršnim programom procesa.

Drugi alat je naredba *su*. Korištenjem ove naredbe korisnik može da promijeni svoj identitet i UID u drugog korisnika (podrazumijevano *root*) broj navodeći kao parametar lozinku tog drugog korisnika. Da bi se izbjeglo širenje *root* lozinke, postoji i treći alat *sudo*. Za pristup administratorskim pravima korisnik navodi kroz ovaj alat

svoju lozinku, ali postoji datoteka koja dalje govori koji korisnici mogu dobiti puna prava i za izvršenje kojih naredbi.

Unix bazirani sistemi poznaju tri vrste prava i tri vrste korisnika. Stoga se prava pristupa definišu sa devet bita ili 12 bita. Da se za svaku datoteku ne bi evidentirala prava pristupa za svakog korisnika pojedinačno, korisno je sve korisnike razvrstati u klase i za svaku od njih vezati spomenuta prava pristupa. Unix sistemi poznaje tri klase korisnika. Jednoj pripada vlasnik datoteke, drugoj članovi grupe korisnika vezanih uz datoteku, a trećoj ostali korisnici. Za svaku vrstu korisnika se navode po tri bita, za pravo čitanja, pravo pisanja i pravo izvršavanja. Neke kombinacije prava date su na slici Slika 178.

Binarno	Oktalno	Simbolički	Prava s pristupom datoteka
111000000	700	rwX-----	Samo vlasnik može čitati pisati i izvršavati
110110110	666	rw-rw-rw-	Svi mogu čitati i pisati
000000000	000	-----	Niko nema pristupa
110100100	644	rw-r--r--	Vlasnik može čitati i pisati, ostali samo čitati
100100000	440	r--r-----	Vlasnik i grupa imaju pravo čitanja

Slika 178 Neka prava nad Unix datotekama

Da bi se pročitala ili postavila prava nad datotekama i srodnim objektima, postoje određene funkcije za očitavanje i postavljanje prava. Parametri ovih funkcija su specijalni brojevi UID koji je identifikator korisnika i GID, identifikator grupe.

Funkcija	Namjena
chmod(path,mode)	promjena režima datoteke
access(path,mode)	Provjera pristupa koristeći UID i GID
getuid()	vraća stvarni UID
getgid()	vraća stvarni GID
chown(path,owner,group)	mijenja vlasnika i grupu
setuid(uid)	postavlja UID
setgid(gid)	postavlja GID

Nedostatak Unix mehanizma pristupa je u nemogućnosti definiranja složenijih pravila. Recimo da nekoj datoteci koja je rezultat kompajliranja programa treba dodijeliti ACL element tako da članovi grupe programeri mogu da je mijenjaju, članovi grupe *backup* operatora da je čitaju ali ne mijenjaju, a tester i da je izvršavaju. Osnovni mehanizam prava poznaje samo jednu grupu pridruženu datoteci, pa ovo nije moguće. Da se ovaj nedostatak prevaziđe, neki Unix sistemi imaju *setfacl* komandu kojom se mogu praviti fleksibilnije liste kontrole pristupa, navodeći pojedina prava nad datotekom za različite grupe ili korisnike.

14.14. Sigurnost u Windows sistemima

Malo koji podsistem Windows-a kroz njegove razvojne generacije se toliko promijenio kao sigurnosni podsistem. Generacije Windows 1-3 i Windows 9x su kao podlogu koristile MS DOS, operativni sistem dizajniran za samo jednog korisnika, tako da nije bilo nikakve kontrole pristupa datotekama, izuzev kada se koriste lokalne mreže. S druge strane, serija Windows NT je uvela izuzetno sofisticiran sistem listi pristupa, *token*-a, domena korisnika i aktivnog direktorija (od Windows 2000), koji pruža puno veće mogućnosti u odnosu na Unix sistem. Nažalost, u međuvremenu su nastale aplikacije za starije verzije Windows-a koje su zahtijevale

često instaliranje i međuzavisnost komponenti, pa su na Windows XP korisnici imali administratorska prava, što je bila prilična rupa za pojavu virusa. Od Windows Vista korisnicima su prava smanjena i prije pokretanja aplikacije koja zahtijeva veća prava, korisnik je upozoravan na to. Same aplikacije su instalirane u stvarne direktorije, dok su naknadna upisivanja u njihove direktorije koje bi aplikacije obavile u toku rada obavljana u korisničke direktorije vezane simboličkom vezom s aplikacijom. Taj pristup je ostao i u kasnijim verzijama Windows-a uz nešto rjeđe upozoravanje korisnika.

Svaki subjekt u sistemu, a to su korisnici, grupe i kompjuteri u mreži imaju sigurnosne identifikatore (SID). Windows NT i kasniji poznaje dvije vrste grupa, lokalne (koje predstavljaju više korisnika na jednom računaru) i globalne (svi korisnici u računarskoj mreži koji se mogu prijaviti na računare u toj mreži).

Polje	Značenje
User	SID korisnika uzet iz lokalne baze ili aktivnog direktorija.
Groups	Lista SID-ova grupa koje uključuju tog korisnika
Privileges	Lista privilegija na lokalnom kompjuteru za korisnika i njegove sigurnosne grupe
Default Owner	SID korisnika ili sigurnosne grupe koji podrazumijevano postaje vlasnik objekta kreiranog od strane korisnika pridruženog ovom
Primary Group	SID korisnikove primarne grupe, samo u POSIX podsistemu
Default DACL	Ugrađeni skup dopuštenja koja će biti nad objektima kreiranim od strane ovog korisnika ako nema nikakvih drugih informacija o kontroli pristupa
Source	Proces koji je kreirao pristupni token
Type	Da li je token primarni ili personalizirajući
Impersonation Level	Do kojeg nivoa se može primiti sigurnosni kontekst klijenta predstavljenog ovim pristupnim tokenom.
Statistics	Informacija o samom pristupnom tokenu
Restricting SIDs	Opciona lista SID koji se dodaju pristupnom tokenu da kreirane niti imaju manja prava nego što je korisnik dopustio.
TS Session ID	Da li je token pridružen Terminal Services klijentskoj sesiji.
Session Reference	Za internu upotrebu.
SandBox Inert	Token uključuje SANDBOX_INERT flag.
Audit Policy	Od Windows Server 2003, za praćenje po korisniku.
Origin	Od Windows Server 2003, sadrži oznaku sesije prijave na lokalnoj mašini ili 0 ako je prijavljen preko mreže.

Slika 179 Struktura pristupnog tokena

SID je broj koji ima 128 bita i predstavlja se odvojenim grupama, tako neki od subjekata može imati oznaku poput S-1-5-21-2025429065-492874223-1748137768-500. Podsistem *Security Account Manager*, čuva listu svih korisnika i obavi prijavu. Kada se korisnik prijavi, pokupe se SID-ovi korisnika i svih grupa kojima on pripada i sastavi lista koja se pridruži procesu. Ta lista zajedno s još nekim podacima se pakuje u **pristupni token procesa**. Ako proces kreira nove procese, njegovi sigurnosni tokeni se inicijalno nasljeđuju. No, niti koje kreira proces mogu imati i

sasvim druge sigurnosne tokene, ukoliko je potrebno da se izvršavaju pod pravima udaljenih korisnika za lokalne servise.

S druge strane, svaki imenovani objekt (datoteka, direktorij, proces, semafor, imenovana cijev...) ima drugu strukturu njemu pridruženu, koja se zove **sigurnosni deskriptor**. Sigurnosni deskriptor opisuje sigurnosne attribute svakog objekta. Informacija u sigurnosnom deskriptoru uključuje vlasnika objekta i DACL, **diskrecionu** listu kontrole pristupa. Svaki element ove liste (ACE) definiše prava pristupa za pojedinog korisnika ili grupu korisnika. Vlasnik objekta kontroliše DACL i koristi je da odredi ko treba, a ko ne treba da pristupi objektu i s kojim pravima. Unutar sigurnosnog deskriptora je i **kontrolna** lista sistemskog pristupa (SACL) koju kontrolišu administratori sistema. Administratori koriste SACL da navedu zahtjeve praćenja za pristup objektu. Na primjer, administrator može postaviti SACL koji će generirati događaj u zabilježkama svaki put kada se pokuša obrisati datoteka.

Ako liste kontrole pristupa nema, smatra se da svi korisnici imaju puna prava nad objektom. Ako je ima, ali je prazna, to znači da niko nema prava pristupa. U listu se stavlja spisak korisnika i prava nad objektom. Pošto korisnik može da pripada u više grupa, prava korisnika se sabiraju. Na primjer, ako članovi grupe Inspektori imaju pravo čitanja datoteke, a članovi grupe Cistaci imaju pravo brisanja, korisnik koji je uključen u obje grupe moći će tu datoteku i da čita i da briše. Korisnik koji po bilo kom osnovu nad objektom ima negativno pravo "*No access*" neće moći pristupiti datoteci čak i ako po svim ostalim pravima može. Da bi se izbjeglo pravljenje liste pristupa za svaki objekt, uveden je indikator nasljeđivanja. Tako datoteke u direktoriju imaju podrazumijevano liste pristupa koje su date za cijeli direktorij.

Prema načinu provjere prijave, računarske mreže se organizuju u tri oblika. Prvi su **radne grupe**. Kod radne grupe, svi računari u mreži su ravnopravni i na svakom od njih se nalazi spisak korisnika i lokalnih grupa koje se mogu na njega prijaviti. Korisnici mogu dijeliti diskove, ali je broj korisnika koji mogu istovremeno pristupiti ograničen. Postavljanje radne grupe je jednostavno, ali je održavanje otežano, jer korisnik mora imati svoja imena na svim računarima na kojim namjerava raditi. U slučaju promjene lozinke, ona se mora obaviti na svim računarima. U drugom obliku, **NT domena**, u lokalnoj mreži postoji jedan računar koji obavlja poslove prijave i on se zove kontroler domene. Domenu čine svi računari čiji se korisnici prijavljuju preko istog kontrolera. Nakon što se korisnik prijavi sa radne stanice, kontroler domene provjeri kojoj grupi on pripada i vrati odgovarajuće liste pristupa. Ovdje se korisnička imena i lozinke čuvaju centralizovano, pa se promjena lozinke i dodavanje novih korisnika rade na samo jednom mjestu. Kod većih mreža domenski kontroler postaje usko grlo, jer se svi korisnici prijavljuju preko njega. Često i organizaciona struktura firme ima hijerarhiju koja se ne može uklopiti u linearnu strukturu jednog domenskog kontrolera i mnogo računara povezanog na njega. Stoga je od Windows 2000 uveden **aktivni direktorij**, imenski servis koji čuva informacije o objektima. Ti objekti su korisnici, računari, štampači i domene. Podaci u njima se čuvaju kroz hijerarhijski prostor imena, u odvojenim serverima za pojedine dijelove mreže, ali koji i dalje mogu da sarađuju po potrebi. Od ove verzije Windows unaprijeđeni su i kriptografski protokoli prijave, Kerberos i TSL (*Transport Layer Security*).

Kada se korisnik prijavi, on unese korisničko ime i mjesto gdje će se obaviti provjera (lokalna mašina, kontroler domene ili aktivnog direktorija). Korisnik se može prijaviti lozinkom, ali se mogu tražiti i opcioni akreditivi, poput certifikata i pametnih kartica. Windows korisnik može biti lokalni na računaru, poznat na NT domeni ili Aktivnom direktoriju. Na NT domeni evidentiraju se korisničko ime, lozinka i grupe kojima pripada, dok Aktivni direktorij sadrži i kontaktne informacije, organizacione podatke i certifikate. Ako se korisnik uspješno prijavio, sistem kreira sesiju prijave koja predstavlja sigurnosni kontekst korisnika.

Svaki novi proces, počevši od školjke Explorer, sadrži pristupni token i novokreiranim procesima prosljeđuje njegovu kopiju. Nekad je potrebno kreirati niti pod drugim korisničkim pravima. Na primjer, server za bazu podataka treba da obradi korisničke zahtjeve izvršavajući nit sa pravima tog korisnika, a ne pravima administratora servera koji je pokrenuo serversku aplikaciju. Ta aktivnost se zove impersonizacija. Za vrijeme njenog trajanja, nit dobiva privremeni pristupni token s pravima drugog korisnika, i po završetku niti, prava se vraćaju na podrazumijevana prava procesa.

Kada proces pokuša otvoriti osigurani objekt, na primjer, prilikom poziva *CreateFile* funkcije, proces navodi skup prava pristupa potrebnih da se pozove *CreateFile*. Sigurnosni sistem tada pogleda sigurnosni deskriptor sistema i provjeri elemente u listi pristupa da nađe onaj SID koji odgovara grupi ili korisniku tako da on odgovara pristupnom tokenu procesa. Kada je zadatak završen, korisniku se zabranjuje pristup objektu (ako je nađen element DACL liste koji to zabranjuje) ili se korisniku dodijeli odgovarajući skup prava pristupa. On može biti traženi skup prava ili njegov podskup. Na primjer, korisnik je pozvao *CreateFile* s zahtjevom za čitanje i pisanje, ali mu je nakon uvida u liste pristupa bilo moguće dodijeliti samo pravo čitanja.

Programerima su dostupne API funkcije vezane za sigurnost i prijavu iz tabele..

Funkcija	Značenje
LogonUser(lpszUsername, lpszDomain, lpszPassword, dwLogonType, dwLogonProvider, phToken)	Prijava korisnika i dodjeljuje mu pristupni token
DeleteAce(pAcl, dwAceIndex)	Briše element liste pristupa
AddAccessAllowedAce(pAcl, dwAceRevision, AccessMask, pSid)	Dodaje pravo pristupa u listu pristupa
AddAccessDeniedAce(pAcl, dwAceRevision, AccessMask, pSid)	Dodaje zabranu u listu pristupa
InitializeAcl(pAcl, nAclLength, dwAclRevision)	Kreira novu listu pristupa

Zbog opasnosti koje prijete s Interneta, sigurnosni sistem u Windows je dalje proširen od verzije Vista uvođenjem nivoa integriteta. Ovo je djelomično uvođenje MACL, obavezne kontrole pristupa. Kako korisnik može da pokrene više procesa, različitog nivoa opasnosti (neki od njih su i virusi, za koje korisnik ne zna, ali se pokreću pod njegovim imenom) treba spriječiti interakciju između nekih njegovih procesa. Procesima se dodjeljuju nivoi povjerenja, redom od najmanjeg ka najvišem: nepovjerljiv, nizak, srednji, visok, sistemski i instalator. Objekti (datoteke, ključevi registry, prozori) imaju nivoe integriteta koji predstavljaju najniži nivo povjerenja

procesa. Ako proces nižeg nivoa povjerenja od datog nivoa integriteta pokuša da pristupi navedenom objektu, čak i ako njegov vlasnik ima puna prava nad njim, neće mu biti dopušteno.

14.15.Zadaci

Zadatak 1. U Unix sistemu, rezultat `ls -l` komande je sljedeći.

```
-rwxr-xr-x 1 kemo uprava 4096 Nov 26 2007 sample.txt
```

Ako je sejo član grupe uprava, koja će on imati prava nad ovom datotekom?

Rješenje: Ako podijelimo `-rwxr-xr-x` na sljedećih 10 simbola, smatrajući da crtica znači da to pravo nije dodijeljeno:

Obična	Da	da	da	da	ne	da	da	ne	da
Vrsta	Vlasnik	Vlasnik	Vlasnik	Grupa	Grupa	Grupa	Ostali	Ostali	Ostali
Datoteke	Čitanje	pisanje	izvršenje	čitanje	Pisanje	izvršenje	čitanje	pisanje	izvršenje

Vlasnik je kemo, a grupa pridružena uz ovu datoteku je uprava. Stoga, sejo ima čitanje i izvršavanje.

Zadatak 2: Uzevši $p=19$ i $q=31$, šifrujte i dešifrujte podatak $P=23$ RSA algoritmom.

Rješenje: Za $p=19$ i $q=31$ dobija se $N=p \cdot q=589$. Dobija se da je

$$(p-1)(q-1)=2 \cdot 3 \cdot 3 \cdot 2 \cdot 3 \cdot 5=540$$

Uzmimo za e proizvoljan broj relativno prosti sa 540. Najmanji takav broj je $e=7$. Nađimo d za koje važi $7d \bmod 540=1$

Pretraga d se može izvršiti program skim kodom.

```
for (int a=0;a<100000;a++) if ((7*a)%540==1) printf("%d ",a);
```

Prvo d koje zadovoljava uslov $d=463$. Da se ne računa stepenovanje ogromnim brojevima, koristiće se relacija distributivnosti u modularnoj aritmetici :

$$(ab) \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$$

Za šifrovanje podatka $P=23$ i račun $C = P^e \bmod N = 23^7 \bmod 589$, izračunaće se pomoćni rezultati:

$$23^2 \bmod 589=529$$

$$23^4 \bmod 589=(23^2 \cdot 23^2) \bmod 589=(529 \cdot 529) \bmod 589=66$$

$$23^6 \bmod 589=(23^4 \cdot 23^2) \bmod 589=(66 \cdot 529) \bmod 589=163$$

$$C = P^e \bmod N = 23^7 \bmod 589 = 23^{6+1} \bmod 589 = (23^6 \cdot 23^1) \bmod 589 =$$

$$=((23^6 \bmod 589)(23^1 \bmod 589)) \bmod 589 = (163 \cdot 23) \bmod 589 = 215$$

$$\text{Za dešifrovanje podatka } C=215 \text{ i račun } P = C^d \bmod N = 215^{463} \bmod 589 =$$

$215^{256+128+64+8+4+2+1} \bmod 589$ redom će se na sličan način izračunati $215^2 \bmod 589$, iz njega dobiti $215^4 \bmod 589$, pa $215^8 \bmod 589$, $215^{16} \bmod 589$, i tako do $215^{256} \bmod 589$. Iz ovih međurezultata se dalje dobivaju $215^{256+128} \bmod 589$, pa $215^{256+128+64} \bmod 589$, pa $215^{256+128+64+8} \bmod 589$, pa $215^{256+128+64+8+4} \bmod 589$ pa $215^{256+128+64+8+4+2} \bmod 589$ i na kraju $215^{256+128+64+8+4+2+1} \bmod 589$

Rezultat je $P=23$.

15.ARHITEKTURA MICROSOFT-OVIH SISTEMA

Ogromnu većinu personalnih računara, kao i dobar dio serverskih računara pogoni neka od verzija Windows operativnog sistema. U ovom poglavlju daće se osvrst na historiju i arhitekturu Microsoft-ovih operativnih sistema. Microsoft je nastojao da novi operativni sistemi izvršavaju što više softvera namijenjenog za starije sisteme, pa će biti riječi i o arhitekturi verzija operativnih sistema koje se danas malo koriste, kao i najnovijih verzija.

15.1.MS DOS

Nakon uspjeha tada manjih kompanija Apple, Microsoft, Commodore, Tandy i Atari, i IBM je 1981. godine odlučio da napravi svoj prvi personalni računar, IBM PC 5150, koristeći Intel 8088 procesor. Nakon neuspjelog dogovora sa Digital Research (vlasnicima CP/M) za razvoj operativnog sistema, Microsoft preuzima razvoj operativnog sistema. Ova firma je prije toga uglavnom razvijala BASIC interpretere za osmobicne računare. Tim Paterson iz firme Seattle Computers prelazi u Microsoft i dovršio svoj raniji operativni sistem QDOS, preimenujući ga u MSDOS, odnosno u IBM verziji PC DOS. MSDOS je jedan od najjednostavnijih operativnih sistema koji se i danas ponegdje koristi. Prva verzija MS DOS-a je bila veoma primitivna, ali nakon nekoliko promjena omogućila je sve neophodne funkcije koje pruža operativni sistem. MS DOS je prošao kroz više promjena tokom godina da bi omogućio bolju funkcionalnost. MS-DOS je jednozadaćni operativni sistem, što znači da može da izvršava samo jedan program u isto vrijeme. Grafičkog okruženja nema, pa korisnik mora kucati tekstualne komande i odgovore kada komunicira sa operativnim sistemom. Kasnije verzije uključuju dodatni program DOSSHELL koji omogućava pokretanje programa iz menija.

MS DOS smatra svaki odvojeni program i podatke individualnom datotekom. Svaka datoteka ima ime, koje je podijeljeno u dva dijela, ime datoteke i ekstenziju. Ime datoteke može biti dugačko do osam znakova, a ekstenzija može biti dužine do tri znaka. DOS je između 1980 i 1999. prošao kroz 17 verzija pri čemu je zadnjih sedam godina bio dio arhitekture Windows sistema, sve do Windows ME. Nove verzije su uglavnom imale podršku za sve veći kapacitet *floppy* i *hard* diskova i poneki novi alat. Za MS DOS je napisan ogroman broj programa, pa i nakon što je Microsoft prestao s podrškom za ovaj operativni sistem, njegovi kasniji sistemi su čuvali kompatibilnost s MS DOS sistemom.

DOS cijeli memorijski prostor vidi kao područje od jednog megabajta memorije (Slika 180), pri čemu se na nižim adresama od adrese 0 nalaze interrupt vektori, zatim određeni prostor zauzima jezgro sistema, nakon koga slijedi prostor za korisnički program. Od 640-g kilobajta smješteni su video memorija, nakon nje ROM video kartice, mali prostor za proširenje RAM-a i na vrhu memorije ROM BIOS. Memorijskim lokacijama se ne pristupa linearno, nego koristeći segmentni i ofsetni dio adrese, ali su u ovoj tabeli date apsolutne adrese. No, kako ovaj operativni sistem većinu vremena provodi u realnom režimu rada, apsolutna adresa se dobiva zbirom ofsetnog dijela adrese i segmentnog dijela pomnoženog s 16.

Adresa	Područje
00000	Interrupt vektori
00400	ROM BIOS područje
00500	DOS parametarsko područje
00700	IO.SYS (u PC DOS IBMBIO.COM)
Promjenjivo	MSDOS.SYS (u PC DOS IBMDOS.COM)
Promjenjivo	Prostor koji je rezervirao CONFIG.SYS za drajvere i buffere
Promjenjivo	Rezidentni dio COMMAND.COM
Promjenjivo	Pokazivač na varijable okruženja
Promjenjivo	Blok varijabli okruženja rezidentnog programa
Promjenjivo	Rezidentni program
Promjenjivo	Blok varijabli okruženja trenutnog programa
Promjenjivo	Trenutni program
Promjenjivo	Izbrisivi dio COMMAND.COM
8FFFF	Kraj konvencionalnog memorijskog područja (640 K)
A0000	Video memorija
C0000	ROM video kartica
E0000	Memorijska proširenja EMS (proširena memorija)
F0000	ROM BIOS
FFFFF	Kraj 8086 dostupnog područja
100000	Visoka memorija od DOS 5.0, tu se prebaci MSDOS.SYS
110000	Produžena memorija preko ekstendera VCPI ili DPPI
FFFFFF	Krajnja adresa dostupna preko ekstendera (32 M)

Slika 180 MS DOS, memorijska mapa

Neophodni dio MS DOS, koji se stalno nalazi u memoriji, se sastoji od tri datoteke.

- IO.SYS, koji sadrži osnovne drajvere perifernih uređaja i učitava u memoriju glavni dio DOS-a MSDOS.SYS. U PC DOS on se zove IBMBIO.COM
- MSDOS.SYS, je jezgro operativnog sistema i sadrži implementaciju sistemskih poziva MSDOS-a, kroz softverske trap-ove između INT 20h i INT 2Fh, od kojih je posebno značajan INT 21h. U PC DOS on se zove IBMDOS.COM
- COMMAND.COM je interpreter komandne linije, koji od korisnika očekuje unos komande s parametrima i izvršava komandu ugrađenu u njega ili eksterni program. Ova datoteka se ne čuva cijelo vrijeme u RAM-u u potpunosti, ali je jedan njen dio uvijek prisutan. On ima mali skriptni jezik koji omogućava pokretanje programa s ekstenzijom .BAT. Jedan od njih AUTOEXEC.BAT se pokreće pri svakom podizanju MSDOS.

Ostatak MSDOSa se tipično čuva na disketi ili fiksnom disku, u direktoriju sa imenom C:\DOS. Datoteke smještene u tom direktoriju obavljaju zadatke operativnog sistema kao što je program za formatiranje diskova, editor teksta, napredni programi za kopiranje datoteka, program za promjenu atributa, ali nisu neophodne za normalni rad operativnog sistema.

Na operativnom sistemu MS DOS, API funkcije se pozivaju mašinskom instrukcijom INT 21h, (i još nekim drugim INT naredbama), pa izgledaju kao sistemski pozivi, iako ovaj operativni sistem nema razliku između korisničkog režima i režima jezgra. Prije poziva ove instrukcije u registru AH se nalazi broj funkcije, a u drugim registrima poput AL, BH, SI itd. dodatni parametri. Vrijednosti

AH registra kojim se bira poziv su navedene u tabeli Slika 181. Pozivi su implementirani u datoteci MSDOS.SYS.

01	Čitaj znak s standardnog ulaza	02	Piši znak na standardni izlaz
05	Piši znak na štampač	06	Konzolni ulaz izlaz
07	Direktno čitanje znaka bez pisanja	08	Čitanje znaka s STDIN bez pisanja
09	Piši znak na standardni izlaz	0A	Ulaz koji koristi buffer
0B	Dobavi status standardnog izlaza	0C	Isprazni ulazni buffer
0D	Disk reset	0E	Odaberi podrazumijevani disk
19	Trenutni podrazumijevani disk	25	Postavi interrupt vektor
2A	Čitaj sistemski datum	2B	Postavi sistemski datum
2C	Čitaj sistemsko vrijeme	2D	Postavi sistemsko vrijeme
2E	Uključi verifikaciju	30	Dobavi verziju DOS-a
31	Završi program ali ostaje u memoriji	35	Dobavi interrupt vektor
36	Dobavi slobodan disk prostor	39	Kreiraj poddirektorij
3A	Obriši poddirektorij	3B	Postavi radni direktorij
3C	Kreiraj datoteku	3D	Otvori datoteku
3E	Zatvori datoteku	3F	Čitaj datoteku
40	Piši u datoteku	41	Obriši datoteku
42	Pozicioniraj se na datoteku	43	Pročitaj/postavi atribut datoteke
47	Dobavi trenutni direktorij	4C	Napusti program
4D	Dobavi povratni kôd	54	Očitaj fleg verifikacije
56	Preimenuj datoteku	57	Pročitaj/postavi datum datoteke

Slika 181 DOS API/ Sistemski pozivi

Sljedeći primjer u asemblerskom jeziku poziva funkciju za završetak programa.

```
MOV AH, 4Ch
MOV AL, 0
INT 21h
```

Jedini datotečni sistem koji MS DOS poznaje je FAT u verzijama FAT 12 i FAT 16. Za podršku datotečnim sistemima na CD mora se instalirati poseban drajver MSCDEX što dodatno smanjuje raspoloživu memoriju.

Ograničenja DOS-a na jedan program u jednom trenutku i limit na 640 K memorije, vremenom su rješavana na više načina. Uvođenje sistemskog poziva INT 27h (*Terminate and Stay Resident*, TSR) je omogućilo da program ostane u memoriji i nakon njegovog završetka, a za nove aplikativne programe ostaje prostor iza posljednjeg bajta koji je zaustavljeni program iskoristio. Ta se osobina koristila uglavnom za drajvere, a zaustavljenom programu se onda pristupa preusmjeravajući prekide. Nedovoljan memorijski prostor od 640K tzv. konvencionalne memorije se prevazilazio jednom od sljedećih tehnika:

Overlay: Dijelovi programa se nalaze u eksternoj datoteci na disku, slično mehanizmu *swapping*, ali bez spašavanja trenutnog stanja. Poziv potprograma koji se nalazi u *overlay* se prevede kao poziv kratke rutina koja preusmjeri poziv potprograma na prebacivanje potprograma iz datoteke u memoriju ili na njegov stvarni programski kôd, kada je on već u memoriji.

Expanded memorija: Neiskorišteni memorijski prostor između apsolutnih adresa E0000 i F0000 se popunjavao RAM memorijom podijeljenom u banke veličine po 16 K. U jednom trenutku u memoriji mogu biti četiri takve banke, ali ukupni broj raspoloživih stranica iz kojih se te banke biraju nije ograničen.

Extended memorija: Dodatnim programima, kao što je EMM386.EXE, procesor se prebacuje u zaštićeni režim rada, čime mu postaje dostupna memorija i iznad 1M, prebaci dijelove memorije aplikativnom programu koji se nalazi u dijelu memorije dostupnom pod MS DOS i vraća u realni režim rada ili radi u virtualnom režimu rada (koji je zaštićeni režim, ali oponaša reaktivni režim rada procesora).

15.2.Xenix, OS 2, Azure Sphere

Bilo je jasno da sa pojavom moćnijih procesora, kao što su 80286 i 80386, jedan jednogprocesni operativni sistem kao što je MS DOS ograničen u području memorije na 1M nije adekvatan. Zato su u Microsoftu krenuli u više projekata moćnijih operativnih sistema, ali nakon uspjeha Windows, ti sistemi su napušteni.

Xenix: Microsoft je uzeo 1979 licencu od AT&T da napravi UNIX kompatibilan operativni sistem, i angažujući druge kompanije Altos, Tandy i SCO napravljene su verzije za PDP-11, Zilog Z8001, 68000 i Intel 8086. Godine 1987, prava potpuno predana firmi SCO.

OS /2: Zamišljen kao višeproceneni i grafički nasljednik DOSa, sistem OS/2 se pojavio u doba kada mašine nisu bile dovoljno jeftine. Razvijen je zajedno s firmom IBM. Nakon razlaza 1990, nastavljen je razvoj od strane IBM-a, do 2005, kada i ova firma prestaje s razvojem OS/2. Arhitektura je dijelom iskorištena za arhitekturu Windows NT.

Nakon 25 godina isključivog razvoja Windows platforme, Microsoft je 2018. godine objavio svoj prvi Linux bazirani operativni sistem, Azure Sphere, namijenjen za Internet of Things uređaje. To su neračunarski uređaji povezani na Internet, poput veš mašina i medicinskih uređaja. Kernel je tipa sistema u realnom vremenu sa akcentom na sigurnost.

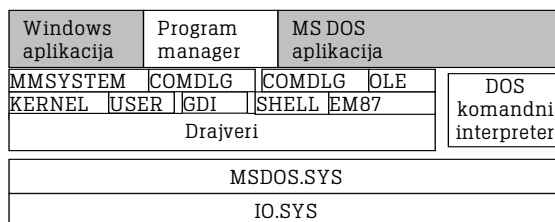
15.3.Windows

Ipak, ono što je obilježilo smjenu dva milenija je skup operativnih sistema koji se zovu Windows. Iako pokrenut sa zakašnjenjem u odnosu na MacOS na Apple Macintosh računarima, sveobuhvatna dostupnost klonova IBM PC računara kombinovana sa njihovom niskom cijenom, prethodnom popularnošću MS DOS, lakoćom upotrebe i dobrim marketingom postigla je njegovu apsolutnu dominaciju na personalnim računarima, Windows se razvijao u 4 različite razvojne linije: šesnaestbitne verzije (koje su samo produžetak MS-DOS-a grafičkim sučeljem), verzije 9x (32-bitni sa ugrađenim MS-DOS), Windows NT verzije (sa sigurnim 32 bitnim ili 64 bitnim jezgrom) i verzije CE (jezgro u ROM-u za džepne uređaje).

15.4.Šesnaestbitne verzije Windows

Nakon uspjeha Apple Macintosh, Microsoft-ov odgovor je bio u okruženju Windows. Prve verzije Microsoft Windows su zapravo samo dodatak grafičkog okruženja u MS DOS. U ranim verzijama, Windows 1.0 i 2.0 bio je moguć samo rad u realnom režimu na 8086 procesoru, ali sa mogućnošću prebacivanja između procesa. To je značilo i dalje ograničenje adresnog prostora na 1M i nikakvu zaštitu između aplikacija. Ni raspoređivanje nije bilo preemptivno, nego se zasnivalo na uredno pisanim petljama poruka, opisano u poglavlju o Windows GUI. Windows 1.0 nije imao preklapanje prozora a Windows 2.0 jeste. Ove verzije Windowsa su sličnije

DOSShellu nego Windowsu 3.1. Budućnost je još uvijek gledana u OS/2. Prve dvije verzije Windows-a su uglavnom ignorirane, jer je malo aplikacija pisano za njih. Verzije Windows 3.0, 3.1 i 3.11 *for workgroups* su preokrenule softverski svijet mikroracunara, iako se za 3.x seriju još uvijek može reći da nisu operativni sistem nego samo grafičko korisničko sučelje za MS DOS. Dakle, MS-DOS je primarni operativni sistem, a Windows 3.x omogućava GUI. Kada se podiže kompjuter, MSDOS se mora prvi učitati, a Windows se pokreće komandom WIN. Nakon što je GUI pokrenut, vidi se školjka Windowsa 3.x, *Program Manager*.

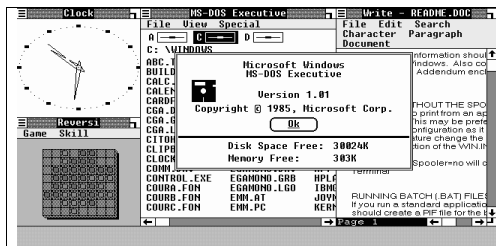


Slika 182 Arhitektura DOS+Windows 3

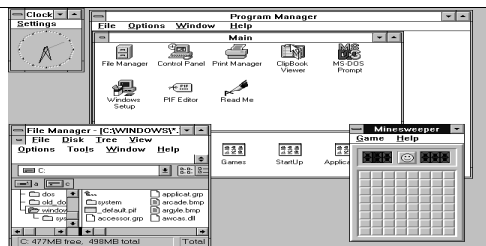
Arhitektura Windows 3.x je relativno jednostavna. Pored 3 datoteke potrebne za pokretanje MS-DOS, za pokretanje Windows u 16 bitnom režimu potrebne su datoteke koje sadrže systemske pozive. Za razliku od MSDOS, API pozivi se ne pozivaju mašinskom INT instrukcijom nego kao obični potprogrami, ali su smješteni u dijeljene biblioteke. Zavisno od raspoložive memorije i procesora, Windows 3.0 se mogao pokrenuti u jednom od tri režima. U realnom režimu (potreban 8086 procesor i 640 K RAM) Windows je samo GUI za MS-DOS, bez ikakve zaštite. Standardni režim (procesor 80286 u zaštićenom režimu uz 1 M do 16 M memorije) koristi segmentaciju sa zaštitom. U proširenom režimu (bar 80386 sa 2 M RAM) koristi se segmentacija sa straničenjem, teoretsko adresiranje do 4 G (u praksi do 512 M) i preemptivno raspoređivanje MS-DOS programa. Ali, u sva tri režima se Windows programi raspoređuju kooperativno, što i predstavlja najveću manu ove serije. Krah jedne aplikacije izaziva krah cijelog sistema, a česta je i potreba za nekim MS DOS drajverima.

Glavne datoteke Windows 3.x serije su date u tabeli

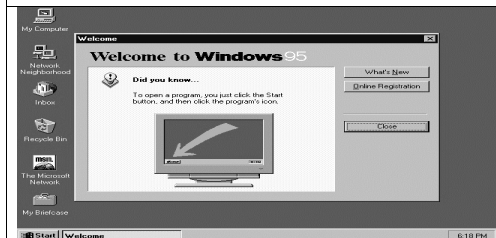
KERNEL.EXE KRNL386.EXE	Alocira i prati systemske resurse, koordinira U/I zahtjeve prema hardveru, upravljanje memorijom i pokretanje programa.
USER.EXE	Obrađuje U/I zahtjeve, poruke tastaturu, miša, zvuk, korisničko sučelje
GDI.EXE	<i>Graphic Driver Interface</i> , odgovoran za crtanje i štampanje
SHELL.DLL	API za pristup korisničkoj školjci (od 3.0)
MMSYSTEM.DLL	Funkcije za multimediju (od 3.1)
COMMDLG.DLL	Standardni dijaloški prozori (od verzije 3.1)
COMMCTRL.DLL	Korisničko sučelje: kartice, potencijometri, indikatori napretka. (od 3.11)
PROGMAN.EXE, WINFILE.EXE, MSDOS.EXE	Korisničke školjke: Program manager (od 3.0), File manager (od 3.0) i MSDOS Executive (od 2.0)
*.DRV	Datoteke sa drajverima za periferijske uređaje
386SPART.PAR	Datoteka u kojoj se čuva sadržaj virtualne memorije
CONTROL.INI	Datoteka za konfiguraciju alatom Control Panel
SYSTEM.INI	Datoteka za konfiguraciju drajvera i memorije
PROGRAM.INI	Sadržaj prozora za izbor programa za pokretanje
WIN.INI	Postavke čuvara ekrana, boje, fontove i autostart programa



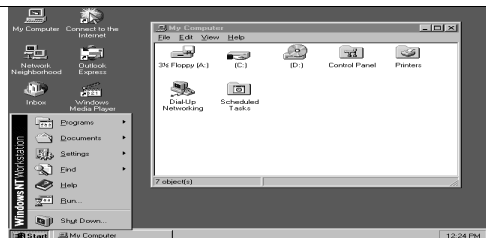
Windows 1.0



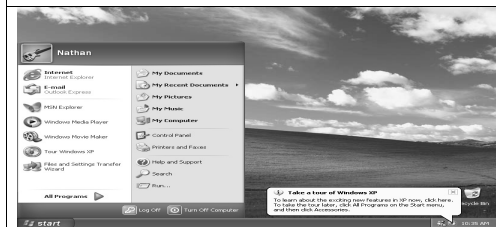
Windows 3.1



Windows 95



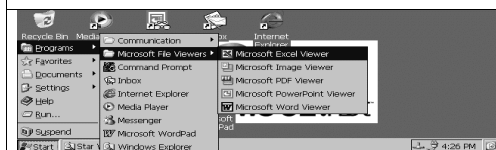
Windows NT 4.0



Windows XP



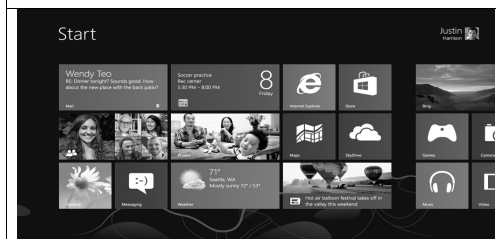
Windows 7



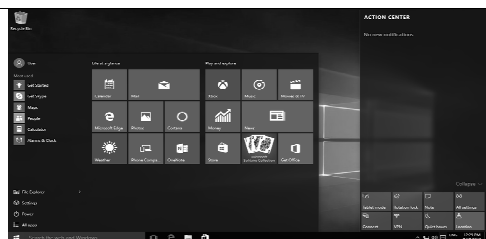
Windows CE 4.0



MS-DOS 6.22



Windows 8

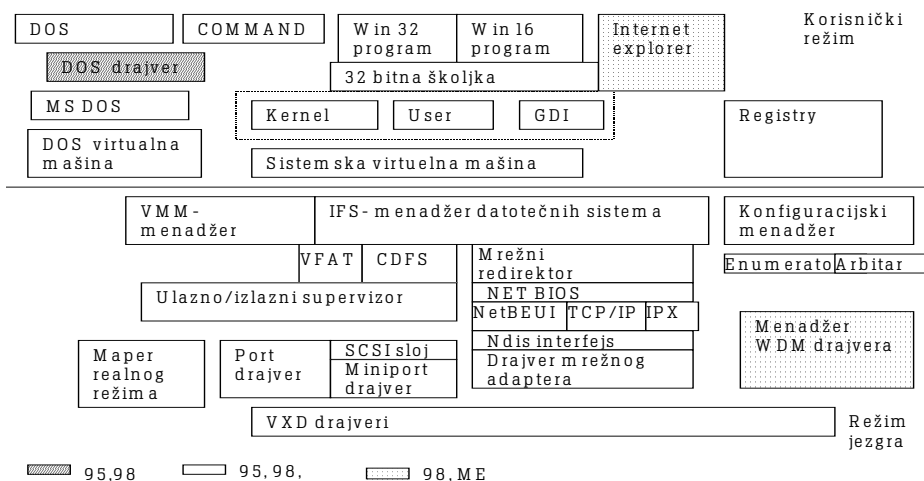


Windows 10

15.5.Windows 95, 98, ME

Veliki broj programa za MS DOS, i Windows 3.1, kao i mali hardverski zahtjevi ovih operativnih sistema, naveli su Microsoft da razvije seriju Windows koja se često naziva 9x i uključuje Windows 95, Windows 98, Windows 98SE i Windows ME. One se odlikuju 32 bitnim grafičkim okruženjem, ali velikom kompatibilnošću s starijim aplikacijama, i manjim hardverskim zahtjevima od NT serije, po cijenu manje sigurnosti i prenosivosti. Jezgro je modifikovani MS DOS.

Windows 95 se pojavio u više izdanja, od početnog na 15 disketa, do posljednjeg OSR 2.5 na CD sa podrškom za USB i FAT 32. Windows 95 ima MS-DOS interfejs za zaštićeni režim DPMS i GUI u zaštićenom režimu. DPMS (*DOS Protected Mode Interface*) režim je unapređena verzija MSDOS-a koja podržava upotrebu produžene memorije. Windows 95 prvo starta DPMS, a zatim učitava GUI. DPMS zahtijeva iste tri datoteke kao i MS-DOS: IO.SYS, MSDOS.SYS, i COMMAND.COM. Ove datoteke se razlikuju u odnosu na istoimene u ranijim verzijama Windows-a, podržavaju duga imena datoteka, a MSDOS.SYS je sada tekstualna datoteka koja sadrži konfiguracione postavke, dok IO.SYS sadrži pored drajvera uređaja i implementaciju sistemskih poziva (što je u MS DOS bilo u datoteci MSDOS.SYS). Kada se pokrene GUI, on preklapa DPMS i učitava 32 bitne drajvere koji ne zauzimaju memoriju potrebnu MSDOS programima.



Slika 184 Arhitektura Windows 9x

Arhitektura Windows 95 (Slika 184) zbog kompatibilnosti s MS DOS je dosta miješana. Postoje dvije vrste drajvera. Drajveri u realnom režimu se učitavaju iz Config.sys (linije device=sxss) i Autoexec.bat (programi koji ostaju u memoriji nakon završetka). Drajveri u zaštićenom režimu se učitavaju iz GUI-a, pri čemu oni najčešći su u datoteci VMM386.VXD, ali se mogu zamijeniti drugom datotekom. Upravljač virtualnom memorijom VMM podržava upotrebu memorije na nivou DPMS i GUI. VMM može kreirati virtualne mašine (VM), područja memorije u kojima se aplikacije izvršavaju. Upotreba više virtualnih mašina omogućava MSDOS programima da se izvršavaju u vlastitom memorijskom prostoru, poboljšavajući

performanse operativnog sistema. Kada se radi u GUI režimu, DOS pozivi (npr. INT 21h) se preusmjeru na odgovarajuće drajvere unutar VMM.

Uz VMM je instalabilni datotečni sistem (IFS) koji pruža podršku za hard diskove, CDROM-ove i mrežne diskove. IFS podržava duga imena datoteka, a radi i u DPMI i u GUI režimu.

U GUI režimu se nalazi srce Windowsa 95: Kernel, GDI i USER. Ove komponente održavaju sve glavne funkcije unutar Windowsa 95. Windows 95 ima preemtivni *multitasking* (svaki program ima određeno vrijeme procesora) za razliku od Windowsa 3.x koji ima kooperativni *multitasking* (vrijeme koje program ima bazirano je na načinu kako je aplikacija pisana). Ove tri komponente imaju potpuno istu funkciju kao kod Windows 3.1 – kernel32.dll je 32 bitna verzija kernel.exe, gdi32.dll je 32 verzija gdi.exe, a user32.dll je 32 bitna verzija user.exe. Na vrhu arhitekture Windowsa 95 je korisničko sučelje sa ikonama, prozorima i alatka.

Kako su INI datoteke postale mnogobrojne, zamijenjene su *Registry* bazom. To je centralizovana konfiguracijska baza za softverske postavke i hardverske postavke. Svi konfiguracijski parametri su smješteni u datoteke User.dat i System.dat, pri čemu operativni sistem pravi njihove kopije kao User.da0 i System.da0. Registry se čita alatom regedit.exe.

Datotečna struktura za Windows 95 je veoma unapređena u odnosu na Windows 3.x. Windows 95 podržava duga imena datoteka (LFN) prevazilazeći ograničenje imena na 8 slova. Imena Windows 95 datoteka sadrže do 255 znakova uključujući razmake i tačke. Svako dugačko ime datoteka ima ekvivalentno kratko ime, npr. datoteka *This is a really really really long file name.txt* se također zove *thisis~1.txt*

Microsoft Windows 98 uključuje bolje urađeno upravljanje napajanjem, podršku za više monitora i integriranu podršku za najnoviji hardver. Također je uključena podrška za novi *Win32 Driver Model* (WDM), koji dopušta WDM uređaju da radi i pod Windowsom 98, kao i budućim verzijama Windowsa NT koristeći isti drajver. Pojavio se u dvije verzije, Windows 98 i Windows 98SE.

Posljednji Windows iz 9x serije je **Windows ME**, (*Millenium Edition*) i on je i posljednja inkarnacija MS DOS, iako je ovog puta DOS dobro sakriven. Windows ME je Windows 4.90 iznad MSDOS-a 8.00. U ovom Windowsu povećane su mogućnosti zaštite sistema od slučajnog brisanja datoteka (*System Restore*, *Windows File Protection*) i ovo je prvi Windows koji ima automatsko ažuriranje. Izbačene su mogućnosti prelaska u čisti MSDOS režim i spriječeno učitavanje MS DOS *driver*-a. Po riječima Microsofta, ovo je urađeno radi povećanja brzine i stabilnosti sistema, no upravo su mogućnost izvršavanja široke baze DOS programa kojima smeta zaštićeni režim rada i manji hardverski zahtjevi bile glavne prednosti Windows 9x serije u odnosu na Windows NT seriju. S druge strane, zadržavanje DOS jezgra, dizajniranog za jednoprocenjski rad nije moglo ukloniti nestabilnost sistema, tako da je ova verzija često smatrana najgorom verzijom Windows-a svih vremena

15.6.Windows NT verzije

U toku razvoja OS/2 3.0, kao stabilnog, portabilnog, više-procesnog operativnog sistema uključen je David Neil Cutler kao prvi glavni projektant (ranije projektovao RSX 11 i VMS). Operativni sistem je dizajniran sa razdvojenim API od jezgra

operativnog sistema. Nakon uspjeha Windows 3.0, odlučeno je da se umjesto OS/2 API, primarno razvija 32 bitna verzija Windows API. Tako je 1993 nastao Windows NT. Nasljednici Windows NT su i danas aktualne verzije Windows-a i dijele zajednički osnovni dizajn jezgra, više od 20 godina, i uz manje razlike koje će biti kasnije objašnjene, opis je relevantan za većinu verzija Windows. Sve verzije Windows, koje pripadaju NT seriji su date na slici Slika 185).

Ime	Izdanja	Datum
Windows NT 3.1	Windows NT, Advanced Server	27.7.1993
Windows NT 3.5	Workstation, Server	21.9.1994
Windows NT 3.51	Workstation, Server	30.5.1995
Windows NT 4.0	Workstation, Server, Server Enterprise Edition, Terminal Server, Embedded	29.7.1996
Windows 2000	Professional, Server, Advanced Server, Datacenter Server	17.2.2000,
Windows XP	Home, Professional, Media Center (original, 2003, 2004 & 2005), Tablet PC (original i 2005), Starter, Embedded, Home N, Professional N	25.10.2001
Windows Fundamentals for Legacy PCs		8.7.2006
Windows XP	64-bit Edition Version 2003	28.3.2003
Windows Server 2003	Standard, Enterprise, Datacenter, Web, Storage, Small Business Server, Compute Cluster	24.4.2003
Windows XP	Professional x64 Edition	25.4.2005
Windows Server 2003 R2	Standard, Enterprise, Datacenter, Web, Storage, Small Business Server, Compute Cluster	6.12.2005
Windows Home Server		16.7.2007
Windows Vista	Starter, Home Basic, Home Premium, Business, Enterprise, Ultimate, Home Basic N, Business N	30.11.2006
Windows Server 2008	Foundation, Standard, Enterprise, Datacenter, Web Server, HPC Server, Itanium-Based Systems	27.2.2008
Windows 7	Starter, Home Basic, Home Premium, Professional, Enterprise, Ultimate, Thin PC	22.10.2009
Windows Server 2008 R2	Foundation, Standard, Enterprise, Datacenter, Web Server, HPC Server, Itanium-Based Systems	22.10.2009
Windows Home Server 2011		6.4.2011
Windows 8	8, Pro, Enterprise, RT	26.10.2012
Windows Server 2012	Foundation, Essentials, Standard, Datacenter	4.9.2012
Windows 8.1	8.1, Pro, Enterprise, RT 8.1	18.10.2013
Windows Server 2012 R2	Foundation, Essentials, Standard, Datacenter	18.10.2013
Windows 10	Home, Pro, Enterprise, LTSC, Education, Mobile, Mobile Enterprise, IoT	29.08.2015
Windows Server 2016	Essentials, Standard, Datacenter	26.09.2016
Windows Server 2019		

Slika 185 Sve verzije i izdanja sa Windows NT jezgrom

Nakon tri verzije vizualno slične (Windows NT 3.1, NT 3.5 i NT 3.51) sa Windows 3.1, ali diametralno različite arhitekture, jer koriste odlično dizajnirano jezgro, za razliku od MS DOS, slijedile su dvije verzije koje su vizualno slične Windows 95 i Windows ME. **Windows NT 4.0** uvodi grafičko sučelje sa Start dugmetom i ubranu grafiku jer je GDI prebačen u jezgro, uz blago smanjenje stabilnosti. **Windows 2000** uvodi dva nova podsistema u jezgro, *Plug and Play* i *Power Manager*. *Plug and play*, uveden sa Windows 95, sada je dio Windows 2000. Mogu se priključiti kompatibilni uređaji i Windows će konfigurirati uređaj. To će možda zahtijevati promjenu konfiguracije drugih uređaja. Serverska verzija Windows 2000 je uvela novi način grupiranja računara i prava. Umjesto dotadašnjeg sistema domena koje su predstavljale skup korisnika i računara, uveden je hijerarhijski organizovani aktivni direktorij.

Windows XP je vjerovatno najpopularnija verzija Windows svih vremena i najduže u upotrebi, jer je period podrške bio od 2001 do 2014. godine. Ovo je prva verzija sa NT jezgrom namijenjena ne samo profesionalnoj nego i širokoj publici. Pored vizualnih promjena u izgledu prozora, napravljeno je i više, ne mnogo revolucionarnih, ali dosta korisnih promjena u jezgru. Promijenjeno je jezgro da se ubrza dizanje sistema. Stabilnost je povećana time što Windows XP provjerava da li je driver certificiran u *Windows Hardware Quality Lab* (WHQL). Necertificirani *driver*-i mogu i dalje raditi na korisnikov zahtjev, ali postoji i crna lista *driver*-a koji ne rade. Windows XP uključuje *System Restore* koji vraća sistem u stanje prije instalacije problematičnog drajvera. Dopušteno je da se u memoriji nalazi više različitih verzija DLL-ova kako bi aplikacije koje su ovisne o specifičnoj verziji DLL-a radile. Ubrzano je vrijeme podizanja operativnog sistema pomoću *Simple Boot Flag* (SBF) specifikacije.

Jezgro bazirano na kodu Windows XP korišteno je u serverskoj verziji, Windows Server 2003. **Window Server 2003** je dizajniran kao nastavak Windows 2000 servera. Windows 2003 server uključuje *.NET Framework 1.1* i *XML Web Services*. Jedno vrijeme se zvao i Windows 2003 .NET server. Omogućeno je brže i lakše upravljanje Aktivnim direktorijem. Uključen je *Internet Information Server* (IIS) 6. Postojala je i 64 bitna verzija Windows Server 2003 predviđena je za Intel 64-bit Itanium 2, ostale verzije su za 32 bitni Pentium.

Microsoft je počeo planirati Longhorn 2001. godine, odmah nakon izlaska XP-a. Bio je očekivan krajem 2003. "Longhorn" je trebao uključiti mnogo novih mogućnosti. Ali zbog tehničkih problema, originalni "Longhorn", zasnovan na XP-ovom izvornom kodu, je odbačen i krenulo se otpočetka, izradom na bazi sistema Windows Server 2003. Operativni sistem je na kraju objavljen pod imenom **Windows Vista**. Neki najavljeni noviteti kao što je datotečni sistem baziran na SQL su odbačeni ili odgođeni. Vista se pokazala kao imunija na viruse, manje stabilna od XP, manje kompatibilna s DOS-om, jer više ne rade DOS aplikacije u grafičkom režimu. Windows Vista je uveo dosta novosti u korisničkom režimu rada, kao što su grafičko sučelje *Aero* sa novom školjkom, prepoznavanje govora, *Internet Explorer 7*, *Media player 11*, *Mail*, *Calendar*, *Photo Gallery*, *DVD maker*, direktoriji koji koriste simboličke veze, veća sigurnost, alat za prijavu problema i *Windows update*. Što se tiče jezgra, promijenjen je *boot loader* i radi druge arhitekture *driver*-a, *Windows Driver Foundation* (umjesto dotadašnjeg WDM) manje *driver*-a je u jezgru. Upravljanje memorijom je promijenjeno tako da se programi mogu učitati na nepredvidive, slučajne lokacije, kako bi se prevarili virusi. Kako je radi sigurnosti ova verzija često tražila dodatna odobrenja korisnika za akciju i zahtijevala jači hardver nego XP, ova verzija Windowsa nije bila popularna.

Serverski ekvivalent za Windows Vista je **Windows Server 2008** (radno ime Longhorn Server). Ova serverska verzija ima široku podršku virtualizaciji, aktivnom direktoriju (mreži računara s zajedničkom prijavom) i naprednom komandnom interpreteru PowerShell. Windows server 2008 R2 koristi jezgro iz Windows 7.

Windows 7 je prvenstveno namijenjen da bude nadogradnja i popravak mana Windows Vista, uz zadržavanje kompatibilnosti sa hardverom i softverom dizajniranim za Vista. Zadržavajući sličan izgled kao Vista, u Windows 7 sučelje

pojednostavljeno, uz dodatak redizajniranog *taskbar*-a koji omogućava aplikacijama da se dodaju na njega, i nove funkcije upravljanja prozorima. Ostale nove mogućnosti su dodane u operativni sistem, uključujući biblioteke, novi sistem dijeljenja datoteka HomeGroup, i podrška za *multitouch* unos. "Action Center" pruža pregled sigurnosti sistema i informacije o održavanju, a podešavanja su napravljena u kontroli korisničkih naloga da bi bio manje nametljiv. Windows 7 se također isporučuje s ažuriranim verzijama nekoliko standardnih aplikacija: *Internet Explorer*, *Windows Media Player* i *Windows Media Center*. Od NT 4, sve veća integracija Win32 u jezgro, učinila udaljšavanje od originalne ideje o fleksibilnom NT jezgru sa proizvoljnim podsistemima. Iz tog razloga se dosadašnje verzije Windowsa nisu mogle koristiti bez grafičkog okruženja, koje je nepotrebno na serverima i ugrađenim uređajima. Stoga se krenulo u redizajn, napravljen je MinWin, minimum da se Windows 7 pokrene (bez GUI, sa mrežnim servisima, ukupno 25 M). To je zahtijevalo redizajn sistemskih biblioteka Win32 podsistema, pa su neke funkcije iz *Kernel32.dll* i *Advapi32.dll* prebačene u manji *Kernelbase.dll*, a dinamičke biblioteke se mogu dijeliti u više virtualnih biblioteka, što se zove API skupovi. Obrada konzole je prebačena iz *csrss* u *conhost.exe*

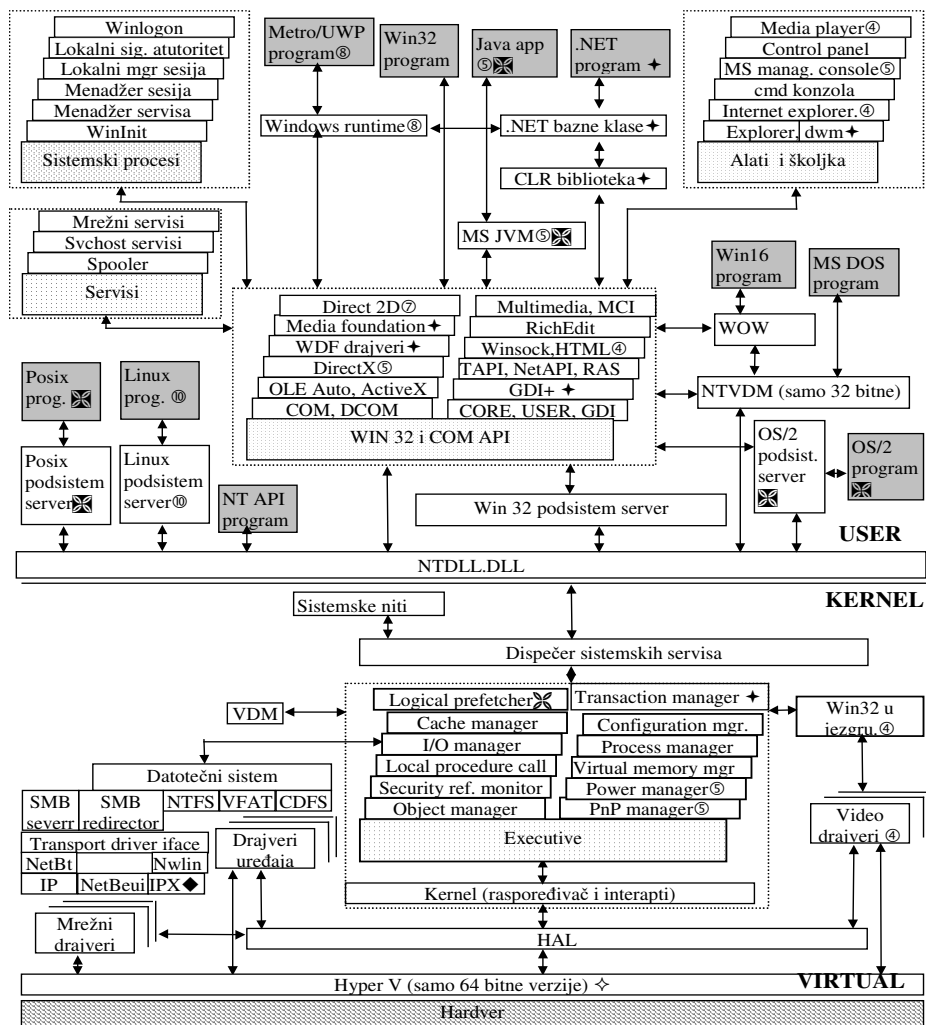
Windows 8 je verzija Windows-a s najdrastičnijom promjenom korisničkog sučelja još od Windows 95. Umjesto start menija uvedeno je novo okruženje prilagođeno tablet računarima. Novije aplikacije se sada prikazuju punom ekranu, umjesto rada u prozoru, za koje je rezerviran *Desktop* režim. Nakon Win32 i .NET API, uvodi se novi API zvani *Windows Runtime*, a glavni razvojni jezici postaju HTML 5 i JavaScript. Distribucija softvera se pomjera prema centralizovanom serverskom sistemu: *Windows Store*. Glavne izmjene u jezgru su učinjene s podizanjem sistema. *Secure boot*, sa javnim ključem koristi nove verzije BIOS-a i pazi na eventualne viruse prilikom podizanja sistema. Pri podizanju sistem koristi djelomičnu hibernaciju, radi ubrzanja podizanja. Dodjela tačne lokacije fizičke memorije aplikacijama je manje deterministička, u cilju smanjenja vjerovatnoće napada. Verzija se pokazala kao najveći promašaj, jer želja za identičnim korisničkim sučeljem za *desktop* i mobilne računare dovela je do vrlo neugodnog rada *desktop* korisnicima. U Windows 8.1 u *Desktop* režimu je vraćeno Start dugme, dodana podrška za 3D štampače i šifrovanje uređaja.

Windows 10, iz 2015. godine, omogućava da i *Metro* aplikacije po želji rade u prozoru ili punom ekranu, a Start meni je bliži klasičnom iz verzije 7, ali uključuje u polovini i start ekran iz Windows 8. Windows korisničko sučelje je izmijenjeno da omogući prijelaz između sučelja prilagođenog mišu i prilagođenog dodirnom ekranu na osnovu dostupnih ulaznih uređaja, što je posebno potrebno na 2-u-1 PC-u. Windows 10 i uvodi sistem virtualnih *desktop*-a, mogućnost upravljanja prozorima i *desktop*-om pod nazivom *Task View*, *Microsoft Edge web* preglednik kao zamjena za *Internet Explorer*, podršku za prepoznavanje otiska prsta i izgleda lica prilikom prijava, nove sigurnosne osobine za poslovna okruženja, i DirectX 12 i WDDM 2.0 za poboljšanje grafičkih mogućnosti operativnog sistema za igre. Postoji i verzija za *Raspberry Pi 2*. Iako će vrijeme pokazati da li je to istina, tvrdi se da je ovo posljednja verzija Windows-a koja ima ime. Nakon ovoga, Windows se ažurira isključivo preko

Interneta. Tipično dva puta godišnje nova ažuriranja donose nove aplikacije i mogućnosti sistema.

Windows NT jezgro se koristi u još nekim sistemima, manje poznatim. Igrače konzole XBOX-360 i XBOX imaju operativne sisteme sa jezgrom preuzetim od Windows 2000 odnosno Windows 8, ali drugačijim ostatkom operativnog sistema. *Windows Embedded*, predstavljaju skraćene i konfigurabilne verzije Windows-a za ugradnju u različite uređaje (npr. POS kase, bankomate...).

15.7.Windows NT – Arhitektura



Uvedeno od: ③NT 3.5 ④NT 4 ⑤2000 ⑥XP ⑦Vista ⑧7 ⑨8 ⑩10
 Ukinito od: ③NT 3.5 ④NT 4 ⑤2000 ⑥XP ⑦Vista ⑧7 ⑨8 ⑩10

Slika 186Arhitektura Windows sa NT jezgrom (32 bitna verzija)

Šematski prikaz arhitekture Windows NT dat je na slici Slika 186. Podsistemi Windows NT su podijeljeni u dva ili tri područja prema režimu privilegija: Režim jezgra (*Ring 0*), režim *Hyper V* virtualizatora (opcionalno prisutan, *Ring 1*) i korisnički režim (*Ring 3*). NT koristi hibridno jezgro koje je između čistog mikrokernels i monolitnog dizajna. Jezgro i podsistemi *Executive* kao što su *Process Manager* i *Virtual Memory Manager*, izvršavaju se u režimu privilegija jezgra i kompajlirani su u jednoj datoteci NTOSKRNL.EXE. Ovi podsistemi u režimu jezgra nisu odvojeni procesi i međusobno komuniciraju pozivom funkcija radi maksimalnih performansi.

Sistem je dizajniran da podrži različite API-je, tako da se jezgro može iskoristiti da po potrebi bude jezgro različitih vrsta operativnih sistema, npr. da bude nova verzija Windowsa, Unix-a, MS DOS-a ili OS/2. Svaki od ovih operativnih sistema ima svoje API-je, ali je nivo implementiranosti različit. DOS aplikacije su ograničene na one koje ne pristupaju direktno hardveru, POSIX i OS/2 imaju minimalan skup funkcija, dok se WIN32 stalno unapređuje i on je vremenom dominirao NT linijom. Sada će komponente Windows-NT biti opisane slojevito, od hardvera do aplikacijskih programa. Windows NT je uglavnom pisan u C i C++, uz vrlo male mašinski specifične dijelove u asemblerskom jeziku.

15.8.Hardver

Većina ranijih Microsoftovih operativnih sistema je vezana za računare iz familije IBM PC, uglavnom *desktop* tipa. MS DOS je bio dizajniran isključivo za 8086 procesor (uz mnogo kasnije dodane *driver*-e za DPMI koji su koristili 80386), Windows 3.1 je imao verzije za 80286 i 80386 procesor, a Windows 95 je bio isključivo za 80386.

Pošto je zamišljen i za servere, NT je dizajniran za podršku znatno široj klasi računara, iako se po broju podržanih platformi ne može porediti sa Linux-om. Sve verzije Windows NT rade na procesorima IA-32 arhitekture (80386, 486, razne verzije Pentium i Core procesora). Verzija NT 4 podržavala je i PowerPC, DEC Alpha i MIPS R4000 procesore, ali je podrška ukinuta sa Windows 2000 zbog slabe prodaje. Od Windows XP do Windows server 2003 podržan je Intel Itanium. Od Windows XP uvedena je podrška sa x64 procesore (AMD Athlon 64 i Intel Core u 64 bitnom režimu), a od Windows 8 podrška za ARM procesore.

Iako broj podržanih procesora nije veliki, postoji obilje periferijske opreme koju treba podržati jer je za najpopularniju računarsku platformu razvijen ogroman broj grafičkih kartica, štampača, mrežnih uređaja i kontrolera na matičnim pločama.

15.9.HAL i Hyper V virtualizator

U dizajnu jezgra donesena je jedna, za to vrijeme neobična odluka. Kako je OS zamišljen za različite platforme, uveden je još jedan sloj između hardvera i jezgra. HAL (*Hardare Abstraction Layer*) je NT-ov interfejs prema samom procesoru. Kako je NT dizajniran da bude prenosiv između različitih računarskih arhitektura. Dijelovi koda specifični za pojedini procesor pišu se u ovom modulu (datoteka HAL.DLL). HAL eksportuje standardni procesor i *driver*-i se pišu za njega. Čak i na pločama sa istim procesorom može biti razlika, recimo ako su jednoprocesorske i višeprocorske. Tako se prave različite verzije HAL.DLL-a, npr. jednoprocesorska, višeprocorska i debug verzija. Primjer funkcije koju HAL eksportuje prema jezgru

je `WRITE_PORT_UCHAR`, koja na periferni port šalje osmobarbitnu vrijednost. Prebacivanjem ove akcije u potprogram jezgro i *driver*-i su rasterećeni brige o tome da li je periferni uređaj memorijski ili U/I mapiran. Slično tome, funkcija `HalEnableInterrupt` za uključivanje prekida smanjuje potrebu pisanja dijelova jezgra u asemblerskom jeziku.

Od Windows server 2008 i Windows 8 može se instalirati i još niži sloj, *Hyper V* virtualizator (datoteka `hvx64.exe`), ali samo na x86-64 platformi. On se ponaša kao Hipervizor tipa 1, koji omogućava istovremeno izvršavanje više operativnih sistema sa maksimalnom brzinom.

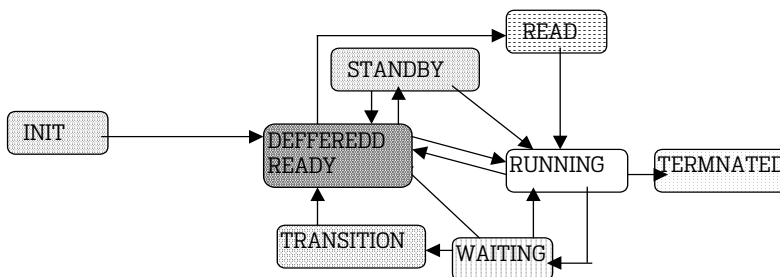
Intel-ovi procesori IA-32 i x64 poznaju četiri izolovana nivoa privilegija. Originalno su se koristila samo dva: nivo 0 za HAL, *Kernel*, *Executive* i *driver*-e i nivo 3 za korisničke procese i biblioteke. Ako je virtualizator uključen, on radi u nivou 1.

15.10.NT Kernel

Unutar datoteke `NTOSKRNL.EXE` nalaze se dva osnovna dijela, *Kernel* i *Executive*. *Kernel* radi bliže sa hardverom nego *Executive*. Obavlja dvije funkcije: raspoređivanje niti i procesa, te upravljanje prekidima. NT raspoređivač niti, koji se zove dispečer, nalazi se u *Kernel*-u. Dispečer implementira 32 nivoa prioriteta. Prioritet 0 je rezerviran za sistemsku nit koja izbacuje nekoristene memorijske stranice. Programi se izvršavaju u nivoima prioriteta 1 do 15, dok nivoima 16 do 31 mogu pristupiti samo administratori.

NT dispečer je preemptivni raspoređivač. Procesorsko vrijeme je podijeljeno u dijelove koji se zovu kvantumi. Kada nit dođe do kraja svog kvantuma i dispečer predaje zadatak niti istog prioriteta koji čeka.

U Windows 2000 niti mogu biti u stanjima *Init* (početno stanje), *Ready* (čeka na izvršenje), *Running* (izvršava se), *Standby* (odabrana za izvršenje na određenom procesoru, ali se još ne izvršava), *Waiting* (nit čeka na sinhronizaciju), *Transition* (spremna za izvršenje, ali *Memory manager* ju je izbacio iz memorije), *Terminated* (nit je završena i njen blok u *Executive* se može obrisati). U cilju manjeg vremena zaključavanja tabele procesa, u kasnijim verzijama uvedeno je još jedno stanje *Deferred ready* (nit je dodijeljena procesoru, ali još nije ubačena u raspoređivač). Dijagram stanja i prijelaza između niti dat je na sljedećoj slici.



Slika 187: Dijagram stanja procesa u Windows 10

Algoritam raspoređivanja niti je *Multi level feedback*. Za svaki od 32 nivoa prioriteta postoji po jedan red čekanja. Niti se izvršavaju u kvantumima koji su podrazumijevano 2 vremenska intervala za radne stanice, a 12 vremenskih intervala

za servere. Vremenski interval je do 10 ms za višeprocorske sisteme i do 15,6 ms za jednoprocesorske sisteme i mijenja se po potrebi. Kada proces prelazi u prvi plan (prozor izabran mišem) njegov kvantum se udvostručava. Kada kvantum istekne, izvršavanje se prepušta drugoj niti u istom prioritetu. Prioritet niti se dinamički mijenja nakon završetka U/I operacija, nakon čekanja na semafore, nakon što niti procesa u prvom planu završe *wait* operaciju, kada se GUI nit probudi zbog aktivnosti prozora i ako je neka od niti izgladnjela (duže od 4s nije izvršavana)..

Sinhronizaciju omogućavaju *mutex*-i, semafori, *event*-i, i *spinlock*-ovi. Objekte Kernel-a mapira Executive kroz *Object Manager* i *Process Manager*, i oni su takvi vidljivi aplikacijama.

Druga uloga Kernel-a je upravljanje prekidima. Prekidi su definirani osobinama procesora, tako da svaki procesor ima vlastitu tabelu prekidnih vektora (IDT, *Interrupt descriptor table*) koja pokazuje na odgovarajuću rutinu u jezgru.

15.11.NT Executive

NT Executive upravlja resursima u operativnom sistemu. Sastoji se od sljedećih dijelova: *Object Manager*, *Security Reference Monitor*, *Virtual Memory Manager*, *I/O Manager*, *Cache Manager*, *Local Procedure Call (LPC) Facility*, *Configuration Manager*, *Process Manager*, Win32 (od NT 4.0), *Plug-and-Play Manager* (od Windows 2000), *Power Manager* (od Windows 2000) i *Transaction Manager* (od Windows Vista).

Object Manager obavlja zadatke upravljanja objektima koji uključuju identifikaciju i brojanje referenci. Kada aplikacija otvori resurs, OM locira objekt ili kreira novi. Objekti se označavaju identifikatorima koji se zovu *handle*. Oni su jedinstveni na nivou aplikacije, ali ne i između aplikacija. Brojač referenci prati koliko je procesa pristupilo resursu. Identifikacija objekta se nalazi u NT-jevom prostoru za imena. Svi djeljivi resursi imaju imena, kao što su imena datoteka, registry ključeva ili semafora. Svi drugi podsistemi u *Executive* koriste *Object Manager* da definišu objekte i upravljaju objektima koji predstavljaju resurse. Na primjer, kroz *Object manager*, *Process manager* definiše objekt "proces" kojim prati aktivne procese

Objekti su *Adapter*, *Callback*, *Controller*, *DebugObject*, *Desktop*, *Device*, *Directory*, *Driver*, *Event*, *EventPair*, *File*, *IoCompletion*, *Job*, *Key*, *KeyedEvent*, *Mutant*, *Port*, *Process*, *Profile*, *Section*, *Semaphore*, *SymbolicLink*, *Thread*, *Timer*, *Token*, *Type*, *WaitablePort*, *WindowsStation*, *WMIGuid*.

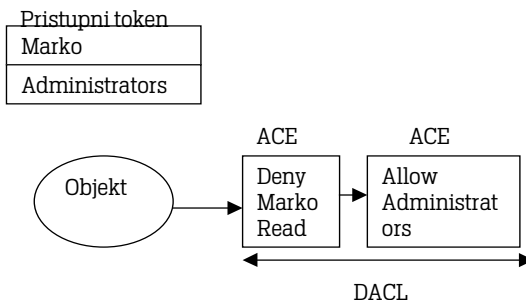
Tako npr. ako je Windows API funkcijom *CreateSemaphore* kreiran semafor imena "GLOBAL\TESTSEM", *Object Manager* će kreirati objekt TESTSEM, u svom internom direktoriju *BaseNamedObjects*. Bez prefiksa GLOBAL, objekt se kreira u privatnom imenskom prostoru. Aplikativni programi ne vide objekte preko imena, nego preko numeričkog identifikatora – *handle* (rezultat funkcije *CreateSemaphore*). Svaki objekt koji se evidentira u *Object Manageru* ima podatke o imenu, tipu, imenu direktorija OM, listi *handle*-ova, sigurnosnom deskriptoru i broju referenci.

Process Manager radi sa Kernelom da definiše objekte procesa i tredova. *Process Manager* analizira Kernelov objekt *Process* i dodaje mu identifikator procesa PID, pristupni token, adresnu mapu i tabelu *handle*-ova. Opis svakog procesa drži se u strukturi EPROCESS, koja pruža pogled na proces iz *Executive*. Ona ima istu funkciju

kao kontrolni blok procesa. U toj strukturi je smještena druga struktura KPROCESS, sa informacijama o procesu koje koristi Kernel. KPROCESS sadrži objekt tipa *Process*, listu KTHREAD struktura sa podacima o svakoj niti, podatke o baznom prioritetu, kvantumu i procesorima na kojima je preporučljivo izvršavanje. Ostatak EPROCESS strukture čuva identifikator procesa, limite za broj fizičkih stranica, informacije o adresnom prostoru, radnom skupu, virtualnoj memoriji, pristupnim *token*-ima, tabeli *handle*-ova objekata, procesnom bloku Windows podsistema itd.

Security Reference Monitor

je vezan s *Object Manager*-om. *Object Manager* poziva *Security Reference Monitor*, koji provjerava prava pristupa prije nego dopusti aplikaciji da otvori objekt. *Object manager* također zove *Security Reference Monitor* prije nego dopusti procesu da obavlja druge operacije nad objektima, kao što su čitanje i pisanje.



Slika 188 Pristupni token

SRM implementira sigurnosni model baziran na sigurnosnim identifikatorima (SID) i Diskrecionim listama za kontrolu pristupa (DACL). Svaki proces u NT-u ima pridruženi token pristupa koji sadrži SID korisnika, a koji posjeduje proces i SID-ove grupa kojima on pripada. Objekti koje *Object Manager* evidentira pokazuju na sigurnosni deskriptor, koji sadrži SID vlasnika objekta, njegovu primarnu grupu i pokazivač na diskrecionu listu. Ona se sastoji od elemenata kontrole pristupa (ACE) čijom kombinacijom se dolazi do zaključka o pravu nad objektom. Na slici Slika 188 je DACL koji ne dopušta vlasniku procesa Marko, da čita objekt, premda dopušta njegovoj grupi (Administrators).

Virtual Memory Manager transformiše virtualne adrese procesa u fizičke adrese i kontroliše alokaciju fizičke memorije. Na IA-32 Windows sistemima, virtualna memorija je velika 4G. Aplikacije mogu pristupiti prva 2G (*user mode* polovina, koja se mijenja kako se pokreću novi procesi). Druga polovina (2-4G) je Kernel polovina i ona se ne mijenja. Virtualni prostor procesa obično je podijeljen ovako:

Adresa (hex)	Značenje
00000000	Podaci koje može program koristiti
00400000	Mapirani izvršni program i globalni podaci
promjenjivo	Stack za programske niti
promjenjivo	Mapirane dinamičke djeljive biblioteke
Promjenjivo	Alocirani privatni podaci
80000000	Kernel, executive
Promjenjivo	HAL
Promjenjivo	Driver-i
C0000000	Tabela stranica procesa
C0400000	Tablice radnog skupa
C0800000	Prostor za sistemski <i>cache</i>
FFFFFFFF	Kraj virtualnog adresnog prostora

Serverske verzije na IA-32 se mogu pokrenuti i sa opcijom da se najviša adresa korisničkog procesa poveća na C0000000, što daje procesu virtualni prostor od 3 gigabajta.

Verzije za 64 bitne sisteme (IA-64 i x64) imaju daleko veće adresne prostore. Windows, međutim, ne dodjeljuje procesu teoretski maksimum adresnog prostora jer je on 16 eksabajta (oko 17 milijardi gigabajta), što je u ovom trenutku besmisleno veliko, a ni fizički 64 bitni procesori nemaju dovoljno adresnih linija za 64 bitnu adresu. Stoga je na x64 virtualni adresni prostor procesa organizovan tako da se procesu dodjeljuje dostupnih 8 terabajta virtualne memorije (do Windows 8) ili 128 terabajta (Windows 8.1 i Windows 10). Skraćeni prikaz adresnog područja procesa na x64 izgleda ovako.

Adresa (hex) Windows 8	Adresa (hex) Windows 10	Značenje
0000000000000000	0000000000000000	Područje dodijeljeno procesu
000007FFFFFFF	00007FFFFFFF	Kraj područja procesa
0000080000000000	0000800000000000	Prazno područje
FFFFFF0000000000	FFFFF00000000000	Početak sistemskog područja
FFFFFF6800000000	FFFFF68000000000	Tabela stranica procesa (4 nivoa)
FFFFFF7000000000	FFFFF70000000000	Podaci o radnom skupu
FFFFFF9000000000	FFFFF90000000000	Sesijsko područje
FFFFFF9800000000	FFFFF98000000000	Prostor za sistemski keš
FFFFFFA800000000	FFFFFA8000000000	Područje dodijeljeno jezgri
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	Kraj virtualnog adresnog prostora

Virtual Memory Manager implementira virtualnu memoriju koristeći straničenje. Na 32 bitnim arhitekturama, na IA-32 stranica je velika 4K, (na Alpha arhitekturi na Windows NT4 mogla je biti 8 K). Na IA-32 segmentacijski dio adrese je gotovo isključen, preciznije svi deskriptori segmenata pokazuju na istu početnu adresu, 0000000. Tako se virtualna adresa sastoji od tri dijela: broj tablice u imeniku (10 najviših bita), broj stranice u tablici (10 srednjih bita) i adresa unutar stranice (12 najnižih bita). Na x64 koristi se straničenje na četiri nivoa: najviših 16 bita virtualne adrese se ne koristi, biti na pozicijama 39-47 se zovu *Page map* (najviši nivo tabele straničenja), biti 30-38 *Page directory*, biti 21-29 se zovu *Page table*, a biti 12-20 *Page table entry*. Biti 0-11, kao i na IA-32 predstavljaju poziciju unutar stranice.

Novopokrenuti proces se mapira (njegov izvršni program na disku) u fizičku memoriju. Ubacivanje stranice u memoriju se obavlja na zahtjev (greška stranice, mapiranje novog procesa u memoriji), sa dodatkom elemenata ubacivanja sa predviđanjem: uključujući traženu stranicu ubacuje se susjednih 3 stranice (podaci) ili 7 stranica (programski kod). Ako procesi zahtijevaju više memorije nego što je fizički ima, koristi se zamjenska datoteka za virtualnu memoriju. Može ih biti i više na različitim diskovima. Kada je nedovoljno fizičke memorije za nove zahtjeve procesa za memorijom koristi se kombinacija lokalne i globalne strategije za izbacivanje stranice: model radnog skupa. Najprije se izbacuju stranice dodijeljene procesu dok se ne smanji radni skup na predviđeni minimum, a zatim se smanjuje radni skup i drugih procesa. Algoritam za izbor stranice za brisanje je **NFU sa starenjem u radnom skupu** tog procesa ili drugih procesa..

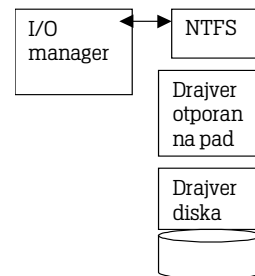
Ako procesu ili driveru treba alocirati malu količinu memorije, to se obavlja alokacijom na *heap*-u, kojim također može upravljati VMM. Druge mogućnosti VMM su memorijsko mapiranje datoteka, dijeljenje memorije i zaštita pri pisanju strane.

Logical prefetcher: Veći pomak od straničenja na zahtjev prema straničenju sa predviđanjem učinjen je od Windows XP. Tada je dodana mogućnost *Prefetch* radi bržeg startanja programa: u direktoriju *C:\windows\prefetch* čuva se spisak biblioteka koje program zahtijeva i logičkih stranica koje su korištene u prvih 10 sekundi izvršavanja. Naredna pokretanja programa će biti daleko brža, jer će se iskoristiti ove informacije.

I/O manager integriše dodatne drajvere za uređaje sa NT-om. Drajver uređaja prevodi komande koje NT i aplikacije šalju uređaju i proslijeđuju zahtjeve hardveru. Ako Microsoft nije isporučio drajver, proizvođač hardvera jeste.

I/O manager podržava asinhroni paketni I/O prijenos. Npr., ako MS Word čita iz datoteke, servis *NtReadFile* alocira I/O request packet (IRP) koji ima informacije o adresi gdje se smještaju podaci, objektu koji predstavlja datoteku, poziciji u datoteci, količini podataka itd. IRP se zatim proslijedi driver-u datotečnog sistema. Kada drajveri prime IRP oni rade svoje aktivnosti, a aplikacija nastavlja sa izvršenjem. Pošto to može biti teško za programiranje, standardni Win32 API ubacuje potrebna čekanja, ali je moguće pozivati i napredne funkcije koje koriste ovu asinhronost.

I/O manager podržava 64 bitne pozicije u datotekama i višeslojne drajvere uređaja. Npr., NTFS drajver je iznad *fault tolerant* driver-a koji je iznad standardnog driver-a (Slika 189).



Slika 189 I/O manager i drajveri

Cache manager sarađuje sa VMM i drajverima za datotečni sistem. *Cache manager* održava NT globalni (dijeljen od svih datotečnih sistema) keš za datoteke. NT keš je datotečno orijentiran (za razliku od Windows 95 koji je blokovo orijentiran). Područje predviđeno za *cache* datoteka alocira se u virtualnom području procesa, tako da ako nema dovoljno fizičke memorije *cache* će se smanjivati postojećim algoritmima straničenja. Virtualno područje predviđeno za *cache* dijeli se na poglede: svaki je velik 256 kilobajta i sadrži komad datoteke te veličine koji je čitan ili pisan. *Cache* se generalno dodjeljuje po FIFO principu. Na primjer, ako je najprije čitano prvih 50 K datoteke A.txt, pa 500 K datoteke B.txt i zatim 30 K datoteke C.txt, tada će datoteka A.txt će zauzeti pogled 0, datoteka B.txt će zauzeti poglede 1 i 2, a datoteka C.txt zauzeti pogled 2.

Fizičku memoriju kešu dodjeljuje *Working-set tuner*. Kada on izbacuje memoriju iz prostora predviđenog za *cache*, datotečni sistem snima datoteku na disk.

Power manager (od Windows 2000) upravlja promjenama statusa napajanja za sve uređaje koji podržavaju te promjene. To se često radi kroz složeni niz uređaja za kontrolu drugih uređaja. Drajveri treba da reaguju na sljedeće informacije: Nivo aktivnosti sistema, nivo baterije, trenutni zahtjevi na zatvaranje, spavanje ili

hibernacije, korisničke akcije kao što je pritisak na dugme napajanja, postavke *Control panel*, kao što je automatsko gašenje na 10 posto baterije.

Plug and Play manager (od Windows 2000) određuje koji su drajveri potrebni za podršku odgovarajućem uređaju i učitava te drajvere. Preuzima zahtjeve svakog uređaja za resursima (kao što su I/O portovi, DMA kanali, prekidne IRQ linije i memorijske lokacije) tokom njihove prozivke.

Configuration Manager upravlja *Registry*-jem. *Registry* je velika konfiguracijska baza sistema. Organizovan je hijerarhijski kroz stablo. Grane ovog stabla se zovu ključevi (poput direktorija u datotečnom sistemu), a listovi se zovu vrijednosti (poput datoteka, ali sadrže samo jedan podatak). Postavke vezane za trenutnog korisnika čuvaju se u ključu HKEY_CURRENT_USER, postavke koje koriste svi korisnici u ključu HKEY_USERS, postavke vezane za prepoznavanje COM objekata u ključu HKEY_CLASSES_ROOT, postavke vezane za sistem u ključu HKEY_LOCAL_MACHINE, informacije o performansama u ključu HKEY_PERFORMANCE_DATA, a postavke o trenutnom hardverskom profilu u ključu HKEY_CURRENT_CONFIG.

Na primjer, ako se otvori ključ HKEY_CURRENT_USER\Control Panel\Colors i u njemu promijeni vrijednost Background na 0 0 200, pozadina ekrana postaje plava, nakon što se korisnik sljedeći put prijavi.

Poziv lokalne procedure (LPC) je komunikacijski mehanizam između procesa. On nije dostupan kroz Windows API, nego se koristi isključivo za komunikaciju između komponenti Windows operativnog sistema. Primjer aplikacija koje ga koriste su LSASS i CSRSS. Postoje 3 tipa LPC, kopiranje podataka iz procesa u proces (za poruke do 256 bajtova), dijeljena memorija i dijeljena memorija sa parovima.

Transakcijski menadžer (od Windows Vista) implementira transakciju u režimu jezgra. Pri obavljanju više operacija nad podacima, nekad je potrebno da se sve izvrše u potpunosti ili uopće ne izvrše. Ako dođe do greške, transakcija se može vratiti tako da se podaci vraćaju u prvobitno stanje.

15.12. Dispečer sistemskih servisa

Spomenuti podsistemi *Executive* pružaju odgovarajuće potprograme koji se mogu pozvati iz drajvera i drugih programa u režimu jezgra. Neki od ovih potprograma su ekportovani prema korisničkom režimu rada. Na primjer iz drajvera se može pozvati potprogram MmAllocateContiguousMemory koji će pozvati *Virtual memory manager* da dodijeli kontinualni blok memorije navedene veličine koji neće biti izbacivan zbog straničenja. Više oko 2600 ovih potprograma je dostupno iz korisničkog režima.

Prelazak iz korisnički režim u sistemski režim rada na procesorima 80486 i Pentium 1 obavlja se instrukcijom INT 2C čime se prelazi na posebnu rutinu za obradu prekida. Registar EAX prije poziva ove instrukcije treba da sadrži redni broj sistemskog poziva, a registar EBX pokazuje na njegove parametre. Od procesora Pentium 2 uvedena je brža instrukcija SYSENTER sa istim parametrima. Dispečer sistemskih servisa prema rednom broju sistemskog poziva pogleda u tabeli adresu gdje se u Kernelu ili Executive nalazi potprogram koji implementira i pozove poziv.

15.13.Drajveri

Pored HAL, Kernel i Executive, u režimu jezgra se takođe nalaze *driver*-i. Svaki *driver* je u posebnoj datoteci, obično ekstenzije *.sys*. *Driver*-i se dijele u sljedeće kategorije.

- *Driver*-i datotečnog sistema primaju ulazno/izlazne zahtjeve za datotekama i samo šalju zahtjeve prema *driver*-ima memorije ili mreže.
- *Plug and Play driver*-i rade sa hardverom i integrišu se sa *Power manager*-om i *PnP manager*-om. Oni uključuju *driver*-e za diskove, video adaptere, ulazne uređaje i mrežne adaptere.
- *Non-PnP driver*-i su takođe ekstenzije jezgra, ali nemaju interakciju sa *PnP* i *Power manager*-om. Primjeri ovih drajvera su drajveri za mrežne protokole.

WDM drajveri se pišu u skladu sa *Windows Driver Model* (WDM). Specifikacijom. Tri tipa WDM drajvera, prema funkcijama su:

- *Driver*-i sabirnice su odgovorni za informisanje PnP menadžera o uređajima zakačenim za sabirnicu,
- Funkcionalni *driver*-i: obavljaju stvarne akcije nad uređajem i slanje na odgovarajuće U/I portove uređaja.
- Filter *driver*-i: mijenjaju ponašanje *driver*-a (podatke koji se šalju uređaju).

WDM *driver*-i se sastoje od sljedećih dijelova.

- Inicijalizacijska rutina, se poziva kada *I/O manager* inicijalizira drajver
- Rutina za dodavanje uređaja se poziva kada *PnP manager* pošalje poruku *driver*-u da je detektovan uređaj za koji je *driver* odgovoran.
- Rutine za prosljeđivanje zahtjeva uređaju, realizuju operacije otvaranja, zatvaranja, čitanja, pisanja uređaja itd. Za ove rutine I/O menadžer generiše paket U/I zahtjeva (IRP) i šalje ih *driver*-u.
- Rutina za start U/I operacija iniciraju prijenos podataka od uređaja ili prema uređaju.
- Prekidna rutina se izvršava kad uređaj generiše prekid.
- Odložena prekidna rutina obavlja aktivnosti na prekide uređaja koje se kasnije izvršavaju, uz dopuštenje da drugi uređaji generišu nove prekide.
- Rutina koja se poziva kada je U/I operacija završena.
- Rutina koja se poziva kada je U/I operacija nasilno prekinuta
- Rutina koja se poziva pri gašenju sistema

Drajveri se pišu u slojevima. *Class* driveri implementiraju zajedničke funkcije za cijelu klasu uređaja, npr. za sve IDE diskove. *Miniclass* driveri se učitavaju kao njihov dodatak i sadrže specifičnosti pojedinog uređaja. Slično ovome, *port* driveri implementiraju zajedničke funkcionalnosti U/I priključka – npr SCSI, a *miniport* drajveri realiziraju specifične dijelove. Na primjer, raspređivanje zahtjeva prema disku koristi C-LOOK algoritam i realizovano je u *driver*-ima *scsiport.sys* i *ataport.sys*

Datotečni sistemi nisu u kernelu ni executive, nego su realizovani kao poseban *driver*. U Windows NT koriste se FAT, NTFS, ISO 9660, exFAT i novi ReFS.

15.14.WIN 32 u kernel dijelu

Poseban *driver* win32.sys ubačen je od Windows NT 4.0. Njegova svrha je ubrzanje nekih grafičkih funkcija koje pruža Win32 podsistem (USER32.DLL, GDI32.DLL i dio DirectX). Do tada je ovaj podsistem u potpunosti bio u korisničkom području..

15.15.Prirodni API

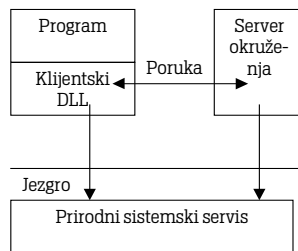
Sada slijedi dio Windows NT sistema koji se izvršava sa privilegijama korisničkih procesa. Sistemski pozivi koje prima dispečer sistemskih servisa i dalje prosljede ka Executive treba da budu realizovani i sa druge strane, iz korisničkih procesa. Prije svega, svakom od tih sistemskih poziva treba poslati parametre u formatu kakav očekuju programi u jeziku C i omogućiti njihovo pozivanje iz aplikacija. Ovi potprogrami se zovu NT-jev prirodni API, smješten u datoteci NTDLL.DLL. Ovaj API ima oko 2500 funkcija koji se pozivaju koristeći softverske izuzetke. Softverski izuzetak je hardverski potpomognuti način da se prelazi između korisničkog režima rada i režima jezgra. Sistemski servisi prirodnog API-ja su napravljeni da ih dijelovi Windows NT-a u korisničkom režimu rada mogu koristiti. Prirodni API mogu zvati i aplikacije, ali neće dobiti ništa više što ne bi mogle dobiti Win32 API-jem. Imena sistemskih servisa počinju sa Nt. Na primjer Win32 funkcija *CreateProces* poziva *ntCreateProces*, a ovaj poziva *psCreateProcess* iz *Process Manager*-a.

Vrlo malo aplikacija koristi isključivo prirodni API, a rijetke su one koje ga koriste i u kombinaciji sa drugim API. On je uveden da se razlikuje od API poziva drugih operativnih sistema, kao što su Unix, OS/2 ili Windows 95, ali da se ti API pozivi mogu mapirati na prirodni API. Na taj način se mogu prenositi aplikacije namijenjene za različite operativne sisteme, uz zanemarljiv gubitak na brzini izvršavanja.

15.16.Serveri okruženja

Skup API funkcija koje su nastale od nekog operativnog sistema, a koji se veže na prirodni API da bi pozvao usluge jezgra zove se okruženje. Do verzije Windows 2000 bila su tri takva okruženja Win32, POSIX i OS/2. Zbog male količine softvera razvijenog pod njima, okruženja OS/2 i POSIX su od Windows XP ukinuta. Ali od Windows 10 (*build* juni 2017) uveden je novi *Windows Subsystem for Linux*, koji omogućava pokretanje Linux programa pod Windows-om, pod uslovom da su kompajlirani za isti procesor. Pri pokretanju pojedinog programa iz njegovog izvršnog programa se pročita koje okruženje zahtijeva i pokrene odgovarajući server okruženja, ako već nije pokrenut, pored samog programa. Okruženja operativnih sistema su implementirana kao klijent/server.

Sve aplikacije su povezane sa nekim dinamičkim bibliotekama koje eksportuju API. Biblioteke koje se vežu sa aplikacijom se zovu klijentske biblioteke, dok se program koji je pokrenut kao ispomoć zove server okruženja. Na primjer program NOTEPAD.EXE je klijent okruženja operativnog sistema Win32. Stoga se on povezuje sa klijentskim



Slika 190 Klijent server model u jezgri

DLL-ovima, Kernel32.dll, gdi32.dll, i user32.dll, dok je server program CSRSS.EXE. POSIX program cp.exe se veže za psxdll.dll, a serverski program je POSIX.EXE. Linux programi mogu biti i u svom izvršnom formatu (ELF) a serverski program je LXRUN.EXE.

Klijentski DLL komunicira sa serverom koristeći LPC mehanizme jezgra, sa ranije uspostavljenom dijeljenom memorijom.

U nekim slučajevima klijentski DLL može u potpunosti implementirati API bez poziva servera (u sebi samom ili preko NTDLL.DLL), dok u drugim slučajevima server mora pomoći. Serverska pomoć je potrebna obično samo kada se trebaju ažurirati globalne informacije o sistemu. Na primjer, za *CreateProcess* funkciju, mora se pozivati server, dok za čitanje datoteke ne mora (Slika 190).

Sesijski menadžer (SMSS.EXE) je prvi proces koji se starta nakon inicijalizacije jezgra. On kreira varijable okruženja, napravi datoteku za virtualnu memoriju i pokrene serverske programe za podsisteme za koje je rečeno u *registry* da su potrebni. To je obično win32 podsistem i njegov server. Nakon toga pokreće program **wininit** za inicijalizaciju ovog podsistema.

15.17.Server Win32

Win32 okruženje je najvažnije i dalje će biti riječi samo o njemu. Server ovog okruženja je CSRSS.EXE, koji koristi samo prirodni API. Biblioteke Win32 (kernel32.dll, user32.dll, gdi32.dll) komuniciraju sa njim u sljedećim situacijama: funkcije koje rade sa konzolnim prozorom (tekstualne), kreiranje procesa i niti i zahtjev za specifičnom verzijom DLL biblioteka ako ih ima više sa istim imenom

15.18.Win32 i Win64 bazni API

Aplikativni programi pozivaju potprograme koji su implementirani u dinamičkim bibliotekama. U tipičnom Windows sistemu instalirano je više stotina ovakvih biblioteka, a svaka od njih pruža desetine pa i stotine funkcija.

Win32 programi mogu da se izvršavaju na različitim operativnim sistemima, tako postoje verzije biblioteka za Windows 9x, Windows 3.1 (uz dodatak Win32s), Windows NT ili WINE (pod Linux-om). Za svaki od ovih operativnih sistema biblioteke se realizuju na različit način, ali bi u principu program koji poziva funkcije iz ovih biblioteka trebao da radi ako su te funkcije propisno realizovane za svaku verziju biblioteke za pojedini operativni sistem. Bazne biblioteke (npr. kernel32.dll) za Windows NT pozivaju jezgro preko NTDLL.DLL ili servera CSRSS.EXE. Pokušaj ubacivanja takve biblioteke u Windows 98 bi doveo do kraha, ali neke biblioteke su platformski manje ovisne, pa je dovoljna jedna verzija.

Funkcije koje se pozivaju iz Windows programa prilagođene su jeziku C i drugim proceduralnim jezicima. Kada se pokrene proces, unutar njegove izvršne datoteke EXE zapisana je lista dinamičkih biblioteka koje se pozivaju i funkcija unutar njih. Poziv funkcije se obavlja kao poziv običnog potprograma (dakle nije sistemski poziv, ali neke funkcije mogu da pozovu rutinu nižeg nivoa koja će pozvati jezgro). U 32 bitnim verzijama parametri funkcija se smještaju na *stack* prije poziva, a sama funkcija ih nakon završetka obriše sa steka. Parametri su najčešće 32 bitne vrijednosti, čak i kada po prirodi podatka zauzimaju manje prostora (npr. logičke vrijednosti).

Najvažnije biblioteke baznog Windows API date su u tabeli.

Biblioteka	Namjena
ADVAPI32.DLL	Pozivi vezani za sigurnost i pristup registry-ju
CFGMR32.DLL	Instalacija drajvera
IMAGEHLP.DLL	Analiza izvršnih programa
KERNEL32.DLL	Pristup datotekama, procesima, sinhronizacijskim objektima
MSVCRT.DLL	Osnovna biblioteka za programe u jeziku C
SFC.DLL	Zaštita sistemskih datoteka
COMCTL32.DLL	Elementi korisničkog sučelja, npr. pogled stabla i trake progres
COMDLG32.DLL	Standardni dijaloški prozori za izbor datoteka, štampača i boja
RICHED20.DLL	Komponenta editor teksta sa fontovima i slikama
SHELL32.DLL	Pristup elementima školjke Explorer
USER32.DLL	Komunikacija i crtanje osnovnih komponenti korisničkog sučelja
WTSAPI32.DLL	Pristup udaljenoj radnoj površini
GDI32.DLL	Crtanje likova na ekranu i štampaču
MSIMG32.DLL	Manipulacija bit mapiranim slikama
OPENGL32.DLL	Crtanje trodimenzionalnih objekata
AVIFIL32.DLL	Upravljanje AVI datotekama sa filmovima
MSVFW32.DLL	Obrada video podataka
WINMM.DLL	Pristup multimedijalnim uređajima
CABINET.DLL	Kompresija i dekompresija u arhivama tipa CAB
CRYPT32.DLL	Kriptografske funkcije
LZ32.DLL	Kompresija i dekompresija datoteka
IMM32.DLL	Metode unosa podataka za azijske jezike
USP10.DLL	Detalji o alfabetima za nacionalne jezike
MAPI32.DLL	Slanje i prijem elektronske pošte
NETAPI32.DLL	Komunikacija u Windows mrežama
RASMAN.DLL	Dial-up konekcija
URL.DLL	Čitanje elemenata URL adresa
WININET.DLL	HTTP, FTP i Gopher protokol
WSOCK32.DLL	Mrežna komunikacija preko socket-a
ODBC32.DLL	Pristup serverima za baze podataka koristeći SQL
TAPI32.DLL	Komunikacija sa telefonskim uređajima

Od Windows 7 neke od ovih biblioteka su razbijene na više manjih biblioteka, ali i dalje pružaju standardnu funkcionalnost.

Na x64 platformama, *Core API* je otprilike jednak, kao na IA-32, ali je prosljeđivanje parametara drugačije. Prva četiri parametra potprograma se prosljeđuju u registrima RCX, RDX, R8 i R9, a ostali preko steka.

15.19.OLE, COM/DCOM, ActiveX

U sistemima sa grafikom i prozorima, kao što je Windows, pojavila se potreba za većom komunikacijom između procesa, od one koju pružaju klasični metodi poput poruka, dijeljene memorije i cijevi. U poslovnim primjenama nije rijedak scenarij da korisnik piše tekst i unutar njega ubacuje tablične proračune i slike, kao i podatke iz baze podataka, a na kraju doda u memorandum adrese primaoca elektronske pošte. Sada je dilema da li razviti jedan program koji radi i obradu teksta i može računati sa tablicama i crtati slike i slati elektronsku poštu, ili uzeti specijalizovane

programe za to. Jedinstveni program bi po mogućnostima svake od ovih funkcionalnosti zaostajao za specijalizovanim, a nepotrebno zauzimao ogromnu memoriju i bio preopterećen opcijama za one korisnike kojima treba samo obrada teksta. S druge strane, upotreba specijalizovanih programa bi tražila stalne prelaske između programa, snimanje različitih datoteka, konverziju formata svaki put kada se nešto promijeni. Kako kombinovati ova dva pristupa? Rješenje je u načinu pisanja svakog specijalizovanog programa, tako da on pruža više ulaznih tačaka, pa kada se želi ubaciti u tekst tablica, tada program za obradu teksta poziva neki od potprograma programa za rad sa tablicama i omogućava kvalitetan način uređivanja tablice uz automatsko kopiranje rezultata u program za obradu teksta.

Osnovni Win32 API, dizajniran za programe u jeziku C nije dovoljan za takve aplikacije, pa je uveden pristup komponenti. Komponenta je skup srodnih funkcija spojenih u zajedničku datoteku (DLL ili EXE) koje se mogu pozivati i ugrađivati u druge programe. Za razliku od objekata u C++ ove komponente se ne opisuju pravilima programskog jezika, nego opisom binarnog formata u koji su one prevedene, ali se mogu nalaziti u adresnim prostorima drugih procesa, pa čak i drugim računarima, a pri tome biti pisane u različitim programskim jezicima.

Već od Windows NT 3.1 definisan je standard za takve komponente koji se zove *Component Object Model* (**COM**). Za dobro napisane komponente, COM dozvoljava ponovnu upotrebu objekata bez znanja o njihovoj unutrašnjoj realizaciji. Svaki COM objekt sadrži tablicu pokazivača na potprograme (metode) koji se u toj tablici nalaze u strogo određenom redoslijedu. Redoslijed ovisi od namjene COM objekta i zajedno sa potrebnim parametrima čini njegov interfejs. Svi COM objekti imaju tri metode: *QueryInterface*, *AddRef* i *Release*, nakon kojih se u toj tabeli nalaze ostale metode zavisno od interfejsa. Prije nego ih program koji koristi COM objekta smije pozvati, pozove se *QueryInterface* sa parametrom koji predstavlja ogroman broj GUID, dizajniran da bude jedinstven pri svakom pokretanju generatora ovih brojeva. COM objekt u metodi *QueryInterface* provjeri prosljeđeni broj i ako je jednak onom koji poznaje, vrati pokazivač na tabelu metoda, dopustivši dalje pozivanje ostalih metoda. *AddRef* povećava jedan brojač svaki put kada se komponenta inicijalizira iz drugih programa, dok *Release* umanjuje taj brojač i kada je on 0 oslobađa memoriju koju je komponenta zauzela.

Distribuirani COM (DCOM) proširuje COM programski model tako da radi preko mreže. Jedan od njegovih zadataka je pakovanje parametara za njihovo prosljeđivanje kroz mrežu. Struktura COM objekata iskorištena je za niz različitih vrsta komponenti, koje su obuhvaćene zajedničkim imenom ActiveX.

OLE (*Object linking and embedding*) Aplikacije koje koriste OLE upravljaju složenim dokumentima - to su dokumenti sastavljeni od podataka iz različitih aplikacija. OLE pruža usluge koje olakšavaju aplikacijama da pozivaju druge aplikacije za uređivanje podataka. Na primjer, procesor riječi koji koristi OLE može ugrađivati dijagram iz tabličnog kalkulatora. OLE vodi računa o pokretanju tabličnog kalkulatora i prezentaciji grafikona za uređivanje. Kada korisnik napusti tabelu, dijagram bi se ažurirao u originalnom dokumentu za obradu teksta. Tako izgleda da je tabelarni proširenje procesora za obradu teksta.

OLE automatizacija je namijenjena pozivanju dijelova aplikacije programski. Na primjer, svaka opcija koju pruža Microsoft Word može se pozvati iz eksternog programa napisanog u C++ ili Visual BASICu. Tako postaje moguće iz programa kreirati Word dokumente, upisivati tekst u njih i formatirati ga bez korisničkog pokretanja Word-a i diranja tastature ili miša.

ActiveX kontrole su male komponente (programska ekstenzija je obično .OCX) koje proširuju mogućnosti aplikacija. Primjeri su dodatne komponente u Visual BASIC-u za prikaz kalendara koje postaju po načinu upotrebe skoro ravnopravne osnovnim komponentama koje su već ugrađene u jezik. Ove kontrole se ugrađuju i u Internet Explorer radi povećanja njegovih mogućnosti (npr. otvaranje PDF dokumenata) ili izvršavanje aplikacija unutar web stranice.

15.20. DirectX

Dok su ostale vrste aplikacija već naveliko razvijane za 16-bitne i 32-bitne Windows, igre su bile zadnja kategorija softvera koja je razvijana u MS DOS-u, jer su zbog brzine prikaza zahtijevale direktan pristup video memoriji i zvučnim uređajima. Zato je najprije za Windows 95 razvijen skup drajvera i API za brži pristup grafici, a od NT 4.0, ovaj skup drajvera i API poziva uključen je i u NT seriju. *DirectX* funkcije su također upakovane u COM objekte. Prvi među *DirectX* interfejsima bili su *DirectDraw* (koji je omogućavao crtanje upisom vrijednosti u zonu memorije kao što se radilo u MS DOS igrama, ali sa specijalnim driver-om kojim se postizala neovisnost od fizičke organizacije video memorije), *Direct3D* (za iscrtavanje trodimenzionalne grafike koristeći hardversku akceleraciju), *DirectSound* (za smplovani zvuk), *DirectMusic* (za muziku po principu nota), *DirectPlay* (za mrežnu komunikaciju u igrama). Kako su grafičke kartice unapređivane i uvedena grafička konzola Xbox, razvijane su nove verzije *DirectX*, pa je *DirectDraw* zamijenjen za *Direct2D* i *DirectWrite* (ispis teksta), *DirectSound* sa *XAudio2*, a *DirectMusic* sa *XACT3*. Sa interfejsom *DirectCompute* grafičke kartice se mogu koristiti i za generalni račun.

15.21. Sistemski procesi

Sesijski menadžer je pokrenuo Wininit. **Wininit** pripremi privremene direktorije, varijable okruženja, postavke u *registry* i pokreće dalje procese.

Services će pogledati koje Windows servise treba pokrenuti i redom ih inicijalizirati. **Winlogon** prikazuje prozorčić za prijavu korisnika na sistem. **LSASS** (*Local Security Authority Subsystem Service*) će provjeriti da li se korisnik uspješno prijavio i da li je lozinka koju je unio u skladu sa onom koja je evidentirana lokalno ili na serveru aktivnog direktorija i ako jeste dodjeljuje kreira pristupni token za tog korisnika. **Userinit** dovršava inicijalizaciju specifičnu za korisnika i pokreće školjku **Explorer**, nakon čega završava sa radom.

15.22. Servisi

Servisi su pozadinski programi koji nemaju interakciju sa korisnikom, nego isključivo sa datotekama, mrežom ili *Registry*-jem. Proces *services.exe* pročita u *registry* koji servisi trebaju biti pokrenuti i aktivira ih kao niti unutar procesa *svchost.exe*, uz neke izuzetke poput *printer spooler* servisa koji se pokreće u procesu

spoolsv.exe. Servisi rade i kada nijedan korisnik nije prijavljen na sistem. Mogu se pokrenuti automatski ili ručno. Neki važniji servisi dati su u tabeli.

Ime servisa	Opis
Active Directory Service	Omogućava mrežnu autentifikaciju korisnika
Computer Browser	Pronalazi računare na mreži koji imaju dijeljene diskove
Delivery Optimization	Ubrzava Windows update preko računara u lokalnoj mreži
DNS Client	Pretvara domenska imena kao en.wikipedia.org u IP adrese
Event Log	Čita i upisuje događaje u event log
Indexing Service	Indeksira datoteke za bržu pretragu
Internet Connection Sharing	Internet pristup drugih računara ako je jedan povezan
Plug and Play	Automatska detekcija i konfiguracija hardvera
Windows Print spooler	Upravlja štampačima i šalje datoteke na štampanje
Remote Procedure Call	Omogućava poziv potprograma na drugim računarima
Routing and Remote Access	Pretvara računar u mrežni router
Secondary Logon	Omogućava pokretanje aplikacija pod drugim korisnikom
Server	Čini računar vidljivim na mreži za dijeljene diskove
Superfetch	Omogućava učitavanje datoteka sa predviđanjem
Task Scheduler	Raspoređuje programe za start u određeno vrijeme
Windows Error Reporting	Javlja razvoju operativnog sistema o krahovima aplikacija
Windows Firewall	Blokira neodobrene konekcije prema računar u od njega
Windows Image Acquisition	Podrška za skenere i kamere
Windows Time	Sinhonizacija vremena za eksternim
Windows Update	Ažurira operativni sistem

15.23.NTVDM i WOW

Jedan od ciljeva Windows NT je da izvršava i veliki broj ranijih programa za MS DOS i Windows 3.1. NTVDM je specijalni program koji se pokreće kada se pokrene program namijenjen za ove operativne sisteme. Procesor se tada prebaci u specijalni Virtual 86 režim u kome se koriste segmentni registri kao na 8086 procesoru. Uz program se u njegov virtualni prostor učitava modifikovana verzija MSDOS-a 5.0 u posebnim datotekama NTIO.SYS, NTDOS.SYS i COMMAND.COM. Na svim mjestima u ovako izmijenjenom MS DOS-u koja direktno pristupaju disku ubačene su nelegalne instrukcije koje izazivaju skok u NTVDM koji dalje poziva odgovarajuće rutine za simuliranje željenog ponašanja. Od svih verzija Windows NT, ovaj podsistem je najbolje implementiran u Windows XP, jer u toj verziji rade grafičke aplikacije i simulira se zvučna kartica. Kasnije verzije simuliraju izvršavanje isključivo tekstualnih aplikacija.

NTVDM se pokreće i kada se želi na 32 bitnoj verziji Windows-a pokrenuti 16 bitni Windows program, npr. Word 2.0. Uz njega se aktivira Windows on Windows (WoW). Podsistem WOW operativnog sistema preusmjerava stare 16-bitne pozive u nove 32-bitne ekvivalente kako bi pružili podršku za 16-bitne pokazivače, modele memorije i adresni prostor. Isporučene su 16 bitne verzije biblioteka i OLE objekata. Svi 16-bitni programi pokreću podrazumijevano u jednoj virtualnoj DOS mašini sa dijeljenim memorijskim prostorom. Međutim, oni se mogu konfigurirati za pokretanje u svom

odvojenom memorijskom prostoru, u kom slučaju svaki 16-bitni proces ima vlastitu virtualnu mašinu.

NTVDM i WoW ne postoje u 64 bitnoj verziji Windows, jer procesor nema u 64 bitnom režimu podršku za 16 bitne programe. No, ovdje je potrebno naći način pokretanja 32 bitnih programa, kojih je daleko više napisano nego 64 bitnih. Podsistem WoW64 to omogućava. On sadrži sloj komponenti za kompatibilnost koji ima slične interfejsa na svim 64-bitnim verzijama Windowsa. Njegov cilj je da stvori 32-bitno okruženje potrebno za pokretanje neizmijenjenih 32-bitnih Windows aplikacija na 64-bitnom sistemu. WOW64 se implementira pomoću biblioteka:

Wow64.dll: osnovni interfejs za jezgru Windows NT koji prevodi između 32-bitnih i 64-bitnih poziva, uključujući pointere i parametre na steku; Wow64win.dll, koji pruža odgovarajuće ulazne tačke za 32-bitne aplikacije i Wow64cpu.dll, koji vodi računa o prebacivanju procesora sa 32-bitnog na 64-bitni režim. Uz Wow64 isporučuju se dinamičke biblioteke koje se instaliraju na 32 bitnim Windows.

15.24.Školjka

Nakon prijave, korisnik radi sa podrazumijevanim softverom za pokretanje aplikacija. U prvim verzijama to je bio Program manager, a od NT 4.0 Windows Explorer. Windows Explorer ima dvije osnovne funkcije: Pokretanje programa menijem ili kraticama na desktopu i upravljanje datotekama (dodavanje, kopiranje, premještanje, brisanje i otvaranje specifičnom aplikacijom. U verzijama NT 4.0, 2000, XP, 7 i 10 ekran je podijeljen na radnu površinu koja zauzima većinu ekrana na kome se slažu prozori i alatnu traku koja sadrži startno dugme za pokretanje menija, listu trenutno pokrenutih aplikacija i kratice za pokretanje malih programa. Verzije 8 i 8.1 umjesto startnog menija otvaraju odvojeni startni ekran. Školjka je sastavljena od ActiveX kontrola. Školjke komandne linije na Windows imaju sekundarni značaj. Školjka CMD omogućuje komande slične onim u MSDOS, dok školjka PowerShell predstavlja školjku sa naprednim skriptnim jezikom.

15.25.Framework .NET

Nakon relativno kratkog postojanja izvršne platforme ugrađene u Windows za programe pisane u jeziku Java i sudskih tužbi između firmi Microsoft i Sun oko uslova ugradnje te platforme, u Microsoftu su 2001 razvili prvu verziju okruženja .NET koje će biti usporedivo sa Java okruženjem. Od Windows Vista dolazi standardno instaliran, dok se za starije operativne sisteme mora posebno instalirati.

Objedinjujući ideje iz Java i COM modela, **.NET okruženje** namijenjeno je za razvoj i izvođenje programa na različitim operativnim sistemima i hardverskim platformama, neovisno o programskom jeziku u kojem su ti programi pisani. Program napisan u višem programskom jeziku ne prevodi se mašinski jezik, već u međujezik (engl. *Intermediate Language*, IL). U tom se obliku aplikacija isporučuje krajnjem korisniku. Prilikom izvođenja programa na odredišnom računaru, taj međukôd .NET okruženje prevodi u izvedbeni kôd te ga izvodi. Samo okruženje sastoji se od dva dijela: izvršnog jezičkog okruženja CLR i biblioteke klasa FCL.

Izvršno jezičko okruženje (*Common Language Runtime*, CLR) je sistem koji je zadužen za izvođenje aplikacija, prevodeći ih pri tome iz međujezika u mašinski kôd. CLR među ostalim sadrži objektno-orijentirani model koji podržava nasljeđivanje,

polimorfizam, rukovanje izuzecima, sistem za automatsko upravljanje memorijom koji sam oslobađa dijelove memorije ako ih proces ne koristi, sigurnosni model pješčanika, sve osnovne .NET tipove podataka, mnoge dodatne klase .NET okruženja, prevodilac međujezika (IL-a) u mašinski kôd te optimizator kôda. Izvršno okruženje upravlja izvođenjem kôda samo ako je taj kôd definiran kao upravljani kôd (engl. *managed code*).

Biblioteka klasa .NET okruženja (*Framework Class Library*, FCL) jest biblioteka klasa i metoda koja dolazi s .NET okruženjem. Tu biblioteku mogu koristiti svi programi koji se izvode na .NET okruženju, bez obzira na jezik u kojem je program pisan. Biblioteka se sastoji od nekoliko hiljada klasa koje pokrivaju široko područje primjene, uključujući elemente korisničkog interfejsa, rad s datotekama, povezivanje sa bazama podataka, za rad s različitim tipovima podataka i njihovim kolekcijama, razvoj web aplikacija, numeričke algoritme, komunikaciju između aplikacija i drugo. Zbog tako velikog broja klasa, one su razvrstani po srodnosti u odgovarajuće imenske prostore od kojih je najznačajniji *System* u kojem je definisan tip *Object* iz kojega su izvedeni svi ostali tipovi. Prevedeni programi za .NET platformu sastoje se od skupa datoteka u formatu IL koji su upakovani u jedinstvenu EXE ili DLL datoteku koja se zove asemblija.

15.26. Sistemski alati i korisnički procesi

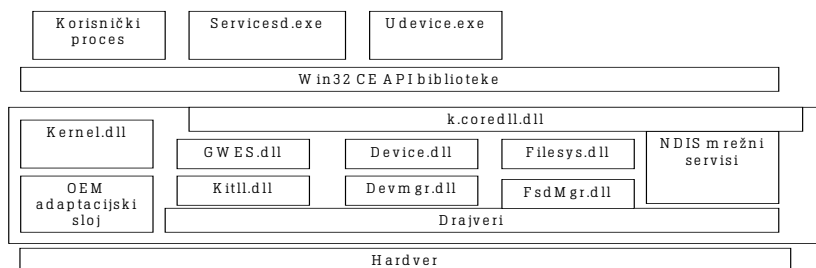
Već i tek instaliran Windows uključuje desetine alata za sistemsku administraciju (*Control Panel*, *Microsoft Management Console*, *Event Viewer*, *Task Scheduler*, *Backup*, *Disk Cleanup* ...) i korisničkih aplikacija (*Calculator*, *Internet explorer*, *Edge*, *Media Player*, *Paint*, *Notepad*, *Wordpad*, *Skype*, *OneNote*...) kao i manji broj igara (uglavnom verzije pasijansa). U svakoj verziji Windowsa pojavljuju se nove i ukidaju zastarjele aplikacije.

15.27. Windows Runtime

Pad prodaje desktop računara, porast upotrebe mobilnih uređaja, a vrlo slab uspjeh operativnog sistema Windows Phone, doveo je do potrebe za novim API-jem i stilom aplikacija, počevši od Windows 8. API je nazvan *Windows Runtime*, u Windows 10 *Universal Windows Platform*, a stil aplikacija je u kratkom periodu promijenio više imena: *Metro*, *Modern*, *Store*, *Universal Windows Application*. API je objektno orijentisan, a klase su smještene u *windows*.dll* datotekama. Može se koristiti i sa aplikacijama u C++ (prirodni kôd) i sa .NET aplikacijama (kontrolisani kôd). Aplikacije su univerzalne u smislu da im se veličina prilagođava uređaju, od mobilnog telefona, Xbox konzole, desktopa ili televizora. Distribucija programa ide preko web prodavnice Windows Store.

15.28. Windows CE, Phone, Mobile i Pocket PC

Četvrta grupa operativnih sistema sa imenom Windows je daleko manje kompleksnosti. U Windows CE sistemima cijeli operativni sistem je u ROM-u kao i aplikacije koje dolaze sa sistemom. Ako programski modul nije kompresovan, ROM programi se izvršavaju na lokaciji u ROM-u gdje se već nalaze. Ako je modul u ROM-u kompresovan, dekompresuje se i prebaci u RAM. Upisivi podaci se smještaju u RAM.



Slika 191 Windows CE arhitektura

Windows CE je operativni sistem dijeljenog izvornog koda, gdje proizvođači uređaja licenciraju izvorni kôd. Na raspolaganju je nekoliko arhitektura. Nije kompatibilan s WIN32, a (za razliku od XP embedded koji zahtijeva bar 8 megabajta) CE 5 može se smijesiti u 350 kilobajta. Razvijen je u verzijama 1,2,3,4,5,6 i 7. Na bazi jezgra Windows CE, razvijeni su i Windows Phone, Mobile i Pocket PC. Oni su operativni sistemi za mobilne uređaje, bazirani na CE kernelu, ali nisu tako modularni kao Windows CE.

CE kernel smješten je u datoteku nk.exe, koja uključuje određene dinamičke biblioteke: kernel.dll (u kome je jezgro operativnog sistema, upravljanje procesima i memorijom), gwes.dll (realizacija prozora i grafike), device.dll i devmgr.dll (upravljanje driver-ima), filesys.dll i fstd.dll (upravljanje datotekama), k.coredll.dll (svi korisnički programi su vezani za nju). Specifičnosti svakog od sistema smještaju se u OAL (OEM adaptacijski sloj) koji realizuje interapte, upravljanje napajanjem i tajmer. API za programe ima dosta funkcija Win32 API, iako ne nužno smještenim u iste biblioteke.

15.29.Zadaci

Zadatak 1. Identifikujte na Windows računar podsysteme opisane u ovom poglavlju tražeći datoteke u C:\Windows\System32 direktoriju.

Zadatak 2. Potražite skup alata Sysinternals i njim testirajte ponašanje Windows podsistema. Ponašanje nekih od podsistema jezgra istražite alatima iz ovog paketa, navedenim u tabeli.

Object manager	WinObj, Handle,
Security reference monitor	AccessChk, AccessEnum, AdExplorer
Virtual memory manager	RAMMap, VMMap
Cache manager	CacheSet
I/O manager	LoadOrder, NTFSInfo, DiskExt, DiskView, TCPView, PortMon
Configuration manager	Autoruns, RegJump
Process manager	PsList, Process Explorer, ClockRes, ListDLLs, Pipe-List
Service manager	PsService

Zadatak 3. Potražite na Internetu izvorni kod dijela jezgra Windows NT pod imenom Windows Research Kernel. Uočite direktorije u stablu izvornog koda koji čine ntoskrnl.exe datoteku, i povežite ih sa podsistemima kernel i executive.

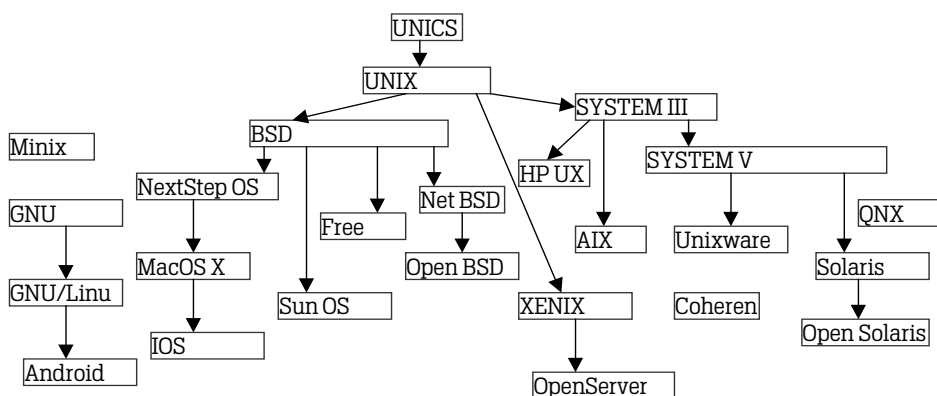
16.ARHITEKTURA UNIX I LINUX

16.1.UNIX historijat

MULTICS iz 1964. je operativni sistem konzorcija više američkih organizacija, univerziteta i korporacija kao operativni sistem opće namjene. Ovaj projekt je uveo puno značajnih novina u svijet operativnih sistema, ali je imao za to vrijeme prevelike hardverske zahtjeve. Stoga su inženjeri Bell Labs-a, nakon što se ova firma povukla iz MULTICS projekta, Denis Richie i Ken Thompson 1969. kreirali prvi UNICS, kasnije **UNIX** sistem, napisan u asemblerskom jeziku, za računare PDP 7, koje su koristili u laboratoriji. Kasnije je čitav sistem ponovo napisan u programskom jeziku C (koga su također razvili), čime je znatno olakšano prenošenje UNIX operativnog sistema na druge vrste računara. Za njihov projekt zainteresovali su se univerziteti, prije svega Berkley koji je nastavio razvoj i izmjene poslao nazad. Verzija UNIX-a razvijana na Berkley univerzitetu se zove BSD.

Zainteresovala se, međutim, i njihova firma, AT&T, vlasnik Bell Labs-a, koji poduzima korake na komercijalizaciji UNIX-a. Njihova verzija se zove System V. Proizvođači hardvera i softverske kompanije licenciraju UNIX od AT&T da bi kreirale vlastite verzije UNIX-a. Njihove promjene su ostale interne, pa ljudi izvana nisu mogli vidjeti šta je izmijenjeno.

Nezadovoljan razvojem događaja, Richard M. Stallman, profesor na MIT-u, 1984 godine osniva Free Software Foundation i započinje rad na GNU sistemu. Iako je razvijeno dosta alata i programa opće namjene (implementacija komandi, školjka, C kompajler), GNU sistemu je nedostajalo jezgro operativnog sistema. Započet je razvoj HURD kernela ali je išao sporije nego što je očekivano.



Slika 192 Unix porodično stablo

I dok su komercijalni Unix sistemi (uz Free BSD sisteme otvorenog koda) nastavili da se razvijaju iz početnog izvornog koda Unix-a, (Slika 192), paralelno su nastala još dva jezgra pisana od početka, a sa stanovišta sistemskih poziva slična Unix jezgru. Prvo je **Minix**, prije svega namijenjen u obrazovne svrhe, objavljen u okviru knjige Andrew Tanenbaum-a, "Dizajn operativnih sistema". Iz razloga da to

jezgro ostane dovoljno malo, kako bi bilo upotrebljivo za nastavu, autor nije dopuštao dodavanje novih osobina u Minix. Drugo jezgro je **Linux**. Linus Torvalds je napisao novo jezgro koje je bilo kompatibilno sa UNIX jezgrom, ali sa dostupnim izvornim kodom i liberalnom politikom oko izmjena tog izvornog koda. Linux jezgro je licencirano pod GNU GPL licencom, koja određuje da softver licenciran pod njom se može mijenjati, prilagođavati, pa čak i prodavati, ali prilikom redistribucije tog istog softvera, ista prava moraju imati i drugi korisnici. Ubrzo je Linux jezgro postalo nedostajuća karika GNU sistema.

16.2.POSIX

Najvažniji standard koji omogućava međusobnu kompatibilnost operativnih sistema, na nivou izvornog koda aplikacija, nastalih iz Unix-a je POSIX. POSIX, skraćenica od *Portable Operating System Interface* je zajedničko ime za porodicu povezanih standarda koje definiše Institut inženjera elektrotehnike i elektronike (IEEE). Cilj standarda je da definiše oblik i ponašanje programskog okruženja (API-ja) operativnog sistema, ljuske i komandi kompatibilnih sa operativnim sistemom Unix. Međunarodno ime za ove standarde je ISO/IEC 9945 ili IEEE 1003. Trenutna verzija ovog standarda podijeljena je u 4 dijela

- Dio 1: Osnovne definicije – terminološki rječnik, varijable okruženja, preporučena struktura direktorija, terminalski interfejs, preporučeni način pisanja alata, C biblioteke.
- Dio 2: Sistemski interfejsi – opisuje funkcije pozivane iz biblioteke za C za pristup sistemskim pozivima, kao i standardnu C biblioteku.
- Dio 3: Školjka i alati – Interne komande školjke i sintaksa alata.
- Dio 4: Razlozi - napomene o poglavljima prethodna tri dijela o motivaciji za standard.

Jedan dio operativnih sistema iz Unix porodice je certificiran prema POSIX standardu (AIX, HP-UX, IRIX, Solaris, UnixWare, QNX Neutrino). Drugi dio je necertificiran, ali je dovoljno POSIX kompatibilan za rad većine aplikacija (Android, BeOS, Darwin, FreeBSD, Linux, NetBSD, OpenBSD, OpenSolaris, Xenix). Pored ovoga, postoje i skupovi aplikacija za operativne sisteme koji nisu Unix tipa sa komandama i C bibliotekama za razvoj programa čiji je izvorni kôd pisan za POSIX sisteme (Cygwin za Windows, DJGPP za MS DOS, Unix System services za z/OS).

POSIX standard ne govori o strukturi jezgra, formatu izvršnih programa, ali ni višim slojevima sistema kao što je grafički interfejs. S obzirom na različitosti POSIX sistema, hardverske razlike i dužinu historije razvoja, u daljnjem će biti opisan samo jedan od njih, trenutno najpopularniji, Linux.

16.3. Linux

Linus Torvalds, tada student Univerziteta u Helsinkiju je 1991. Započeo razvoj Linux-a kao hobi projekta, no vrlo brzo su se uključili novi istraživači, korisnici i razvojni timovi. Linux je izvorno razvijen za Intel 80386 procesor na IBM kompatibilnim personalnim računarima. Tokom godina prenesen je na različite platforme: DEC Alpha, Sun SPARC, Motorola 680x0, HP PA-RISC, Intel Itanium, x64, ARM, kao i na multiprocesorske platforme.

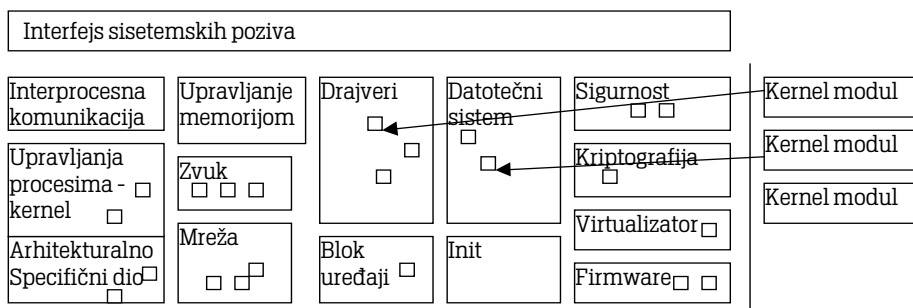
Linux nije komercijalni proizvod koga kontroliše i prodaje jedna kompanija. On je proizvod otvorenog koda i njegov izvorni kôd je slobodno dostupan. Aktualni izvorni kôd jezgra se može naći na web stranici <http://www.kernel.org> ili u paketu sa dodatnim aplikacijama. Najveće vrijednosti Linux-a su kompatibilnost sa ranijim i novim Unix softverom, cijena (obično besplatno ili simbolično), raznovrsnost mogućnosti instalacije, podrška velikom broju hardverskih platformi, skalabilnost (od minimalnih Raspberry Pi do najjačih superkompjuteri), pogodnost za istraživa-nje principa operativnih sistema (izvorni kôd je dostupan), sigurnost (sistem prava ugrađen od samog početka) itd.

Linux jezgro je u svom razvoju prošlo kroz više faza. Verzije se označavaju s 3 broja, pri čemu su prva dva značajnija (ako izuzmemo nestabilne grane) 0.1 (samo 386, Minix FS), 1.0 (mreža, dodatni hardver), 1.2 (posljednja samo za PC) 2.0, 2.2 (više arhitektura), 2.4 i 2.6 (simetrično multiprocesiranje). Nakon 2.6.39 prešlo se na seriju 3.0, a nakon 3.19 na seriju 4.0. U trenutku pisanja ove knjige verzija je 4.19.8 (decembar 2018).

Samo jezgro nije, naravno dovoljno za rad. Standardni, kompajlirani skup paketa, ili distribucije, uključuju osnovni Linux sistem, alate za instalaciju i upravljanje sistemom, te osnovne UNIX alate. Prve distribucije su samo raspakivale datoteke na mjesta, kasnije imaju naprednije rukovanje paketima. Neke najpoznatije distribucije su Slackware, RedHat, Debian, Ubuntu, Gentoo, Suse, Mandriva. One se međusobno razlikuju po formatu datoteka u kojima se isporučuju aplikacije. Dva najpopularnija takva formata su RPM i DEB.

16.4.Komponente Linux sistema

Glavne grupe komponenti Linux sistema su jezgro, systemske biblioteke, sistemski alati i korisnički programi. Jezgro održava glavne aktivnosti operativnog sistema. Izvršava se u privilegovanom režimu, a njegove strukture i kôd su u istom adresnom prostoru. Systemske biblioteke definišu standardni skup funkcija kroz koje aplikacije surađuju s jezgrom i funkcionalnost OS, koja ne zahtijeva pune privilegije jezgra. Sistemski alati obavljaju specijalizovane zadatke upravljanja sistemom.



Slika 193 Arhitektura Linux jezgra

Na sistemima na kojima se nalazi izvorni kôd jezgra, on je obično smješten u direktoriju `/usr/src/linux` dok je prevedena verzija najčešće u direktoriju `/boot` pod imenom koje zavisi od načina njegove kompresije: `vmlinuz`, `vmlinuz`, `ZImage`,

bzImage i druga. Izvorni kôd je organizovan u više poddirektorija. Dijelovi koji su ovisni od platforme smješteni su u direktorij *arch*. Podissemi jezgra su smješteni u posebne direktorije. Linux jezgro je u principu monolitno. To nije mikro-kernel arhitektura, nego je jezgro jedna datoteka, čiji se dijelovi pozivaju koristeći funkcije u jeziku C. Na slici Slika 193 se vide komponente jezgra koje odgovaraju odgovarajućim direktorijima u stablu njegovog izvornog koda. Izvorni kôd jezgra je znatno veći od njegove prevedene verzije jer uključuje podršku za veliki broj procesora, perifernih uređaja, mrežnih protokola i algoritama raspoređivanja. Svi se nikada ne koriste istovremeno, pa se jezgro kompajlira izabirajući koji dijelovi pojedinih komponenti ulaze u prevedeno jezgro a koji ne.

Ipak, zbog velikog potencijalnog broja drajvera za perifernijske uređaje, moguće je jezgro razdvojiti u više modula koji su u odvojenim datotekama i posebno se učitavaju, ali su u istom adresnom prostoru kao i ostatak jezgra. Moduli se koriste za drajvere za opcione perifernijske uređaje, koji se učitavaju po potrebi, na primjer kada se ubaci USB disk. Sistem modula ima tri komponente.

1)Sistem za upravljanje modulima učitava module u memoriju i komunicira s jezgrom. Za učitavanje i izbacivanje modula koriste se alati u korisničkom režimu za učitavanje i izbacivanje modula. To su naredbe *insmod* i *rmmod*.

2)Sistem registracije drajvera dopušta modulima da kažu ostatku kernela da postoji novi drajver. Spisak raspoloživih modula se čuva u posebnim tablicama. Moduli mogu biti drajveri uređaja, datotečni formati, datotečni sistemi i mrežni protokoli.

3)Mehanizam razrješenja konflikta rezerviše hardverske resurse za jedan drajver i štiti ih od drugog drajvera. Korisnik to vidi kao alat iz komandne linije *modprobe* za učitavanje modula koji provjeri koji su moduli potrebni da se učitaju prije zahtijevanog modula i da li je neki od njih već zauzeo određene resurse (npr. I/O portove)

Linux distribucije se međusobno razlikuju po izboru softvera koji je uključen u njih i načinu njihove instalacije i pakovanja. Kako se vidi na slici Slika 194, Linux distribucije se sastoje od jezgra koje se nalazi u glavnoj datoteci i modulima, sistema za pokretanje (*boot loader*, inicijalizacijski program i inicijalni RAM disk), skupa biblioteka od kojih je najvažnija *libc*, pozadinskih programa, školjke, alata iz komandne linije, grafičke infrastrukture oko *X.org* servera i alata u grafičkom režimu. Menadžer paketa brine o tome da su instalirani dijelovi sistema međusobno konzistentni. Nijedna komponenta, osim jezgra, nije obavezna. Naravno, neke komponente se ne mogu koristiti bez drugih (npr. Koffice zahtijeva KDE koji lančano ovisi od QT biblioteke, X.Org servera, nekih sistemskih biblioteka i jezgra), a nekim slučajima dvije komponente ne mogu da koegzistiraju (npr. treba instalirati samo jedan *boot loader* ili *display manager*).

Rad s datotekama Thunar Dolphin Nautilus Krusader	Grafika: GIMP Krita Inkscape blender	Kancelarijski programi OpenOffice koffice abiword gnumeric	Editori: vim kate gedit	Igre	Razvojna okruženja eclipse kdevelop anjuta lazarus
Web browseri: Firefox Konqueror	E-mail: Kmail Thunderbird Evolution	Mrežni klijenti: kopete pidgin mirc	Emulatori: qemu dosbox wine mame mess vice fuse	Tekst alati: meld kdiff3	Zvuk i video: xine mplayer amarok ffmpeg gstreamer vlc
Menadžeri prozora: metacity kwm fwvm twm jwm openstep fluxbox compiz		Desktop okruženja: GNOME XFCE KDE			
		Displej menadžeri: xdm kdm gdm	GTK + biblioteka	QT biblioteka	Open Motif
X.org server xlib		svgalib		Framebuffer,	
Paketi softvera Dpkg apt rpm portage yum slackpkg tar bzip2 gzip zip	Administracija: lspci lshal lsusb adduser top ps jobs procpd free slabtop	Mrežni alati: iwconfig ip iptables netstat route ifconfig host socklist tcpdump wget netcat curl traceroute ping rsync	Mrežni klijenti: mutt mail lynx ftp telnet ssh	Razvojni alati gcc binutils gdb javac fpc make f2c	Tekst alati: diff sed grep latex
Coreutils: pwd printf printenv vmstat ln dd df du cp rm uniq sort comm join cat paste chown chmod echo nice stat sync ls mkdir ...		Util-linux: mkfs fdisk mount su more kill	Interpreter Runtime perl php awk python java	Školjka: BASH : jvm mono getty login	
Servisi: klogd acpid getty login syslogd crond dbus udev hotplug hald	Štampač: lpd cups	DBMS: MySQL SQLite PostgreSQL	Mrežni serveri: telnetd sshd apache postfix inetd portmap named NetworkManager ftpd nfs samba iSCSI aMULE	Razvojni serveri: svn cvs bugzilla	
Libselinux	zlib libexpat	libstdc++	libssl	Lokalizacija konzola	
Libcrypt	ld.so libxml	libm	libresolv libgettext	Initrd (RAM disk)	
Librt pthread	libdl	GNU C biblioteka		Školjka: Busybox	
Kernel moduli	procfs	Linux kernel		Boot: LILO, GRUB	

Slika 194 Arhitektura Linux distribucije

16.5.Upravljanje procesima u Linux

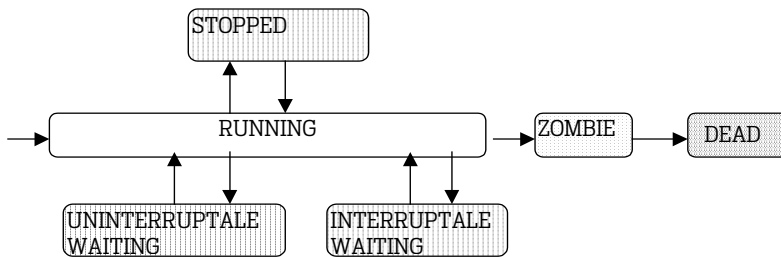
Za pokretanje procesa pod Linux koriste se uobičajene UNIX operacije za procese, kao što je poziv *fork* koji kreira novi proces, a on onda može da pozove *execve* da učita u memoriju sliku drugog procesa.

Linux koristi niti u jezgru. Nit je iste strukture kao proces i ona je novi proces s istim adresnim prostorom kao roditelj. Razlika je u kreiranju. Proces se kreira pozivom *fork* koji kreira novi proces s novim kontekstom i sadržajem memorije podataka, dok poziv *clone* kreira novi proces s svojim identitetom, ali dijeli strukture podataka o procesu s roditeljem.

Procesi u jezgru se evidentiraju kroz deskriptore procesa. Deskriptor procesa ili niti je sastavljen od strukture *task_struct* i svih objekata na koje ona pokazuje. O svakom procesu su sačuvane informacije o stanju, adresnom prostoru, otvorenim datotekama, signalima, resursima i pokazivačima na druge strukture. Svaki *task_struct* objekt je stavljen u ulančanu listu koju koristi raspoređivač, a koja počinje objektom *init_task*.

Kada se proces ne izvršava, njegovo stanje se dijelom čuva u deskriptoru procesa, a dijelom u području jezgra predviđenom za *stack*. To uključuje programski brojač, pokazivač *stack*-a, registre opće namjene, statusnu riječ, registre za upravljanje memorijom itd. Kada proces nastavlja sa izvršenjem, ove zapamćene vrijednosti se vraćaju u procesor.

Stanja u kojima proces može biti su: *Running* (trenutno se izvršava ili čeka u redu spremnih procesa), *Interruptible waiting* (u stanju čekanja, ali ga POSIX signal može reaktivirati), *Uninterruptible waiting* (u stanju čekanja, signali ga ne mogu reaktivirati), *Stopped* (zaustavljen signalom SIGSTOP), *Traced* (u trenutku sesije sa *debugger*-om), *Zombie* (završio, ali njegov deskriptor se još čuva), *Dead* (kernel briše proces). Za razliku od nekih Unix sistema kao što je OpenBSD, Linux ne evidentira posebnu razliku između stanja *Ready* i *Running*.



Slika 195: Dijagram stanja procesa u Linux

Nakon pokretanja jezgra, jezgro samo kreira jedan proces, koga zovu *process 0*, *swapper* ili *idle* proces. Identifikacijski broj tog procesa je 0. U višeprocorskim sistemima postoji onoliko procesa sa ID 0 koliko procesora. Osim tog procesa, svaki drugi je kreiran od strane roditeljskog procesa. Proces 0 inicijalizira strukturu jezgra i pokreće proces 1 koji se zove *init* proces. *Idle* proces ostaje u memoriji, i kada nijedan drugi proces ne radi, on se izvršava. *Init* proces pokreće ostale procese i on je u suštini predak svih procesa.

Svaki proces se identificira cijelim brojem koji se zove PID (*process identifier*). On se čuva u *pid* polju strukture *task_struct*. Maksimalna dopuštena vrijednost za PID je 32767, a vezu između PID i odgovarajućeg *task_struct* objekta održava tablica *pidhash*. Drugi identifikatori procesa su PGRP (grupa procesa) i sesija, uglavnom korištene za među-procesnu komunikaciju.

Adresni prostor procesa se kreira pri pozivu *fork*. U početku se koristi ista kopija programa i podataka kod oba procesa, osim povratne vrijednosti poziva *fork* koja je kod dijete-procesa jednaka 0, a roditelj proces dobiva vrijednost PID dijete procesa. Program i podaci se ne dupliciraju, nego se koristi tehnika COW, *Copy on Write*. Linux postavi dijeljenje stranica između roditeljskog i dijete procesa. Kada dijete proces pokuša promijeniti sadržaj dijeljene stranice, desi se izuzetak greške stranice, kreira se stranica sa novim sadržajem i promijeni tablica stranica da uključi i nju.

Niti se evidentiraju u istoj *task_struct* strukturi kao i procesi. Umjesto poziva *fork*, nit se kreira pozivom *clone*, koji dijete proces drži u istom adresnom prostoru kao roditelj proces. Kada dijete proces mijenja podatke, *Copy on Write* se ne izvršava.

Razlika u odnosu na POSIX standard je što u POSIX standardu sve niti istog procesa dijele zajednički PID. U Linux-u niti istog procesa imaju različite PID vrijednosti, ali se u *task_struct* čuva i polje *tgid*, koje evidentira PID procesa koji je pozvao *clone*. Ako ta vrijednost postoji, onda se ona vraća kao vrijednost poziva *getpid*.

Algoritmi za raspoređivanje procesa su se mijenjali u raznim verzijama jezgra. Linux koristi dva raspoređivačka algoritma, koje određuje klasa procesa, algoritam u dijeljenom vremenu za pravično raspoređivanje i algoritam za realno vrijeme gdje je apsolutni prioritet važniji od pravičnosti.

Za procese u dijeljenom vremenu, Linux koristi prioritetni algoritam sa dinamičkim podešavanjem. U verziji 2.6 algoritam izgleda ovako: Za procese u dijeljenom vremenu prioritet je između 100 (najviši) i 139 najniži, podrazumijevano 120. Prioritet se računa od statičkog i dinamičkog dijela. Pozivom *nice* mijenja se statički prioritet. Kvantum se dobiva od statičkog prioriteta, prioritetniji procesi dobivaju veći kvantum. Kvantum je cjelobrojni umnožak otkucaja koji predstavlja milisekundu. Računa se po formuli

```
if (static_prio < 120) base = (140-static_prio) * 20;
else if (static_prio >= 120) base = (140-static_prio)*5;
```

Dinamički prioritet se dobiva iz statičkog prioriteta i bonusa. Bonus ima vrijednost između 0 i 10, a zavisi od prosječnog vremena procesa provedenog u čekanju. Bonus i dinamički prioritet računaju se po formulama

```
bonus = MIN(sleep_avg/ 100, 10)
prio = MAX(100, MIN(static_prio - bonus + 5, 139))
```

Pri svakom vremenskom otkucaju (svake milisekunde) ažurira se *sleep_avg* (srednje vrijeme uspavanosti). Uvećava se ako je proces u stanjima *TASK_INTERRUPTIBLE* i *TASK_UNINTERRUPTIBLE*, a umanjuje ako se proces u tom trenutku izvršava. Maksimalno dopušteno vrijeme uspavanosti je 1 sekunda.

U Linux-u aplikativni procesi mogu biti prekinuti u bilo kom trenutku kada su u korisničkom režimu, ali kada su u režimu jezgra (tokom sistemskog poziva ili greške stranice) jezgro odlaže njihovo prekidanje postavljajući indikator *tif_need_resched* u *thread_info* objektu.

Za procese u realnom vremenu Linux koristi FCFS i round-robin klase; u oba slučaja još ima i prioritet. Raspoređivač odabere proces najvećeg prioriteta, za jednako prioritetne procese, odabere onaj koji je najduže čekao. FCFS procesi nastavljaju do završetka ili svog blokiranja. A round-robin procesi se prekidaju nakon isteka kvantuma i prebace na kraj reda raspoređivanja, što znači da se round-robin procesi jednakog prioriteta automatski raspoređuju. Prioritet procesa klase u realnom vremenu su između 1 i 99, manji broj znači veći prioritet. Rokovi u realnom vremenu nisu podržani.

Nakon verzije 2.6.0 raspoređivač je promijenjen. Ranije verzije su prolazile kroz cijelu tabelu procesa u vremenu $O(N)$, pa je za veći broj procesa, raspoređivanje bilo sporo. U verzijama 2.6.0 do 2.6.23 primijenjen je $O(1)$ raspoređivač koji raspoređuje procese u konstantnom vremenu, bez obzira koliko procesa ima u operativnom sistemu. Algoritam se zasniva na 140 redova prioriteta i mape bitova u kojoj svaki bit označava da li ima elemenata u datom redu čekanja. Na taj način se brzo nalazi koji proces se naredni izvršava. Od verzije 2.6.24 zamijenjen je s *Completely Fair*

Scheduler, čije je vrijeme $O(\log N)$. Raspoređivač čuva procese umjesto ulančane liste u formi balansirano *red-black* stabla, koristeći proteklo vrijeme kao ključ i birajući najdalji lijevi čvor.

Prilikom izmjene konteksta, čuva se stanje procesora. Neki registri se čuvaju u steku jezgra, a ostali u deskriptoru procesa (*task_struct*). U slučaju i386, kada se desi izmjena konteksta, jezgro snima registre u deskriptor procesa i na kraju pročita sadržaj ESP registra iz deskriptora procesa na koji se prelazi. Snimanje registara opće namjene obavlja se uz pomoć *Task state segment* strukture koja omogućava izmjenu registara opće namjene jednom instrukcijom skoka.

16.6.Među-procesna komunikacija u Linux

Linux u jezgru podržava klasične UNIX komunikacijske mehanizme, signale i cijevi. Pored ovoga podržani su semafori, poruke i dijeljena memorija. Signali su najjednostavniji komunikacijski mehanizam prema korisničkim procesima, ali ih jezgro ne koristi. Koriste se za informaciju procesima o izuzecima. Na primjer, ako proces pristupi nedopuštenom području memorije, generiše se signal SIGSEGV. Dva člana *task_struct* objekta, *pending_signal* i *blocked*, čuvaju za svaki od mogućih signala po jedan bit da li je signal proslijeđen i da li je zabranjen. Procesori sa 32 bitnom riječi mogu imati do 32 signala.

Linux realizuje cijevi preko dva datotečna objekta. Oni dijele isti privremeni indeksni čvor. Indeksni čvor vezan za cijev ne predstavlja datoteku, nego se koristi specijalni, za korisnika nevidljivi, datotečni sistem *pipefs*. Taj datotečni sistem je u memoriji, u području jezgra. Dva datotečna objekta imaju različite zadatke, jedan je samo za čitanje a drugi samo za pisanje prema cijevi. Kada proces 1 piše u cijev, podaci iz njegovog privatnog područja se kopiraju u memorijski okvir sa dijeljenim podacima. Kada proces 2 čita iz cijevi, podaci se kopiraju iz dijeljenog okvira u privatni memorijski prostor procesa.

Imenovane cijevi se u Linux zovu FIFO (nema veze sa više puta spominjanim algoritmom). FIFO je specijalna datoteka u datotečnom sistemu, koja ima indeksne čvorove, ali ne i sadržaj.

Postoje dvije vrste semafora, semafori za među-procesnu komunikaciju i semafori za sinhronizaciju jezgra. Semafori za sinhronizaciju korisničkih procesa se čuvaju u jedinstvenom nizu *sem_array*. Proces koji želi obaviti semaforske operacije poziva sistemski poziv *semop*.

Procesi mogu komunicirati i redovima poruka. Proces koji želi komunicirati sa drugim preko poruka poziva poziv *msgget* i šalje odnosno čita poruke pozivima *msgsnd* i *msgrcv*. Poruke u redu čekanja su u objektima *msg_queue* koji sadrže dva reda čekanja, za pošiljaoca i primaoca. Red čekanja poruka je ograničene veličine. Ako pošiljalac ne može dalje slati poruke jer je red pun, on se blokira.

16.7.Sinhronizacija jezgra

Iz korisničkog režima rada zahtjev jezgru se može desiti iz dva razloga. Prvi razlog je kada aktivni program zahtijeva jezgro sistemskim pozivom ili greškom stranice. Drugi razlog je ukoliko *driver* izaziva hardverski prekid. Taj drugi razlog se dešava asinhrono i jezgro se može nalaziti u kritičnoj sekciji. Sinhronizacija jezgra pazi da

kritična sekcija jezgra ne bude prekinuta drugom kritičnom sekcijom. Linux koristi dvije tehnike za zaštitu kritične sekcije:

1. Normalno izvršavanje jezgra se ne prekida. Kad se desi prekid sata usred servisne rutine jezgra, postavi se *need_resched* varijabla, pa se raspoređivač pokrene nakon završetka sistemskog poziva.

2. Unutar obrade prekida, novi prekidi su isključeni. Da se izbjegne zaostajanje rada ostatka sistema, Linux jezgro smanjuje isključenost prekida u kritičnoj sekciji. Obrada prekida se dijeli na rutine u gornjoj polovini i donjoj polovini. U gornjoj polovini svi prekidi su isključeni. U donjoj polovini oni su uključeni, a mali raspoređivač sprječava da se donje polovine međusobno prekidaju.

U jezgru se koriste tri mehanizma sinhronizacije, atomske operacije, semafori jezgra i spinlock-ovi.

Atomske operacije se realizuju u mašinskom jeziku koristeći instrukcije koje se ne mogu prekidati, čak i u višeprocorskim sistemima. Na i386 to su instrukcije koje imaju specijalni kôd ispred same instrukcije LOCK.

Semafori jezgra su tipa *struct semaphore* i realizuju dvije atomske operacije *up* i *down*.

Spinlock je binarni semafor bez liste čekanja. Oni isključuju prekidanje izvršenja jezgra dok se ne oslobode. Posebno su bitni u višeprocorskim sistemima.

16.8.Upravljanje memorijom

Kernel područje, nevidljivo za program
Stek područje
.bss sekcija (neinicijalizirani podaci)
.data sekcija (podaci)
.text sekcija (programski kod)
Zabranjeno područje

Slika 196 Memorijsko područje procesa

Svaki proces ima virtualni i fizički pogled na memorijsko područje. Novi virtualni prostori se kreiraju pri pozivima *fork* i *exec*. Linux jezgro za svaki proces ima svoje, mašinski ovisno područje u memoriji, podijeljeno na pokazivače na sve dostupne stranice i nerezervisani dio. Procesima je dostupno samo njihovo vlastito područje. (Slika 196). Segmentacijski dio na i386 arhitekturi je malo korišten, svi procesi dijele istu deskriptorsku tabelu i razlikuju se samo u dijelu straničenja. Na i386 linearni adresni prostor je 4 Gb. Adrese između 00000000 BFFFFFFF se koriste kada je proces u korisničkom režimu, a adrese C0000000 do FFFFFFFF se koriste kada je proces u režimu jezgra.

Prilikom kreiranja procesa sa *fork* pozivom novonastali proces dijeli memoriju s roditeljskim. Sistem za upravljanje memorijom tada postavi stranice oba procesa u režim samo za čitanje. Onog trenutka kada jedan od njih pokuša upisati podatak, dešava se prekid, i kreira se nova memorijska stranica s kopiranim sadržajem

stranice u koju se pokušalo upisati. Nakon toga se upiše tražena vrijednost u obje stranice i ubuduće one postaju upisive. Ova tehnika se zove *Copy on write* koja znatno ubrzava pokretanje procesa i štedi memoriju.

Da bi se operativni sistem mogao prebaciti i na arhitekture koje nisu Intel tipa, Linux ne koristi segmente, nego samo stranice. Na Intel i386 platformama se onda svi segmenti postavljaju da obuhvataju cijelu memoriju.

Zavisno od arhitekture, primijenjeno je straničenje na 2-4 nivoa. Stoga u opštem slučaju, linearna adresa se sastoji od 5 dijelova: *Page Global Dir*, *Page Upper Dir*, *Page Middle Dir*, *Page Table* i *Page Offset* pri čemu se neki od njih ne koriste ako nisu hardverski podržani.

Algoritam zamjene stranica je približan WSClock algoritmu. Linux održava dvije liste stranica: listu stranica koje se koriste (*active_list*) i listu stranica koje su kandidati za brisanje (*inactive_list*). Kada je memorija kritična, stranice s početka liste *active_list* se prebacuju na kraj liste *inactive_list*. Ako se dva puta pristupi stranici koja je u *inactive_list*, ona se prebaci u *active_list*. Iz memorije se, kada je to potrebno, izbacuju stranice iz *inactive_list*.

Pri dodjeli fizičke memorije stranice se dodjeljuju i prema namjeni. Na PC računarima, Linux razlikuje tri područja memorije po hardverskim karakteristikama. *ZONE_DMA* predstavlja područje memorije u kome svi uređaji mogu da koriste DMA, čak i oni koji imaju 24 bitni brojač memorijske lokacije. *ZONE_NORMAL* je uobičajeno memorijsko područje za dodjelu memorije, ali samo neki uređaji mogu da koriste DMA u tom području. *ZONE_HIGH* predstavlja područje koje jezgro mora posebno tretirati, na primjer na 64 bitnim Linux sistemima to je područje iznad 4G i njegove adrese se ne mogu dodjeljivati 32 bitnim aplikacijama.

Prilikom zahtjeva za blokom fizičke memorije koji se sastoji od određenog broja memorijskih stranica, koristi se *buddy* sistem za praćenje slobodne fizičke memorije. Kada se dva susjedna regiona oslobode oni formiraju veći region. Ako se zahtjev ne može zadovoljiti alokacijom malog regiona, veći se dijeli na dva dijela rekurzivno dok se ne dođe do dijela zadovoljavajuće veličine.

Za zahtjeve manje od stranice koristi se alokator koji smješta više objekata navedene ili standardne veličine (stepen broja 2) u neku stranicu, što je približno algoritmu *quick fit*.

16.9.Datotečni sistemi

Linux poznaje veliki broj fizičkih datotečnih sistema: minix, ext, ext2, ext3, ext4, Reiserfs, XFS, JFS, xia, msdos, umsdos, vfat, ntfs, proc, nfs, iso9660, hpfs, sysv, smb, ncpfs. Oni se međusobno razlikuju po ograničenjima na veličinu datoteka, imena datoteka, upisivost podataka, upotrebu žurnala i sigurnost. Ipak jezgro apstrahuje sve sisteme kao jedinstveni *virtual file system* (VFS). U njemu se datoteke opisuju kroz četiri strukture, *superblock* (informacije o montiranom datotečnom sistemu), *inode* (informacije o pojedinoj datoteci), *file* (informacije o otvorenoj datoteci) i *dentry* (informacije o pojedinom elementu direktorija). Čak i datotečni sistemi koji nemaju indeksne čvorove (fat, ntfs) se premapiraju u ovaj format.

Najčešće korišten fizički datotečni format je ext2fs koji alokira podatke u blokovima od 1K i smješta logički susjedne blokove datoteke u fizički susjedne blokove na

disku. Njegova žurnalska verzija je ext4fs. Razvijen je iz klasičnog Unix datotečnog sistema, ali se indeksni čvorovi, bitmape i blokovi podataka ne drže na samo jednom mjestu. Particija se dijeli na više podblokova, od kojih svaki sadrži superblok, opise grupe, bitmape indeksnih čvorova, bitmape blokova podataka, tabelu indeksnih čvorova i same podatke. Ext2 pokušava da alocira blokove podataka tako da podaci budu u istoj logičkoj grupi gdje su njihovi indeksni čvorovi. Ako nema slobodnih takvih blokova, traže se blokovi u susjednim grupama.

Proc datotečni sistem je namijenjen za pristup parametrima jezgra. Ovaj datotečni sistem ne čuva podatke, nego se automatski generiše na UI zahtjeve. On mora implementirati virtualnu strukturu direktorija sa brojevima indeksnih čvorova. Broj indeksnog čvora određuje vrstu informacija koje se traže od jezgra.

16.10.Drajveri uređaja

Znakovnim uređajima, najjednostavnijim od Linux uređaja se pristupa kao datotekama. Aplikacije koriste standardne systemske pozive da ih otvore, čitaju iz njih, pišu i zatvaraju kao da su datoteke. Pri inicijalizaciji znakovnog uređaja, njegov drajver se registruje dodajući element u *chrdevs* vektor strukture *device_struct*. Identifikator glavnog broja uređaja (npr 4 za tty uređaj) se koristi kao indeks ovog vektora. Glavni broj uređaja je fiksiran.

Svaki element *chrdevs* vektora, je struktura *device_struct* koja sadrži dva elementa; pointer na ime registrovanog drajvera i pointer na blok datotečnih operacija. Blok datotečnih operacija predstavlja adrese rutina koje obavljaju specifične datotečne operacije kao što su *open*, *read*, *write* i *close*.

Kada se otvara specijalna datoteka koja predstavlja znakovni uređaj (na primjer /dev/cua0) jezgro mora pripremiti pozive pravih rutina za rad s datotekama. Svaki uređaj je predstavljen indeksnim čvorom u virtualnom datotečnom sistemu i sadrži glavni i sporedni identifikator uređaja. Svaki indeksni čvor ima povezan skup datotečnih operacija koje su moguće sa njim.

Blok uređajima se također može pristupiti kao datotekama. Registracija ovih uređaja je slična registraciji znakovnih uređaja. Linux čuva skup registrovanih uređaja u *blkdevs* vektoru koji je indeksiran glavnim brojem uređaja. I njegovi elementi su strukture *device_struct* tipa. Za razliku od znakovnih uređaja, postoje klase blok uređaja: SCSI uređaji, IDE uređaji i drugi. Klasa se registruje u jezgru i pruža datotečne operacije za njega. Drajveri blok uređaja određene klase pružaju specifične interfejs za klasu.

Svaki drajver blok uređaja mora takođe pružiti interfejs prema disk kešu. Drajver blok uređaja puni element *blk_dev* vektora strukture podatka *blk_dev_struct*. Indeks za ovaj vektor je glavni broj uređaja. Struktura *blk_dev_struct* se sastoji od adrese rutine za obradu zahtjeva i pointera na listu strukture podataka za zahtjev, koja predstavlja zahtjeve iz disk keša prema drajveru za čitanje i pisanje bloka podataka. Svaki put kada disk keš želi čitati ili pisati podatke na registrovani uređaj, on dodaje strukturu zahtjeva za podacima u svoju *blk_dev_struct*.

Mrežni uređaji šalju i primaju pakete podataka. Neki od njih su fizički, npr. *Ethernet* kartica, a neko čisto softverski, npr. *loopback* koji služi za slanje podataka samom sebi. Svaki mrežni uređaj je predstavljen strukturom koja ga opisuje. Ova

struktura sadrži podatke o uređaju i adrese funkcija koje dopuštaju mrežnim protokolima da koriste usluge uređaja. Ove funkcije se uglavnom odnose na prijenos podataka na mrežni uređaj. Svi mrežni paketi su predstavljeni *sk_buff* strukturama podataka koje omogućavaju da se zaglavlja protokola dodaju ili oduzimaju. Za razliku od blok i znakovnih uređaja koje treba kreirati u datotečnom sistemu s *mknod* naredbom, mrežni uređaji se pojavljuju spontano kako se inicijaliziraju. Elementi strukture uređaja su ime, irq, bazna adresa uređaja, DMA kanali koje koristi, flegovi s njegovim stanjem, veličina paketa, familija mrežnih protokola, tip uređaja, adresa, red čekanja na pakete i skup funkcija koje uređaj podržava.

16.11. Osnovne biblioteke

Sada će biti riječi o dijelu Linux sistema u korisničkom režimu rada. Aplikacije se vezuju sa dinamičkim bibliotekama koje realizuju najvažnije funkcije. Gotovo svi programi u binarnom formatu vezuju se za biblioteku *libc*. To je najvažnija biblioteka u Linux sistemima. Ona pruža funkcionalnost standardne C biblioteke zajedno sa sistemskim pozivima jezgra. To uključuje funkcije koje direktno pozivaju sistemski poziv poput *fork*, *open*, *read*, *write*, *brk*, *execv*, *exit*, kao i funkcije iz standardne C biblioteke koje su uglavnom realizovane unutar te biblioteke u korisničkom režimu rada (*malloc*, *printf*, *strcpy*, *fopen*...). Osnovnim bibliotekama smatraju se biblioteke navedene u sljedećoj tabeli

Biblioteka	Namjena
<i>libc</i>	Osnovne funkcije jezika C i upakovani sistemski pozivi
<i>libm</i>	Matematičke unkcije iz standardne C biblioteke (<i>sin</i> , <i>cos</i> , <i>atan</i> , <i>sqrt</i> ,...)
<i>libpthread</i>	Funkcije POSIX niti i semafora (<i>pthread_create</i> , <i>pthread_join</i> , <i>sem_wait</i> ...)
<i>libstdc++</i>	Uključuje bazne klase biblioteke za C++
<i>libdl</i>	Pristup dinamičkim bibliotekama (<i>dlopen</i> , <i>dlclose</i> , <i>dlsym</i>)
<i>libcrypt</i>	Funkcije za kriptografisanje podataka
<i>libncurses</i>	Pružaju poboljšani prikaz teksta u konzoli, kontrola pozicije na ekranu
<i>libutil</i>	Funkcije prijave korisnika i ažuriranje datoteke lozinki
<i>libz</i>	Funkcije kompresije (<i>gzopen</i> , <i>inflate</i> , <i>deflate</i>)
<i>libgcc_s</i>	Funkcije za gcc koje omogućavaju praćenje izvršenja programa i izuzetke
<i>librt</i>	Omogućava komunikaciju procesa preko poruka i postavljanje tajmera
<i>libnspr4</i>	Platformski neutralne funkcije za pristup sistemu slične <i>libc</i> funkcijama.
<i>libnss3</i>	Platformski neutralne sigurnosne funkcije
<i>libssl3</i>	Funkcije za mrežnu sigurnost kroz kriptografisanu komunikaciju
<i>libgettext</i>	Prevođenje poruka programa na nacionalne jezike
<i>libxml</i>	Omogućava analizu dokumenata u xml formatu.
<i>libsane</i>	Pruža funkcije za rad sa skanerom.
<i>libxslt</i>	Namijenjen za transformaciju dokumenata u XML formatu
<i>libcups</i>	Omogućava rad sa štampačem, slanje dokumenata i podešavanje drajvera.

Ove biblioteke su obično smještene u */lib* direktoriju ili */usr/lib* direktoriju. Pored njih još mnogo dijeljenih biblioteka se instalira po potrebi za aplikativne programe.

16.12. Pozadinski procesi

Pozadinski procesi (demoni) obavljaju odgovarajuću sistemsku funkciju. Prema Linux i Unix principu modularnosti oni nisu do jezgra, nego su obični procesi, ali nemaju interakcije sa korisnikom. Jedna grupa ovih procesa se pokreće pri inicijali-

zaciji sistema i aktivni su do gašenja, dok se druga grupa pokreće kada su potrebni i rade samo dok su potrebni.

Prvi proces koji se pokreće nakon inicijalizacije jezgra je **init** pa je stoga i najvažniji pozadinski proces. On ima PID=1 i predak je svih procesa. Pri startu on ili pokreće sistem u jednokorisničkom režimu ili pokreće seriju skripti potrebnih za prelazak u višekorisnički režim. U višekorisničkom režimu *init* prati da li se korisnici mogu prijaviti na raznim uređajima sa kojih je prijava moguća. To može biti konzolna tastatura, uređaji spojeni na serijske portove ili prijava preko grafičkog *xdm* procesa. Također, *init* briše zombi procese kada se njihov normalni roditeljski proces završi. Proces *init* se može konfigurirati da se pokreće na različite načine, zavisno od toga koje resurse zahtijeva. Na primjer, moguće je podići Linux sistem da radi kao server, grafička ili jednokorisnička mašina. Izbor načina podizanja vrši se komandom *telinit*, čiji parametar može biti broj između 0 i 6, koji predstavlja nivo izvršavanja. U konfiguracionoj datoteci */etc/inittab* za svaki nivo izvršavanja navedena je odgovarajuća komanda koja se pri tome pokreće i nastavlja inicijalizaciju sistema.

Za raspoređivanje procesa u određenim trenucima (npr. automatski *backup*) koriste se pozadinski procesi **crond** i **atd**. *Crond* je namijenjen za periodične zadatke (npr. preuzimanje nove verzije softvera svakog četvrtka) dok je *atd* predviđen za jednokratno pokretanje u određenom vremenu (npr. slanje novogodišnje čestitke).

Pozadinski procesi **xinetd** ili **inetd** upravljaju drugim pozadinskim procesima. Pomoću *inetd* se pokreću prvenstveno Internet serverske aplikacije. U konfiguracionoj datoteci */etc/inetd.conf* navodi se koji pozadinski procesi se pokreću na koji mrežni zahtjev. Mrežni zahtjevi su opisani u datoteci */etc/services*. Tako na primjer, pozadinski program za slanje elektronske pošte se može konfigurirati da se prvi put pokrene tek kad je inicirano slanje pošte.

Printer *spooler* je realizovan procesima **cupsd** ili **lpd**. Stari stil štampanja obavlja se preko procesa *lpd*, koji prima poslove štampe od korisnika i po potrebi pokreće procese za stvarnu štampu. Mogućnosti *cupsd* su znatno veće, on po potrebi pokreće konverzije programe za štampu, a može mu se pristupiti i preko web interfejsa.

Dva osnovna protokola za realizaciju mrežnih diskova pokrivena su odgovarajućim skupom pozadinskih procesa. Prvi protokol je NFS. Pozadinski proces **rpc.nfsd** se izvršava na datotečnom serveru i prihvata zahtjeve od klijenata za pristup njegovim datotekama preko tog protokola. Drugi takav protokol je SMB ili CIFS koji je popularniji na Windows računarima. Proces **smbd** omogućava dijeljenje datoteka i štampača da budu vidljivi Windows računarima, dok proces **nmbd** pruža ime računara da bude vidljivo Windows mašinama.

Za slanje i prijem elektronske pošte koriste se četiri pozadinska procesa, zavisno od željenog protokola i namjene računara. Slanje elektronske pošte obavlja se pozadinskim procesom **sendmail** koji prihvata poruke od korisnika i šalje ih prema drugim računarima na Internetu. Računari koji očekuju da se na njih pošalje pošta (tipično mail serveri) izvršavaju **smtpd**, koji očitava port 25 za ulazne poruke i prosljeđuje ih za daljnju obradu, drugim serverima ili ih smješta na svoj disk. Ako se elektronska pošta čuva na disku, korisnici mogu čitati svoju elektronsku poštu sa udaljenog servera, ako server izvršava pozadinske procese **popd** (jednostavniji POP3 protokol, koji može samo preuzimati i brisati poruke) ili **imapd** (napredniji IMAP

protokol, namijenjen da se poruke čuvaju stalno na serveru kako bi im se pristupalo sa različitih lokacija).

Da se omogućiti korisniku prijava sa udaljenog računara terminalskim pristupom preko mreže, a da se komande izvršavaju na serveru koristi se **sshd**. Komunikacija se obavlja kriptografisano, što nije slučaj sa ranijim pozadinskim programima za terminalski pristup poput **in.rlogind**, **in.telnetd** i **in.rshd** kod kojih se mogu pročitati lozinke i sadržaj prijenosa osluškivanjem mreže.

Za distribuciju zajedničkih datoteka u mreži poput datoteka sa lozinkama (*/etc/passwd*) ili datoteka sa imenima računara (*/etc/hosts*) koristi se NIS server. Na računaru koji distribuira podatke treba biti pokrenut **ypserver**, a na onim koji ih koriste **yplibind**. Za prijenos jedne datoteke na što jednostavniji način, bez ikakvih prijava (nekad potrebno za podizanje jezgra preko mreže) koristi se **in.tftpd**.

Automatska konfiguracija računara (saznavanje IP adrese i DNS servera) obavlja se DHCP servisom. Na Linux mašinama tu uslugu pruža pozadinski proces **dhcpcd**. Ostale informacije o hardveru sakuplja **hald** iz raznih izvora i smješta ih u aktivnu listu uređaja. Jedna grupa pozadinskih procesa pomaže funkcionalnosti koje inače obavlja jezgro. Takvi su **ksoftirqd** (obrađuje softverske prekide kada je opterećenje preveliko), **kacpid** (kontrolira uređaje koji imaju informacije o napajanju), **kblockd** (prati red čekanja na blok uređaje), **aio** (ponovo pokušava asinhrono operacije), **kswapd** (prebacuje stranice u swap prostor), **ata** (podrška za serijske ATA diskove), **scsi_eh_** (Obavlja upravljanje SCSI greškama), **kjournald** (podrška za žurnalske datotečne sisteme), **events** (obrada generičkih redova čekanja).

Procesi mogu da zapisuju događaje u cilju praćenja. Umjesto da za to koriste konzolu ili datoteke, brine se proces **syslogd** koji evidentira te poruke i sortira ih prema pravilima koje je definirao sistemski administrator.

Ako računar služi za čuvanje web stranica, na njemu se pokreće odgovarajući **httpd World Wide Web server**. Najpopularnijim se smatra Apache httpd. Tada se može pristupiti serveru koristeći *web browser*.

Za slanje datoteka ka serveru i od servera koristi se FTP (*File transfer protocol*) server. Za aktiviranje ovog servisa potrebno je pokrenuti proces **ftpd** i podesiti odgovarajuće direktorije. Može se konfigurirati da mu mogu pristupiti samo registrovani korisnici ili bilo ko. Pošto se FTP ne smatra sigurnim protokolom, alternativa za kopiranje datoteka je program **rsync** koji se može konfigurisati da radi u pozadini.

U većim mrežama administratori treba da prate mrežu radi dobivanja statistike sa podacima o protoku, upotrebi procesora ili slobodnom prostoru na disku. U tu svrhu na računarima se može instalirati SNMP server (**snmpd**) kojim se šalju informacije o sistemu. Server za DNS (domenski sistem imena) se zove **named**. On mapira imena računara u IP mrežne adrese koristeći distribuiranu bazu kako bi se imenovali računari u cijelom svijetu. Ukoliko server ima ulogu preusmjerivača na mreži, mogu se instalirati servisi **routed** ili **gated**, koji imaju tablice koje kažu na koju mrežnu karticu preusmjeriti koji zahtjev.

16.13.Školjke i osnovne komande

Na Linux sistemima postoji veći izbor školjki za rad u komandnoj liniji, ali najčešće se koristi *bash* koji je već opisan. U fazi podizanja sistema koristi se i školjka

busybox koja u istom izvršnom procesu sadrži nešto jednostavniju školjku *ash* i pridodate *coreutils* alate po želji, u cilju manjeg zauzeća diska.

Kolekcija najvažnijih komandi iz komandne linije koji se koriste u Unix, a tako i Linux sistemima zove se *Coreutils*. To uključuje oko 100 komandi za rad sa datotekama i direktorijima, prikaz datoteka i neke systemske alate (*chgrp, chown, chmod, cp, dd, df, dir, ln, ls, mkdir, mkfifo, mknod, mv, rm, rmdir, cat,...*).

Pored *coreutils*, Linux distribucije uključuju veći broj drugih administrativnih alata i korisničkih programa iz komandne linije, poput editora teksta (*vi, emacs, joe*), formatera teksta (*latex*), mrežnih konfiguracijskih alata (*ifconfig, ping*), kompajlera i interpretera za programske jezike (*gcc, python, javac, make*).

16.14. Paketni menadžeri

Linux distribucije se međusobno najviše razlikuju po načinu instalacije softvera. Programi često zavise o većem broju datoteka: sam izvršni program, dijeljene biblioteke koje on koristi, konfiguracije datoteke, interpreteri programskih jezika u kojima su programi napisani, dokumentacije, slike, fontova i drugo. Neke Linux distribucije očekuju od korisnika da sam brine o instalaciji tih pomoćnih datoteka, dok druge pažljivo prate sve potrebne međuzavisnosti između datoteka i omogućavaju automatsku instalaciju dodatnih potrebnih datoteka. Paketni menadžeri brinu o potrebnim datotekama za programe, međuzavisnostima, njihovim preuzimanjem sa mreže i eksternih medija, instalaciji i deinstalaciji. Najpoznatiji paketni menadžeri su *rpm* i *dpkg*.

16.15. X server i bazne grafičke biblioteke

Najčešće korišteni sistem za podlogu grafičkom interfejsu na Linuxu je X.Org, jedna od verzija X Window sistema. X Window System se može podijeliti na nekoliko ključnih komponenti. Jedna od njih je menadžer prikaza (*Display manager*) čiji je glavni zadatak autentifikacija korisnika, prijavljivanje i pokretanje inicijalnog okruženja i skripti za pokretanje. Menadžer prikaza također pokreće X server, koji definiše apstraktni interfejs za grafičke sisteme i ulazne uređaje (tastaturu i miš). Skripte za pokretanje također pokreću menadžer prozora, koji omogućava korisniku da pomjera, promijeni veličinu, minimizira i maksimizira prozore, kao i da upravlja zasebnim virtualnim računarima. Konačno, na najnižem nivou, aplikacije su povezane sa bibliotekom grafičkih komponenti. X server razumije samo vrlo osnovni skup primitivnih funkcija; on ne definiše programski interfejs za entitete na visokom nivou, kao što su dugmad, klizne trake, tekstualna polja i meniji. Ovaj dizajn postiže dva važna cilja. Prvo, omogućava X serveru da radi na računaru koji je potpuno odvojen od onog kod klijentske aplikacije. Drugo, on dozvoljava serveru da podrži razne različite menadžere prozora i skupove komponenti.

Menadžer prikaza predstavlja korisnički (grafički) ekran za prijavljivanje i obično je prva stvar koju korisnik vidi kada sjedne za računar. Nije obavezno imati ga, pa mnogi korisnici onemogućavaju menadžer prikaza i sami pokreću X iz tekstualne konzole komandom *startx* (koja pokreće *xinit* program, a koji pokreće X server). Tri najpoznatija menadžera prikaza su *xdm* (originalni), *gdm* (menadžer prikaza GNOME-a) i *kdm* (menadžer prikaza KDE-a). Menadžer prikaza može upravljati udaljenim prijavama na druge X servere kroz XDMCP protokol. Konfiguracijske

datoteke u poddirektoriju xdm, gdm ili kdm unutar /etc/X11 navode kako će se prikazati menadžer prikaza. U tipičnom scenariju, menadžer prikaza traži korisničko ime i lozinku, a zatim se provjeri autentičnost korisnika.

Krajnji zadatak menadžera prikaza je da izvrši skriptu Xsession, koja postavlja korisničko okruženje na nivou cijelog sistema, a na kraju ona pokreće ~/.xsession, za korisnički specifične postavke. Ova skripta pokreće sve procese u pozadini osim posljednje komande koja je menadžer prozora.

X protokol omogućava udaljeno izvršavanje aplikacija, tj., da se na jednom računaru program izvršava, a na drugom prikazuje. Da bi aplikacija mogla da prikazuje sliku na udaljenom računaru koristi se varijabla okruženja DISPLAY. Varijabla sadrži ime ili IP adresu servera, broj prikaza (identifikaciju određene instance X servera za povezivanje) i opcioni broj ekrana (za prikaz sa više monitora).

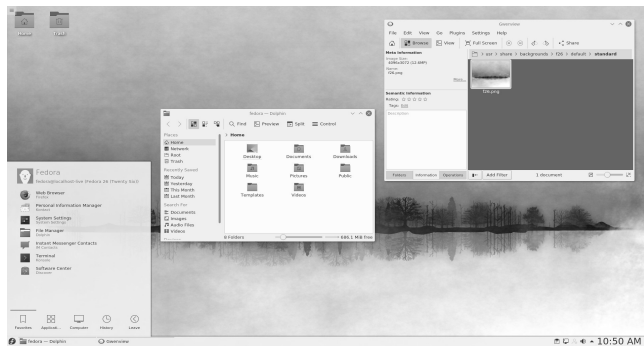
```
DISPLAY = server.mreza.com: 10.2; export DISPLAY
```

Ova postavka ukazuje na se X aplikacije izvršavaju na ekranu 2 servera, koji se zove server.mreza.com, koristeći prikaz 10. Aplikacije uspostavljaju TCP konekciju sa serverom na portu broj 6000 plus broj prikaza (u ovom primjeru, port 6010), koga osluškuje X server koji upravlja tim ekranom.

16.16.Desktop okruženja

Fleksibilnost modela X Window sistema je tokom godina dovela do razvoja skupova grafičkih kontrola, menadžera prozora, pretraživača datoteka, alata i uslužnih programa. Pošto su aplikacije postale naprednije i zahtijevale progresivno veću podršku od osnovnog sistema prozora, postalo je jasno da je neophodan sveobuhvatan pristup unapređenju platforme. Iz ove potrebe su rođena dva velika moderna desktop okruženja: GNOME i KDE.

KDE, koji označava *K Desktop Environment*, napisan je u C++ i izgrađen je na biblioteci alata Qt. KDE često preferiraju ljudi koji prelaze sa Windows ili Mac okruženja zbog ugodnog izgleda i srodnosti korištenja sa Windows. Aplikacije napisane za KDE skoro uvek sadrže K negde u imenu, na primer, Konqueror (web pretraživač), Konsole (terminal emulator) ili KWord (word processor). Podrazumijevani menadžer prozora, kwin, podržava freedesktop.org *Standard Window Manager Specification*, mogućnost promjene ukupnog izgleda i osjećaja i mnoge druge funkcije. Programski paket KOffice sadrži program za obradu teksta, tablice i prezentacije. KDE posjeduje sveobuhvatan skup razvojnih alata, uključujući integrirano razvojno okruženje (IDE).



Slika 197 KDE okruženje

GNOME je napisan u C i baziran je na GTK + skupu kontrola. Uz dodatak, GNOME je uključio grafičke efekte koji su mu nedostajali. GNOME je manje doraden i manje konzistentan od KDE, ali je češće čistiji i jednostavniji. Mnoge Linux distribucije koriste GNOME kao podrazumijevano okruženje za radno okruženje. Kao i KDE, GNOME ima bogatu kolekciju aplikacija. Aplikacije GNOME-a uobičajeno se prepoznaju prisustvom slova G u njihovim imenima. Jedan od izuzetaka je standardni menadžer prozora GNOME-a, pod nazivom *metacity* koji obavlja osnovne funkcije prozora. Upravitelj datoteka se zove Nautilus. Kancelarijske aplikacije uključuju AbiWord za obradu teksta, Gnumeric kao tablični kalkulator i GIMP za obradu slika.

Popularna su i grafička okruženja manjih resursnih zahtjeva kao što su XFCE, LXDE, EDE a ponekad se koriste i samostalni menadžeri prozora Fluxbox, IceWm, twm, jwm, olwm i druga. Izbor desktop okruženja ne diktira koje aplikacije se mogu pokrenuti. Bez obzira na izbor radnog okruženja, mogu se odabrati aplikacije koje pripadaju drugom. Ipak, uočice se primjetne razlike u konzistenciji izgleda i brzini pokretanja.

16.17. Korisničke aplikacije

Pored aplikacija koje dolaze u paketu sa desktop okruženjem, Linux distribucije obično nude instalirane ili spremne za instalaciju kompleksne korisničke aplikacije poslovnog tipa (Open Office, Gnumeric), web preglednike (Firefox, Chrome), web servere (Apache), programe za stono izdavaštvo (Scribus, Latex), servere baza podataka (MySQL, PostgreSQL), kompajlere za većinu programskih jezika i druge. Izbor komercijalnih programa za Linux je znatno manji nego za Windows, ali je mogućnost sastavljanja vlastite kolekcije veća.

16.18. Zadaci

Zadatak 1. Kompajliranje jezgra. Ako je u Linux-u instaliran izvorni kôd njegovog jezgra, on se obično nalazi u direktoriju `/usr/src/linux-xxxxx` gdje je `xxxx` oznaka verzije jezgra. Daljnji poddirektoriji ovog direktorija su *arch* (kôd specifičan za pojedinu procesorsku arhitekturu), *crypto* (kriptografske rutine), *fs* (implementacija datotečnog sistema), *sound* (drajveri za zvuk), *block* (drajveri diskova), *kernel* (raspoređivač), *mm* (upravljanje memorijom), *init* (glavna petlja jezgra), *net* (mrežni drajveri), *drivers* (drajveri perifernih uređaja), *ipc* (interprocesna komunikacija), *security* (sigurnosni podsistem). Pregledajte sadržaj ovog direktorija s

```
ls -lR | more
```

Linux jezgro je monolitno u smislu memorijskog prostora, ali je izrazito modularno po pitanju izbora pojedinih dijelova jezgra. Da se konfigurira šta ulazi u jezgro, koristi se komanda.

```
make menuconfig
```

Nakon izvjesnog vremena, ponudiće se opcije za izbor postavki jezgra. Komponente koje se uključuju mogu biti ugrađene u jezgro, izbačene ili kompajlirane kao kernel modul. Komponente koje su ugrađene u jezgro se brže pokreću od onih koje su u modulima, ali opterećuju memoriju, ako nisu potrebne. Po završetku konfiguriranja izlazi se iz programa za izbor komponenti i pokreće komanda make, nakon koje je potrebno čekati između nekoliko minuta i nekoliko sati.

```
make
```

Po završetku procesa, kreira se datoteka s jezgrom (obično se zove vmlinuz ili bzimage) i više desetina kernel modula (ekstenzija datoteke .ko). Ako se želi nova verzija jezgra instalirati na postojeći operativni sistem, koriste se sljedeće komande.

```
make modules_install  
make install
```

17. OPERATIVNI SISTEMI ZA MOBILNE UREĐAJE

U posljednje vrijeme, mobilni uređaji, kao što su tablet računari, čitači elektronskih knjiga, pametni mobilni telefoni postižu velike uspjehe i preuzimaju primat nad personalnim računarima.

Mobilni uređaji u usporedbi s stonim imaju više prednosti: uvijek su s korisnikom, imaju često Internet pristup, GPS, kompase, kamere i mikrofone. Mana im je u ograničenom ekranu, dužini trajanja baterije, ograničenoj brzini procesora, sporijem mrežnom pristupu, ulaznom uređaju koji je sporiji od klasične tastature (softverska tastatura, dodirni ekran, tasteri na mobitelu, pisaljke). Pošto je riječ o razmjerno novoj oblasti za tržište se bori više platformi. Ovdje će biti riječi o arhitekturi nekih od njih.

17.1. Google Android

Android operativni sistem zasnovan je na Linux jezgru i prilagođen je tako da se može koristiti na većini mobilnih uređaja. On se sastoji od modifikovanog monolitnog Linux jezgra zaduženog za podršku hardveru i funkcije niskog nivoa, skupa biblioteka zaduženih za dodatne zadatke kao što su iscrtavanje grafike, podrška za dekodiranje video snimaka, podrška za SSL enkripciju itd.

Iako je Android Linux distribucija, on po nekim stvarima odstupa od većine standardnih Linux distribucija, npr. Android nema standardni X Window System ili standardni skup GNU biblioteka pa tako nije u mogućnosti da pokreće aplikacije razvijene za druge standardne Linux sisteme. Stoga, dok jezgro Androida ima arhitekturu kao na slici Slika 193, arhitektura cijelog sistema nije poput slike Slika 194 nego je prikazana na slici Slika 198. Veliki dio Android operativnog sistema napisan je u Java programskom jeziku dok su samo kernel i neke biblioteke niskog nivoa napisani u C i C ++.

App Launcher	Barcode maker	Barcode reader	Basic calculator	Batery level	Browser
Caller ID	Clock	Compass	Contact list	Current location	Time/date
E-mail	ePub reader	MP3 player	Navigation aid	Notetaker	Scheduler
Signal strength	SMS messenger	Telephone	Video magnifier	Voice recorder	Voice search
Notification Manager		Package Manager		Pružaoici sadržaja	
View System					
Activity Manager		Resource Manager		Window Manager	
LocationManager					
Surface manager		Media framework		SQLite	
FreeType		Osnovne biblioteke			
init		zygote		adbd	
Webkit		SGL		SSL	
Libc		Open GL		Dalvik VM	
Linux kernel					
Displej		Keypad		Memorija	
Napajanje		Kamera		WiFi	
Audio		Binder			

Slika 198 Arhitektura Android sistema

Android podržava širok spektar Googleovih usluga kao što su: Gmail, kalendar i sinhronizacija kontakata, razmjena poruka između *cloud*-a i uređaja kao i mnoge druge usluge, neke su vidljive za korisnika, a neki nisu. Najvažnija usluga je *Play Store* (Prodavnica). Kada programeri naprave Android aplikacije, oni će ih objaviti na *Play Store*-u. Time je osigurano poštovanje standarda za aplikacije. Kao i u tradicionalnom Linux sistemu, prvi Androidov proces u korisničkom okruženju je *init*, što je roditelj svih ostalih procesa. Pozadinski procesi koje Android *init* pokreće, međutim, više se fokusiraju na detalje niskog nivoa (upravljanje datotekama i hardverskom pristupu), a ne na korisničke mogućnosti višeg nivoa kao što je elektronska pošta.

Pozadinski proces **zygote**, pokreće Java procese višeg nivoa. On je vezan za sistemske biblioteke i Java biblioteke. U sklopu biblioteka se nalazi i odvojeni Android Runtime koji sadrži osnovne, bazne, biblioteke i Dalvik virtualnu mašinu koja je zadužena za pokretanje aplikacija višeg nivoa napisanih u Java programskom jeziku. Java kôd u Android aplikacijama je dostupan u Dalvik-ovom formatu. Ovaj format simulira mašinu baziranu na registrima umjesto Java tradicionalnog *stack* baziranog formata. Format Dalvikovog *bytecode* omogućava brže izvršavanje, dok i dalje podržava JIT (*Just-in-Time*) kompilaciju. Dalvikov *bytecode* je također efikasniji u pogledu prostora diska i RAM-a.

Mobilnim uređajima potrebna je velika brzina pokretanja procesa, a Java platforma nema tu osobinu. Proces *zygote* rješava taj problem. *Zygote* je odgovoran za podizanje i inicijalizaciju Dalvik-a do tačke gdje je spreman za pokretanje sistema ili aplikacijskog koda napisanog u Java. Kreiranje novog procesa iz *zygote* uključuje Linux *fork*, ali ne postoji *exec* poziv. Novi proces je kopija izvornog *zygote* procesa, sa svojim unaprijed inicijaliziranim stanjem, postavljenim i spremnim za rad. Nakon *fork*, novi proces ima svoje vlastito Dalvik okruženje, premda dijeli sve unaprijed učitane i inicijalizirane podatke sa *zygote* putem *copy-on-write* stranica. Sve što je sada potrebno da novi aktivirani proces bude spreman jeste da dobije tačan identitet (UID itd.), završi svaku inicijalizaciju Dalvika koji zahtijeva početne niti, i učitavanje aplikacije ili sistemskog koda za pokretanje.

Prvi proces koji *zygote* uvijek pokreće, zove se *system_server* (sistemski server), koji sadrži sve osnovne servise operativnog sistema. Ključni dijelovi su menadžer napajanja, menadžer paketa, menadžer prozora i menadžer aktivnosti.

Iz *zygote* će se po potrebi kreirati ostali procesi. Neki od njih će se izvršavati svo vrijeme, kao što je telefonski proces. Dodatni procesi aplikacija će se kreirati i zaustaviti po potrebi dok je sistem pokrenut.

Pošto tipični Android uređaj nema lokalnu konzolu, školjka se ne pokreće na tradicionalni način, nego pozadinski proces *adb* osluškuje zahtjeve konekcije (kao što je konekcija preko USB-a) koji zahtijevaju pristup školjci.

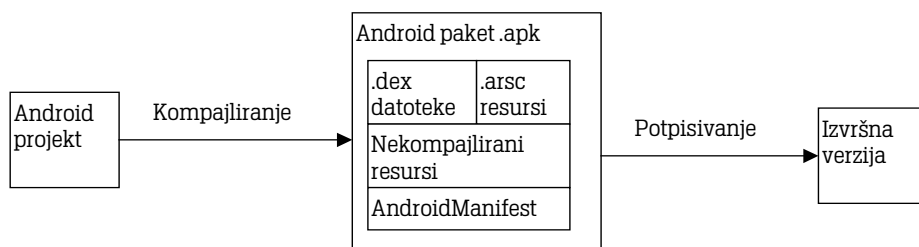
Aplikacije komuniciraju sa operativnim sistemom preko poziva kroz biblioteke koje on pruža, a koje zajedno čine Android framework. Neke od ovih biblioteka mogu obavljati svoj rad unutar tog procesa, ali mnoge će morati obavljati komunikaciju sa drugim procesima, a to su česti servisi unutar *system_server* procesa.

Komunikacija između aplikacija ne koristi tradicionalne mehanizme poput cijevi, Unix poruka ili semafora. Umjesto toga razvijen je poseban kernel modul *binder*, baziran na porukama. Poruka u *Binder*-u označena je kao transakcija i na višem nivou može se posmatrati kao funkcionalni poziv kroz procese. Svaka transakcija koju korisničko okruženje šalje kernelu je potpuna operacija: identifikira cilj operacije i identitet pošiljaoca, kao i sve podatke koji se isporučuju. Kernel određuje odgovarajući proces za primanje te transakcije, isporučujući je na thread čekanja u procesu.

Pokretanje samih aplikacija se ne vrši direktno, već se aplikacije pokreću u okruženju odvojenom od ostatka sistema, gdje dobivaju samo određeni dio sistemskih resursa, pa tako nemaju pristup dijelovima sistema koji su im nepotrebni, što donekle poboljšava sigurnost i stabilnost sistema. Također, pri instalaciji aplikacija korisnik dobiva listu svih dozvola koje jedna aplikacija zahtijeva da bi se instalirala, što korisniku daje mogućnost da uoči potencijalno štetne aplikacije i obustavi njihovu instalaciju prije nego što dođe do oštećenja. Svaka aplikacija ima svoj Linux proces, a svaki proces ima svoju Dalvik VM. Svakoj aplikaciji se dodjeljuje jedinstveni Linux ID, dok su prava postavljena da su aplikacijske datoteke vidljive samo njoj.

Na višem nivou od biblioteka su sistemске aplikacije neophodne za upotrebu sistema od strane korisnika i tu se nalaze, *window manager*, menadžer resursa, menadžer instalacijskih paketa, kao i aplikacije zadužene za obavljanje osnovnih funkcija vezanih za mobilne telefone ili uređaj na kom je instaliran Android. Na najvišem nivou se nalaze krajnje korisničke aplikacije, odnosno aplikacije koje direktno koristi korisnik.

Android aplikacije se grade koristeći Java i SDK biblioteke, ali ne koriste uobičajene *Swing* ili *AWT* biblioteke za grafičko okruženje. Java code se kompajlira u Dalvik *byte code* (.dex). Ova virtualna mašina je optimizovana za mobilne uređaje (bolje upravljanje memorijom, upotreba baterije, itd.). Dalvik VM izvršava .dex datoteke. Nakon kompajliranja, aplikacije se pakuju u pakete s ekstenzijom .apk (Slika 199).



Slika 199 Proces gradnje Android aplikacije

Aplikacija nije izvršna datoteka sa glavnom ulaznom tačkom, nego paket svega što čini tu aplikaciju: njen kôd, grafički resursi, deklaracije o tome šta je ona u sistemu i drugi podaci. Android aplikacija prema konvenciji je datoteka sa apk ekstenzijom, za Android paket. Ova datoteka je zapravo normalna zip arhiva, koja

sadrži sve o aplikaciji. Sastoji se od manifesta, resursa potrebnih za aplikaciju, kompajliranih Dalvik klasa i informacija za prijavljivanje.

Manifest opisuje aplikaciju, šta ona radi i kako je pokrenuti. Manifest mora da sadrži ime paketa za aplikaciju, (npr. `com.android.app.painter`) koje ga jedinstveno identifikuje. Manifest je XML datoteka nazvana `AndroidManifest.xml`. Početak i kraj te datoteke izgledaju ovako:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication"
    android:versionCode="1"
    android:versionName="1.0" >
    ...
</manifest>
```

Android aplikacije nemaju jednu glavnu ulaznu tačku koja se izvršava kada ih korisnik pokrene. Umjesto toga, objavljuju pod manifest <application> oznakom razne ulazne tačke koje opisuju različite stvari koje aplikacija može učiniti. Ove ulazne tačke su izražene kao četiri različita tipa, definišući osnovne tipove ponašanja koje aplikacije mogu pružiti: aktivnost, prijemnik, servis i pružalac sadržaja.

Aktivnost je ulazna tačka za interakciju sa korisnikom. Ona predstavlja jedan ekran sa korisničkim interfejsom. Na primjer, aplikacija za elektronsku poštu može imati jednu aktivnost koja prikazuje listu nove pošte, drugu aktivnost za sastavljanje elektronske pošte i treću aktivnost za čitanje e-pošte. Iako aktivnosti rade nad istom aplikacijom, svaka od njih je nezavisna od ostalih. Stoga, druga aplikacija može pokrenuti bilo koju od ovih aktivnosti ako aplikacija za e-poštu to dozvoljava. Na primjer, aplikacija za kameru može započeti aktivnost u aplikaciji za e-poštu koja sastavlja novu poštu kako bi omogućila korisniku da dijeli sliku. Aktivnosti u Java programu, pišu se kao klase koje nasljeđuju klasu `Activity`. Nakon toga se eksportuju u manifestu kroz XML element <activity> unutar elementa <application>.

```
<manifest ... >
    <application ... >
        <activity android:name="com.example.myapplication.MainActivity" ... >
        </activity>
    </application>
</manifest>
```

Usluga je opšta ulazna tačka održavanje aplikacije da radi svo vrijeme u pozadini. To je komponenta koja radi u pozadini za obavljanje dugotrajnih operacija ili za obavljanje posla za daljinske procese. Usluga ne pruža korisnički interfejs. Na primjer, usluga može da reprodukuje muziku u pozadini dok je korisnik radi sa drugom aplikacijom ili može izvlačiti podatke preko mreže bez blokiranja interakcije korisnika sa aktivnošću. Druga komponenta, kao što je aktivnost, može pokrenuti uslugu i sarađivati s njom. Usluge u Java programu pišu se kao klase koje nasljeđuju klasu `Service` i navode u manifestu kroz XML element <service> unutar elementa <application>.

Prijemnik je komponenta koja omogućava sistemu da isporučuje događaje u aplikaciju izvan redovnog protokola korisnika, omogućavajući aplikaciji da odgovara na sistemske programe. Pošto su prijemnici još jedan ulaz u aplikaciju, sistem može da emituje događaje čak i na aplikacije koje trenutno nisu pokrenute. Prijemnik se

u Java programu realizuje kao klasa koja nasljeđuje klasu `BroadcastReceiver` i svako obavještenje se šalje kao `Intent` objekt. U manifestu se za svaki registrovani prijemnik navodi koju vrstu događaja će obrađivati.

```
<receiver android:name="com.example.myapplication.Receive">
  <intent-filter>
    <action android:name="android.intent.action.BATTERY_LOW"></action>
  </intent-filter>
</receiver>
```

Pružalac sadržaja upravlja zajedničkim skupom podataka koje aplikacija može spremati u datotečnom sistemu, u bazi podataka `SQLite`, na webu ili na bilo kojoj drugoj stalnoj lokaciji za skladištenje kojoj aplikacija može pristupiti. Kroz davaoca sadržaja, druge aplikacije mogu da pošalju ili izmijene podatke ako su vezani sa davaocem sadržaja. Na primjer, foto aplikacija može da pristupa adresaru..

U manifestu se navodi i koje usluge uređaja aplikacija zahtijeva (pristup Internetu, telefon, GPS i slično).

Android je jednokorisnički operativni sistem, ali ipak koristi Unix-ove korisničke identifikatore. Zapravo koristi ih intenzivnije nego klasični Unix ili Linux. Za razliku od običnih Linux distribucija gdje se sve aplikacije koje je pokrenuo isti korisnik izvršavaju pod njegovim pravima, ovdje svaka aplikacija ima svoj korisnički identifikator (UID). Kada je instalirana Android aplikacija, za nju se stvara novi jedinstveni korisnički ID (ili UID), a sav njen kôd se pokreće kao taj „korisnik“. Korisnički ID-ovi Linux-a stoga stvaraju pješčaničnik za svaku aplikaciju, sa svojim vlastitim izolovanim područjem datotečnog sistema. Na ovaj način se smanjuje rizik od zlonamjernog koda i slučajnih grešaka korisnika da obriše sve aplikacije.

Iznad menadžera paketa nalazi se još jedan važan sistemski servis, menadžer aktivnosti. Iako je menadžer paketa odgovoran za održavanje statičkih informacija o svim instaliranim aplikacijama, menadžer aktivnosti određuje kada, gdje i kako se te aplikacije trebaju pokrenuti. Uprkos svom imenu, zapravo je odgovoran za pokretanje sve četiri vrste komponenti aplikacija i implementaciju odgovarajućeg ponašanja za svaku od njih. Menadžer aktivnosti je također odgovoran za određivanje kada procesi više nisu potrebni. On prati sve aktivnosti, prijemnike, servise i pružaoce sadržaja koji su pokrenuti u procesu. Iz ovoga može se odrediti koliko je proces važan ili nije važan.

17.2. Apple iOS

Apple iOS je sistem koji se koristi na iPhone mobilnim telefonima, iPad tabletima i Apple TV. Ovaj operativni sistem se koristi samo na Apple uređajima. Korisničko sučelje iOS bazira na gesturama, posebnim pokretima ruke preko dodirnog ekrana.

Kako pokazuje Slika 200, Apple iOS koristi četiri sloja apstrakcije. Donji slojevi sadrže osnovne usluge i tehnologije. Slojevi višeg nivoa pružaju više sofisticirane usluge.

Cocoa Touch
Mediji
Osnovni OS
Osnovne usluge

Slika 200 Arhitektura Apple iOS

Core OS sadrži Unix Mach jezgro, radne okvire za sigurnost, digitalnu obradu signala i softver koji omogućava priključenje dodatnih uređaja. Core Services pruža podršku za dijeljenje datoteka, izvršavanje zadataka, nabavku aplikacija, računarstvo u oblaku, podršku za XML, JavaScript itd. Media sloj uključuje većinu grafičkih, audio i video tehnologija. Cocoa Touch predstavlja implementaciju korisničkog sučelja za mobilne uređaje.

17.3.Drugi operativni sistemi za mobilne uređaje

Ogromna većina mobilnih uređaja koristi Android ili iOS. Neki stariji mobiteli ili novi u najjeftinijoj kategoriji koriste ili su koristili i druge sisteme.

Palm OS je bio jedan od prvih operativnih sistema za mobilne uređaje. Dizajniran za jednostavnu upotrebu sa dodirnim ekranima namijenjen je uređajima za lične podatke, a kasnije proširen za mobitele.

Mobilni uređaji sa Windows operativnim sistemom prošli su kroz dvije vrste arhitektura. Verzije Windows Mobile i Windows Phone do verzije 7 bile su bazirane na Windows CE jezgru, dok je Windows Phone baziran na Windows NT jezgru. Korisničko sučelje je slično onom u Windows 8, Metro, odnosno Windows 10. Ugrađene su mobilne verzije aplikacija Internet Explorer, Microsoft Office, Contacts. U trenutku pisanja ove knjige najavljeno je da se Windows 10 mobile neće dalje razvijati.

Tizen je kao i Android baziran na Linux jezgru, ali za razvoj aplikacija umjesto Java platforme koristi HTML 5.

Nokia OS koristi se na manjim Nokia mobilima. Razvijen u više generacija, S30, S30+ i S40. Sistem je jednog procesni, brz za ugrađene aplikacije, ali za eksterne aplikacije podržava jedino relativno sporu JavaME platformu.

Symbian OS je nastao iz Psion EPOC, i bio je dosta korišten u Nokia sistemima do 2012. Godine 2008 je predstavljao 50% tržišta mobilnih operativnih sistema, ali četiri godine kasnije se gotovo ugasio. Jezgro ovog operativnog sistema je mikro-kernel tipa sa serverima koji komuniciraju preko poruka. Objekti jezgra su predstavljeni handle-ovima koji nemaju veze s internom strukturom (npr. kod Unix-a oni često predstavljaju pokazivače).

BlackBerry OS kao jezgro koristi QNX, prvi uspješni operativni sistem s pravom mikro-kernel arhitekturom. Ovo jezgro se pokazalo kao efikasnije od Mach jezgra, jer se slanje poruka između servera obavlja bez pozivanja raspoređivača, tako da se poruke mogu obraditi brže. Korisničko sučelje ovog operativnog sistema je zasnovano na gesturama i čini tastaturu nepotrebnom.

18. IZVORNI KÔD MALOG JEZGRA

U svrhu boljeg razumijevanja principa operativnih sistema razvijeno je minimalističko jezgro operativnog sistema, pod imenom LEKOS (*Layered Educational Kernel of Operating System*). Ovo jezgro je namijenjeno za PC računare, sa IDE diskom, procesorom 80386 ili novijim, tastaturom i tekstualnim ekranom. Može se pokrenuti pod emulatorom qemu, ili na realnoj mašini. Izuzev *boot* sektora (koji je u asemblerskom jeziku, jer mora biti vrlo kratak) napisano je u jeziku C. Jezgro je organizovano slojevito iz metodičkih razloga. Svaka nova datoteka izvornog koda predstavljena u ovoj knjizi može da koristi prethodne, ali uz dodatak *main* funkcije sa njima može predstavljati i zaokruženu cjelinu.

18.1. Boot sektor

Ovo je jedini potprogram koji je napisan u asemblerskom jeziku. Boot sektor mora biti velik 512 bajta i završiti se signaturom. Sastoji se iz dva osnovna dijela, 16 bitni i 32 bitni. Šesnaest-bitni dio koda, koji se izvršava na adresi 000:7c00 najprije dva puta poziva BIOS *interrupt* 13h. Ovom pozivu se kao parametar prosljeđuju vrijednosti u registrima. Pri prvom pozivu se kontroler diska resetuje, a pri drugom pozivu se prebaci određen broj sektora sa diska u RAM. U drugoj polovini šesnaest-bitnog dijela se učitava GDTR registar da pokazuje na tabelu koja jednostavno mapira segmente u linearno adresiranje.

```
[BITS 16]          ; Za realni rezim 16 bitne instrukcije
[ORG 0x7C00]       ; BIOS učitava boot sektor na 0x7C00
reset_drive:
    mov ah, 0          ; RESET komanda kontroleru
    int 13h           ; Pozovi interapt 13h
    or ah, ah          ; Provjeri kôd greške
    jnz reset_drive    ; Pokušaj opet ako ah != 0
    mov ax, 0
    mov es, ax
    mov bx, 0x1000      ; Sektor će se učitati na = 0000:1000
    mov ah, 02h         ; READ SECTOR-komanda
    mov al, 51          ; Koliko sektora čitamo - 51
    mov ch, 0           ; Cilindar = 0
    mov cl, 2h          ; Sektor = 2,
    mov dh, 1           ; na hard disku prva particija
    int 13h            ; Bios poziv za direktno čitanje diska
    or ah, ah          ; Provjeri kôd greške
    jnz reset_drive    ; Pokušaj opet ako ah != 0
    cli                ; Isključi interapt
    xor ax, ax
    mov ds, ax          ; Postavi DS-registar na 0 za lgdt
    lgdt [gdt_desc]     ; Učitaj GDT registar
    mov eax, cr0         ; Pročitaj CR0 u EAX
    or eax, 1           ; Najniži bit protected mode
    mov cr0, eax        ; Vрати CR0, protected mode
    jmp 08h:clear_pipe   ; Skoči u kodni segment,
```

32-bitni dio sada postavi segmentne registre, stack pointer i skače u jezgro.

```
[BITS 32] ; Sada smo u 32 bitnom režimu
clear_pipe: mov ax, 10h ; Deskriptor 2 (*8=10h)
            mov ds, ax ; ide u data segment
            mov ss, ax ; ide u stack segment
            mov esp, 090000h ; Postavi stek pointer na 090000h
            jmp 08h:01000h ; Skoči u segment 08h (code), offset 01000h
```

Sada slijedi tablica deskriptora. Mala je, sastoji se od samo tri deskriptora, jednog praznog, jednog za kôd i jednog za podatke.

```
gdt: ; Adresa GDT
gdt_null: dd 0 ; Prazan segment dd 0
gdt_code: dw 0FFFFh ; limit FFFFFFFF
            dw 0 ; bazna adresa 0
            db 0 ; bazna adresa 0
            db 10011010b ; Code segment, read/execute, nonconforming
            db 11001111b ; limit FFFFFFFF
            db 0 ; bazna adresa 0
gdt_data: dw 0FFFFh ; limit FFFF
            dw 0 ; bazna adresa 0
            db 0 ; bazna adresa 0
            db 10010010b ; Data segment, read/write, expand down
            db 11001111b ; limit FFFF
            db 0 ; bazna adresa 0
gdt_end: ; Račun veličine GDT
gdt_desc: ; GDT deskriptor
            dw gdt_end-gdt-1; Limit (veličina)
            dd gdt ; Adresa of the GDT
times 510-($-$$) db 0 ; Ostatak sektora popuni nulama
dw 0AA55h ; Signatura boot sektora
```

18.2.Podprogrami za i/o portove

Za direktno očitavanje ulazno/izlaznih portova mora se imati malo ubačenog asemblerskog koda, jer ove rutine nisu dio C jezika. Sljedeće četiri rutine (datoteka ports.c) su omotač oko odgovarajućih mašinskih instrukcija i omogućavaju pristup 16 bitnim i 8 bitnim portovima. Ovdje se koristi jedno proširenje GNU C kompajlera, naredba `__asm__` koja ubacuje rutine u asemblerskom jeziku u C program.

```
// in, Čitanje sa osmobitnog porta
unsigned char in(unsigned short _port) {
    unsigned char result;
    // Reci GNU kompajleru da generiše instrukciju IN AL,DX
    // pri čemu će result biti mapitan na AL a _port na DX
    __asm__ ("in %dx, %%al" : "=a" (result) : "d" (_port));
    return result;
}

// out, slanje na osmobitni port
void out(unsigned short _port, unsigned char _data) {
    // Reci GNU kompajleru da generiše instrukciju OUT DX,AL
    // pri čemu će _data biti mapitan na AL a _port na DX
    __asm__ ("out %%al, %dx" : : "a" (_data), "d" (_port));
}
```

```
// Čitaj sa 16 bitnog porta
unsigned short inw(unsigned short _port) {
    unsigned short result;
    // Reci GNU kompajleru da generiše instrukciju IN AX,DX
    // pri čemu će result biti mapitan na AL a _port na DX
    __asm__ ("inw %dx, %ax" : "=a" (result) : "d" (_port));
    return result;
}

void outw(unsigned short _port, unsigned short _data) {
    // Reci GNU kompajleru da generiše instrukciju OUT DX,AX
    // pri čemu će _data biti mapitan na AX a _port na DX
    __asm__ ("outw %ax, %dx" : : "a" (_data), "d" (_port));
}
```

18.3. Video drajver

Prvi dio jednostavnog video drajvera (datoteka video.c) se bavi pozicioniranjem kursora. Pozicija kursora se čuva u varijabli *cursorpos*, koja sadrži 80*red+kolona. Grafička kartica ima registre kojima se pristupa preko portova 0x3D4 i 0x3D5, kojima se uključuje tekstualni kursor. U njih treba upisati *cursorpos*.

```
short cursorpos;
// hardwarecursor mijenja poziciju tekstualnog kursora
void hardwarecursor() {
    out(0x3D4, 0x0F); // Mijenjamo niži bajt adrese hardverskog kursora
    out(0x3D5, (char)(cursorpos&0xFF)); // Upisujemo ga
    out(0x3D4, 0x0E); // Mijenjamo viši bajt adrese hardverskog kursora
    out(0x3D5, (char)((cursorpos>>8)&0xFF)); // Upisujemo ga
}
// SetCursor računa poziciju tekstualnog kursora od koordinata
void SetCursor(short row, short col) {
    cursorpos=((row*80) + col); // Izračunaj
    hardwarecursor(); // Na novu poziciju
}
// Vрати trenutnu vrijednost pozicije kursora
void GetCursor(short * row, short * col) {
    *row = cursorpos/80; // Podijeli poziciju sa 80 i vrati kao red
    *col=cursorpos % 80; // Kolona je ostatak dijeljenja pozicije s 80
}
```

Ispis teksta na ekran se obavlja petljom koja ubacuje ASCII kodove poruke, preskačući attribute u video memoriji.

```
void Print(const char *_message) {
    unsigned long i; // Brojač petlje
    unsigned char *vidmem; // Video memorija
    vidmem= (unsigned char *)0xB8000; // Adresa video memorije
    vidmem += cursorpos<<1; // Počni ispis od kursora*2, znak zauzme 2 bajta
    i = 0; // brojač na početak petlje
    while (_message[i] != 0) { // Nastavi do praznog znaka
        *vidmem = _message[i++]; // Prebaci znak u video memoriju
        vidmem += 2; // Pređi na narednu poziciju u video memoriji
    }
    cursorpos += i; // Kursor na novu poziciju
    hardwarecursor(); // Iscrtaj hardverski kursor
}
```

Korisno je imati i rutinu koja ispisuje cijeli broj.


```

#define LSHIFT 0x2a // Sken kôd lijevog shift-a
#define RSHIFT 0x36 // Sken kôd desnog shift-a
#define LALT 0x38 // Sken kôd Alt tastera
#define LCTL 0x1d // Sken kôd lijevog ctrl tastera
#define CAPSL 0x3a // Sken kôd CAPS Lock tastera
#define EXTESC 0xe0 // Prvi bajt dvobajtnog sken koda
#define INCTL 0x01 // Pritisnut ctrl
#define INSHIFT 0x02 // Pritisnut Shift
#define CAPSLOCK 0x04 // Pritisnut Caps lock
#define INALT 0x08 // Pritisnut Alt
#define EXTENDED 0x10 // Pritisnut Taster sa dvobajtnm sken kodom
#define NOCHAR 0 // Nema tastera
static int state; // Status tastature */
// extchar izlazi iz režima dvobajtnog sken koda
static int extchar(unsigned char code) {
    return state &= ~EXTENDED; // Ugasi EXTENDED bit
}
// Preko tabele konvertuje Scan u ASCII kôd
unsigned char ScanToAscii(unsigned char code) {
    unsigned int ch;
    if (state & EXTENDED) // Ako je prošli znak dvobajtni
        return extchar(code); // ne vraćamo ASCII
    if (code & 0x80) { // Otpuštanje ima scan kôd uvećan za 0x80
        switch (code & 0x7f) { // Tada obrišemo najviši bit
            case LSHIFT: // Otpušten Shift
                state &= ~INSHIFT; break; // Gasi stanje Shift
            case CAPSL: state &= ~CAPSLOCK; break; // Otpušten CapsLock gasi
            case LCTL: state &= ~INCTL; break; // Otpušten Ctrl gasi stanje Ctrl
            case LALT: state &= ~INALT; break; //Otpušten Alt gasi stanje ALT
        }
        return NOCHAR; // Otpuštanje nas ne interesuje inače
    }
    switch (code) { // Ako je pritisnut neki od posebnih tastera
        case LSHIFT:
            case RSHIFT: state |= INSHIFT; return NOCHAR; // U stanje Shift
            case CAPSL: state |= CAPSLOCK; return NOCHAR; // U stanje Caps
            case LCTL: state |= INCTL; return NOCHAR; // U stanje Ctrl
            case LALT: state |= INALT; return NOCHAR; // U stanje Alt
            case EXTESC: state |= EXTENDED; return NOCHAR; // Dvobajtni scan kôd
    }
    ch = NOCHAR; // Za početak ništa nije pritisnuto
    if (code < sizeof(kbcode)) { // Scan kôd ima u tabeli
        if (state & CAPSLOCK || state & INSHIFT) // U Caps ili shift stanju
            ch = kbshift[code]; // Uzmi iz Shift konverzije tabele
        else
            ch = kbcode[code]; // Iz obične ako nije CAPS lock ili Shift
        if (state & INCTL) // U Ctrl stanju uzmi iz kbctl tabele
            ch = kbctl[code];
        if (state & INALT) ch += 0x80; // U Alt stanju uvećaj kôd za 0x80
    }
    return ch;
}

```

18.5.Drajver diska

Implementiran je samo IDE disk, kao najjednostavnija vrsta diska za pristup. Kod ove vrste diskova koriste se portovi 0x1f1 do 0x1f7 na koje se može poslati LBA ili CHS adresa prvog sektora u grupi sektora koji se želi prebaciti, te koliko se sektora prebacuje. Slanjem vrijednosti na port 0x1f7 se aktivira komanda kojom započinje prijenos u jednom ili drugom pravcu. Nakon toga se mogu očitavati vrijednosti ili upisivati.

```
// Izabira sektor slanjem na I/O portove
void SetFirstSector(int first_sector) {
    out(0x3f6,2); //ne koristimo interapte i DMS
    out(0x1f2,1); // Port 1f2 koliko sektora: 1
    out( 0x1f3,first_sector & 0xff ); // Početni sektor se upiše
    out( 0x1f4,(first_sector>>8) & 0xff ); // kao 28 bitni broj
    out( 0x1f5,(first_sector>>16) & 0xff );// na četiri porta
    out( 0x1f6,((first_sector>>24) & 0x0f)|0xe0 ); //sec+broj diska
}
// ReadHDBlock rutina čita num sektora počevši od first_sector u na
//lokaciju buf
int ReadHDBlock( void* buf, int first_sector, int num ){
    int i; // Brojač
    unsigned short* dat = (unsigned short*) buf; // Pointer na buffer
    while(num > 0 ) { // Petlja sektor po sektor
        while( in(0x1f7) & 0x80 ){ //Kada disk bude spreman, javiće na port
            SetFirstSector(first_sector); // Prvi sektor
            out(0x1f7,0x20); //Kaži disk kontroleru da šalje bajtove
            // Disk je spreman kada je 7. Bit porta 1f7 jednak 0 a 3. bit je 1
            while( in(0x1f7) & 0x80 ) {} //Čekaj da je disk spreman
            while( (in(0x1f7) & 0x08) == 0 ) {} //Čekaj da dođu podaci
            for(i=0;i<256;++i) // IDE je 16 bitni interfejs, dolaze 2 po 2 bajta
                *dat++ = inw(0x1f0); // Pročitaj 16 bita u memoriju
            --num; ++first_sector; // Pređi na sljedeći sektor
        }
        return 1; // Vрати se
    }
}
// WriteHDBlock rutina piše num sektora počevši od first_sector sa
// lokacije buf
int WriteHDBlock( void* buf, int first_sector, int num ){
    int i; // Brojač
    unsigned short* dat = (unsigned short*) buf; // Pointer na buffer
    while(num > 0 ){ // Petlja sektor po sektor
        while( in(0x1f7) & 0x80 ){ //Kada disk bude spreman, javiće na port
            SetFirstSector(int first_sector); // Izaberi prvi sektor
            out(0x1f7,0x30); //Javi kontroleru da prima bajtove
            while( in(0x1f7) & 0x80 ){ // Čekaj da disk bude spreman
                for(i=0;i<256;++i){ // Sektor je 512 bajta, prebacuje se 256*2
                    outw(0x1f0,*dat); dat +=1; // Naredna 16 bitna riječ
                }
                --num; ++first_sector; // novi sektor
            }
        }
        return 1; //Kraj rutine
    }
}
```

18.6. Obradivači interapta

U zaštićenom režimu rada registar IDTR pokazuje na tabelu interapt vektora. Svaki element te tabele se sastoji od osam bajta. Postavljanje početka te tabele se obavlja naredbom LIDT koja je upakovana u C potprogram.

```
struct IDTDescr{ // Tablica interapta ima element od 8 bajtova
    unsigned short offset_1; // Biti 0..15 ofsetnog dijela adrese
    unsigned short selector; // izbor kodnog segmenta u GDT ili LDT
    unsigned short reserved : 5; // rezervisani biti
    unsigned short signature : 8; // oznaka vrste deskriptora
    unsigned short dpl : 2; // predviđeni nivo privilegija
    unsigned short present : 1; // Prisutan u memoriji ili ne
    unsigned short offset_2; // Biti 16..31 ofsetnog dijela adrese
};

struct IDTDescr * InterruptTable; // Pokazuje na tablicu interapt rutina
// Rutina lidt inicijalizira tablicu interapta
static inline void lidt(void* base, unsigned short size) {
    struct {
        unsigned short length; // U ovoj strukturi su dužina i
        unsigned int base; // početna adresa tabele interapta
    } __attribute__((packed)) IDTR;
    IDTR.length = size; // Popuni veličinu tabele interapta
    IDTR.base = (unsigned int) base; // Popuni početnu adresu
    // Asemblerska instrukcija lidt će popuniti IDTR registar koji
    // će iz memorije uzeti početnu adresu tabele interapta i njenu veličinu
    __asm__ ( "lidt (%0)" : : "p"(&IDTR) );
}
// Sljedeća dva podprograma pozivaju asemblerske instrukcije
void enable_interrupts () {
    __asm__ ("sti");//Uključuje interapte, postavi interapt fleg
}
void disable_interrupts () {
    __asm__ ("cli");//Isključi interapte, obriši interapt fleg
}
}
```

Rani interapt kontroleri na PC su postavljali vektore za obradu hardverskih zahtjeva počevši od IRQ0 na vektore interapta od 8 nadalje. No od procesora 80386 ti vektori se koriste za druge svrhe, pa je potrebno premapirati interapt vektore, da se hardverski zahtjevi obrađuju od vektora 32 nadalje.

```
void SetCmdWords(char first,char second) { // Upisuje komandne riječi na
    out(0x21,first); out (0x80,0); // Prvu riječ na prvi kontroler
    out(0xA1,second); out (0x80,0); // Drugu riječ na drugi kontroler
}
// PIC_remap pomijera tablice interapta kontrolera
void PIC_remap(int offset1, int offset2) {
    unsigned char a1, a2;
    a1 = in(0x21); a2 = in(0xA1); // Pročitaj staro stanje
    out(0x20, 0x10+0x01); out (0x80,0);// Predefiniši interapte
    out(0xA0, 0x10+0x01); out (0x80,0); // Nakon ovoga šaljemo 4 podatka
    SetCmdWords (offset1,offset2); //za koliko interapt vektora se pomijera
    SetCmdWords (4,2); // IRQ za oba kontrolera
    SetCmdWords (1,1); // 8086/88 (MCS-80/85)
    SetCmdWords (a1,a2); } // vrati sačuvane maske
```

Sljedeća rutina puni dijelove interapt vektora.

```
// Parametri su broj rutine i pokazivač na funkciju koja obrađuje
void SetCustomInterrupt(int intnum, void (*fp)(int)){
    unsigned int ad; // adresa rutine
    struct IDTDescr * myint; // Element tabele interapta
    myint=InterruptTable+intnum; // pozicija u tabeli interapta
    ad=(unsigned int) fp; // Pročitaj adresu
    myint->offset_1=ad & 0xFFFF; // Upiši u tabelu bite adrese 0..16
    myint->offset_2=ad >> 16; // Upiši u tabelu bite adrese 16..31
    myint->selector=8; // Kodni segment je 8
    myint->signature=0x70; // Signatura 70h je interapt kapija
    myint->reserved=0; myint->dpl=0; myint->present=1;
}
```

Sada slijedi implementacija rutina koje se izvršavaju u interaptima. Format C potprograma nije jednak formatu interapt rutine. Zato rutine koje se izvršavaju u interaptima imaju određene asemblerske instrukcije na početku i kraju kako bi imale pravilni format steka. Prvo će se realizovati tajmer interapt. Njegov zadatak je jednostavan, uvećava brojač za 1 svakih 10 ms.

```
int elapsedtime; // Varijabla koja mjeri pređeno vrijeme
// Sljedeći potprogrami postavljaju brojač vremena
void SetTime(unsigned int t) {
    elapsedtime=t;
}
unsigned int ReadTime() {
    return elapsedtime;
}
// Rutina se poziva iz interapta, na svaki tajmer otkucaj
void Clock() {
    __asm__ __volatile__ ("pushal"); // Čuva sve registre na steku
    SetTime(ReadTime()+1); // Uvećaj vrijeme za 1
    out(0x20,0x20); // Pošalji komandu kraj interapta
    __asm__ __volatile__ ("popal ; leave ; iret"); // vrati registre
}
void InitTimer() { // Ovom rutinom se inicijalizira tajmer
    unsigned int tmp = 1193180 / 100; // 100 Hz ili 10 ms
    out(0x43, 0x36); // Naredi da se mijenja Frevencija tajmera 0
    out(0x40, (unsigned char) tmp); // niži bajt
    out(0x40, (unsigned char) (tmp >> 8)); // viši bajt
    out(0x21,in(0x21) & 0xfe); // Promijeni masku da tajmer pravi interapt
    elapsedtime=0;
    SetCustomInterrupt(32,Clock); // Vektor 32 se odnosi na Clock potprogram
}
```

Sljedeća interapt rutina je vezana za očitavanje tastature. Primljeni sken kôd tastature se smješta u kružni buffer. Rutina za očitavanje tastature preuzima znak iz tog buffera.


```

char kdbuffer[256];
unsigned int readchar,storedchar; // Pozicija pročitano i ubačeno bajta
// Ova rutina se poziva kada je pritisnut taster
void kbdirq() {
    static unsigned char c;
    __asm__ __volatile__ ("pushal; "); // Zapamti registre
    c=GetScanCode(); // Uzmi sken kôd
    if (c != 0) { // Ako je regularni taster
        storedchar=(storedchar+1) % 256; // Stavi ga u kružni buffer
        kdbuffer[storedchar]=c;
    }
    out(0x20,0x20); // Signaliziraj kraj interapta
    __asm__ __volatile__ ("popal; leave ; iret");// Vрати registre, izađi
}
// Narednom funkcijom čita se sken kôd iz buffera
unsigned char readintscancode() {
    if (readchar!=storedchar) { // Ako se razlikuju pozicije ima novih
        readchar=(readchar+1) % 256; // Naredna pozicija pročitano bajta
        return kdbuffer[readchar]; // vrati znak
    }
    else return 0;
}
// Sljedeći potprogram inicijalizira interapt tastature
void SetKeyboardInterrupt(){
    SetCustomInterrupt(33,kbdirq); // Interapt broj 33 za kbdirq rutinu
    out(0x21,in(0x21) & 0xfd); // Postavi masku da se prima interapt
    readchar=storedchar=0; // Pozicije buffera na 0
}

```

Inicijalizacija interapt sistema postavlja interapt tabelu i aktivira vektore.

```

void InitInterrupts() {
    out(0x21,0xff); out(0xa1,0xff); // Maske oba kontrolera na neaktivno
    InterruptTable=(void *)0; // Interapt tabela na sam početak memorije
    lidt(InterruptTable, 2048); // Popuni je sa 256 vektora
    PIC_remap(32,40); // Vektori 1. kontrolera počinju od 32, drugog od 40
    InitTimer(); // Tajmer interapt
    SetKeyboardInterrupt(); // Interapt tastature
    enable_interrupts (); // Uključi interapte
}

```

18.7.Upravljanje memorijom

U modulu za upravljanje (memory.c) memorijom implementirane su dvije strategije, za fizičku memoriju i virtualnu memoriju. Fizička memorija se upravlja *first fit* algoritmom. Fizički blok memorije je predstavljen elementom *memtable*.

```

#define TABLELEN 200
#define EOUTMEMORY -1
// Lista kontrolnih blokova memorije (MCB) je tabela struktura table
struct table {
    unsigned int startaddress; // Početna adresa bloka
    unsigned int length:31; // 31 bit dužina bloka
    unsigned int busy:1; // 1 bit da li je blok slobodan
} * memtable;
int maxused=0; // Koliko elemenata ima tabela

```

Inicijalizacija memorijskog podsistema postavlja tablicu elemenata fizičke memorije na jedan blok od 128 megabajta, počevši od adrese 1 M. Područje ispod 1M koristi kernel.

```
// Init memory pravi početni memorijski kontrolni blok koji će dijeliti
void initmemory() {
    maxused=0; // Nula elemenata
    memtable=(void *) 0x20000; // Tabela se smješta na adresu 0x20000
    memtable[0].startaddress=1024*1024; // Predvidimo da je memorija
    memtable[0].length=128*1024*1024; // 128 M, počevši od adrese 1M
}
```

AllocMemory će onda dodijeliti prvi slobodan blok. Fizička memorija se dodjeljuje kontinualno, first fit algoritam, neiskorišteni dio šupljine postaje nova šupljina.

```
// Dodjeljuje memoriju procesu
void * AllocMemory(int size) {
    int i;
    if (maxused>=TABLELEN) // Ako su svi MCB zauzeti
        return (void *) EOUTMEMORY; // ne može se alocirati
    for (i=0; i<=maxused; i++) { // Kreni kroz sve MCB
        // First fit: prvi blok koji je dovoljne veličine i slobodan
        if ((memtable[i].length>=size) && (memtable[i].busy==0)) {
            memtable[i].busy=1; // Nađen, pa ga alociraj
            if (memtable[i].length>size) { // Ako tražimo više od veličine
                maxused++; // dodajemo novi kontrolni blok
                memtable[maxused].busy=0; // on je slobodan
                memtable[maxused].startaddress=
                    memtable[i].startaddress+size; // a počinje iza zauzetog
                // dužina novog bloka će biti dužina presječenog - alocirano
                memtable[maxused].length=memtable[i].length-size;
                memtable[i].length=size; // dužina alociranog
            }
            return (void *) memtable[i].startaddress; // vrati adresu bloka
        }
    }
    return (void *) EOUTMEMORY; // Ni jedan slobodan blok, nema memorije
}
```

Rutina za oslobađanje zauzetog bloka fizičke memorije brine o ukрупnjavanju susjednih šupljina, gledajući sva četiri slučaja položaja postojećih i nove šupljine.

```

// FreeMemory Oslobađamo memoriju na adresi addr
void FreeMemory(void * addr) {
    unsigned int i;
    unsigned int req,prev,next; // indeks Traženi, blok ispred i blok iza
    unsigned int npos; // Memorijska adresa traženog bloka
    req=prev=next=-1;
    for (i=0; i<=maxused; i++) // Kreni kroz tabelu memorijskih blokova
        if (memtable[i].startaddress==(unsigned int)addr &&
            memtable[i].busy==1) { //Našli smo zauzeti blok na toj adresi
            req=i; // Zapamti koji je blok u pitanju
            npos=memtable[i].startaddress+memtable[i].length;
            break;
        }
    if (req !=-1) { // Našli smo blok. Ima li praznih iza i ispred?
        for (i=0; i<=maxused; i++) { // Prođi kroz tabelu ponovo
            if (memtable[i].startaddress==npos && memtable[i].busy==0)
                next=i; // Ima slobodni blok čija adresa je odmah iza brisanog
            if (memtable[i].startaddress+memtable[i].length==
                (unsigned int)addr && memtable[i].busy==0)
                prev=i;//Ima slobodni blok čija adresa je odmah ispred brisanog
            if (prev!=-1 && next !=-1)
                break; // Skrati traženje ako su nađena oba susjedna bloka
        }
        memtable[req].busy=0; // Oslobađamo blok
        if (next != -1) { // Ako je iza njega slobodan blok
            memtable[req].length += memtable[next].length; // Povećaj brisani
            memtable[next]=memtable[maxused]; // U sljedbenika prebaci zadnji
            if (prev==maxused) // Specijalni slučaj, prethodnik je zadnji
                prev=next; // prethodnik ide u oslobođenog sljedmenika
            maxused--; // smanjujemo broj memorijskih kontrolnih blokova
        }
        if (prev != -1) { // Ako je ispred njega slobodan blok
            // Povećaj blok prethodnika
            memtable[prev].length += memtable[req].length;
            memtable[req]=memtable[maxused]; //Stavi zadnji umjesto obrisano
            maxused--; // smanjujemo broj memorijskih kontrolnih blokova
        }
    }
}

```

Procesi mogu da vide memoriju drugačije. Koristi se straničenje na dva nivoa. Adresni prostor svakog procesa od 4 megabajta se vidi kao da je između adresa 0 i 4M, a mapira se na odgovarajuću fizičku adresu iznad 1M. Prvi megabajt procesa je mapiran na kernel, a ostala 3 na memoriju iznad 1M. Kako Intelovi procesori koriste straničenje na 2 nivoa, potrebna su dva elementa u tabeli. Prvi megabajt se mapira svim procesima jednako.

```
// Virtualni adresni prostor od 4M mapiraće se da logička adresa 0
// Počinje od fizičke adrese koja je data kao drugi parametar funkcije
// MapVirtualSpace. Prvi parametar sadrži tabelu stranica za dati proces
void MapVirtualSpace(void * tableaddr,void * useraddr) {
    unsigned int *page_directory, *first_page_table, i,address;
    page_directory=tableaddr; // Imenik stranica na navedenu adresu
    Address=0; // Počni od adrese 0
    for(i = 1; i < 1024; i++) // Imenik stranica ima 1024 elementa
        page_directory[i] = 0 | 2;//Svi osim prvog su not present
    first_page_table = page_directory + 1024;// Tabela stranica je
    //odmah iza imenika stranica, nulti element pokazuje na nju
    page_directory[0] = (unsigned int)first_page_table;
    page_directory[0] |= 3;//supervisor level, read/write, present
    for(i = 0; i < 1024; i++) {
        first_page_table[i] = address | 3; // Element tabele pokazuje adresu
        // Ove stranice su prisutne u memoriji
        address = address + 4096; //Nova 4 kilobajta
        if (i==255) address=(unsigned int) useraddr; // od 255 stranice
    } // mapira se na korisničku memoriju
}
```

Naredne dvije funkcije mijenjaju registre CR0 i CR3 i time mijenjaju režim straničenja na uključen ili isključen.

```
// Uključi straničenje i postavi tabelu stranica na page_directory
void ActivatePaging ( int *page_directory) {
    static unsigned int cr0;
    //postavi CR3 na page_directory
    asm volatile("mov %0, %%cr3:: \"b\"(page_directory));
    // Pročitaj registar CR0 i postavi njegov najviši bit na 1
    asm volatile("mov %%cr0, %0:: \"b\"(cr0));
    cr0 |= 0x80000000;
    asm volatile("mov %0, %%cr0:: \"b\"(cr0));
}
// Ovim se isključuje straničenje
void LinearMode ( ) {
    unsigned int cr0;
    // Pročitaj registar CR0 i postavi njegov najviši bit na 0
    asm volatile("mov %%cr0, %0:: \"b\"(cr0));
    cr0 &= 0x7FFFFFFF;
    asm volatile("mov %0, %%cr0:: \"b\"(cr0));
}
```

18.8.Datotečni sistem

Datotečni sistem je dizajniran da bude što jednostavniji. Stoga je implementirana direktorijska struktura na jednom nivou, sa kontinualnom alokacijom. Ipak, strukture podataka koje opisuju elemente direktorija pored podataka o imenu i poziciji datoteke na disku, posjeduju i polje directory koje omogućuje proširenje na direktorije s dva ili više nivoa. Struktura direntry sadrži podatke o svim datotekama na disku, a openentry o otvorenim.

```
#include "string.h"
// Particija počinje boot blokom iza koga je kernel, pa direktorijska
// struktura.
// Element direktorija može biti zadnji, obrisana datoteka, obična
enum filetypes {ENDFS,FREEZONE,REGULARFILE};
enum filedescriptorstatus {OPEN,CLOSED}; // kada je dir. u memoriji
enum errors {OK,EINVALID,EFULLDISK,ENOMOREFILES,EFILENOTFOUND};
#define NAMELEN 47
unsigned int startfs,maxfiles,maxopenfiles,
    endfs,dirtableondisk,dataoffset;
struct direntry{ // Element direktorija na disku
    enum filetypes entrytype; // Jedan od tipova datoteka
    unsigned char name[NAMELEN]; // Ime datoteke
    unsigned int startblock; // Početni blok
    unsigned int maxsizeinblocks; // Maksimalna veličina
    unsigned int actualsizeinbytes; // Stvarna veličina
    unsigned int directory; // Predviđeno za dodavanje poddirektorija
}; // veličina mora biti sadržioc broja 512 koliko je sektor
struct openentry{ // Element direktorija u memoriji
    enum filedescriptorstatus status; // status datoteke (open/closed)
    unsigned int dirtablepos; // Na koji se element direktorija odnosi
    unsigned int startblock; // Početni blok
    unsigned int maxsizeinblocks; // Maksimalna veličina
    unsigned int actualsizeinbytes; // Stvarna veličina
    unsigned int currentseekpos; // Trenutna pozicija čitanja
};
struct direntry * dirtable; // Tabela svih datoteka
struct openentry * opentable; // Tabela otvorenih datoteka
```

Izmjene sadržaja datoteke rezultuju upisom elementa direktorija na disk, što radi sljedeća procedura.

```
// Snimi element direktorija num na disk
int SaveDirEntry(unsigned int num) {
    if (num<0 || num>=maxfiles) // Greška ako je broj neregularan
        return -EINVAL;
    int dirsperblock=512/sizeof(struct direntry); //elemenata po sektoru
    // koji sektor brojeno od početka direktorija na disku
    int relativepostionondisk=num/dirsperblock;
    // Pošto moramo snimit cijeli sektor, snimiće se i nekoliko elemenata
    // direktorija ispred, račun zaokruživanja
    int direntrytowrite=relativepostionondisk*dirsperblock;
    WriteHDBlock( &(dirtable[direntrytowrite]), // snimi 512 bajta
        dirtableondisk+relativepostionondisk,1 ); // na fizički sektor
    return OK;
}
```

Otvaranje datoteke kopira element s proslijeđenim rednim brojem u tabeli direktorija u slobodno mjesto u tabeli otvorenih datoteka uz punjenje elemenata tabele otvorenih datoteka koji ne postoje u tabeli direktorija. Druga rutina `OpenFile` traži datoteku s navedenim imenom i poziva `OpenFileByNumber`.

```
// Otvori datoteku rednog broja num
int OpenFileByNumber(int num) {
    int i,foundpos;
    struct openentry * currentopenentry; // Pokazivač na tabelu otvorenih
    struct dirent * currentdirent;
    if (num<0 || num>=maxfiles) // Nije nađena
        return -EINVAL;
    currentdirent=dirtable+num; //Pozicioniraj se na mjesto u direktoriju
    currentopenentry=openable;
    Foundpos=-1; // Traži slobodno mjesto u tabeli otvorenih datoteka
    for (i=0;i<maxopenfiles;i++,currentopenentry++)
        if (currentopenentry->status==CLOSED) { // Zatvorena, može iskoristi
            currentopenentry->status=OPEN; // Stanje otvorena
            currentopenentry->dirtablepos=num; // Pozicijau direktoriju
            currentopenentry->startblock= // Ostale podatke iskopiraj iz dir.
            currentdirent->startblock;
            currentopenentry->maxsizeinblocks=currentdirent-> maxsizeinblocks;
            currentopenentry->actualsizeinbytes=
                currentdirent-> actualsizeinbytes;
            currentopenentry->currentseekpos=0;
            return i;
        }
    return -ENOMOREFILES;
}

int OpenFile(char * name) {
    int i;
    struct dirent * currentdirent;
    currentdirent=dirtable;
    for (i=0;i<maxfiles;i++,currentdirent++) {
        if(currentdirent->entrytype==REGULARFILE &&
            ! strcmp(currentdirent->name,name,NAMELEN))
            return OpenFileByNumber(i);
    }
    return -EFILENOTFOUND;
}
```

Zatvaranje datoteke prosto promijeni statusno polje u tabeli otvorenih datoteka.

```
int CloseFile(int fd) {
    if (fd<0 || fd>=maxopenfiles) return -EINVAL;
    opentable[fd].status=CLOSED;
    return OK;
}
```

Čitanje datoteke je nešto složenija operacija. Izračuna se iz pozicije navedene u otvorenoj datoteci pozicija bloka na disku. Nakon toga se poziva funkcija za čitanje sektora sa diska u memoriju.

```

// Čita datoteku deskriptora fd, na poziciju buffer veličine size
int ReadFile(int fd, char * buffer,unsigned int size) {
    int kernelbufpos,needtoread,userbufpos,
        transferred,currentblock,startoffile;
    char kernelbuf[512]; // Ovdje se učitava sektor
    if (fd<0 || fd>=maxopenfiles)
        return -EINVAL; // Greška ako je neispravan deskriptor
    if(opentable[fd].status!=OPEN)
        return -EINVAL; // greška ako nije otvorena
    if (size==-1) // ako je veličina -1
        size=opentable[fd].actualsizeinbytes; // to znači cijela datoteka
    userbufpos=0; // Pozicija u korisničkom bufferu je 0
    // Prvi komad ide u dio kernel buffera, blok se dobije dijeljenjem
    // pozicije sa veličinom sektora (512)
    currentblock=opentable[fd].currentseekpos / 512;
    kernelbufpos=opentable[fd].currentseekpos % 512;
    // dataoffset sadrži sektore gdje su podaci
    startoffile=dataoffset+opentable[fd].startblock;
    needtoread=1; // Čitaćemo sa diska
    transferred=0; // Ništa još nije preneseno
    if ( opentable[fd].currentseekpos==opentable[fd].actualsizeinbytes-1)
        return 0; // Kraj datoteke
    while (size >0) { // Bajt po bajt
        if (needtoread) { // Pokupi fizički disk
            ReadHDBlock( kernelbuf, startoffile+currentblock, 1 );
            currentblock++;
            needtoread=0; // dok se ne prebaci sve, nove blokove ne čitamo
        }
        buffer[userbufpos++] =
            kernelbuf[kernelbufpos++]; // iz kernela u korisnički prostor
        opentable[fd].currentseekpos++; // Povećaj seek poziciju
        if (kernelbufpos==512) { // Na kraju kernel buffera
            kernelbufpos=0; // Pozicija 0
            needtoread=1; // Čitaj novi blok
        }
        transferred++;
        if (opentable[fd].currentseekpos>=opentable[fd].actualsizeinbytes){
            opentable[fd].currentseekpos=opentable[fd].actualsizeinbytes-1;
            return transferred; // Ako je kraj datoteke prekini
        }
        size--; // Umanji brojač veličine
    } // Petlja se ponavlja dok se ne pročita sve
    return transferred;
}

```

Pisanje započinje slično kao i čitanje, nalaženjem pozicije. Ovdje se mora paziti još da snimljeni blokovi ne pređu limit predviđen za tu datoteku. Ako je upisano više od trenutne dužine, ažurira se odgovarajući element direktorijske strukture na novu dužinu.

```

int WriteFile(int fd, char * buffer,unsigned int size) {
    int kernelbufpos,userbufpos,transferred,
        fpondisk,currentblock,startoffile;
    char kernelbuf[512]; // Ovdje se čuva sektor
    if (fd<0 || fd>=maxopenfiles)
        return -EINVAL; // Greška ako je neispravan deskriptor
    if(opentable[fd].status!=OPEN)
        return -EINVAL; // Greška ako nije otvorena
    fpondisk=opentable[fd].dirtablepos;
    currentblock=opentable[fd].currentseekpos / 512;
    kernelbufpos=opentable[fd].currentseekpos % 512;
    if (currentblock>=opentable[fd].maxsizeinblocks)
        return -EFULLDISK; // Greška ako prelazimo limit kontin. alokacije
    userbufpos=0; // Izračunaj poziciju gdje upisujemo
    startoffile=dataoffset+opentable[fd].startblock;
    // Ali prvo učitaj taj blok jer se može raditi samo sektor po sektor
    ReadHDBlock(kernelbuf, startoffile+currentblock, 1 );
    while (size >0) { // Puni kernel buffer iz korisničkog buffera
        kernelbuf[kernelbufpos++]=buffer[userbufpos++];
        opentable[fd].currentseekpos++; // Povećaj seek poziciju
        if (kernelbufpos==512) { // na kraju kernel buffera
            kernelbufpos=0; // vrati na početak i
            WriteHDBlock( kernelbuf, startoffile+currentblock, 1 );//snimi
            currentblock++; // novi sektor
            if (currentblock>=opentable[fd].maxsizeinblocks) {
                // Limit veličine
                opentable[fd].actualsizeinbytes=
                    dirtable[fpondisk].actualsizeinbytes=
                        opentable[fd].maxsizeinblocks*512;
                SaveDirEntry(fpondisk); // snimi koliko je stalo
                return -EFULLDISK;
            }
        }
        Transferred++;
        if (opentable[fd].currentseekpos>=opentable[fd].actualsizeinbytes){
            opentable[fd].actualsizeinbytes++; // Ažuriraj veličinu datoteke
            dirtable[fpondisk].actualsizeinbytes++;
        }
        size--;
    }
    // Snimi zadnji blok
    WriteHDBlock( kernelbuf, startoffile+currentblock, 1 );
    SaveDirEntry(fpondisk); // Snimi element direktorija
    return transferred;
}

```

Zbog kontinualne alokacije kreiranje datoteke treba u parametru da ima i njenu maksimalnu veličinu. Nakon toga se traži algoritmom BestFit slobodni blok na disku.


```

int CreateFile(char * name, unsigned int maxsize) {
    int i, foundpos, bestfitsize, requiredblocks;
    struct dirent * currentdirent = dirtable;
    requiredblocks = 1 + maxsize / 512; // Koliko treba blokova=
    if (maxsize % 512 == 0) requiredblocks--;
    foundpos = -1; // Još nismo ništa našli
    for (i = 0; i < maxfiles; i++, currentdirent++) {
        // Nađi prvi veći ili jednak komad diska
        if (currentdirent->entrytype == FREEZONE &&
            currentdirent->maxsizeinblocks >= requiredblocks) {
            if (foundpos == -1) {
                foundpos = i; // Prvo nađeno je trenutno najbolje uklapanje
                bestfitsize = currentdirent->maxsizeinblocks;
            }
            else {
                if (currentdirent->maxsizeinblocks < bestfitsize) {
                    foundpos = i; // Našli još manji komad u koji se uklapa
                    bestfitsize = currentdirent->maxsizeinblocks;
                }
            }
        }
    }
    // Kraj direktorija
    if (currentdirent->entrytype == ENDFS && foundpos == -1) {
        // Nismo ranije našli prazninu
        if (currentdirent->maxsizeinblocks >= requiredblocks) {
            if (i == maxfiles - 1) return -ENOMOREFILES; // Limit broja datoteka
            foundpos = i; // Zadnji element (prazan prostor) se smanjuje
            dirtable[foundpos + 1] = dirtable[foundpos];
            dirtable[foundpos + 1].maxsizeinblocks -= requiredblocks;
            dirtable[foundpos + 1].startblock += requiredblocks;
            dirtable[foundpos].maxsizeinblocks = requiredblocks;
            break;
        }
        else return -EFULLDISK;
    }
}

if (foundpos == -1) return -ENOMOREFILES;
strncpy(dirtable[foundpos].name, name, NAMELEN); // Unesi ime u dir.
dirtable[foundpos].entrytype = REGULARFILE; // doznaka datoteke
dirtable[foundpos].actualsizeinbytes = 0; // prazna
dirtable[foundpos].directory = 0;
SaveDirEntry(foundpos); // Snimi direktorijsku strukturu
return OpenFileByNumber( foundpos); // vrati deskriptor
}

```

Pozicioniranje je jednostavno.

```

int SeekFile(int fd, unsigned int position) {
    if (fd < 0 || fd >= maxopenfiles) return -EINVAL; // neregularni deskriptor
    if (opentable[fd].status != OPEN) return -EINVAL; // neotvorena datoteka
    if (position >= opentable[fd].actualsizeinbytes) // Zaokruži poziciju
        position = opentable[fd].actualsizeinbytes - 1;
    opentable[fd].currentseekpos = position; // Upiši je u tabelu
    return position;
}

```

Brisanje datoteke je pretvaranje njenog sadržaja direktorija u slobodnu zonu.

```
// Briši datoteku navedenog imena
int DeleteFile(char * name) {
    int i;
    struct dirent * currentdirent=dirtable; // Pogledaj direktorij
    for (i=0;i<maxfiles;i++,currentdirent++) {
        if(currentdirent->entrytype==REGULARFILE) {
            if (strncmp(currentdirent->name,name,NAMELEN)==0) {
                // Našli datoteku sa datom imenom
                currentdirent->entrytype=FREEZONE; // Postaje slobodni blok
                return SaveDirEntry(i); // Snimi direktorijsku strukturu
            }
        }
    }
    return -EFILENOTFOUND;
}
```

Postavljanje početnih vrijednosti datotečnog sistema je formatiranje particije.

```
int FormatPartition(int startsector, int maxfiles, int partitionsize ) {
    dirtable[0].entrytype=ENDFS; // Samo jedan element u direktoriju
    dirtable[0].startblock=0; // Cijela particija od početka
    dirtable[0].maxsizeinblocks=partitionsizesize;
    SaveDirEntry(0);
}
```

Inicijalizacija postavlja potrebne varijable i učitava u memoriju strukturu direktorija.

```
int filesysteminit () {
    int dirsiz, i;
    // vrijednosti promijeniti za drugi disk
    dirtable=(void *) 0x8000; maxfiles=1024; opentable=(void *) 0x18000;
    maxopenfiles=256; dirtableondisk=180; // apsolutno
    dirsiz=maxfiles*sizeof(struct dirent)/512;
    dataoffset=dirtableondisk+dirsiz;
    ReadHDBlock(dirtable, dirtableondisk, dirsiz); // direktorij u memoriju
    for (i=0;i<maxopenfiles;i++)
        opentable[i].status=CLOSED; // Sve datoteke zatvorene
}
```

18.9.Procesi

Radi jednostavnosti, raspoređivač je kooperativnog tipa, što znači da aplikacije ili niti u jezgu moraju povremeno zvati systemske pozive koji obavljaju raspoređivanje. Za čuvanje ranijeg konteksta procesa ili systemske niti koristi se *jmp_buf* struktura, u koju se sačuva stanje registara i pripremi skok.

```
#define NUM_TASKS 50
struct __jmp_buf {
    unsigned int ebx; // Registri koje treba čuvati
    unsigned int esp;
    unsigned int ebp; unsigned int esi;
    unsigned int edi; unsigned int eip;
};
#define NULL (void *) 0
```

Potprogramom *setjmp* se pripremi povratna točka a potprogramom *longjmp* se skače na neku od postavljenih povratnih tačaka.

```

typedef struct __jmp_buf jmp_buf[1];
asm (".global _setjmp;"
     "_setjmp:"
     "movl 4(%esp),%edx;" // Ova pamti na steku stanje registara
     "popl %ecx;"
     "movl %ebx, (%edx);"
     "movl %esp, 4(%edx);"
     "pushl %ecx;"
     "xorl %eax, %eax;"
     "movl %ebp, 8(%edx);"
     "movl %esi, 12(%edx);"
     "movl %edi, 16(%edx);"
     "movl %ecx, 20(%edx);"
     "ret;");
asm (".global _longjmp;"
     "_longjmp:"
     "movl 4(%esp),%edx;" // Ova vraća sa steka stanje registara
     "movl 8(%esp),%eax;"
     "movl (%edx),%ebx;"
     "movl 4(%edx),%esp;"
     "movl 8(%edx),%ebp;"
     "movl 12(%edx),%esi;"
     "movl 16(%edx),%edi;"
     "jmp *20(%edx);");
jmp_buf buf;

```

U strukturi *task_t* se nalazi kontrolni blok procesa. On se sastoji od statusa procesa, pokazivača na njegovu memoriju, stranice sadržaja registara iz *setjmp*. Kontrolni blokovi procesa su poredani u niz.

```

enum procstate {DELETED, READY, SUSPENDED, RUNNING };
typedef struct // Kontrolni blok procesa
{
    int state; // Stanje
    void * memoryblock; // Fizička memorija
    void * pagetable; // Tabela stranica
    jmp_buf regcontext; // registri
} task_t;
task_t * g_tasks;
unsigned char * stacks;
int enabled=0, idlecpu;
unsigned current = NUM_TASKS ; // Trenutni proces na kraj

```

Ovo je jednostavna funkcija za promjenu statusa procesa.

```

void setprocstate(int pid, int state) {
    if ((pid >= 0 && pid < NUM_TASKS &&
        state >= DELETED && state <= RUNNING)) {
        g_tasks[pid].state = state;
    }
}

```

Raspoređivač radi po Round robin principu iako je kooperativan. Kada se pozove *schedule* funkcija sa *setjmp* se sačuva stanje trenutno izvršavanog procesa i zatim se s *longjmp* prelazi na naredni proces.

```

void schedule(void) {
    unsigned prev;
    static int cr0;
    prev = current;
    do { // Ponovi dok ne nađe spreman posao
        current++; // sljedeći proces
        if(current >= NUM_TASKS) // kraj liste?
            current = 0; // na početak liste
    } while (g_tasks[current].state!=READY);
    idlecpu=(prev==current==0); // Nema procesa ako je izabran idle
    if(setjmp(g_tasks[prev].regcontext) != 0) // Izmjena konteksta, snimi
        return;
    ActivatePaging ( g_tasks[current].pagetable); // Podesi tabelu stranica
    longjmp(g_tasks[current].regcontext, 1); // Skoči na snimljeni proces
}

```

Naredna funkcija predstavlja prazan proces. On poziva hlt instrukciju radi smanjenja potrošnje (hlađenje).

```

void idle () {
    for (;;) {
        if (idlecpu) asm("hlt");
        schedule();
    }
}

```

Pokretanje novog procesa alocira memoriju od 3M, postavi registre, premapira memoriju i ako nije sistemaska nit učita datoteku s procesom.

```

// Otvara novi proces iz izvornog koda jezgra ili satoteke
int newprocess(void * routine, void * stack, char * filename) {
    unsigned int i,fd,memat4K;
    void * mem,*page_directory;
    mem=(void *)AllocMemory(3*1024*1024+4096*3); //3 M + 3 strane
    for(i = 0; i < NUM_TASKS; i++) { // Nađi mjesto u tabeli procesa
        if (g_tasks[i].state==DELETED) { // Pripremi registre u PCB
            g_tasks[i].regcontext[0].eip = (unsigned)routine;
            g_tasks[i].regcontext[0].esp = (unsigned)stack;
            g_tasks[i].state=READY; // Proces u stanju spreman
            g_tasks[i].memoryblock=mem; // Memorija procesa je ovako određena:
            // imenik stranica, pa tabela stranica, pa privatni prostor
            memat4K=((unsigned int)mem & 0xFFFFF000)+0x1000; //Imenik stranica
            page_directory=(void *)memat4K;
            g_tasks[i].pagetable=page_directory; // Tabela stranica
            MapVirtualSpace(page_directory,memat4K+8192);// privatni prostor
            if (filename !=NULL) { // Navedena datoteka s programom
                LinearMode();
                if ((fd=OpenFile(filename))>0) { // Otvori je
                    ReadFile(fd,(void *) (memat4K+8192),-1); // Učitaj
                    CloseFile(fd); // Zatvori
                }
            }
        }
    }
    return i;
}
} return -1;
}

```

Inicijalizacija tabele procesa će sve procese u tabeli proglašiti praznim i pokrenuti idle proces.

```
#define    STACK_SIZE 5120
void initprocesstable() {
    int i;
    g_tasks=(void *) 0x80000; // Kontrolni blokovi procesa
    stacks=(void *) 0x70000; // stek područja
    for(i = 0; i < NUM_TASKS; i++) {
        (void)setjmp(g_tasks[i].regcontext); /* registri procesa */
        g_tasks[i].state=DELETED; // Svi taskovi obrisani
    }
    // Mora biti bar jedan proces
    newprocess((void *)idle,(void *)(stacks+ STACK_SIZE),NULL);
}
```

Završetak procesa oslobađa njegovu memoriju, a funkcija Close završava trenutno izvršavani proces.

```
void Terminate(int pid) {
    if (pid >=0 && pid < NUM_TASKS ) {
        setprocstate(pid,DELETED); // Oslobodi PCB
        LinearMode(); // Isključi straničenje
        FreeMemory(g_tasks[pid].memoryblock); // Oslobodi memoriju
        schedule(); // Raspoređivač
    }
}
void Close() {
    Terminate(current);
}
```

18.10. Sistemski poziv

Sistemski pozivi su obični pozivi potprograma iz jezgra uz dodatak schedule poziva. Pozivi s podvlačicom obave i raspoređivanje.

```
int _OpenFile(char * name) {int i=OpenFile(name); schedule(); return i;}
int _DeleteFile(char *name){int i=DeleteFile(name);schedule();return i;}
int _CreateFile(char * name, unsigned int maxsize) { int
i=CreateFile(name, maxsize);schedule(); return i;}
int _CloseFile(int fd) { int i=CloseFile( fd);schedule();return i;}
int _SeekFile(int fd, unsigned int position) {
    int i=SeekFile( fd, position);schedule(); return i;}
int _ReadFile(int fd, char * buffer,unsigned int size) { schedule();
return ReadFile( fd, buffer,size);}
int _WriteFile(int fd, char * buffer,unsigned int size) { schedule();
return WriteFile( fd, buffer,size);}
int _FormatPartition(int startsector,int maxfiles,int partitionsize) {
    schedule();return FormatPartition(startsector,maxfiles,partitionsize);}
unsigned char _ReadIntScanCode(){schedule();return readintscanCode();}
void _SetTime(unsigned int t) { schedule(); SetTime(t);}
unsigned int _ReadTime() { schedule(); return ReadTime(); }
void *_AllocMemory(int size){schedule();return(void*)AllocMemory(size);}
void _FreeMemory(void * addr) { schedule(); FreeMemory(addr);}
unsigned char _ScanToAscii(unsigned char code) {schedule(); return
ScanToAscii(code);}
```

```

unsigned char _GetScanCode(){ schedule(); return GetScanCode();}
void _SetCursor(short row, short col) { SetCursor( row,
col);schedule();}
void _GetCursor(short * row, short * col) { GetCursor( row,
col);schedule();}
void _ClrScr() { ClrScr();schedule();}
void _Print(const char * message){ Print(message);schedule();}
void _PrintNum(int num) { PrintNum(num);schedule();}
int _NewProcess(void * routine, void * stack, char * filename) {
    newprocess(routine, stack, filename);schedule();}
void _Terminate(int pid) { Terminate(pid);schedule();}
void _Close() { Close();schedule(); }

```

18.11.Glavni program

Glavni program poziva inicijalizaciju podsistema.

```

void main() {
    ClrScr(); InitInterrupts(); initmemory();
    filesysteminit (); initprocesstable(); Cooperative();
    while (1) {};
}

```

Kao ilustracija pokrenuće se dvije systemske niti, sa različito mapiranom memorijom iznad jednog megabajta. Jedna nit se završava ranije, a obje broje i prikazuju slovo koje se nalazi na lokaciji 1M.

```

static void task0(void){
    int i=120;
    char *a;
    a=(char *) (1024*1024);a[0]='C';a[1]=0;
    while(1){
        i=i+1; SetCursor(1,10);Print(a);SetCursor(1,11); _PrintNum(i);
    }
}
static void task1(void){
    int i=120;
    char *a;
    a=(char *) (1024*1024);
    a[0]='B';a[1]=0;
    while(1){
        i=i+1;if (i==1000) Terminate(1);
        SetCursor(10,10);Print(a);SetCursor(10,11); _PrintNum(i);
    }
}
int Cooperative(void) {
    volatile unsigned i; unsigned adr;
    newprocess((void *)task0,(void *)((void *) 0x71000),0);
    newprocess((void *)task1,(void *)((void *) 0x72000),0);
    schedule();
    return 0;
}

```

Kompajliranjem kernela dobivaju se dvije datoteke. Jedna sadrži boot sektor, a druga kernel. Njih treba posebnim alatom staviti na pravo mjesto na disku. Nakon paljenja računara, ili pokretanja emulatora, pokreće se ovaj minijaturni OS.

19.LITERATURA

Knjige

Bach, Maurice J. *The design of the UNIX operating system*. Vol. 5. Englewood Cliffs, NJ: Prentice-Hall, 1986.

Deitel, Harvey, Deitel, Paul, & Choffnes, David *Operating systems*. Pearson/Prentice Hall, 2004.

Dhamdhere, Dhananjay M. *Operating Systems: A Concept-based Approach*, 2nd ed. Tata McGraw-Hill Education, 2006.

Dorđević, Borislav, Dragan Pleskonjić, Nemanja Maček. *Operativni sistemi: teorija, praksa i rešeni zadaci*. Mikro knjiga, 2005.

Eric Dumas, Franck Mevel, Remy Card *The Linux Kernel Book*, Willey 2013

Feitelson, Dror G. *Notes on Operating Systems* School of Computer Science and Engineering, The Hebrew University of Jerusalem (2011).

Petzold, Charles *Programming Windows*, 5th edition, Microsoft press, 1998

Pietrek Matt, *Windows 95 System programming Secrets*, IDG Books Worldwide, Inc., 1995

Portable Applications Standards Committee of the IEEE Computer Society and The Open Group, Standard for Information Technology—*Portable Operating System Interface (POSIX®), Base Specifications, Issue 7*, 2008

Prasad Dabak, Sandeep Phadke, Milind Borate, *Undocumented Windows NT*, Hungry Minds, 1999

Rubini, Alessandro, Jonathan Corbet. *Linux device drivers*. O'Reilly Media, Inc., 2001.

Sibsankar, Haldar, Aravind, Alex. *Operating systems* Pearson Education, 2010.

Silberschatz, Abraham., Peter B Galvin, and Greg Gagne. *Operating System Concepts* 10th ed. Wiley Publishing, 2018.

Simon, Richard, *Windows NT Win32 API SuperBible*, Waite Group Press 1997

Stallings, William, and Goutam Kumar Paul. *Operating systems: internals and design principles.*, 9th ed., Pearson, 2018.

Tanenbaum, Andrew, Bos, Herbert, *Modern Operating Systems*, 4th ed. Prentice Hall Press, 2014

Tanenbaum, Andrew, Wetherall, David, *Computer Networks*, 5th ed. Pearson, 2011

Yosifovich, Pavel, Solomon, David, Russinovich, Mark, Ionescu, Alex. *Windows Internals, Part 1: System architecture, processes, threads, memory management, and more*. 7th ed., Microsoft Press, 2017.

Članci

Aho, A. V.; P. J. Denning; and J. D. Ullman, "Principles of Optimal Page Replacement," *Journal of the ACM*, Vol. 18, No. 1, January 1971, pp. 80-93

Bays, C., "A Comparison of Next-fit, First-fit, and Best-fit," *Communications of the ACM*, Vol. 20, No. 3, March 1977, pp. 191-192.

Belady, L. A., "A Study of Replacement Algorithms for Virtual Storage Computers," IBM Systems Journal, Vol. 5, No. 2, 1966, pp. 78-101.

Belady, L.A., Nelson R, Shedler G. "An anomaly in space-time characteristics of certain programs running in a paging machine." Communications of the ACM 12.6 (1969): 349-353.

Berners - Lee, T., et al, "The World-Wide Web," Communications of the ACM, Vol. 37, No. 8, August 1994, pp. 76-82.

Denning, P. J., "The Working Set Model for Program Behavior," Communications of the ACM, Vol. 11, No. 5, May 1968, pp. 323-333.

Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in F. Genuys, ed., Programming Languages, New York: Academic Press, 1968, pp. 43-112.

Gyanendra K., and Parul T. "A Novel Longest Distance First Page Replacement Algorithm." Indian Journal of Science and Technology 10.30 (2017).

Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," Communications of the ACM, Vol. 17, No. 10, October 1974, pp. 549-557.

Megiddo, N, and Dharmendra S. . "Outperforming LRU with an adaptive replacement cache algorithm." Computer 37.4 (2004): 58-65.

Muqaddas, A., Abdulsalam, H., & Salman, A. S-LOOK: A Preemptive Disk Scheduling Algorithm for Offline and Online Environments. Computer Science & Information Technologies, Lviv, Ukraine. (2009).

Peterson, G.L. "Myths About the Mutual Exclusion Problem" Information Processing Letters, 12(3), pages 115-116, 1981

Ribić S, Salihbegović A, "Tiny operating system kernel for education purposes", 38th International Convention on Information and Communication Technology, Electronics and Microelectronics, 2015

Ritchie, D. M., and Thompson K., "The UNIX Time-sharing System," Communications of the ACM, Vol. 17, No. 7, July 1974, pp. 365-375

Teorey, T., and T. Pinkerton, "A Comparative Analysis of Disk Scheduling Policies," Communications of the ACM, Vol. 15, No. 3, March 1972, pp. 177-184.

20.INDEKS

A

ActiveX, 27, 337, 338, 339, 341
 Ada, 88
 ADSL, 174, 224, 225
 Aktivni direktorij, 313
 alarmi, 97
 algoritmi raspoređivanja
 procesa
 Ciljevi, 50
 Dinamičko, 70
 Dodjela posvećenog
 procesora, 70
 Družbeno, 69, 76
 FCFS, prvi došao prvi
 uslužen, 53
 Fer raspodjela, 63
 Garantovano, 62
 Globalno, 68
 HRRN, Najbolji odnos
 odziva sljedeći, 56
 Kategorije, 52
 Kooperativno, 58
 Lutrijsko, 63
 Monotonom stopom, 67
 Najraniji rok prvo, 66
 Particionirano, 68
 Podjela opterećenja, 68, 69
 Potpuno fer
 raspoređivanje, 64
 Prioritetno bez istiskivanja,
 56
 Prioritetno sa
 istiskivanjem, 65, 67
 Prioritetno sa
 prilagođivanjem
 prioriteta, 58
 Round Robin sa
 prioritetima, 60
 Round-Robin- kružno, 57
 Ručno raspoređivanje, 65
 SJF, Najkraći posao prvi, 54
 SRTN, Najkraće preostalo
 vrijeme, 56
 Više nivoa sa povratnom
 spregom, 61
 Višestruki redovi čekanja,
 61
 algoritmi zamjene
 segmenata, 158
 alokacija datoteka

ekstentna, 285
 indeksna na više nivoa, 283
 kontinualna, 276
 ulančana lista, 278
 ulančana lista i tablica
 pokazivača, 279
 alokacija uređaja, 181
 Amdalov zakon, 70
 ANSI escape sekvence, 189
 API pozivi, 26, 202, 319, 335
 aplikativni protokoli, 232
 Apple iOS, 28, 367, 368
 Azure Sphere, 318

B

backup, 261, 290, 291, 299,
 308, 310, 357
 bafer, 181
 barijere, 88
 BASIC, 20, 205, 315
 batch sistemi
 multiprogramirani, 16
 serijski, 15, 16, 49
 Bélády anomalija, 155
 bežični interfejs, 174
 boot sektor, 286, 369
 BPB-BIOS Parameter Block,
 275
 brojač referenci, 156
 Buddy system, 123
 burst režim, 173

C

Cache Manager, 329
 CD-ROM, 183, 254
 CHS-Cilindar, glava, sektor,
 236, 255, 374
 cijevi, 93, 94, 95
 Clipboard, 92, 97
 CMD.EXE, 206
 COM, 27, 92, 206, 207, 316,
 333, 337, 338, 339, 341
 COMMAND.COM, 206, 207,
 316, 321, 340
 Configuration Manager, 329,
 333
 crond proces, 349, 357
 cupsd proces, 357
 CUPS-Sistem štampanja za
 Unix, 184, 203, 204

D

DACL-Diskreciona lista
 kontrola pristupa, 311, 312,
 313, 330
 Dalvik VM, 363, 364, 365, 366
 datotečni atributi, 259
 datotečni deskriptor, 266
 datotečni sistem, 259, 265,
 269, 289, 334, 354, 380
 Performanse, 291
 datotečni sistemi, 269, 334,
 354
 datoteka, 92, 259
 fizički sadržaj, 262
 DCOM, 337, 338
 defragmentacija, 292
 deskriptor, 94, 161, 165, 194,
 265, 266, 272, 273, 274, 277,
 312, 313, 330, 350, 352, 370,
 383, 384, 385
 deskriptorski keš, 136
 deskriptorski registar
 GDTR, 160, 161, 164, 165,
 369
 LDTR, 160, 161, 164, 165
 Desktop okruženja, 349, 360
 dijeljena memorija, 92, 93,
 156
 dijeljene datoteke, 92
 dijeljene stranice, 155
 dinamička relokacija, 118
 DirectX, 27, 28, 325, 335, 339
 direktorij, 208, 266, 271, 287
 aciklični graf, 269
 dvonivovski, 268
 hijerarhijski, 268
 jednonivovski, 267
 opšti graf, 270
 dispečer, 328, 333
 dispečer sistemskih servisa,
 333
 distribuirani sistemi, 15, 18
 DLL-Dinamički dijeljene
 biblioteke, 126, 209, 319,
 327, 335, 336, 338, 342
 Windows API, 127
 DMA kontroler, 170, 172, 173,
 179, 182
 DMA-Direktni pristup
 memoriji, 170, 172, 173, 176,

179, 182, 186, 291, 333, 354, 356
 Dodirni ekrani, 204
 drajver, 178, 180, 184, 187, 188, 202, 319, 322, 324, 329, 332, 334, 355, 372, 374
 blokovskih uređaja, 176
 memorijskih uređaja, 177
 mrežnih uređaja, 177
 znakovnih uređaja, 176
 drajver diska, 374
 drajver tastature, 187, 372
 drajveri uređaja, 355
 dugo ime datoteka, 280
 DVD, 114, 252, 254, 255, 277, 324

E

ekranski hardver, 187
 elementi tabele stranica
 bit modifikacije, 139
 bit prisutnosti, 138
 bit referenciranja, 139
 Stranični okvir, 138, 139, 140, 141, 148, 155, 162
 ESC/P, 201
 Ethernet, 168, 174, 221, 222, 225, 227, 228, 229, 230, 234, 356
 exokernel, 24
 Explorer, 14, 28, 313, 324, 325, 337, 339, 341, 344, 368
 ext2fs, 292

F

FAT datotečni sistem, 209, 260, 270, 271, 275, 276, 279, 280, 281, 282, 287, 288, 289, 292, 293, 295, 297, 298, 317, 321, 334
 fiksne particije, 116
 flopi disk, 253
 fragmentacija
 datoteke, 236
 fragmentacija memorije
 eksterna, 121, 124, 137, 285
 interna, 118, 121, 140, 281
 Framework .NET, 341
 FTP protokol, 233, 337, 358
 Fuchsia, 22

G

GID-identifikator grupe, 310

glavni program LEKOS, 44, 192, 390
 GNOME, 349, 360, 361
 Google Android, 363
 greška stranica, 139, 142, 143, 144, 145, 146, 148, 149, 150, 152, 154, 331, 350, 351
 GUI-Grafičko korisničko sučelje, 218

H

HAL-Hardware abstraction layer, 19, 327, 328, 330, 334
 Harvard arhitektura, 133
 hibridni sistemi, 15, 17
 hibridno jezgro, 24
 hijerarhijsko stranicenje, 141
 hipervizor, 23, 328
 HTTP protokol, 233, 337
 Hyper V virtualizator, 327, 328

I

I/O Manager, 329
 identifikacija, 300
 imenik stranica, 162, 166, 388
 imenovanje uređaja, 183
 alokacija datoteka, 281
 indeksni čvor, 294, 352
 inetd proces, 349, 357
 init proces, 357
 Intel i386, 37, 160, 318, 319, 327, 346, 352, 353, 354, 369, 375
 memorijska arhitektura, 160
 interaktivni sistemi, 15, 16, 17
 invertovana tabela stranica, 140, 141
 IP protokol, 230
 ISO 9960, 277, 278
 Izbor fizičkih okvira, 143
 izbor slobodne particije
 Best fit-najbolje uklapanje, 129
 First fit-prvo uklapanje, 129, 158, 378
 Worst fit-najgore uklapanje, 129
 izbor stranice za zamjenu
 Adaptivna zamjena, 146, 151
 Algoritam sa radnim skupom, 152

Algoritam starenja, 150, 151
 Druga šansa (Second Chance), 146, 148
 FIFO - Prvi učitani okvir, 146
 GClock, 150
 LDF - Najduža udaljenost prvo, 146
 LFU-Najčešće korištena, 146, 150
 LFU-Najrjeđe korištena, 146, 150, 151, 152, 331
 LRU-Najdavnije zadnji put korišten, 149
 NRU - okvir koji nije skoro korišten, 146, 147, 148
 Optimalni, 147, 164
 Satni algoritam, 148
 Satni algoritam sa dvije kazaljke, 155
 Satni algoritam sa starenjem, 151
 Slučajni izbor stranice, 146, 147
 WSClock, 154
 izmjena konteksta, 36, 37, 388
 izmjenjivi mediji, 252
 izmjenjivi poluprovodnički mediji, 255

J

Java, 27, 88, 107, 304, 341, 363, 364, 365, 366, 367, 368
 jednokorisnički sistemi, 14, 15
 jednoprocensni sistemi, 15, 115, 131

K

KDE, 220, 348, 349, 360, 361
 keš-cache, 114, 183, 329, 332, 344
 komponente Linux sistema, 347
 komunikacija porukama, 95
 kontroler prekida, 171
 kontrolna tačka, 108
 kriptografija, 305
 kritična sekcija, 78
 kvantum, 57, 351

L

LBA-Logička adresa bloka, 236, 239, 255, 374

LEKOS, 369
 Linux, 11, 15, 18, 21, 26, 28, 53,
 59, 78, 86, 92, 124, 158, 180,
 181, 183, 184, 203, 209, 210,
 211, 214, 215, 270, 290, 303,
 305, 318, 335, 336, 345, 346,
 347, 348, 349, 350, 351, 352,
 353, 354, 355, 356, 357, 358,
 359, 361, 362, 363, 364, 365,
 367, 368, 391
 međuprocena
 komunikacija, 352
 osnovne biblioteke, 356,
 363
 štampanje, 203
 upravljanje procesima, 349
 Linux biblioteka
 libc, 26, 348, 356
 libcrypt, 356
 libcups, 356
 libdl, 349, 356
 libgcc_s, 356
 libgettext, 349, 356
 libm, 349, 356
 libncurses, 356
 libnspr4, 356
 libnss3, 356
 libpthread, 356
 librt, 356
 libsane, 356
 libssl3, 356
 libstdc++, 349, 356
 libutil, 356
 libxml, 349, 356
 libxslt, 356
 libz, 356
 Linux drajver, 180
 LPC-Local procedure call, 329

M

magnetna traka, 110, 168, 253
 magnetni diskovi, 235
 formatiranje, 238
 klasifikacija, 237
 organizacija, 235
 pravljenje particija, 238
 vrijeme pristupa, 237
 MAR-Memorijski adresni
 registar, 113
 MBR-Master boot record, 238,
 290
 MCB-Kontrolni blok
 memorije, 120, 125, 377, 378

među-procesna
 komunikacija, 92, 352
 međusobno isključivanje
 Dijeljena varijabla, 79
 Peterson-ovo rješenje, 80
 Striktna promjena, 79
 TSL instrukcija, 81
 Zabrana prekida, 80
 Memorija
 Off-line, 114
 Primarna, 114
 Sekundarna, 114
 Tercijarna, 114
 Memorijske adrese, 113
 memorijske banke, 133
 memorijski mapirane
 datoteke, 266
 memorijski mapirani uređaji,
 173
 metoda jednake raspodjele,
 143
 metoda proporcionalne
 raspodjele, 143
 MFT datoteka, 275, 286, 287
 mikrokernell, 19, 21, 22
 Minix, 22
 miš, 168, 187
 MMU-Jedinica za upravljanje
 memorijom, 37, 113, 118, 133,
 134, 137, 139, 140, 145, 164
 Mnix, 275, 345, 346
 modem, 167, 168, 170, 176, 197,
 222, 223, 224, 225
 modulacija, 224
 modularno jezgro, 21
 monitori, 87, 88
 monolitni sistemi, 19
 monolitno jezgro, 20
 mrežni servisi, 185
 MS DOS, 17, 18, 57, 58, 116, 184,
 202, 206, 207, 208, 265,
 269, 279, 305, 310, 315, 316,
 317, 318, 319, 320, 321, 322,
 323, 327, 339, 340, 346
 Multics, 17, 159
 mutex, 85

N

natjecanje, 77
 Nokia OS, 28, 368
 NT domena, 312
 NT Executive, 329
 NT Kernel, 328

NTFS, 209, 259, 262, 263, 271,
 275, 285, 287, 289, 290, 332,
 334
 NTOSKRNL.EXE, 14, 327, 328
 NTVDM, 340

O

Object Manager, 329, 330
 obrađivač interapta, 375
 obrađivač prekida, 175
 OLE, 337, 338, 339, 340
 operativni sistem
 definicija, 13
 distribuirani, 15, 18
 džepni, 19
 klasifikacija, 14
 mainframe, 18, 115, 197
 mrežni, 18
 preko virtualnih mašina,
 22, 23, 24
 serverski, 18
 udruženi, 18
 OS/2, 24, 27, 159, 318, 319, 322,
 327, 335
 OS/360 MFT, 116
 overlay, 131, 317

P

Paketni menadžer, 359
 Palm OS, 368
 paralelni interfejs, 174
 PCL, 201
 PDL
 Jezik za opis stranice, 200,
 201, 202
 Plug-and-Play Manager, 329
 podprogrami za i/o portove,
 370
 POSIX, 24, 27, 28, 46, 71, 77, 86,
 125, 157, 179, 180, 185, 273,
 311, 327, 335, 336, 346, 350,
 351, 356, 391
 Posix API, 24, 27, 28, 86, 125,
 157, 179, 185, 273
 Posix API za upravljanje
 memorijom, 125
 posix funkcije
 pthread_attr_destroy, 46
 pthread_attr_init, 46
 pthread_create, 46, 47, 48,
 77, 87, 356
 pthread_detach, 46
 pthread_exit, 46

- pthread_join, 46, 47, 48, 77, 87, 356
- Posix funkcije
- access, 273, 310
 - alarm, 40, 98, 99
 - brk, 125
 - chdir, 273
 - chmod, 273, 310
 - chown, 273, 310
 - close, 94, 95, 265, 266, 273, 274
 - closedir, 273
 - creat, 273, 274
 - dup2, 273
 - execl, 40
 - execvp, 40
 - execv, 40
 - fork, 40, 41, 48, 94, 155, 206
 - fstat, 273
 - getcwd, 273
 - getpid, 40, 41, 71
 - getppid, 40, 41
 - getpriority, 71
 - kill, 40
 - link, 273
 - lseek, 273, 274
 - madvise, 157, 158
 - mkdir, 273
 - mlock, 157, 158
 - mlockall, 157
 - mprotect, 157
 - munlock, 157, 158
 - munlockall, 157
 - nice, 71
 - open, 95, 101, 185, 266, 273, 274
 - opendir, 273
 - pause, 40
 - pipe, 94
 - read, 89, 94, 95, 185, 265, 266, 273, 274, 303
 - readdir, 273
 - rename, 273
 - rewinddir, 273
 - rmdir, 273
 - sbrk, 125, 126
 - sem_close, 86
 - sem_destroy, 86
 - sem_getvalue, 86
 - sem_init, 86, 87
 - sem_open, 86
 - sem_post, 86, 87
 - sem_trywait, 86
 - sem_unlink, 86
 - setpriority, 71
 - signal, 82, 89, 90, 91, 92, 98, 99, 102, 301
 - sleep, 40, 41, 48, 90, 91
 - stat, 273
 - unlink, 273
 - wait, 40, 41, 82, 86, 87, 88, 89, 90, 91, 92, 101, 102, 206, 301
 - waitpid, 40
- Posix funkcije access, 273, 310
- Posix funkcije chmod, 273, 310
- Posix funkcije chown, 273, 310
- Posix funkcije getgid, 310
- Posix funkcije getuid, 310
- Posix funkcije setgid, 310
- Posix funkcije setuid, 310
- PostScript, 168, 201, 202, 203, 204
- potpuni zastoje, 102, 104, 108
- Bankarov algoritam, 109
 - Detekcija grafom alokacije resursa, 105
 - Detekcija matricnom metodom, 106
 - Detekcija mjerenjem vremena, 105
 - Izbjegavanje, 108
 - Modeliranje grafom alokacije resursa, 103
 - Nojev algoritam, 104
 - Oporavak, 107
 - Strategije rješavanja, 104
 - Uslovi nastanka, 102
 - Windows API, 111
- Power Manager, 323, 329
- PowerShell, 324, 341
- pozadinski procesi, 357, 364
- PPP protokol, 222, 225
- praćenje statusa, 183
- preemptivno raspoređivanje, 392
- prekidi, 24, 49, 81, 170, 171, 177, 179, 186, 315, 317, 329, 369, 375, 376, 377
- hardverski, 25
 - izuzeci, 25
 - softverski, 24
- prepletanje, 239
- priključivanje diskova, 251
- princip dražvera
- ulaz/izlaz vođen
 - prekidima, 178
- principi zaštite, 300
- prirodni API, 335
- pristup datotekama
- direktni, 264
 - indeksni, 264
 - sekvencijalni, 264
- pristupni raspoređivač, 49
- pristupni token, 330
- Problem čitača i pisaca, 91
- Problem pet filozofa, 90
- proces, 11, 32, 35, 49, 63, 64, 66, 70, 87, 92, 93, 117, 131, 349, 352, 358, 379, 386
- model, 29, 30
 - pokretanje, 32
 - Posix funkcije, 40, 71
 - stanja, 33, 35
 - Windows API funkcije, 38, 71
 - završavanje, 37
- Process Manager, 327, 329, 335
- programske niti, 41, 42, 80, 89, 349
- hibridna implementacija, 43
 - implementacija, 43
 - Posix funkcije, 46, 77, 356
 - u jezgri, 43, 44, 46, 349, 386
 - u korisničkom prostoru, 43
 - Windows API funkcije, 44
- promjenjive particije, 118
- prstenasti sistem, 21
- ## R
- radni skup, 152, 153, 154, 156, 331
- RAID strukture, 245
- RAID strukture
- RAID 0, 245, 246, 248, 249, 258
 - RAID 0+1, 249
 - RAID 1, 246, 248, 249, 258
 - RAID 10, 249
 - RAID 2, 246, 258
 - RAID 3, 247, 248, 258
 - RAID 4, 247, 248, 258

RAID 5, 247, 248, 249, 250, 258
 RAID 50, 249, 250
 RAID 6, 247, 248, 249
 raspoređivač, 33, 49, 56, 57, 59, 64, 83, 351, 352, 387, 389
 raspoređivanje diska
 C-LOOK, 240, 242, 243, 256, 257, 334
 C-SCAN, 240, 242, 256, 257
 FCFS, 53, 54, 58, 61, 69, 72, 181, 240, 241, 256, 351
 F-LOOK, 243
 F-SCAN, 240, 243
 LOOK, 240, 241, 242, 243, 256, 257
 SCAN, 240, 241, 242, 243, 244, 256
 SCAN-EDF, 244
 S-LOOK, 240, 241, 242, 392
 unutar jedne staze, 240
 višestepeni SCAN, 243
 Raspoređivanje diska
 SSTF, 240, 241, 243, 256
 raspoređivanje ulazno-izlaznih operacija, 181
 redovi raspoređivanja, 34
 registrovanje drajvera, 181
 relokacioni registar, 134, 135
 resursi, 101, 111, 301
 neoduzivi, 101
 oduzivi, 101
 rezervni sektori, 245
 režim krađe ciklusa, 173
 RPC - Poziv udaljene procedure, 28, 92, 99, 252
 RS232, 170, 174, 176, 222, 223, 224, 225, 226
 RSA algoritam, 307, 308, 314

S

Security Reference Monitor, 329, 330
 segmentacija, 127, 134, 159
 segmentacija sa straničenjem, 159
 semafori, 82, 83, 89, 352, 353
 sa blokiranjem, 83
 sa zaposlenim čekanjem, 82
 serijski interfejs, 174
 server Win32, 336
 serveri okruženja, 335

servisi, 339, 349
 SID-sigurnosni identifikator, 311, 313, 330
 signali, 92, 97, 352
 sigurnosni deskriptor, 312
 simetrični kriptosistemi, 306, 307
 sinhronizacija
 POSIX API za sinhronizaciju, 86
 Windows API, 84
 sinhronizacija jezgra, 352, 353
 sistemi u realnom vremenu, 15, 18, 19, 28, 52, 64, 65, 66, 75, 190, 240, 318, 351
 sistemski poziv, 25, 26, 38, 40, 94, 317, 335, 389
 SLIP protokol, 225, 230
 slobodnim blokovi diska, 287
 slojevi mrežnih modela, 221
 slojeviti sistemi, 20
 SMTP protokol, 222, 233, 234
 spooler, 185, 202
 SSD disk, 114, 243, 244, 250, 251, 255
 stack, 25, 29, 31, 42, 44, 134, 135, 137, 263, 336, 350, 364, 369, 370, 388, 389, 390
 štampači, 198, 200
 statička relokacija, 117
 straničenje, 137, 142
 strategija straničenja
 globalna, 144
 lokalna, 144
 struktura particije, 275
 swapping, 116
 Symbian, 22, 368

T

tabela otvorenih datoteka, 266, 381, 382
 tabela segmenata, 135, 160
 tabela stranica, 163, 330, 331, 380, 387
 tanki klijent, 197, 198
 tastatura, 168, 187, 218, 372
 TCP protokol, 234
 Tizen, 368
 TLB registri, 140, 141, 149, 162
 Transaction Manager, 329
 transparentni režim, 173
 trojanac, 300

U

U/I kontroler, 113, 114, 168, 170, 171, 172, 173, 174
 registri, 170
 U/I mapirani uređaji, 168, 173
 udarni štampači, 198
 UDP protokol, 232
 UID-identifikator korisnika, 309, 310, 364, 367
 ulazno-izlazni uređaji
 Klasificiranje, 167
 Struktura, 169
 unikernel, 20
 Unix datotečni sistem, 275, 283, 284
 Unix GUI, 219
 UNIX historijat, 345
 Unix školjke, 209
 Programiranje, 216
 Unix/Linux naredbe
 komandne linije, 210
 upravljanje memorijom, 324, 353, 377
 USB, 114, 174, 202, 224, 225, 226, 227, 237, 251, 255, 279, 321, 348
 uslovne varijable, 88

V

veličina disk bloka, 276, 281
 veličina stranice, 140, 163
 video drajver, 371
 video RAM, 189
 Virtual Memory Manager, 327, 329, 330, 331
 virtualna adresa, 134, 138
 virtualna memorija, 133
 virus, 300
 višekorisnički sistemi, 14, 15
 višeprocenjski sistemi, 116
 vlakna, 45
 vrijeme brisanja stranica, 145

W

WDM Drajeri, 322, 324, 334
 WiFi, 229
 Win32 bazni API, 336
 Win32 u jezgru, 335
 Windows, 11, 14, 15, 17, 18, 19, 24, 26, 27, 28, 32, 38, 40, 44, 45, 48, 52, 53, 58, 59, 62, 70, 71, 84, 85, 92, 93, 96, 97, 101,

- 105, 111, 124, 125, 127, 135,
- 144, 156, 159, 179, 180, 181,
- 184, 190, 192, 197, 198, 202,
- 204, 206, 207, 208, 209,
- 218, 219, 252, 261, 262, 263,
- 265, 268, 269, 271, 272, 279,
- 280, 281, 290, 291, 303, 305,
- 310, 311, 312, 313, 315, 318,
- 319, 320, 321, 322, 323, 324,
- 325, 326, 327, 328, 329, 330,
- 331, 332, 333, 334, 335, 336,
- 337, 338, 339, 340, 341, 342,
- 343, 344, 346, 357, 360, 361,
- 368, 391
- Štampanje, 202
- Windows 1.0, 318, 320
- Windows 10, 15, 26, 28, 206,
- 219, 320, 323, 325, 328, 331,
- 335, 342, 368
- Windows 2000, 310, 312, 323,
- 324, 326, 327, 328, 329, 332,
- 333, 335
- Windows 3.1, 52, 58, 135, 320,
- 321, 322, 323, 327, 336, 340
- Windows 7, 28, 209, 281, 320,
- 323, 324, 337
- Windows 8, 32, 320, 323, 325,
- 326, 327, 328, 331, 342, 368
- Windows 95, 159, 190, 280,
- 291, 320, 321, 322, 323, 325,
- 327, 332, 335, 339, 391
- Windows 98, 15, 279, 280, 281,
- 321, 322, 336
- Windows API, 27
- Windows API funkcije
- AbortDoc, 203
- AddAccessAllowedAce, 313
- AddAccessDeniedAce, 313
- AppendMenu, 191
- Arc, 191
- BeginPaint, 191, 194
- BitBlt, 191, 202
- CheckDlgButton, 191
- CloseHandle, 39, 85, 271,
- 273
- CloseThreadWaitChainSession, 111
- ConnectNamedPipe, 95
- ConvertThreadToFiber, 45,
- 46
- CopyFile, 271
- CopyMemory, 125
- CreateBitmap, 191
- CreateDirectory, 271
- CreateFiber, 45, 46
- CreateFile, 180, 271, 272,
- 273, 313, 385, 389
- CreateFontIndirect, 191
- CreateHatchBrush, 191
- CreateMenu, 191
- CreateMutex, 85
- CreateNamedPipe, 95
- CreatePipe, 95
- CreateProcess, 27, 32, 38,
- 39, 336
- CreateSemaphore, 85, 86,
- 100, 329
- CreateThread, 44, 45, 48,
- 100
- CreateWindow, 191, 192
- DefWindowProc, 193, 194
- DeleteAce, 313
- DeleteCriticalSection, 85
- DeleteFiber, 45
- DeleteFile, 271, 386, 389
- DestroyWindow, 191, 194
- DeviceCapabilities, 203
- DeviceIoControl, 180
- DispatchMessage, 58, 97,
- 191, 193
- Ellipse, 191
- EndDoc, 202, 203
- EndPage, 203
- EndPaint, 191, 194
- EnterCriticalSection, 85
- EnumPrinters, 203
- EnumProcesses, 38, 39
- EnumWindows, 191
- ExitProcess, 38, 39
- ExitThread, 44, 45
- ExtFloodFill, 191
- FillMemory, 125
- FindClose, 271
- FindFirstFile, 271
- FindNextFile, 272
- FlushFileBuffers, 272
- FreeLibrary, 127
- GetConsoleWindow, 190
- GetCurrentProcessId, 38
- GetCurrentProcessorNumber, 71
- GetCurrentThread, 44
- GetDC, 191
- GetDefaultPrinter, 203
- GetFileAttributes, 272
- GetFileInformationByHand-
- le, 272
- GetKeyState, 191
- GetMessage, 58, 97, 191, 193
- GetModuleFileName, 127
- GetModuleFileNameEx, 38,
- 39
- GetPrinter, 203
- GetPriorityClass, 71, 72
- GetProcAddress, 127, 128
- GetProcessHeap, 125
- GetProcessTimes, 71
- GetProcessWorkingSetSize,
- 156, 157
- GetScrollPos, 191
- GetThreadPriority, 71
- GetThreadWaitChain, 111
- HeapAlloc, 125
- HeapFree, 125
- HeapReAlloc, 125
- HeapSize, 125
- InitializeAcl, 313
- InitializeCriticalSection, 85
- LeaveCriticalSection, 85
- LoadLibrary, 127, 128
- LockFileEx, 272
- LogonUser, 313
- MoveFile, 272
- OpenThreadWaitChainSession, 111
- PeekConsoleInput, 190
- PeekMessage, 97
- PlayEnhMetaFile, 191
- PolyBezier, 191
- Polygon, 191
- PrinterProperties, 203
- ReadConsole, 190
- ReadFile, 180, 272, 273, 383,
- 388, 389
- Rectangle, 191, 194
- RegisterClass, 191, 192
- ReleaseMutex, 85, 86
- ReleaseSemaphore, 85, 86,
- 100
- RemoveDirectory, 272
- ResumeThread, 44
- SendMessage, 97
- SetConsoleCursorPosition,
- 190
- SetConsoleTitle, 190
- SetEndOfFile, 272
- SetFileAttributes, 272
- SetFilePointer, 272, 273
- SetMenu, 191

- SetPixelV, 191
 - SetPrinter, 203
 - SetPriorityClass, 71, 72
 - SetProcessWorkingSetSize, 156, 157
 - SetTextColor, 191
 - SetThreadAffinityMask, 71
 - SetThreadPriority, 71
 - ShowCursor, 191
 - Sleep, 44
 - StartDoc, 202, 203
 - StartPage, 203
 - SuspendThread, 44
 - SwitchToFiber, 45, 46
 - SwitchToThread, 44
 - TerminateProcess, 38, 39
 - TerminateThread, 44, 45
 - TextOut, 191, 202
 - TryEnterCriticalSection, 85
 - UnlockFileEx, 272
 - VirtualAlloc, 101, 125
 - VirtualFree, 125
 - VirtualLock, 156, 157
 - VirtualProtect, 156, 157
 - VirtualUnlock, 156
 - WaitForMultipleObjects, 85
 - WaitForSingleObject, 45, 85, 86, 100, 105
 - WriteConsole, 190
 - WriteConsoleInput, 190
 - Windows API za rad sa datotekama i direktorijima, 271
 - Windows API za upravljanje memorijom, 124
 - Windows biblioteka
 - ADVAPI32.DLL, 337
 - AVIFIL32.DLL, 337
 - CABINET.DLL, 337
 - CFGMR32.DLL, 337
 - COMCTL32.DLL, 337
 - COMDLG32.DLL, 337
 - CRYPT32.DLL, 337
 - GDI32.DLL, 335, 337
 - IMAGEHLP.DLL, 337
 - IMM32.DLL, 337
 - KERNEL32.DLL, 271, 337
 - LZ32.DLL, 337
 - MAPI32.DLL, 337
 - MSIMG32.DLL, 337
 - MSVCRT.DLL, 337
 - MSVFW32.DLL, 337
 - NETAPI32.DLL, 337
 - ODBC32.DLL, 337
 - OPENGL32.DLL, 337
 - RASMAN.DLL, 337
 - RICHED20.DLL, 337
 - SFC.DLL, 337
 - SHELL32.DLL, 337
 - TAPI32.DLL, 337
 - URL.DLL, 337
 - USER32.DLL, 335, 337
 - USP10.DLL, 337
 - WININET.DLL, 337
 - WINMM.DLL, 28, 337
 - WSOCK32.DLL, 337
 - WTSAPI32.DLL, 337
 - Windows CE, 320, 342, 343, 368
 - Windows grafičke aplikacije, 192, 193
 - Windows korisnički procesi, 342
 - Windows ME, 315, 321, 322, 323
 - Windows Meta File, 202
 - Windows Mobile, 28, 323, 342, 343, 368
 - Windows NT, 19, 24, 26, 27, 53, 62, 184, 207, 261, 262, 263, 291, 310, 311, 318, 320, 322, 323, 326, 327, 331, 334, 335, 336, 338, 340, 341, 344, 368, 391
 - Windows Phone, 342, 343, 368
 - Windows Pocket PC, 342, 343
 - Windows Runtime, 325, 342
 - Windows Server 2003, 311, 323, 324
 - Windows Server 2008, 209, 323, 324
 - Windows sistemski procesi, 339
 - Windows Vista, 111, 190, 261, 311, 323, 324, 329, 333, 341
 - Windows XP, 15, 27, 45, 111, 190, 303, 310, 320, 323, 324, 327, 332, 335, 340
- X**
- X klijent, 194
 - X server, 194, 195, 196, 220, 359, 360
 - X terminal, 194
 - Xenix, 318, 346
 - Xlib, 194, 195, 196, 219
 - Xlib funkcije
 - XClearWindow, 195
 - XCloseDisplay, 195
 - XCreateSimpleWindow, 195, 196
 - XCreateWindow, 195
 - XDrawArc, 195
 - XDrawLine, 195
 - XDrawPoint, 195
 - XDrawRectangle, 195, 196
 - XDrawString, 195
 - XFillPolygon, 195
 - XFillRectangle, 195
 - XgrabKeyboard, 195
 - XLoadFont, 195
 - XLowerWindow, 195
 - XNextEvent, 195, 196
 - XOpenDisplay, 195, 196
 - XPeekEvent, 195
 - XRaiseWindow, 195
 - XSelectInput, 195, 196
 - XSetBackground, 195
 - XSetFillStyle, 195
 - XSetFont, 195
 - XSetForeground, 195, 196
 - XSetLineAttributes, 195
- XML, 324, 356, 366, 368
- Z**
- zakretanje, 239
 - znakovni terminal, 197
 - zonski zapis, 236
 - žurnal, 290
 - zygote, 364

